APPENDIX A

Campus Reserve Source Code Listing

Generated from 178 tracked, first-party source and configuration files.

Repository revision: 710447e

Scope and exclusions

This appendix provides the application source code, database migrations, views and frontend code, routes, tests, and selected deployment/configuration files.

Private environment files, credentials, dependencies, runtime data, generated build output, binary assets, and unrelated documents are intentionally excluded.

Source code sections

Use the PDF bookmarks panel to jump directly to a project section or an individual file. File headings include their repository-relative paths; line numbers are for reference and are not part of the source code.

app

app/Console/Commands/CleanupExpiredOtps.php

```php
<?php

namespace App\Console\Commands;

use App\Models\Otp;
use Illuminate\Console\Command;

class CleanupExpiredOtps extends Command
{
    protected $signature = 'otps:cleanup';

    protected $description = 'Delete expired OTP records from database';

    public function handle()
    {
        $deleted = Otp::where('expires_at', '<', now())->delete();

        $this->info("Deleted {$deleted} expired OTP records.");

        return 0;
    }
}
```

app/Console/Commands/CompleteExpiredReservations.php

```php
<?php

namespace App\Console\Commands;

use App\Models\Reservation;
use App\Services\ReservationNotificationDispatcher;
use Illuminate\Console\Command;

class CompleteExpiredReservations extends Command
{
    protected $signature = 'reservations:complete-expired';

    protected $description = 'Complete approved reservations whose reservation time has passed';

    public function handle(ReservationNotificationDispatcher $reservationNotifications): int
    {
        $now = now();
        $completed = 0;

        Reservation::query()
            ->where('status', 'approved')
            ->where(function ($query) use ($now): void {
                $query->whereDate('reservation_date', '<', $now->toDateString())
                    ->orWhere(function ($query) use ($now): void {
                        $query->whereDate('reservation_date', $now->toDateString())
                            ->whereTime('end_time', '<=', $now->format('H:i:s'));
                    });
            })
            ->orderBy('id')
            ->chunkById(100, function ($reservations) use ($now, $reservationNotifications, &$completed): void {
                foreach ($reservations as $reservation) {
                    $updated = Reservation::query()
                        ->whereKey($reservation->id)
                        ->where('status', 'approved')
                        ->where(function ($query) use ($now): void {
                            $query->whereDate('reservation_date', '<', $now->toDateString())
                                ->orWhere(function ($query) use ($now): void {
                                    $query->whereDate('reservation_date', $now->toDateString())
                                        ->whereTime('end_time', '<=', $now->format('H:i:s'));
                                });
                        })
                        ->update([
```

```
                            'status' => 'completed',
                            'updated_at' => now(),
                        ]);

                    if ($updated === 0) {
                        continue;
                    }

                    $reservation->refresh();
                    $reservationNotifications->notifyBorrower($reservation);
                    $completed++;
                }
            });

        $this->info("Completed {$completed} expired approved reservation(s).");

        return self::SUCCESS;
    }
}
```

app/Console/Commands/DeleteExpiredCompletedReservations.php

```
<?php

namespace App\Console\Commands;

use App\Models\Reservation;
use Illuminate\Console\Command;

class DeleteExpiredCompletedReservations extends Command
{
    protected $signature = 'reservations:delete-expired-completed';

    protected $description = 'Delete completed reservations three days after their scheduled end time';

    public function handle(): int
    {
        $cutoff = now()->subDays(3);

        $deleted = Reservation::query()
            ->where('status', 'completed')
            ->where(function ($query) use ($cutoff): void {
                $query->whereDate('reservation_date', '<', $cutoff->toDateString())
                    ->orWhere(function ($query) use ($cutoff): void {
                        $query->whereDate('reservation_date', $cutoff->toDateString())
                            ->whereTime('end_time', '<=', $cutoff->format('H:i:s'));
                    });
            })
            ->delete();

        $this->info("Deleted {$deleted} completed reservation(s) older than three days.");

        return self::SUCCESS;
    }
}
```

app/Console/Commands/DeleteExpiredRejectedReservations.php

```
<?php

namespace App\Console\Commands;

use App\Models\Reservation;
use Illuminate\Console\Command;

class DeleteExpiredRejectedReservations extends Command
{
    protected $signature = 'reservations:delete-expired-rejected';

    protected $description = 'Delete rejected reservations after three full days';

    public function handle(): int
    {
```

```
        $deleted = Reservation::query()
            ->where('status', 'rejected')
            ->whereNotNull('rejected_at')
            ->where('rejected_at', '<=', now()->subDays(3))
            ->delete();

        $this->info("Deleted {$deleted} rejected reservation(s) older than three days.");

        return self::SUCCESS;
    }
}
```

app/Console/Commands/RejectExpiredReservations.php

```
<?php

namespace App\Console\Commands;

use App\Models\Reservation;
use App\Services\ReservationNotificationDispatcher;
use Illuminate\Console\Command;

class RejectExpiredReservations extends Command
{
    protected $signature = 'reservations:reject-expired';

    protected $description = 'Reject pending reservations whose reservation date has passed';

    public function handle(ReservationNotificationDispatcher $reservationNotifications): int
    {
        $cutoffDate = now()->toDateString();
        $rejected = 0;

        Reservation::query()
            ->where('status', 'pending')
            ->whereDate('reservation_date', '<', $cutoffDate)
            ->orderBy('id')
            ->chunkById(100, function ($reservations) use ($cutoffDate, $reservationNotifications, &$rejected): void {
                foreach ($reservations as $reservation) {
                    $updated = Reservation::query()
                        ->whereKey($reservation->id)
                        ->where('status', 'pending')
                        ->whereDate('reservation_date', '<', $cutoffDate)
                        ->update([
                            'status' => 'rejected',
                            'rejected_at' => now(),
                            'admin_remarks' => 'Automatically rejected because the reservation date has passed.',
                            'updated_at' => now(),
                        ]);

                    if ($updated === 0) {
                        continue;
                    }

                    $reservation->refresh();
                    $reservationNotifications->notifyBorrower($reservation);
                    $rejected++;
                }
            });

        $this->info("Rejected {$rejected} expired pending reservation(s).");

        return self::SUCCESS;
    }
}
```

app/Console/Commands/SendPropertyReturnReminders.php

```
<?php

namespace App\Console\Commands;

use App\Models\Reservation;
```

```php
use App\Notifications\PropertyReturnDue;
use Illuminate\Console\Command;
use Illuminate\Support\Facades\DB;

class SendPropertyReturnReminders extends Command
{
    protected $signature = 'property-returns:notify-due';

    protected $description = 'Notify borrowers when approved property reservations reach their return time';

    public function handle(): int
    {
        $now = now();
        $sent = 0;

        Reservation::query()
            ->where('status', 'approved')
            ->whereNull('return_reminded_at')
            ->whereHas('items')
            ->where(function ($query) use ($now): void {
                $query->whereDate('reservation_date', '<', $now->toDateString())
                    ->orWhere(function ($query) use ($now): void {
                        $query->whereDate('reservation_date', $now->toDateString())
                            ->whereTime('end_time', '<=', $now->format('H:i:s'));
                    });
            })
            ->orderBy('id')
            ->chunkById(100, function ($reservations) use (&$sent): void {
                foreach ($reservations as $reservation) {
                    DB::transaction(function () use ($reservation, &$sent): void {
                        $dueReservation = Reservation::query()
                            ->with(['user', 'facility', 'items'])
                            ->lockForUpdate()
                            ->find($reservation->id);

                        if (! $dueReservation
                            || $dueReservation->status !== 'approved'
                            || $dueReservation->return_reminded_at
                            || $dueReservation->items->isEmpty()) {
                            return;
                        }

                        $dueReservation->user->notify(new PropertyReturnDue($dueReservation));
                        $dueReservation->forceFill(['return_reminded_at' => now()])->save();
                        $sent++;
                    });
                }
            });

        $this->info("Sent {$sent} property return reminder(s).");

        return self::SUCCESS;
    }
}
```

app/Console/Kernel.php

```php
<?php

namespace App\Console;

use Illuminate\Console\Scheduling\Schedule;
use Illuminate\Foundation\Console\Kernel as ConsoleKernel;

class Kernel extends ConsoleKernel
{
    protected function schedule(Schedule $schedule)
    {
        // Clean up expired OTPs every minute
        $schedule->command('otps:cleanup')->everyMinute();
    }

    protected function commands()
    {
```

```
        $this->load(__DIR__.'/Commands');

        require base_path('routes/console.php');
    }
}
```

app/Http/Controllers/AdminController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\AuditLog;
use App\Models\Facility;
use App\Models\FacilityMaintenanceBlock;
use App\Models\FacilityItem;
use App\Models\PropertyItemMaintenanceBlock;
use App\Models\Reservation;
use App\Models\User;
use App\Services\FacilityItemAvailability;
use App\Services\FacilityMaintenanceSchedule;
use App\Services\PropertyItemMaintenanceSchedule;
use App\Services\ReservationNotificationDispatcher;
use App\Services\ReservationDateWindow;
use App\Services\ReservationTimeWindow;
use App\Services\UserNameSearchIndex;
use Illuminate\Http\Request;
use Illuminate\Pagination\LengthAwarePaginator;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\DB;
use Illuminate\Validation\ValidationException;
use Illuminate\Validation\Rule;

class AdminController extends Controller
{
    private function requireAdmin()
    {
        // I check that the user is logged in and is an admin
        // If not, I block them from accessing this page
        if (! Auth::check() || ! Auth::user()->isAdmin()) {
            abort(403, 'Admin access only.');
        }
    }

    private function requireSuperAdmin()
    {
        // Only the head admin can access super-admin features
        // I block everyone else
        if (! Auth::check() || ! Auth::user()->isSuperAdmin()) {
            abort(403, 'Head admin access only.');
        }
    }

    public function dashboard(
        Request $request,
        ReservationDateWindow $reservationDateWindow,
        UserNameSearchIndex $userNameSearchIndex
    )
    {
        // Show the admin dashboard with pending requests, recent activity, and user search
        $this->requireAdmin();

        $search = trim($request->get('search', ''));
        $role = $request->validate([
            'role' => ['nullable', 'in:all,user,admin,super_admin'],
        ])['role'] ?? 'all';
        $memberSort = $request->validate([
            'member_sort' => ['nullable', 'in:name,joined_newest,joined_oldest'],
        ])['member_sort'] ?? 'joined_newest';
        $users = \App\Models\User::query();

        if ($role === 'user') {
            $users->where('is_admin', false)->where('is_super_admin', false);
        } elseif ($role === 'admin') {
```

```
            $users->where('is_admin', true)->where('is_super_admin', false);
        } elseif ($role === 'super_admin') {
            $users->where('is_super_admin', true);
        }

        if ($search !== '') {
            $searchEmailHash = filter_var($search, FILTER_VALIDATE_EMAIL)
                ? hash('sha256', strtolower($search))
                : null;
            $candidateIds = $userNameSearchIndex->matchingUserIds($search);
            if ($searchEmailHash !== null) {
                $emailUserId = (clone $users)->where('email_hash', $searchEmailHash)->value('id');
                if ($emailUserId !== null) {
                    $candidateIds->push($emailUserId);
                }
            }

            $allUsers = (clone $users)
                ->whereIn('id', $candidateIds->unique())
                ->when($memberSort !== 'name', fn ($query) => $query
                    ->orderBy('created_at', $memberSort === 'joined_oldest' ? 'asc' : 'desc')
                    ->orderBy('id', $memberSort === 'joined_oldest' ? 'asc' : 'desc'))
                ->get()
                ->filter(fn (User $user): bool =>
                    $userNameSearchIndex->matches($user->name, $search)
                    || ($searchEmailHash !== null
                        && $user->email_hash !== null
                        && hash_equals($user->email_hash, $searchEmailHash))
                );
            $allUsers = $allUsers->values();

            if ($memberSort === 'name') {
                $allUsers = $allUsers
                    ->sortBy(fn (User $user) => mb_strtolower($user->name))
                    ->values();
            }

            $currentPage = LengthAwarePaginator::resolveCurrentPage();
            $users = new LengthAwarePaginator(
                $allUsers->forPage($currentPage, 15)->values(),
                $allUsers->count(),
                15,
                $currentPage,
                [
                    'path' => LengthAwarePaginator::resolveCurrentPath(),
                    'query' => $request->query(),
                ]
            );
            $users->withQueryString();
        } elseif ($memberSort === 'name') {
            $allUsers = $users->get()
                ->sortBy(fn (User $user) => mb_strtolower($user->name))
                ->values();
            $currentPage = LengthAwarePaginator::resolveCurrentPage();
            $users = new LengthAwarePaginator(
                $allUsers->forPage($currentPage, 15)->values(),
                $allUsers->count(),
                15,
                $currentPage,
                [
                    'path' => LengthAwarePaginator::resolveCurrentPath(),
                    'query' => $request->query(),
                ]
            );
        } else {
            $users->orderBy('created_at', $memberSort === 'joined_oldest' ? 'asc' : 'desc')
                ->orderBy('id', $memberSort === 'joined_oldest' ? 'asc' : 'desc');
            $users = $users->paginate(15)->withQueryString();
        }

        if ($search === '' && $memberSort === 'name') {
            $users->withQueryString();
        }

```

```
        if ($request->ajax()) {
            return response()->view('admin.partials.members', compact('users', 'search', 'role', 'memberSort'));
        }

        $pendingReservations = Reservation::with(['user', 'facility', 'items'])
            ->where('status', 'pending')
            ->orderBy('reservation_date')
            ->orderBy('start_time')
            ->paginate(15, ['*'], 'pendingPage')
            ->withQueryString();

        $reservationSort = $request->validate([
            'reservation_sort' => ['nullable', 'in:recent,oldest,alphabetical'],
        ])['reservation_sort'] ?? 'recent';
        $approvedReservationsQuery = Reservation::with(['user', 'facility', 'items'])
            ->where('status', 'approved');

        if ($reservationSort === 'alphabetical') {
            $approvedReservationsQuery->orderByRaw("COALESCE(
                (SELECT LOWER(facilities.name) FROM facilities WHERE facilities.id = reservations.facility_id),
                (SELECT LOWER(facility_items.name)
                 FROM reservation_items
                 INNER JOIN facility_items ON facility_items.id = reservation_items.facility_item_id
                 WHERE reservation_items.reservation_id = reservations.id
                 ORDER BY LOWER(facility_items.name)
                 LIMIT 1),
                ''
            ) ASC");
        } else {
            $approvedReservationsQuery->orderBy('reservation_date', $reservationSort === 'oldest' ? 'asc' : 'desc')
                ->orderBy('start_time', $reservationSort === 'oldest' ? 'asc' : 'desc');
        }

        $approvedReservations = $approvedReservationsQuery
            ->orderBy('id', 'desc')
            ->paginate(20, ['*'], 'approvedPage')
            ->withQueryString()
            ->fragment('reservations');

        $recentReservations = Reservation::with(['user', 'facility'])
            ->whereIn('status', ['approved', 'rejected'])
            ->orderBy('updated_at', 'desc')
            ->take(20)
            ->get();

        $reservationDecisionTotals = Reservation::query()
            ->selectRaw('status, COUNT(*) as total')
            ->whereIn('status', ['approved', 'rejected'])
            ->groupBy('status')
            ->pluck('total', 'status');
        $approvedReservationCount = (int) $reservationDecisionTotals->get('approved', 0);
        $rejectedReservationCount = (int) $reservationDecisionTotals->get('rejected', 0);

        $auditLogs = AuditLog::with(['user', 'reservation.facility'])
            ->latest()
            ->take(25)
            ->get();
        $maximumReservationDate = $reservationDateWindow->maximumDate();

        return view('admin.dashboard', compact(
            'pendingReservations',
            'approvedReservations',
            'recentReservations',
            'approvedReservationCount',
            'rejectedReservationCount',
            'auditLogs',
            'users',
            'search',
            'role',
            'memberSort',
            'reservationSort',
            'maximumReservationDate',
        ));
    }
```

```
215
216     private function reservationActionResponse(Request $request, string $message, ?AuditLog $auditLog = null)
217     {
218         if ($request->expectsJson()) {
219             return response()->json([
220                 'status' => $message,
221                 'pending_count' => Reservation::where('status', 'pending')->count(),
222                 'recent_decision_count' => min(20, Reservation::whereIn('status', ['approved', 'rejected'])->count()),
223                 'audit_log_count' => min(25, AuditLog::count()),
224                 'audit_log' => $auditLog ? [
225                     'action' => $auditLog->action,
226                     'details' => $auditLog->details,
227                     'created_at' => $auditLog->created_at->format('M d, Y H:i:s'),
228                     'user' => Auth::user()->name,
229                 ] : null,
230             ]);
231         }
232
233         return back()->with('status', $message);
234     }
235
236     public function createFacility()
237     {
238         $this->requireAdmin();
239
240         $facilities = Facility::query()
241             ->orderBy('name')
242             ->get()
243             ->unique(fn ($facility) => strtolower(trim($facility->name)) . '|' . strtolower(trim($facility->building)))
244             ->values()
245             ->map(function ($facility) {
246                 $facility->description = $facility->description ?: (
247                     $facility->building
248                         ? $facility->building . ' campus facility.'
249                         : 'Campus facility.'
250                 );
251
252                 return $facility;
253             });
254
255         return view('admin.facilities.create', compact('facilities'));
256     }
257
258     public function storeFacility(Request $request)
259     {
260         $this->requireAdmin();
261
262         $validated = $request->validate([
263             'name' => [
264                 'required',
265                 'string',
266                 'max:100',
267                 Rule::unique('facilities')->where(fn ($query) => $query->where('building', trim($request->input('building', '')))),
268             ],
269             'building' => ['required', 'string', 'max:100'],
270             'type' => ['required', 'string', 'max:50'],
271             'capacity' => ['required', 'integer', 'min:1'],
272             'requires_approval' => ['sometimes', 'boolean'],
273         ]);
274
275         $validated['requires_approval'] = $request->boolean('requires_approval');
276         $facility = Facility::create($validated);
277
278         AuditLog::create([
279             'user_id' => Auth::id(),
280             'reservation_id' => null,
281             'action' => 'facility_created',
282             'details' => "Created facility {$facility->name} in {$facility->building}.",
283         ]);
284
285         return redirect()->route('admin.facilities.create')->with('status', 'Facility created successfully.');
286     }
287
288     public function updateFacility(Request $request, Facility $facility)
```

```
    {
        $this->requireAdmin();

        $validated = $request->validate([
            'name' => [
                'required',
                'string',
                'max:100',
                Rule::unique('facilities')->where(fn ($query) => $query->where('building', trim($request->input('building', '')))->ignore($facility->id),
            ],
            'building' => ['required', 'string', 'max:100'],
            'type' => ['required', 'string', 'max:50'],
            'capacity' => ['required', 'integer', 'min:1'],
            'requires_approval' => ['sometimes', 'boolean'],
        ]);

        $validated['requires_approval'] = $request->boolean('requires_approval');

        $facility->update($validated);

        AuditLog::create([
            'user_id' => Auth::id(),
            'reservation_id' => null,
            'action' => 'facility_updated',
            'details' => "Updated facility {$facility->name} in {$facility->building}.",
        ]);

        return redirect()->route('admin.facilities.create')->with('status', 'Facility updated successfully.');
    }

    public function destroyFacility(Facility $facility)
    {
        $this->requireAdmin();

        $facilityName = $facility->name;
        $facility->delete();

        AuditLog::create([
            'user_id' => Auth::id(),
            'reservation_id' => null,
            'action' => 'facility_deleted',
            'details' => "Removed facility {$facilityName}.",
        ]);

        return redirect()->route('admin.facilities.create')->with('status', 'Facility removed successfully.');
    }

    public function propertyManagement()
    {
        $this->requireAdmin();

        $items = FacilityItem::orderBy('name')->get();

        return view('admin.property-management', compact('items'));
    }

    public function reservationSettings(
        ReservationTimeWindow $reservationTimeWindow,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        PropertyItemMaintenanceSchedule $propertyItemMaintenanceSchedule
    )
    {
        $this->requireAdmin();

        $reservationHours = $reservationTimeWindow->current();
        $settingsStorageAvailable = $reservationTimeWindow->isAvailable();
        $maintenanceStorageAvailable = $maintenanceSchedule->isAvailable();
        $propertyMaintenanceStorageAvailable = $propertyItemMaintenanceSchedule->isAvailable();
        $facilities = Facility::active()->orderBy('name')->get();
        $propertyItems = FacilityItem::with('facility')->orderBy('name')->get();
        $maintenanceBlocks = $maintenanceStorageAvailable
            ? FacilityMaintenanceBlock::with('facility')
                ->whereDate('maintenance_date', '>=', today())
                ->orderBy('maintenance_date')
```

```
                ->orderBy('start_time')
                ->get()
            : collect();
        $propertyMaintenanceBlocks = $propertyMaintenanceStorageAvailable
            ? PropertyItemMaintenanceBlock::with('item')
                ->whereDate('maintenance_date', '>=', today())
                ->orderBy('maintenance_date')
                ->orderBy('start_time')
                ->get()
            : collect();

        return view('admin.reservation-settings', compact(
            'reservationHours',
            'settingsStorageAvailable',
            'maintenanceStorageAvailable',
            'propertyMaintenanceStorageAvailable',
            'facilities',
            'propertyItems',
            'maintenanceBlocks',
            'propertyMaintenanceBlocks'
        ));
    }

    public function updateReservationSettings(Request $request, ReservationTimeWindow $reservationTimeWindow)
    {
        $this->requireAdmin();

        if (! $reservationTimeWindow->isAvailable()) {
            return back()->with('error', 'Reservation settings cannot be saved until the reservation settings database migration has been applied.');
        }

        $validated = $request->validate([
            'start_time' => ['required', 'date_format:H:i'],
            'end_time' => ['required', 'date_format:H:i', 'after:start_time'],
            'current_password' => ['required', 'current_password'],
        ]);

        $reservationTimeWindow->update($validated['start_time'], $validated['end_time']);

        return redirect()
            ->route('admin.reservation-settings')
            ->with('status', 'Reservation hours updated successfully.');
    }

    public function storeMaintenanceBlock(
        Request $request,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        ReservationNotificationDispatcher $reservationNotifications
    ) {
        $this->requireAdmin();

        if (! $maintenanceSchedule->isAvailable()) {
            return back()->with('error', 'Maintenance scheduling is unavailable until its database migration has been applied.');
        }

        $validated = $request->validate([
            'facility_id' => ['required', 'integer', Rule::exists('facilities', 'id')->where('is_active', true)],
            'maintenance_date' => ['required', 'date', 'after_or_equal:today'],
            'is_all_day' => ['required', 'boolean'],
            'start_time' => ['nullable', 'required_unless:is_all_day,1', 'date_format:H:i'],
            'end_time' => ['nullable', 'required_unless:is_all_day,1', 'date_format:H:i', 'after:start_time'],
            'reason' => ['required', 'string', 'max:255'],
            'current_password' => ['required', 'current_password'],
        ]);

        $isAllDay = $request->boolean('is_all_day');
        $startTime = $isAllDay ? null : $validated['start_time'];
        $endTime = $isAllDay ? null : $validated['end_time'];
        $facility = Facility::findOrFail($validated['facility_id']);

        [$block, $overlappingReservations] = DB::transaction(function () use (
            $validated,
            $facility,
            $isAllDay,
```

```
            $startTime,
            $endTime
        ) {
            $block = FacilityMaintenanceBlock::create([
                'facility_id' => $facility->id,
                'maintenance_date' => $validated['maintenance_date'],
                'start_time' => $startTime,
                'end_time' => $endTime,
                'is_all_day' => $isAllDay,
                'reason' => $validated['reason'],
            ]);

            $overlappingReservations = Reservation::query()
                ->where('facility_id', $facility->id)
                ->whereDate('reservation_date', $validated['maintenance_date'])
                ->whereIn('status', ['pending', 'approved'])
                ->when(! $isAllDay, fn ($query) => $query
                    ->where('start_time', '<', $endTime)
                    ->where('end_time', '>', $startTime))
                ->lockForUpdate()
                ->get();

            foreach ($overlappingReservations as $reservation) {
                $reservation->update([
                    'status' => 'rejected',
                    'rejected_at' => now(),
                    'admin_remarks' => "Automatically rejected due to facility maintenance: {$validated['reason']}.",
                ]);

                AuditLog::create([
                    'user_id' => Auth::id(),
                    'reservation_id' => $reservation->id,
                    'action' => 'rejected',
                    'details' => "Automatically rejected due to facility maintenance: {$validated['reason']}.",
                ]);
            }

            AuditLog::create([
                'user_id' => Auth::id(),
                'reservation_id' => null,
                'action' => 'facility_maintenance_created',
                'details' => "Scheduled maintenance for {$facility->name} on {$validated['maintenance_date']}: {$validated['reason']}.",
            ]);

            return [$block, $overlappingReservations];
        });

        foreach ($overlappingReservations as $reservation) {
            $reservationNotifications->notifyBorrower($reservation);
        }

        return redirect()
            ->route('admin.reservation-settings')
            ->with('status', $overlappingReservations->isEmpty()
                ? 'Maintenance block scheduled successfully.'
                : 'Maintenance block scheduled. Overlapping reservations were automatically rejected and their owners notified.');
    }

    public function destroyMaintenanceBlock(Request $request, FacilityMaintenanceBlock $block)
    {
        $this->requireAdmin();
        $request->validate([
            'current_password' => ['required', 'current_password'],
        ]);

        $facilityName = $block->facility->name;
        $description = "Removed facility maintenance block for {$facilityName} on {$block->maintenance_date->format('Y-m-d')}.";
        $block->delete();

        AuditLog::create([
            'user_id' => Auth::id(),
            'reservation_id' => null,
            'action' => 'facility_maintenance_removed',
            'details' => $description,
```

```
        ]);

        return redirect()->route('admin.reservation-settings')->with('status', 'Maintenance block removed.');
    }

    public function updateMaintenanceBlock(
        Request $request,
        FacilityMaintenanceBlock $block,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        ReservationNotificationDispatcher $reservationNotifications
    ) {
        $this->requireAdmin();

        if (! $maintenanceSchedule->isAvailable()) {
            return back()->with('error', 'Maintenance scheduling is unavailable until its database migration has been applied.');
        }

        $validated = $request->validate([
            'facility_id' => ['required', 'integer', Rule::exists('facilities', 'id')->where('is_active', true)],
            'maintenance_date' => ['required', 'date', 'after_or_equal:today'],
            'is_all_day' => ['required', 'boolean'],
            'start_time' => ['nullable', 'required_unless:is_all_day,1', 'date_format:H:i'],
            'end_time' => ['nullable', 'required_unless:is_all_day,1', 'date_format:H:i', 'after:start_time'],
            'reason' => ['required', 'string', 'max:255'],
            'current_password' => ['required', 'current_password'],
        ]);

        $isAllDay = $request->boolean('is_all_day');
        $startTime = $isAllDay ? null : $validated['start_time'];
        $endTime = $isAllDay ? null : $validated['end_time'];
        $facility = Facility::findOrFail($validated['facility_id']);

        $overlappingReservations = DB::transaction(function () use (
            $block,
            $validated,
            $facility,
            $isAllDay,
            $startTime,
            $endTime
        ) {
            $block = FacilityMaintenanceBlock::query()->lockForUpdate()->findOrFail($block->id);
            $block->update([
                'facility_id' => $facility->id,
                'maintenance_date' => $validated['maintenance_date'],
                'start_time' => $startTime,
                'end_time' => $endTime,
                'is_all_day' => $isAllDay,
                'reason' => $validated['reason'],
            ]);

            $overlappingReservations = Reservation::query()
                ->where('facility_id', $facility->id)
                ->whereDate('reservation_date', $validated['maintenance_date'])
                ->whereIn('status', ['pending', 'approved'])
                ->when(! $isAllDay, fn ($query) => $query
                    ->where('start_time', '<', $endTime)
                    ->where('end_time', '>', $startTime))
                ->lockForUpdate()
                ->get();

            foreach ($overlappingReservations as $reservation) {
                $reservation->update([
                    'status' => 'rejected',
                    'rejected_at' => now(),
                    'admin_remarks' => "Automatically rejected due to facility maintenance: {$validated['reason']}.",
                ]);

                AuditLog::create([
                    'user_id' => Auth::id(),
                    'reservation_id' => $reservation->id,
                    'action' => 'rejected',
                    'details' => "Automatically rejected due to facility maintenance: {$validated['reason']}.",
                ]);
            }
```

```
585
586             AuditLog::create([
587                 'user_id' => Auth::id(),
588                 'reservation_id' => null,
589                 'action' => 'facility_maintenance_updated',
590                 'details' => "Updated maintenance for {$facility->name} on {$validated['maintenance_date']}: {$validated['reason']}.",
591             ]);
592
593             return $overlappingReservations;
594         });
595
596         foreach ($overlappingReservations as $reservation) {
597             $reservationNotifications->notifyBorrower($reservation);
598         }
599
600         return redirect()
601             ->route('admin.reservation-settings')
602             ->with('status', $overlappingReservations->isEmpty()
603                 ? 'Maintenance block updated successfully.'
604                 : 'Maintenance block updated. Overlapping reservations were automatically rejected and their owners notified.');
605     }
606
607     public function storeFacilityItem(Request $request)
608     {
609         $this->requireAdmin();
610
611         $validated = $request->validate([
612             'name' => ['required', 'string', 'max:100'],
613             'quantity' => ['required', 'integer', 'min:1'],
614         ]);
615
616         $item = FacilityItem::create($validated);
617
618         AuditLog::create([
619             'user_id' => Auth::id(),
620             'reservation_id' => null,
621             'action' => 'facility_item_created',
622             'details' => "Added property item {$item->name} (quantity: {$item->quantity}).",
623         ]);
624
625         return redirect()->route('admin.property-management.index')->with('status', 'Facility item added successfully.');
626     }
627
628     public function updateFacilityItem(Request $request, FacilityItem $item)
629     {
630         $this->requireAdmin();
631
632         $validated = $request->validate([
633             'name' => ['required', 'string', 'max:100'],
634             'quantity' => ['required', 'integer', 'min:1'],
635         ]);
636
637         $item->update($validated);
638
639         AuditLog::create([
640             'user_id' => Auth::id(),
641             'reservation_id' => null,
642             'action' => 'facility_item_updated',
643             'details' => "Updated property item {$item->name} (quantity: {$item->quantity}).",
644         ]);
645
646         return redirect()->route('admin.property-management.index')->with('status', 'Facility item updated successfully.');
647     }
648
649     public function destroyFacilityItem(FacilityItem $item)
650     {
651         $this->requireAdmin();
652
653         $itemName = $item->name;
654         $item->delete();
655
656         AuditLog::create([
657             'user_id' => Auth::id(),
658             'reservation_id' => null,
```

```
            'action' => 'facility_item_deleted',
            'details' => "Removed property item {$itemName}.",
        ]);

        return redirect()->route('admin.property-management.index')->with('status', 'Facility item removed successfully.');
    }

    public function approve(
        Request $request,
        Reservation $reservation,
        FacilityItemAvailability $itemAvailability,
        ReservationNotificationDispatcher $reservationNotifications,
        ReservationTimeWindow $reservationTimeWindow,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        PropertyItemMaintenanceSchedule $propertyItemMaintenanceSchedule
    )
    {
        // Mark the reservation as approved and send the user a notification
        $this->requireAdmin();

        if ($reservation->status !== 'pending') {
            if ($request->expectsJson()) {
                return response()->json(['message' => 'Only pending reservations can be approved.'], 409);
            }

            if (! $reservationTimeWindow->allows(
                $reservation->start_time->format('H:i'),
                $reservation->end_time->format('H:i')
            )) {
                $approvalError = $reservationTimeWindow->validationMessage();

                if ($request->expectsJson()) {
                    return response()->json(['message' => $approvalError], 409);
                }

                return back()->with('error', $approvalError);
            }

            return back()->with('error', 'Only pending reservations can be approved.');
        }

        $approvalError = DB::transaction(function () use (
            $request,
            $reservation,
            $itemAvailability,
            $maintenanceSchedule,
            $propertyItemMaintenanceSchedule
        ) {
            if ($reservation->facility_id !== null) {
                $maintenanceBlock = $maintenanceSchedule->conflictingBlock(
                    $reservation->facility_id,
                    $reservation->reservation_date->format('Y-m-d'),
                    $reservation->start_time->format('H:i'),
                    $reservation->end_time->format('H:i')
                );

                if ($maintenanceBlock) {
                    return $maintenanceSchedule->validationMessage($maintenanceBlock);
                }

                $conflictingApproved = Reservation::where('facility_id', $reservation->facility_id)
                    ->where('reservation_date', $reservation->reservation_date)
                    ->where('status', 'approved')
                    ->where(function ($q) use ($reservation) {
                        $q->whereBetween('start_time', [$reservation->start_time, $reservation->end_time])
                            ->orWhereBetween('end_time', [$reservation->start_time, $reservation->end_time])
                            ->orWhere(function ($q2) use ($reservation) {
                                $q2->where('start_time', '<=', $reservation->start_time)
                                    ->where('end_time', '>=', $reservation->end_time);
                            });
                    })
                    ->exists();

                if ($conflictingApproved) {
```

```
                    return 'Cannot approve this reservation because an approved booking already occupies the same room and time slot. Please reject or reschedule it first.';
                }
            }

            $reservedItems = $reservation->items()->orderBy('facility_items.id')->get();
            if ($reservation->facility_id === null && $reservedItems->isEmpty()) {
                return 'Cannot approve this reservation because it has no facility or property items.';
            }
            $lockedItems = FacilityItem::query()
                ->whereIn('id', $reservedItems->modelKeys())
                ->orderBy('id')
                ->lockForUpdate()
                ->get()
                ->keyBy('id');

            foreach ($reservedItems as $item) {
                    $lockedItem = $lockedItems->get($item->id);
                    if (!$lockedItem) {
                        return 'Cannot approve this reservation because one of its facility items is no longer available.';
                    }
                    $itemMaintenanceBlock = $propertyItemMaintenanceSchedule->conflictingBlock(
                        $lockedItem->id,
                        $reservation->reservation_date->format('Y-m-d'),
                        $reservation->start_time->format('H:i'),
                        $reservation->end_time->format('H:i')
                    );
                    if ($itemMaintenanceBlock) {
                        return "Cannot approve this reservation because {$lockedItem->name} is under maintenance: {$itemMaintenanceBlock->reason}.";
                    }

                    $availableQuantity = $itemAvailability->availableQuantity(
                        $lockedItem,
                        $reservation->reservation_date->format('Y-m-d'),
                        $reservation->start_time->format('H:i'),
                        $reservation->end_time->format('H:i')
                );

                if ($item->pivot->quantity > $availableQuantity) {
                    return "Cannot approve this reservation because only {$availableQuantity} of {$item->name} remain available for this date and time.";
                }
            }

            $reservation->update([
                'status' => 'approved',
                'admin_remarks' => $request->input('remarks', 'Approved by admin.'),
            ]);

            AuditLog::create([
                'user_id' => Auth::id(),
                'reservation_id' => $reservation->id,
                'action' => 'approved',
                'details' => 'Reservation approved by admin',
            ]);

            return null;
        });

        if ($approvalError) {
            if ($request->expectsJson()) {
                return response()->json(['message' => $approvalError], 409);
            }

            return back()->with('error', $approvalError);
        }

        $reservationNotifications->notifyBorrower($reservation);

        return $this->reservationActionResponse(
            $request,
            'Reservation approved successfully.',
            AuditLog::where('reservation_id', $reservation->id)->where('action', 'approved')->latest()->first()
        );
    }

```

```
    public function reject(
        Request $request,
        Reservation $reservation,
        ReservationNotificationDispatcher $reservationNotifications
    )
    {
        // Mark the reservation as rejected and let the user know
        $this->requireAdmin();

        if ($reservation->status !== 'pending') {
            if ($request->expectsJson()) {
                return response()->json(['message' => 'Only pending reservations can be rejected.'], 409);
            }

            return back()->with('error', 'Only pending reservations can be rejected.');
        }

        $reservation->update([
            'status' => 'rejected',
            'rejected_at' => now(),
            'admin_remarks' => $request->input('remarks', 'Rejected by admin.'),
        ]);

        $auditLog = AuditLog::create([
            'user_id' => Auth::id(),
            'reservation_id' => $reservation->id,
            'action' => 'rejected',
            'details' => $request->input('remarks', 'Reservation rejected by admin'),
        ]);

        $reservationNotifications->notifyBorrower($reservation);

        return $this->reservationActionResponse($request, 'Reservation rejected successfully.', $auditLog);
    }

    public function updateApprovedReservation(
        Request $request,
        Reservation $reservation,
        FacilityItemAvailability $itemAvailability,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        PropertyItemMaintenanceSchedule $propertyItemMaintenanceSchedule,
        ReservationTimeWindow $reservationTimeWindow,
        ReservationDateWindow $reservationDateWindow
    ) {
        $this->requireAdmin();

        if ($reservation->status !== 'approved') {
            return back()->with('error', 'Only approved reservations can be edited.');
        }

        $validated = $request->validate([
            'reservation_date' => array_merge($reservationDateWindow->rules(), ['after_or_equal:today']),
            'start_time' => ['required', 'date_format:H:i'],
            'end_time' => ['required', 'date_format:H:i', 'after:start_time'],
            'purpose' => ['required', 'string', 'max:255'],
        ]);

        $reservation->loadMissing('items');
        $startTime = $validated['start_time'];
        $endTime = $validated['end_time'];
        $date = $validated['reservation_date'];

        if (! $reservationTimeWindow->allows($startTime, $endTime)) {
            throw ValidationException::withMessages([
                'start_time' => $reservationTimeWindow->validationMessage(),
            ]);
        }

        if ($reservation->facility_id !== null) {
            $maintenanceBlock = $maintenanceSchedule->conflictingBlock(
                $reservation->facility_id,
                $date,
                $startTime,
                $endTime
```

```
            );

            if ($maintenanceBlock) {
                throw ValidationException::withMessages([
                    'reservation_date' => $maintenanceSchedule->validationMessage($maintenanceBlock),
                ]);
            }

            $conflict = Reservation::query()
                ->where('facility_id', $reservation->facility_id)
                ->whereDate('reservation_date', $date)
                ->where('status', 'approved')
                ->where('id', '!=', $reservation->id)
                ->where('start_time', '<', $endTime)
                ->where('end_time', '>', $startTime)
                ->exists();

            if ($conflict) {
                throw ValidationException::withMessages([
                    'start_time' => 'Another approved reservation already occupies this facility at that time.',
                ]);
            }
        }

        if ($reservation->items->isNotEmpty()) {
            foreach ($reservation->items as $item) {
                $maintenanceBlock = $propertyItemMaintenanceSchedule->conflictingBlock(
                    $item->id,
                    $date,
                    $startTime,
                    $endTime
                );

                if ($maintenanceBlock) {
                    throw ValidationException::withMessages([
                        'reservation_date' => "{$item->name} is under maintenance: {$maintenanceBlock->reason}.",
                    ]);
                }
            }

            $availableQuantities = $itemAvailability->availableQuantities(
                $reservation->items,
                $date,
                $startTime,
                $endTime,
                $reservation->id
            );

            foreach ($reservation->items as $item) {
                if ($item->pivot->quantity > ($availableQuantities[$item->id] ?? 0)) {
                    throw ValidationException::withMessages([
                        'reservation_date' => "Only ".($availableQuantities[$item->id] ?? 0)." of {$item->name} are available for that schedule.",
                    ]);
                }
            }
        }

        $reservation->update([
            'reservation_date' => $date,
            'start_time' => $startTime,
            'end_time' => $endTime,
            'purpose' => $validated['purpose'],
        ]);

        $auditLog = AuditLog::create([
            'user_id' => Auth::id(),
            'reservation_id' => $reservation->id,
            'action' => 'reservation_updated',
            'details' => "Updated approved reservation #{$reservation->id} to {$date}, {$startTime}-{$endTime}.",
        ]);

        if ($request->expectsJson()) {
            return response()->json(['status' => 'Approved reservation updated successfully.']);
        }
```

```
955
956         return redirect()->to(route('admin.dashboard').'#reservations')
957             ->with('status', 'Approved reservation updated successfully.');
958     }
959
960     public function destroyReservation(
961         Request $request,
962         Reservation $reservation,
963         ReservationNotificationDispatcher $reservationNotifications
964     )
965     {
966         $this->requireAdmin();
967
968         $reservation->loadMissing(['user', 'facility', 'items']);
969         $reservation->status = 'cancelled';
970         $auditLog = DB::transaction(function () use ($reservation) {
971             $auditLog = AuditLog::create([
972                 'user_id' => Auth::id(),
973                 'reservation_id' => $reservation->id,
974                 'action' => 'deleted',
975                 'details' => 'Reservation cancelled and deleted by admin.',
976             ]);
977             $reservation->delete();
978
979             return $auditLog;
980         });
981
982         $reservationNotifications->notifyBorrower($reservation);
983
984         return $this->reservationActionResponse(
985             $request,
986             'Reservation deleted successfully and the borrower was notified.',
987             $auditLog
988         );
989     }
990
991     public function updateUserRole(Request $request, \App\Models\User $user)
992     {
993         // Only the head admin can change user roles, including admin promotion/demotion.
994         $this->requireSuperAdmin();
995
996         if ($user->isSuperAdmin()) {
997             if ($request->expectsJson()) {
998                 return response()->json(['message' => 'Cannot change head admin role.'], 403);
999             }
1000
1001            return redirect()->route('admin.dashboard', [
1002                'search' => $request->input('return_search'),
1003                'role' => $request->input('return_role'),
1004            ])
1005                ->with('error', 'Cannot change head admin role.');
1006        }
1007
1008        $request->validate([
1009            'role' => 'required|in:user,admin',
1010        ]);
1011
1012        $user->update([
1013            'is_admin' => $request->input('role') === 'admin',
1014        ]);
1015
1016        AuditLog::create([
1017            'user_id' => Auth::id(),
1018            'reservation_id' => null,
1019            'action' => 'role_changed',
1020            'details' => sprintf('Changed role for user %s to %s', $user->email, $request->input('role')),
1021        ]);
1022
1023        if ($request->expectsJson()) {
1024            return response()->json(['status' => 'User role updated successfully.']);
1025        }
1026
1027        return redirect()->route('admin.dashboard', [
1028            'search' => $request->input('return_search'),
```

```
            'role' => $request->input('return_role'),
        ])
            ->with('status', 'User role updated successfully.');
    }

    public function destroyUser(Request $request, \App\Models\User $user)
    {
        // Delete a user account. Only the head admin may perform this action.
        $this->requireSuperAdmin();

        if ($user->isSuperAdmin()) {
            if ($request->expectsJson()) {
                return response()->json(['message' => 'Cannot delete the head admin account.'], 403);
            }

            return redirect()->route('admin.dashboard', [
                'search' => $request->input('return_search'),
                'role' => $request->input('return_role'),
            ])
                ->with('error', 'Cannot delete the head admin account.');
        }

        if ($user->id === Auth::id()) {
            if ($request->expectsJson()) {
                return response()->json(['message' => 'You cannot delete your own account here.'], 403);
            }

            return redirect()->route('admin.dashboard', [
                'search' => $request->input('return_search'),
                'role' => $request->input('return_role'),
            ])
                ->with('error', 'You cannot delete your own account here.');
        }

        $user->delete();

        AuditLog::create([
            'user_id' => Auth::id(),
            'reservation_id' => null,
            'action' => 'user_deleted',
            'details' => sprintf('Deleted user %s', $user->email),
        ]);

        if ($request->expectsJson()) {
            return response()->json(['status' => 'User deleted successfully.']);
        }

        return redirect()->route('admin.dashboard', [
            'search' => $request->input('return_search'),
            'role' => $request->input('return_role'),
        ])
            ->with('status', 'User deleted successfully.');
    }
}
```

app/Http/Controllers/AuthController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Otp;
use App\Models\User;
use App\Models\UserProfilePicture;
use App\Notifications\PasswordChanged;
use App\Notifications\PasswordResetLink;
use Endroid\QrCode\QrCode;
use Endroid\QrCode\Writer\SvgWriter;
use GuzzleHttp\Client;
use Illuminate\Http\Request;
use Illuminate\Support\Carbon;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Cookie;
use Illuminate\Support\Facades\DB;
```

```
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Mail;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;
use Laravel\Socialite\Facades\Socialite;
use OTPHP\TOTP;

class AuthController extends Controller
{
    /**
     * Show the login form
     */
    public function showLogin()
    {
        $pendingUserId = session('pending_2fa_user_id');
        if ($pendingUserId) {
            $otp = Otp::where('user_id', $pendingUserId)->latest()->first();

            if (! $otp || $otp->isExpired()) {
                session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'time_left', 'pending_admin_2fa']);
            }
        }

        return view('auth.login');
    }

    public function showForgotPassword()
    {
        return view('auth.forgot-password');
    }

    public function sendPasswordResetLink(Request $request)
    {
        $request->validate(['email' => ['required', 'email']]);
        $email = strtolower($request->email);
        $user = User::where('email_hash', hash('sha256', $email))->first();

        if ($user) {
            $this->sendPasswordResetLinkToEmail($email);
        }

        return back()->with('status', 'If an account exists for that email, a password reset link has been sent.');
    }

    public function sendAuthenticatedPasswordResetLink(Request $request)
    {
        $user = Auth::user();
        $this->sendPasswordResetLinkToEmail(strtolower($user->email));

        return response()->json([
            'status' => 'A password reset email was queued for your account address. Check your inbox and spam folder shortly.',
        ]);
    }

    private function sendPasswordResetLinkToEmail(string $email): void
    {
        $token = Str::random(64);
        DB::table('password_reset_tokens')->updateOrInsert(
            ['email' => $email],
            ['token' => Hash::make($token), 'created_at' => now()]
        );

        $resetUrl = route('password.reset', ['token' => $token, 'email' => $email]);
        $user = User::where('email_hash', hash('sha256', $email))->firstOrFail();
        $user->notify(new PasswordResetLink($resetUrl));
    }

    public function showResetPassword(Request $request, string $token)
    {
        return view('auth.reset-password', [
            'token' => $token,
            'email' => $request->query('email'),
        ]);
    }
```

```
92
93      public function resetPassword(Request $request)
94      {
95          $request->validate([
96              'token' => ['required', 'string'],
97              'email' => ['required', 'email'],
98              'password' => ['required', 'confirmed', 'min:5', 'max:30'],
99          ]);
100
101         $email = strtolower($request->email);
102         $reset = DB::table('password_reset_tokens')->where('email', $email)->first();
103         if (! $reset || Carbon::parse($reset->created_at)->addHour()->isPast() || ! Hash::check($request->token, $reset->token)) {
104             return back()->withErrors(['email' => 'This password reset link is invalid or expired.'])->withInput();
105         }
106
107         $user = User::where('email_hash', hash('sha256', $email))->first();
108         if (! $user) {
109             return back()->withErrors(['email' => 'This password reset link is invalid or expired.'])->withInput();
110         }
111
112         $user->update(['password' => $request->password]);
113         DB::table('password_reset_tokens')->where('email', $email)->delete();
114
115         return redirect()->route('login')->with('status', 'Your password has been reset. You can now log in.');
116     }
117
118     /**
119      * Show the hidden admin login page.
120      */
121     public function showAdminLogin()
122     {
123         session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'time_left', 'pending_admin_2fa']);
124
125         return view('auth.admin-login');
126     }
127
128     /**
129      * Show admin 2FA verification page.
130      */
131     public function showAdmin2fa()
132     {
133         if (! session('pending_2fa_user_id') || ! session('pending_admin_2fa')) {
134             return redirect()->route('admin.login');
135         }
136
137         return view('auth.admin-login');
138     }
139
140     /**
141      * Handle admin login request.
142      */
143     public function loginAdmin(Request $request)
144     {
145         // Check admin email and password. This route is hidden from regular users.
146         $request->validate([
147             'email' => ['required', 'email'],
148             'password' => ['required', 'min:5', 'max:30'],
149         ]);
150
151         $emailHash = hash('sha256', strtolower($request->email));
152         $user = User::where('email_hash', $emailHash)->first();
153
154         logger()->debug('Admin login request received', [
155             'email' => $request->email,
156             'remember_device' => $request->boolean('remember_device'),
157         ]);
158
159         if (! $user || ! $user->is_admin || ! Hash::check($request->password, $user->password)) {
160             logger()->debug('Admin login request failed credentials', [
161                 'email' => $request->email,
162                 'user_exists' => $user !== null,
163                 'is_admin' => $user ? $user->is_admin : null,
164             ]);
165
```

```
            return back()->withErrors([
                'email' => 'The provided admin credentials are invalid.',
            ])->onlyInput('email');
        }

        if (! $user->two_fa_enabled) {
            logger()->debug('Admin login user has 2FA disabled', ['user_id' => $user->id]);
            Auth::login($user);
            $request->session()->regenerate();

            return redirect('/admin/dashboard');
        }

        logger()->debug('Admin login user requires 2FA', ['user_id' => $user->id]);
        session()->put('remember_device', $request->boolean('remember_device'));

        try {
            $this->prepareLoginTwoFactor($user);
        } catch (\Exception $e) {
            logger()->error('Admin OTP generation failed', [
                'user_id' => $user->id,
                'exception' => $e->getMessage(),
            ]);

            return back()
                ->withErrors(['email' => 'Unable to generate the 2FA code. Please try again.'])
                ->onlyInput('email');
        }

        session()->put([
            'pending_2fa_user_id' => $user->id,
            'pending_2fa_email' => $user->email,
            'pending_admin_2fa' => true,
            'time_left' => 30,
        ]);

        return redirect('/admin-2fa')->with('show_2fa_modal', true);
    }

    /**
     * Handle login request
     */
    public function login(Request $request)
    {
        // Check the user's email and password
        // If they have 2FA enabled, I'll send them a code
        $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required', 'min:5', 'max:30'],
        ]);

        logger()->debug('Login request received', [
            'route' => $request->path(),
            'email' => $request->email,
            'remember_device' => $request->boolean('remember_device'),
        ]);

        $emailHash = hash('sha256', strtolower($request->email));
        $user = User::where('email_hash', $emailHash)->first();

        if (! $user || ! Hash::check($request->password, $user->password)) {
            logger()->debug('Login request failed credentials', [
                'email' => $request->email,
                'user_exists' => $user !== null,
            ]);

            session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'time_left', 'pending_admin_2fa']);

            if ($request->expectsJson()) {
                return response()->json(['message' => 'The provided credentials do not match our records.'], 422);
            }

            return back()->withErrors([
                'email' => 'The provided credentials do not match our records.',
```

```
            ])->onlyInput('email');
        }

        if ($this->hasValidRememberedDevice($request, $user)) {
            Auth::login($user);
            $request->session()->regenerate();

            if ($request->expectsJson()) {
                return response()->json(['redirect' => url('/reserve')])->withCookie($this->refreshRememberDeviceCookie($user));
            }

            return redirect('/reserve')->withCookie($this->refreshRememberDeviceCookie($user));
        }

        if (! $user->two_fa_enabled) {
            logger()->debug('Login user has 2FA disabled', ['user_id' => $user->id]);
            Auth::login($user);
            $request->session()->regenerate();

            if ($request->expectsJson()) {
                return response()->json(['redirect' => url('/reserve')]);
            }

            return redirect('/reserve');
        }

        logger()->debug('Login user requires 2FA', ['user_id' => $user->id]);
        session()->put('remember_device', $request->boolean('remember_device'));

        try {
            $this->prepareLoginTwoFactor($user);
        } catch (\Exception $e) {
            logger()->error('OTP generation failed', [
                'user_id' => $user->id,
                'exception' => $e->getMessage(),
            ]);

            if ($request->expectsJson()) {
                return response()->json(['message' => 'Unable to generate the 2FA code. Please try again.'], 422);
            }

            return back()->withErrors(['email' => 'Unable to generate the 2FA code. Please try again.'])->onlyInput('email');
        }

        session()->put([
            'pending_2fa_user_id' => $user->id,
            'pending_2fa_email' => $user->email,
        ]);

        if ($request->expectsJson()) {
            return response()->json([
                'two_factor_required' => true,
                'method' => session('pending_2fa_method'),
                'email' => $user->email,
            ]);
        }

        return back()->with([
            'show_2fa_modal' => true,
            'status' => session('pending_2fa_method') === 'authenticator'
                ? 'Enter the code from your authenticator app.'
                : 'A 6-digit verification code has been sent to your email.',
        ]);
    }

    private function issueOtp(User $user)
    {
        // Create a brand new one-time password (OTP) for 2FA
        // Hash it and set to expire in 30 seconds
        logger()->debug('issueOtp called', ['user_id' => $user->id, 'email' => $user->email]);
        $otpCode = str_pad(random_int(0, 999999), 6, '0', STR_PAD_LEFT);

        // Delete any existing active OTPs for this user
        Otp::where('user_id', $user->id)->delete();
```

```
314
315         // Create new OTP record in database
316         Otp::create([
317             'user_id' => $user->id,
318             'code' => Hash::make($otpCode),
319             'expires_at' => now()->addSeconds(30),
320             'attempts' => 0,
321         ]);
322
323         $this->sendOtpEmail($user, $otpCode);
324
325         session()->put('time_left', 30);
326     }
327
328     private function prepareLoginTwoFactor(User $user): void
329     {
330         if ($user->two_fa_secret) {
331             session()->put([
332                 'pending_2fa_method' => 'authenticator',
333                 'authenticator_2fa_attempts' => 0,
334                 'two_fa_attempts_remaining' => 3,
335             ]);
336
337             return;
338         }
339
340         session()->put('pending_2fa_method', 'email');
341         $this->issueOtp($user);
342     }
343
344     private function sendOtpEmail(User $user, string $otp): void
345     {
346         Mail::raw(
347             "Campus Reserve\n\nYour one-time password (OTP) is:\n\n{$otp}\n\nEnter this 6-digit code on the verification screen. It expires in 30 seconds. If you did not try
    to sign in, you can ignore this email.",
348             function ($message) use ($user): void {
349                 $message->to($user->email)
350                     ->subject('Your Campus Reserve OTP code');
351             }
352         );
353     }
354
355     public function setupAuthenticator(Request $request)
356     {
357         $user = Auth::user();
358
359         $isChangingPassword = $request->filled('password');
360         if ($isChangingPassword) {
361             $request->validate([
362                 'current_password' => ['required', 'string'],
363                 'password' => ['required', 'confirmed', 'min:5', 'max:30'],
364                 'password_2fa_otp' => ['nullable', 'digits:6'],
365             ]);
366
367             if (! Hash::check($request->current_password, $user->password)) {
368                 return $this->profileValidationResponse($request, 'current_password', 'The current password is incorrect.');
369             }
370
371             if ($user->two_fa_secret && ! $request->filled('password_2fa_otp')) {
372                 return $this->profileValidationResponse($request, 'password_2fa_otp', 'Enter your authenticator code to change your password.');
373             }
374
375             if ($user->two_fa_secret && ! TOTP::createFromSecret($user->two_fa_secret)->verify($request->password_2fa_otp)) {
376                 $attempts = (int) session('password_2fa_attempts', 0) + 1;
377                 if ($attempts >= 3) {
378                     session()->forget('password_2fa_attempts');
379                     if ($request->expectsJson()) {
380                         return response()->json(['message' => 'Too many invalid authenticator codes. Try again.'], 429);
381                     }
382
383                     return back()->withErrors(['password_2fa_otp' => 'Too many invalid authenticator codes. Try again.'])->withInput();
384                 }
385
386                 session(['password_2fa_attempts' => $attempts]);
```

```
                if ($request->expectsJson()) {
                    return response()->json(['message' => 'Invalid Authenticator Code', 'attempts_remaining' => 3 - $attempts], 422);
                }

                return $this->profileValidationResponse($request, 'password_2fa_otp', 'Invalid Authenticator Code');
            }

            session()->forget('password_2fa_attempts');
        }
        if (! Hash::check($request->input('setup_current_password'), $user->password)) {
            if ($request->expectsJson()) {
                return response()->json(['message' => 'The current password is incorrect.'], 422);
            }

            return back()->withErrors(['setup_current_password' => 'The current password is incorrect.']);
        }

        $totp = TOTP::generate();
        $totp->setLabel($user->email);
        $totp->setIssuer('Campus Reserve');
        session(['pending_authenticator_secret' => $totp->getSecret()]);

        $setup = [
            'authenticator_setup_uri' => $totp->getProvisioningUri(),
            'authenticator_setup_qr' => (new SvgWriter)->write(new QrCode($totp->getProvisioningUri()))->getDataUri(),
            'authenticator_setup_secret' => $totp->getSecret(),
        ];

        if ($request->expectsJson()) {
            return response()->json($setup);
        }

        return back()->with($setup);
    }

    public function confirmAuthenticator(Request $request)
    {
        $request->validate(['authenticator_otp' => ['required', 'digits:6']]);
        $secret = session('pending_authenticator_secret');
        $user = Auth::user();

        if (! $secret || ! TOTP::createFromSecret($secret)->verify($request->authenticator_otp, null, 29)) {
            if ($request->expectsJson()) {
                return response()->json([
                    'message' => 'The authenticator code is invalid.',
                ], 422);
            }

            return back()->withErrors(['authenticator_otp' => 'The authenticator code is invalid.'])->withInput();
        }

        $user->update(['two_fa_secret' => $secret, 'two_fa_enabled' => true]);
        session()->forget('pending_authenticator_secret');

        if ($request->expectsJson()) {
            return response()->json([
                'status' => 'Authenticator app 2FA is now enabled.',
            ]);
        }

        return back()->with('status', 'Authenticator app 2FA is now enabled.');
    }

    public function showProfile()
    {
        return view('profile');
    }

    public function verifyCurrentPassword(Request $request)
    {
        $request->validate([
            'current_password' => ['required', 'string'],
        ]);

```

```
        if (! Hash::check($request->current_password, Auth::user()->password)) {
            return response()->json(['message' => 'The current password is incorrect.'], 422);
        }

        return response()->json(['valid' => true]);
    }

    public function updateEmailNotifications(Request $request)
    {
        $validated = $request->validate([
            'email_notifications' => ['required', 'boolean'],
        ]);

        Auth::user()->update([
            'email_notifications' => $validated['email_notifications'],
        ]);

        return response()->json([
            'status' => $validated['email_notifications']
                ? 'Email notifications enabled.'
                : 'Email notifications disabled.',
        ]);
    }

    public function updateProfile(Request $request)
    {
        $isChangingPassword = $request->filled('password');

        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'country_code' => ['nullable', 'string', 'in:+1,+44,+63,+81,+91,+86,+33,+49,+39,+61'],
            'phone' => ['nullable', 'string', 'regex:/^[\+]?[0-9]{7,15}$/'],
            'two_fa_enabled' => ['sometimes', 'boolean'],
            'email_notifications' => ['sometimes', 'boolean'],
            'profile_picture' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
        ]);

        $user = Auth::user();

        if ($isChangingPassword) {
            $request->validate([
                'current_password' => ['required', 'string'],
                'password' => ['required', 'confirmed', 'min:5', 'max:30'],
                'password_2fa_otp' => ['nullable', 'digits:6'],
            ]);

            if (! Hash::check($request->current_password, $user->password)) {
                return $this->profileValidationResponse($request, 'current_password', 'The current password is incorrect.');
            }

            if ($user->two_fa_enabled && ! $user->two_fa_secret) {
                return $this->profileValidationResponse($request, 'password_2fa_otp', 'Authenticator verification is unavailable. Please set up 2FA again.');
            }

            if ($user->two_fa_enabled && ! $request->filled('password_2fa_otp')) {
                return $this->profileValidationResponse($request, 'password_2fa_otp', 'Enter your authenticator code to change your password.');
            }

            if ($user->two_fa_enabled && ! TOTP::createFromSecret($user->two_fa_secret)->verify($request->password_2fa_otp)) {
                $attempts = (int) session('password_2fa_attempts', 0) + 1;
                if ($attempts >= 3) {
                    session()->forget('password_2fa_attempts');

                    return response()->json(['message' => 'Too many invalid authenticator codes. Try again.'], 429);
                }

                session(['password_2fa_attempts' => $attempts]);

                return response()->json(['message' => 'Invalid Authenticator Code', 'attempts_remaining' => 3 - $attempts], 422);
            }

            session()->forget('password_2fa_attempts');
        }

```

```
        if ($request->boolean('two_fa_enabled') && ! $user->two_fa_enabled && ! $user->two_fa_secret) {
            return redirect()->route('profile.show')->withErrors([
                'two_fa_enabled' => 'Set up and confirm your authenticator app before enabling 2FA.',
            ]);
        }

        $isDisablingTwoFactor = $user->two_fa_enabled && ! $request->boolean('two_fa_enabled');
        if ($isDisablingTwoFactor) {
            $request->validate([
                'disable_current_password' => ['required', 'string'],
                'disable_2fa_otp' => ['nullable', 'digits:6'],
            ]);

            if (! Hash::check($request->disable_current_password, $user->password)) {
                if ($request->expectsJson()) {
                    return $this->profileValidationResponse($request, 'disable_current_password', 'The current password is incorrect.');
                }

                return back()->withErrors([
                    'disable_current_password' => 'The current password is incorrect.',
                ])->withInput();
            }

            if (! $request->filled('disable_2fa_otp')) {
                if ($user->two_fa_secret) {
                    if ($request->expectsJson()) {
                        return response()->json(['message' => 'Enter the current code from your authenticator app to disable 2FA.'], 422);
                    }

                    return back()->with('show_disable_2fa_otp', true)->with(
                        'status',
                        'Enter the current code from your authenticator app to disable 2FA.'
                    )->withInput();
                }

                $this->issueOtp($user);

                return back()->with('show_disable_2fa_otp', true)->with(
                    'status',
                    'A verification code was sent to your email. Enter it to disable 2FA.'
                )->withInput();
            }

            if ($user->two_fa_secret) {
                if (! TOTP::createFromSecret($user->two_fa_secret)->verify($request->disable_2fa_otp, null, 29)) {
                    if ($request->expectsJson()) {
                        return $this->profileValidationResponse($request, 'disable_2fa_otp', 'Invalid Authenticator Code');
                    }

                    return back()->withErrors([
                        'disable_2fa_otp' => 'Invalid Authenticator Code',
                    ])->with('show_disable_2fa_otp', true)->withInput();
                }

                $user->update(['two_fa_enabled' => false]);

                return back()->with('status', 'Two-factor authentication has been disabled.');
            }

            $otp = Otp::where('user_id', $user->id)->latest()->first();
            if (! $otp || ! $otp->isValid() || ! Hash::check($request->disable_2fa_otp, $otp->code)) {
                if ($otp && $otp->isValid()) {
                    $otp->incrementAttempts();
                }

                if ($request->expectsJson()) {
                    return $this->profileValidationResponse($request, 'disable_2fa_otp', 'Invalid Authenticator Code');
                }

                return back()->withErrors([
                    'disable_2fa_otp' => 'Invalid Authenticator Code',
                ])->with('show_disable_2fa_otp', true)->withInput();
            }

```

```
            $otp->delete();
        }

        $data = [
            'name' => $request->name,
            'two_fa_enabled' => $request->boolean('two_fa_enabled'),
            'email_notifications' => $request->boolean('email_notifications'),
        ];

        if ($request->phone) {
            // If the phone number already has a country code (starts with +), keep it
            // Otherwise, I'll add the country code they selected
            if (str_starts_with($request->phone, '+')) {
                $data['phone'] = $request->phone;
            } else {
                $countryCode = $request->country_code ?? '+63';
                $data['phone'] = $countryCode.$request->phone;
            }
        }

        if ($request->hasFile('profile_picture')) {
            $data['profile_picture'] = $request->file('profile_picture')->store('profile_pictures', 'public');
        }

        if ($isChangingPassword) {
            $data['password'] = $request->password;
        }

        $user->update($data);

        if ($isChangingPassword) {
            try {
                $user->notify(new PasswordChanged);
            } catch (\Throwable $exception) {
                logger()->error('Password change security email could not be sent.', [
                    'user_id' => $user->id,
                    'exception' => $exception->getMessage(),
                ]);
            }
        }

        if ($request->expectsJson()) {
            return response()->json([
                'status' => $isChangingPassword ? 'Password changed successfully.' : 'Profile updated successfully.',
                'password_changed' => $isChangingPassword,
                'user' => [
                    'name' => $user->name,
                    'phone' => $user->phone,
                    'two_fa_enabled' => $user->two_fa_enabled,
                ],
            ]);
        }

        return back()->with('status', $isChangingPassword ? 'Password changed successfully.' : 'Profile updated successfully.');
    }

    public function updateEmail(Request $request)
    {
        $validated = $request->validate([
            'email' => ['required', 'string', 'email', 'max:255'],
            'current_password' => ['required', 'string'],
        ]);

        $user = Auth::user();
        if (! Hash::check($validated['current_password'], $user->password)) {
            return response()->json([
                'message' => 'The current password is incorrect.',
                'errors' => ['current_password' => ['The current password is incorrect.']],
            ], 422);
        }

        $email = trim($validated['email']);
        $emailHash = hash('sha256', strtolower($email));

```

```
        if (User::where('email_hash', $emailHash)->where('id', '!=', $user->id)->exists()) {
            return response()->json([
                'message' => 'This email address is already in use.',
                'errors' => ['email' => ['This email address is already in use.']],
            ], 422);
        }

        $user->email = $email;
        $user->save();

        return response()->json([
            'status' => 'Email address updated successfully.',
            'email' => $user->email,
        ]);
    }

    public function updateProfileName(Request $request)
    {
        $validated = $request->validate([
            'name' => ['required', 'string', 'max:255'],
        ]);

        $user = Auth::user();
        $user->name = $validated['name'];
        $user->save();

        return response()->json([
            'status' => 'Name saved automatically.',
            'name' => $user->name,
        ]);
    }

    public function uploadProfilePicture(Request $request)
    {
        $validated = $request->validate([
            'profile_picture' => ['required', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
        ]);

        $user = Auth::user();
        $previousPicture = $user->profile_picture;
        $imageContents = file_get_contents($validated['profile_picture']->getRealPath());
        if ($imageContents === false) {
            return response()->json([
                'message' => 'The profile picture could not be saved. Please try again.',
            ], 500);
        }

        DB::transaction(function () use ($user, $validated, $imageContents): void {
            $user->storedProfilePicture()->updateOrCreate([], [
                'mime_type' => $validated['profile_picture']->getMimeType(),
                'image_data' => base64_encode($imageContents),
            ]);

            $user->update(['profile_picture' => null]);
        });

        if ($previousPicture
            && str_starts_with($previousPicture, 'profile_pictures/')
            && $previousPicture !== 'profile_pictures/default-avatar.png') {
            Storage::disk('public')->delete($previousPicture);
        }

        return response()->json([
            'status' => 'Profile picture uploaded successfully.',
            'picture_url' => $user->profilePictureUrl(),
        ]);
    }

    public function showProfilePicture(User $user)
    {
        $picture = UserProfilePicture::where('user_id', $user->id)->firstOrFail();
        $imageContents = base64_decode($picture->image_data, true);
        abort_if($imageContents === false, 404);

```

```
        return response($imageContents)
            ->header('Content-Type', $picture->mime_type)
            ->header('Cache-Control', 'private, max-age=3600')
            ->header('X-Content-Type-Options', 'nosniff');
    }

    private function profileValidationResponse(Request $request, string $field, string $message)
    {
        if ($request->expectsJson()) {
            return response()->json(['message' => $message, 'field' => $field], 422);
        }

        return back()->withErrors([$field => $message])->withInput();
    }

    /**
     * Update theme preference via AJAX
     */
    private function hasValidRememberedDevice(Request $request, User $user): bool
    {
        $token = $request->cookie('remember_device');

        if (! $token || ! $user->remember_device_token || ! $user->remember_device_expires_at) {
            return false;
        }

        if ($user->remember_device_expires_at->isPast()) {
            return false;
        }

        return Hash::check($token, $user->remember_device_token);
    }

    private function rememberDeviceCookie(User $user)
    {
        $token = Str::random(60);
        $expires = now()->addDays(30);

        $user->update([
            'remember_device_token' => Hash::make($token),
            'remember_device_expires_at' => $expires,
        ]);

        return cookie(
            'remember_device',
            $token,
            60 * 24 * 30,
            '/',
            null,
            app()->environment('production'),
            true,
            false,
            'Strict'
        );
    }

    private function refreshRememberDeviceCookie(User $user)
    {
        if (! $user->remember_device_token) {
            return null;
        }

        return $this->rememberDeviceCookie($user);
    }

    private function forgetRememberDeviceCookie()
    {
        return Cookie::forget('remember_device');
    }

    /**
     * Show the registration form
     */
    public function showRegister()
```

```
    {
        $googleSignup = session('pending_google_signup');

        return view('auth.register', [
            'googleSignup' => ! empty($googleSignup),
            'googleName' => $googleSignup['name'] ?? null,
            'googleEmail' => $googleSignup['email'] ?? null,
        ]);
    }

    /**
     * Handle registration request
     */
    public function register(Request $request)
    {
        $googleSignup = session('pending_google_signup');

        if ($googleSignup) {
            $request->merge([
                'name' => $request->input('name', $googleSignup['name'] ?? ''),
                'email' => $googleSignup['email'] ?? $request->input('email'),
            ]);
        }

        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255'],
            'password' => ['required', 'confirmed', 'min:5', 'max:30'],
        ]);

        $emailHash = hash('sha256', strtolower($request->email));
        if (User::where('email_hash', $emailHash)->exists()) {
            return back()->withErrors([
                'email' => 'This email is already registered.',
            ])->onlyInput('email', 'name');
        }

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
            'profile_picture' => 'profile_pictures/default-avatar.png', // Default avatar
        ]);

        session()->forget('pending_google_signup');
        Auth::login($user);

        return redirect('/reserve');
    }

    /**
     * Show 2FA verification form
     */
    public function show2faVerify(Request $request)
    {
        $userId = session('pending_2fa_user_id');
        if (! $userId) {
            return redirect('/login');
        }

        $user = User::findOrFail($userId);
        $otp = Otp::where('user_id', $userId)->latest()->first();

        if (! $user->two_fa_secret && (! $otp || $otp->isExpired())) {
            $request->session()->forget([
                'pending_2fa_user_id',
                'pending_2fa_email',
                'pending_2fa_method',
                'pending_2fa_provider',
                'pending_2fa_redirect',
                'pending_google_2fa',
                'show_2fa_modal',
                'time_left',
            ]);
```

```
905
906             return redirect('/login')->with('error', '2FA code expired. Please login again.');
907         }
908
909         $method = session('pending_2fa_method', $user->two_fa_secret ? 'authenticator' : 'email');
910         $provider = session('pending_2fa_provider');
911         $attemptsRemaining = $user->two_fa_secret
912             ? max(0, 3 - (int) session('authenticator_2fa_attempts', 0))
913             : max(0, 3 - (int) $otp->attempts);
914
915         return view('auth.2fa-verify', compact('method', 'provider', 'attemptsRemaining'));
916     }
917
918     /**
919      * Verify 2FA code
920      */
921     public function verify2fa(Request $request)
922     {
923         $request->validate([
924             'otp' => 'required|digits:6',
925         ]);
926
927         $userId = session('pending_2fa_user_id');
928         $user = User::findOrFail($userId);
929         $otp = Otp::where('user_id', $userId)->latest()->first();
930
931         if ($user->two_fa_secret) {
932             $attempts = (int) session('authenticator_2fa_attempts', 0);
933             if ($attempts >= 3) {
934                 $request->session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'pending_2fa_method', 'pending_2fa_provider',
    'pending_2fa_redirect', 'pending_google_2fa', 'authenticator_2fa_attempts']);
935                 if ($request->expectsJson()) {
936                     return response()->json(['message' => 'Too many invalid authenticator codes. Please log in again.', 'redirect' => route('login')], 429);
937                 }
938
939                 return redirect()->route('login')->withErrors(['email' => 'Too many invalid authenticator codes. Please log in again.']);
940             }
941
942             if (! TOTP::createFromSecret($user->two_fa_secret)->verify($request->otp, null, 29)) {
943                 $attempts++;
944                 if ($attempts >= 3) {
945                     $request->session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'pending_2fa_method', 'pending_2fa_provider',
    'pending_2fa_redirect', 'pending_google_2fa', 'authenticator_2fa_attempts']);
946                     if ($request->expectsJson()) {
947                         return response()->json(['message' => 'Too many invalid authenticator codes. Please log in again.', 'redirect' => route('login')], 429);
948                     }
949
950                     return redirect()->route('login')->withErrors(['email' => 'Too many invalid authenticator codes. Please log in again.']);
951                 }
952
953                 session([
954                     'authenticator_2fa_attempts' => $attempts,
955                     'two_fa_attempts_remaining' => 3 - $attempts,
956                 ]);
957                 if ($request->expectsJson()) {
958                     return response()->json(['message' => 'Invalid Authenticator Code', 'attempts_remaining' => 3 - $attempts], 422);
959                 }
960
961                 return back()->with(['show_2fa_modal' => true, 'two_fa_attempts_remaining' => 3 - $attempts])->withErrors(['otp' => 'Invalid Authenticator Code']);
962             }
963
964             $redirectUrl = session()->pull('pending_2fa_redirect');
965             $request->session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'time_left', 'pending_2fa_method', 'pending_2fa_provider',
    'pending_google_2fa', 'authenticator_2fa_attempts']);
966             $isAdminLogin = session()->pull('pending_admin_2fa', false);
967             Auth::login($user);
968             $request->session()->regenerate();
969             if ($request->expectsJson()) {
970                 return response()->json(['redirect' => $redirectUrl ?? ($isAdminLogin ? url('/admin/dashboard') : url('/reserve'))]);
971             }
972
973             return redirect($redirectUrl ?? ($isAdminLogin ? '/admin/dashboard' : '/reserve'));
974         }
975
```

```
        if (! $otp || ! $otp->isValid()) {
            return back()->with([
                'show_2fa_modal' => true,
            ])->withErrors(['otp' => 'Code expired or too many attempts. Please resend a new code.']);
        }

        if (Hash::check($request->otp, $otp->code)) {
            $otp->delete();

            $redirectUrl = session()->pull('pending_2fa_redirect');
            $request->session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'time_left', 'pending_2fa_method', 'pending_2fa_provider',
    'pending_google_2fa']);
            $isAdminLogin = session()->pull('pending_admin_2fa', false);

            Auth::login($user);
            $request->session()->regenerate();

            $response = redirect($redirectUrl ?? ($isAdminLogin ? '/admin/dashboard' : '/reserve'));
            if (session()->pull('remember_device', false)) {
                $response->withCookie($this->rememberDeviceCookie($user));
            }

            return $response;
        }

        $otp->incrementAttempts();

        return back()->with([
            'show_2fa_modal' => true,
        ])->withErrors(['otp' => 'Invalid code. Try again.']);
    }

    /**
     * Resend OTP
     */
    public function resend2fa(Request $request)
    {
        $userId = session('pending_2fa_user_id');
        $user = User::find($userId);

        if (! $user) {
            session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'time_left', 'pending_admin_2fa']);
            if ($request->wantsJson()) {
                return response()->json(['error' => 'Session expired. Login again.'], 440);
            }

            return redirect('/login')->with('error', 'Session expired. Login again.');
        }

        $this->prepareLoginTwoFactor($user);

        session()->flash('show_2fa_modal', true);

        if ($request->wantsJson()) {
            return response()->json([
                'status' => 'A new 6-digit code has been generated and logged.',
                'time_left' => 30,
            ]);
        }

        return back()->with([
            'show_2fa_modal' => true,
            'status' => 'A new 6-digit code has been generated and logged.',
        ]);
    }

    /**
     * Handle logout request
     */
    public function logout(Request $request)
    {
        $user = Auth::user();
        if ($user) {
            Otp::where('user_id', $user->id)->delete();
```

```
        }
        Auth::logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
        $request->session()->forget(['pending_2fa_user_id', 'pending_2fa_email', 'show_2fa_modal', 'time_left', 'pending_admin_2fa', 'pending_google_signup']);

        return redirect('/')->withCookie($this->forgetRememberDeviceCookie());
    }

    /**
     * Redirect to Google OAuth
     */
    public function redirectToGoogle(Request $request)
    {
        $redirectUrl = $request->getSchemeAndHttpHost().'/auth/google/callback';

        return Socialite::driver('google')
            ->redirectUrl($redirectUrl)
            ->redirect();
    }

    /**
     * Handle Google OAuth callback
     */
    public function handleGoogleCallback(Request $request)
    {
        try {
            // Skip SSL checking for testing on localhost
            $httpClient = new Client(['verify' => false]);
            $googleUser = Socialite::driver('google')
                ->setHttpClient($httpClient)
                ->user();
        } catch (\Exception $e) {
            // Record the error for debugging
            \Log::error('Google OAuth Error: '.$e->getMessage());

            return redirect('/login')->with('error', 'Failed to authenticate with Google.');
        }

        // Check if user exists by email hash
        $emailHash = hash('sha256', strtolower($googleUser->getEmail()));
        $user = User::where('email_hash', $emailHash)->first();

        // New Google users complete the normal signup flow with their own password confirmation.
        if (! $user) {
            session()->put('pending_google_signup', [
                'name' => $googleUser->getName() ?: 'User',
                'email' => $googleUser->getEmail(),
            ]);

            return redirect()->route('register');
        }

        if ($user->two_fa_enabled) {
            try {
                $this->prepareLoginTwoFactor($user);
            } catch (\Exception $e) {
                logger()->error('Google sign-in 2FA setup failed', [
                    'user_id' => $user->id,
                    'exception' => $e->getMessage(),
                ]);

                return redirect('/login')->withErrors([
                    'email' => 'Unable to start two-factor verification. Please try again.',
                ]);
            }

            $request->session()->put([
                'pending_2fa_user_id' => $user->id,
                'pending_2fa_email' => $user->email,
                'pending_2fa_provider' => 'google',
                'pending_2fa_redirect' => '/reserve',
                'pending_google_2fa' => true,
            ]);
```

```
            return redirect()->route('2fa.show')->with(
                'status',
                session('pending_2fa_method') === 'authenticator'
                    ? 'Google sign-in succeeded. Enter your Campus Reserve authenticator code to continue.'
                    : 'Google sign-in succeeded. Enter the verification code sent to your email to continue.'
            );
        }

        // Sign users without Campus Reserve two-factor authentication in directly.
        Auth::login($user);
        $request->session()->regenerate();

        return redirect('/reserve');
    }
}
```

app/Http/Controllers/CalendarController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Reservation;
use App\Models\Facility;
use App\Models\FacilityMaintenanceBlock;
use App\Models\FacilityItem;
use App\Models\User;
use App\Notifications\ReservationPendingApproval;
use App\Services\FacilityItemAvailability;
use App\Services\FacilityMaintenanceSchedule;
use App\Services\ReservationNotificationDispatcher;
use App\Services\ReservationDateWindow;
use App\Services\ReservationTimeWindow;
use Illuminate\Database\UniqueConstraintViolationException;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Notification;
use Illuminate\Validation\ValidationException;

class CalendarController extends Controller
{
    // Get approved reservations in the visible calendar range and format them as events.
    public function events(
        Request $request,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        ReservationTimeWindow $reservationTimeWindow
    )
    {
        // The shared calendar shows approved facility bookings, while keeping
        // each user's reservation purpose private from other users.
        $validated = $request->validate([
            'start' => ['required', 'date_format:Y-m-d'],
            'end' => ['required', 'date_format:Y-m-d', 'after:start'],
            'facility' => ['sometimes', 'string', 'max:100'],
            'facility_id' => ['sometimes', 'integer', 'exists:facilities,id'],
        ]);

        $query = Reservation::with('facility')
            ->where('status', 'approved')
            ->whereNotNull('facility_id')
            ->where('reservation_date', '>=', $validated['start'])
            ->where('reservation_date', '<', $validated['end']);

        if (! empty($validated['facility_id'])) {
            $query->where('facility_id', $validated['facility_id']);
        } elseif (! empty($validated['facility'])) {
            $facilityName = $validated['facility'];
            $query->whereHas('facility', function ($q) use ($facilityName) {
                $q->where('name', $facilityName);
            });
        }
        $reservations = $query->get();
```

```
        $events = $reservations->map(function ($reservation) {
            // Format the dates and times so the calendar can understand them
            $dateStr = $reservation->reservation_date->format('Y-m-d');
            // Make sure times are in the right format (hours:minutes:seconds)
            $startTime = $reservation->start_time->format('H:i:00');
            $endTime = $reservation->end_time->format('H:i:00');
            $start = $dateStr . 'T' . $startTime;
            $end = $dateStr . 'T' . $endTime;
            return [
                'id' => $reservation->id,
                'title' => $reservation->facility->name,
                'start' => $start,
                'end' => $end,
                'color' => $this->getReservationColor($reservation->id),
                'extendedProps' => [
                    'purpose' => $reservation->user_id === Auth::id() ? $reservation->purpose : null,
                    'status' => $reservation->status,
                    'room' => $reservation->facility->name,
                    'date' => $dateStr,
                    'is_owner' => $reservation->user_id === Auth::id(),
                ],
            ];
        });

        $hours = $reservationTimeWindow->current();
        $maintenanceEvents = $maintenanceSchedule
            ->forDateRange(
                $validated['start'],
                $validated['end'],
                $validated['facility'] ?? null,
                isset($validated['facility_id']) ? (int) $validated['facility_id'] : null
            )
            ->map(function (FacilityMaintenanceBlock $block) use ($hours) {
                $date = $block->maintenance_date->format('Y-m-d');
                $startTime = $block->is_all_day ? $hours['start_time'] : $block->start_time;
                $endTime = $block->is_all_day ? $hours['end_time'] : $block->end_time;

                return [
                    'id' => 'maintenance-'.$block->id,
                    'title' => 'Unavailable: '.$block->facility->name,
                    'start' => $date.'T'.$startTime.':00',
                    'end' => $date.'T'.$endTime.':00',
                    'color' => '#FEE2E2',
                    'extendedProps' => [
                        'type' => 'maintenance',
                        'is_maintenance' => true,
                        'is_all_day' => $block->is_all_day,
                        'reason' => $block->reason,
                        'purpose' => $block->reason,
                        'status' => 'unavailable',
                        'room' => $block->facility->name,
                        'date' => $date,
                        'is_owner' => false,
                    ],
                ];
            });

        $events = $events->concat($maintenanceEvents)->values();
        return response()->json($events);
    }

    // Save a new reservation to the database
    public function store(
        Request $request,
        FacilityItemAvailability $itemAvailability,
        ReservationNotificationDispatcher $reservationNotifications,
        ReservationTimeWindow $reservationTimeWindow,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        \App\Services\PropertyItemMaintenanceSchedule $propertyItemMaintenanceSchedule,
        ReservationDateWindow $reservationDateWindow
    )
    {
        // First, I validate that all the required info is provided
        // Then I check if the room is actually available at that time
```

```
        $hasItemBookings = $request->has('items');
        $validated = $request->validate([
            'facility' => $hasItemBookings ? ['nullable', 'string'] : ['required', 'string'],
            'purpose' => 'required|string|max:255',
            'date' => $reservationDateWindow->rules(),
            'start_time' => 'required|date_format:H:i',
            'end_time' => 'required|date_format:H:i|after:start_time',
            'items' => $hasItemBookings ? ['required', 'array', 'min:1'] : ['sometimes', 'array'],
            'items.*.facility_item_id' => ['required', 'integer', 'distinct', 'exists:facility_items,id'],
            'items.*.quantity' => ['required', 'integer', 'min:1'],
        ]);

        if (! $reservationTimeWindow->allows($validated['start_time'], $validated['end_time'])) {
            return response()->json([
                'error' => $reservationTimeWindow->validationMessage(),
            ], 422);
        }

        $facility = null;
        if (! $hasItemBookings) {
            $facilityInput = $validated['facility'];
            $facility = ctype_digit($facilityInput)
                ? Facility::active()->findOrFail((int) $facilityInput)
                : Facility::active()->where('name', $facilityInput)->firstOrFail();
        }

        if ($facility) {
            $maintenanceBlock = $maintenanceSchedule->conflictingBlock(
                $facility->id,
                $validated['date'],
                $validated['start_time'],
                $validated['end_time']
            );
            if ($maintenanceBlock) {
                return response()->json([
                    'code' => 'facility_maintenance',
                    'error' => $maintenanceSchedule->validationMessage($maintenanceBlock),
                ], 409);
            }

            if (! $facility->isAvailable($validated['date'], $validated['start_time'], $validated['end_time'])) {
                return response()->json([
                    'code' => 'schedule_occupied',
                    'error' => "This schedule and room is already occupied!\nPlease try another time or room!",
                ], 409);
            }
        }

        try {
            $reservation = DB::transaction(function () use ($validated, $facility, $hasItemBookings, $itemAvailability, $propertyItemMaintenanceSchedule) {
                $selectedItems = collect($validated['items'] ?? []);

                if ($hasItemBookings) {
                    $itemIds = $selectedItems->pluck('facility_item_id')->map(fn ($id) => (int) $id)->all();
                    $lockedItems = FacilityItem::query()
                        ->whereIn('id', $itemIds)
                        ->orderBy('id')
                        ->lockForUpdate()
                        ->get()
                        ->keyBy('id');

                    if ($lockedItems->count() !== count($itemIds)) {
                        throw ValidationException::withMessages([
                            'items' => 'One or more selected items are no longer available.',
                        ]);
                    }

                    foreach ($selectedItems as $selection) {
                        $item = $lockedItems->get((int) $selection['facility_item_id']);
                        $propertyMaintenanceBlock = $propertyItemMaintenanceSchedule->conflictingBlock(
                            $item->id,
                            $validated['date'],
                            $validated['start_time'],
                            $validated['end_time']
```

```
                        );
                        if ($propertyMaintenanceBlock) {
                            return response()->json([
                                'code' => 'property_item_maintenance',
                                'error' => $propertyItemMaintenanceSchedule->validationMessage($propertyMaintenanceBlock),
                            ], 409);
                        }

                        $availableQuantity = $itemAvailability->availableQuantity(
                            $item,
                            $validated['date'],
                            $validated['start_time'],
                            $validated['end_time']
                        );

                        if ((int) $selection['quantity'] > $availableQuantity) {
                            return response()->json([
                                'error' => "{$item->name} has only {$availableQuantity} available for that date and time.",
                            ], 409);
                        }
                    }
                }

                $reservation = Reservation::create([
                    'user_id' => Auth::id(),
                    'facility_id' => $facility?->id,
                    'reservation_date' => $validated['date'],
                    'start_time' => $validated['start_time'],
                    'end_time' => $validated['end_time'],
                    'purpose' => $validated['purpose'],
                    'status' => 'pending',
                ]);

                if ($hasItemBookings) {
                    $reservation->items()->attach($selectedItems->mapWithKeys(fn ($selection) => [
                        (int) $selection['facility_item_id'] => ['quantity' => (int) $selection['quantity']],
                    ])->all());
                }

                return $reservation;
            });
        } catch (UniqueConstraintViolationException $exception) {
            if (! $facility) {
                throw $exception;
            }

            $sameStartTimeExists = Reservation::query()
                ->where('facility_id', $facility->id)
                ->whereDate('reservation_date', $validated['date'])
                ->get(['start_time'])
                ->contains(fn (Reservation $existingReservation): bool =>
                    $existingReservation->start_time->format('H:i') === $validated['start_time']
                );

            if (! $sameStartTimeExists) {
                throw $exception;
            }

            return response()->json([
                'code' => 'schedule_occupied',
                'error' => "This schedule and room is already occupied!\nPlease try another time or room!",
            ], 409);
        }

        if ($reservation instanceof \Illuminate\Http\JsonResponse) {
            return $reservation;
        }

        $admins = User::where('is_admin', true)
            ->orWhere('is_super_admin', true)
            ->get();

        Notification::send($admins, new ReservationPendingApproval($reservation));
        $reservationNotifications->notifyBorrower($reservation);
```

```
278
279         return response()->json(['success' => true, 'reservation' => $reservation]);
280     }
281
282     public function itemAvailability(
283         Request $request,
284         Facility $facility,
285         FacilityItemAvailability $itemAvailability,
286         ReservationTimeWindow $reservationTimeWindow,
287         FacilityMaintenanceSchedule $maintenanceSchedule,
288         \App\Services\PropertyItemMaintenanceSchedule $propertyItemMaintenanceSchedule,
289         ReservationDateWindow $reservationDateWindow
290     ) {
291         abort_unless($facility->is_active, 404);
292
293         $validated = $request->validate([
294             'date' => $reservationDateWindow->rules(),
295             'start_time' => ['required', 'date_format:H:i'],
296             'end_time' => ['required', 'date_format:H:i', 'after:start_time'],
297         ]);
298
299         if (! $reservationTimeWindow->allows($validated['start_time'], $validated['end_time'])) {
300             return response()->json([
301                 'message' => $reservationTimeWindow->validationMessage(),
302             ], 422);
303         }
304
305         $maintenanceBlock = $maintenanceSchedule->conflictingBlock(
306             $facility->id,
307             $validated['date'],
308             $validated['start_time'],
309             $validated['end_time']
310         );
311         if ($maintenanceBlock) {
312             return response()->json([
313                 'message' => $maintenanceSchedule->validationMessage($maintenanceBlock),
314             ], 409);
315         }
316
317         $items = $facility->items()
318             ->orderBy('name')
319             ->get();
320         $availableQuantities = $itemAvailability->availableQuantities(
321             $items,
322             $validated['date'],
323             $validated['start_time'],
324             $validated['end_time']
325         );
326
327         $items = $items->map(function (FacilityItem $item) use (
328             $availableQuantities,
329             $propertyItemMaintenanceSchedule,
330             $validated
331         ): array {
332             $maintenanceBlock = $propertyItemMaintenanceSchedule->conflictingBlock(
333                 $item->id,
334                 $validated['date'],
335                 $validated['start_time'],
336                 $validated['end_time']
337             );
338
339             return [
340                 'id' => $item->id,
341                 'name' => $item->name,
342                 'stock' => $item->quantity,
343                 'available' => $maintenanceBlock ? 0 : $availableQuantities[$item->id],
344                 'maintenance_reason' => $maintenanceBlock?->reason,
345             ];
346         });
347
348         return response()->json(['items' => $items]);
349     }
350
351     public function propertyItemAvailability(
```

```
        Request $request,
        FacilityItemAvailability $itemAvailability,
        ReservationTimeWindow $reservationTimeWindow,
        \App\Services\PropertyItemMaintenanceSchedule $propertyItemMaintenanceSchedule,
        ReservationDateWindow $reservationDateWindow
    ) {
        $validated = $request->validate([
            'date' => $reservationDateWindow->rules(),
            'start_time' => ['required', 'date_format:H:i'],
            'end_time' => ['required', 'date_format:H:i', 'after:start_time'],
        ]);

        if (! $reservationTimeWindow->allows($validated['start_time'], $validated['end_time'])) {
            return response()->json([
                'message' => $reservationTimeWindow->validationMessage(),
            ], 422);
        }

        $items = FacilityItem::query()->orderBy('name')->get();
        $availableQuantities = $itemAvailability->availableQuantities(
            $items,
            $validated['date'],
            $validated['start_time'],
            $validated['end_time']
        );

        return response()->json([
            'items' => $items->map(function (FacilityItem $item) use (
                $availableQuantities,
                $propertyItemMaintenanceSchedule,
                $validated
            ): array {
                $maintenanceBlock = $propertyItemMaintenanceSchedule->conflictingBlock(
                    $item->id,
                    $validated['date'],
                    $validated['start_time'],
                    $validated['end_time']
                );

                return [
                    'id' => $item->id,
                    'name' => $item->name,
                    'stock' => $item->quantity,
                    'available' => $maintenanceBlock ? 0 : $availableQuantities[$item->id],
                    'maintenance_reason' => $maintenanceBlock?->reason,
                ];
            }),
        ]);
    }

    public function pending(Request $request)
    {
        $pendingReservations = Reservation::with('facility')
            ->where('user_id', Auth::id())
            ->whereNotNull('facility_id')
            ->where('status', 'pending')
            ->orderBy('reservation_date')
            ->orderBy('start_time')
            ->get()
            ->map(function ($reservation) {
                return [
                    'id' => $reservation->id,
                    'facility' => $reservation->facility->name,
                    'date' => $reservation->reservation_date->format('M d, Y'),
                    'start_time' => $reservation->start_time->format('H:i'),
                    'end_time' => $reservation->end_time->format('H:i'),
                    'purpose' => $reservation->purpose,
                    'status' => $reservation->status,
                ];
            });

        return response()->json($pendingReservations);
    }

```

```
    // Pick a color for each reservation so they look different on the calendar
    private function getReservationColor($reservationId)
    {
        $colors = [
            '#DCEBE0', '#EAF0DE', '#F4E8CE', '#E2EDE8',
            '#F0E6D3', '#DDE9DF', '#E7EBDD', '#EFE8DA',
        ];

        $colorIndex = (intval($reservationId) % count($colors));
        return $colors[$colorIndex];
    }
}
```

app/Http/Controllers/ChatbotController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Facility;
use App\Models\FacilityItem;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationPendingApproval;
use App\Services\FacilityItemAvailability;
use App\Services\FacilityMaintenanceSchedule;
use App\Services\PropertyItemMaintenanceSchedule;
use App\Services\ReservationNotificationDispatcher;
use App\Services\ReservationDateWindow;
use App\Services\ReservationTimeWindow;
use Carbon\Carbon;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Notification;

class ChatbotController extends Controller
{
    public function reset(): JsonResponse
    {
        session()->forget([
            'chatbot_reservation',
            'chatbot_cancellation',
            'chatbot_last_action',
            'chatbot_last_reservation_type',
        ]);

        return response()->json(['success' => true]);
    }

    public function respond(
        Request $request,
        FacilityItemAvailability $itemAvailability,
        ReservationNotificationDispatcher $reservationNotifications,
        ReservationTimeWindow $reservationTimeWindow,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        PropertyItemMaintenanceSchedule $propertyItemMaintenanceSchedule,
        ReservationDateWindow $reservationDateWindow
    ): JsonResponse {
        $message = trim((string) $request->input('message'));
        $normalized = $this->normalizeMessage($message);

        if ($message === '') {
            return response()->json(['reply' => 'Ask me about available rooms, schedules, or reservations.'], 422);
        }

        [$safeMessage, $containsAbuse] = $this->moderateMessage($message);
        if ($containsAbuse) {
            return response()->json([
                'reply' => 'Please keep the conversation respectful. I can help with reservations, room availability, schedules, and your bookings.',
                'user_message' => $safeMessage,
                'moderated' => true,
            ], 422);
```

```
        }

        $state = session('chatbot_reservation', []);
        $providedPurpose = false;
        if ($state['awaiting_purpose'] ?? false) {
            if ($this->isNegative($normalized)) {
                session()->forget('chatbot_reservation');
                return response()->json(['reply' => 'Okay, I cancelled that reservation request.']);
            }

            if (mb_strlen($message) > 255) {
                return response()->json(['reply' => 'Please keep the reservation purpose under 255 characters.']);
            }

            $state['purpose'] = $message;
            unset($state['awaiting_purpose']);
            $providedPurpose = true;

            if ($state['confirm_after_purpose'] ?? false) {
                unset($state['confirm_after_purpose']);
                $normalized = 'yes';
            }

            session(['chatbot_reservation' => $state]);
        }

        $facilities = null;
        if (!$providedPurpose && !$this->isSupportedMessage($normalized)) {
            $facilities = Facility::active()->with('items')->orderBy('name')->get();
            $matchesFacility = $facilities->contains(
                fn (Facility $facility) => $this->messageMatchesFacility($facility, $normalized)
            );
            $matchesItem = $this->parsePropertyItems(FacilityItem::query()->get(), $normalized, [])->isNotEmpty();

            if (!$matchesFacility && !$matchesItem) {
                return response()->json([
                    'reply' => 'I can help with room and item availability, facility or property reservations, reservation status, schedules, and cancellations. For example, ask “What is my next booking?” or “Is Room 101 free next Friday from 9 to 11 AM?”',
                    'user_message' => $safeMessage,
                ]);
            }
        }

        $cancellation = session('chatbot_cancellation', []);
        if (($cancellation['awaiting_confirmation'] ?? false) && $this->isConfirmation($normalized)) {
            $reservation = Reservation::with(['facility', 'items'])
                ->where('user_id', Auth::id())
                ->find($cancellation['reservation_id'] ?? null);

            if (!$reservation || !$reservation->canBeCancelled()) {
                session()->forget('chatbot_cancellation');
                return response()->json(['reply' => 'That reservation can no longer be cancelled. Please check your current reservations.']);
            }

            $facilityName = $this->reservationFacilityLabel($reservation);
            $itemSummary = $this->reservationItemSummary($reservation);
            $reservation->delete();
            session()->forget('chatbot_cancellation');

            return response()->json(['reply' => "Your reservation for {$facilityName}{$itemSummary} on " . $reservation->reservation_date->format('F j, Y') . ' was cancelled successfully.']);
        }

        if (($cancellation['awaiting_confirmation'] ?? false) && $this->isNegative($normalized)) {
            session()->forget('chatbot_cancellation');
            return response()->json(['reply' => 'Okay, I did not cancel the reservation.']);
        }

        if (($cancellation['awaiting_selection'] ?? false)) {
            $reservation = Reservation::with(['facility', 'items'])
                ->where('user_id', Auth::id())
                ->whereIn('status', ['pending', 'approved'])
                ->whereDate('reservation_date', '>=', now()->toDateString())
                ->orderBy('reservation_date')
```

```
                ->orderBy('start_time')
                ->get()
                ->filter(fn (Reservation $reservation) => $reservation->canBeCancelled())
                ->first(function (Reservation $reservation) use ($normalized) {
                    $facilityName = preg_replace('/[^a-z0-9]/', '', strtolower($reservation->facility?->name ?? ''));
                    $question = preg_replace('/[^a-z0-9]/', '', $normalized);
                    $itemNames = $reservation->items->map(
                        fn (FacilityItem $item) => preg_replace('/[^a-z0-9]/', '', strtolower($item->name))
                    );

                    return ($facilityName !== '' && str_contains($question, $facilityName))
                        || $itemNames->contains(fn (string $itemName) => $itemName !== '' && str_contains($question, $itemName));
                });

            if ($reservation) {
                session(['chatbot_cancellation' => [
                    'reservation_id' => $reservation->id,
                    'awaiting_confirmation' => true,
                ]]);

                return response()->json(['reply' => "Do you want to cancel your {$this->reservationFacilityLabel($reservation)} reservation on " .
    $reservation->reservation_date->format('F j, Y') . '? Reply yes to confirm cancellation.']);
            }
        }

        if (($state['awaiting_confirmation'] ?? false) && $this->isConfirmation($normalized)) {
            if (empty($state['purpose'])) {
                $state['awaiting_purpose'] = true;
                $state['confirm_after_purpose'] = true;
                session(['chatbot_reservation' => $state]);

                return response()->json(['reply' => 'What is the purpose of this reservation?']);
            }

            $isPropertyReservation = ($state['reservation_type'] ?? 'facility') === 'property';
            $facility = $isPropertyReservation ? null : Facility::active()->find($state['facility_id'] ?? null);
            $date = $state['date'] ?? null;
            $startTime = $state['start_time'] ?? null;
            $endTime = $state['end_time'] ?? null;

            if ((! $facility && ! $isPropertyReservation) || ! $date || ! $startTime || ! $endTime) {
                session()->forget('chatbot_reservation');
                return response()->json(['reply' => $isPropertyReservation
                    ? 'I need the item, date, and time again before I can submit the request.'
                    : 'I need the room, date, and time again before I can submit the request.']);
            }

            if (! $reservationTimeWindow->allows($startTime, $endTime)) {
                return response()->json(['reply' => $reservationTimeWindow->validationMessage()]);
            }

            if ($facility) {
                $maintenanceBlock = $maintenanceSchedule->conflictingBlock(
                    $facility->id,
                    $date,
                    $startTime,
                    $endTime
                );
                if ($maintenanceBlock) {
                    session()->forget('chatbot_reservation');
                    return response()->json(['reply' => $maintenanceSchedule->validationMessage($maintenanceBlock)]);
                }

                if (! $facility->isAvailable($date, $startTime, $endTime)) {
                    session()->forget('chatbot_reservation');
                    return response()->json(['reply' => 'That time is no longer available. Please choose another time or room.']);
                }
            }

            $requestedItems = collect($state['items'] ?? []);
            if ($isPropertyReservation && $requestedItems->isEmpty()) {
                session()->put('chatbot_reservation.awaiting_confirmation', false);
                return response()->json(['reply' => 'Select at least one property item and quantity before I can submit the request. Tell me the item name and quantity.']);
            }
```

```
205
206             $reservation = DB::transaction(function () use (
207                 $facility,
208                 $date,
209                 $startTime,
210                 $endTime,
211                 $state,
212                 $requestedItems,
213                 $itemAvailability,
214                 $isPropertyReservation,
215                 $propertyItemMaintenanceSchedule
216             ) {
217                 if ($isPropertyReservation) {
218                     $itemIds = $requestedItems->pluck('facility_item_id')->map(fn ($id) => (int) $id)->all();
219                     $items = FacilityItem::query()
220                         ->whereIn('id', $itemIds)
221                         ->orderBy('id')
222                         ->lockForUpdate()
223                         ->get()
224                         ->keyBy('id');
225
226                     if ($items->count() !== count($itemIds) || $requestedItems->isEmpty()) {
227                         return response()->json(['reply' => 'I could not verify the selected items. Please start the property reservation again.']);
228                     }
229
230                     foreach ($requestedItems as $selection) {
231                         $item = $items->get((int) $selection['facility_item_id']);
232                         if (!$item) {
233                             return response()->json(['reply' => 'A selected property item is no longer available. Please start the property reservation again.']);
234                         }
235                         $maintenanceBlock = $propertyItemMaintenanceSchedule->conflictingBlock(
236                             $item->id,
237                             $date,
238                             $startTime,
239                             $endTime
240                         );
241                         if ($maintenanceBlock) {
242                             return response()->json([
243                                 'reply' => $propertyItemMaintenanceSchedule->validationMessage($maintenanceBlock),
244                             ]);
245                         }
246                     }
247
248                     $availableQuantities = $itemAvailability->availableQuantities($items, $date, $startTime, $endTime);
249                     foreach ($requestedItems as $selection) {
250                         $item = $items->get((int) $selection['facility_item_id']);
251                         $available = $availableQuantities[$item->id] ?? 0;
252                         if ((int) $selection['quantity'] > $available) {
253                             return response()->json([
254                                 'reply' => "{$item->name} has only {$available} available for that date and time. Please choose a smaller quantity or another time.",
255                             ]);
256                         }
257                     }
258                 }
259
260                 $reservation = Reservation::create([
261                     'user_id' => Auth::id(),
262                     'facility_id' => $facility?->id,
263                     'reservation_date' => $date,
264                     'start_time' => $startTime,
265                     'end_time' => $endTime,
266                     'purpose' => $state['purpose'],
267                     'status' => 'pending',
268                 ]);
269
270                 if ($isPropertyReservation) {
271                     $reservation->items()->attach($requestedItems->mapWithKeys(fn ($selection) => [
272                         (int) $selection['facility_item_id'] => ['quantity' => (int) $selection['quantity']],
273                     ])->all());
274                 }
275
276                 return $reservation;
277             });
278
```

```
            if ($reservation instanceof JsonResponse) {
                return $reservation;
            }

            $admins = User::where('is_admin', true)
                ->orWhere('is_super_admin', true)
                ->get();
            $reservation->load(['facility', 'user', 'items']);
            Notification::send($admins, new ReservationPendingApproval($reservation));
            $reservationNotifications->notifyBorrower($reservation);
            session()->forget('chatbot_reservation');
            session([
                'chatbot_last_action' => 'reservation_submitted',
                'chatbot_last_reservation_type' => ($state['reservation_type'] ?? 'facility'),
            ]);

            $requestType = ($state['reservation_type'] ?? 'facility') === 'property' ? 'property reservation' : 'reservation';
            $itemSummary = $this->reservationItemSummary($reservation);
            $reservationDescription = ($state['reservation_type'] ?? 'facility') === 'property'
                ? "property items{$itemSummary}"
                : "{$facility->name}{$itemSummary}";

            $timeSummary = ($state['reservation_type'] ?? 'facility') === 'property'
                ? ' with use at ' . Carbon::createFromFormat('H:i', $startTime)->format('g:i A') . ' and return at ' . Carbon::createFromFormat('H:i', $endTime)->format('g:i A')
                : ' from ' . Carbon::createFromFormat('H:i', $startTime)->format('g:i A') . ' to ' . Carbon::createFromFormat('H:i', $endTime)->format('g:i A');

            return response()->json([
                'reply' => "Your {$requestType} request for {$reservationDescription} on " . Carbon::parse($date)->format('F j, Y') . "{$timeSummary} was submitted as a pending request for administrator approval.",
            ]);
        }

        if (session('chatbot_last_action') === 'reservation_submitted') {
            if ($this->isConfirmation($normalized)) {
                $requestType = session('chatbot_last_reservation_type') === 'property' ? 'property' : 'facility';
                return response()->json(['reply' => "Would you like to make another {$requestType} reservation? Tell me the room, date, and time" . ($requestType === 'property' ? ', and the items and quantities' : '') . ', or reply “no” when you are finished.']);
            }

            if ($this->isNegative($normalized) || preg_match('/^(done|finished|that is all|that\'s all)\b/i', $normalized)) {
                $requestType = session('chatbot_last_reservation_type') === 'property' ? 'property reservation' : 'reservation';
                session()->forget(['chatbot_last_action', 'chatbot_last_reservation_type']);
                return response()->json(['reply' => "Okay. Your {$requestType} request is waiting for administrator approval."]);
            }

            session()->forget(['chatbot_last_action', 'chatbot_last_reservation_type']);
        }

        if (preg_match('/\b(new|start over|reset|forget that|different)\b.*\b(reservation|booking|request|room|time)\b/i', $normalized)) {
            session()->forget('chatbot_reservation');
        }

        if (preg_match('/^\s*(hello|hi|hey)\s*[!.?]*\s*$/', $normalized)) {
            return response()->json(['reply' => 'Hi! I can check facility and property-item availability, help reserve them separately, and check your reservation status. Try: “Reserve 2 projectors tomorrow from 9 AM to 11 AM.”']);
        }

        if (preg_match('/\b(help|what can you do|what do you do|website|site|account|login|profile)\b/', $normalized)) {
            return response()->json(['reply' => 'I can check facility and property-item availability, request facility or property reservations, show your reservation status including requested items, and help cancel bookings. For property, include the item name and quantity, date, time to use, and time to return.']);
        }

        if ($this->isReservationStatusQuestion($normalized)) {
            $reservationsQuery = Reservation::with(['facility', 'items'])
                ->where('user_id', Auth::id());
            if ($this->isUpcomingReservationQuestion($normalized)) {
                $reservations = $reservationsQuery
                    ->upcoming()
                    ->limit(5)
                    ->get();

                if ($reservations->isEmpty()) {
                    return response()->json(['reply' => 'You do not have any upcoming pending or approved reservations.']);
```

```
                }

                $nextReservation = $reservations->first();
                $status = $nextReservation->status === 'approved' ? 'accepted' : 'pending approval';
                $time = $nextReservation->items->isNotEmpty()
                    ? 'use at ' . $nextReservation->start_time->format('g:i A') . ', return at ' . $nextReservation->end_time->format('g:i A')
                    : $nextReservation->start_time->format('g:i A') . ' to ' . $nextReservation->end_time->format('g:i A');

                return response()->json([
                    'reply' => 'Your next reservation is '
                        . $this->reservationFacilityLabel($nextReservation)
                        . $this->reservationItemSummary($nextReservation)
                        . ' on ' . $nextReservation->reservation_date->format('F j, Y')
                        . ', ' . $time . " ({$status}).",
                ]);
            }

            $reservations = $reservationsQuery
                ->whereIn('status', ['pending', 'approved', 'rejected'])
                ->orderByDesc('reservation_date')
                ->orderByDesc('start_time')
                ->limit(10)
                ->get();

            if ($reservations->isEmpty()) {
                return response()->json(['reply' => 'You do not have any pending, accepted, or declined reservations.']);
            }

            $summary = $reservations->map(function (Reservation $reservation) {
                $status = match ($reservation->status) {
                    'approved' => 'accepted',
                    'rejected' => 'declined',
                    default => $reservation->status,
                };
                $facility = $this->reservationFacilityLabel($reservation);
                $date = $reservation->reservation_date->format('F j, Y');
                $time = $reservation->items->isNotEmpty()
                    ? 'use at ' . $reservation->start_time->format('g:i A') . ', return at ' . $reservation->end_time->format('g:i A')
                    : $reservation->start_time->format('g:i A') . ' to ' . $reservation->end_time->format('g:i A');
                $items = $this->reservationItemSummary($reservation);
                $purpose = $reservation->purpose !== '' ? " (purpose: {$reservation->purpose})" : '';

                return "{$facility}{$items}{$purpose} on {$date}, {$time} - {$status}";
            })->join("\n");

            return response()->json(['reply' => "Here are your most recent reservations:\n{$summary}"]);
        }

        if (preg_match('/\b(cancel|remove|delete|get rid of)\b.*\b(reservation|booking|request|it|that)\b/i', $normalized)
            || preg_match('/\b(cancel|remove|delete)\b.*\b(my|the)\b/i', $normalized)) {
            $reservations = Reservation::with(['facility', 'items'])
                ->where('user_id', Auth::id())
                ->whereIn('status', ['pending', 'approved'])
                ->whereDate('reservation_date', '>=', now()->toDateString())
                ->orderBy('reservation_date')
                ->orderBy('start_time')
                ->get()
                ->filter(fn (Reservation $reservation) => $reservation->canBeCancelled());

            $requestedReservation = $reservations->first(function (Reservation $reservation) use ($normalized) {
                $facilityName = preg_replace('/[^a-z0-9]/', '', strtolower($reservation->facility?->name ?? ''));
                $question = preg_replace('/[^a-z0-9]/', '', $normalized);
                $itemNames = $reservation->items->map(fn (FacilityItem $item) => preg_replace('/[^a-z0-9]/', '', strtolower($item->name)));

                return ($facilityName !== '' && str_contains($question, $facilityName))
                    || $itemNames->contains(fn (string $itemName) => $itemName !== '' && str_contains($question, $itemName));
            });

            if ($requestedReservation) {
                $facilityName = $this->reservationFacilityLabel($requestedReservation);
                $itemSummary = $this->reservationItemSummary($requestedReservation);
                session(['chatbot_cancellation' => [
                    'reservation_id' => $requestedReservation->id,
                    'awaiting_confirmation' => true,
```

```
                ]]);

                return response()->json(['reply' => "Do you want to cancel your reservation for {$facilityName}{$itemSummary} on " .
$requestedReservation->reservation_date->format('F j, Y') . '? Reply yes to confirm cancellation.']);
            }

            if ($reservations->isEmpty()) {
                return response()->json(['reply' => 'You do not have any pending or accepted reservations that can be cancelled.']);
            }

            $options = $reservations->map(function (Reservation $reservation) {
                return $this->reservationFacilityLabel($reservation) . $this->reservationItemSummary($reservation) . ' on ' . $reservation->reservation_date->format('F j,
Y');
            })->join('; ');

            session(['chatbot_cancellation' => ['awaiting_selection' => true]]);
            return response()->json(['reply' => "Which reservation do you want to cancel? Your cancellable reservations are: {$options}. Reply with the facility or item name,
 then I will ask you to confirm."]);
        }

        $facilities ??= Facility::active()->with('items')->orderBy('name')->get();
        $propertyItems = FacilityItem::query()->orderBy('name')->get();
        $propertyIntent = (bool) preg_match('/\b(property|properties|item|items|equipment|equipments|inventory|asset|assets|supply|supplies)\b/i', $normalized)
            || ($state['reservation_type'] ?? null) === 'property'
            || $this->parsePropertyItems($propertyItems, $normalized, [])->isNotEmpty();

        $isItemListRequest = $propertyIntent
            && (bool) preg_match('/\b(show|list|what|which|available|have)\b/', $normalized)
            && !(bool) preg_match('/\b(reserve|book|booking)\b/', $normalized)
            && !$this->hasDateOrTimeRequest($normalized);

        if ($isItemListRequest) {
            $requestedFacility = $facilities->first(function (Facility $facility) use ($normalized) {
                return $this->messageMatchesFacility($facility, $normalized);
            });

            if ($requestedFacility) {
                return response()->json(['reply' => 'Property items are managed independently of rooms. Browse the property inventory to see available items.']);
            }

            $inventory = $propertyItems->map(fn (FacilityItem $item) => "{$item->name} ({$item->quantity} in stock)");

            if ($inventory->isEmpty()) {
                return response()->json(['reply' => 'There are no property items listed for reservation yet.']);
            }

            return response()->json([
                'reply' => 'Available property items include: ' . $inventory->join(', ') . '. Name the item, quantity, date, and time to request a property reservation.',
            ]);
        }

        $isFacilityListRequest = (
            preg_match('/\b(show|list|what|which|name)\b.*\b(rooms?|facilit(?:y|ies)|spaces?)\b/', $normalized)
            && preg_match('/\b(available|active|have|offer|there)\b/', $normalized)
        ) || preg_match('/\b(show|list|what) (rooms?|facilit(?:y|ies)|spaces?)\b/', $normalized);

        if ($isFacilityListRequest && !$this->hasDateOrTimeRequest($normalized)) {
            if ($facilities->isEmpty()) {
                return response()->json(['reply' => 'There are no active facilities available right now.']);
            }

            $rooms = $facilities->map(fn (Facility $facility) => $facility->name)->join(', ');
            return response()->json(['reply' => "Our active facilities are: {$rooms}. Ask me for a date and time to check availability."]);
        }

        $date = $this->parseDate($normalized);
        [$startTime, $endTime] = $this->parseTimeRange($normalized);
        $requestedFacility = $facilities->first(fn (Facility $facility) => $this->messageMatchesFacility($facility, $normalized));

            $askingForAlternatives = (bool) preg_match('/\b(other|another|different|all)\s+(rooms?|facilit(?:y|ies))\b/i', $normalized);
            if ($askingForAlternatives) {
                $requestedFacility = null;
                $state['facility_id'] = null;
                $state['awaiting_confirmation'] = false;
```

```
            }

        $date ??= !empty($state['date']) ? Carbon::parse($state['date']) : null;
        if ($date && ! $reservationDateWindow->allows($date->toDateString())) {
            session()->forget('chatbot_reservation');

            return response()->json([
                'reply' => 'Reservations can be made up to '.$reservationDateWindow->maximumDate().'. Please choose an earlier date.',
            ]);
        }
        $startTime ??= $state['start_time'] ?? null;
        $endTime ??= $state['end_time'] ?? null;
        $requestedFacility ??= $facilities->firstWhere('id', $state['facility_id'] ?? null);
        $previousFacilityId = $state['facility_id'] ?? null;
        $previousItems = $previousFacilityId && $requestedFacility && (int) $previousFacilityId !== $requestedFacility->id
            ? []
            : ($state['items'] ?? []);
        $requestedItems = $propertyIntent
            ? $this->parsePropertyItems($propertyItems, $normalized, $previousItems)
            : ($requestedFacility
                ? $this->parseRequestedItems($requestedFacility, $normalized, $previousItems)
                : collect());
        $isReservationRequest = (bool) preg_match('/\b(reserve|book|booking)\b/i', $normalized)
            || (bool) ($state['reservation_intent'] ?? false);

        $state = [
            'date' => $date?->toDateString(),
            'start_time' => $startTime,
            'end_time' => $endTime,
            'facility_id' => $propertyIntent ? null : $requestedFacility?->id,
            'reservation_type' => $propertyIntent ? 'property' : 'facility',
            'items' => $requestedItems->values()->all(),
            'purpose' => $state['purpose'] ?? null,
            'reservation_intent' => $isReservationRequest,
        ];
        session(['chatbot_reservation' => $state]);

        if ($isReservationRequest && !$requestedFacility && !$propertyIntent) {
            return response()->json(['reply' => 'Which room or facility would you like to reserve?']);
        }

        if ($isReservationRequest
            && ($requestedFacility || ($propertyIntent && $requestedItems->isNotEmpty()))
            && empty($state['purpose'])) {
            $state['awaiting_purpose'] = true;
            session(['chatbot_reservation' => $state]);

            return response()->json(['reply' => $propertyIntent
                ? 'What is the purpose of your property reservation?'
                : "What is the purpose of your reservation for {$requestedFacility->name}?"]);
        }

        if (!$date) {
            return response()->json(['reply' => $propertyIntent
                ? 'What date should I check for the property reservation? You can say “today”, “tomorrow”, or use YYYY-MM-DD.'
                : 'What date should I check? You can say “today”, “tomorrow”, or use YYYY-MM-DD.']);
        }

        if (!$startTime || !$endTime) {
            if ($this->hasTimeRangeInput($normalized)) {
                return response()->json(['reply' => 'Please provide a valid time range where the end time is later than the start time, such as “9 to 10 AM” or “8 AM to 7 PM”.']);
            }

            session(['chatbot_reservation' => $state]);
            return response()->json(['reply' => $propertyIntent
            ? 'What time will you use the items, and what time will you return them? For example: “use from 9 AM and return at 11 AM”.'
                : 'What time range should I check? For example: “from 9 AM to 11 AM”.']);
        }

        $matches = $requestedFacility ? collect([$requestedFacility]) : $facilities;
        $unavailableFacilityIds = $propertyIntent ? collect() : Reservation::query()
            ->whereIn('facility_id', $matches->pluck('id')->all())
            ->whereDate('reservation_date', $date->toDateString())
```

```
            ->where('status', 'approved')
            ->where('start_time', '<', $endTime)
            ->where('end_time', '>', $startTime)
            ->distinct()
            ->pluck('facility_id');
        $available = $matches->reject(fn (Facility $facility) => $unavailableFacilityIds->contains($facility->id));
        $dateLabel = $date->format('F j, Y');
        $timeLabel = Carbon::createFromFormat('H:i', $startTime)->format('g:i A') . ' to ' . Carbon::createFromFormat('H:i', $endTime)->format('g:i A');

        if (! $propertyIntent && $available->isEmpty()) {
            $subject = $requestedFacility ? $requestedFacility->name : 'the requested rooms';
            session()->forget('chatbot_reservation');
            return response()->json(['reply' => "No availability was found for {$subject} on {$dateLabel} from {$timeLabel}. Try another time or date."]);
        }

        $names = $available->map(fn (Facility $facility) => $facility->name)->join(', ');

        if ($propertyIntent) {
            if ($requestedItems->isEmpty()) {
                $availableQuantities = $itemAvailability->availableQuantities(
                    $propertyItems,
                    $date->toDateString(),
                    $startTime,
                    $endTime
                );
                $itemAvailabilityList = $propertyItems
                    ->map(function (FacilityItem $item) use (
                        $availableQuantities,
                        $propertyItemMaintenanceSchedule,
                        $date,
                        $startTime,
                        $endTime
                    ): string {
                        $maintenanceBlock = $propertyItemMaintenanceSchedule->conflictingBlock(
                            $item->id,
                            $date->toDateString(),
                            $startTime,
                            $endTime
                        );
                        $available = $maintenanceBlock ? 0 : ($availableQuantities[$item->id] ?? 0);

                        return "{$item->name} ({$available} available)";
                    })
                    ->join(', ');
                $inventory = $itemAvailabilityList !== '' ? $itemAvailabilityList : 'no property items are listed';
                session(['chatbot_reservation' => array_merge($state, [
                    'date' => $date->toDateString(),
                    'start_time' => $startTime,
                    'end_time' => $endTime,
                    'facility_id' => null,
                    'reservation_type' => 'property',
                ])]);

                return response()->json([
                    'reply' => "Property item availability on {$dateLabel} from {$timeLabel}: {$inventory}. Tell me the item name and quantity to continue.",
                    'action_url' => route('reserve', ['tab' => 'property']),
                    'action_label' => 'Open property reservations',
                ]);
            }

            $itemAvailabilitySummary = [];
            $selectedPropertyItems = $requestedItems->map(
                fn (array $selection) => $propertyItems->firstWhere('id', (int) $selection['facility_item_id'])
            )->filter();
            $availableQuantities = $itemAvailability->availableQuantities(
                $selectedPropertyItems,
                $date->toDateString(),
                $startTime,
                $endTime
            );
            foreach ($requestedItems as $selection) {
                $item = $propertyItems->firstWhere('id', (int) $selection['facility_item_id']);
                if (!$item) {
                    continue;
```

```
                }

                $maintenanceBlock = $propertyItemMaintenanceSchedule->conflictingBlock(
                    $item->id,
                    $date->toDateString(),
                    $startTime,
                    $endTime
                );
                $availableQuantity = $maintenanceBlock ? 0 : ($availableQuantities[$item->id] ?? 0);
                $itemAvailabilitySummary[] = "{$item->name}: {$availableQuantity} available";

                if ((int) $selection['quantity'] > $availableQuantity) {
                    session(['chatbot_reservation' => $state]);
                    return response()->json([
                        'reply' => "{$item->name} has only {$availableQuantity} available on {$dateLabel} from {$timeLabel}. Tell me a smaller quantity or another date and time.",
                    ]);
                }
            }

            if ($isReservationRequest) {
                if (empty($state['purpose'])) {
                    $state['awaiting_purpose'] = true;
                    $state['confirm_after_purpose'] = true;
                    session(['chatbot_reservation' => $state]);

                    return response()->json(['reply' => 'What is the purpose of this property reservation?']);
                }

                session([
                    'chatbot_reservation' => array_merge($state, [
                        'facility_id' => null,
                        'awaiting_confirmation' => true,
                    ]),
                ]);

                $requestedItemSummary = $this->formatPropertyItems($propertyItems, $requestedItems);
                return response()->json([
                    'reply' => "{$requestedItemSummary} are available on {$dateLabel}. The use time is " . Carbon::createFromFormat('H:i', $startTime)->format('g:i A') . ' and the return time is ' . Carbon::createFromFormat('H:i', $endTime)->format('g:i A') . '. Should I submit this property reservation request to the administrators for approval? Reply yes to confirm.',
                ]);
            }

            session([
                'chatbot_reservation' => array_merge($state, [
                    'facility_id' => null,
                    'awaiting_confirmation' => true,
                ]),
            ]);

            return response()->json([
                'reply' => "Available on {$dateLabel}: " . implode(', ', $itemAvailabilitySummary) . '. Use time: ' . Carbon::createFromFormat('H:i', $startTime)->format('g:i A') . '; return time: ' . Carbon::createFromFormat('H:i', $endTime)->format('g:i A') . '. Would you like me to submit a property reservation request for those quantities?',
            ]);
        }

        if ($isReservationRequest && $requestedFacility) {
            session([
                'chatbot_reservation' => array_merge($state, [
                    'facility_id' => $requestedFacility->id,
                    'awaiting_confirmation' => true,
                ]),
            ]);

            return response()->json([
                'reply' => "{$requestedFacility->name} is available on {$dateLabel} from {$timeLabel}. Should I submit this reservation request to the administrators for approval? Reply yes to confirm reservation.",
            ]);
        }

        if ($requestedFacility) {
            session([
                'chatbot_reservation' => array_merge($state, [
```

```
                    'facility_id' => $requestedFacility->id,
                    'awaiting_confirmation' => true,
                ]),
            ]);

            return response()->json([
                'reply' => "Available on {$dateLabel} from {$timeLabel}: {$requestedFacility->name}. Would you like me to submit this reservation request to the administrators for approval? Reply yes to confirm reservation.",
            ]);
        }

        session([
            'chatbot_reservation' => array_merge($state, [
                'date' => $date->toDateString(),
                'start_time' => $startTime,
                'end_time' => $endTime,
                'facility_id' => null,
                'awaiting_confirmation' => false,
            ]),
        ]);
        return response()->json([
            'reply' => ($isReservationRequest ? 'The requested time is available. ' : '') . "Available on {$dateLabel} from {$timeLabel}: {$names}. You can name a room if you want me to submit a reservation request.",
            'action_url' => route('reserve'),
            'action_label' => 'Open reservation form',
        ]);
    }

    private function messageMatchesFacility(Facility $facility, string $message): bool
    {
        $question = preg_replace('/[^a-z0-9]/', '', strtolower($message));
        $facilityName = preg_replace('/[^a-z0-9]/', '', strtolower($facility->name));
        $roomNumber = preg_replace('/[^a-z0-9]/', '', strtolower((string) $facility->room_number));

        return ($facilityName !== '' && str_contains($question, $facilityName))
            || ($roomNumber !== '' && str_contains($question, $roomNumber));
    }

    private function parseRequestedItems(Facility $facility, string $message, array $previousItems): \Illuminate\Support\Collection
    {
        return $this->parsePropertyItems($facility->items, $message, $previousItems);
    }

    private function parsePropertyItems(\Illuminate\Support\Collection $items, string $message, array $previousItems): \Illuminate\Support\Collection
    {
        $requestedItems = collect($previousItems)->keyBy(fn (array $item) => (int) $item['facility_item_id']);
        $normalizedMessage = strtolower($message);

        foreach ($items as $item) {
            $itemPattern = str_replace(' ', '\\s+', preg_quote(strtolower($item->name), '/'));
            $itemPattern = '(?<![a-z0-9])' . $itemPattern . 's?(?![a-z0-9])';
            $quantity = null;

            if (preg_match('/\b(\d+)\s*(?:x\s*)?' . $itemPattern . '/i', $normalizedMessage, $matches)) {
                $quantity = (int) $matches[1];
            } elseif (preg_match('/' . $itemPattern . '\s*(?:x|qty|quantity)\s*(\d+)\b/i', $normalizedMessage, $matches)) {
                $quantity = (int) $matches[1];
            } elseif (preg_match('/\b(one|two|three|four|five|six|seven|eight|nine|ten)\s+' . $itemPattern . '/i', $normalizedMessage, $matches)) {
                $quantity = array_search(strtolower($matches[1]), [
                    1 => 'one', 2 => 'two', 3 => 'three', 4 => 'four', 5 => 'five',
                    6 => 'six', 7 => 'seven', 8 => 'eight', 9 => 'nine', 10 => 'ten',
                ], true);
            } elseif (preg_match('/' . $itemPattern . '/i', $normalizedMessage)) {
                $quantity = 1;
            }

            if ($quantity !== null && $quantity > 0) {
                $requestedItems->put($item->id, [
                    'facility_item_id' => $item->id,
                    'quantity' => $quantity,
                ]);
            }
        }

```

```
        return $requestedItems->values();
    }

    private function formatPropertyItems(\Illuminate\Support\Collection $items, \Illuminate\Support\Collection $requestedItems): string
    {
        return $requestedItems
            ->map(function (array $selection) use ($items) {
                $item = $items->firstWhere('id', (int) $selection['facility_item_id']);

                return $item ? "{$selection['quantity']} × {$item->name}" : null;
            })
            ->filter()
            ->join(', ');
    }

    private function reservationItemSummary(Reservation $reservation): string
    {
        if ($reservation->items->isEmpty()) {
            return '';
        }

        $items = $reservation->items
            ->map(fn (FacilityItem $item) => "{$item->name} × {$item->pivot->quantity}")
            ->join(', ');

        return " (items: {$items})";
    }

    private function reservationFacilityLabel(Reservation $reservation): string
    {
        return $reservation->facility?->name
            ?? ($reservation->items->isNotEmpty() ? 'Property items' : 'Unknown facility');
    }

    private function formatRequestedItems(Facility $facility, \Illuminate\Support\Collection $requestedItems): string
    {
        return $requestedItems
            ->map(function (array $selection) use ($facility) {
                $item = $facility->items->firstWhere('id', (int) $selection['facility_item_id']);

                return $item ? "{$selection['quantity']} × {$item->name}" : null;
            })
            ->filter()
            ->join(', ');
    }

    private function parseDate(string $message): ?Carbon
    {
        if (preg_match('/\b(?:in\s+)?(\d{1,3})\s+days?\b/', $message, $matches)) {
            $days = (int) $matches[1];
            if ($days <= 365) {
                return now()->addDays($days)->startOfDay();
            }
        }

        if (preg_match('/\b(?:(next|this|coming)\s+)?(monday|tuesday|wednesday|thursday|friday|saturday|sunday)\b/', $message, $matches)) {
            $today = now()->startOfDay();
            $weekday = Carbon::parse($matches[2])->dayOfWeek;
            $daysUntil = ($weekday - $today->dayOfWeek + 7) % 7;

            if (($matches[1] ?? '') === 'next' && $daysUntil === 0) {
                $daysUntil = 7;
            }

            return $today->addDays($daysUntil);
        }

        if (preg_match('/\b(tomorrow|next day)\b/', $message)) {
            return now()->addDay()->startOfDay();
        }

        if (preg_match('/\b(today|this day)\b/', $message)) {
            return now()->startOfDay();
        }
```

```
855
856         if (preg_match('/\b(20\d{2}-\d{2}-\d{2})\b/', $message, $matches)) {
857             try {
858                 return Carbon::createFromFormat('Y-m-d', $matches[1])->startOfDay();
859             } catch (\Throwable) {
860                 return null;
861             }
862         }
863
864         if (preg_match('/\b(January|February|March|April|May|June|July|August|September|October|November|December)\s+\d{1,2}(?:st|nd|rd|th)?(?:,)?(?:\s+20\d{2})?\b/i',
    $message, $matches)) {
865             try {
866                 $dateText = preg_replace('/(st|nd|rd|th)\b/i', '', $matches[0]);
867                 $date = Carbon::parse($dateText);
868                 if (!preg_match('/20\d{2}/', $dateText)) {
869                     $date->year(now()->year);
870                     if ($date->isBefore(now()->startOfDay())) {
871                         $date->addYear();
872                     }
873                 }
874
875                 return $date->startOfDay();
876             } catch (\Throwable) {
877                 return null;
878             }
879         }
880
881         return null;
882     }
883
884     private function parseTimeRange(string $message): array
885     {
886         $message = preg_replace('/\b(?:12\s*)?noon\b/i', '12 pm', $message);
887         preg_match('/\b(?:from|between)?\s*(\d{1,2})(?::(\d{2}))?\s*(am|pm)?\s*(?:to|until|through|and|-)\s*(\d{1,2})(?::(\d{2}))?\s*(am|pm)?\b/i', $message, $range);
888
889         if ($range) {
890             $startPeriod = isset($range[3]) && $range[3] !== '' ? strtolower($range[3]) : null;
891             $endPeriod = isset($range[6]) && $range[6] !== '' ? strtolower($range[6]) : $startPeriod;
892             $startPeriodWasMissing = $startPeriod === null;
893             $endPeriodWasMissing = !isset($range[6]) || $range[6] === '';
894
895             if (!$startPeriod && !$endPeriod) {
896                 if ((int) $range[1] <= 12 && (int) $range[4] <= 12) {
897                     $startPeriod = 'am';
898                     $endPeriod = 'am';
899                 }
900             } elseif (!$startPeriod) {
901                 $startPeriod = $endPeriod;
902             }
903
904             $times = [
905                 $this->formatTime($range[1], $range[2] ?? '', $startPeriod),
906                 $this->formatTime($range[4], $range[5] ?? '', $endPeriod),
907             ];
908
909             if ($times[0] !== null && $times[1] !== null && $times[0] >= $times[1]) {
910                 if ($startPeriodWasMissing && $endPeriod) {
911                     $startPeriod = $endPeriod === 'pm' ? 'am' : 'pm';
912                     $times[0] = $this->formatTime($range[1], $range[2] ?? '', $startPeriod);
913                 } elseif ($endPeriodWasMissing && $startPeriod === 'am') {
914                     $endPeriod = 'pm';
915                     $times[1] = $this->formatTime($range[4], $range[5] ?? '', $endPeriod);
916                 }
917             }
918
919             if ($times[0] === null || $times[1] === null || $times[0] >= $times[1]) {
920                 return [null, null];
921             }
922
923             return $times;
924         }
925
926         preg_match_all('/\b(\d{1,2})(?::(\d{2}))?\s*(am|pm)\b/i', $message, $matches, PREG_SET_ORDER);
927         $times = [];
```

```
928
929         foreach ($matches as $match) {
930             $time = $this->formatTime($match[1], $match[2], strtolower($match[3]));
931
932             if ($time !== null) {
933                 $times[] = $time;
934             }
935         }
936
937         if (count($times) < 2 || $times[0] >= $times[1]) {
938             return [null, null];
939         }
940
941         return [$times[0], $times[1]];
942     }
943
944     private function formatTime(string $hourText, string $minuteText, ?string $period): ?string
945     {
946         $hour = (int) $hourText;
947         $minute = $minuteText !== '' ? (int) $minuteText : 0;
948
949         if ($period === 'pm' && $hour < 12) {
950             $hour += 12;
951         } elseif ($period === 'am' && $hour === 12) {
952             $hour = 0;
953         }
954
955         if ($hour > 23 || $minute > 59 || ($period !== null && $hourText > 12)) {
956             return null;
957         }
958
959         return sprintf('%02d:%02d', $hour, $minute);
960     }
961
962     private function hasTimeRangeInput(string $message): bool
963     {
964         return (bool) preg_match('/\b\d{1,2}(?::\d{2})?\s*(?:am|pm)?\s*(?:to|until|through|-)\s*\d{1,2}(?::\d{2})?\s*(?:am|pm)?\b/i', $message);
965     }
966
967     private function hasDateOrTimeRequest(string $message): bool
968     {
969         return (bool)
    preg_match('/\b(today|tomorrow|20\d{2}-\d{2}-\d{2}|January|February|March|April|May|June|July|August|September|October|November|December|\d{1,2}(?::\d{2})?\s*(?:am|pm))\b/i',
     $message);
970     }
971
972     private function isReservationStatusQuestion(string $message): bool
973     {
974         if (preg_match('/\b(cancel|remove|delete)\b/i', $message)) {
975             return false;
976         }
977
978         return (bool) preg_match('/\b(reservations?|bookings?|schedule|appointments?)\b/i', $message)
979             && (bool) preg_match('/\b(my|mine|do i have|what|show|list|status|when|next|upcoming|pending|approved|accepted|declined|rejected)\b/i', $message);
980     }
981
982     private function isUpcomingReservationQuestion(string $message): bool
983     {
984         return (bool) preg_match('/\b(next|upcoming|soon|future)\b/i', $message)
985             && (bool) preg_match('/\b(reservations?|bookings?|schedule|appointments?)\b/i', $message);
986     }
987
988     private function isSupportedMessage(string $message): bool
989     {
990         return (bool) preg_match('/\b(hello|hi|hey|help|what can you do|website|site|account|login|profile|campus|room|rooms|facility|facilities|property|properties|item|item
    s|equipment|equipments|inventory|asset|assets|supply|supplies|available|availability|schedule|reservation|reservations|reserve|reserved|book|booking|bookings|cancel|cancelled
    |delete|remove|pending|approved|accepted|declined|rejected|status|today|tomorrow|am|pm|noon|yes|yeah|yep|yup|sure|okay|ok|correct|right|confirm|submit|go ahead|do
    it|no|nope|nah|not now|done|finished|\d{4}-\d{2}-\d{2})\b/i', $message)
991             || $this->hasTimeRangeInput($message);
992     }
993
994     private function normalizeMessage(string $message): string
995     {
996         $normalized = strtolower(trim($message));
```

```
        $normalized = str_replace(['’', '`'], "'", $normalized);
        $normalized = preg_replace("/\bwhat's\b/", 'what is', $normalized);
        $normalized = preg_replace("/\bthat's\b/", 'that is', $normalized);
        $normalized = preg_replace('/\b(reservation request|booking request)\b/', 'reservation', $normalized);
        $normalized = preg_replace('/\s+/', ' ', $normalized);

        return $normalized;
    }

    private function isConfirmation(string $message): bool
    {
        return (bool) preg_match('/^(yes|yeah|yep|yup|sure|okay|ok|correct|right|that is right|that works|please do|go ahead|do it|submit|confirm)(?:\b|[!.?])/i', trim($message));
    }

    private function isNegative(string $message): bool
    {
        return (bool) preg_match('/^(no|nope|nah|not now|do not|don\'t|cancel)(?:\b|[!.?])/i', trim($message));
    }

    private function moderateMessage(string $message): array
    {
        $terms = [
            'fuck', 'fucker', 'fucking', 'shit', 'bitch', 'asshole', 'bastard',
            'damn', 'crap', 'dick', 'piss', 'nigger', 'nigga', 'faggot',
            'fag', 'dyke', 'homo', 'tranny', 'chink', 'spic', 'kike', 'gook',
        ];
        $normalized = strtolower($message);
        $normalized = strtr($normalized, [
            '0' => 'o', '1' => 'i', '3' => 'e', '4' => 'a',
            '@' => 'a', '$' => 's', '5' => 's', '7' => 't',
        ]);
        $normalized = preg_replace('/[^a-z]/', '', $normalized);
        $normalized = preg_replace('/(.)\1+/', '$1', $normalized);
        $found = false;

        foreach ($terms as $term) {
            if (str_contains($normalized, $term)) {
                $found = true;
                break;
            }
        }

        if (!$found && preg_match('/(?:hate|kill|hurt|destroy|disgusting|dirty|stupid|gross)(?:all|the|those|these)?(?:gay|lesbian|lgbt|trans)\w*|(?:gay|lesbian|lgbt|trans)\w*(?:hate|kill|hurt|destroy|disgusting|dirty|stupid|gross)|(?:gay|lesbian|lgbt|trans)\w*(?:bad|wrong|inferior|unnatural|perverted|disgusting)/', $normalized)) {
            $found = true;
        }

        if ($found) {
            return ['[Censored]', true];
        }

        return [$message, false];
    }
}
```

app/Http/Controllers/Controller.php

```
<?php

namespace App\Http\Controllers;

abstract class Controller
{
    //
}
```

app/Http/Controllers/CredentialController.php

```
<?php

namespace App\Http\Controllers;

```

```php
use App\Models\SavedCredential;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class CredentialController extends Controller
{
    /**
     * Show all the user's saved passwords/credentials.
     */
    public function index()
    {
        $credentials = Auth::user()->savedCredentials()->latest()->get();
        return view('credentials.index', compact('credentials'));
    }

    /**
     * Show the form to create a new saved credential.
     */
    public function create()
    {
        return view('credentials.create');
    }

    /**
     * Save the new credential to the database.
     * Everything I save here gets encrypted automatically by the model.
     */
    public function store(Request $request)
    {
        $validated = $request->validate([
            'site_name' => 'required|string|max:255',
            'site_url' => 'nullable|string|max:500',
            'username' => 'required|string|max:255',
            'password' => 'required|string|max:500',
            'notes' => 'nullable|string|max:2000',
        ]);

        $validated['user_id'] = Auth::id();

        SavedCredential::create($validated);

        return redirect()->route('credentials.index')
            ->with('success', 'Credential saved successfully with AES encryption.');
    }

    /**
     * Show a single credential (only if the user owns it).
     */
    public function show(SavedCredential $credential)
    {
        // Make sure this credential belongs to the logged-in user
        if ($credential->user_id !== Auth::id()) {
            abort(403);
        }

        return view('credentials.show', compact('credential'));
    }

    /**
     * Show the form to edit a credential.
     */
    public function edit(SavedCredential $credential)
    {
        // Make sure this credential belongs to the logged-in user
        if ($credential->user_id !== Auth::id()) {
            abort(403);
        }

        return view('credentials.edit', compact('credential'));
    }

    /**
     * Save changes to an existing credential.
     */
```

```
    public function update(Request $request, SavedCredential $credential)
    {
        // Make sure this credential belongs to the logged-in user
        if ($credential->user_id !== Auth::id()) {
            abort(403);
        }

        $validated = $request->validate([
            'site_name' => 'required|string|max:255',
            'site_url' => 'nullable|string|max:500',
            'username' => 'required|string|max:255',
            'password' => 'required|string|max:500',
            'notes' => 'nullable|string|max:2000',
        ]);

        $credential->update($validated);

        return redirect()->route('credentials.index')
            ->with('success', 'Credential updated successfully.');
    }

    /**
     * Delete a credential from the database.
     */
    public function destroy(SavedCredential $credential)
    {
        // Make sure this credential belongs to the logged-in user
        if ($credential->user_id !== Auth::id()) {
            abort(403);
        }

        $credential->delete();

        return redirect()->route('credentials.index')
            ->with('success', 'Credential deleted successfully.');
    }

    /**
     * Send back the decrypted password via AJAX (for copy-to-clipboard).
     */
    public function getPassword(SavedCredential $credential)
    {
        // Ensure user owns this credential
        if ($credential->user_id !== Auth::id()) {
            return response()->json(['error' => 'Unauthorized'], 403);
        }

        return response()->json([
            'password' => $credential->password
        ]);
    }
}
```

app/Http/Controllers/DebugController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Crypt;
use Illuminate\Support\Str;

class DebugController extends Controller
{
    /**
     * Show the decrypt utility with member selection.
     */
    public function showDecryptForm()
    {
        if (! auth()->check() || ! auth()->user()->isAdmin()) {
            return redirect()->route('home');
        }

```

```
        if (! session('decrypt_otp_verified')) {
            return view('debug.decrypt-otp');
        }

        // Fetch all users for the dropdown
        $users = \App\Models\User::select('id', 'name', 'email')->orderBy('name')->get();

        return view('debug.decrypt', ['users' => $users]);
    }

    /**
     * Issue an OTP code for decrypt utility access verification.
     */
    public function issueDecryptOtp(Request $request)
    {
        if (! auth()->check() || ! auth()->user()->isAdmin()) {
            return redirect()->route('home');
        }

        $user = auth()->user();
        $otp = rand(100000, 999999);

        // Store OTP in session with 10-second expiry
        session([
            'decrypt_otp' => $otp,
            'decrypt_otp_expires_at' => now()->addSeconds(10),
            'decrypt_otp_attempts' => 0,
        ]);

        // Log OTP to otp.log file for reference
        $logMessage = "[" . now()->format('Y-m-d H:i:s') . "] Admin '{$user->email}' requested decrypt OTP: {$otp} (expires in 10 seconds)\n";
        file_put_contents(storage_path('logs/otp.log'), $logMessage, FILE_APPEND);

        return view('debug.decrypt-otp-verify', ['otp_issued' => true, 'timeLeft' => 10]);
    }

    /**
     * Resend OTP code for decrypt utility access.
     */
    public function resendDecryptOtp(Request $request)
    {
        if (! auth()->check() || ! auth()->user()->isAdmin()) {
            return redirect()->route('home');
        }

        $user = auth()->user();
        $otp = rand(100000, 999999);

        // Store new OTP in session with 10-second expiry
        session([
            'decrypt_otp' => $otp,
            'decrypt_otp_expires_at' => now()->addSeconds(10),
            'decrypt_otp_attempts' => 0,
        ]);

        // Log resent OTP to otp.log file
        $logMessage = "[" . now()->format('Y-m-d H:i:s') . "] Admin '{$user->email}' resent decrypt OTP: {$otp} (expires in 10 seconds)\n";
        file_put_contents(storage_path('logs/otp.log'), $logMessage, FILE_APPEND);

        return view('debug.decrypt-otp-verify', ['otp_issued' => true, 'timeLeft' => 10]);
    }

    /**
     * Verify the OTP code for decrypt utility access.
     */
    public function verifyDecryptOtp(Request $request)
    {
        if (! auth()->check() || ! auth()->user()->isAdmin()) {
            return redirect()->route('home');
        }

        $request->validate(['otp' => 'required|numeric|digits:6']);

        $storedOtp = session('decrypt_otp');
```

```
        $expiresAt = session('decrypt_otp_expires_at');
        $attempts = session('decrypt_otp_attempts', 0);

        if (! $storedOtp || now()->isAfter($expiresAt)) {
            return back()->with('error', 'OTP has expired. Request a new one.');
        }

        if ($attempts >= 5) {
            session()->forget(['decrypt_otp', 'decrypt_otp_expires_at', 'decrypt_otp_attempts']);

            return back()->with('error', 'Too many attempts. Please request a new OTP.');
        }

        if ((int) $request->input('otp') !== (int) $storedOtp) {
            session(['decrypt_otp_attempts' => $attempts + 1]);

            return back()->with('error', 'Invalid OTP. Please try again.');
        }

        // OTP verified! Mark session as verified and clear OTP
        session(['decrypt_otp_verified' => true]);
        session()->forget(['decrypt_otp', 'decrypt_otp_expires_at', 'decrypt_otp_attempts']);

        return redirect()->route('debug.decrypt.form');
    }

    /**
     * Get user details for auto-filling fields (AJAX endpoint).
     */
    public function getUserDetails(Request $request)
    {
        if (! auth()->check() || ! auth()->user()->isAdmin()) {
            abort(403);
        }

        $request->validate(['user_id' => 'required|integer|exists:users,id']);

        $user = \App\Models\User::findOrFail($request->input('user_id'));

        return response()->json([
            'name' => $user->name,
            'email' => $user->email,
            'phone' => $user->phone ?? 'N/A',
            'email_encrypted' => $user->getRawOriginal('email'),
            'phone_encrypted' => $user->getRawOriginal('phone'),
        ]);
    }

    /**
     * Decrypt and display user information.
     */
    public function decrypt(Request $request)
    {
        if (! auth()->check() || ! auth()->user()->isAdmin()) {
            return redirect()->route('home');
        }

        if (! session('decrypt_otp_verified')) {
            return redirect()->route('debug.decrypt.form');
        }

        $request->validate(['user_id' => 'required|integer|exists:users,id']);

        $user = \App\Models\User::findOrFail($request->input('user_id'));

        // Fetch all users for dropdown
        $users = \App\Models\User::select('id', 'name', 'email')->orderBy('name')->get();

        // Prepare decrypted user data
        $decryptedData = [
            'user' => $user->name,
            'email' => $user->email,
            'phone' => $user->phone ?? 'N/A',
        ];
```

```

        return view('debug.decrypt', [
            'users' => $users,
            'selectedUserId' => $user->id,
            'decrypted' => $decryptedData,
        ]);
    }
}
```

app/Http/Controllers/FacilityController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Facility;

class FacilityController extends Controller
{
    /**
     * Send a list of all active facilities to the frontend as JSON.
     * The form uses this to populate the room dropdown.
     */
    public function list()
    {
        $facilities = Facility::active()
            ->orderBy('name')
            ->get(['id', 'name']);

        return response()->json($facilities);
    }

    public function propertyItems(\App\Services\PropertyItemMaintenanceSchedule $maintenanceSchedule)
    {
        $items = \App\Models\FacilityItem::query()
            ->when($maintenanceSchedule->isAvailable(), fn ($query) => $query->whereNotIn(
                'id',
                $maintenanceSchedule->activeItemIds()
            ))
            ->orderBy('name')
            ->get(['id', 'name', 'quantity']);

        return response()->json($items);
    }
}
```

app/Http/Controllers/NotificationController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Reservation;
use App\Models\User;
use App\Notifications\PropertyReturnDue;
use App\Notifications\PropertyReturnNotConfirmed;
use Carbon\Carbon;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Notification;

class NotificationController extends Controller
{
    public function unreadSummary()
    {
        $summary = auth()->user()->unreadNotifications()
            ->selectRaw('COUNT(*) as count, MAX(created_at) as latest_at')
            ->first();

        return response()->json([
            'count' => (int) $summary->count,
            'latest_at' => $summary->latest_at,
```

```
        ]);
    }

    public function index()
    {
        // Get the logged-in user's notifications, showing the newest first, with 15 per page
        $notifications = auth()->user()->notifications()->latest()->paginate(15);

        return view('notifications.index', compact('notifications'));
    }

    public function respondToPropertyReturn(Request $request, Reservation $reservation)
    {
        $validated = $request->validate([
            'notification_id' => ['required', 'uuid'],
            'response' => ['required', 'in:returned,not_returned'],
        ]);

        [$response, $reservation, $alreadyResponded, $adminNotified] = DB::transaction(function () use ($validated, $reservation) {
            $user = auth()->user();
            $notification = $user->notifications()
                ->whereKey($validated['notification_id'])
                ->lockForUpdate()
                ->firstOrFail();

            abort_unless(
                $notification->type === PropertyReturnDue::class
                    && (int) ($notification->data['reservation_id'] ?? 0) === $reservation->id,
                403
            );

            if (! empty($notification->data['response'])) {
                return [$notification->data['response'], $reservation, true, false];
            }

            $lockedReservation = Reservation::query()
                ->whereKey($reservation->id)
                ->where('user_id', $user->id)
                ->lockForUpdate()
                ->firstOrFail();

            abort_unless(
                $lockedReservation->status === 'approved'
                    && $lockedReservation->items()->exists(),
                422,
                'This property reservation is no longer awaiting a return confirmation.'
            );

            $returnAt = Carbon::parse(
                $lockedReservation->reservation_date->toDateString()
                    .' '.$lockedReservation->end_time->format('H:i:s')
            );
            abort_if(now()->lt($returnAt), 422, 'The scheduled return time has not passed yet.');

            if ($validated['response'] === 'returned') {
                $lockedReservation->update(['status' => 'completed']);
            }

            $notification->forceFill([
                'data' => array_merge($notification->data, [
                    'response' => $validated['response'],
                    'responded_at' => now()->toIso8601String(),
                    'status' => $validated['response'] === 'returned' ? 'completed' : 'approved',
                ]),
                'read_at' => now(),
            ])->save();

            $lockedReservation->load(['user', 'facility', 'items']);
            $adminNotified = false;

            if ($validated['response'] === 'not_returned') {
                $admins = User::query()
                    ->where(fn ($query) => $query->where('is_admin', true)->orWhere('is_super_admin', true))
                    ->get();
```

```

                if ($admins->isEmpty()) {
                    logger()->warning('Borrower reported property items not returned, but no admins were available to notify.', [
                        'reservation_id' => $lockedReservation->id,
                    ]);
                } else {
                    Notification::send($admins, new PropertyReturnNotConfirmed($lockedReservation));
                    $adminNotified = true;
                }
            }

            return [$validated['response'], $lockedReservation, false, $adminNotified];
        });

        if ($alreadyResponded) {
            return back()->with('info', 'You already responded to this return reminder.');
        }

        if ($response === 'not_returned') {
            if (! $adminNotified) {
                return back()->with('error', 'Your response was recorded, but no administrators are available to notify. Please contact the campus administrator directly.');
            }

            return back()->with('success', 'Thanks for letting us know. An administrator has been notified.');
        }

        if ($response === 'returned') {
            return back()->with('success', 'Thanks for confirming. The reservation has been marked as completed.');
        }

        return back()->with('info', 'You already responded to this return reminder.');
    }

    public function markAllRead(Request $request)
    {
        // Mark all the user's unread notifications as read
        auth()->user()->unreadNotifications->markAsRead();

        return back();
    }

    public function destroySelected(Request $request)
    {
        // Delete selected notifications
        // If the notification is about a pending reservation, I also delete that reservation
        // This keeps everything in sync
        $request->validate([
            'selected_notifications' => 'array',
            'selected_notifications.*' => 'string',
        ]);

        $selectedIds = $request->input('selected_notifications', []);
        $notifications = auth()->user()->notifications()->whereIn('id', $selectedIds)->get();

        foreach ($notifications as $notification) {
            if ($notification->type === \App\Notifications\ReservationPendingApproval::class
                && ! empty($notification->data['reservation_id'])) {
                $reservation = Reservation::where('id', $notification->data['reservation_id'])
                    ->where('user_id', auth()->id())
                    ->first();

                if ($reservation && in_array($reservation->status, ['pending', 'approved', 'rejected'])) {
                    $reservation->delete();
                }
            }

            $notification->delete();
        }

        return back()->with('success', 'Selected notifications were removed successfully. Pending reservation requests were also cleared.');
    }
}
```

app/Http/Controllers/PropertyItemMaintenanceController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\AuditLog;
use App\Models\FacilityItem;
use App\Models\PropertyItemMaintenanceBlock;
use App\Models\Reservation;
use App\Services\PropertyItemMaintenanceSchedule;
use App\Services\ReservationNotificationDispatcher;
use Illuminate\Http\Request;
use Illuminate\Support\Collection;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\DB;
use Illuminate\Validation\Rule;

class PropertyItemMaintenanceController extends Controller
{
    public function store(
        Request $request,
        PropertyItemMaintenanceSchedule $maintenanceSchedule,
        ReservationNotificationDispatcher $reservationNotifications
    ) {
        $this->requireAdmin();

        if (! $maintenanceSchedule->isAvailable()) {
            return back()->with('error', 'Property maintenance scheduling is unavailable until its database migration has been applied.');
        }

        $validated = $this->validateBlock($request);
        $isAllDay = $request->boolean('is_all_day');
        $startTime = $isAllDay ? null : $validated['start_time'];
        $endTime = $isAllDay ? null : $validated['end_time'];
        $item = FacilityItem::findOrFail($validated['facility_item_id']);

        [$block, $overlappingReservations] = DB::transaction(function () use (
            $validated,
            $item,
            $isAllDay,
            $startTime,
            $endTime
        ): array {
            $lockedItem = FacilityItem::query()->lockForUpdate()->findOrFail($item->id);
            $block = PropertyItemMaintenanceBlock::create([
                'facility_item_id' => $lockedItem->id,
                'maintenance_date' => $validated['maintenance_date'],
                'start_time' => $startTime,
                'end_time' => $endTime,
                'is_all_day' => $isAllDay,
                'reason' => $validated['reason'],
            ]);

            $overlappingReservations = $this->rejectOverlappingReservations(
                $lockedItem,
                $validated['maintenance_date'],
                $isAllDay,
                $startTime,
                $endTime,
                $validated['reason']
            );

            AuditLog::create([
                'user_id' => Auth::id(),
                'reservation_id' => null,
                'action' => 'property_item_maintenance_created',
                'details' => "Scheduled maintenance for {$item->name} on {$validated['maintenance_date']}: {$validated['reason']}.",
            ]);

            return [$block, $overlappingReservations];
        });

        $this->notifyRejectedReservations($overlappingReservations, $reservationNotifications);
```

```

        return redirect()
            ->route('admin.reservation-settings')
            ->with('status', $overlappingReservations->isEmpty()
                ? 'Property item maintenance scheduled successfully.'
                : 'Property item maintenance scheduled. Overlapping reservations were automatically rejected and their owners notified.');
    }

    public function update(
        Request $request,
        PropertyItemMaintenanceBlock $block,
        PropertyItemMaintenanceSchedule $maintenanceSchedule,
        ReservationNotificationDispatcher $reservationNotifications
    ) {
        $this->requireAdmin();

        if (! $maintenanceSchedule->isAvailable()) {
            return back()->with('error', 'Property maintenance scheduling is unavailable until its database migration has been applied.');
        }

        $validated = $this->validateBlock($request);
        $isAllDay = $request->boolean('is_all_day');
        $startTime = $isAllDay ? null : $validated['start_time'];
        $endTime = $isAllDay ? null : $validated['end_time'];
        $item = FacilityItem::findOrFail($validated['facility_item_id']);

        $overlappingReservations = DB::transaction(function () use (
            $block,
            $validated,
            $item,
            $isAllDay,
            $startTime,
            $endTime
        ): Collection {
            $lockedBlock = PropertyItemMaintenanceBlock::query()->lockForUpdate()->findOrFail($block->id);
            $lockedItem = FacilityItem::query()->lockForUpdate()->findOrFail($item->id);
            $lockedBlock->update([
                'facility_item_id' => $lockedItem->id,
                'maintenance_date' => $validated['maintenance_date'],
                'start_time' => $startTime,
                'end_time' => $endTime,
                'is_all_day' => $isAllDay,
                'reason' => $validated['reason'],
            ]);

            $overlappingReservations = $this->rejectOverlappingReservations(
                $lockedItem,
                $validated['maintenance_date'],
                $isAllDay,
                $startTime,
                $endTime,
                $validated['reason']
            );

            AuditLog::create([
                'user_id' => Auth::id(),
                'reservation_id' => null,
                'action' => 'property_item_maintenance_updated',
                'details' => "Updated maintenance for {$item->name} on {$validated['maintenance_date']}: {$validated['reason']}.",
            ]);

            return $overlappingReservations;
        });

        $this->notifyRejectedReservations($overlappingReservations, $reservationNotifications);

        return redirect()
            ->route('admin.reservation-settings')
            ->with('status', $overlappingReservations->isEmpty()
                ? 'Property item maintenance updated successfully.'
                : 'Property item maintenance updated. Overlapping reservations were automatically rejected and their owners notified.');
    }

    public function destroy(Request $request, PropertyItemMaintenanceBlock $block)
```

```
    {
        $this->requireAdmin();
        $request->validate([
            'current_password' => ['required', 'current_password'],
        ]);

        DB::transaction(function () use ($block): void {
            $lockedBlock = PropertyItemMaintenanceBlock::query()->with('item')->lockForUpdate()->findOrFail($block->id);
            $details = "Removed property item maintenance for {$lockedBlock->item->name} on {$lockedBlock->maintenance_date->format('Y-m-d')}.";
            $lockedBlock->delete();

            AuditLog::create([
                'user_id' => Auth::id(),
                'reservation_id' => null,
                'action' => 'property_item_maintenance_removed',
                'details' => $details,
            ]);
        });

        return redirect()->route('admin.reservation-settings')->with('status', 'Property item maintenance removed.');
    }

    private function validateBlock(Request $request): array
    {
        return $request->validate([
            'facility_item_id' => ['required', 'integer', Rule::exists('facility_items', 'id')],
            'maintenance_date' => ['required', 'date', 'after_or_equal:today'],
            'is_all_day' => ['required', 'boolean'],
            'start_time' => ['nullable', 'required_unless:is_all_day,1', 'date_format:H:i'],
            'end_time' => ['nullable', 'required_unless:is_all_day,1', 'date_format:H:i', 'after:start_time'],
            'reason' => ['required', 'string', 'max:255'],
            'current_password' => ['required', 'current_password'],
        ]);
    }

    private function rejectOverlappingReservations(
        FacilityItem $item,
        string $date,
        bool $isAllDay,
        ?string $startTime,
        ?string $endTime,
        string $reason
    ): Collection {
        $overlappingReservations = Reservation::query()
            ->whereHas('items', fn ($query) => $query->where('facility_items.id', $item->id))
            ->whereDate('reservation_date', $date)
            ->whereIn('status', ['pending', 'approved'])
            ->when(! $isAllDay, fn ($query) => $query
                ->where('start_time', '<', $endTime)
                ->where('end_time', '>', $startTime))
            ->lockForUpdate()
            ->get();

        foreach ($overlappingReservations as $reservation) {
            $details = "Automatically rejected due to property item maintenance for {$item->name}: {$reason}.";
            $reservation->update([
                'status' => 'rejected',
                'rejected_at' => now(),
                'admin_remarks' => $details,
            ]);

            AuditLog::create([
                'user_id' => Auth::id(),
                'reservation_id' => $reservation->id,
                'action' => 'rejected',
                'details' => $details,
            ]);
        }

        return $overlappingReservations;
    }

    private function notifyRejectedReservations(
        Collection $reservations,
```

```
        ReservationNotificationDispatcher $reservationNotifications
    ): void {
        foreach ($reservations as $reservation) {
            $reservationNotifications->notifyBorrower($reservation);
        }
    }

    private function requireAdmin(): void
    {
        abort_unless(Auth::check() && Auth::user()->isAdmin(), 403, 'Admin access only.');
    }
}
```

app/Http/Middleware/RequireAdminActionPassword.php

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Symfony\Component\HttpFoundation\Response;

class RequireAdminActionPassword
{
    public function handle(Request $request, Closure $next): Response
    {
        $user = $request->user();

        if (! $user || ! $user->isAdmin()) {
            abort(403, 'Admin access only.');
        }

        if (str_starts_with($request->path(), 'admin/users/') && ! $user->isSuperAdmin()) {
            abort(403, 'Head admin access only.');
        }

        if ($request->isMethodSafe()) {
            return $next($request);
        }

        $password = $request->input('current_password');
        if (! is_string($password) || ! Hash::check($password, $user->password)) {
            $message = 'Enter your current password to confirm this admin action.';

            if ($request->expectsJson()) {
                return response()->json([
                    'message' => $message,
                    'errors' => ['current_password' => [$message]],
                ], 422);
            }

            return back()
                ->withErrors(['current_password' => $message])
                ->withInput($request->except('current_password'));
        }

        return $next($request);
    }
}
```

app/Http/Middleware/SecurityHeaders.php

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class SecurityHeaders
```

```
{
    public function handle(Request $request, Closure $next): Response
    {
        $response = $next($request);

        // Remove headers that leak info about server software being used
        // Harder for attackers to find vulnerabilities specific to this setup

        // Check if working with a Response object before adding headers
        if ($response instanceof \Illuminate\Http\Response || $response instanceof Response) {
            // Prevent site from being framed in other websites
            $response->headers->set('X-Frame-Options', 'SAMEORIGIN');

            // Prevent browsers from guessing what type of content is being sent
            $response->headers->set('X-Content-Type-Options', 'nosniff');

            // Set security policies for what resources can be accessed
            $response->headers->set('Referrer-Policy', 'strict-origin-when-cross-origin');
            $response->headers->set('Permissions-Policy', 'geolocation=(), microphone=(), camera=()');
            $response->headers->set('X-XSS-Protection', '1; mode=block');

            // Tell browsers not to cache sensitive pages, prevents showing old versions
            $response->headers->set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0');
            $response->headers->set('Pragma', 'no-cache');
            $response->headers->set('Expires', '0');

            // Set up strong Content Security Policy rules
            // Controls what resources (scripts, images, etc) can load on this site
            $frameAncestors = $request->is('calendar/manage') && $request->boolean('embedded')
                ? "frame-ancestors 'self'"
                : "frame-ancestors 'none'";

            $cspDirectives = [
                "default-src 'self'",
                "script-src 'self' 'unsafe-inline' https://cdn.tailwindcss.com https://cdn.jsdelivr.net",
                "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com https://cdn.jsdelivr.net",
                "img-src 'self' data:",
                "font-src 'self' https://fonts.gstatic.com https://cdn.jsdelivr.net",
                "connect-src 'self' https://fonts.googleapis.com https://cdn.jsdelivr.net",
                "form-action 'self'",
                "base-uri 'self'",
                "object-src 'none'",
                $frameAncestors,
                'block-all-mixed-content',
            ];

            if (config('app.debug') || app()->environment('local')) {
                $devOrigins = [
                    'http://127.0.0.1:5173',
                    'http://localhost:5173',
                ];

                $httpDevOrigins = implode(' ', $devOrigins);
                $wsDevOrigins = implode(' ', array_map(fn ($origin) => preg_replace('#^http#', 'ws', $origin), $devOrigins));

                $cspDirectives[1] .= ' ' . $httpDevOrigins;
                $cspDirectives[2] .= ' ' . $httpDevOrigins;
                $cspDirectives[5] .= ' ' . $httpDevOrigins . ' ' . $wsDevOrigins;
            }

            $response->headers->set('Content-Security-Policy', implode('; ', $cspDirectives) . ';');

            // HSTS (Strict-Transport-Security) always set
            $response->headers->set('Strict-Transport-Security', 'max-age=31536000; includeSubDomains; preload');
        }

        return $response;
    }
}
```

app/Mail/Transport/GmailApiTransport.php

```
<?php

```

```php
namespace App\Mail\Transport;

use Illuminate\Http\Client\ConnectionException;
use Illuminate\Support\Facades\Http;
use Symfony\Component\Mailer\Exception\TransportException;
use Symfony\Component\Mailer\SentMessage;
use Symfony\Component\Mailer\Transport\AbstractTransport;

class GmailApiTransport extends AbstractTransport
{
    private ?string $accessToken = null;

    private int $accessTokenExpiresAt = 0;

    public function __construct(
        private readonly string $clientId,
        private readonly string $clientSecret,
        private readonly string $refreshToken,
    ) {
        parent::__construct();
    }

    protected function doSend(SentMessage $message): void
    {
        $rawMessage = rtrim(strtr(base64_encode($message->getOriginalMessage()->toString()), '+/', '-_'), '=');

        try {
            $response = Http::withToken($this->getAccessToken())
                ->acceptJson()
                ->timeout(20)
                ->post('https://gmail.googleapis.com/gmail/v1/users/me/messages/send', [
                    'raw' => $rawMessage,
                ]);
        } catch (ConnectionException $exception) {
            throw new TransportException('Could not connect to the Gmail API: '.$exception->getMessage(), 0, $exception);
        }

        if (! $response->successful()) {
            $detail = $response->json('error.message') ?? $response->json('error_description') ?? 'No error message provided.';

            throw new TransportException(sprintf(
                'The Gmail API rejected the email with HTTP %d: %s',
                $response->status(),
                $detail
            ));
        }

        $messageId = $response->json('id');
        if (! is_string($messageId) || $messageId === '') {
            throw new TransportException('The Gmail API response did not include an email ID.');
        }
    }

    private function getAccessToken(): string
    {
        if ($this->clientId === '' || $this->clientSecret === '' || $this->refreshToken === '') {
            throw new TransportException('Gmail API OAuth credentials are not fully configured.');
        }

        if ($this->accessToken !== null && time() < $this->accessTokenExpiresAt - 60) {
            return $this->accessToken;
        }

        try {
            $response = Http::asForm()
                ->acceptJson()
                ->timeout(20)
                ->post('https://oauth2.googleapis.com/token', [
                    'client_id' => $this->clientId,
                    'client_secret' => $this->clientSecret,
                    'refresh_token' => $this->refreshToken,
                    'grant_type' => 'refresh_token',
                ]);
        } catch (ConnectionException $exception) {
```

```
            throw new TransportException('Could not obtain a Gmail API access token: '.$exception->getMessage(), 0, $exception);
        }

        $accessToken = $response->json('access_token');
        if (! $response->successful() || ! is_string($accessToken) || $accessToken === '') {
            $detail = $response->json('error_description') ?? $response->json('error') ?? 'No error message provided.';

            throw new TransportException(sprintf(
                'Google OAuth could not refresh the Gmail API access token (HTTP %d): %s',
                $response->status(),
                is_string($detail) ? $detail : 'No error message provided.'
            ));
        }

        $this->accessToken = $accessToken;
        $this->accessTokenExpiresAt = time() + (int) $response->json('expires_in', 3600);

        return $this->accessToken;
    }

    public function __toString(): string
    {
        return 'gmail-api';
    }
}
```

app/Mail/Transport/ResendApiTransport.php

```
<?php

namespace App\Mail\Transport;

use Illuminate\Http\Client\ConnectionException;
use Illuminate\Support\Facades\Http;
use Symfony\Component\Mailer\Envelope;
use Symfony\Component\Mailer\Exception\TransportException;
use Symfony\Component\Mailer\SentMessage;
use Symfony\Component\Mailer\Transport\AbstractTransport;
use Symfony\Component\Mime\Address;
use Symfony\Component\Mime\Email;
use Symfony\Component\Mime\MessageConverter;

class ResendApiTransport extends AbstractTransport
{
    public function __construct(private readonly string $apiKey)
    {
        parent::__construct();
    }

    protected function doSend(SentMessage $message): void
    {
        if ($this->apiKey === '') {
            throw new TransportException('The Resend API key is not configured.');
        }

        $email = MessageConverter::toEmail($message->getOriginalMessage());
        $envelope = $message->getEnvelope();
        $headers = [];

        foreach ($email->getHeaders()->all() as $header) {
            if (! in_array(strtolower($header->getName()), ['from', 'to', 'cc', 'bcc', 'reply-to', 'sender', 'subject', 'content-type'], true)) {
                $headers[$header->getName()] = $header->getBodyAsString();
            }
        }

        $payload = [
            'from' => $envelope->getSender()->toString(),
            'to' => $this->stringifyAddresses($this->getRecipients($email, $envelope)),
            'subject' => $email->getSubject(),
            'html' => $email->getHtmlBody(),
            'text' => $email->getTextBody(),
        ];

        foreach ([
```

```
            'cc' => $email->getCc(),
            'bcc' => $email->getBcc(),
            'reply_to' => $email->getReplyTo(),
        ] as $key => $addresses) {
            if ($addresses !== []) {
                $payload[$key] = $this->stringifyAddresses($addresses);
            }
        }

        if ($headers !== []) {
            $payload['headers'] = $headers;
        }

        if ($email->getAttachments() !== []) {
            $payload['attachments'] = array_map(function ($attachment): array {
                $attachmentHeaders = $attachment->getPreparedHeaders();
                $contentType = $attachmentHeaders->get('Content-Type')->getBody();
                $content = $contentType === 'text/calendar'
                    ? $attachment->getBody()
                    : str_replace("\r\n", '', $attachment->bodyToString());

                return [
                    'content_type' => $contentType,
                    'content' => $content,
                    'filename' => $attachmentHeaders->getHeaderParameter('Content-Disposition', 'filename'),
                ];
            }, $email->getAttachments());
        }

        try {
            $response = Http::withToken($this->apiKey)
                ->acceptJson()
                ->timeout(20)
                ->post('https://api.resend.com/emails', $payload);
        } catch (ConnectionException $exception) {
            throw new TransportException('Could not connect to the Resend API: '.$exception->getMessage(), 0, $exception);
        }

        if (! $response->successful()) {
            throw new TransportException(sprintf(
                'The Resend API rejected the email with HTTP %d: %s',
                $response->status(),
                $response->json('message') ?? 'No error message provided.'
            ));
        }

        $messageId = $response->json('id');
        if (! is_string($messageId) || $messageId === '') {
            throw new TransportException('The Resend API response did not include an email ID.');
        }

        $email->getHeaders()->addTextHeader('X-Resend-Email-ID', $messageId);
    }

    protected function getRecipients(Email $email, Envelope $envelope): array
    {
        return array_filter($envelope->getRecipients(), function (Address $address) use ($email): bool {
            return ! in_array($address, array_merge($email->getCc(), $email->getBcc()), true);
        });
    }

    public function __toString(): string
    {
        return 'resend';
    }
}
```

app/Models/AuditLog.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class AuditLog extends Model
{
    // Fields I can assign when logging an admin action
    protected $fillable = [
        'user_id',
        'reservation_id',
        'action',
        'details',
    ];

    // Link to the admin who performed the action
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    // Link to the reservation that was modified (if applicable)
    public function reservation(): BelongsTo
    {
        return $this->belongsTo(Reservation::class);
    }
}
```

app/Models/Facility.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Facility extends Model
{
    /**
     * Fields I can assign when creating/updating a facility.
     */
    protected $fillable = [
        'name',
        'building',
        'room_number',
        'type',
        'capacity',
        'description',
        'image',
        'is_active',
        'requires_approval',
    ];

    /**
     * Tell Laravel what type each field is (arrays, booleans, etc.)
     */
    protected $casts = [
        'is_active' => 'boolean',
        'requires_approval' => 'boolean',
    ];

    /**
     * Link to all reservations for this facility.
     */
    public function reservations(): HasMany
    {
        return $this->hasMany(Reservation::class);
    }

    public function items(): HasMany
    {
        return $this->hasMany(FacilityItem::class);
    }

    /**
     * Get a readable name for the facility type.
```

```
     */
    public function getTypeLabelAttribute(): string
    {
        return match($this->type) {
            'classroom' => 'Classroom',
            'laboratory' => 'Laboratory',
            'conference_room' => 'Conference Room',
            'auditorium' => 'Auditorium',
            'sports_hall' => 'Sports Hall',
            'study_room' => 'Study Room',
            'other' => 'Other',
            default => ucfirst($this->type),
        };
    }

    /**
     * Only return active facilities.
     */
    public function scopeActive($query)
    {
        return $query->where('is_active', true);
    }

    /**
     * Check if this room is free at a specific date and time.
     */
    public function isAvailable($date, $startTime, $endTime, $excludeReservationId = null): bool
    {
        // I only block approvals and calendar bookings against already approved reservations.
        // Pending reservations may overlap, but they are not shown on the calendar until approved.
        $query = $this->reservations()
            ->whereDate('reservation_date', $date)
            ->where('status', 'approved')
            ->where('start_time', '<', $endTime)
            ->where('end_time', '>', $startTime);

        if ($excludeReservationId) {
            $query->where('id', '!=', $excludeReservationId);
        }

        return $query->count() === 0;
    }
}
```

app/Models/FacilityItem.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class FacilityItem extends Model
{
    protected $fillable = [
        'facility_id',
        'name',
        'quantity',
    ];

    protected $casts = [
        'quantity' => 'integer',
    ];

    public function facility(): BelongsTo
    {
        return $this->belongsTo(Facility::class);
    }

    public function reservations(): BelongsToMany
    {
        return $this->belongsToMany(Reservation::class, 'reservation_items')
```

```
            ->withPivot('quantity')
            ->withTimestamps();
    }
}
```

app/Models/FacilityMaintenanceBlock.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class FacilityMaintenanceBlock extends Model
{
    protected $fillable = [
        'facility_id',
        'maintenance_date',
        'start_time',
        'end_time',
        'is_all_day',
        'reason',
    ];

    protected $casts = [
        'maintenance_date' => 'date',
        'is_all_day' => 'boolean',
    ];

    public function facility(): BelongsTo
    {
        return $this->belongsTo(Facility::class);
    }
}
```

app/Models/Otp.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Otp extends Model
{
    protected $table = 'otps';

    protected $fillable = [
        'user_id',
        'code',
        'expires_at',
        'attempts',
    ];

    protected $casts = [
        'expires_at' => 'datetime',
    ];

    protected $hidden = [
        'code',
    ];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    public function isExpired(): bool
    {
        return now()->isAfter($this->expires_at);
    }
```

```

    public function isValid(): bool
    {
        return !$this->isExpired() && $this->attempts < 3;
    }

    public function incrementAttempts(): void
    {
        $this->increment('attempts');
    }
}
```

app/Models/PropertyItemMaintenanceBlock.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class PropertyItemMaintenanceBlock extends Model
{
    protected $fillable = [
        'facility_item_id',
        'maintenance_date',
        'start_time',
        'end_time',
        'is_all_day',
        'reason',
    ];

    protected $casts = [
        'maintenance_date' => 'date',
        'is_all_day' => 'boolean',
    ];

    public function item(): BelongsTo
    {
        return $this->belongsTo(FacilityItem::class, 'facility_item_id');
    }
}
```

app/Models/Reservation.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class Reservation extends Model
{
    /**
     * Fields I can set when creating/updating a reservation.
     */
    protected $fillable = [
        'user_id',
        'facility_id',
        'reservation_date',
        'start_time',
        'end_time',
        'purpose',
        'notes',
        'status',
        'rejected_at',
        'admin_remarks',
        'attendees_count',
    ];

    /**
```

```
     * Here I tell Laravel what type each field is (dates, times, etc.)
     */
    protected $casts = [
        'reservation_date' => 'date',
        'rejected_at' => 'datetime',
        'start_time' => 'datetime:H:i',
        'end_time' => 'datetime:H:i',
    ];

    /**
     * Link to the user who made this reservation.
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    /**
     * Link to the facility being reserved.
     */
    public function facility(): BelongsTo
    {
        return $this->belongsTo(Facility::class);
    }

    public function items(): BelongsToMany
    {
        return $this->belongsToMany(FacilityItem::class, 'reservation_items')
            ->withPivot('quantity')
            ->withTimestamps();
    }

    /**
     * Get a nice label and color for the current status.
     */
    public function getStatusBadgeAttribute(): array
    {
        return match($this->status) {
            'pending' => ['label' => 'Pending', 'class' => 'bg-indigo-100 text-indigo-800'],
            'approved' => ['label' => 'Approved', 'class' => 'bg-green-100 text-green-800'],
            'rejected' => ['label' => 'Rejected', 'class' => 'bg-red-100 text-red-800'],
            'cancelled' => ['label' => 'Cancelled', 'class' => 'bg-gray-100 text-gray-800'],
            'completed' => ['label' => 'Completed', 'class' => 'bg-blue-100 text-blue-800'],
            default => ['label' => ucfirst($this->status), 'class' => 'bg-gray-100 text-gray-800'],
        };
    }

    /**
     * Scope to get upcoming reservations.
     */
    public function scopeUpcoming($query)
    {
        // Query only reservations that are still active and not in the past.
        return $query->where('reservation_date', '>=', now()->toDateString())
            ->whereIn('status', ['pending', 'approved'])
            ->orderBy('reservation_date')
            ->orderBy('start_time');
    }

    /**
     * Scope to get past/completed reservations.
     */
    public function scopeCompleted($query)
    {
        return $query->where(function ($q) {
            $q->where('reservation_date', '<', now()->toDateString())
                ->orWhere('status', 'completed');
        });
    }

    /**
     * Check if reservation can be cancelled.
     */
    public function canBeCancelled(): bool
```

```
    {
        // A reservation can only be cancelled if it is pending or approved
        // and its date is today or in the future.
        return in_array($this->status, ['pending', 'approved'])
            && $this->reservation_date >= now()->toDateString();
    }
}
```

app/Models/SavedCredential.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Support\Facades\Crypt;

class SavedCredential extends Model
{
    use HasFactory;

    /**
     * Fields I can assign when creating/updating a saved credential.
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'user_id',
        'site_name',
        'site_url',
        'username',
        'password',
        'notes',
    ];

    /**
     * Don't expose the password when converting to JSON.
     *
     * @var array<int, string>
     */
    protected $hidden = [
        'password',
    ];

    /**
     * Link to the user who saved this credential.
     */
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    /**
     * When saving the site name, I encrypt it for security.
     */
    public function setSiteNameAttribute($value)
    {
        $this->attributes['site_name'] = Crypt::encryptString($value);
    }

    /**
     * When getting the site name, I decrypt it so it's readable.
     */
    public function getSiteNameAttribute($value)
    {
        try {
            return Crypt::decryptString($value);
        } catch (\Exception $e) {
            return $value;
        }
    }

    /**
```

```
     * When saving the site URL, I encrypt it (but allow it to be empty).
     */
    public function setSiteUrlAttribute($value)
    {
        $this->attributes['site_url'] = $value ? Crypt::encryptString($value) : null;
    }

    /**
     * When getting the site URL, I decrypt it if it exists.
     */
    public function getSiteUrlAttribute($value)
    {
        if (!$value) return null;
        try {
            return Crypt::decryptString($value);
        } catch (\Exception $e) {
            return $value;
        }
    }

    /**
     * When saving the username, I encrypt it so it's secure.
     */
    public function setUsernameAttribute($value)
    {
        $this->attributes['username'] = Crypt::encryptString($value);
    }

    /**
     * When getting the username, I decrypt it.
     */
    public function getUsernameAttribute($value)
    {
        try {
            return Crypt::decryptString($value);
        } catch (\Exception $e) {
            return $value;
        }
    }

    /**
     * When saving the password, I encrypt it so it stays secure in the database.
     */
    public function setPasswordAttribute($value)
    {
        // Make sure the password is encrypted before saving
        $this->attributes['password'] = Crypt::encryptString($value);
    }

    /**
     * When getting the password, I decrypt it.
     */
    public function getPasswordAttribute($value)
    {
        try {
            return Crypt::decryptString($value);
        } catch (\Exception $e) {
            return $value;
        }
    }

    /**
     * When saving notes, I encrypt them (but allow them to be empty).
     */
    public function setNotesAttribute($value)
    {
        $this->attributes['notes'] = $value ? Crypt::encryptString($value) : null;
    }

    /**
     * When getting notes, I decrypt them if they exist.
     */
    public function getNotesAttribute($value)
    {
```

```
        if (!$value) return null;
        try {
            return Crypt::decryptString($value);
        } catch (\Exception $e) {
            return $value;
        }
    }
}
```

app/Models/User.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Illuminate\Support\Facades\Crypt;
use Illuminate\Database\Eloquent\Relations\HasOne;
use App\Services\UserNameSearchIndex;

class User extends Authenticatable
{
    use HasFactory, Notifiable;

    protected static function booted(): void
    {
        static::created(function (User $user): void {
            app(UserNameSearchIndex::class)->sync($user);
        });

        static::updated(function (User $user): void {
            if ($user->wasChanged('name')) {
                app(UserNameSearchIndex::class)->sync($user);
            }
        });
    }

    /**
     * The attributes that can be assigned directly when creating or updating a user.
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'name',
        'email',
        'password',
        'phone',
        'profile_picture',
        'two_fa_enabled',
        'two_fa_secret',
        'remember_device_token',
        'remember_device_expires_at',
        'is_admin',
        'is_super_admin',
        'two_fa_secret',
        'email_notifications',
    ];

    protected $attributes = [
        'email_notifications' => false,
    ];

    /**
     * These fields are hidden when returning user data as JSON.
     * Passwords, tokens, and sensitive admin flags should not be exposed.
     *
     * @var array<int, string>
     */
    protected $hidden = [
        'password',
        'remember_token',
        'email_hash',
```

```
        'remember_device_token',
        'is_admin',
        'is_super_admin',
    ];

    /**
     * Define field types for casts (dates, booleans, etc.).
     *
     * @var array<string, string>
     */
    protected $casts = [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
        'remember_device_expires_at' => 'datetime',
        'two_fa_enabled' => 'boolean',
        'is_admin' => 'boolean',
        'is_super_admin' => 'boolean',
        'email_notifications' => 'boolean',
    ];

    /**
     * Link to the user's saved credentials.
     */
    public function savedCredentials()
    {
        return $this->hasMany(SavedCredential::class);
    }

    /**
     * Link to the user's OTP records.
     */
    public function otps()
    {
        return $this->hasMany(Otp::class);
    }

    public function storedProfilePicture(): HasOne
    {
        return $this->hasOne(UserProfilePicture::class);
    }

    public function profilePictureUrl(): string
    {
        $picture = $this->storedProfilePicture;

        if ($picture) {
            return route('profile.picture.show', [
                'user' => $this->id,
                'v' => $picture->updated_at->timestamp,
            ]);
        }

        return $this->profile_picture
            ? asset('storage/'.$this->profile_picture)
            : asset('images/guest-avatar.svg');
    }

    /**
     * Encrypts the user name when saving to prevent direct database access.
     */
    public function setNameAttribute($value)
    {
        $this->attributes['name'] = Crypt::encryptString($value);
    }

    /**
     * Decrypts the user name when retrieving from database.
     */
    public function getNameAttribute($value)
    {
        try {
            return Crypt::decryptString($value);
        } catch (\Exception $e) {
            return $value; // Return unchanged if decryption fails
```

```
        }
    }

    /**
     * Encrypts and hashes email on save.
     * Hash is used for lookups while keeping the actual email encrypted.
     */
    public function setEmailAttribute($value)
    {
        // Save the encrypted email in the database
        // Also save a hashed version for lookups without decrypting
        $this->attributes['email'] = Crypt::encryptString($value);
        $this->attributes['email_hash'] = hash('sha256', strtolower($value));
    }

    /**
     * Decrypts email with error handling.
     */
    public function getEmailAttribute($value)
    {
        try {
            return Crypt::decryptString($value);
        } catch (\Exception $e) {
            return $value; // Return unchanged if decryption fails
        }
    }

    /**
     * Encrypts phone number for privacy. Allows null values.
     */
    public function setPhoneAttribute($value)
    {
        // Save encrypted if provided, otherwise null
        $this->attributes['phone'] = $value ? Crypt::encryptString($value) : null;
    }

    /**
     * Decrypts phone number if it exists.
     */
    public function getPhoneAttribute($value)
    {
        if (! $value) {
            return null;
        }

        try {
            return Crypt::decryptString($value);
        } catch (\Exception $e) {
            return $value;
        }
    }

    /**
     * Safely encrypt the 2FA secret when it is set.
     */
    public function setTwoFaSecretAttribute($value)
    {
        $this->attributes['two_fa_secret'] = $value === null || $value === '' ? null : Crypt::encryptString($value);
    }

    /**
     * Safely decrypt the 2FA secret. If it was encrypted with an older app key,
     * treat it as absent instead of crashing the login flow.
     */
    public function getTwoFaSecretAttribute($value)
    {
        if ($value === null || $value === '') {
            return null;
        }

        try {
            return Crypt::decryptString($value);
        } catch (\Throwable $e) {
            return null;
```

```
        }
    }

    /**
     * Check if user is an admin.
     */
    public function isAdmin(): bool
    {
        return $this->getAttribute('is_admin') === true;
    }

    public function isSuperAdmin(): bool
    {
        return $this->getAttribute('is_super_admin') === true;
    }
}
```

app/Models/UserProfilePicture.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class UserProfilePicture extends Model
{
    protected $fillable = [
        'user_id',
        'mime_type',
        'image_data',
    ];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

app/Notifications/PasswordChanged.php

```
<?php

namespace App\Notifications;

use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Notifications\Messages\MailMessage;
use Illuminate\Notifications\Notification;

class PasswordChanged extends Notification implements ShouldQueue
{
    use Queueable;

    public function via($notifiable): array
    {
        return ['mail'];
    }

    public function toMail($notifiable): MailMessage
    {
        return (new MailMessage)
            ->subject('Your Campus Reserve password was changed')
            ->greeting('Password changed')
            ->line('Your Campus Reserve password was changed successfully.')
            ->line('If you made this change, you can safely ignore this email.')
            ->line('If you did not change your password, your account may be compromised. Secure it immediately.')
            ->action('Secure my account', route('security.compromised'))
            ->salutation('Campus Reserve Security');
    }
}
```

app/Notifications/PasswordResetLink.php

```php
<?php

namespace App\Notifications;

use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Notifications\Messages\MailMessage;
use Illuminate\Notifications\Notification;

class PasswordResetLink extends Notification implements ShouldQueue
{
    use Queueable;

    public function __construct(public string $resetUrl) {}

    public function via(object $notifiable): array
    {
        return ['mail'];
    }

    public function toMail(object $notifiable): MailMessage
    {
        return (new MailMessage)
            ->subject('Reset your Campus Reserve password')
            ->greeting('Password reset requested')
            ->line('Use the button below to reset your Campus Reserve password. This link expires in 60 minutes.')
            ->action('Reset password', $this->resetUrl)
            ->line('If you did not request a reset, you can ignore this email.');
    }
}
```

app/Notifications/PropertyReturnDue.php

```php
<?php

namespace App\Notifications;

use App\Models\Reservation;
use Illuminate\Bus\Queueable;
use Illuminate\Notifications\Notification;

class PropertyReturnDue extends Notification
{
    use Queueable;

    public function __construct(public Reservation $reservation) {}

    public function via($notifiable): array
    {
        return ['database'];
    }

    public function toArray($notifiable): array
    {
        $items = $this->reservation->items
            ->map(fn ($item) => "{$item->name} × {$item->pivot->quantity}")
            ->join(', ');

        return [
            'reservation_id' => $this->reservation->id,
            'title' => 'Property items are due for return',
            'message' => "Your return time for {$items} has passed. Have you returned these items?",
            'action_url' => route('notifications.index'),
            'status' => 'approved',
            'response_required' => true,
        ];
    }
}
```

app/Notifications/PropertyReturnNotConfirmed.php

```php
<?php

namespace App\Notifications;

use App\Models\Reservation;
use Illuminate\Bus\Queueable;
use Illuminate\Notifications\Notification;

class PropertyReturnNotConfirmed extends Notification
{
    use Queueable;

    public function __construct(public Reservation $reservation) {}

    public function via($notifiable): array
    {
        return ['database'];
    }

    public function toArray($notifiable): array
    {
        $items = $this->reservation->items
            ->map(fn ($item) => "{$item->name} × {$item->pivot->quantity}")
            ->join(', ');

        return [
            'reservation_id' => $this->reservation->id,
            'title' => 'Property items not returned',
            'message' => "{$this->reservation->user->name} reported that {$items} have not been returned by the scheduled return time.",
            'action_url' => route('admin.dashboard'),
            'status' => $this->reservation->status,
        ];
    }
}
```

app/Notifications/ReservationPendingApproval.php

```php
<?php

namespace App\Notifications;

use Illuminate\Bus\Queueable;
use Illuminate\Notifications\Notification;
use App\Models\Reservation;

class ReservationPendingApproval extends Notification
{
    use Queueable;

    // I keep the reservation data so I can include details in the notification
    public Reservation $reservation;

    public function __construct(Reservation $reservation)
    {
        $this->reservation = $reservation;
    }

    public function via($notifiable)
    {
        return ['database'];
    }

    public function toArray($notifiable)
    {
        $reservation = $this->reservation;
        $userName = optional($reservation->user)->name ?? 'A user';
        $facilityName = $reservation->items->isNotEmpty()
            ? 'property items'
            : (optional($reservation->facility)->name ?? 'a facility');
        $date = optional($reservation->reservation_date)?->format('M d, Y') ?? 'a date';
        $start = optional($reservation->start_time)?->format('H:i') ?? '--:--';
        $end = optional($reservation->end_time)?->format('H:i') ?? '--:--';
```

```
        $items = $reservation->items
            ->map(fn ($item) => "{$item->name} × {$item->pivot->quantity}")
            ->join(', ');
        $itemDetails = $items !== '' ? " Items: {$items}." : '';
        $timeDetails = $items !== ''
            ? " Use time: {$start}; return time: {$end}."
            : " from {$start} to {$end}.";
        $purposeDetails = $reservation->purpose !== '' ? " Purpose: {$reservation->purpose}." : '';

        return [
            'reservation_id' => $reservation->id,
            'title' => 'Reservation change request',
            'message' => "{$userName} requested {$facilityName} on {$date}.{$timeDetails}{$purposeDetails}{$itemDetails}",
            'action_url' => route('admin.dashboard'),
            'status' => $reservation->status,
        ];
    }
}
```

app/Notifications/ReservationStatusChanged.php

```php
<?php

namespace App\Notifications;

use App\Models\Reservation;
use Illuminate\Bus\Queueable;
use Illuminate\Notifications\Messages\MailMessage;
use Illuminate\Notifications\Notification;

class ReservationStatusChanged extends Notification
{
    use Queueable;

    // Reservation object used to create the status change notification details.
    public Reservation $reservation;

    public function __construct(Reservation $reservation)
    {
        $this->reservation = $reservation;
    }

    public function via($notifiable)
    {
        return $notifiable->email_notifications ? ['database', 'mail'] : ['database'];
    }

    public function toMail($notifiable): MailMessage
    {
        $data = $this->toArray($notifiable);

        return (new MailMessage)
            ->subject($data['title'].' - Campus Reserve')
            ->greeting('Hello '.$notifiable->name.',')
            ->line($data['message'])
            ->action('View reservation', $data['action_url'])
            ->line('You can change email notification preferences in your profile settings.');
    }

    public function toArray($notifiable)
    {
        $reservation = $this->reservation;
        $facilityName = $reservation->items->isNotEmpty()
            ? 'property items'
            : (optional($reservation->facility)->name ?? 'a facility');
        $date = optional($reservation->reservation_date)?->format('M d, Y') ?? 'a date';
        $start = optional($reservation->start_time)?->format('H:i') ?? '--:--';
        $end = optional($reservation->end_time)?->format('H:i') ?? '--:--';
        $status = ucfirst($reservation->status);
        $items = $reservation->items
            ->map(fn ($item) => "{$item->name} × {$item->pivot->quantity}")
            ->join(', ');
        $itemDetails = $items !== '' ? " Items: {$items}." : '';
        $timeDetails = $items !== ''
```

```
            ? " Use time: {$start}; return time: {$end}."
            : " from {$start} to {$end}.";
        $statusMessage = match ($reservation->status) {
            'pending' => 'is pending and awaiting approval',
            'approved' => 'has been approved',
            'rejected' => 'has been denied',
            default => "has been {$reservation->status}",
        };

        return [
            'reservation_id' => $reservation->id,
            'title' => "Reservation {$status}",
            'message' => "Your reservation for {$facilityName} on {$date}{$timeDetails} {$statusMessage}.{$itemDetails}",
            'action_url' => route('reserve'),
            'status' => $reservation->status,
        ];
    }

    public function toDatabase($notifiable): array
    {
        return $this->toArray($notifiable);
    }
}
```

app/Providers/AppServiceProvider.php

```
<?php

namespace App\Providers;

use App\Mail\Transport\GmailApiTransport;
use App\Mail\Transport\ResendApiTransport;
use Illuminate\Support\Facades\Mail;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     */
    public function register(): void
    {
        //
    }

    /**
     * Bootstrap any application services.
     */
    public function boot(): void
    {
        Mail::extend('gmail-api', function (array $config): GmailApiTransport {
            return new GmailApiTransport(
                is_string($config['client_id'] ?? null) ? $config['client_id'] : '',
                is_string($config['client_secret'] ?? null) ? $config['client_secret'] : '',
                is_string($config['refresh_token'] ?? null) ? $config['refresh_token'] : '',
            );
        });

        Mail::extend('resend', function (array $config): ResendApiTransport {
            $key = $config['key'] ?? config('services.resend.key');

            return new ResendApiTransport(is_string($key) ? $key : '');
        });
    }
}
```

app/Services/FacilityItemAvailability.php

```
<?php

namespace App\Services;

use App\Models\FacilityItem;
```

```php
use Illuminate\Support\Carbon;
use Illuminate\Support\Facades\DB;

class FacilityItemAvailability
{
    public function availableQuantity(
        FacilityItem $item,
        string $date,
        string $startTime,
        string $endTime,
        ?int $excludeReservationId = null
    ): int {
        $reservations = $item->reservations()
            ->where('reservations.status', 'approved')
            ->whereDate('reservations.reservation_date', $date)
            ->where('reservations.start_time', '<', $endTime)
            ->where('reservations.end_time', '>', $startTime);
        if ($excludeReservationId !== null) {
            $reservations->where('reservations.id', '!=', $excludeReservationId);
        }
        $reservedQuantity = $reservations->sum('reservation_items.quantity');

        return max(0, $item->quantity - (int) $reservedQuantity);
    }

    public function availableQuantities(
        iterable $items,
        string $date,
        string $startTime,
        string $endTime,
        ?int $excludeReservationId = null
    ): array {
        $quantities = [];
        foreach ($items as $item) {
            $quantities[$item->id] = $item->quantity;
        }

        if ($quantities === []) {
            return [];
        }

        $nextDate = Carbon::parse($date)->addDay()->toDateString();
        $reservedQuantities = DB::table('reservation_items')
            ->join('reservations', 'reservations.id', '=', 'reservation_items.reservation_id')
            ->whereIn('reservation_items.facility_item_id', array_keys($quantities))
            ->where('reservations.status', 'approved')
            ->where('reservations.reservation_date', '>=', $date)
            ->where('reservations.reservation_date', '<', $nextDate)
            ->where('reservations.start_time', '<', $endTime)
            ->where('reservations.end_time', '>', $startTime)
            ->when($excludeReservationId !== null, fn ($query) => $query->where('reservations.id', '!=', $excludeReservationId))
            ->select('reservation_items.facility_item_id')
            ->selectRaw('SUM(reservation_items.quantity) AS reserved_quantity')
            ->groupBy('reservation_items.facility_item_id')
            ->pluck('reserved_quantity', 'facility_item_id');

        foreach ($quantities as $itemId => $quantity) {
            $quantities[$itemId] = max(0, $quantity - (int) ($reservedQuantities[$itemId] ?? 0));
        }

        return $quantities;
    }
}
```

app/Services/FacilityMaintenanceSchedule.php

```php
<?php

namespace App\Services;

use App\Models\FacilityMaintenanceBlock;
use Illuminate\Database\Eloquent\Collection;
use Illuminate\Support\Facades\Schema;

```

```php
class FacilityMaintenanceSchedule
{
    public function isAvailable(): bool
    {
        return Schema::hasTable('facility_maintenance_blocks');
    }

    public function conflictingBlock(int $facilityId, string $date, string $startTime, string $endTime): ?FacilityMaintenanceBlock
    {
        if (! $this->isAvailable()) {
            return null;
        }

        return FacilityMaintenanceBlock::query()
            ->where('facility_id', $facilityId)
            ->whereDate('maintenance_date', $date)
            ->where(function ($query) use ($startTime, $endTime) {
                $query->where('is_all_day', true)
                    ->orWhere(function ($query) use ($startTime, $endTime) {
                        $query->where('start_time', '<', $endTime)
                            ->where('end_time', '>', $startTime);
                    });
            })
            ->first();
    }

    public function forDateRange(
        string $start,
        string $end,
        ?string $facilityName = null,
        ?int $facilityId = null
    ): Collection
    {
        if (! $this->isAvailable()) {
            return new Collection();
        }

        return FacilityMaintenanceBlock::query()
            ->with('facility')
            ->where('maintenance_date', '>=', $start)
            ->where('maintenance_date', '<', $end)
            ->when($facilityName, fn ($query) => $query->whereHas(
                'facility',
                fn ($facilityQuery) => $facilityQuery->where('name', $facilityName)
            ))
            ->when($facilityId, fn ($query) => $query->where('facility_id', $facilityId))
            ->orderBy('maintenance_date')
            ->orderBy('start_time')
            ->get();
    }

    public function validationMessage(FacilityMaintenanceBlock $block): string
    {
        return "This facility is unavailable: {$block->reason}.";
    }
}
```

app/Services/PropertyItemMaintenanceSchedule.php

```php
<?php

namespace App\Services;

use App\Models\PropertyItemMaintenanceBlock;
use Illuminate\Support\Collection;
use Illuminate\Support\Facades\Schema;

class PropertyItemMaintenanceSchedule
{
    private ?bool $tableAvailable = null;

    public function isAvailable(): bool
    {
        if ($this->tableAvailable === null) {
```

```
            $this->tableAvailable = Schema::hasTable('property_item_maintenance_blocks');
        }

        return $this->tableAvailable;
    }

    public function conflictingBlock(
        int $itemId,
        string $date,
        string $startTime,
        string $endTime,
        ?int $excludeBlockId = null
    ): ?PropertyItemMaintenanceBlock {
        if (! $this->isAvailable()) {
            return null;
        }

        return PropertyItemMaintenanceBlock::query()
            ->where('facility_item_id', $itemId)
            ->whereDate('maintenance_date', $date)
            ->where(function ($query) use ($startTime, $endTime): void {
                $query->where('is_all_day', true)
                    ->orWhere(function ($query) use ($startTime, $endTime): void {
                        $query->where('start_time', '<', $endTime)
                            ->where('end_time', '>', $startTime);
                    });
            })
            ->when($excludeBlockId !== null, fn ($query) => $query->where('id', '!=', $excludeBlockId))
            ->first();
    }

    public function activeItemIds(): Collection
    {
        if (! $this->isAvailable()) {
            return collect();
        }

        $now = now();

        return PropertyItemMaintenanceBlock::query()
            ->whereDate('maintenance_date', $now->toDateString())
            ->where(function ($query) use ($now): void {
                $query->where('is_all_day', true)
                    ->orWhere(function ($query) use ($now): void {
                        $query->where('start_time', '<=', $now->format('H:i:s'))
                            ->where('end_time', '>', $now->format('H:i:s'));
                    });
            })
            ->distinct()
            ->pluck('facility_item_id');
    }

    public function validationMessage(PropertyItemMaintenanceBlock $block): string
    {
        return "This property item is unavailable: {$block->reason}.";
    }
}
```

app/Services/ReservationDateWindow.php

```
<?php

namespace App\Services;

use Illuminate\Support\Carbon;

class ReservationDateWindow
{
    public function maximumDate(): string
    {
        return now()->addYears(2)->toDateString();
    }

    public function rules(): array
```

```
    {
        return [
            'required',
            'date',
            'before_or_equal:'.$this->maximumDate(),
        ];
    }

    public function allows(string $date): bool
    {
        return Carbon::parse($date)->startOfDay()->lessThanOrEqualTo(
            Carbon::parse($this->maximumDate())->startOfDay()
        );
    }
}
```

app/Services/ReservationNotificationDispatcher.php

```
<?php

namespace App\Services;

use App\Models\Reservation;
use App\Notifications\ReservationStatusChanged;
use Illuminate\Support\Facades\Notification;
use Throwable;

class ReservationNotificationDispatcher
{
    public function notifyBorrower(Reservation $reservation): void
    {
        $reservation->loadMissing(['user', 'facility', 'items']);
        $user = $reservation->user;

        if (! $user) {
            return;
        }

        $notification = new ReservationStatusChanged($reservation);
        Notification::sendNow($user, $notification, ['database']);

        if (! $user->email_notifications) {
            return;
        }

        try {
            Notification::sendNow($user, $notification, ['mail']);
        } catch (Throwable $exception) {
            logger()->error('Reservation email notification could not be sent.', [
                'user_id' => $user->id,
                'reservation_id' => $reservation->id,
                'exception' => $exception->getMessage(),
            ]);
        }
    }
}
```

app/Services/ReservationTimeWindow.php

```
<?php

namespace App\Services;

use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Schema;

class ReservationTimeWindow
{
    private const DEFAULT_START = '00:00';
    private const DEFAULT_END = '23:59';

    public function isAvailable(): bool
    {
```

```
        return Schema::hasTable('reservation_settings');
    }

    public function current(): array
    {
        if (! $this->isAvailable()) {
            return [
                'start_time' => self::DEFAULT_START,
                'end_time' => self::DEFAULT_END,
            ];
        }

        $settings = DB::table('reservation_settings')->where('id', 1)->first();

        return [
            'start_time' => $settings->start_time ?? self::DEFAULT_START,
            'end_time' => $settings->end_time ?? self::DEFAULT_END,
        ];
    }

    public function allows(string $startTime, string $endTime): bool
    {
        $settings = $this->current();

        return $startTime >= $settings['start_time']
            && $endTime <= $settings['end_time'];
    }

    public function validationMessage(): string
    {
        $settings = $this->current();

        return 'Reservations must be scheduled between '
            .date('g:i A', strtotime($settings['start_time']))
            .' and '
            .date('g:i A', strtotime($settings['end_time']))
            .'.';
    }

    public function update(string $startTime, string $endTime): void
    {
        if (! $this->isAvailable()) {
            throw new \RuntimeException('Reservation settings are unavailable until the database migration is applied.');
        }

        $settings = DB::table('reservation_settings')->where('id', 1);
        $values = [
            'start_time' => $startTime,
            'end_time' => $endTime,
            'updated_at' => now(),
        ];

        if ($settings->exists()) {
            $settings->update($values);
            return;
        }

        DB::table('reservation_settings')->insert([
            'id' => 1,
            ...$values,
            'created_at' => now(),
        ]);
    }
}
```

app/Services/UserNameSearchIndex.php

```
<?php

namespace App\Services;

use App\Models\User;
use Illuminate\Support\Collection;
use Illuminate\Support\Facades\DB;
```

```php

class UserNameSearchIndex
{
    public function sync(User $user): void
    {
        $tokens = $this->tokens($user->name);
        DB::table('user_name_search_tokens')
            ->where('user_id', $user->id)
            ->delete();

        if ($tokens === []) {
            return;
        }

        $now = now();
        DB::table('user_name_search_tokens')->insertOrIgnore(
            array_map(fn (string $token): array => [
                'user_id' => $user->id,
                'token_hash' => $this->hashToken($token),
                'created_at' => $now,
                'updated_at' => $now,
            ], $tokens)
        );
    }

    public function matchingUserIds(string $search): Collection
    {
        $tokens = $this->tokens($search);
        if ($tokens === []) {
            return collect();
        }

        $tokenHashes = array_map(fn (string $token): string => $this->hashToken($token), $tokens);

        return DB::table('user_name_search_tokens')
            ->whereIn('token_hash', $tokenHashes)
            ->groupBy('user_id')
            ->havingRaw('COUNT(DISTINCT token_hash) = ?', [count($tokenHashes)])
            ->pluck('user_id');
    }

    public function matches(string $name, string $search): bool
    {
        return mb_strpos($this->normalize($name), $this->normalize($search)) !== false;
    }

    private function tokens(string $value): array
    {
        $normalized = $this->normalize($value);
        $length = mb_strlen($normalized);
        if ($length === 0) {
            return [];
        }

        $gramSize = min(3, $length);
        $tokens = [];
        for ($index = 0; $index <= $length - $gramSize; $index++) {
            $tokens[] = mb_substr($normalized, $index, $gramSize);
        }

        return array_values(array_unique($tokens));
    }

    private function normalize(string $value): string
    {
        $normalized = mb_strtolower(trim($value));

        return preg_replace('/\s+/u', ' ', $normalized) ?? $normalized;
    }

    private function hashToken(string $token): string
    {
        return hash_hmac('sha256', $token, (string) config('app.key'));
    }
```

```
}
```

bootstrap

bootstrap/app.php

```
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->trustProxies(at: '*');

        // Apply security headers to all HTTP responses globally
        $middleware->append(\App\Http\Middleware\SecurityHeaders::class);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        // Render exceptions securely - no debug details in production
        $exceptions->shouldRenderJsonWhen(function ($request) {
            return $request->expectsJson();
        });
    })->create();
```

bootstrap/providers.php

```
<?php

return [
    App\Providers\AppServiceProvider::class,
];
```

config

config/app.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Application Name
    |--------------------------------------------------------------------------
    |
    | This value is the name of your application, which will be used when the
    | framework needs to place the application's name in a notification or
    | other UI elements where an application name needs to be displayed.
    |
    */

    'name' => env('APP_NAME', 'Laravel'),

    /*
    |--------------------------------------------------------------------------
    | Application Environment
    |--------------------------------------------------------------------------
    |
    | This value determines the "environment" your application is currently
    | running in. This may determine how you prefer to configure various
```

```
    | services the application utilizes. Set this in your ".env" file.
    |
    */

    'env' => env('APP_ENV', 'production'),

    /*
    |--------------------------------------------------------------------------
    | Application Debug Mode
    |--------------------------------------------------------------------------
    |
    | When your application is in debug mode, detailed error messages with
    | stack traces will be shown on every error that occurs within your
    | application. If disabled, a simple generic error page is shown.
    |
    */

    'debug' => (bool) env('APP_DEBUG', false),

    /*
    |--------------------------------------------------------------------------
    | Application URL
    |--------------------------------------------------------------------------
    |
    | This URL is used by the console to properly generate URLs when using
    | the Artisan command line tool. You should set this to the root of
    | the application so that it's available within Artisan commands.
    |
    */

    'url' => env('APP_URL', 'http://localhost'),

    /*
    |--------------------------------------------------------------------------
    | Application Timezone
    |--------------------------------------------------------------------------
    |
    | Here you may specify the default timezone for your application, which
    | will be used by the PHP date and date-time functions. The timezone
    | is set to "UTC" by default as it is suitable for most use cases.
    |
    */

    'timezone' => env('APP_TIMEZONE', 'Asia/Manila'),

    /*
    |--------------------------------------------------------------------------
    | Application Locale Configuration
    |--------------------------------------------------------------------------
    |
    | The application locale determines the default locale that will be used
    | by Laravel's translation / localization methods. This option can be
    | set to any locale for which you plan to have translation strings.
    |
    */

    'locale' => env('APP_LOCALE', 'en'),

    'fallback_locale' => env('APP_FALLBACK_LOCALE', 'en'),

    'faker_locale' => env('APP_FAKER_LOCALE', 'en_US'),

    /*
    |--------------------------------------------------------------------------
    | Encryption Key
    |--------------------------------------------------------------------------
    |
    | This key is utilized by Laravel's encryption services and should be set
    | to a random, 32 character string to ensure that all encrypted values
    | are secure. You should do this prior to deploying the application.
    |
    */

    'cipher' => 'AES-256-CBC',
```

```

    'key' => env('APP_KEY'),

    'previous_keys' => [
        ...array_filter(
            explode(',', (string) env('APP_PREVIOUS_KEYS', ''))
        ),
    ],

    /*
    |--------------------------------------------------------------------------
    | Maintenance Mode Driver
    |--------------------------------------------------------------------------
    |
    | These configuration options determine the driver used to determine and
    | manage Laravel's "maintenance mode" status. The "cache" driver will
    | allow maintenance mode to be controlled across multiple machines.
    |
    | Supported drivers: "file", "cache"
    |
    */

    'maintenance' => [
        'driver' => env('APP_MAINTENANCE_DRIVER', 'file'),
        'store' => env('APP_MAINTENANCE_STORE', 'database'),
    ],

];
```

config/auth.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Authentication Defaults
    |--------------------------------------------------------------------------
    |
    | This option defines the default authentication "guard" and password
    | reset "broker" for your application. You may change these values
    | as required, but they're a perfect start for most applications.
    |
    */

    'defaults' => [
        'guard' => env('AUTH_GUARD', 'web'),
        'passwords' => env('AUTH_PASSWORD_BROKER', 'users'),
    ],

    /*
    |--------------------------------------------------------------------------
    | Authentication Guards
    |--------------------------------------------------------------------------
    |
    | Next, you may define every authentication guard for your application.
    | Of course, a great default configuration has been defined for you
    | which utilizes session storage plus the Eloquent user provider.
    |
    | All authentication guards have a user provider, which defines how the
    | users are actually retrieved out of your database or other storage
    | system used by the application. Typically, Eloquent is utilized.
    |
    | Supported: "session"
    |
    */

    'guards' => [
        'web' => [
            'driver' => 'session',
            'provider' => 'users',
        ],
    ],
```

```
44
45      /*
46      |--------------------------------------------------------------------------
47      | User Providers
48      |--------------------------------------------------------------------------
49      |
50      | All authentication guards have a user provider, which defines how the
51      | users are actually retrieved out of your database or other storage
52      | system used by the application. Typically, Eloquent is utilized.
53      |
54      | If you have multiple user tables or models you may configure multiple
55      | providers to represent the model / table. These providers may then
56      | be assigned to any extra authentication guards you have defined.
57      |
58      | Supported: "database", "eloquent"
59      |
60      */
61
62      'providers' => [
63          'users' => [
64              'driver' => 'eloquent',
65              'model' => env('AUTH_MODEL', App\Models\User::class),
66          ],
67
68          // 'users' => [
69          //     'driver' => 'database',
70          //     'table' => 'users',
71          // ],
72      ],
73
74      /*
75      |--------------------------------------------------------------------------
76      | Resetting Passwords
77      |--------------------------------------------------------------------------
78      |
79      | These configuration options specify the behavior of Laravel's password
80      | reset functionality, including the table utilized for token storage
81      | and the user provider that is invoked to actually retrieve users.
82      |
83      | The expiry time is the number of minutes that each reset token will be
84      | considered valid. This security feature keeps tokens short-lived so
85      | they have less time to be guessed. You may change this as needed.
86      |
87      | The throttle setting is the number of seconds a user must wait before
88      | generating more password reset tokens. This prevents the user from
89      | quickly generating a very large amount of password reset tokens.
90      |
91      */
92
93      'passwords' => [
94          'users' => [
95              'provider' => 'users',
96              'table' => env('AUTH_PASSWORD_RESET_TOKEN_TABLE', 'password_reset_tokens'),
97              'expire' => 60,
98              'throttle' => 60,
99          ],
100     ],
101
102     /*
103     |--------------------------------------------------------------------------
104     | Password Confirmation Timeout
105     |--------------------------------------------------------------------------
106     |
107     | Here you may define the number of seconds before a password confirmation
108     | window expires and users are asked to re-enter their password via the
109     | confirmation screen. By default, the timeout lasts for three hours.
110     |
111     */
112
113     'password_timeout' => env('AUTH_PASSWORD_TIMEOUT', 10800),
114
115 ];
```

config/cache.php

```php
<?php

use Illuminate\Support\Str;

return [

    /*
    |--------------------------------------------------------------------------
    | Default Cache Store
    |--------------------------------------------------------------------------
    |
    | This option controls the default cache store that will be used by the
    | framework. This connection is utilized if another isn't explicitly
    | specified when running a cache operation inside the application.
    |
    */

    'default' => env('CACHE_STORE', 'file'),

    /*
    |--------------------------------------------------------------------------
    | Cache Stores
    |--------------------------------------------------------------------------
    |
    | Here you may define all of the cache "stores" for your application as
    | well as their drivers. You may even define multiple stores for the
    | same cache driver to group types of items stored in your caches.
    |
    | Supported drivers: "array", "database", "file", "memcached",
    |                    "redis", "dynamodb", "octane",
    |                    "failover", "null"
    |
    */

    'stores' => [

        'array' => [
            'driver' => 'array',
            'serialize' => false,
        ],

        'database' => [
            'driver' => 'database',
            'connection' => env('DB_CACHE_CONNECTION'),
            'table' => env('DB_CACHE_TABLE', 'cache'),
            'lock_connection' => env('DB_CACHE_LOCK_CONNECTION'),
            'lock_table' => env('DB_CACHE_LOCK_TABLE'),
        ],

        'file' => [
            'driver' => 'file',
            'path' => storage_path('framework/cache/data'),
            'lock_path' => storage_path('framework/cache/data'),
        ],

        'memcached' => [
            'driver' => 'memcached',
            'persistent_id' => env('MEMCACHED_PERSISTENT_ID'),
            'sasl' => [
                env('MEMCACHED_USERNAME'),
                env('MEMCACHED_PASSWORD'),
            ],
            'options' => [
                // Memcached::OPT_CONNECT_TIMEOUT => 2000,
            ],
            'servers' => [
                [
                    'host' => env('MEMCACHED_HOST', '127.0.0.1'),
                    'port' => env('MEMCACHED_PORT', 11211),
                    'weight' => 100,
                ],
            ],
```

```
        ],

        'redis' => [
            'driver' => 'redis',
            'connection' => env('REDIS_CACHE_CONNECTION', 'cache'),
            'lock_connection' => env('REDIS_CACHE_LOCK_CONNECTION', 'default'),
        ],

        'dynamodb' => [
            'driver' => 'dynamodb',
            'key' => env('AWS_ACCESS_KEY_ID'),
            'secret' => env('AWS_SECRET_ACCESS_KEY'),
            'region' => env('AWS_DEFAULT_REGION', 'us-east-1'),
            'table' => env('DYNAMODB_CACHE_TABLE', 'cache'),
            'endpoint' => env('DYNAMODB_ENDPOINT'),
        ],

        'octane' => [
            'driver' => 'octane',
        ],

        'failover' => [
            'driver' => 'failover',
            'stores' => [
                'database',
                'array',
            ],
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Cache Key Prefix
    |--------------------------------------------------------------------------
    |
    | When utilizing the APC, database, memcached, Redis, and DynamoDB cache
    | stores, there might be other applications using the same cache. For
    | that reason, you may prefix every cache key to avoid collisions.
    |
    */

    'prefix' => env('CACHE_PREFIX', Str::slug((string) env('APP_NAME', 'laravel')).'-cache-'),

];
```

config/database.php

```
<?php

use Illuminate\Support\Str;

return [

    /*
    |--------------------------------------------------------------------------
    | Default Database Connection Name
    |--------------------------------------------------------------------------
    |
    | Here you may specify which of the database connections below you wish
    | to use as your default connection for database operations. This is
    | the connection which will be utilized unless another connection
    | is explicitly specified when you execute a query / statement.
    |
    */

'default' => env('DB_CONNECTION', 'mysql'),

    /*
    |--------------------------------------------------------------------------
    | Database Connections
    |--------------------------------------------------------------------------
    |
    | Below are all of the database connections defined for your application.
```

```
    | An example configuration is provided for each database system which
    | is supported by Laravel. You're free to add / remove connections.
    |
    */

    'connections' => [

        'sqlite' => [
            'driver' => 'sqlite',
            'url' => env('DB_URL'),
            'database' => env('DB_DATABASE', database_path('database.sqlite')),
            'prefix' => '',
            'foreign_key_constraints' => env('DB_FOREIGN_KEYS', true),
            'busy_timeout' => null,
            'journal_mode' => null,
            'synchronous' => null,
            'transaction_mode' => 'DEFERRED',
        ],

        'mysql' => [
            'driver' => 'mysql',
            'url' => env('DB_URL'),
            'host' => env('DB_HOST', '127.0.0.1'),
            'port' => env('DB_PORT', '3306'),
            'database' => env('DB_DATABASE', 'laravel'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'unix_socket' => env('DB_SOCKET', ''),
            'charset' => env('DB_CHARSET', 'utf8mb4'),
            'collation' => env('DB_COLLATION', 'utf8mb4_unicode_ci'),
            'prefix' => '',
            'prefix_indexes' => true,
            'strict' => true,
            'engine' => null,
            'options' => extension_loaded('pdo_mysql') ? array_filter([
                (PHP_VERSION_ID >= 80500 ? \Pdo\Mysql::ATTR_SSL_CA : \PDO::MYSQL_ATTR_SSL_CA) => env('MYSQL_ATTR_SSL_CA'),
            ]) : [],
        ],

        'mariadb' => [
            'driver' => 'mariadb',
            'url' => env('DB_URL'),
            'host' => env('DB_HOST', '127.0.0.1'),
            'port' => env('DB_PORT', '3306'),
            'database' => env('DB_DATABASE', 'laravel'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'unix_socket' => env('DB_SOCKET', ''),
            'charset' => env('DB_CHARSET', 'utf8mb4'),
            'collation' => env('DB_COLLATION', 'utf8mb4_unicode_ci'),
            'prefix' => '',
            'prefix_indexes' => true,
            'strict' => true,
            'engine' => null,
            'options' => extension_loaded('pdo_mysql') ? array_filter([
                (PHP_VERSION_ID >= 80500 ? \Pdo\Mysql::ATTR_SSL_CA : \PDO::MYSQL_ATTR_SSL_CA) => env('MYSQL_ATTR_SSL_CA'),
            ]) : [],
        ],

        'pgsql' => [
            'driver' => 'pgsql',
            'url' => env('DB_URL'),
            'host' => env('DB_HOST', '127.0.0.1'),
            'port' => env('DB_PORT', '5432'),
            'database' => env('DB_DATABASE', 'laravel'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'charset' => env('DB_CHARSET', 'utf8'),
            'prefix' => '',
            'prefix_indexes' => true,
            'search_path' => 'public',
            'sslmode' => env('DB_SSLMODE', 'prefer'),
        ],

```

```
        'sqlsrv' => [
            'driver' => 'sqlsrv',
            'url' => env('DB_URL'),
            'host' => env('DB_HOST', 'localhost'),
            'port' => env('DB_PORT', '1433'),
            'database' => env('DB_DATABASE', 'laravel'),
            'username' => env('DB_USERNAME', 'root'),
            'password' => env('DB_PASSWORD', ''),
            'charset' => env('DB_CHARSET', 'utf8'),
            'prefix' => '',
            'prefix_indexes' => true,
            // 'encrypt' => env('DB_ENCRYPT', 'yes'),
            // 'trust_server_certificate' => env('DB_TRUST_SERVER_CERTIFICATE', 'false'),
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Migration Repository Table
    |--------------------------------------------------------------------------
    |
    | This table keeps track of all the migrations that have already run for
    | your application. Using this information, we can determine which of
    | the migrations on disk haven't actually been run on the database.
    |
    */

    'migrations' => [
        'table' => 'migrations',
        'update_date_on_publish' => true,
    ],

    /*
    |--------------------------------------------------------------------------
    | Redis Databases
    |--------------------------------------------------------------------------
    |
    | Redis is an open source, fast, and advanced key-value store that also
    | provides a richer body of commands than a typical key-value system
    | such as Memcached. You may define your connection settings here.
    |
    */

    'redis' => [

        'client' => env('REDIS_CLIENT', 'phpredis'),

        'options' => [
            'cluster' => env('REDIS_CLUSTER', 'redis'),
            'prefix' => env('REDIS_PREFIX', Str::slug((string) env('APP_NAME', 'laravel')).'-database-'),
            'persistent' => env('REDIS_PERSISTENT', false),
        ],

        'default' => [
            'url' => env('REDIS_URL'),
            'host' => env('REDIS_HOST', '127.0.0.1'),
            'username' => env('REDIS_USERNAME'),
            'password' => env('REDIS_PASSWORD'),
            'port' => env('REDIS_PORT', '6379'),
            'database' => env('REDIS_DB', '0'),
            'max_retries' => env('REDIS_MAX_RETRIES', 3),
            'backoff_algorithm' => env('REDIS_BACKOFF_ALGORITHM', 'decorrelated_jitter'),
            'backoff_base' => env('REDIS_BACKOFF_BASE', 100),
            'backoff_cap' => env('REDIS_BACKOFF_CAP', 1000),
        ],

        'cache' => [
            'url' => env('REDIS_URL'),
            'host' => env('REDIS_HOST', '127.0.0.1'),
            'username' => env('REDIS_USERNAME'),
            'password' => env('REDIS_PASSWORD'),
            'port' => env('REDIS_PORT', '6379'),
            'database' => env('REDIS_CACHE_DB', '1'),
```

```
            'max_retries' => env('REDIS_MAX_RETRIES', 3),
            'backoff_algorithm' => env('REDIS_BACKOFF_ALGORITHM', 'decorrelated_jitter'),
            'backoff_base' => env('REDIS_BACKOFF_BASE', 100),
            'backoff_cap' => env('REDIS_BACKOFF_CAP', 1000),
        ],

    ],

];
```

config/filesystems.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Default Filesystem Disk
    |--------------------------------------------------------------------------
    |
    | Here you may specify the default filesystem disk that should be used
    | by the framework. The "local" disk, as well as a variety of cloud
    | based disks are available to your application for file storage.
    |
    */

    'default' => env('FILESYSTEM_DISK', 'local'),

    /*
    |--------------------------------------------------------------------------
    | Filesystem Disks
    |--------------------------------------------------------------------------
    |
    | Below you may configure as many filesystem disks as necessary, and you
    | may even configure multiple disks for the same driver. Examples for
    | most supported storage drivers are configured here for reference.
    |
    | Supported drivers: "local", "ftp", "sftp", "s3"
    |
    */

    'disks' => [

        'local' => [
            'driver' => 'local',
            'root' => storage_path('app/private'),
            'serve' => true,
            'throw' => false,
            'report' => false,
        ],

        'public' => [
            'driver' => 'local',
            'root' => storage_path('app/public'),
            'url' => rtrim(env('APP_URL', 'http://localhost'), '/').'/storage',
            'visibility' => 'public',
            'throw' => false,
            'report' => false,
        ],

        's3' => [
            'driver' => 's3',
            'key' => env('AWS_ACCESS_KEY_ID'),
            'secret' => env('AWS_SECRET_ACCESS_KEY'),
            'region' => env('AWS_DEFAULT_REGION'),
            'bucket' => env('AWS_BUCKET'),
            'url' => env('AWS_URL'),
            'endpoint' => env('AWS_ENDPOINT'),
            'use_path_style_endpoint' => env('AWS_USE_PATH_STYLE_ENDPOINT', false),
            'throw' => false,
            'report' => false,
        ],

```

```
    ],

    /*
    |--------------------------------------------------------------------------
    | Symbolic Links
    |--------------------------------------------------------------------------
    |
    | Here you may configure the symbolic links that will be created when the
    | `storage:link` Artisan command is executed. The array keys should be
    | the locations of the links and the values should be their targets.
    |
    */

    'links' => [
        public_path('storage') => storage_path('app/public'),
    ],

];
```

config/logging.php

```
<?php

use Monolog\Handler\NullHandler;
use Monolog\Handler\StreamHandler;
use Monolog\Handler\SyslogUdpHandler;
use Monolog\Processor\PsrLogMessageProcessor;

return [

    /*
    |--------------------------------------------------------------------------
    | Default Log Channel
    |--------------------------------------------------------------------------
    |
    | This option defines the default log channel that is utilized to write
    | messages to your logs. The value provided here should match one of
    | the channels present in the list of "channels" configured below.
    |
    */

    'default' => env('LOG_CHANNEL', 'stack'),

    /*
    |--------------------------------------------------------------------------
    | Deprecations Log Channel
    |--------------------------------------------------------------------------
    |
    | This option controls the log channel that should be used to log warnings
    | regarding deprecated PHP and library features. This allows you to get
    | your application ready for upcoming major versions of dependencies.
    |
    */

    'deprecations' => [
        'channel' => env('LOG_DEPRECATIONS_CHANNEL', 'null'),
        'trace' => env('LOG_DEPRECATIONS_TRACE', false),
    ],

    /*
    |--------------------------------------------------------------------------
    | Log Channels
    |--------------------------------------------------------------------------
    |
    | Here you may configure the log channels for your application. Laravel
    | utilizes the Monolog PHP logging library, which includes a variety
    | of powerful log handlers and formatters that you're free to use.
    |
    | Available drivers: "single", "daily", "slack", "syslog",
    |                    "errorlog", "monolog", "custom", "stack"
    |
    */

    'channels' => [
```

```
54
55          'stack' => [
56              'driver' => 'stack',
57              'channels' => explode(',', (string) env('LOG_STACK', 'single')),
58              'ignore_exceptions' => false,
59          ],
60
61          'single' => [
62              'driver' => 'single',
63              'path' => storage_path('logs/laravel.log'),
64              'level' => env('LOG_LEVEL', 'debug'),
65              'replace_placeholders' => true,
66          ],
67
68          'otp' => [
69              'driver' => 'single',
70              'path' => storage_path('logs/otp.log'),
71              'level' => env('LOG_LEVEL', 'debug'),
72              'replace_placeholders' => true,
73          ],
74
75          'daily' => [
76              'driver' => 'daily',
77              'path' => storage_path('logs/laravel.log'),
78              'level' => env('LOG_LEVEL', 'debug'),
79              'days' => env('LOG_DAILY_DAYS', 14),
80              'replace_placeholders' => true,
81          ],
82
83          'slack' => [
84              'driver' => 'slack',
85              'url' => env('LOG_SLACK_WEBHOOK_URL'),
86              'username' => env('LOG_SLACK_USERNAME', 'Laravel Log'),
87              'emoji' => env('LOG_SLACK_EMOJI', ':boom:'),
88              'level' => env('LOG_LEVEL', 'critical'),
89              'replace_placeholders' => true,
90          ],
91
92          'papertrail' => [
93              'driver' => 'monolog',
94              'level' => env('LOG_LEVEL', 'debug'),
95              'handler' => env('LOG_PAPERTRAIL_HANDLER', SyslogUdpHandler::class),
96              'handler_with' => [
97                  'host' => env('PAPERTRAIL_URL'),
98                  'port' => env('PAPERTRAIL_PORT'),
99                  'connectionString' => 'tls://'.env('PAPERTRAIL_URL').':'.env('PAPERTRAIL_PORT'),
100             ],
101             'processors' => [PsrLogMessageProcessor::class],
102         ],
103
104         'stderr' => [
105             'driver' => 'monolog',
106             'level' => env('LOG_LEVEL', 'debug'),
107             'handler' => StreamHandler::class,
108             'handler_with' => [
109                 'stream' => 'php://stderr',
110             ],
111             'formatter' => env('LOG_STDERR_FORMATTER'),
112             'processors' => [PsrLogMessageProcessor::class],
113         ],
114
115         'syslog' => [
116             'driver' => 'syslog',
117             'level' => env('LOG_LEVEL', 'debug'),
118             'facility' => env('LOG_SYSLOG_FACILITY', LOG_USER),
119             'replace_placeholders' => true,
120         ],
121
122         'errorlog' => [
123             'driver' => 'errorlog',
124             'level' => env('LOG_LEVEL', 'debug'),
125             'replace_placeholders' => true,
126         ],
127
```

```
        'null' => [
            'driver' => 'monolog',
            'handler' => NullHandler::class,
        ],

        'emergency' => [
            'path' => storage_path('logs/laravel.log'),
        ],

    ],

];
```

config/mail.php

```
<?php

return [
    'default' => env('MAIL_MAILER', 'log'),

    'mailers' => [
        'smtp' => [
            'transport' => 'smtp',
            'scheme' => env('MAIL_SCHEME'),
            'url' => env('MAIL_URL'),
            'host' => env('MAIL_HOST', '127.0.0.1'),
            'port' => env('MAIL_PORT', 2525),
            'username' => env('MAIL_USERNAME'),
            'password' => env('MAIL_PASSWORD'),
            'timeout' => null,
            'local_domain' => env('MAIL_EHLO_DOMAIN'),
        ],

        'resend' => [
            'transport' => 'resend',
            'key' => env('RESEND_KEY'),
        ],

        'gmail_api' => [
            'transport' => 'gmail-api',
            'client_id' => env('GMAIL_MAIL_CLIENT_ID'),
            'client_secret' => env('GMAIL_MAIL_CLIENT_SECRET'),
            'refresh_token' => env('GMAIL_MAIL_REFRESH_TOKEN'),
        ],

        'log' => [
            'transport' => 'log',
            'channel' => env('MAIL_LOG_CHANNEL'),
        ],

        'array' => [
            'transport' => 'array',
        ],

        'failover' => [
            'transport' => 'failover',
            'mailers' => ['smtp', 'log'],
        ],
    ],

    'from' => [
        'address' => env('MAIL_FROM_ADDRESS', 'hello@example.com'),
        'name' => env('MAIL_FROM_NAME', env('APP_NAME', 'Campus Reserve')),
    ],
];
```

config/queue.php

```
<?php

return [

    /*
```

```
    |--------------------------------------------------------------------------
    | Default Queue Connection Name
    |--------------------------------------------------------------------------
    |
    | Laravel's queue supports a variety of backends via a single, unified
    | API, giving you convenient access to each backend using identical
    | syntax for each. The default queue connection is defined below.
    |
    */

    'default' => env('QUEUE_CONNECTION', 'database'),

    /*
    |--------------------------------------------------------------------------
    | Queue Connections
    |--------------------------------------------------------------------------
    |
    | Here you may configure the connection options for every queue backend
    | used by your application. An example configuration is provided for
    | each backend supported by Laravel. You're also free to add more.
    |
    | Drivers: "sync", "database", "beanstalkd", "sqs", "redis",
    |          "deferred", "background", "failover", "null"
    |
    */

    'connections' => [

        'sync' => [
            'driver' => 'sync',
        ],

        'database' => [
            'driver' => 'database',
            'connection' => env('DB_QUEUE_CONNECTION'),
            'table' => env('DB_QUEUE_TABLE', 'jobs'),
            'queue' => env('DB_QUEUE', 'default'),
            'retry_after' => (int) env('DB_QUEUE_RETRY_AFTER', 90),
            'after_commit' => false,
        ],

        'beanstalkd' => [
            'driver' => 'beanstalkd',
            'host' => env('BEANSTALKD_QUEUE_HOST', 'localhost'),
            'queue' => env('BEANSTALKD_QUEUE', 'default'),
            'retry_after' => (int) env('BEANSTALKD_QUEUE_RETRY_AFTER', 90),
            'block_for' => 0,
            'after_commit' => false,
        ],

        'sqs' => [
            'driver' => 'sqs',
            'key' => env('AWS_ACCESS_KEY_ID'),
            'secret' => env('AWS_SECRET_ACCESS_KEY'),
            'prefix' => env('SQS_PREFIX', 'https://sqs.us-east-1.amazonaws.com/your-account-id'),
            'queue' => env('SQS_QUEUE', 'default'),
            'suffix' => env('SQS_SUFFIX'),
            'region' => env('AWS_DEFAULT_REGION', 'us-east-1'),
            'after_commit' => false,
        ],

        'redis' => [
            'driver' => 'redis',
            'connection' => env('REDIS_QUEUE_CONNECTION', 'default'),
            'queue' => env('REDIS_QUEUE', 'default'),
            'retry_after' => (int) env('REDIS_QUEUE_RETRY_AFTER', 90),
            'block_for' => null,
            'after_commit' => false,
        ],

        'deferred' => [
            'driver' => 'deferred',
        ],

```

```
        'background' => [
            'driver' => 'background',
        ],

        'failover' => [
            'driver' => 'failover',
            'connections' => [
                'database',
                'deferred',
            ],
        ],

    ],

    /*
    |--------------------------------------------------------------------------
    | Job Batching
    |--------------------------------------------------------------------------
    |
    | The following options configure the database and table that store job
    | batching information. These options can be updated to any database
    | connection and table which has been defined by your application.
    |
    */

    'batching' => [
        'database' => env('DB_CONNECTION', 'sqlite'),
        'table' => 'job_batches',
    ],

    /*
    |--------------------------------------------------------------------------
    | Failed Queue Jobs
    |--------------------------------------------------------------------------
    |
    | These options configure the behavior of failed queue job logging so you
    | can control how and where failed jobs are stored. Laravel ships with
    | support for storing failed jobs in a simple file or in a database.
    |
    | Supported drivers: "database-uuids", "dynamodb", "file", "null"
    |
    */

    'failed' => [
        'driver' => env('QUEUE_FAILED_DRIVER', 'database-uuids'),
        'database' => env('DB_CONNECTION', 'sqlite'),
        'table' => 'failed_jobs',
    ],

];
```

config/services.php

```
<?php

return [

    /*
    |--------------------------------------------------------------------------
    | Third Party Services
    |--------------------------------------------------------------------------
    |
    | This file is for storing the credentials for third party services such
    | as Slack, Google, and more. This file provides the de facto
    | location for this type of information, allowing packages to have
    | a conventional file to locate the various service credentials.
    |
    */

    'slack' => [
        'notifications' => [
            'bot_user_oauth_token' => env('SLACK_BOT_USER_OAUTH_TOKEN'),
            'channel' => env('SLACK_BOT_USER_DEFAULT_CHANNEL'),
        ],
```

```
    ],

    'google' => [
        'client_id' => env('GOOGLE_CLIENT_ID'),
        'client_secret' => env('GOOGLE_CLIENT_SECRET'),
        'redirect' => env('GOOGLE_REDIRECT_URL', env('APP_URL').'/auth/google/callback'),
    ],

    'resend' => [
        'key' => env('RESEND_KEY'),
    ],

];
```

config/session.php

```
<?php

use Illuminate\Support\Str;

return [

    /*
    |--------------------------------------------------------------------------
    | Default Session Driver
    |--------------------------------------------------------------------------
    |
    | This option determines the default session driver that is utilized for
    | incoming requests. Laravel supports a variety of storage options to
    | persist session data. Database storage is a great default choice.
    |
    | Supported: "file", "cookie", "database", "memcached",
    |            "redis", "dynamodb", "array"
    |
    */

'driver' => env('SESSION_DRIVER', 'file'),

    /*
    |--------------------------------------------------------------------------
    | Session Lifetime
    |--------------------------------------------------------------------------
    |
    | Here you may specify the number of minutes that you wish the session
    | to be allowed to remain idle before it expires. If you want them
    | to expire immediately when the browser is closed then you may
    | indicate that via the expire_on_close configuration option.
    |
    */

    'lifetime' => (int) env('SESSION_LIFETIME', 120),

    'expire_on_close' => env('SESSION_EXPIRE_ON_CLOSE', false),

    /*
    |--------------------------------------------------------------------------
    | Session Encryption
    |--------------------------------------------------------------------------
    |
    | This option allows you to easily specify that all of your session data
    | should be encrypted before it's stored. All encryption is performed
    | automatically by Laravel and you may use the session like normal.
    |
    */

    'encrypt' => env('SESSION_ENCRYPT', false),

    /*
    |--------------------------------------------------------------------------
    | Session File Location
    |--------------------------------------------------------------------------
    |
    | When utilizing the "file" session driver, the session files are placed
    | on disk. The default storage location is defined here; however, you
```

```
    | are free to provide another location where they should be stored.
    |
    */

    'files' => storage_path('framework/sessions'),

    /*
    |--------------------------------------------------------------------------
    | Session Database Connection
    |--------------------------------------------------------------------------
    |
    | When using the "database" or "redis" session drivers, you may specify a
    | connection that should be used to manage these sessions. This should
    | correspond to a connection in your database configuration options.
    |
    */

    'connection' => env('SESSION_CONNECTION'),

    /*
    |--------------------------------------------------------------------------
    | Session Database Table
    |--------------------------------------------------------------------------
    |
    | When using the "database" session driver, you may specify the table to
    | be used to store sessions. Of course, a sensible default is defined
    | for you; however, you're welcome to change this to another table.
    |
    */

    'table' => env('SESSION_TABLE', 'sessions'),

    /*
    |--------------------------------------------------------------------------
    | Session Cache Store
    |--------------------------------------------------------------------------
    |
    | When using one of the framework's cache driven session backends, you may
    | define the cache store which should be used to store the session data
    | between requests. This must match one of your defined cache stores.
    |
    | Affects: "dynamodb", "memcached", "redis"
    |
    */

    'store' => env('SESSION_STORE'),

    /*
    |--------------------------------------------------------------------------
    | Session Sweeping Lottery
    |--------------------------------------------------------------------------
    |
    | Some session drivers must manually sweep their storage location to get
    | rid of old sessions from storage. Here are the chances that it will
    | happen on a given request. By default, the odds are 2 out of 100.
    |
    */

    'lottery' => [2, 100],

    /*
    |--------------------------------------------------------------------------
    | Session Cookie Name
    |--------------------------------------------------------------------------
    |
    | Here you may change the name of the session cookie that is created by
    | the framework. Typically, you should not need to change this value
    | since doing so does not grant a meaningful security improvement.
    |
    */

    'cookie' => env(
        'SESSION_COOKIE',
        Str::slug((string) env('APP_NAME', 'laravel')).'-session'
```

```
    ),

    /*
    |--------------------------------------------------------------------------
    | Session Cookie Path
    |--------------------------------------------------------------------------
    |
    | The session cookie path determines the path for which the cookie will
    | be regarded as available. Typically, this will be the root path of
    | your application, but you're free to change this when necessary.
    |
    */

    'path' => env('SESSION_PATH', '/'),

    /*
    |--------------------------------------------------------------------------
    | Session Cookie Domain
    |--------------------------------------------------------------------------
    |
    | This value determines the domain and subdomains the session cookie is
    | available to. By default, the cookie will be available to the root
    | domain without subdomains. Typically, this shouldn't be changed.
    |
    */

    'domain' => env('SESSION_DOMAIN'),

    /*
    |--------------------------------------------------------------------------
    | HTTPS Only Cookies
    |--------------------------------------------------------------------------
    |
    | By setting this option to true, session cookies will only be sent back
    | to the server if the browser has a HTTPS connection. This will keep
    | the cookie from being sent to you when it can't be done securely.
    |
    */

    'secure' => env('SESSION_SECURE_COOKIE'),

    /*
    |--------------------------------------------------------------------------
    | HTTP Access Only
    |--------------------------------------------------------------------------
    |
    | Setting this value to true will prevent JavaScript from accessing the
    | value of the cookie and the cookie will only be accessible through
    | the HTTP protocol. It's unlikely you should disable this option.
    |
    */

    'http_only' => env('SESSION_HTTP_ONLY', true),

    /*
    |--------------------------------------------------------------------------
    | Same-Site Cookies
    |--------------------------------------------------------------------------
    |
    | This option determines how your cookies behave when cross-site requests
    | take place, and can be used to mitigate CSRF attacks. By default, we
    | will set this value to "lax" to permit secure cross-site requests.
    |
    | See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie#samesitesamesite-value
    |
    | Supported: "lax", "strict", "none", null
    |
    */

    'same_site' => env('SESSION_SAME_SITE', 'lax'),

    /*
    |--------------------------------------------------------------------------
    | Partitioned Cookies
```

```
    |--------------------------------------------------------------------------
    |
    | Setting this value to true will tie the cookie to the top-level site for
    | a cross-site context. Partitioned cookies are accepted by the browser
    | when flagged "secure" and the Same-Site attribute is set to "none".
    |
    */

    'partitioned' => env('SESSION_PARTITIONED_COOKIE', false),

];
```

database

database/factories/UserFactory.php

```php
<?php

namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;

/**
 * @extends \Illuminate\Database\Eloquent\Factories\Factory<\App\Models\User>
 */
class UserFactory extends Factory
{
    /**
     * The current password being used by the factory.
     */
    protected static ?string $password;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'name' => fake()->name(),
            'email' => fake()->unique()->safeEmail(),
            'email_verified_at' => now(),
            'password' => static::$password ??= Hash::make('password'),
            'remember_token' => Str::random(10),
        ];
    }

    /**
     * Indicate that the model's email address should be unverified.
     */
    public function unverified(): static
    {
        return $this->state(fn (array $attributes) => [
            'email_verified_at' => null,
        ]);
    }
}
```

database/migrations/0001_01_01_000000_create_users_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
```

```
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });

        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->longText('payload');
            $table->integer('last_activity')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

database/migrations/0001_01_01_000001_create_cache_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('cache', function (Blueprint $table) {
            $table->string('key')->primary();
            $table->mediumText('value');
            $table->integer('expiration')->index();
        });

        Schema::create('cache_locks', function (Blueprint $table) {
            $table->string('key')->primary();
            $table->string('owner');
            $table->integer('expiration')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
```

```
    {
        Schema::dropIfExists('cache');
        Schema::dropIfExists('cache_locks');
    }
};
```

database/migrations/0001_01_01_000002_create_jobs_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('jobs', function (Blueprint $table) {
            $table->id();
            $table->string('queue')->index();
            $table->longText('payload');
            $table->unsignedTinyInteger('attempts');
            $table->unsignedInteger('reserved_at')->nullable();
            $table->unsignedInteger('available_at');
            $table->unsignedInteger('created_at');
        });

        Schema::create('job_batches', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->string('name');
            $table->integer('total_jobs');
            $table->integer('pending_jobs');
            $table->integer('failed_jobs');
            $table->longText('failed_job_ids');
            $table->mediumText('options')->nullable();
            $table->integer('cancelled_at')->nullable();
            $table->integer('created_at');
            $table->integer('finished_at')->nullable();
        });

        Schema::create('failed_jobs', function (Blueprint $table) {
            $table->id();
            $table->string('uuid')->unique();
            $table->text('connection');
            $table->text('queue');
            $table->longText('payload');
            $table->longText('exception');
            $table->timestamp('failed_at')->useCurrent();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('jobs');
        Schema::dropIfExists('job_batches');
        Schema::dropIfExists('failed_jobs');
    }
};
```

database/migrations/0002_01_01_000000_add_remember_device_to_users_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('remember_device_token')->nullable()->after('remember_token');
            $table->timestamp('remember_device_expires_at')->nullable()->after('remember_device_token');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn(['remember_device_token', 'remember_device_expires_at']);
        });
    }
};
```

database/migrations/2024_01_01_000000_add_email_hash_to_users_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            // Change email column to TEXT to store encrypted data
            $table->text('email')->change();

            // Add email_hash column for lookups (since email is now encrypted)
            $table->string('email_hash')->after('email')->unique();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('email_hash');
            $table->string('email')->unique()->change();
        });
    }
};
```

database/migrations/2024_01_02_000000_create_saved_credentials_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
```

```
     *
     * Creates a table for storing AES-encrypted login credentials.
     * All sensitive fields (site_name, site_url, username, password, notes)
     * are stored as TEXT to accommodate encrypted data.
     */
    public function up(): void
    {
        Schema::create('saved_credentials', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->text('site_name');           // AES encrypted
            $table->text('site_url')->nullable(); // AES encrypted
            $table->text('username');             // AES encrypted
            $table->text('password');             // AES encrypted
            $table->text('notes')->nullable();    // AES encrypted
            $table->timestamps();

            // Index for faster user lookups
            $table->index('user_id');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('saved_credentials');
    }
};
```

database/migrations/2024_01_03_000000_create_facilities_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('facilities', function (Blueprint $table) {
            $table->id();
            $table->string('name', 100);
            $table->string('building', 100);
            $table->string('type', 50);
            $table->integer('capacity');
            $table->text('amenities')->nullable();
            $table->boolean('requires_approval')->default(false);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('facilities');
    }
};
```

database/migrations/2024_01_04_000000_create_reservations_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
```

```
     */
    public function up(): void
    {
        Schema::create('reservations', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->foreignId('facility_id')->constrained()->onDelete('cascade');
            $table->date('reservation_date');
            $table->time('start_time');
            $table->time('end_time');
            $table->string('purpose');
            $table->text('notes')->nullable();
            $table->enum('status', ['pending', 'approved', 'rejected', 'cancelled', 'completed'])->default('pending');
            $table->text('admin_remarks')->nullable();
            $table->integer('attendees_count')->nullable();
            $table->timestamps();

            // Prevent double booking
            $table->unique(['facility_id', 'reservation_date', 'start_time'], 'unique_facility_booking');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('reservations');
    }
};
```

database/migrations/2024_11_01_000000_add_2fa_columns_to_users_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('2fa_code')->nullable();
            $table->timestamp('2fa_expires_at')->nullable();
            $table->tinyInteger('2fa_attempts')->default(0);
        });
    }

    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn(['2fa_code', '2fa_expires_at', '2fa_attempts']);
        });
    }
};

```

database/migrations/2026_03_15_000001_add_is_active_to_facilities_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('facilities', function (Blueprint $table) {
            $table->boolean('is_active')->default(true)->after('amenities');
```

```
        });
    }

    public function down(): void
    {
        Schema::table('facilities', function (Blueprint $table) {
            $table->dropColumn('is_active');
        });
    }
};
```

database/migrations/2026_04_12_000000_add_profile_phone_and_2fa_settings_to_users_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('phone')->nullable()->after('email_hash');
            $table->boolean('two_fa_enabled')->default(false)->after('phone');
            $table->string('profile_picture')->nullable()->after('two_fa_enabled');
        });
    }

    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn(['phone', 'two_fa_enabled', 'profile_picture']);
        });
    }
};
```

database/migrations/2026_04_17_000000_add_is_admin_to_users_and_create_audit_logs_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->boolean('is_admin')->default(false)->after('remember_device_expires_at');
            $table->boolean('is_super_admin')->default(false)->after('is_admin');
        });

        Schema::create('audit_logs', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->foreignId('reservation_id')->nullable()->constrained()->nullOnDelete();
            $table->string('action');
            $table->text('details')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('audit_logs');

        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn(['is_admin', 'is_super_admin']);
        });
    }
```

```
};
```

database/migrations/2026_04_18_000000_add_theme_preference_to_users_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('theme_preference')->default('light')->after('two_fa_enabled');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('theme_preference');
        });
    }
};
```

database/migrations/2026_04_18_000000_create_notifications_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up()
    {
        Schema::create('notifications', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->string('type');
            $table->morphs('notifiable');
            $table->text('data');
            $table->timestamp('read_at')->nullable();
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('notifications');
    }
};
```

database/migrations/2026_04_23_000000_create_otps_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
```

```
        Schema::create('otps', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->string('code');
            $table->timestamp('expires_at');
            $table->integer('attempts')->default(0);
            $table->timestamps();

            $table->index('user_id');
            $table->index('expires_at');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('otps');
    }
};
```

database/migrations/2026_08_28_000000_add_two_fa_secret_to_users_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table): void {
            $table->text('two_fa_secret')->nullable();
        });
    }

    public function down(): void
    {
        Schema::table('users', function (Blueprint $table): void {
            $table->dropColumn('two_fa_secret');
        });
    }
};
```

database/migrations/2026_10_07_000001_add_unique_index_to_facilities_name_and_building.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        if (! Schema::hasTable('facilities')) {
            return;
        }

        $driver = Schema::getConnection()->getDriverName();

        if ($driver === 'pgsql') {
            DB::statement('CREATE UNIQUE INDEX facilities_name_building_unique_idx ON facilities (LOWER(name), LOWER(building));');
            return;
        }

        Schema::table('facilities', function (Blueprint $table) {
            $table->unique(['name', 'building']);
```

```
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        if (! Schema::hasTable('facilities')) {
            return;
        }

        $driver = Schema::getConnection()->getDriverName();

        if ($driver === 'pgsql') {
            DB::statement('DROP INDEX IF EXISTS facilities_name_building_unique_idx;');
            return;
        }

        Schema::table('facilities', function (Blueprint $table) {
            $table->dropUnique(['name', 'building']);
        });
    }
};
```

database/migrations/2026_10_13_000001_create_facility_items_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('facility_items', function (Blueprint $table) {
            $table->id();
            $table->foreignId('facility_id')->constrained()->cascadeOnDelete();
            $table->string('name', 100);
            $table->unsignedInteger('quantity');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('facility_items');
    }
};
```

database/migrations/2026_10_14_000001_create_reservation_items_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('reservation_items', function (Blueprint $table) {
            $table->id();
            $table->foreignId('reservation_id')->constrained()->cascadeOnDelete();
            $table->foreignId('facility_item_id')->constrained()->cascadeOnDelete();
            $table->unsignedInteger('quantity');
            $table->timestamps();
            $table->unique(['reservation_id', 'facility_item_id']);
        });
    }
```

```

    public function down(): void
    {
        Schema::dropIfExists('reservation_items');
    }
};
```

database/migrations/2026_10_15_000001_add_return_reminded_at_to_reservations_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('reservations', function (Blueprint $table) {
            $table->timestamp('return_reminded_at')->nullable();
            $table->index(
                ['status', 'return_reminded_at', 'reservation_date', 'end_time'],
                'reservations_due_return_lookup_index'
            );
        });
    }

    public function down(): void
    {
        Schema::table('reservations', function (Blueprint $table) {
            $table->dropIndex('reservations_due_return_lookup_index');
            $table->dropColumn('return_reminded_at');
        });
    }
};
```

database/migrations/2026_10_15_000002_add_email_notifications_to_users_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->boolean('email_notifications')->default(false);
        });
    }

    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('email_notifications');
        });
    }
};
```

database/migrations/2026_10_19_000001_add_calendar_and_item_availability_indexes.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
```

```
    {
        Schema::table('reservations', function (Blueprint $table) {
            $table->index(
                ['status', 'reservation_date'],
                'reservations_status_date_index'
            );
        });

        Schema::table('reservation_items', function (Blueprint $table) {
            $table->index('facility_item_id', 'reservation_items_facility_item_index');
        });
    }

    public function down(): void
    {
        Schema::table('reservation_items', function (Blueprint $table) {
            $table->dropIndex('reservation_items_facility_item_index');
        });

        Schema::table('reservations', function (Blueprint $table) {
            $table->dropIndex('reservations_status_date_index');
        });
    }
};
```

database/migrations/2026_10_19_000002_add_rejected_at_to_reservations_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('reservations', function (Blueprint $table): void {
            $table->timestamp('rejected_at')->nullable()->after('status');
            $table->index(['status', 'rejected_at']);
        });

        DB::table('reservations')
            ->where('status', 'rejected')
            ->update(['rejected_at' => DB::raw('updated_at')]);
    }

    public function down(): void
    {
        Schema::table('reservations', function (Blueprint $table): void {
            $table->dropIndex(['status', 'rejected_at']);
            $table->dropColumn('rejected_at');
        });
    }
};
```

database/migrations/2026_10_19_000003_add_user_reservation_history_indexes.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('reservations', function (Blueprint $table) {
            $table->index(
                ['user_id', 'reservation_date', 'start_time'],
                'reservations_user_history_index'
```

```
            );
            $table->index(
                ['user_id', 'status', 'reservation_date'],
                'reservations_user_stats_index'
            );
        });
    }

    public function down(): void
    {
        Schema::table('reservations', function (Blueprint $table) {
            $table->dropIndex('reservations_user_history_index');
            $table->dropIndex('reservations_user_stats_index');
        });
    }
};
```

database/migrations/2026_10_23_000001_create_reservation_settings_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('reservation_settings', function (Blueprint $table) {
            $table->unsignedTinyInteger('id')->primary();
            $table->string('start_time', 5)->default('00:00');
            $table->string('end_time', 5)->default('23:59');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('reservation_settings');
    }
};
```

database/migrations/2026_10_23_000002_create_facility_maintenance_blocks_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('facility_maintenance_blocks', function (Blueprint $table) {
            $table->id();
            $table->foreignId('facility_id')->constrained()->cascadeOnDelete();
            $table->date('maintenance_date');
            $table->time('start_time')->nullable();
            $table->time('end_time')->nullable();
            $table->boolean('is_all_day')->default(false);
            $table->string('reason', 255);
            $table->timestamps();

            $table->index(['facility_id', 'maintenance_date']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('facility_maintenance_blocks');
    }
```

```
};
```

database/migrations/2026_10_23_000003_make_property_reservations_independent.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        DB::table('reservations')
            ->whereIn('id', DB::table('reservation_items')->select('reservation_id'))
            ->delete();

        DB::table('facility_items')->update(['facility_id' => null]);

        Schema::table('facility_items', function (Blueprint $table) {
            $table->dropForeign(['facility_id']);
            $table->unsignedBigInteger('facility_id')->nullable()->change();
            $table->foreign('facility_id')->references('id')->on('facilities')->nullOnDelete();
        });

        Schema::table('reservations', function (Blueprint $table) {
            $table->dropForeign(['facility_id']);
            $table->unsignedBigInteger('facility_id')->nullable()->change();
            $table->foreign('facility_id')->references('id')->on('facilities')->nullOnDelete();
        });
    }

    public function down(): void
    {
        throw new RuntimeException(
            'This migration deletes legacy property reservations and cannot be reversed without losing newer independent property data.'
        );
    }
};
```

database/migrations/2026_10_25_000001_create_property_item_maintenance_blocks_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('property_item_maintenance_blocks', function (Blueprint $table): void {
            $table->id();
            $table->foreignId('facility_item_id')->constrained()->cascadeOnDelete();
            $table->date('maintenance_date');
            $table->time('start_time')->nullable();
            $table->time('end_time')->nullable();
            $table->boolean('is_all_day')->default(false);
            $table->string('reason', 255);
            $table->timestamps();

            $table->index(['facility_item_id', 'maintenance_date']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('property_item_maintenance_blocks');
    }
};
```

database/migrations/2026_10_25_000002_create_user_profile_pictures_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('user_profile_pictures', function (Blueprint $table): void {
            $table->id();
            $table->foreignId('user_id')->unique()->constrained()->cascadeOnDelete();
            $table->string('mime_type', 32);
            $table->longText('image_data');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('user_profile_pictures');
    }
};
```

database/migrations/2026_10_25_000003_create_user_name_search_tokens_table.php

```php
<?php

use App\Models\User;
use App\Services\UserNameSearchIndex;
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('user_name_search_tokens', function (Blueprint $table): void {
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->char('token_hash', 64);
            $table->timestamps();
            $table->primary(['user_id', 'token_hash']);
            $table->index(['token_hash', 'user_id']);
        });

        $searchIndex = app(UserNameSearchIndex::class);
        User::query()->chunkById(200, function ($users) use ($searchIndex): void {
            foreach ($users as $user) {
                $searchIndex->sync($user);
            }
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('user_name_search_tokens');
    }
};
```

database/seeders/DatabaseSeeder.php

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

```

```
class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // User::factory(10)->create();

        // Only seed dummy reservations (assumes users/facilities already exist)
$this->call(UserSeeder::class);
        $this->call(FacilitySeeder::class);
        $this->call(DummyReservationSeeder::class);
    }
}
```

database/seeders/DummyReservationSeeder.php

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Reservation;
use App\Models\Facility;
use App\Models\User;
use Illuminate\Support\Facades\DB;

class DummyReservationSeeder extends Seeder
{
    public function run()
    {
        // Get a user (first user)
        $user = User::first();
        if (!$user) return;

        // Get facilities by name
        $facilities = Facility::whereIn('name', [
            'Conference Room A',
            'Computer Lab',
            'Auditorium',
        ])->get()->keyBy('name');

        // Insert dummy reservations
        $dummyReservations = [
            [
                'facility' => 'Conference Room A',
                'date' => '2026-03-20',
                'start_time' => '09:00:00',
                'end_time' => '11:00:00',
                'purpose' => 'Team Meeting',
            ],
            [
                'facility' => 'Computer Lab',
                'date' => '2026-03-22',
                'start_time' => '13:00:00',
                'end_time' => '15:00:00',
                'purpose' => 'Lab Session',
            ],
            [
                'facility' => 'Auditorium',
                'date' => '2026-03-25',
                'start_time' => '10:00:00',
                'end_time' => '12:00:00',
                'purpose' => 'Presentation',
            ],
        ];

        foreach ($dummyReservations as $dummy) {
            $facility = $facilities[$dummy['facility']] ?? null;
            if (!$facility) continue;
            Reservation::firstOrCreate([
```

```
                'user_id' => $user->id,
                'facility_id' => $facility->id,
                'reservation_date' => $dummy['date'],
                'start_time' => $dummy['start_time'],
                'end_time' => $dummy['end_time'],
                'purpose' => $dummy['purpose'],
            ], [
                'status' => 'pending',
            ]);
        }
    }
}
```

database/seeders/FacilitySeeder.php

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Facility;
use Illuminate\Support\Facades\DB;

class FacilitySeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        // Clear old facilities first - disable foreign key checks for truncate
        if (DB::getDriverName() === 'mysql') {
            DB::statement('SET FOREIGN_KEY_CHECKS=0;');
        } else {
            DB::statement('PRAGMA foreign_keys=OFF;');
        }

        Facility::truncate();

        if (DB::getDriverName() === 'mysql') {
            DB::statement('SET FOREIGN_KEY_CHECKS=1;');
        } else {
            DB::statement('PRAGMA foreign_keys=ON;');
        }

        $facilities = [
            // Computer Labs
            [
                'name' => 'Com LAB 1',
                'building' => 'Technology Building',
                'type' => 'Computer Lab',
                'capacity' => 30,
                'amenities' => 'Desktop computers, Projector, Whiteboard, Air conditioning',
                'requires_approval' => false,
                'is_active' => true,
            ],
            [
                'name' => 'Com LAB 2',
                'building' => 'Technology Building',
                'type' => 'Computer Lab',
                'capacity' => 30,
                'amenities' => 'Desktop computers, Projector, Whiteboard, Air conditioning',
                'requires_approval' => false,
                'is_active' => true,
            ],
            [
                'name' => 'Com LAB 3',
                'building' => 'Technology Building',
                'type' => 'Computer Lab',
                'capacity' => 30,
                'amenities' => 'Desktop computers, Projector, Whiteboard, Air conditioning',
                'requires_approval' => false,
                'is_active' => true,
            ],
```

```
            [
                'name' => 'Com LAB 4',
                'building' => 'Technology Building',
                'type' => 'Computer Lab',
                'capacity' => 30,
                'amenities' => 'Desktop computers, Projector, Whiteboard, Air conditioning',
                'requires_approval' => false,
                'is_active' => true,
            ],
            [
                'name' => 'Mac LAB 4',
                'building' => 'Technology Building',
                'type' => 'Computer Lab',
                'capacity' => 20,
                'amenities' => 'Mac computers, Projector, Whiteboard, Air conditioning',
                'requires_approval' => false,
                'is_active' => true,
            ],

            // Classrooms
            [
                'name' => 'Classroom A',
                'building' => 'Academic Building',
                'type' => 'Classroom',
                'capacity' => 50,
                'amenities' => 'Projector, Whiteboard, Tables and chairs, Air conditioning',
                'requires_approval' => false,
            ],
            [
                'name' => 'Classroom B',
                'building' => 'Academic Building',
                'type' => 'Classroom',
                'capacity' => 50,
                'amenities' => 'Projector, Whiteboard, Tables and chairs, Air conditioning',
                'requires_approval' => false,
            ],

            // Meeting Rooms
            [
                'name' => 'Meeting Room 1',
                'building' => 'Administrative Building',
                'type' => 'Meeting Room',
                'capacity' => 15,
                'amenities' => 'Conference table, Video conferencing, Whiteboard',
                'requires_approval' => false,
            ],
            [
                'name' => 'Meeting Room 2',
                'building' => 'Administrative Building',
                'type' => 'Meeting Room',
                'capacity' => 20,
                'amenities' => 'Conference table, Video conferencing, Projector',
                'requires_approval' => false,
            ],

            // Kitchen Classroom
            [
                'name' => 'Kitchen Classroom',
                'building' => 'Culinary Building',
                'type' => 'Kitchen',
                'capacity' => 25,
                'amenities' => 'Kitchen equipment, Cooking stations, Ventilation system',
                'requires_approval' => true,
            ],

            // Science Lab
            [
                'name' => 'Science Lab',
                'building' => 'Science Building',
                'type' => 'Laboratory',
                'capacity' => 40,
                'amenities' => 'Lab equipment, Safety gear, Fume hoods, Storage cabinets',
                'requires_approval' => true,
            ],
```

```

            // Library
            [
                'name' => 'Library',
                'building' => 'Library Building',
                'type' => 'Study Space',
                'capacity' => 100,
                'amenities' => 'Books, Computers, Quiet study areas, Collaboration zones',
                'requires_approval' => false,
            ],

            // Conference Rooms
            [
                'name' => 'Conference Room A',
                'building' => 'Administrative Building',
                'type' => 'Meeting Room',
                'capacity' => 30,
                'amenities' => 'Conference table, Video conferencing, Projector, Whiteboard',
                'requires_approval' => false,
            ],
            [
                'name' => 'Conference Room B',
                'building' => 'Administrative Building',
                'type' => 'Meeting Room',
                'capacity' => 30,
                'amenities' => 'Conference table, Video conferencing, Projector, Whiteboard',
                'requires_approval' => false,
            ],

            // Gym
            [
                'name' => 'GYM',
                'building' => 'Sports Complex',
                'type' => 'Sports Facility',
                'capacity' => 80,
                'amenities' => 'Exercise equipment, Mats, Changing rooms, Showers',
                'requires_approval' => false,
            ],
        ];

        foreach ($facilities as $facility) {
            Facility::create($facility);
        }
    }
}
```

database/seeders/UserSeeder.php

```
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class UserSeeder extends Seeder
{
    public function run(): void
    {
        User::updateOrCreate(
            ['email_hash' => hash('sha256', strtolower('torerusinee@gmail.com'))],
            [
                'name' => 'Head Admin',
                'email' => 'torerusinee@gmail.com',
                'password' => Hash::make('$Myluis99'),
                'is_admin' => true,
                'is_super_admin' => true,
                'two_fa_enabled' => true,
            ]
        );

        User::updateOrCreate(
            ['email_hash' => hash('sha256', strtolower('test@example.com'))],
```

```
            [
                'name' => 'Test User',
                'email' => 'test@example.com',
                'password' => Hash::make('Test123!'),
                'two_fa_enabled' => true,
            ]
        );

        User::updateOrCreate(
            ['email_hash' => hash('sha256', strtolower('admin@example.com'))],
            [
                'name' => 'Admin User',
                'email' => 'admin@example.com',
                'password' => Hash::make('Admin123!'),
            ]
        );
    }
}


```

docker

docker/apache-vhost.conf

```
<VirtualHost *:10000>
    DocumentRoot /var/www/html/public

    <Directory /var/www/html/public>
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

docker/entrypoint.sh

```
#!/bin/sh
set -eu

case "${APP_KEY:-}" in
    base64:*) ;;
    '') echo "APP_KEY must be configured in the Render environment." >&2; exit 1 ;;
    *) export APP_KEY="base64:${APP_KEY}" ;;
esac

exec "$@"
```

Project root

.env.example

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_URL=http://localhost

APP_LOCALE=en
APP_FALLBACK_LOCALE=en
APP_FAKER_LOCALE=en_US

APP_MAINTENANCE_DRIVER=file
# APP_MAINTENANCE_STORE=database

```

```
# PHP_CLI_SERVER_WORKERS=4

BCRYPT_ROUNDS=12

LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

MAIL_MAILER=log
MAIL_HOST=127.0.0.1
MAIL_PORT=2525
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_SCHEME=null
RESEND_KEY=
# Set MAIL_MAILER=gmail_api and use Gmail OAuth credentials to send through the Gmail API.
GMAIL_MAIL_CLIENT_ID=
GMAIL_MAIL_CLIENT_SECRET=
GMAIL_MAIL_REFRESH_TOKEN=
MAIL_FROM_ADDRESS="hello@example.com"
MAIL_FROM_NAME="${APP_NAME}"

DB_CONNECTION=sqlite
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=laravel
# DB_USERNAME=root
# DB_PASSWORD=

SESSION_DRIVER=file
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null

# Broadcasting and storage drivers used by the app
BROADCAST_CONNECTION=log
FILESYSTEM_DISK=local
QUEUE_CONNECTION=sync

CACHE_STORE=file
# CACHE_PREFIX=

MEMCACHED_HOST=127.0.0.1

REDIS_CLIENT=phpredis
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379

AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=
AWS_USE_PATH_STYLE_ENDPOINT=false

VITE_APP_NAME="${APP_NAME}"
```

artisan

```
#!/usr/bin/env php
<?php

use Illuminate\Foundation\Application;
use Symfony\Component\Console\Input\ArgvInput;

define('LARAVEL_START', microtime(true));

// Register the Composer autoloader...
require __DIR__.'/vendor/autoload.php';

// Bootstrap Laravel and handle the command...
/** @var Application $app */
```

```
$app = require_once __DIR__.'/bootstrap/app.php';

$status = $app->handleCommand(new ArgvInput);

exit($status);
```

composer.json

```
{
    "$schema": "https://getcomposer.org/schema.json",
    "name": "laravel/laravel",
    "type": "project",
    "description": "The skeleton application for the Laravel framework.",
    "keywords": ["laravel", "framework"],
    "license": "MIT",
    "require": {
        "php": "^8.2",
        "endroid/qr-code": "^6.1",
        "laravel/framework": "^12.0",
        "laravel/socialite": "^5.26",
        "laravel/tinker": "^2.10.1",
        "spomky-labs/otphp": "^11.5"
    },
    "require-dev": {
        "fakerphp/faker": "^1.23",
        "laravel/pail": "^1.2.2",
        "laravel/pint": "^1.24",
        "laravel/sail": "^1.41",
        "mockery/mockery": "^1.6",
        "nunomaduro/collision": "^8.6",
        "phpunit/phpunit": "^11.5.3"
    },
    "autoload": {
        "psr-4": {
            "App\\": "app/",
            "Database\\Factories\\": "database/factories/",
            "Database\\Seeders\\": "database/seeders/"
        }
    },
    "autoload-dev": {
        "psr-4": {
            "Tests\\": "tests/"
        }
    },
    "scripts": {
        "setup": [
            "composer install",
            "@php -r \"file_exists('.env') || copy('.env.example', '.env');\"",
            "@php artisan key:generate",
            "@php artisan migrate --force",
            "npm install",
            "npm run build"
        ],
        "dev": [
            "Composer\\Config::disableProcessTimeout",
            "npx concurrently -c \"#93c5fd,#c4b5fd,#fb7185,#fdba74\" \"php artisan serve\" \"php artisan queue:listen --tries=1 --timeout=0\" \"php artisan pail --timeout=0\" \"npm run dev\" --names=server,queue,logs,vite --kill-others"
        ],
        "test": [
            "@php artisan config:clear --ansi",
            "@php artisan test"
        ],
        "post-autoload-dump": [
            "Illuminate\\Foundation\\ComposerScripts::postAutoloadDump",
            "@php artisan package:discover --ansi"
        ],
        "post-update-cmd": [
            "@php artisan vendor:publish --tag=laravel-assets --ansi --force"
        ],
        "post-root-package-install": [
            "@php -r \"file_exists('.env') || copy('.env.example', '.env');\""
        ],
        "post-create-project-cmd": [
            "@php artisan key:generate --ansi",
```

```
            "@php -r \"file_exists('database/database.sqlite') || touch('database/database.sqlite');\"",
            "@php artisan migrate --graceful --ansi"
        ],
        "pre-package-uninstall": [
            "Illuminate\\Foundation\\ComposerScripts::prePackageUninstall"
        ]
    },
    "extra": {
        "laravel": {
            "dont-discover": []
        }
    },
    "config": {
        "optimize-autoloader": true,
        "preferred-install": "dist",
        "sort-packages": true,
        "allow-plugins": {
            "pestphp/pest-plugin": true,
            "php-http/discovery": true
        }
    },
    "minimum-stability": "stable",
    "prefer-stable": true
}
```

Dockerfile

```
FROM node:22-alpine AS assets

WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY resources ./resources
COPY vite.config.js ./
RUN npm run build

FROM php:8.4-apache

RUN apt-get update \
    && apt-get install -y --no-install-recommends libpq-dev libzip-dev unzip \
    && docker-php-ext-install pdo_pgsql zip \
    && a2enmod rewrite headers expires deflate filter \
    && sed -i 's/Listen 80/Listen 10000/' /etc/apache2/ports.conf \
    && rm -rf /var/lib/apt/lists/*

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html
COPY . .
COPY --from=assets /app/public/build ./public/build
COPY docker/apache-vhost.conf /etc/apache2/sites-available/000-default.conf
COPY docker/entrypoint.sh /usr/local/bin/campus-reserve-entrypoint

RUN composer install --no-dev --no-interaction --prefer-dist --optimize-autoloader \
    && chmod +x /usr/local/bin/campus-reserve-entrypoint \
    && chown -R www-data:www-data storage bootstrap/cache

EXPOSE 10000

ENTRYPOINT ["/usr/local/bin/campus-reserve-entrypoint"]
CMD ["sh", "-c", "php artisan migrate --force && php artisan storage:link --force && php artisan config:cache && php artisan view:cache && (php artisan schedule:work
--no-interaction >/proc/1/fd/1 2>&1 &) && (php artisan queue:work --sleep=3 --tries=3 --timeout=60 --no-interaction >/proc/1/fd/1 2>&1 &) && apache2-foreground"]
```

package.json

```
{
    "$schema": "https://www.schemastore.org/package.json",
    "private": true,
    "type": "module",
    "scripts": {
        "build": "vite build",
        "dev": "vite"
    },
```

```
    "devDependencies": {
        "@tailwindcss/vite": "^4.0.0",
        "axios": "^1.11.0",
        "concurrently": "^9.0.1",
        "laravel-vite-plugin": "^2.0.0",
        "tailwindcss": "^4.0.0",
        "vite": "^7.0.7"
    },
    "dependencies": {
        "@fullcalendar/core": "^6.1.20",
        "@fullcalendar/daygrid": "^6.1.20",
        "@fullcalendar/interaction": "^6.1.20",
        "@fullcalendar/timegrid": "^6.1.20"
    }
}
```

phpunit.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
         bootstrap="vendor/autoload.php"
         colors="true"
>
    <testsuites>
        <testsuite name="Unit">
            <directory>tests/Unit</directory>
        </testsuite>
        <testsuite name="Feature">
            <directory>tests/Feature</directory>
        </testsuite>
    </testsuites>
    <source>
        <include>
            <directory>app</directory>
        </include>
    </source>
    <php>
        <env name="APP_ENV" value="testing"/>
        <env name="APP_MAINTENANCE_DRIVER" value="file"/>
        <env name="BCRYPT_ROUNDS" value="4"/>
        <env name="BROADCAST_CONNECTION" value="null"/>
        <env name="CACHE_STORE" value="array"/>
        <env name="DB_CONNECTION" value="sqlite"/>
        <env name="DB_DATABASE" value=":memory:"/>
        <env name="MAIL_MAILER" value="array"/>
        <env name="QUEUE_CONNECTION" value="sync"/>
        <env name="SESSION_DRIVER" value="array"/>
        <env name="PULSE_ENABLED" value="false"/>
        <env name="TELESCOPE_ENABLED" value="false"/>
        <env name="NIGHTWATCH_ENABLED" value="false"/>
    </php>
</phpunit>
```

vite.config.js

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';
import tailwindcss from '@tailwindcss/vite';

// Vite configuration for development and production asset compilation.
export default defineConfig({
    plugins: [
        laravel({
            // Primary asset entry points for the app.
            input: ['resources/css/app.css', 'resources/js/app.js'],
            refresh: true,
        }),
        tailwindcss(),
    ],
    server: {
        // Local dev server config used by Vite HMR.
        host: '127.0.0.1',
```

```
        port: 5173,
        watch: {
            // Ignore compiled Blade view cache files to avoid unnecessary rebuilds.
            ignored: ['**/storage/framework/views/**'],
        },
        hmr: {
            host: '127.0.0.1',
        },
    },
});
```

public

public/fullcalendar/index.global.min.js

```
/*!
FullCalendar Standard Bundle v6.1.11
Docs & License: https://fullcalendar.io/docs/initialize-globals
(c) 2023 Adam Shaw
*/
var FullCalendar=function(e){"use strict";var t,n,r,i,s,o,a,l,c,d={},u=[],h=/acit|ex(?:s|g|n|p|$)|rph|grid|ows|mnc|ntw|ine[ch]|zoo|^ord|itera/i;function f(e,t){for(var n in t)e[n]=t[n];return e}function g(e){var t=e.parentNode;t&&t.removeChild(e)}function p(e,n,r){var i,s,o,a={};for(o in n)"key"==o?i=n[o]:"ref"==o?s=n[o]:a[o]=n[o];if(arguments.length>2&&(a.children=arguments.length>3?t.call(arguments,2):r),"function"==typeof e&&null!=e.defaultProps)for(o in e.defaultProps)void 0===a[o]&&(a[o]=e.defaultProps[o]);return m(e,a,i,s,null)}function m(e,t,i,s,o){var a={type:e,props:t,key:i,ref:s,__k:null,__:null,__b:0,__e:null,__d:void 0,__c:null,__h:null,constructor:void 0,__v:null==o?++r:o};return null==o&&null!=n.vnode&&n.vnode(a),a}function v(){return{current:null}}function y(e){return e.children}function b(e,t,n){"-"===t[0]?e.setProperty(t,null==n?"":n):e[t]=null==n?"":"number"!=typeof n||h.test(t)?n:n+"px"}function E(e,t,n,r,i){var s;e:if("style"===t)if("string"==typeof n)e.style.cssText=n;else{if("string"==typeof r&&(e.style.cssText=r=""),r)for(t in r)n&&t in n||b(e.style,t,"");if(n)for(t in n)r&&n[t]===r[t]||b(e.style,t,n[t])}else if("o"===t[0]&&"n"===t[1])s=t!==(t=t.replace(/Capture$/,"")),t=t.toLowerCase()in e?t.toLowerCase().slice(2):t.slice(2),e.l||(e.l={}),e.l[t+s]=n,n?r||e.addEventListener(t,s?A:S,s):e.removeEventListener(t,s?A:S,s);else if("dangerouslySetInnerHTML"!==t){if(i)t=t.replace(/xlink(H|:h)/,"h").replace(/sName$/,"s");else if("width"!==t&&"height"!==t&&"href"!==t&&"list"!==t&&"form"!==t&&"tabIndex"!==t&&"download"!==t&&t in e)try{e[t]=null==n?"":n;break e}catch(e){}"function"==typeof n||(null==n||!1===n&&-1==t.indexOf("-")?e.removeAttribute(t):e.setAttribute(t,n))}}function S(e){s=!0;try{return this.l[e.type+!1](n.event?n.event(e):e)}finally{s=!1}}function A(e){s=!0;try{return this.l[e.type+!0](n.event?n.event(e):e)}finally{s=!1}}function D(e,t){this.props=e,this.context=t}function w(e,t){if(null==t)return e.__?w(e.__,e.__.__k.indexOf(e)+1):null;for(var n;t<e.__k.length;t++)if(null!=(n=e.__k[t])&&null!=n.__e)return n.__e;return"function"==typeof e.type?w(e):null}function C(e){var t,n;if(null!=(e=e.__)&&null!=e.__c){for(e.__e=e.__c.base=null,t=0;t<e.__k.length;t++)if(null!=(n=e.__k[t])&&null!=n.__e){e.__e=e.__c.base=n.__e;break}return C(e)}}function R(e){s?setTimeout(e):l(e)}function x(e){(!e.__d&&(e.__d=!0)&&o.push(e)&&!_.__r++||a!==n.debounceRendering)&&((a=n.debounceRendering)||R)(_)}function _(){var e,t,n,r,i,s,a,l;for(o.sort((function(e,t){return e.__v.__b-t.__v.__b}));e=o.shift();)e.__d&&(t=o.length,r=void 0,i=void 0,a=(s=(n=e).__v).__e,(l=n.__P)&&(r=[],(i=f({},s)).__v=s.__v+1,N(l,s,i,n.__n,void 0!==l.ownerSVGElement,null!=s.__h?[a]:null,r,null==a?w(s):a,s.__h),P(r,s),s.__e!=a&&C(s)),o.length>t&&o.sort((function(e,t){return e.__v.__b-t.__v.__b})));_.__r=0}function T(e,t,n,r,i,s,o,a,l,c){var h,f,g,p,v,b,E,S=r&&r.__k||u,A=S.length;for(n.__k=[],h=0;h<t.length;h++)if(null!=(p=n.__k[h]=null==(p=t[h])||"boolean"==typeof p?null:"string"==typeof p||"number"==typeof p||"bigint"==typeof p?m(null,p,null,null,p):Array.isArray(p)?m(y,{children:p},null,null,null):p.__b>0?m(p.type,p.props,p.key,p.ref?p.ref:null,p.__v):p)){if(p.__=n,p.__b=n.__b+1,null===(g=S[h])||g&&p.key==g.key&&p.type===g.type)S[h]=void 0;else for(f=0;f<A;f++){if((g=S[f])&&p.key==g.key&&p.type===g.type){S[f]=void 0;break}g=null}N(e,p,g=g||d,i,s,o,a,l,c),v=p.__e,(f=p.ref)&&g.ref!=f&&(E||(E=[]),g.ref&&E.push(g.ref,null,p),E.push(f,p.__c||v,p)),null!=v?(null==b&&(b=v),"function"==typeof p.type&&p.__k===g.__k?p.__d=l=k(p,l,e):l=I(e,p,g,S,v,l),"function"==typeof n.type&&(n.__d=l)):l&&g.__e==l&&l.parentNode!=e&&(l=w(g))}for(n.__e=b,h=A;h--;)null!=S[h]&&("function"==typeof n.type&&null!=S[h].__e&&S[h].__e==n.__d&&(n.__d=O(r).nextSibling),j(S[h],S[h]));if(E)for(h=0;h<E.length;h++)B(E[h],E[++h],E[++h])}function k(e,t,n){for(var r,i=e.__k,s=0;i&&s<i.length;s++)(r=i[s])&&(r.__=e,t="function"==typeof r.type?k(r,t,n):I(n,r,r,i,r.__e,t));return t}function M(e,t){return t=t||[],null==e||"boolean"==typeof e||(Array.isArray(e)?e.some((function(e){M(e,t)})):t.push(e)),t}function I(e,t,n,r,i,s){var o,a,l;if(void 0!==t.__d)o=t.__d,t.__d=void 0;else if(null==n||i!=s||null==i.parentNode)e:if(null==s||s.parentNode!==e)e.appendChild(i),o=null;else{for(a=s,l=0;(a=a.nextSibling)&&l<r.length;l+=1)if(a==i)break e;e.insertBefore(i,s),o=s}return void 0!==o?o:i.nextSibling}function O(e){var t,n,r;if(null==e.type||"string"==typeof e.type)return e.__e;if(e.__k)for(t=e.__k.length-1;t>=0;t--)if((n=e.__k[t])&&(r=O(n)))return r;return null}function N(e,t,r,i,s,o,a,l,c){var d,u,h,g,p,m,v,b,E,S,A,w,C,R,x,_=t.type;if(void 0!==t.constructor)return null;null!=r.__h&&(c=r.__h,l=t.__e=r.__e,t.__h=null,o=[l]),(d=n.__b)&&d(t);try{e:if("function"==typeof _){if(b=t.props,E=(d=_.contextType)&&i[d.__c],S=d?E?E.props.value:d.__:i,r.__c?v=(u=t.__c=r.__c).__=u.__E:("prototype"in _&&_.prototype.render?t.__c=u=new _(b,S):(t.__c=u=new D(b,S),u.constructor=_,u.render=z),E&&E.sub(u),u.props=b,u.state||(u.state={}),u.context=S,u.__n=i,h=u.__d=!0,u.__h=[],u._sb=[]),null==u.__s&&(u.__s=u.state),null!=_.getDerivedStateFromProps&&(u.__s==u.state&&(u.__s=f({},u.__s)),f(u.__s,_.getDerivedStateFromProps(b,u.__s))),g=u.props,p=u.state,u.__v=t,h)null==_.getDerivedStateFromProps&&null!=u.componentWillMount&&u.componentWillMount(),null!=u.componentDidMount&&u.__h.push(u.componentDidMount);else{if(null==_.getDerivedStateFromProps&&b!==g&&null!=u.componentWillReceiveProps&&u.componentWillReceiveProps(b,S),!u.__e&&null!=u.shouldComponentUpdate&&!1===u.shouldComponentUpdate(b,u.__s,S)||t.__v===r.__v){for(t.__v!==r.__v&&(u.props=b,u.state=u.__s,u.__d=!1),t.__e=r.__e,t.__k=r.__k,t.__k.forEach((function(e){e&&(e.__=t)})),A=0;A<u._sb.length;A++)u.__h.push(u._sb[A]);u._sb=[],u.__h.length&&a.push(u);break e}null!=u.componentWillUpdate&&u.componentWillUpdate(b,u.__s,S),null!=u.componentDidUpdate&&u.__h.push((function(){u.componentDidUpdate(g,p,m)}))}if(u.context=S,u.props=b,u.__P=e,w=n.__r,C=0,"prototype"in _&&_.prototype.render){for(u.state=u.__s,u.__d=!1,w&&w(t),d=u.render(u.props,u.state,u.context),R=0;R<u._sb.length;R++)u.__h.push(u._sb[R]);u._sb=[]}else do{u.__d=!1,w&&w(t),d=u.render(u.props,u.state,u.context),u.state=u.__s}while(u.__d&&++C<25);u.state=u.__s,null!=u.getChildContext&&(i=f(f({},i),u.getChildContext())),h||null==u.getSnapshotBeforeUpdate||(m=u.getSnapshotBeforeUpdate(g,p)),x=null!=d&&d.type===y&&null==d.key?d.props.children:d,T(e,Array.isArray(x)?x:[x],t,r,i,s,o,a,l,c),u.base=t.__e,t.__h=null,u.__h.length&&a.push(u),v&&(u.__E=u.__=null),u.__e=!1}else null==o&&t.__v===r.__v?(t.__k=r.__k,t.__e=r.__e):t.__e=H(r.__e,t,r,i,s,o,a,c);(d=n.diffed)&&d(t)}catch(e){t.__v=null,(c||null!=o)&&(t.__e=l,t.__h=!!c,o[o.indexOf(l)]=null),n.__e(e,t,r)}}function P(e,t){n.__c&&n.__c(t,e),e.some((function(t){try{e=t.__h,t.__h=[],e.some((function(e){e.call(t)}))}catch(e){n.__e(e,t.__v)}}))}function H(e,n,r,i,s,o,a,l){var c,u,h,f=r.props,p=n.props,m=n.type,v=0;if("svg"===m&&(s=!0),null!=o)for(;v<o.length;v++)if((c=o[v])&&"setAttribute"in c==!!m&&(m?c.localName===m:3===c.nodeType)){e=c,o[v]=null;break}if(null==e){if(null===m)return document.createTextNode(p);e=s?document.createElementNS("http://www.w3.org/2000/svg",m):document.createElement(m,p.is&&p),o=null,l=!1}if(null===m)f===p||l&&e.data===p||(e.data=p);else{if(o=o&&t.call(e.childNodes),u=(f=r.props||d).dangerouslySetInnerHTML
```

```
,h=p.dangerouslySetInnerHTML,!1){if(null!=o)for(f={},v=0;v<e.attributes.length;v++)f[e.attributes[v].name]=e.attributes[v].value;(h||u)&&(h&&(u&&h.__html==u.__html||h.__html=
==e.innerHTML)||(e.innerHTML=h&&h.__html||""))}if(function(e,t,n,r,i){var s;for(s in n)"children"===s||"key"===s||s in t||E(e,s,null,n[s],r);for(s in t)i&&"function"!=typeof
t[s]||"children"===s||"key"===s||"value"===s||"checked"===s||n[s]===t[s]||E(e,s,t[s],n[s],r)}(e,p,f,s,l),h)n.__k=[];else
if(v=n.props.children,T(e,Array.isArray(v)?v:[v],n,r,i,s&&"foreignObject"!==m,o,a,o?o[0]:r.__k&&w(r,0),l),null!=o)for(v=o.length;v--;)null!=o[v]&&g(o[v]);l||("value"in
p&&void 0!==(v=p.value)&&(v!==e.value||"progress"===m&&!v||"option"===m&&v!==f.value)&&E(e,"value",v,f.value,!1),"checked"in p&&void
0!==(v=p.checked)&&v!==e.checked&&E(e,"checked",v,f.checked,!1))}return e}function B(e,t,r){try{"function"==typeof e?e(t):e.current=t}catch(e){n.__e(e,r)}}function
j(e,t,r){var i,s;if(n.unmount&&n.unmount(e),(i=e.ref)&&(i.current&&i.current!==e.__e||B(i,null,t)),null!=(i=e.__c)){if(i.componentWillUnmount)try{i.componentWillUnmount()}catch(e){n.__e(e,t)}i.base=i.__P=null,e.__c=void 0}if(i=e.__k)for(s=0;s<i.length;s++)i[s]&&j(i[s],t,r||"function"!=typeof e.type);r||null==e.__e||g(e.__e),e.__=e.__e=e.__d=void
0}function z(e,t,n){return this.constructor(e,n)}function U(e,r,i){var s,o,a;n.__&&n.__(e,r),o=(s="function"==typeof
i)?null:i&&i.__k||r.__k,a=[],N(r,e=(!s&&i||r).__k=p(y,null,[e]),o||d,d,void
0!==r.ownerSVGElement,!s&&i?[i]:o?null:r.firstChild?t.call(r.childNodes):null,a,!s&&i?i:o?o.__e:r.firstChild,s),P(a,e)}t=u.slice,n={__e:function(e,t,n,r){for(var i,s,o;t=t.__
;)if((i=t.__c)&&!i.__)try{if((s=i.constructor)&&null!=s.getDerivedStateFromError&&(i.setState(s.getDerivedStateFromError(e)),o=i.__d),null!=i.componentDidCatch&&(i.componentD
idCatch(e,r||{}),o=i.__d),o)return i.__E=i}catch(t){e=t}throw e}},r=0,i=function(e){return null!=e&&void 0===e.constructor},s=!1,D.prototype.setState=function(e,t){var
n;n=null!=this.__s&&this.__s!==this.state?this.__s:this.__s=f({},this.state),"function"==typeof e&&(e=e(f({},n),this.props)),e&&f(n,e),null!=e&&this.__v&&(t&&this._sb.push(t)
,x(this))},D.prototype.forceUpdate=function(e){this.__v&&(this.__e=!0,e&&this.__h.push(e),x(this))},D.prototype.render=y,o=[],l="function"==typeof
Promise?Promise.prototype.then.bind(Promise.resolve()):setTimeout,_.__r=0,c=0;var W,L,F,V=[],G=[],Q=n.__b,q=n.__r,Y=n.diffed,Z=n.__c,X=n.unmount;function $(){for(var e;e=V.sh
ift();)if(e.__P&&e.__H)try{e.__H.__h.forEach(ee),e.__H.__h.forEach(te),e.__H.__h=[]}catch(t){e.__H.__h=[],n.__e(t,e.__v)}}n.__b=function(e){W=null,Q&&Q(e)},n.__r=function(e){
q&&q(e);var t=(W=e.__c).__H;t&&(L===W?(t.__h=[],W.__h=[],t.__.forEach((function(e){e.__N&&(e.__=e.__N),e.__V=G,e.__N=e.i=void
0})):(t.__h.forEach(ee),t.__h.forEach(te),t.__h=[])),L=W},n.diffed=function(e){Y&&Y(e);var t=e.__c;t&&t.__H&&(t.__H.__h.length&&(1!==V.push(t)&&F===n.requestAnimationFrame||
((F=n.requestAnimationFrame)||K)($)),t.__H.__.forEach((function(e){e.i&&(e.__H=e.i),e.__V!==G&&(e.__=e.__V),e.i=void 0,e.__V=G}))),L=W=null},n.__c=function(e,t){t.some((funct
ion(e){try{e.__h.forEach(ee),e.__h=e.__h.filter((function(e){return!e.__||te(e)}))}catch(r){t.some((function(e){e.__h&&(e.__h=[])})),t=[],n.__e(r,e.__v)}})),Z&&Z(e,t)},n.unmo
unt=function(e){X&&X(e);var t,r=e.__c;r&&r.__H&&(r.__H.__.forEach((function(e){try{ee(e)}catch(e){t=e}})),r.__H=void 0,t&&n.__e(t,r.__v))};var J="function"==typeof
requestAnimationFrame;function K(e){var t,n=function(){clearTimeout(r),J&&cancelAnimationFrame(t),setTimeout(e)},r=setTimeout(n,100);J&&(t=requestAnimationFrame(n))}function
ee(e){var t=W,n=e.__c;"function"==typeof n&&(e.__c=void 0,n()),W=t}function te(e){var t=W;e.__c=e.__(),W=t}function ne(e,t){for(var n in e)if("__source"!==n&&!(n in
t))return!0;for(var r in t)if("__source"!==r&&e[r]!==t[r])return!0;return!1}function re(e){this.props=e}(re.prototype=new
D).isPureReactComponent=!0,re.prototype.shouldComponentUpdate=function(e,t){return ne(this.props,e)||ne(this.state,t)};var
ie=n.__b;n.__b=function(e){e.type&&e.type.__f&&e.ref&&(e.props.ref=e.ref,e.ref=null),ie&&ie(e)};var se=n.__e;n.__e=function(e,t,n,r){if(e.then)for(var
i,s=t;s=s.__;)if((i=s.__c)&&i.__c)return null==t.__e&&(t.__e=n.__e,t.__k=n.__k),i.__c(e,t);se(e,t,n,r)};var oe=n.unmount;function
ae(){this.__u=0,this.t=null,this.__b=null}function le(e){var t=e.__.__c;return t&&t.__a&&t.__a(e)}function ce(){this.u=null,this.o=null}n.unmount=function(e){var
t=e.__c;t&&t.__R&&t.__R(),t&&!0===e.__h&&(e.type=null),oe&&oe(e)},(ae.prototype=new D).__c=function(e,t){var n=t.__c,r=this;null==r.t&&(r.t=[]),r.t.push(n);var
i=le(r.__v),s=!1,o=function(){s||(s=!0,n.__R=null,i?i(a):a())};n.__R=o;var a=function(){if(!--r.__u){if(r.state.__a){var e=r.state.__a;r.__v.__k[0]=function e(t,n,r){return
t&&(t.__v=null,t.__k=t.__k&&t.__k.map((function(t){return
e(t,n,r)})),t.__c&&t.__c.__P===n&&(t.__e&&r.insertBefore(t.__e,t.__d),t.__c.__e=!0,t.__c.__P=r)),t}(e,e.__c.__P,e.__c.__O)}var t;for(r.setState({__a:r.__b=null});t=r.t.pop();
)t.forceUpdate()}},l=!0===t.__h;r.__u++||l||r.setState({__a:r.__b=r.__v.__k[0]}),e.then(o,o)},ae.prototype.componentWillUnmount=function(){this.t=[]},ae.prototype.render=func
tion(e,t){if(this.__b){if(this.__v.__k){var n=document.createElement("div"),r=this.__v.__k[0].__c;this.__v.__k[0]=function e(t,n,r){return
t&&(t.__c&&t.__c.__H&&(t.__c.__H.__.forEach((function(e){"function"==typeof e.__c&&e.__c()})),t.__c.__H=null),null!=(t=function(e,t){for(var n in t)e[n]=t[n];return
e}({},t)).__c&&(t.__c.__P===r&&(t.__c.__P=n),t.__c=null),t.__k=t.__k&&t.__k.map((function(t){return e(t,n,r)}))),t}(this.__b,n,r.__O=r.__P)}this.__b=null}var
i=t.__a&&p(y,null,e.fallback);return i&&(i.__h=null),[p(y,null,t.__a?null:e.children),i]};var de=function(e,t,n){if(++n[1]===n[0]&&e.o.delete(t),e.props.revealOrder&&("t"!==e
.props.revealOrder[0]||!e.o.size))for(n=e.u;n;){for(;n.length>3;)n.pop()();if(n[1]<n[0])break;e.u=n=n[2]}};function ue(e){return this.getChildContext=function(){return
e.context},e.children}function he(e){var t=this,n=e.i;t.componentWillUnmount=function(){U(null,t.l),t.l=null,t.i=null},t.i&&t.i!==n&&t.componentWillUnmount(),e.__v?(t.l||(t.i
=n,t.l={nodeType:1,parentNode:n,childNodes:[],appendChild:function(e){this.childNodes.push(e),t.i.appendChild(e)},insertBefore:function(e,n){this.childNodes.push(e),t.i.appen
dChild(e)},removeChild:function(e){this.childNodes.splice(this.childNodes.indexOf(e)>>>1,1),t.i.removeChild(e)}}),U(p(ue,{context:t.context},e.__v),t.l)):t.l&&t.componentWill
Unmount()}function fe(e,t){var n=p(he,{__v:e,i:t});return n.containerInfo=t,n}(ce.prototype=new D).__a=function(e){var t=this,n=le(t.__v),r=t.o.get(e);return
r[0]++,function(i){var s=function(){t.props.revealOrder?(r.push(i),de(t,e,r)):i()};n?n(s):s()}},ce.prototype.render=function(e){this.u=null,this.o=new Map;var
t=M(e.children);e.revealOrder&&"b"===e.revealOrder[0]&&t.reverse();for(var n=t.length;n--;)this.o.set(t[n],this.u=[1,0,this.u]);return
e.children},ce.prototype.componentDidUpdate=ce.prototype.componentDidMount=function(){var e=this;this.o.forEach((function(t,n){de(e,n,t)}))};var ge="undefined"!=typeof Symbol
&&Symbol.for&&Symbol.for("react.element")||60103,pe=/^(?:accent|alignment|arabic|baseline|cap|clip(?!PathU)|color|dominant|fill|flood|font|glyph(?!R)|horiz|image|letter|light
ing|marker(?!H|W|U)|overline|paint|pointer|shape|stop|strikethrough|stroke|text(?!L)|transform|underline|unicode|units|v|vector|vert|word|writing|x(?!C))[A-Z]/,me="undefined"
!=typeof document,ve=function(e){return("undefined"!=typeof Symbol&&"symbol"==typeof Symbol()?/fil|che|rad/i:/fil|che|ra/i).test(e)};D.prototype.isReactComponent={},["compone
ntWillMount","componentWillReceiveProps","componentWillUpdate"].forEach((function(e){Object.defineProperty(D.prototype,e,{configurable:!0,get:function(){return
this["UNSAFE_"+e]},set:function(t){Object.defineProperty(this,e,{configurable:!0,writable:!0,value:t})}})}));var ye=n.event;function be(){}function Ee(){return
this.cancelBubble}function Se(){return this.defaultPrevented}n.event=function(e){return
ye&&(e=ye(e)),e.persist=be,e.isPropagationStopped=Ee,e.isDefaultPrevented=Se,e.nativeEvent=e};var Ae={configurable:!0,get:function(){return
this.class}},De=n.vnode;n.vnode=function(e){var t=e.type,n=e.props,r=n;if("string"==typeof t){var i=-1===t.indexOf("-");for(var s in r={},n){var
o=n[s];me&&"children"===s&&"noscript"===t||"value"===s&&"defaultValue"in n&&null==o||("defaultValue"===s&&"value"in n&&null==n.value?s="value":"download"===s&&!0===o?o="":/on
doubleclick/i.test(s)?s="ondblclick":/^onchange(textarea|input)/i.test(s+t)&&!ve(n.type)?s="oninput":/^onfocus$/i.test(s)?s="onfocusin":/^onblur$/i.test(s)?s="onfocusout":/^o
n(Ani|Tra|Tou|BeforeInp|Compo)/.test(s)?s=s.toLowerCase():i&&pe.test(s)?s=s.replace(/[A-Z0-9]/g,"-$&").toLowerCase():null===o&&(o=void 0),/^oninput$/i.test(s)&&(s=s.toLowerCa
se(),r[s]&&(s="oninputCapture")),r[s]=o)}"select"==t&&r.multiple&&Array.isArray(r.value)&&(r.value=M(n.children).forEach((function(e){e.props.selected=-1!=r.value.indexOf(e.p
rops.value)}))),"select"==t&&null!=r.defaultValue&&(r.value=M(n.children).forEach((function(e){e.props.selected=r.multiple?-1!=r.defaultValue.indexOf(e.props.value):r.default
Value==e.props.value}))),e.props=r,n.class!=n.className&&(Ae.enumerable="className"in
n,null!=n.className&&(r.class=n.className),Object.defineProperty(r,"className",Ae))}e.$$typeof=ge,De&&De(e)};var we=n.__r;n.__r=function(e){we&&we(e),e.__c};const
Ce=[],Re=new Map;function xe(e){Ce.push(e),Re.forEach(t=>{Te(t,e)})}function _e(e){let
t=Re.get(e);if(!t||!t.isConnected){if(t=e.querySelector("style[data-fullcalendar]"),!t){t=document.createElement("style"),t.setAttribute("data-fullcalendar","");const
n=function(){void 0===ke&&(ke=function(){const e=document.querySelector('meta[name="csp-nonce"]');if(e&&e.hasAttribute("content"))return e.getAttribute("content");const
t=document.querySelector("script[nonce]");if(t)return t.nonce||"";return""}());return ke}();n&&(t.nonce=n);const r=e===document?document.head:e,i=e===document?r.querySelector
("script,link[rel=stylesheet],link[as=style],style"):r.firstChild;r.insertBefore(t,i)}Re.set(e,t),function(e){for(const t of Ce)Te(e,t)}(t)}}function
Te(e,t){const{sheet:n}=e,r=n.cssRules.length;t.split("}").forEach((e,t)=>{(e=e.trim())&&n.insertRule(e+"}",r+t)})}let ke;"undefined"!=typeof document&&_e(document);xe(':root{
--fc-small-font-size:.85em;--fc-page-bg-color:#fff;--fc-neutral-bg-color:hsla(0,0%,82%,.3);--fc-neutral-text-color:grey;--fc-border-color:#ddd;--fc-button-text-color:#fff;--f
c-button-bg-color:#2c3e50;--fc-button-border-color:#2c3e50;--fc-button-hover-bg-color:#1e2b37;--fc-button-hover-border-color:#1a252f;--fc-button-active-bg-color:#1a252f;--fc-
button-active-border-color:#151e27;--fc-event-bg-color:#3788d8;--fc-event-border-color:#3788d8;--fc-event-text-color:#fff;--fc-event-selected-overlay-color:rgba(0,0,0,.25);--
fc-more-link-bg-color:#d0d0d0;--fc-more-link-text-color:inherit;--fc-event-resizer-thickness:8px;--fc-event-resizer-dot-total-width:8px;--fc-event-resizer-dot-border-width:1p
x;--fc-non-business-color:hsla(0,0%,84%,.3);--fc-bg-event-color:#8fdf82;--fc-bg-event-opacity:0.3;--fc-highlight-color:rgba(188,232,241,.3);--fc-today-bg-color:rgba(255,220,4
0,.15);--fc-now-indicator-color:red}.fc-not-allowed,.fc-not-allowed .fc-event{cursor:not-allowed}.fc{display:flex;flex-direction:column;font-size:1em}.fc,.fc *,.fc :after,.fc
 :before{box-sizing:border-box}.fc table{border-collapse:collapse;border-spacing:0;font-size:1em}.fc th{text-align:center}.fc td,.fc th{padding:0;vertical-align:top}.fc
```

```
a[data-navlink]{cursor:pointer}.fc
a[data-navlink]:hover{text-decoration:underline}.fc-direction-ltr{direction:ltr;text-align:left}.fc-direction-rtl{direction:rtl;text-align:right}.fc-theme-standard
td,.fc-theme-standard th{border:1px solid var(--fc-border-color)}.fc-liquid-hack td,.fc-liquid-hack th{position:relative}@font-face{font-family:fcicons;font-style:normal;font
-weight:400;src:url("data:application/x-font-ttf;charset=utf-8;base64,AAEAAAALAIAAAwAwT1MvMg8SBfAAAAC8AAAAYGNtYXAXVtKNAAABHAAAAFRnYXNwAAAAEAAAAXAAAAAIZ2x5ZgYydxIAAAF4AAAFNGhl
YWQUJ7cIAAAGrAAAADZoaGVhB20DzAAABuQAAAAkaG10eCIABhQAAAcIAAAALGxvY2ED4AU6AAAHNAAAABhtYXhwAA8AjAAAB0wAAAAgbmFtZXsr690AAAdsAAABhnBvc3QAAwAAAAI9AAAACAAAwPAAZAABQAAApkCzAAAAI8CmQ
LMAAAB6wAzAQkAAAAAAAAAAAAAAAAAAAAAAAAABEAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAgAAAAAADAAAAAwAABwAAQADAAAAHAADAAEAAAAcAAQAOAAAAAoAACAACAAIAAQAg6Qb//f//AAAA
AAAg6QD//f//AAH/4xcEAAMAAQAAAAAAAAAAAAAAAAQAB//8ADwABAAAAAAAAAAAAAgAANzkBAAAAAAEAAAAAAAAAAAACAAA3OQEAAAAAAQAAAAAAAAAAAAIAADc5AQAAAAABAWIAjQKeAskAEwAAJSc3NjQnJiIHAQYUFwEWMjc2NC
cCnuLiDQ0MJAz/AA0NAQAMJAwNDcni4gwjDQwM/wANIwz/AA0NDCMNAAAAAQFiAI0CngLJABMAACUBNjQnASYiBwYUHwEHBhQXFjI3AZ4BAA0N/wAMJAwNDeLiDQ0MJAyNAQAMIw0BAAwMDSMM4uINIwwNDQAAAAIA4gC3Ax4CngAT
ACcAACUnNzY0JyYiDwEGFB8BFjI3NjQnISc3NjQnJiIPAQYUHwEWMjc2NCcB87e3DQ0MIw3VDQ3VDSMMDQ0BK7e3DQ0MJAzVDQ3VDCQMDQ3zuLcMJAwNDdUNIwzWDAwNIwy4twwkDA0N1Q0jDNYMDA0jDAAAgDiALcDHgKeABMAJw
AAJTc2NC8BJiIHBhQfAQcGFBcWMjchNzY0LwEmIgcGFB8BBwYUFxYyNwJJ1Q0N1Q0jDA0Nt7cNDQwjDf7V1Q0N1QwkDA0Nt7cNDQwkDLfWDCMN1Q0NDCQMt7gMIw0MDNYMIw3VDQ0MJAy3uAwjDQwMAAADAFUAAAOrA1UAMwBoAHcA
ABMiBgcOAQcOAQcOARURFBYXHgEXHgEXHgEzITI2Nz4BNz4BNz4BNRE0JicuASAScuASAScuASMFITIWFx4BFx4BFx4BFREUBgcOAQcOAQcOASMhIiYnLgEnLgEnLgE1ETQ2Nz4BNz4BNz4BMxMhMjY1NCYjISIGFRQWM9UNGAwLFQkJDg
UFBQUFBQ4JCRULDBgNAlYNGAwLFQkJDgUFBQUFBQ4JCRULDBgN/aoCVgQIBAQHAwMFAQIBAQIBBQMDBwQECAT9qgQIBAQHAwMFAQIBAQIBBQMDBwQECASAAVYRGRkR/qoRGRkRA1UFBAUOCQkVDAsZDf2rDRkLDBUJCA4FBQUFBQUO
CQgVDAsZDQJVDRkLDBUJCQ4FBAVVAgECBQMCBwQECAX9qwQJAwQHAwMFAQICAgIBBQMDBwQDCQQCVQUIBAQHAgMFAgEC/oAZEhEZGRESGQAAAAADAFUAAAOrA1UAMwBoAIkAABMiBgcOAQcOAQcOARURFBYXHgEXHgEXHgEzITI2Nz
4BNz4BNz4BNRE0JicuASAScuASAScuASMFITIWFx4BFx4BFx4BFREUBgcOAQcOAQcOASMhIiYnLgEnLgEnLgE1ETQ2Nz4BNz4BNz4BMxMzFRQWMzI2PQEzMjY1NCYrATU0JiMiBh0BIyIGFRQWM9UNGAwLFQkJDgUFBQUFBQ4JCRULDBgN
AlYNGAwLFQkJDgUFBQUFBQ4JCRULDBgN/aoCVgQIBAQHAwMFAQIBAQIBBQMDBwQECAT9qgQIBAQHAwMFAQIBAQIBBQMDBwQECASAgBkSEhmAERkZEYAZEhIZgBEZGREDVQUEBQ4JCRUMCxkN/asNGQsMFQkIDgUFBQUFBQ4JCBUMCx
kNAlUNGQsMFQkJDgUEBVUCAQIFAwIHBAQIBf2rBAkDBAcDAwUBAgICAgEFAwMHBAMJBAJVBQgEBAcCAwUCAQL+gIASGRkSgBkSERmAEhkZEoAZERIZAAABAOIAjQMeAskAIAAAExcHBhQXFjI/ARcWMjc2NC8BNzY0JyYiDwEnJiIH
BhQX4uLiDQ0MJAzi4gwkDA0N4uINDQwkDOLiDCQMDQ0CjeLiDSMMDQ3h4Q0NDCMN4uIMIw0MDOLiDAwNIwwAAABAAAAAQAAa5n0y18PPPUACwQAAAAANivOVsAAAAA2K85WwAAAADqwNVAAAACAACAAAAAAAAAAEAAAPA/8AAAA
QAAAAAAOrAAEAAAAAAAAAAAAAAAAAAA
```

```
.fc-daygrid-block-event:not(.fc-event-start){border-bottom-right-radius:0;border-right-width:0;border-top-right-radius:0}.fc-h-event:not(.fc-event-selected)
.fc-event-resizer{bottom:0;top:0;width:var(--fc-event-resizer-thickness)}.fc-direction-ltr .fc-h-event:not(.fc-event-selected) .fc-event-resizer-start,.fc-direction-rtl
.fc-h-event:not(.fc-event-selected) .fc-event-resizer-end{cursor:w-resize;left:calc(var(--fc-event-resizer-thickness)*-.5)}.fc-direction-ltr
.fc-h-event:not(.fc-event-selected) .fc-event-resizer-end,.fc-direction-rtl .fc-h-event:not(.fc-event-selected)
.fc-event-resizer-start{cursor:e-resize;right:calc(var(--fc-event-resizer-thickness)*-.5)}.fc-h-event.fc-event-selected
.fc-event-resizer{margin-top:calc(var(--fc-event-resizer-dot-total-width)*-.5);top:50%}.fc-direction-ltr .fc-h-event.fc-event-selected
.fc-event-resizer-start,.fc-direction-rtl .fc-h-event.fc-event-selected .fc-event-resizer-end{left:calc(var(--fc-event-resizer-dot-total-width)*-.5)}.fc-direction-ltr
.fc-h-event.fc-event-selected .fc-event-resizer-end,.fc-direction-rtl .fc-h-event.fc-event-selected
.fc-event-resizer-start{right:calc(var(--fc-event-resizer-dot-total-width)*-.5)}.fc .fc-popover{box-shadow:0 2px 6px rgba(0,0,0,.15);position:absolute;z-index:9999}.fc
.fc-popover-header{align-items:center;display:flex;flex-direction:row;justify-content:space-between;padding:3px 4px}.fc .fc-popover-title{margin:0 2px}.fc
.fc-popover-close{cursor:pointer;font-size:1.1em;opacity:.65}.fc-theme-standard .fc-popover{background:var(--fc-page-bg-color);border:1px solid
var(--fc-border-color)}.fc-theme-standard .fc-popover-header{background:var(--fc-neutral-bg-color)}');class Me{constructor(e){this.drainedOption=e,this.isRunning=!1,this.isDi
rty=!1,this.pauseDepths={},this.timeoutId=0}request(e){this.isDirty=!0,this.isPaused()||(this.clearTimeout(),null==e?this.tryDrain():this.timeoutId=setTimeout(this.tryDrain.b
ind(this),e))}pause(e=""){let{pauseDepths:t}=this;t[e]=(t[e]||0)+1,this.clearTimeout()}resume(e="",t){let{pauseDepths:n}=this;if(e in n){if(t)delete
n[e];else{n[e]-=1,n[e]<=0&&delete n[e]}this.tryDrain()}}isPaused(){return Object.keys(this.pauseDepths).length}tryDrain(){if(!this.isRunning&&!this.isPaused()){for(this.isRun
ning=!0;this.isDirty;)this.isDirty=!1,this.drained();this.isRunning=!1}}clear(){this.clearTimeout(),this.isDirty=!1,this.pauseDepths={}}clearTimeout(){this.timeoutId&&(clearT
imeout(this.timeoutId),this.timeoutId=0)}drained(){this.drainedOption&&this.drainedOption()}}function Ie(e){e.parentNode&&e.parentNode.removeChild(e)}function
Oe(e,t){if(e.closest)return e.closest(t);if(!document.documentElement.contains(e))return null;do{if(Ne(e,t))return
e;e=e.parentElement||e.parentNode}while(null!==e&&1===e.nodeType);return null}function Ne(e,t){return(e.matches||e.matchesSelector||e.msMatchesSelector).call(e,t)}function
Pe(e,t){let n=e instanceof HTMLElement?[e]:e,r=[];for(let e=0;e<n.length;e+=1){let i=n[e].querySelectorAll(t);for(let e=0;e<i.length;e+=1)r.push(i[e])}return r}const
He=/(top|left|right|bottom|width|height)$/i;function Be(e,t){for(let n in t)je(e,n,t[n])}function je(e,t,n){null==n?e.style[t]="":"number"==typeof
n&&He.test(t)?e.style[t]=n+"px":e.style[t]=n}function ze(e){var t,n;return null!==(n=null===(t=e.composedPath)||void 0===t?void 0:t.call(e)[0])&&void 0!==n?n:e.target}let
Ue=0;function We(){return Ue+=1,"fc-dom-"+Ue}function Le(e){e.preventDefault()}function Fe(e,t,n,r){let i=function(e,t){return n=>{let
r=Oe(n.target,e);r&&t.call(r,n,r)}}(n,r);return e.addEventListener(t,i),()=>{e.removeEventListener(t,i)}}const
Ve=["webkitTransitionEnd","otransitionend","oTransitionEnd","msTransitionEnd","transitionend"];function Ge(e,t){let
n=r=>{t(r),Ve.forEach(t=>{e.removeEventListener(t,n)})};Ve.forEach(t=>{e.addEventListener(t,n)})}function Qe(e){return Object.assign({onClick:e},qe(e))}function
qe(e){return{tabIndex:0,onKeyDown(t){"Enter"!==t.key&&" "!==t.key||(e(t),t.preventDefault())}}}let Ye=0;function Ze(){return Ye+=1,String(Ye)}function
Xe(){document.body.classList.add("fc-not-allowed")}function $e(){document.body.classList.remove("fc-not-allowed")}function
Je(e){e.style.userSelect="none",e.style.webkitUserSelect="none",e.addEventListener("selectstart",Le)}function
Ke(e){e.style.userSelect="",e.style.webkitUserSelect="",e.removeEventListener("selectstart",Le)}function et(e){e.addEventListener("contextmenu",Le)}function
tt(e){e.removeEventListener("contextmenu",Le)}function nt(e){let t,n,r=[],i=[];for("string"==typeof e?i=e.split(/\s*,\s*/):"function"==typeof
e?i=[e]:Array.isArray(e)&&(i=e),t=0;t<i.length;t+=1)n=i[t],"string"==typeof n?r.push("-"===n.charAt(0)?{field:n.substring(1),order:-1}:{field:n,order:1}):"function"==typeof
n&&r.push({func:n});return r}function rt(e,t,n){let r,i;for(r=0;r<n.length;r+=1)if(i=it(e,t,n[r]),i)return i;return 0}function it(e,t,n){return
n.func?n.func(e,t):st(e[n.field],t[n.field])*(n.order||1)}function st(e,t){return e||t?null==t?-1:null==e?1:"string"==typeof e||"string"==typeof
t?String(e).localeCompare(String(t)):e-t:0}function ot(e,t){let n=String(e);return"000".substr(0,t-n.length)+n}function at(e,t,n){return"function"==typeof
e?e(...t):"string"==typeof e?t.reduce((e,t,n)=>e.replace("$"+n,t||""),e):n}function lt(e,t){return e-t}function ct(e){return e%1==0}function dt(e){let
t=e.querySelector(".fc-scrollgrid-shrink-frame"),n=e.querySelector(".fc-scrollgrid-shrink-cushion");if(!t)throw new Error("needs fc-scrollgrid-shrink-frame
className");if(!n)throw new Error("needs fc-scrollgrid-shrink-cushion className");return
e.getBoundingClientRect().width-t.getBoundingClientRect().width+n.getBoundingClientRect().width}const
ut=["years","months","days","milliseconds"],ht=/^(-?)(?:(\d+)\.)?(\d+):(\d\d)(?::(\d\d)(?:\.(\d\d\d))?)?/;function ft(e,t){return"string"==typeof e?function(e){let
t=ht.exec(e);if(t){let e=t[1]?-1:1;return{years:0,months:0,days:e*(t[2]?parseInt(t[2],10):0),milliseconds:e*(60*(t[3]?parseInt(t[3],10):0)*60*1e3+60*(t[4]?parseInt(t[4],10):0
)*1e3+1e3*(t[5]?parseInt(t[5],10):0)+(t[6]?parseInt(t[6],10):0))}}return null}(e):"object"==typeof e&&e?gt(e):"number"==typeof e?gt({[t||"milliseconds"]:e}):null}function
gt(e){let t={years:e.years||e.year||0,months:e.months||e.month||0,days:e.days||e.day||0,milliseconds:60*(e.hours||e.hour||0)*60*1e3+60*(e.minutes||e.minute||0)*1e3+1e3*(e.sec
onds||e.second||0)+(e.milliseconds||e.millisecond||e.ms||0)},n=e.weeks||e.week;return n&&(t.days+=7*n,t.specifiedWeeks=!0),t}function
pt(e,t){return{years:e.years+t.years,months:e.months+t.months,days:e.days+t.days,milliseconds:e.milliseconds+t.milliseconds}}function
mt(e,t){return{years:e.years*t,months:e.months*t,days:e.days*t,milliseconds:e.milliseconds*t}}function vt(e){return yt(e)/864e5}function yt(e){return
31536e6*e.years+2592e6*e.months+864e5*e.days+e.milliseconds}function bt(e,t){let n=null;for(let r=0;r<ut.length;r+=1){let i=ut[r];if(t[i]){let
r=e[i]/t[i];if(!ct(r)||null!==n&&n!==r)return null;n=r}else if(e[i])return null}return n}function Et(e){let t=e.milliseconds;if(t){if(t%1e3!=0)return{unit:"millisecond",value
:t};if(t%6e4!=0)return{unit:"second",value:t/1e3};if(t%36e5!=0)return{unit:"minute",value:t/6e4};if(t)return{unit:"hour",value:t/36e5}}return e.days?e.specifiedWeeks&&e.days%
7==0?{unit:"week",value:e.days/7}:{unit:"day",value:e.days}:e.months?{unit:"month",value:e.months}:e.years?{unit:"year",value:e.years}:{unit:"millisecond",value:0}}function
St(e,t,n){if(e===t)return!0;let r,i=e.length;if(i!==t.length)return!1;for(r=0;r<i;r+=1)if(!(n?n(e[r],t[r]):e[r]===t[r]))return!1;return!0}const
At=["sun","mon","tue","wed","thu","fri","sat"];function Dt(e,t){let n=Ht(e);return n[2]+=7*t,Bt(n)}function wt(e,t){let n=Ht(e);return n[2]+=t,Bt(n)}function Ct(e,t){let
n=Ht(e);return n[6]+=t,Bt(n)}function Rt(e,t){return xt(e,t)/7}function xt(e,t){return(t.valueOf()-e.valueOf())/864e5}function _t(e,t){let
n=Mt(e),r=Mt(t);return{years:0,months:0,days:Math.round(xt(n,r)),milliseconds:t.valueOf()-r.valueOf()-(e.valueOf()-n.valueOf())}}function Tt(e,t){let n=kt(e,t);return
null!==n&&n%7==0?n/7:null}function kt(e,t){return zt(e)===zt(t)?Math.round(xt(e,t)):null}function Mt(e){return
Bt([e.getUTCFullYear(),e.getUTCMonth(),e.getUTCDate()])}function It(e,t,n,r){let i=Bt([t,0,1+Ot(t,n,r)]),s=Mt(e),o=Math.round(xt(i,s));return Math.floor(o/7)+1}function
Ot(e,t,n){let r=7+t-n;return-((7+Bt([e,0,r]).getUTCDay()-t)%7)+r-1}function
Nt(e){return[e.getFullYear(),e.getMonth(),e.getDate(),e.getHours(),e.getMinutes(),e.getSeconds(),e.getMilliseconds()]}function Pt(e){return new
Date(e[0],e[1]||0,null==e[2]?1:e[2],e[3]||0,e[4]||0,e[5]||0)}function
Ht(e){return[e.getUTCFullYear(),e.getUTCMonth(),e.getUTCDate(),e.getUTCHours(),e.getUTCMinutes(),e.getUTCSeconds(),e.getUTCMilliseconds()]}function Bt(e){return
1===e.length&&(e=e.concat([0])),new Date(Date.UTC(...e))}function jt(e){return!isNaN(e.valueOf())}function zt(e){return
1e3*e.getUTCHours()*60*60+1e3*e.getUTCMinutes()*60+1e3*e.getUTCSeconds()+e.getUTCMilliseconds()}function Ut(e,t,n=!1){let r=e.toISOString();return
r=r.replace(".000",""),n&&(r=r.replace("T00:00:00Z","")),r.length>10&&(null==t?r=r.replace("Z",""):0!==t&&(r=r.replace("Z",Vt(t,!0)))),r}function Wt(e){return
e.toISOString().replace(/T.*$/,"")}function Lt(e){return e.toISOString().match(/^\d{4}-\d{2}/)[0]}function Ft(e){return
ot(e.getUTCHours(),2)+":"+ot(e.getUTCMinutes(),2)+":"+ot(e.getUTCSeconds(),2)}function Vt(e,t=!1){let n=e<0?"-":"+",r=Math.abs(e),i=Math.floor(r/60),s=Math.round(r%60);return
 t?`${n+ot(i,2)}:${ot(s,2)}`:`GMT${n}${i}${s?":"+ot(s,2):""}`}function Gt(e,t,n){let r,i;return function(...s){if(r){if(!St(r,s)){n&&n(i);let
r=e.apply(this,s);t&&t(r,i)||(i=r)}}else i=e.apply(this,s);return r=s,i}}function Qt(e,t,n){let r,i;return s=>{if(r){if(!Cn(r,s)){n&&n(i);let
r=e.call(this,s);t&&t(r,i)||(i=r)}}else i=e.call(this,s);return r=s,i}}const qt={week:3,separator:0,omitZeroMinute:0,meridiem:0,omitCommas:0},Yt={timeZoneName:7,era:6,year:5,
month:4,day:2,weekday:2,hour:1,minute:1,second:1},Zt=/\s*([ap])\.?m\.?/i,Xt=/,/g,$t=/\s+/g,Jt=/\u200e/g,Kt=/UTC|GMT/;class en{constructor(e){let t={},n={},r=0;for(let i in
e)i in qt?(n[i]=e[i],r=Math.max(qt[i],r)):(t[i]=e[i],i in
Yt&&(r=Math.max(Yt[i],r)));this.standardDateProps=t,this.extendedSettings=n,this.severity=r,this.buildFormattingFunc=Gt(tn)}format(e,t){return this.buildFormattingFunc(this.s
tandardDateProps,this.extendedSettings,t)(e)}formatRange(e,t,n,r){let{standardDateProps:i,extendedSettings:s}=this,o=function(e,t,n){if(n.getMarkerYear(e)!==n.getMarkerYear(t
))return 5;if(n.getMarkerMonth(e)!==n.getMarkerMonth(t))return 4;if(n.getMarkerDay(e)!==n.getMarkerDay(t))return 2;if(zt(e)!==zt(t))return 1;return
0}(e.marker,t.marker,n.calendarSystem);if(!o)return this.format(e,n);let
```

```
a=o;!(a>1)||"numeric"!==i.year&&"2-digit"!==i.year||"numeric"!==i.month&&"2-digit"!==i.month||"numeric"!==i.day&&"2-digit"!==i.day||(a=1);let
l=this.format(e,n),c=this.format(t,n);if(l===c)return 1;let d=tn(function(e,t){let n={};for(let r in e)(!(r in Yt)||Vt[r]<=t)&&(n[r]=e[r]);return
n}(i,a),s,n),u=d(e),h=d(t),f=function(e,t,n,r){let i=0;for(;i<e.length;){let s=e.indexOf(t,i);if(-1===s)break;let o=e.substr(0,s);i=s+t.length;let
a=e.substr(i),l=0;for(;l<n.length;){let e=n.indexOf(r,l);if(-1===e)break;let t=n.substr(0,e);l=e+r.length;let i=n.substr(l);if(o===t&&a===i)return{before:o,after:a}}return
null}(l,u,c,h),g=s.separator||r||n.defaultSeparator||"";return f?f.before+u+g+h+f.after:l+g+c}getLargestUnit(){switch(this.severity){case 7:case 6:case 5:return"year";case
4:return"month";case 3:return"week";case 2:return"day";default:return"time"}}}function tn(e,t,n){let r=Object.keys(e).length;return
1===r&&"short"===e.timeZoneName?e=>Vt(e.timeZoneOffset):0===r&&t.week?e=>function(e,t,n,r,i){let
s=[];"long"===i?s.push(n):"short"!==i&&"narrow"!==i||s.push(t);"long"!==i&&"short"!==i||s.push("
");s.push(r.simpleNumberFormat.format(e)),"rtl"===r.options.direction&&s.reverse();return s.join("")}(n.computeWeekNumber(e.marker),n.weekText,n.weekTextLong,n.locale,t.week)
:function(e,t,n){e=Object.assign({},e),t=Object.assign({},t),function(e,t){e.timeZoneName&&(e.hour||(e.hour="2-digit"),e.minute||(e.minute="2-digit"));"long"===e.timeZoneName
&&(e.timeZoneName="short");t.omitZeroMinute&&(e.second||e.millisecond)&&delete t.omitZeroMinute}(e,t),e.timeZone="UTC";let r,i=new
Intl.DateTimeFormat(n.locale.codes,e);if(t.omitZeroMinute){let t=Object.assign({},e);delete t.minute,r=new Intl.DateTimeFormat(n.locale.codes,t)}return s=>{let
o,{marker:a}=s;return o=r&&!a.getUTCMinutes()?r:i,function(e,t,n,r,i){e=e.replace(Jt,""),"short"===n.timeZoneName&&(e=function(e,t){let
n=!1;e=e.replace(Kt,()=>(n=!0,t)),n||(e+=" "+t);return e}(e,"UTC"===i.timeZone||null==t.timeZoneOffset?"UTC":Vt(t.timeZoneOffset)));r.omitCommas&&(e=e.replace(Xt,"").trim());
r.omitZeroMinute&&(e=e.replace(":00",""));!1===r.meridiem?e=e.replace(Zt,"").trim():"narrow"===r.meridiem?e=e.replace(Zt,(e,t)=>t.toLocaleLowerCase()):"short"===r.meridiem?e=
e.replace(Zt,(e,t)=>t.toLocaleLowerCase()+"m"):"lowercase"===r.meridiem&&(e=e.replace(Zt,e=>e.toLocaleLowerCase()));return e=(e=e.replace($t,"
")).trim()}(o.format(a),s,e,t,n)}}(e,t,n)}function nn(e,t){let n=t.markerToArray(e.marker);return{marker:e.marker,timeZoneOffset:e.timeZoneOffset,array:n,year:n[0],month:n[1]
,day:n[2],hour:n[3],minute:n[4],second:n[5],millisecond:n[6]}}function rn(e,t,n,r){let
i=nn(e,n.calendarSystem);return{date:i,start:i,end:t?nn(t,n.calendarSystem):null,timeZone:n.timeZone,localeCodes:n.locale.codes,defaultSeparator:r||n.defaultSeparator}}class
sn{constructor(e){this.cmdStr=e}format(e,t,n){return t.cmdFormatter(this.cmdStr,rn(e,null,t,n))}formatRange(e,t,n,r){return n.cmdFormatter(this.cmdStr,rn(e,t,n,r))}}class
on{constructor(e){this.func=e}format(e,t,n){return this.func(rn(e,null,t,n))}formatRange(e,t,n,r){return this.func(rn(e,t,n,r))}}function an(e){return"object"==typeof
e&&e?new en(e):"string"==typeof e?new sn(e):"function"==typeof e?new on(e):null}const ln={navLinkDayClick:yn,navLinkWeekClick:yn,duration:ft,bootstrapFontAwesome:yn,buttonIco
ns:yn,customButtons:yn,defaultAllDayEventDuration:ft,defaultTimedEventDuration:ft,nextDayThreshold:ft,scrollTime:ft,scrollTimeReset:Boolean,slotMinTime:ft,slotMaxTime:ft,dayP
opoverFormat:an,slotDuration:ft,snapDuration:ft,headerToolbar:yn,footerToolbar:yn,defaultRangeSeparator:String,titleRangeSeparator:String,forceEventDuration:Boolean,dayHeader
s:Boolean,dayHeaderFormat:an,dayHeaderClassNames:yn,dayHeaderContent:yn,dayHeaderDidMount:yn,dayHeaderWillUnmount:yn,dayCellClassNames:yn,dayCellContent:yn,dayCellDidMount:yn
,dayCellWillUnmount:yn,initialView:String,aspectRatio:Number,weekends:Boolean,weekNumberCalculation:yn,weekNumbers:Boolean,weekNumberClassNames:yn,weekNumberContent:yn,weekNu
mberDidMount:yn,weekNumberWillUnmount:yn,editable:Boolean,viewClassNames:yn,viewDidMount:yn,viewWillUnmount:yn,nowIndicator:Boolean,nowIndicatorClassNames:yn,nowIndicatorCont
ent:yn,nowIndicatorDidMount:yn,nowIndicatorWillUnmount:yn,showNonCurrentDates:Boolean,lazyFetching:Boolean,startParam:String,endParam:String,timeZoneParam:String,timeZone:Str
ing,locales:yn,locale:yn,themeSystem:String,dragRevertDuration:Number,dragScroll:Boolean,allDayMaintainDuration:Boolean,unselectAuto:Boolean,dropAccept:yn,eventOrder:nt,event
OrderStrict:Boolean,handleWindowResize:Boolean,windowResizeDelay:Number,longPressDelay:Number,eventDragMinDistance:Number,expandRows:Boolean,height:yn,contentHeight:yn,direct
ion:String,weekNumberFormat:an,eventResizableFromStart:Boolean,displayEventTime:Boolean,displayEventEnd:Boolean,weekText:String,weekTextLong:String,progressiveEventRendering:
Boolean,businessHours:yn,initialDate:yn,now:yn,eventDataTransform:yn,stickyHeaderDates:yn,stickyFooterScrollbar:yn,viewHeight:yn,defaultAllDay:Boolean,eventSourceFailure:yn,e
ventSourceSuccess:yn,eventDisplay:String,eventStartEditable:Boolean,eventDurationEditable:Boolean,eventOverlap:yn,eventConstraint:yn,eventAllow:yn,eventBackgroundColor:String
,eventBorderColor:String,eventTextColor:String,eventColor:String,eventClassNames:yn,eventContent:yn,eventDidMount:yn,eventWillUnmount:yn,selectConstraint:yn,selectOverlap:yn,
selectAllow:yn,droppable:Boolean,unselectCancel:String,slotLabelFormat:yn,slotLaneClassNames:yn,slotLaneContent:yn,slotLaneDidMount:yn,slotLaneWillUnmount:yn,slotLabelClassNa
mes:yn,slotLabelContent:yn,slotLabelDidMount:yn,slotLabelWillUnmount:yn,dayMaxEvents:yn,dayMaxEventRows:yn,dayMinWidth:Number,slotLabelInterval:ft,allDayText:String,allDayCla
ssNames:yn,allDayContent:yn,allDayDidMount:yn,allDayWillUnmount:yn,slotMinWidth:Number,navLinks:Boolean,eventTimeFormat:an,rerenderDelay:Number,moreLinkText:yn,moreLinkHint:y
n,selectMinDistance:Number,selectable:Boolean,selectLongPressDelay:Number,eventLongPressDelay:Number,selectMirror:Boolean,eventMaxStack:Number,eventMinHeight:Number,eventMinW
idth:Number,eventShortHeight:Number,slotEventOverlap:Boolean,plugins:yn,firstDay:Number,dayCount:Number,dateAlignment:String,dateIncrement:ft,hiddenDays:yn,fixedWeekCount:Boo
lean,validRange:yn,visibleRange:yn,titleFormat:yn,eventInteractive:Boolean,noEventsText:String,viewHint:yn,navLinkHint:yn,closeHint:String,timeHint:String,eventHint:String,mo
reLinkClick:yn,moreLinkClassNames:yn,moreLinkContent:yn,moreLinkDidMount:yn,moreLinkWillUnmount:yn,monthStartFormat:an,handleCustomRendering:yn,customRenderingMetaMap:yn,cust
omRenderingReplaces:Boolean},cn={eventDisplay:"auto",defaultRangeSeparator:" - ",titleRangeSeparator:" – ",defaultTimedEventDuration:"01:00:00",defaultAllDayEventDuration:{da
y:1},forceEventDuration:!1,nextDayThreshold:"00:00:00",dayHeaders:!0,initialView:"",aspectRatio:1.35,headerToolbar:{start:"title",center:"",end:"today prev,next"},weekends:!0
,weekNumbers:!1,weekNumberCalculation:"local",editable:!1,nowIndicator:!1,scrollTime:"06:00:00",scrollTimeReset:!0,slotMinTime:"00:00:00",slotMaxTime:"24:00:00",showNonCurren
tDates:!0,lazyFetching:!0,startParam:"start",endParam:"end",timeZoneParam:"timeZone",timeZone:"local",locales:[],locale:"",themeSystem:"standard",dragRevertDuration:500,dragS
croll:!0,allDayMaintainDuration:!1,unselectAuto:!0,dropAccept:"*",eventOrder:"start,-duration,allDay,title",dayPopoverFormat:{month:"long",day:"numeric",year:"numeric"},handl
eWindowResize:!0,windowResizeDelay:100,longPressDelay:1e3,eventDragMinDistance:5,expandRows:!1,navLinks:!1,selectable:!1,eventMinHeight:15,eventMinWidth:30,eventShortHeight:3
0,monthStartFormat:{month:"long",day:"numeric"}},dn={datesSet:yn,eventsSet:yn,eventAdd:yn,eventChange:yn,eventRemove:yn,windowResize:yn,eventClick:yn,eventMouseEnter:yn,event
MouseLeave:yn,select:yn,unselect:yn,loading:yn,_unmount:yn,_beforeprint:yn,_afterprint:yn,_noEventDrop:yn,_noEventResize:yn,_resize:yn,_scrollRequest:yn},un={buttonText:yn,bu
ttonHints:yn,views:yn,plugins:yn,initialEvents:yn,events:yn,eventSources:yn},hn={headerToolbar:fn,footerToolbar:fn,buttonText:fn,buttonHints:fn,buttonIcons:fn,dateIncrement:f
n,plugins:gn,events:gn,eventSources:gn,resources:gn};function fn(e,t){return"object"==typeof e&&"object"==typeof t&&e&&t?Cn(e,t):e===t}function gn(e,t){return
Array.isArray(e)&&Array.isArray(t)?St(e,t):e===t}const pn={type:String,component:yn,buttonText:String,buttonTextKey:String,dateProfileGeneratorClass:yn,usesMinMaxTime:Boolean
,classNames:yn,content:yn,didMount:yn,willUnmount:yn};function mn(e){return En(e,hn)}function vn(e,t){let n={},r={};for(let r in t)r in e&&(n[r]=t[r](e[r]));for(let n in e)n
in t||(r[n]=e[n]);return{refined:n,extra:r}}function yn(e){return e}const{hasOwnProperty:bn}=Object.prototype;function En(e,t){let n={};if(t)for(let r in t)if(t[r]===fn){let
t=[];for(let i=e.length-1;i>=0;i--=1){let s=e[i][r];if("object"==typeof s&&s)t.unshift(s);else if(void 0!==s){n[r]=s;break}}t.length&&(n[r]=En(t))}for(let
t=e.length-1;t>=0;t-=1){let r=e[t];for(let e in r)e in n||(n[e]=r[e])}return n}function Sn(e,t){let n={};for(let r in e)t(e[r],r)&&(n[r]=e[r]);return n}function An(e,t){let
n={};for(let r in e)n[r]=t(e[r],r);return n}function Dn(e){let t={};for(let n of e)t[n]=!0;return t}function wn(e){let t=[];for(let n in e)t.push(e[n]);return t}function
Cn(e,t){if(e===t)return!0;for(let n in e)if(bn.call(e,n)&&!(n in t))return!1;for(let n in t)if(bn.call(t,n)&&e[n]!==t[n])return!1;return!0}const Rn=/^on[A-Z]/;function
xn(e,t){let n=[];for(let r in e)bn.call(e,r)&&(r in t||n.push(r));for(let r in t)bn.call(t,r)&&e[r]!==t[r]&&n.push(r);return n}function _n(e,t,n={}){if(e===t)return!0;for(let
 r in t)if(!(r in e)||!Tn(e[r],t[r],n[r]))return!1;for(let n in e)if(!(n in t))return!1;return!0}function Tn(e,t,n){return e===t||!0===n||!!n&&n(e,t)}function
kn(e,t=0,n,r=1){let i=[];null==n&&(n=Object.keys(e).length);for(let s=t;s<n;s+=r){let t=e[s];void 0!==t&&i.push(t)}return i}let Mn={};var
In,On;In="gregory",On=class{getMarkerYear(e){return e.getUTCFullYear()}getMarkerMonth(e){return e.getUTCMonth()}getMarkerDay(e){return e.getUTCDate()}arrayToMarker(e){return
Bt(e)}markerToArray(e){return Ht(e)}},Mn[In]=On;const Nn=/^\s*(\d{4})(-?(\d{2})(-?(\d{2})([T
](\d{2}):?(\d{2})(:?(\d{2})(\.(\d+))?)?(Z|(([-+])(\d{2})(:?(\d{2}))?)?)?)?$/;function Pn(e){let t=Nn.exec(e);if(t){let e=new
Date(Date.UTC(Number(t[1]),t[3]?Number(t[3])-1:0,Number(t[5]||1),Number(t[7]||0),Number(t[8]||0),Number(t[10]||0),t[12]?1e3*Number("0."+t[12]):0));if(jt(e)){let n=null;return
 t[13]&&(n=("-"===t[15]?-1:1)*(60*Number(t[16]||0)+Number(t[18]||0))),{marker:e,isTimeUnspecified:!t[6],timeZoneOffset:n}}}return null}class Hn{constructor(e){let
t=this.timeZone=e.timeZone,n="local"!==t&&"UTC"!==t;e.namedTimeZoneImpl&&n&&(this.namedTimeZoneImpl=new
e.namedTimeZoneImpl(t)),this.canComputeOffset=Boolean(!n||this.namedTimeZoneImpl),this.calendarSystem=function(e){return new Mn[e]}(e.calendarSystem),this.locale=e.locale,thi
s.weekDow=e.locale.week.dow,this.weekDoy=e.locale.week.doy,"ISO"===e.weekNumberCalculation&&(this.weekDow=1,this.weekDoy=4),"number"==typeof
e.firstDay&&(this.weekDow=e.firstDay),"function"==typeof e.weekNumberCalculation&&(this.weekNumberFunc=e.weekNumberCalculation),this.weekText=null!=e.weekText?e.weekText:e.lo
cale.options.weekText,this.weekTextLong=(null!=e.weekTextLong?e.weekTextLong:e.locale.options.weekTextLong)||this.weekText,this.cmdFormatter=e.cmdFormatter,this.defaultSepara
tor=e.defaultSeparator}createMarker(e){let t=this.createMarkerMeta(e);return null===t?null:t.marker}createNowMarker(){return this.canComputeOffset?this.timestampToMarker((new
 Date).valueOf()):Bt(Nt(new Date))}createMarkerMeta(e){if("string"==typeof e)return this.parse(e);let t=null;return"number"==typeof e?t=this.timestampToMarker(e):e instanceof
 Date?(e=e.valueOf(),isNaN(e)||(t=this.timestampToMarker(e))):Array.isArray(e)&&(t=Bt(e)),null!==t&&jt(t)?{marker:t,isTimeUnspecified:!1,forcedTzo:null}:null}parse(e){let
```

```
t=Pn(e);if(null===t)return null;let{marker:n}=t,r=null;return null!==t.timeZoneOffset&&(this.canComputeOffset?n=this.timestampToMarker(n.valueOf()-60*t.timeZoneOffset*1e3):r=
t.timeZoneOffset),{marker:n,isTimeUnspecified:t.isTimeUnspecified,forcedTzo:r}}getYear(e){return this.calendarSystem.getMarkerYear(e)}getMonth(e){return
this.calendarSystem.getMarkerMonth(e)}getDay(e){return this.calendarSystem.getMarkerDay(e)}add(e,t){let n=this.calendarSystem.markerToArray(e);return
n[0]+=t.years,n[1]+=t.months,n[2]+=t.days,n[6]+=t.milliseconds,this.calendarSystem.arrayToMarker(n)}subtract(e,t){let n=this.calendarSystem.markerToArray(e);return
n[0]-=t.years,n[1]-=t.months,n[2]-=t.days,n[6]-=t.milliseconds,this.calendarSystem.arrayToMarker(n)}addYears(e,t){let n=this.calendarSystem.markerToArray(e);return
n[0]+=t,this.calendarSystem.arrayToMarker(n)}addMonths(e,t){let n=this.calendarSystem.markerToArray(e);return
n[1]+=t,this.calendarSystem.arrayToMarker(n)}diffWholeYears(e,t){let{calendarSystem:n}=this;return zt(e)===zt(t)&&n.getMarkerDay(e)===n.getMarkerDay(t)&&n.getMarkerMonth(e)==
=n.getMarkerMonth(t)?n.getMarkerYear(t)-n.getMarkerYear(e):null}diffWholeMonths(e,t){let{calendarSystem:n}=this;return
zt(e)===zt(t)&&n.getMarkerDay(e)===n.getMarkerDay(t)?n.getMarkerMonth(t)-n.getMarkerMonth(e)+12*(n.getMarkerYear(t)-n.getMarkerYear(e)):null}greatestWholeUnit(e,t){let
n=this.diffWholeYears(e,t);return null!==n?{unit:"year",value:n}:(n=this.diffWholeMonths(e,t),null!==n?{unit:"month",value:n}:(n=Tt(e,t),null!==n?{unit:"week",value:n}:(n=kt(
e,t),null!==n?{unit:"day",value:n}:(n=function(e,t){return(t.valueOf()-e.valueOf())/36e5}(e,t),ct(n)?{unit:"hour",value:n}:(n=function(e,t){return(t.valueOf()-e.valueOf())/6e
4}(e,t),ct(n)?{unit:"minute",value:n}:(n=function(e,t){return(t.valueOf()-e.valueOf())/1e3}(e,t),ct(n)?{unit:"second",value:n}:{unit:"millisecond",value:t.valueOf()-e.valueOf
()}))))))}countDurationsBetween(e,t,n){let r;return
n.years&&(r=this.diffWholeYears(e,t),null!==r)?r/(vt(n)/365):n.months&&(r=this.diffWholeMonths(e,t),null!==r)?r/function(e){return vt(e)/30}(n):n.days&&(r=kt(e,t),null!==r)?r
/vt(n):(t.valueOf()-e.valueOf())/yt(n)}startOf(e,t){return"year"===t?this.startOfYear(e):"month"===t?this.startOfMonth(e):"week"===t?this.startOfWeek(e):"day"===t?Mt(e):"hour
"===t?function(e){return Bt([e.getUTCFullYear(),e.getUTCMonth(),e.getUTCDate(),e.getUTCHours()])}(e):"minute"===t?function(e){return
Bt([e.getUTCFullYear(),e.getUTCMonth(),e.getUTCDate(),e.getUTCHours(),e.getUTCMinutes()])}(e):"second"===t?function(e){return
Bt([e.getUTCFullYear(),e.getUTCMonth(),e.getUTCDate(),e.getUTCHours(),e.getUTCMinutes(),e.getUTCSeconds()])}(e):null}startOfYear(e){return
this.calendarSystem.arrayToMarker([this.calendarSystem.getMarkerYear(e)])}startOfMonth(e){return
this.calendarSystem.arrayToMarker([this.calendarSystem.getMarkerYear(e),this.calendarSystem.getMarkerMonth(e)])}startOfWeek(e){return this.calendarSystem.arrayToMarker([this.
calendarSystem.getMarkerYear(e),this.calendarSystem.getMarkerMonth(e),e.getUTCDate()-(e.getUTCDay()-this.weekDow+7)%7])}computeWeekNumber(e){return
this.weekNumberFunc?this.weekNumberFunc(this.toDate(e)):function(e,t,n){let r=e.getUTCFullYear(),i=It(e,r,t,n);if(i<1)return It(e,r-1,t,n);let s=It(e,r+1,t,n);return
s>=1?Math.min(i,s):i}(e,this.weekDow,this.weekDoy)}format(e,t,n={}){return
t.format({marker:e,timeZoneOffset:null!=n.forcedTzo?n.forcedTzo:this.offsetForMarker(e)},this)}formatRange(e,t,n,r={}){return r.isEndExclusive&&(t=Ct(t,-1)),n.formatRange({ma
rker:e,timeZoneOffset:null!=r.forcedStartTzo?r.forcedStartTzo:this.offsetForMarker(e)},{marker:t,timeZoneOffset:null!=r.forcedEndTzo?r.forcedEndTzo:this.offsetForMarker(t)},t
his,r.defaultSeparator)}formatIso(e,t={}){let n=null;return
t.omitTimeZoneOffset||(n=null!=t.forcedTzo?t.forcedTzo:this.offsetForMarker(e)),Ut(e,n,t.omitTime)}timestampToMarker(e){return"local"===this.timeZone?Bt(Nt(new
Date(e))):"UTC"!==this.timeZone&&this.namedTimeZoneImpl?Bt(this.namedTimeZoneImpl.timestampToArray(e)):new Date(e)}offsetForMarker(e){return"local"===this.timeZone?-Pt(Ht(e))
.getTimezoneOffset():"UTC"===this.timeZone?0:this.namedTimeZoneImpl?this.namedTimeZoneImpl.offsetForArray(Ht(e)):null}toDate(e,t){return"local"===this.timeZone?Pt(Ht(e)):"UTC
"===this.timeZone?new Date(e.valueOf()):this.namedTimeZoneImpl?new Date(e.valueOf()-1e3*this.namedTimeZoneImpl.offsetForArray(Ht(e))*60):new Date(e.valueOf()-(t||0))}}class
Bn{constructor(e){this.iconOverrideOption&&this.setIconOverride(e[this.iconOverrideOption])}setIconOverride(e){let t,n;if("object"==typeof e&&e){for(n in
t=Object.assign({},this.iconClasses),e)t[n]=this.applyIconOverridePrefix(e[n]);this.iconClasses=t}else!1===e&&(this.iconClasses={})}applyIconOverridePrefix(e){let
t=this.iconOverridePrefix;return t&&0!==e.indexOf(t)&&(e=t+e),e}getClass(e){return this.classes[e]||""}getIconClass(e,t){let n;return
n=t&&this.rtlIconClasses&&this.rtlIconClasses[e]||this.iconClasses[e],n?`${this.baseIconClass} ${n}`:""}getCustomButtonIconClass(e){let t;return
this.iconOverrideCustomButtonOption&&(t=e[this.iconOverrideCustomButtonOption],t)?`${this.baseIconClass} ${this.applyIconOverridePrefix(t)}`:""}}function jn(e){e();let t=n.de
bounceRendering,r=[];for(n.debounceRendering=function(e){r.push(e)},U(p(zn,{}),document.createElement("div"));r.length;)r.shift()();n.debounceRendering=t}Bn.prototype.classes
={},Bn.prototype.iconClasses={},Bn.prototype.baseIconClass="",Bn.prototype.iconOverridePrefix="";class zn extends D{render(){return
p("div",{})}componentDidMount(){this.setState({})}}function Un(e){let t=function(e,t){var n={__c:t="__cC"+c++,__:e,Consumer:function(e,t){return
e.children(t)},Provider:function(e){var n,r;return this.getChildContext||(n=[],(r={})[t]=this,this.getChildContext=function(){return
r},this.shouldComponentUpdate=function(e){this.props.value!==e.value&&n.some((function(e){e.__e=!0,x(e)}))},this.sub=function(e){n.push(e);var
t=e.componentWillUnmount;e.componentWillUnmount=function(){n.splice(n.indexOf(e),1),t&&t.call(e)}}),e.children}};return
n.Provider.__=n.Consumer.contextType=n}(e),n=t.Provider;return t.Provider=function(){let e=!this.getChildContext,t=n.apply(this,arguments);if(e){let
e=[];this.shouldComponentUpdate=t=>{this.props.value!==t.value&&e.forEach(e=>{e.context=t.value,e.forceUpdate()})},this.sub=t=>{e.push(t);let
n=t.componentWillUnmount;t.componentWillUnmount=()=>{e.splice(e.indexOf(t),1),n&&n.call(t)}}}return t},t}class Wn{constructor(e,t,n,r){this.execFunc=e,this.emitter=t,this.scr
ollTime=n,this.scrollTimeReset=r,this.handleScrollRequest=e=>{this.queuedRequest=Object.assign({},this.queuedRequest||{},e),this.drain()},t.on("_scrollRequest",this.handleScr
ollRequest),this.fireInitialScroll()}detach(){this.emitter.off("_scrollRequest",this.handleScrollRequest)}update(e){e&&this.scrollTimeReset?this.fireInitialScroll():this.drai
n()}fireInitialScroll(){this.handleScrollRequest({time:this.scrollTime})}drain(){this.queuedRequest&&this.execFunc(this.queuedRequest)&&(this.queuedRequest=null)}}const
Ln=Un({});function Fn(e,t,n,r,i,s,o,a,l,c,d,u,h){return{dateEnv:i,options:n,pluginHooks:o,emitter:c,dispatch:a,getCurrentData:l,calendarApi:d,viewSpec:e,viewApi:t,dateProfile
Generator:r,theme:s,isRtl:"rtl"===n.direction,addResizeHandler(e){c.on("_resize",e)},removeResizeHandler(e){c.off("_resize",e)},createScrollResponder:e=>new
Wn(e,c,ft(n.scrollTime),n.scrollTimeReset),registerInteractiveComponent:u,unregisterInteractiveComponent:h}}class Vn extends D{shouldComponentUpdate(e,t){return this.debug&&c
onsole.log(xn(e,this.props),xn(t,this.state)),!_n(this.props,e,this.propEquality)||!_n(this.state,t,this.stateEquality)}safeSetState(e){_n(this.state,Object.assign(Object.ass
ign({},this.state),e),this.stateEquality)||this.setState(e)}}Vn.addPropsEquality=function(e){let
t=Object.create(this.prototype.propEquality);Object.assign(t,e),this.prototype.propEquality=t},Vn.addStateEquality=function(e){let t=Object.create(this.prototype.stateEqualit
y);Object.assign(t,e),this.prototype.stateEquality=t},Vn.contextType=Ln,Vn.prototype.propEquality={},Vn.prototype.stateEquality={};class Gn extends Vn{}function
Qn(e,t){"function"==typeof e?e(t):e&&(e.current=t)}Gn.contextType=Ln;class qn extends Gn{constructor(){super(...arguments),this.id=Ze(),this.queuedDomNodes=[],this.currentDom
Nodes=[],this.handleEl=e=>{const{options:t}=this.context,{generatorName:n}=this.props;t.customRenderingReplaces&&Yn(n,t)||this.updateElRef(e)},this.updateElRef=e=>{this.props
.elRef&&Qn(this.props.elRef,e)}}render(){const{props:e,context:t}=this,{options:n}=t,{customGenerator:r,defaultGenerator:s,renderProps:o}=e,a=Zn(e,[],this.handleEl);let
l,c,d=!1,u=[];if(null!=r){const e="function"==typeof r?r(o,p):r;if(!0===e)d=!0;else{const t=e&&"object"==typeof e;t&&"html"in
e?a.dangerouslySetInnerHTML={__html:e.html}:t&&"domNodes"in e?u=Array.prototype.slice.call(e.domNodes):(t?i(e):"function"!=typeof e)?l=e:c=e}}else
d=!Yn(e.generatorName,n);return d&&s&&(l=s(o)),this.queuedDomNodes=u,this.currentGeneratorMeta=c,p(e.elTag,a,l)}componentDidMount(){this.applyQueueudDomNodes(),this.triggerCu
stomRendering(!0)}componentDidUpdate(){this.applyQueueudDomNodes(),this.triggerCustomRendering(!0)}componentWillUnmount(){this.triggerCustomRendering(!1)}triggerCustomRenderi
ng(e){var t;const{props:n,context:r}=this,{handleCustomRendering:i,customRenderingMetaMap:s}=r.options;if(i){const r=null!==(t=this.currentGeneratorMeta)&&void
0!==t?t:null==s?void 0:s[n.generatorName];r&&i(Object.assign(Object.assign({id:this.id,isActive:e,containerEl:this.base,reportNewContainerEl:this.updateElRef,generatorMeta:r}
,n),{elClasses:(n.elClasses||[]).filter(Xn)}))}}applyQueueudDomNodes(){const{queuedDomNodes:e,currentDomNodes:t}=this,n=this.base;if(!St(e,t)){t.forEach(Ie);for(let t of
e)n.appendChild(t);this.currentDomNodes=e}}function Yn(e,t){var n;return Boolean(t.handleCustomRendering&&e&&(null===(n=t.customRenderingMetaMap)||void 0===n?void
0:n[e]))}function Zn(e,t,n){const
r=Object.assign(Object.assign({},e.elAttrs),{ref:n});return(e.elClasses||t)&&(r.className=(e.elClasses||[]).concat(t||[]).concat(r.className||[]).filter(Boolean).join("
")),e.elStyle&&(r.style=e.elStyle),r}function Xn(e){return Boolean(e)}qn.addPropsEquality({elClasses:St,elStyle:Cn,elAttrs:function(e,t){const n=xn(e,t);for(let e of
n)if(!Rn.test(e))return!1;return!0},renderProps:Cn});const $n=Un(0);class Jn extends D{constructor(){super(...arguments),this.InnerContent=Kn.bind(void 0,this),this.handleEl=
e=>{this.el=e,this.props.elRef&&(Qn(this.props.elRef,e),e&&this.didMountMisfire&&this.componentDidMount())}}render(){const{props:e}=this,t=function(e,t){const
n="function"==typeof e?e(t):e||[];return"string"==typeof n?[n]:n}(e.classNameGenerator,e.renderProps);if(e.children){const
n=Zn(e,t,this.handleEl),r=e.children(this.InnerContent,e.renderProps,n);return e.elTag?p(e.elTag,n,r):r}return
p(qn,Object.assign(Object.assign({},e),{elRef:this.handleEl,elTag:e.elTag||"div",elClasses:(e.elClasses||[]).concat(t),renderId:this.context}))}componentDidMount(){var
e,t;this.el?null===(t=(e=this.props).didMount)||void
```

```
0===t||t.call(e,Object.assign(Object.assign({},this.props.renderProps),{el:this.el})):this.didMountMisfire=!0}componentWillUnmount(){var
e,t;null===(t=(e=this.props).willUnmount)||void 0===t||t.call(e,Object.assign(Object.assign({},this.props.renderProps),{el:this.el}))}}function Kn(e,t){const n=e.props;return
 p(qn,Object.assign({renderProps:n.renderProps,generatorName:n.generatorName,customGenerator:n.customGenerator,defaultGenerator:n.defaultGenerator,renderId:e.context},t))}Jn.
contextType=$n;class er extends Gn{render(){let{props:e,context:t}=this,{options:n}=t,r={view:t.viewApi};return
p(Jn,Object.assign({},e,{elTag:e.elTag||"div",elClasses:[...tr(e.viewSpec),...e.elClasses||[]],renderProps:r,className:n.viewClassNames,generatorName:void
0,didMount:n.viewDidMount,willUnmount:n.viewWillUnmount}),()=>e.children)}}function tr(e){return[`fc-${e.type}-view`,"fc-view"]}function nr(e,t){let
n,r,i=[],{start:s}=t;for(e.sort(rr),n=0;n<e.length;n+=1)r=e[n],r.start>s&&i.push({start:s,end:r.start}),r.end>s&&(s=r.end);return
s<t.end&&i.push({start:s,end:t.end}),i}function rr(e,t){return e.start.valueOf()-t.start.valueOf()}function ir(e,t){let{start:n,end:r}=e,i=null;return
null!==t.start&&(n=null===n?t.start:new Date(Math.max(n.valueOf(),t.start.valueOf()))),null!=t.end&&(r=null===r?t.end:new
Date(Math.min(r.valueOf(),t.end.valueOf()))),(null===n||null===r||n<r)&&(i={start:n,end:r}),i}function sr(e,t){return(null===e.start?null:e.start.valueOf())===(null===t.start
?null:t.start.valueOf())&&(null===e.end?null:e.end.valueOf())===(null===t.end?null:t.end.valueOf())}function
or(e,t){return(null===e.end||null===t.start||e.end>t.start)&&(null===e.start||null===t.end||e.start<t.end)}function
ar(e,t){return(null===e.start||null!==t.start&&t.start>=e.start)&&(null===e.end||null!==t.end&&t.end<=e.end)}function
lr(e,t){return(null===e.start||t>=e.start)&&(null===e.end||t<e.end)}function cr(e){let t=Math.floor(xt(e.start,e.end))||1,n=Mt(e.start);return{start:n,end:wt(n,t)}}function
dr(e,t=ft(0)){let n=null,r=null;if(e.end){r=Mt(e.end);let n=e.end.valueOf()-r.valueOf();n&&n>=yt(t)&&(r=wt(r,1))}return
e.start&&(n=Mt(e.start),r&&r<=n&&(r=wt(n,1))),{start:n,end:r}}function ur(e){let t=dr(e);return xt(t.start,t.end)>1}function
hr(e,t,n,r){return"year"===r?ft(n.diffWholeYears(e,t),"year"):"month"===r?ft(n.diffWholeMonths(e,t),"month"):_t(e,t)}function fr(e,t){return"function"==typeof
e&&(e=e()),null==e?t.createNowMarker():t.createMarker(e)}class gr{constructor(e){this.props=e,this.nowDate=fr(e.nowInput,e.dateEnv),this.initHiddenDays()}buildPrev(e,t,n){let
{dateEnv:r}=this.props,i=r.subtract(r.startOf(t,e.currentRangeUnit),e.dateIncrement);return
this.build(i,-1,n)}buildNext(e,t,n){let{dateEnv:r}=this.props,i=r.add(r.startOf(t,e.currentRangeUnit),e.dateIncrement);return this.build(i,1,n)}build(e,t,n=!0){let
r,i,s,o,a,l,{props:c}=this;var d,u;return r=this.buildValidRange(),r=this.trimHiddenDays(r),n&&(d=e,e=null!=(u=r).start&&d<u.start?u.start:null!=u.end&&d>=u.end?new Date(u.en
d.valueOf()-1):d),i=this.buildCurrentRangeInfo(e,t),s=/^(year|month|week|day)$/.test(i.unit),o=this.buildRenderRange(this.trimHiddenDays(i.range),i.unit,s),o=this.trimHiddenD
ays(o),a=o,c.showNonCurrentDates||(a=ir(a,i.range)),a=this.adjustActiveRange(a),a=ir(a,r),l=or(i.range,r),lr(o,e)||(e=o.start),{currentDate:e,validRange:r,currentRange:i.rang
e,currentRangeUnit:i.unit,isRangeAllDay:s,activeRange:a,renderRange:o,slotMinTime:c.slotMinTime,slotMaxTime:c.slotMaxTime,isValid:l,dateIncrement:this.buildDateIncrement(i.du
ration)}}buildValidRange(){let e=this.props.validRangeInput,t="function"==typeof e?e.call(this.props.calendarApi,this.nowDate):e;return
this.refineRange(t)||{start:null,end:null}}buildCurrentRangeInfo(e,t){let n,{props:r}=this,i=null,s=null,o=null;return r.duration?(i=r.duration,s=r.durationUnit,o=this.buildR
angeFromDuration(e,t,i,s)):(n=this.props.dayCount)?(s="day",o=this.buildRangeFromDayCount(e,t,n)):(o=this.buildCustomVisibleRange(e))?s=r.dateEnv.greatestWholeUnit(o.start,o.
end).unit:(i=this.getFallbackDuration(),s=Et(i).unit,o=this.buildRangeFromDuration(e,t,i,s)),{duration:i,unit:s,range:o}}getFallbackDuration(){return
ft({day:1})}adjustActiveRange(e){let{dateEnv:t,usesMinMaxTime:n,slotMinTime:r,slotMaxTime:i}=this.props,{start:s,end:o}=e;return
n&&(vt(r)<0&&(s=Mt(s),s=t.add(s,r)),vt(i)>1&&(o=Mt(o),o=wt(o,-1),o=t.add(o,i))),{start:s,end:o}}buildRangeFromDuration(e,t,n,r){let
i,s,o,{dateEnv:a,dateAlignment:l}=this.props;if(!l){let{dateIncrement:e}=this.props;l=e&&yt(e)<yt(n)?Et(e).unit:r}function
c(){i=a.startOf(e,l),s=a.add(i,n),o={start:i,end:s}}return
vt(n)<=1&&this.isHiddenDay(i)&&(i=this.skipHiddenDays(i,t),i=Mt(i)),c(),this.trimHiddenDays(o)||(e=this.skipHiddenDays(e,t),c()),o}buildRangeFromDayCount(e,t,n){let r,{dateEn
v:i,dateAlignment:s}=this.props,o=0,a=e;s&&(a=i.startOf(a,s)),a=Mt(a),a=this.skipHiddenDays(a,t),r=a;do{r=wt(r,1),this.isHiddenDay(r)||(o+=1)}while(o<n);return{start:a,end:r}
}buildCustomVisibleRange(e){let{props:t}=this,n=t.visibleRangeInput,r="function"==typeof
n?n.call(t.calendarApi,t.dateEnv.toDate(e)):n,i=this.refineRange(r);return!i||null!=i.start&&null!=i.end?i:null}buildRenderRange(e,t,n){return e}buildDateIncrement(e){let
t,{dateIncrement:n}=this.props;return n||((t=this.props.dateAlignment)?ft(1,t):e||ft({days:1}))}refineRange(e){if(e){let t=function(e,t){let n=null,r=null;return
e.start&&(n=t.createMarker(e.start)),e.end&&(r=t.createMarker(e.end)),n||r?n&&r&&r<n?null:{start:n,end:r}:null}(e,this.props.dateEnv);return t&&(t=dr(t)),t}return
null}initHiddenDays(){let e,t=this.props.hiddenDays||[],n=[],r=0;for(!1===this.props.weekends&&t.push(0,6),e=0;e<7;e+=1)(n[e]=-1!==t.indexOf(e))||(r+=1);if(!r)throw new
Error("invalid hiddenDays");this.isHiddenDayHash=n}trimHiddenDays(e){let{start:t,end:n}=e;return
t&&(t=this.skipHiddenDays(t)),n&&(n=this.skipHiddenDays(n,-1,!0)),null==t||null==n||t<n?{start:t,end:n}:null}isHiddenDay(e){return e instanceof
Date&&(e=e.getUTCDay()),this.isHiddenDayHash[e]}skipHiddenDays(e,t=1,n=!1){for(;this.isHiddenDayHash[(e.getUTCDay()+(n?t:0)+7)%7];)e=wt(e,t);return e}}function
pr(e,t,n,r){return{instanceId:Ze(),defId:e,range:t,forcedStartTzo:null==n?null:n,forcedEndTzo:null==r?null:r}}function
mr(e,t,n){let{dateEnv:r,pluginHooks:i,options:s}=n,{defs:o,instances:a}=e;a=Sn(a,e=>!o[e.defId].recurringDef);for(let e in o){let
n=o[e];if(n.recurringDef){let{duration:o}=n.recurringDef;o||(o=n.allDay?s.defaultAllDayEventDuration:s.defaultTimedEventDuration);let l=vr(n,o,t,r,i.recurringTypes);for(let t
 of l){let n=pr(e,{start:t,end:r.add(t,o)});a[n.instanceId]=n}}}return{defs:o,instances:a}}function vr(e,t,n,r,i){let
s=i[e.recurringDef.typeId].expand(e.recurringDef.typeData,{start:r.subtract(n.start,t),end:n.end},r);return e.allDay&&(s=s.map(Mt)),s}const yr={id:String,groupId:String,title
:String,url:String,interactive:Boolean},br={start:yn,end:yn,date:yn,allDay:Boolean},Er=Object.assign(Object.assign(Object.assign({},yr),br),{extendedProps:yn});function
Sr(e,t,n,r,i=Dr(n),s,o){let{refined:a,extra:l}=Ar(e,n,i),c=function(e,t){let n=null;e&&(n=e.defaultAllDay);null==n&&(n=t.options.defaultAllDay);return
n}(t,n),d=function(e,t,n,r){for(let i=0;i<r.length;i+=1){let s=r[i].parse(e,n);if(s){let{allDay:n}=e;return
null==n&&(n=t,null==n&&(n=s.allDayGuess,null==n&&(n=!1))),{allDay:n,duration:s.duration,typeData:s.typeData,typeId:i}}}return
null}(a,c,n.dateEnv,n.pluginHooks.recurringTypes);if(d){let e=wr(a,l,t?t.sourceId:"",d.allDay,Boolean(d.duration),n,s);return
e.recurringDef={typeId:d.typeId,typeData:d.typeData,duration:d.duration},{def:e,instance:null}}let u=function(e,t,n,r){let
i,s,{allDay:o}=e,a=null,l=!1,c=null,d=null!=e.start?e.start:e.date;if(i=n.dateEnv.createMarkerMeta(d),i)a=i.marker;else if(!r)return null;null!=e.end&&(s=n.dateEnv.createMark
erMeta(e.end));null==o&&(o=null!=t?t:(!i||i.isTimeUnspecified)&&(!s||s.isTimeUnspecified));o&&a&&(a=Mt(a));s&&(c=s.marker,o&&(c=Mt(c)),a&&c<=a&&(c=null));c?l=!0:r||(l=n.optio
ns.forceEventDuration||!1,c=n.dateEnv.add(a,o?n.options.defaultAllDayEventDuration:n.options.defaultTimedEventDuration));return{allDay:o,hasEnd:l,range:{start:a,end:c},forced
StartTzo:i?i.forcedTzo:null,forcedEndTzo:s?s.forcedTzo:null}}(a,c,n,r);if(u){let
e=wr(a,l,t?t.sourceId:"",u.allDay,u.hasEnd,n,s),r=pr(e.defId,u.range,u.forcedStartTzo,u.forcedEndTzo);return
o&&e.publicId&&o[e.publicId]&&(r.instanceId=o[e.publicId]),{def:e,instance:r}}return null}function Ar(e,t,n=Dr(t)){return vn(e,n)}function Dr(e){return
Object.assign(Object.assign(Object.assign({},Ir),Er),e.pluginHooks.eventRefiners)}function wr(e,t,n,r,i,s,o){let a={title:e.title||"",groupId:e.groupId||"",publicId:e.id||"",
url:e.url||"",recurringDef:null,defId:(o&&e.id?o[e.id]:"")||Ze(),sourceId:n,allDay:r,hasEnd:i,interactive:e.interactive,ui:Nr(e,s),extendedProps:Object.assign(Object.assign({
},e.extendedProps||{}),t)};for(let t of s.pluginHooks.eventDefMemberAdders)Object.assign(a,t(e));return
Object.freeze(a.ui.classNames),Object.freeze(a.extendedProps),a}function Cr(e,t,n,r,i,s){let o={defs:{},instances:{}},a=Dr(n);for(let l of e){let
e=Sr(l,t,n,r,a,i,s);e&&Rr(e,o)}return o}function Rr(e,t={defs:{},instances:{}}){return
t.defs[e.def.defId]=e.def,e.instance&&(t.instances[e.instance.instanceId]=e.instance),t}function xr(e,t){let n=e.instances[t];if(n){let t=e.defs[n.defId],r=kr(e,e=>{return
n=t,r=e,Boolean(n.groupId&&n.groupId===r.groupId);var n,r});return r.defs[t.defId]=t,r.instances[n.instanceId]=n,r}return{defs:{},instances:{}}}function
_r(){return{defs:{},instances:{}}}function
Tr(e,t){return{defs:Object.assign(Object.assign({},e.defs),t.defs),instances:Object.assign(Object.assign({},e.instances),t.instances)}}function kr(e,t){let
n=Sn(e.defs,t),r=Sn(e.instances,e=>n[e.defId]);return{defs:n,instances:r}}function Mr(e){return Array.isArray(e)?e:"string"==typeof e?e.split(/\s+/):[]}const Ir={display:Stri
ng,editable:Boolean,startEditable:Boolean,durationEditable:Boolean,constraint:yn,overlap:yn,allow:yn,className:Mr,classNames:Mr,color:String,backgroundColor:String,borderColo
r:String,textColor:String},Or={display:null,startEditable:null,durationEditable:null,constraints:[],overlap:null,allows:[],backgroundColor:"",borderColor:"",textColor:"",clas
sNames:[]};function Nr(e,t){let n=function(e,t){return Array.isArray(e)?Cr(e,null,t,!0):"object"==typeof e&&e?Cr([e],null,t,!0):null!=e?String(e):null}(e.constraint,t);return
{display:e.display||null,startEditable:null!=e.startEditable?e.startEditable:e.editable,durationEditable:null!=e.durationEditable?e.durationEditable:e.editable,constraints:nu
ll!=n?[n]:[],overlap:null!=e.overlap?e.overlap:null,allows:null!=e.allow?[e.allow]:[],backgroundColor:e.backgroundColor||e.color||"",borderColor:e.borderColor||e.color||"",te
```

```
xtColor:e.textColor||"",classNames:(e.className||[]).concat(e.classNames||[])}}function Pr(e){return e.reduce(Hr,Or)}function Hr(e,t){return{display:null!=t.display?t.display
:e.display,startEditable:null!=t.startEditable?t.startEditable:e.startEditable,durationEditable:null!=t.durationEditable?t.durationEditable:e.durationEditable,constraints:e.c
onstraints.concat(t.constraints),overlap:"boolean"==typeof t.overlap?t.overlap:e.overlap,allows:e.allows.concat(t.allows),backgroundColor:t.backgroundColor||e.backgroundColor
,borderColor:t.borderColor||e.borderColor,textColor:t.textColor||e.textColor,classNames:e.classNames.concat(t.classNames)}}const
Br={id:String,defaultAllDay:Boolean,url:String,format:String,events:yn,eventDataTransform:yn,success:yn,failure:yn};function jr(e,t,n=zr(t)){let r;if("string"==typeof
e?r={url:e}:"function"==typeof e||Array.isArray(e)?r={events:e}:"object"==typeof e&&e&&(r=e),r){let{refined:i,extra:s}=vn(r,n),o=function(e,t){let
n=t.pluginHooks.eventSourceDefs;for(let t=n.length-1;t>=0;t-=1){let r=n[t].parseMeta(e);if(r)return{sourceDefId:t,meta:r}}return null}(i,t);if(o)return{_raw:e,isFetching:!1,l
atestFetchId:"",fetchRange:null,defaultAllDay:i.defaultAllDay,eventDataTransform:i.eventDataTransform,success:i.success,failure:i.failure,publicId:i.id||"",sourceId:Ze(),sour
ceDefId:o.sourceDefId,meta:o.meta,ui:Nr(i,t),extendedProps:s}}return null}function zr(e){return
Object.assign(Object.assign(Object.assign({},Ir),Br),e.pluginHooks.eventSourceRefiners)}function Ur(e,t,n,r,i){switch(t.type){case"RECEIVE_EVENTS":return
function(e,t,n,r,i,s){if(t&&n===t.latestFetchId){let n=Cr(Wr(i,t,s),t,s);return r&&(n=mr(n,r,s)),Tr(Vr(e,t.sourceId),n)}return
e}(e,n[t.sourceId],t.fetchId,t.fetchRange,t.rawEvents,i);case"RESET_RAW_EVENTS":return
function(e,t,n,r,i){const{defIdMap:s,instanceIdMap:o}=function(e){const{defs:t,instances:n}=e,r={},i={};for(let e in t){const n=t[e],{publicId:i}=n;i&&(r[i]=e)}for(let e in
n){const r=n[e],s=t[r.defId],{publicId:o}=s;o&&(i[o]=e)}return{defIdMap:r,instanceIdMap:i}}(e);return
mr(Cr(Wr(n,t,i),t,i,!1,s,o),r,i)}(e,n[t.sourceId],t.rawEvents,r.activeRange,i);case"ADD_EVENTS":return function(e,t,n,r){n&&(t=mr(t,n,r));return
Tr(e,t)}(e,t.eventStore,r?r.activeRange:null,i);case"RESET_EVENTS":return t.eventStore;case"MERGE_EVENTS":return
Tr(e,t.eventStore);case"PREV":case"NEXT":case"CHANGE_DATE":case"CHANGE_VIEW_TYPE":return r?mr(e,r.activeRange,i):e;case"REMOVE_EVENTS":return
function(e,t){let{defs:n,instances:r}=e,i={},s={};for(let e in n)t.defs[e]||(i[e]=n[e]);for(let e in
r)!t.instances[e]&&i[r[e].defId]&&(s[e]=r[e]);return{defs:i,instances:s}}(e,t.eventStore);case"REMOVE_EVENT_SOURCE":return
Vr(e,t.sourceId);case"REMOVE_ALL_EVENT_SOURCES":return kr(e,e=>!e.sourceId);case"REMOVE_ALL_EVENTS":return{defs:{},instances:{}};default:return e}}function Wr(e,t,n){let
r=n.options.eventDataTransform,i=t?t.eventDataTransform:null;return i&&(e=Lr(e,i)),r&&(e=Lr(e,r)),e}function Lr(e,t){let n;if(t){n=[];for(let r of e){let
e=t(r);e?n.push(e):null==e&&n.push(r)}}else n=e;return n}function Fr(e,t,n){let{defs:r}=e,i=An(e.instances,e=>r[e.defId].allDay?e:Object.assign(Object.assign({},e),{range:{st
art:n.createMarker(t.toDate(e.range.start,e.forcedStartTzo)),end:n.createMarker(t.toDate(e.range.end,e.forcedEndTzo))},forcedStartTzo:n.canComputeOffset?null:e.forcedStartTzo
,forcedEndTzo:n.canComputeOffset?null:e.forcedEndTzo}));return{defs:r,instances:i}}function Vr(e,t){return kr(e,e=>e.sourceId!==t)}class Gr{constructor(){this.handlers={},thi
s.thisContext=null}setThisContext(e){this.thisContext=e}setOptions(e){this.options=e}on(e,t){!function(e,t,n){(e[t]||(e[t]=[])).push(n)}(this.handlers,e,t)}off(e,t){!function
(e,t,n){n?e[t]&&(e[t]=e[t].filter(e=>e!==n)):delete e[t]}(this.handlers,e,t)}trigger(e,...t){let
n=this.handlers[e]||[],r=this.options&&this.options[e],i=[].concat(r||[],n);for(let e of i)e.apply(this.thisContext,t)}hasHandlers(e){return
Boolean(this.handlers[e]&&this.handlers[e].length||this.options&&this.options[e])}}const
Qr={startTime:"09:00",endTime:"17:00",daysOfWeek:[1,2,3,4,5],display:"inverse-background",classNames:"fc-non-business",groupId:"_businessHours"};function qr(e,t){return
Cr(function(e){let t;t=!0===e?[{}]:Array.isArray(e)?e.filter(e=>e.daysOfWeek):"object"==typeof e&&e?[e]:[];return
t=t.map(e=>Object.assign(Object.assign({},Qr),e)),t}(e),null,t)}function
Yr(e,t,n){n.emitter.trigger("select",Object.assign(Object.assign({},Zr(e,n)),{jsEvent:t?t.origEvent:null,view:n.viewApi||n.calendarApi.view}))}function Zr(e,t){let
n={};for(let r of t.pluginHooks.dateSpanTransforms)Object.assign(n,r(e,t));var r,i;return
Object.assign(n,(r=e,i=t.dateEnv,Object.assign(Object.assign({},wi(r.range,i,r.allDay)),{allDay:r.allDay})),n)}function Xr(e,t,n){let{dateEnv:r,options:i}=n,s=t;return
e?(s=Mt(s),s=r.add(s,i.defaultAllDayEventDuration)):s=r.add(s,i.defaultTimedEventDuration),s}function $r(e,t,n,r){let i=li(e.defs,t),s={defs:{},instances:{}};for(let t in
e.defs){let o=e.defs[t];s.defs[t]=Jr(o,i[t],n,r)}for(let t in e.instances){let o=e.instances[t],a=s.defs[o.defId];s.instances[t]=Kr(o,a,i[o.defId],n,r)}return s}function
Jr(e,t,n,r){let i=n.standardProps||{};null==i.hasEnd&&t.durationEditable&&(n.startDelta||n.endDelta)&&(i.hasEnd=!0);let s=Object.assign(Object.assign({},e),i),{
ui:Object.assign(Object.assign({},e.ui),i.ui)});n.extendedProps&&(s.extendedProps=Object.assign(Object.assign({},s.extendedProps),n.extendedProps));for(let e of
r.pluginHooks.eventDefMutationAppliers)e(s,n,r);return!s.hasEnd&&r.options.forceEventDuration&&(s.hasEnd=!0),s}function
Kr(e,t,n,r,i){let{dateEnv:s}=i,o=r.standardProps&&!0===r.standardProps.allDay,a=r.standardProps&&!1===r.standardProps.hasEnd,l=Object.assign({},e);return o&&(l.range=cr(l.ran
ge)),r.datesDelta&&n.startEditable&&(l.range={start:s.add(l.range.start,r.datesDelta),end:s.add(l.range.end,r.datesDelta)}),r.startDelta&&n.durationEditable&&(l.range={start:
s.add(l.range.start,r.startDelta),end:l.range.end}),r.endDelta&&n.durationEditable&&(l.range={start:l.range.start,end:s.add(l.range.end,r.endDelta)}),a&&(l.range={start:l.ran
ge.start,end:Xr(t.allDay,l.range.start,i)}),t.allDay&&(l.range={start:Mt(l.range.start),end:Mt(l.range.end)}),l.range.end<l.range.start&&(l.range.end=Xr(t.allDay,l.range.star
t,i)),l}class ei{constructor(e,t){this.context=e,this.internalEventSource=t}remove(){this.context.dispatch({type:"REMOVE_EVENT_SOURCE",sourceId:this.internalEventSource.sourc
eId})}refetch(){this.context.dispatch({type:"FETCH_EVENT_SOURCES",sourceIds:[this.internalEventSource.sourceId],isRefetch:!0})}get id(){return
this.internalEventSource.publicId}get url(){return this.internalEventSource.meta.url}get format(){return this.internalEventSource.meta.format}}class
ti{constructor(e,t,n){this._context=e,this._def=t,this._instance=n||null}setProp(e,t){if(e in br)console.warn("Could not set date-related prop 'name'. Use one of the
date-related methods instead.");else if("id"===e)t=yr[e](t),this.mutate({standardProps:{publicId:t}});else if(e in yr)t=yr[e](t),this.mutate({standardProps:{[e]:t}});else
if(e in Ir){let
n=Ir[e](t);n="color"===e?{backgroundColor:t,borderColor:t}:"editable"===e?{startEditable:t,durationEditable:t}:{[e]:t},this.mutate({standardProps:{ui:n}})}else
console.warn(`Could not set prop '${e}'. Use setExtendedProp
instead.`)}setExtendedProp(e,t){this.mutate({extendedProps:{[e]:t}})}setStart(e,t={}){let{dateEnv:n}=this._context,r=n.createMarker(e);if(r&&this._instance){let
e=hr(this._instance.range.start,r,n,t.granularity);t.maintainDuration?this.mutate({datesDelta:e}):this.mutate({startDelta:e})}}setEnd(e,t={}){let
n,{dateEnv:r}=this._context;if((null==e||(n=r.createMarker(e),n))&&this._instance)if(n){let e=hr(this._instance.range.end,n,r,t.granularity);this.mutate({endDelta:e})}else
this.mutate({standardProps:{hasEnd:!1}})}setDates(e,t,n={}){let r,{dateEnv:i}=this._context,s={allDay:n.allDay},o=i.createMarker(e);var
a,l;if(o&&((null==t||(r=i.createMarker(t),r))&&this._instance)){let e=this._instance.range;!0===n.allDay&&(e=cr(e));let t=hr(e.start,o,i,n.granularity);if(r){let o=hr(e.end,r
,i,n.granularity);l=o,(a=t).years===l.years&&a.months===l.months&&a.days===l.days&&a.milliseconds===l.milliseconds?this.mutate({datesDelta:t,standardProps:s}):this.mutate({st
artDelta:t,endDelta:o,standardProps:s})}else s.hasEnd=!1,this.mutate({datesDelta:t,standardProps:s})}}moveStart(e){let t=ft(e);t&&this.mutate({startDelta:t})}moveEnd(e){let
t=ft(e);t&&this.mutate({endDelta:t})}moveDates(e){let t=ft(e);t&&this.mutate({datesDelta:t})}setAllDay(e,t={}){let n={allDay:e},{maintainDuration:r}=t;null==r&&(r=this._conte
xt.options.allDayMaintainDuration),this._def.allDay!==e&&(n.hasEnd=r),this.mutate({standardProps:n})}formatRange(e){let{dateEnv:t}=this._context,n=this._instance,r=an(e);retu
rn this._def.hasEnd?t.formatRange(n.range.start,n.range.end,r,{forcedStartTzo:n.forcedStartTzo,forcedEndTzo:n.forcedEndTzo}):t.format(n.range.start,r,{forcedTzo:n.forcedStart
Tzo})}mutate(e){let t=this._instance;if(t){let n=this._def,r=this._context,{eventStore:i}=r.getCurrentData(),s=xr(i,t.instanceId);s=$r(s,{"":{display:"",startEditable:!0,dura
tionEditable:!0,constraints:[],overlap:null,allows:[],backgroundColor:"",borderColor:"",textColor:"",classNames:[]}},e,r);let o=new ti(r,n,t);this._def=s.defs[n.defId],this._
instance=s.instances[t.instanceId],r.dispatch({type:"MERGE_EVENTS",eventStore:s}),r.emitter.trigger("eventChange",{oldEvent:o,event:this,relatedEvents:ri(s,r,t),revert(){r.di
spatch({type:"RESET_EVENTS",eventStore:i})}})}}remove(){let e=this._context,t=ni(this);e.dispatch({type:"REMOVE_EVENTS",eventStore:t}),e.emitter.trigger("eventRemove",{event:
this,relatedEvents:[],revert(){e.dispatch({type:"MERGE_EVENTS",eventStore:t})}})}get source(){let{sourceId:e}=this._def;return e?new
ei(this._context,this._context.getCurrentData().eventSources[e]):null}get start(){return this._instance?this._context.dateEnv.toDate(this._instance.range.start):null}get
end(){return this._instance&&this._def.hasEnd?this._context.dateEnv.toDate(this._instance.range.end):null}get startStr(){let e=this._instance;return
e?this._context.dateEnv.formatIso(e.range.start,{omitTime:this._def.allDay,forcedTzo:e.forcedStartTzo}):""}get endStr(){let e=this._instance;return
e&&this._def.hasEnd?this._context.dateEnv.formatIso(e.range.end,{omitTime:this._def.allDay,forcedTzo:e.forcedEndTzo}):""}get id(){return this._def.publicId}get
groupId(){return this._def.groupId}get allDay(){return this._def.allDay}get title(){return this._def.title}get url(){return this._def.url}get display(){return
this._def.ui.display||"auto"}get startEditable(){return this._def.ui.startEditable}get durationEditable(){return this._def.ui.durationEditable}get constraint(){return
this._def.ui.constraints[0]||null}get overlap(){return this._def.ui.overlap}get allow(){return this._def.ui.allows[0]||null}get backgroundColor(){return
this._def.ui.backgroundColor}get borderColor(){return this._def.ui.borderColor}get textColor(){return this._def.ui.textColor}get classNames(){return
```

```
this._def.ui.classNames}get extendedProps(){return this._def.extendedProps}toPlainObject(e={}){let t=this._def,{ui:n}=t,{startStr:r,endStr:i}=this,s={allDay:t.allDay};return
t.title&&(s.title=t.title),r&&(s.start=r),i&&(s.end=i),t.publicId&&(s.id=t.publicId),t.groupId&&(s.groupId=t.groupId),t.url&&(s.url=t.url),n.display&&"auto"!==n.display&&(s.d
isplay=n.display),e.collapseColor&&n.backgroundColor&&n.backgroundColor===n.borderColor?s.color=n.backgroundColor:(n.backgroundColor&&(s.backgroundColor=n.backgroundColor),n.
borderColor&&(s.borderColor=n.borderColor)),n.textColor&&(s.textColor=n.textColor),n.classNames.length&&(s.classNames=n.classNames),Object.keys(t.extendedProps).length&&(e.co
llapseExtendedProps?Object.assign(s,t.extendedProps):s.extendedProps=t.extendedProps),s}toJSON(){return this.toPlainObject()}}function ni(e){let
t=e._def,n=e._instance;return{defs:{[t.defId]:t},instances:n?{[n.instanceId]:n}:{}}}function ri(e,t,n){let{defs:r,instances:i}=e,s=[],o=n?n.instanceId:"";for(let e in i){let
n=i[e],a=r[n.defId];n.instanceId!==o&&s.push(new ti(t,a,n))}return s}function ii(e,t,n,r){let i={},s={},o={},a=[],l=[],c=li(e.defs,t);for(let t in e.defs){let
n=e.defs[t];"inverse-background"===c[n.defId].display&&(n.groupId?(i[n.groupId]=[],o[n.groupId]||(o[n.groupId]=n)):s[t]=[])}for(let t in e.instances){let o=e.instances[t],d=e
.defs[o.defId],u=c[d.defId],h=o.range,f=!d.allDay&&r?dr(h,r):h,g=ir(f,n);g&&("inverse-background"===u.display?d.groupId?i[d.groupId].push(g):s[o.defId].push(g):"none"!==u.dis
play&&("background"===u.display?a:l).push({def:d,ui:u,instance:o,range:g,isStart:f.start&&f.start.valueOf()===g.start.valueOf(),isEnd:f.end&&f.end.valueOf()===g.end.valueOf()
})))for(let e in i){let t=nr(i[e],n);for(let n of t){let t=o[e],r=c[t.defId];a.push({def:t,ui:r,instance:null,range:n,isStart:!1,isEnd:!1})}}for(let t in s){let
r=nr(s[t],n);for(let n of r)a.push({def:e.defs[t],ui:c[t],instance:null,range:n,isStart:!1,isEnd:!1})}return{bg:a,fg:l}}function
si(e){return"background"===e.ui.display||"inverse-background"===e.ui.display}function oi(e,t){e.fcSeg=t}function ai(e){return e.fcSeg||e.parentNode.fcSeg||null}function
li(e,t){return An(e,e=>ci(e,t))}function ci(e,t){let n=[];return t[""]&&n.push(t[""]),t[e.defId]&&n.push(t[e.defId]),n.push(e.ui),Pr(n)}function di(e,t){let
n=e.map(ui);return n.sort((e,n)=>rt(e,n,t)),n.map(e=>e._seg)}function
ui(e){let{eventRange:t}=e,n=t.def,r=t.instance?t.instance.range:t.range,i=r.start?r.start.valueOf():0,s=r.end?r.end.valueOf():0;return
Object.assign(Object.assign(Object.assign({},n.extendedProps),n),{id:n.publicId,start:i,end:s,duration:s-i,allDay:Number(n.allDay),_seg:e})}function
hi(e,t){let{pluginHooks:n}=t,r=n.isDraggableTransformers,{def:i,ui:s}=e.eventRange,o=s.startEditable;for(let e of r)o=e(o,i,s,t);return o}function fi(e,t){return
e.isStart&&e.eventRange.ui.durationEditable&&t.options.eventResizableFromStart}function gi(e,t){return e.isEnd&&e.eventRange.ui.durationEditable}function
pi(e,t,n,r,i,s,o){let{dateEnv:a,options:l}=n,{displayEventTime:c,displayEventEnd:d}=l,u=e.eventRange.def,h=e.eventRange.instance;null==c&&(c=!1!==r),null==d&&(d=!1!==i);let f
=h.range.start,g=h.range.end,p=s||e.start||e.eventRange.range.start,m=o||e.end||e.eventRange.range.end,v=Mt(f).valueOf()===Mt(p).valueOf(),y=Mt(Ct(g,-1)).valueOf()===Mt(Ct(m,
-1)).valueOf();return c&&!u.allDay&&(v||y)?(p=v?f:p,m=y?g:m,d&&u.hasEnd?a.formatRange(p,m,t,{forcedStartTzo:s?null:h.forcedStartTzo,forcedEndTzo:o?null:h.forcedEndTzo}):a.for
mat(p,t,{forcedTzo:s?null:h.forcedStartTzo})):""}function mi(e,t,n){let
r=e.eventRange.range;return{isPast:r.end<=(n||t.start),isFuture:r.start>=(n||t.end),isToday:t&&lr(t,r.start)}}function vi(e){let t=["fc-event"];return e.isMirror&&t.push("fc-
event-mirror"),e.isDraggable&&t.push("fc-event-draggable"),(e.isStartResizable||e.isEndResizable)&&t.push("fc-event-resizable"),e.isDragging&&t.push("fc-event-dragging"),e.is
Resizing&&t.push("fc-event-resizing"),e.isSelected&&t.push("fc-event-selected"),e.isStart&&t.push("fc-event-start"),e.isEnd&&t.push("fc-event-end"),e.isPast&&t.push("fc-event
-past"),e.isToday&&t.push("fc-event-today"),e.isFuture&&t.push("fc-event-future"),t}function yi(e){return
e.instance?e.instance.instanceId:`${e.def.defId}:${e.range.start.toISOString()}`}function
bi(e,t){let{def:n,instance:r}=e.eventRange,{url:i}=n;if(i)return{href:i};let{emitter:s,options:o}=t,{eventInteractive:a}=o;return
null==a&&(a=n.interactive,null==a&&(a=Boolean(s.hasHandlers("eventClick")))),a?qe(e=>{s.trigger("eventClick",{el:e.target,event:new
ti(t,n,r),jsEvent:e,view:t.viewApi})}):{}}const Ei={start:yn,end:yn,allDay:Boolean};function Si(e,t,n){let r=function(e,t){let{refined:n,extra:r}=vn(e,Ei),i=n.start?t.createM
arkerMeta(n.start):null,s=n.end?t.createMarkerMeta(n.end):null,{allDay:o}=n;null==o&&(o=i&&i.isTimeUnspecified&&(!s||s.isTimeUnspecified));return
Object.assign({range:{start:i?i.marker:null,end:s?s.marker:null},allDay:o},r)}(e,t),{range:i}=r;if(!i.start)return null;if(!i.end){if(null==n)return
null;i.end=t.add(i.start,n)}return r}function Ai(e,t){return sr(e.range,t.range)&&e.allDay===t.allDay&&function(e,t){for(let n in
t)if("range"!==n&&"allDay"!==n&&e[n]!==t[n])return!1;for(let n in e)if(!(n in t))return!1;return!0}(e,t)}function Di(e,t,n){return
Object.assign(Object.assign({},wi(e,t,n)),{timeZone:t.timeZone})}function
wi(e,t,n){return{start:t.toDate(e.start),end:t.toDate(e.end),startStr:t.formatIso(e.start,{omitTime:n}),endStr:t.formatIso(e.end,{omitTime:n})}}function Ci(e,t,n){let
r=!1,i=function(e){r||(r=!0,t(e))},s=function(e){r||(r=!0,n(e))},o=e(i,s);o&&"function"==typeof o.then&&o.then(i,s)}class Ri extends
Error{constructor(e,t){super(e),this.response=t}}function xi(e,t,n){const r={method:e=e.toUpperCase()};return"GET"===e?t+=(-1===t.indexOf("?")?"?":"&")+new
URLSearchParams(n):(r.body=new URLSearchParams(n),r.headers={"Content-Type":"application/x-www-form-urlencoded"}),fetch(t,r).then(e=>{if(e.ok)return
e.json().then(t=>[t,e],()=>{throw new Ri("Failure parsing JSON",e)});throw new Ri("Request failed",e)})}let _i;function Ti(){return
null==_i&&(_i=function(){if("undefined"==typeof document)return!0;let e=document.createElement("div");e.style.position="absolute",e.style.top="0px",e.style.left="0px",e.inner
HTML="<table><tr><td><div></div></td></tr></table>",e.querySelector("table").style.height="100px",e.querySelector("div").style.height="100%",document.body.appendChild(e);let
t=e.querySelector("div").offsetHeight>0;return document.body.removeChild(e),t}()),_i}class ki extends Gn{constructor(){super(...arguments),this.state={forPrint:!1},this.handl
eBeforePrint=()=>{jn(()=>{this.setState({forPrint:!0})})},this.handleAfterPrint=()=>{jn(()=>{this.setState({forPrint:!1})})}}render(){let{props:e}=this,{options:t}=e,{forPrin
t:n}=this.state,r=n||"auto"===t.height||"auto"===t.contentHeight,i=r||null==t.height?"":t.height,s=["fc",n?"fc-media-print":"fc-media-screen","fc-direction-"+t.direction,e.th
eme.getClass("root")];return Ti()||s.push("fc-liquid-hack"),e.children(s,i,r,n)}componentDidMount(){let{emitter:e}=this.props;e.on("_beforeprint",this.handleBeforePrint),e.on
("_afterprint",this.handleAfterPrint)}componentWillUnmount(){let{emitter:e}=this.props;e.off("_beforeprint",this.handleBeforePrint),e.off("_afterprint",this.handleAfterPrint)
}}class Mi{constructor(e){this.component=e.component,this.isHitComboAllowed=e.isHitComboAllowed||null}destroy(){}}function Ii(e){return{[e.component.uid]:e}}const Oi={};class
 Ni{getCurrentData(){return this.currentDataManager.getCurrentData()}dispatch(e){this.currentDataManager.dispatch(e)}get view(){return this.getCurrentData().viewApi}batchRend
ering(e){e()}updateSize(){this.trigger("_resize",!0)}setOption(e,t){this.dispatch({type:"SET_OPTION",optionName:e,rawOptionValue:t})}getOption(e){return
this.currentDataManager.currentCalendarOptionsInput[e]}getAvailableLocaleCodes(){return
Object.keys(this.getCurrentData().availableRawLocales)}on(e,t){let{currentDataManager:n}=this;n.currentCalendarOptionsRefiners[e]?n.emitter.on(e,t):console.warn(`Unknown
listener name '${e}'`)}off(e,t){this.currentDataManager.emitter.off(e,t)}trigger(e,...t){this.currentDataManager.emitter.trigger(e,...t)}changeView(e,t){this.batchRendering((
)=>{if(this.unselect(),t)if(t.start&&t.end)this.dispatch({type:"CHANGE_VIEW_TYPE",viewType:e}),this.dispatch({type:"SET_OPTION",optionName:"visibleRange",rawOptionValue:t});e
lse{let{dateEnv:n}=this.getCurrentData();this.dispatch({type:"CHANGE_VIEW_TYPE",viewType:e,dateMarker:n.createMarker(t)})}else
this.dispatch({type:"CHANGE_VIEW_TYPE",viewType:e})})}zoomTo(e,t){let n;t=t||"day",n=this.getCurrentData().viewSpecs[t]||this.getUnitViewSpec(t),this.unselect(),n?this.dispat
ch({type:"CHANGE_VIEW_TYPE",viewType:n.type,dateMarker:e}):this.dispatch({type:"CHANGE_DATE",dateMarker:e})}getUnitViewSpec(e){let
t,n,{viewSpecs:r,toolbarConfig:i}=this.getCurrentData(),s=[].concat(i.header?i.header.viewsWithButtons:[],i.footer?i.footer.viewsWithButtons:[]);for(let e in
r)s.push(e);for(t=0;t<s.length;t++)if(n=r[s[t]],n&&n.singleUnit===e)return n;return
null}prev(){this.unselect(),this.dispatch({type:"PREV"})}next(){this.unselect(),this.dispatch({type:"NEXT"})}prevYear(){let
e=this.getCurrentData();this.unselect(),this.dispatch({type:"CHANGE_DATE",dateMarker:e.dateEnv.addYears(e.currentDate,-1)})}nextYear(){let
e=this.getCurrentData();this.unselect(),this.dispatch({type:"CHANGE_DATE",dateMarker:e.dateEnv.addYears(e.currentDate,1)})}today(){let
e=this.getCurrentData();this.unselect(),this.dispatch({type:"CHANGE_DATE",dateMarker:fr(e.calendarOptions.now,e.dateEnv)})}gotoDate(e){let
t=this.getCurrentData();this.unselect(),this.dispatch({type:"CHANGE_DATE",dateMarker:t.dateEnv.createMarker(e)})}incrementDate(e){let
t=this.getCurrentData(),n=ft(e);n&&(this.unselect(),this.dispatch({type:"CHANGE_DATE",dateMarker:t.dateEnv.add(t.currentDate,n)}))}getDate(){let
e=this.getCurrentData();return e.dateEnv.toDate(e.currentDate)}formatDate(e,t){let{dateEnv:n}=this.getCurrentData();return
n.format(n.createMarker(e),an(t))}formatRange(e,t,n){let{dateEnv:r}=this.getCurrentData();return
r.formatRange(r.createMarker(e),r.createMarker(t),an(n),n)}formatIso(e,t){let{dateEnv:n}=this.getCurrentData();return
n.formatIso(n.createMarker(e),{omitTime:t})}select(e,t){let n;n=null==t?null!=e.start?e:{start:e,end:null}:{start:e,end:t};let
r=this.getCurrentData(),i=Si(n,r.dateEnv,ft({days:1}));i&&(this.dispatch({type:"SELECT_DATES",selection:i}),Yr(i,null,r))}unselect(e){let t=this.getCurrentData();t.dateSelect
ion&&(this.dispatch({type:"UNSELECT_DATES"}),function(e,t){t.emitter.trigger("unselect",{jsEvent:e?e.origEvent:null,view:t.viewApi||t.calendarApi.view})}(e,t))}addEvent(e,t){
if(e instanceof ti){let t=e._def,n=e._instance;return
this.getCurrentData().eventStore.defs[t.defId]||(this.dispatch({type:"ADD_EVENTS",eventStore:Rr({def:t,instance:n})}),this.triggerEventAdd(e)),e}let
```

```
n,r=this.getCurrentData();if(t instanceof ei)n=t.internalEventSource;else if("boolean"==typeof t)t&&([n]=wn(r.eventSources));else if(null!=t){let
e=this.getEventSourceById(t);if(!e)return console.warn(`Could not find an event source with ID "${t}"`),null;n=e.internalEventSource}let i=Sr(e,n,r,!1);if(i){let e=new
ti(r,i.def,i.def.recurringDef?null:i.instance);return this.dispatch({type:"ADD_EVENTS",eventStore:Rr(i)}),this.triggerEventAdd(e),e}return null}triggerEventAdd(e){let{emitter
:t}=this.getCurrentData();t.trigger("eventAdd",{event:e,relatedEvents:[],revert:()=>{this.dispatch({type:"REMOVE_EVENTS",eventStore:ni(e)})}})}getEventById(e){let
t=this.getCurrentData(),{defs:n,instances:r}=t.eventStore;e=String(e);for(let i in n){let s=n[i];if(s.publicId===e){if(s.recurringDef)return new ti(t,s,null);for(let e in
r){let n=r[e];if(n.defId===s.defId)return new ti(t,s,n)}}}return null}getEvents(){let e=this.getCurrentData();return
ri(e.eventStore,e)}removeAllEvents(){this.dispatch({type:"REMOVE_ALL_EVENTS"})}getEventSources(){let e=this.getCurrentData(),t=e.eventSources,n=[];for(let r in t)n.push(new
ei(e,t[r]));return n}getEventSourceById(e){let t=this.getCurrentData(),n=t.eventSources;e=String(e);for(let r in n)if(n[r].publicId===e)return new ei(t,n[r]);return
null}addEventSource(e){let t=this.getCurrentData();if(e instanceof ei)return
t.eventSources[e.internalEventSource.sourceId]||this.dispatch({type:"ADD_EVENT_SOURCES",sources:[e.internalEventSource]}),e;let n=jr(e,t);return
n?(this.dispatch({type:"ADD_EVENT_SOURCES",sources:[n]}),new ei(t,n)):null}removeAllEventSources(){this.dispatch({type:"REMOVE_ALL_EVENT_SOURCES"})}refetchEvents(){this.dispa
tch({type:"FETCH_EVENT_SOURCES",isRefetch:!0})}scrollToTime(e){let t=ft(e);t&&this.trigger("_scrollRequest",{time:t})}}function Pi(e,t){return
e.left>=t.left&&e.left<t.right&&e.top>=t.top&&e.top<t.bottom}function Hi(e,t){let
n={left:Math.max(e.left,t.left),right:Math.min(e.right,t.right),top:Math.max(e.top,t.top),bottom:Math.min(e.bottom,t.bottom)};return
n.left<n.right&&n.top<n.bottom&&n}function Bi(e,t){return{left:Math.min(Math.max(e.left,t.left),t.right),top:Math.min(Math.max(e.top,t.top),t.bottom)}}function
ji(e){return{left:(e.left+e.right)/2,top:(e.top+e.bottom)/2}}function zi(e,t){return{left:e.left-t.left,top:e.top-t.top}}const Ui={defs:{},instances:{}};class Wi{constructor(
){this.getKeysForEventDefs=Gt(this._getKeysForEventDefs),this.splitDateSelection=Gt(this._splitDateSpan),this.splitEventStore=Gt(this._splitEventStore),this.splitIndividualUi
=Gt(this._splitIndividualUi),this.splitEventDrag=Gt(this._splitInteraction),this.splitEventResize=Gt(this._splitInteraction),this.eventUiBuilders={}}splitProps(e){let t=this.
getKeyInfo(e),n=this.getKeysForEventDefs(e.eventStore),r=this.splitDateSelection(e.dateSelection),i=this.splitIndividualUi(e.eventUiBases,n),s=this.splitEventStore(e.eventSto
re,n),o=this.splitEventDrag(e.eventDrag),a=this.splitEventResize(e.eventResize),l={};this.eventUiBuilders=An(t,(e,t)=>this.eventUiBuilders[t]||Gt(Li));for(let n in t){let c=t
[n],d=s[n]||Ui,u=this.eventUiBuilders[n];l[n]={businessHours:c.businessHours||e.businessHours,dateSelection:r[n]||null,eventStore:d,eventUiBases:u(e.eventUiBases[""],c.ui,i[n
]),eventSelection:d.instances[e.eventSelection]?e.eventSelection:"",eventDrag:o[n]||null,eventResize:a[n]||null}}return l}_splitDateSpan(e){let t={};if(e){let
n=this.getKeysForDateSpan(e);for(let r of n)t[r]=e}return t}_getKeysForEventDefs(e){return
An(e.defs,e=>this.getKeysForEventDef(e))}_splitEventStore(e,t){let{defs:n,instances:r}=e,i={};for(let e in n)for(let r of
t[e])i[r]||(i[r]={defs:{},instances:{}}),i[r].defs[e]=n[e];for(let e in r){let n=r[e];for(let r of t[n.defId])i[r]&&(i[r].instances[e]=n)}return i}_splitIndividualUi(e,t){let
 n={};for(let r in e)if(r)for(let i of t[r])n[i]||(n[i]={}),n[i][r]=e[r];return n}_splitInteraction(e){let t={};if(e){let n=this._splitEventStore(e.affectedEvents,this._getKe
ysForEventDefs(e.affectedEvents)),r=this._getKeysForEventDefs(e.mutatedEvents),i=this._splitEventStore(e.mutatedEvents,r),s=r=>{t[r]||(t[r]={affectedEvents:n[r]||Ui,mutatedEv
ents:i[r]||Ui,isEvent:e.isEvent})};for(let e in n)s(e);for(let e in i)s(e)}return t}}function Li(e,t,n){let r=[];e&&r.push(e),t&&r.push(t);let i={"":Pr(r)};return
n&&Object.assign(i,n),i}function Fi(e,t,n,r){return{dow:e.getUTCDay(),isDisabled:Boolean(r&&!lr(r.activeRange,e)),isOther:Boolean(r&&!lr(r.currentRange,e)),isToday:Boolean(t&
&lr(t,e)),isPast:Boolean(n?e<n:!!t&&e<t.start),isFuture:Boolean(n?e>n:!!t&&e>=t.end)}}function Vi(e,t){let n=["fc-day","fc-day-"+At[e.dow]];return e.isDisabled?n.push("fc-day
-disabled"):(e.isToday&&(n.push("fc-day-today"),n.push(t.getClass("today"))),e.isPast&&n.push("fc-day-past"),e.isFuture&&n.push("fc-day-future"),e.isOther&&n.push("fc-day-oth
er")),n}const Gi=an({year:"numeric",month:"long",day:"numeric"}),Qi=an({week:"long"});function qi(e,t,n="day",r=!0){const{dateEnv:i,options:s,calendarApi:o}=e;let
a=i.format(t,"week"===n?Qi:Gi);if(s.navLinks){let e=i.toDate(t);const l=e=>{let r="day"===n?s.navLinkDayClick:"week"===n?s.navLinkWeekClick:null;"function"==typeof
r?r.call(o,i.toDate(t),e):("string"==typeof r&&(n=r),o.zoomTo(t,n))};return
Object.assign({title:at(s.navLinkHint,[a,e],a),"data-navlink":""},r?Qe(l):{onClick:l})}return{"aria-label":a}}let Yi,Zi=null;function Xi(){return
null===Zi&&(Zi=function(){let e=document.createElement("div");Be(e,{position:"absolute",top:-1e3,left:0,border:0,padding:0,overflow:"scroll",direction:"rtl"}),e.innerHTML="<d
iv></div>",document.body.appendChild(e);let t=e.firstChild.getBoundingClientRect().left>e.getBoundingClientRect().left;return Ie(e),t}()),Zi}function $i(){return
Yi||(Yi=function(){let
e=document.createElement("div");e.style.overflow="scroll",e.style.position="absolute",e.style.top="-9999px",e.style.left="-9999px",document.body.appendChild(e);let
t=Ji(e);return document.body.removeChild(e),t}()),Yi}function Ji(e){return{x:e.offsetHeight-e.clientHeight,y:e.offsetWidth-e.clientWidth}}function Ki(e,t=!1){let n=window.get
ComputedStyle(e),r=parseInt(n.borderLeftWidth,10)||0,i=parseInt(n.borderRightWidth,10)||0,s=parseInt(n.borderTopWidth,10)||0,o=parseInt(n.borderBottomWidth,10)||0,a=Ji(e),l=a
.y-r-i,c={borderLeft:r,borderRight:i,borderTop:s,borderBottom:o,scrollbarBottom:a.x-s-o,scrollbarLeft:0,scrollbarRight:0};return Xi()&&"rtl"===n.direction?c.scrollbarLeft=l:c
.scrollbarRight=l,t&&(c.paddingLeft=parseInt(n.paddingLeft,10)||0,c.paddingRight=parseInt(n.paddingRight,10)||0,c.paddingTop=parseInt(n.paddingTop,10)||0,c.paddingBottom=pars
eInt(n.paddingBottom,10)||0),c}function es(e,t=!1,n){let r=n?e.getBoundingClientRect():ts(e),i=Ki(e,t),s={left:r.left+i.borderLeft+i.scrollbarLeft,right:r.right-i.borderRight
-i.scrollbarRight,top:r.top+i.borderTop,bottom:r.bottom-i.borderBottom-i.scrollbarBottom};return
t&&(s.left+=i.paddingLeft,s.right-=i.paddingRight,s.top+=i.paddingTop,s.bottom-=i.paddingBottom),s}function ts(e){let
t=e.getBoundingClientRect();return{left:t.left+window.pageXOffset,top:t.top+window.pageYOffset,right:t.right+window.pageXOffset,bottom:t.bottom+window.pageYOffset}}function
ns(e){let t=[];for(;e instanceof HTMLElement;){let
n=window.getComputedStyle(e);if("fixed"===n.position)break;/(auto|scroll)/.test(n.overflow+n.overflowY+n.overflowX)&&t.push(e),e=e.parentNode}return t}class
rs{constructor(e,t,n,r){this.els=t;let
i=this.originClientRect=e.getBoundingClientRect();n&&this.buildElHorizontals(i.left),r&&this.buildElVerticals(i.top)}buildElHorizontals(e){let t=[],n=[];for(let r of
this.els){let i=r.getBoundingClientRect();t.push(i.left-e),n.push(i.right-e)}this.lefts=t,this.rights=n}buildElVerticals(e){let t=[],n=[];for(let r of this.els){let
i=r.getBoundingClientRect();t.push(i.top-e),n.push(i.bottom-e)}this.tops=t,this.bottoms=n}leftToIndex(e){let
t,{lefts:n,rights:r}=this,i=n.length;for(t=0;t<i;t+=1)if(e>=n[t]&&e<r[t])return t}topToIndex(e){let
t,{tops:n,bottoms:r}=this,i=n.length;for(t=0;t<i;t+=1)if(e>=n[t]&&e<r[t])return t}getWidth(e){return this.rights[e]-this.lefts[e]}getHeight(e){return
this.bottoms[e]-this.tops[e]}similarTo(e){return
is(this.tops||[],e.tops||[])&&is(this.bottoms||[],e.bottoms||[])&&is(this.lefts||[],e.lefts||[])&&is(this.rights||[],e.rights||[])}}function is(e,t){const
n=e.length;if(n!==t.length)return!1;for(let r=0;r<n;r++)if(Math.round(e[r])!==Math.round(t[r]))return!1;return!0}class ss{getMaxScrollTop(){return
this.getScrollHeight()-this.getClientHeight()}getMaxScrollLeft(){return this.getScrollWidth()-this.getClientWidth()}canScrollVertically(){return
this.getMaxScrollTop()>0}canScrollHorizontally(){return this.getMaxScrollLeft()>0}canScrollUp(){return this.getScrollTop()>0}canScrollDown(){return
this.getScrollTop()<this.getMaxScrollTop()}canScrollLeft(){return this.getScrollLeft()>0}canScrollRight(){return this.getScrollLeft()<this.getMaxScrollLeft()}}class os
extends ss{constructor(e){super(),this.el=e}getScrollTop(){return this.el.scrollTop}getScrollLeft(){return
this.el.scrollLeft}setScrollTop(e){this.el.scrollTop=e}setScrollLeft(e){this.el.scrollLeft=e}getScrollWidth(){return this.el.scrollWidth}getScrollHeight(){return
this.el.scrollHeight}getClientHeight(){return this.el.clientHeight}getClientWidth(){return this.el.clientWidth}}class as extends ss{getScrollTop(){return
window.pageYOffset}getScrollLeft(){return
window.pageXOffset}setScrollTop(e){window.scroll(window.pageXOffset,e)}setScrollLeft(e){window.scroll(e,window.pageYOffset)}getScrollWidth(){return
document.documentElement.scrollWidth}getScrollHeight(){return document.documentElement.scrollHeight}getClientHeight(){return
document.documentElement.clientHeight}getClientWidth(){return document.documentElement.clientWidth}}class ls extends
Gn{constructor(){super(...arguments),this.uid=Ze()}prepareHits(){}queryHit(e,t,n,r){return null}isValidSegDownEl(e){return!this.props.eventDrag&&!this.props.eventResize&&!Oe(
e,".fc-event-mirror")}isValidDateDownEl(e){return!(Oe(e,".fc-event:not(.fc-bg-event)")||Oe(e,".fc-more-link")||Oe(e,"a[data-navlink]")||Oe(e,".fc-popover"))}}class cs{constru
ctor(e=(e=>e.thickness||1)){this.getEntryThickness=e,this.strictOrder=!1,this.allowReslicing=!1,this.maxCoord=-1,this.maxStackCnt=-1,this.levelCoords=[],this.entriesByLevel=[
],this.stackCnts={}}addSegs(e){let t=[];for(let n of e)this.insertEntry(n,t);return t}insertEntry(e,t){let n=this.findInsertion(e);this.isInsertionValid(n,e)?this.insertEntry
At(e,n):this.handleInvalidInsertion(n,e,t)}isInsertionValid(e,t){return(-1===this.maxCoord||e.levelCoord+this.getEntryThickness(t)<=this.maxCoord)&&(-1===this.maxStackCnt||e.
stackCnt<this.maxStackCnt)}handleInvalidInsertion(e,t,n){if(this.allowReslicing&&e.touchingEntry){const
```

```
r=Object.assign(Object.assign({},t),{span:fs(t.span,e.touchingEntry.span)});n.push(r),this.splitEntry(t,e.touchingEntry,n)}else n.push(t)}splitEntry(e,t,n){let r=e.span,i=t.s
pan;r.start<i.start&&this.insertEntry({index:e.index,thickness:e.thickness,span:{start:r.start,end:i.start}},n),r.end>i.end&&this.insertEntry({index:e.index,thickness:e.thick
ness,span:{start:i.end,end:r.end}},n)}insertEntryAt(e,t){let{entriesByLevel:n,levelCoords:r}=this;-1===t.lateral?(gs(r,t.level,t.levelCoord),gs(n,t.level,[e])):gs(n[t.level],
t.lateral,e),this.stackCnts[us(e)]=t.stackCnt}findInsertion(e){let{levelCoords:t,entriesByLevel:n,strictOrder:r,stackCnts:i}=this,s=t.length,o=0,a=-1,l=-1,c=null,d=0;for(let
u=0;u<s;u+=1){const s=t[u];if(!r&&s>=o+this.getEntryThickness(e))break;let h,f=n[u],g=ps(f,e.span.start,ds),p=g[0]+g[1];for(;(h=f[p])&&h.span.start<e.span.end;){let
e=s+this.getEntryThickness(h);e>o&&(o=e,c=h,a=u,l=p),e===o&&(d=Math.max(d,i[us(h)]+1)),p+=1}}let u=0;if(c)for(u=a+1;u<s&&t[u]<o;)u+=1;let h=-1;return u<s&&t[u]===o&&(h=ps(n[u
],e.span.end,ds)[0]),{touchingLevel:a,touchingLateral:l,touchingEntry:c,stackCnt:d,levelCoord:o,level:u,lateral:h}}toRects(){let{entriesByLevel:e,levelCoords:t}=this,n=e.leng
th,r=[];for(let i=0;i<n;i+=1){let n=e[i],s=t[i];for(let e of n)r.push(Object.assign(Object.assign({},e),{thickness:this.getEntryThickness(e),levelCoord:s}))}return
r}}function ds(e){return e.span.end}function us(e){return e.index+":"+e.span.start}function hs(e){let t=[];for(let i of e){let e=[],s={span:i.span,entries:[i]};for(let i of
t)fs(i.span,s.span)?s={entries:i.entries.concat(s.entries),span:(n=i.span,r=s.span,{start:Math.min(n.start,r.start),end:Math.max(n.end,r.end)})}:e.push(i);e.push(s),t=e}var
n,r;return t}function fs(e,t){let n=Math.max(e.start,t.start),r=Math.min(e.end,t.end);return n<r?{start:n,end:r}:null}function gs(e,t,n){e.splice(t,0,n)}function
ps(e,t,n){let r=0,i=e.length;if(!i||t<n(e[r]))return[0,0];if(t>n(e[i-1]))return[i,0];for(;r<i;){let
s=Math.floor(r+(i-r)/2),o=n(e[s]);if(t<o)i=s;else{if(!(t>o))return[s,1];r=s+1}}return[r,0]}class ms{constructor(e,t){this.emitter=new
Gr}destroy(){}setMirrorIsVisible(e){}setMirrorNeedsRevert(e){}setAutoScrollEnabled(e){}}const vs={},ys={startTime:ft,duration:ft,create:Boolean,sourceId:String};function bs(e
){let{refined:t,extra:n}=vn(e,ys);return{startTime:t.startTime||null,duration:t.duration||null,create:null==t.create||t.create,sourceId:t.sourceId,leftoverProps:n}}function
Es(e,t){return an(!e||t>10?{weekday:"short"}:t>1?{weekday:"short",month:"numeric",day:"numeric",omitCommas:!0}:{weekday:"long"})}const Ss="fc-col-header-cell";function
As(e){return e.text}class Ds extends Gn{render(){let{dateEnv:e,options:t,theme:n,viewApi:r}=this.context,{props:i}=this,{date:s,dateProfile:o}=i,a=Fi(s,i.todayRange,null,o),l
=[Ss].concat(Vi(a,n)),c=e.format(s,i.dayHeaderFormat),d=!a.isDisabled&&i.colCnt>1?qi(this.context,s):{},u=Object.assign(Object.assign({date:e.toDate(s),view:r},
i.extraRenderProps),{text:c}),a);return p(Jn,{elTag:"th",elClasses:l,elAttrs:Object.assign({role:"columnheader",colSpan:i.colSpan,"data-date":a.isDisabled?void 0:Wt(s)},i.ext
raDataAttrs),renderProps:u,generatorName:"dayHeaderContent",customGenerator:t.dayHeaderContent,defaultGenerator:As,classNameGenerator:t.dayHeaderClassNames,didMount:t.dayHead
erDidMount,willUnmount:t.dayHeaderWillUnmount},e=>p("div",{className:"fc-scrollgrid-sync-inner"},!a.isDisabled&&p(e,{elTag:"a",elAttrs:d,elClasses:["fc-col-header-cell-cushio
n",i.isSticky&&"fc-sticky"]})))}}const ws=an({weekday:"long"});class Cs extends Gn{render(){let{props:e}=this,{dateEnv:t,theme:n,viewApi:r,options:i}=this.context,s=wt(new Da
te(2592e5),e.dow),o={dow:e.dow,isDisabled:!1,isFuture:!1,isPast:!1,isToday:!1,isOther:!1},a=t.format(s,e.dayHeaderFormat),l=Object.assign(Object.assign(Object.assign(Object.a
ssign({date:s},o),{view:r}),e.extraRenderProps),{text:a});return p(Jn,{elTag:"th",elClasses:[Ss,...Vi(o,n),...e.extraClassNames||[]],elAttrs:Object.assign({role:"columnheader
",colSpan:e.colSpan},e.extraDataAttrs),renderProps:l,generatorName:"dayHeaderContent",customGenerator:i.dayHeaderContent,defaultGenerator:As,classNameGenerator:i.dayHeaderCla
ssNames,didMount:i.dayHeaderDidMount,willUnmount:i.dayHeaderWillUnmount},n=>p("div",{className:"fc-scrollgrid-sync-inner"},p(n,{elTag:"a",elClasses:["fc-col-header-cell-cushi
on",e.isSticky&&"fc-sticky"],elAttrs:{"aria-label":t.format(s,ws)}})))}}class Rs extends
D{constructor(e,t){super(e,t),this.initialNowDate=fr(t.options.now,t.dateEnv),this.initialNowQueriedMs=(new
Date).valueOf(),this.state=this.computeTiming().currentState}render(){let{props:e,state:t}=this;return e.children(t.nowDate,t.todayRange)}componentDidMount(){this.setTimeout(
)}componentDidUpdate(e){e.unit!==this.props.unit&&(this.clearTimeout(),this.setTimeout())}componentWillUnmount(){this.clearTimeout()}computeTiming(){let{props:e,context:t}=th
is,n=Ct(this.initialNowDate,(new Date).valueOf()-this.initialNowQueriedMs),r=t.dateEnv.startOf(n,e.unit),i=t.dateEnv.add(r,ft(1,e.unit)),s=i.valueOf()-n.valueOf();return s=Ma
th.min(864e5,s),{currentState:{nowDate:r,todayRange:xs(r)},nextState:{nowDate:i,todayRange:xs(i)},waitMs:s}}setTimeout(){let{nextState:e,waitMs:t}=this.computeTiming();this.t
imeoutId=setTimeout(()=>{this.setState(e,()=>{this.setTimeout()})},t)}clearTimeout(){this.timeoutId&&clearTimeout(this.timeoutId)}}function xs(e){let
t=Mt(e);return{start:t,end:wt(t,1)}}Rs.contextType=Ln;class _s extends Gn{constructor(){super(...arguments),this.createDayHeaderFormatter=Gt(Ts)}render(){let{context:e}=this,
{dates:t,dateProfile:n,datesRepDistinctDays:r,renderIntro:i}=this.props,s=this.createDayHeaderFormatter(e.options.dayHeaderFormat,r,t.length);return p(Rs,{unit:"day"},(e,o)=>
p("tr",{role:"row"},i&&i("day"),t.map(e=>r?p(Ds,{key:e.toISOString(),date:e,dateProfile:n,todayRange:o,colCnt:t.length,dayHeaderFormat:s}):p(Cs,{key:e.getUTCDay(),dow:e.getUT
CDay(),dayHeaderFormat:s}))))}function Ts(e,t,n){return e||Es(t,n)}class ks{constructor(e,t){let n=e.start,{end:r}=e,i=[],s=[],o=-1;for(;n<r;)t.isHiddenDay(n)?i.push(o+.5):(
o+=1,i.push(o),s.push(n)),n=wt(n,1);this.dates=s,this.indices=i,this.cnt=s.length}sliceRange(e){let
t=this.getDateDayIndex(e.start),n=this.getDateDayIndex(wt(e.end,-1)),r=Math.max(0,t),i=Math.min(this.cnt-1,n);return
r=Math.ceil(r),i=Math.floor(i),r<=i?{firstIndex:r,lastIndex:i,isStart:t===r,isEnd:n===i}:null}getDateDayIndex(e){let{indices:t}=this,n=Math.floor(xt(this.dates[0],e));return
n<0?t[0]-1:n>=t.length?t[t.length-1]+1:t[n]}}class Ms{constructor(e,t){let
n,r,i,{dates:s}=e;if(t){for(r=s[0].getUTCDay(),n=1;n<s.length&&s[n].getUTCDay()!==r;n+=1);i=Math.ceil(s.length/n)}else
i=1,n=s.length;this.rowCnt=i,this.colCnt=n,this.daySeries=e,this.cells=this.buildCells(),this.headerDates=this.buildHeaderDates()}buildCells(){let e=[];for(let
t=0;t<this.rowCnt;t+=1){let n=[];for(let e=0;e<this.colCnt;e+=1)n.push(this.buildCell(t,e));e.push(n)}return e}buildCell(e,t){let
n=this.daySeries.dates[e*this.colCnt+t];return{key:n.toISOString(),date:n}}buildHeaderDates(){let e=[];for(let t=0;t<this.colCnt;t+=1)e.push(this.cells[0][t].date);return
e}sliceRange(e){let{colCnt:t}=this,n=this.daySeries.sliceRange(e),r=[];if(n){let{firstIndex:e,lastIndex:i}=n,s=e;for(;s<=i;){let
o=Math.floor(s/t),a=Math.min((o+1)*t,i+1);r.push({row:o,firstCol:s%t,lastCol:(a-1)%t,isStart:n.isStart&&s===e,isEnd:n.isEnd&&a-1===i}),s=a}}return r}}class Is{constructor(){t
his.sliceBusinessHours=Gt(this._sliceBusinessHours),this.sliceDateSelection=Gt(this._sliceDateSpan),this.sliceEventStore=Gt(this._sliceEventStore),this.sliceEventDrag=Gt(this
._sliceInteraction),this.sliceEventResize=Gt(this._sliceInteraction),this.forceDayIfListItem=!1}sliceProps(e,t,n,r,...i){let{eventUiBases:s}=e,o=this.sliceEventStore(e.eventS
tore,s,t,n,...i);return{dateSelectionSegs:this.sliceDateSelection(e.dateSelection,t,n,s,r,...i),businessHourSegs:this.sliceBusinessHours(e.businessHours,t,n,r,...i),fgEventSe
gs:o.fg,bgEventSegs:o.bg,eventDrag:this.sliceEventDrag(e.eventDrag,s,t,n,...i),eventResize:this.sliceEventResize(e.eventResize,s,t,n,...i),eventSelection:e.eventSelection}}sl
iceNowDate(e,t,n,r,...i){return this._sliceDateSpan({range:{start:e,end:Ct(e,1)},allDay:!1},t,n,{},r,...i)}_sliceBusinessHours(e,t,n,r,...i){return
e?this._sliceEventStore(mr(e,Os(t,Boolean(n)),r),{},t,n,...i).bg:[]}_sliceEventStore(e,t,n,r,...i){if(e){let
s=ii(e,t,Os(n,Boolean(r)),r);return{bg:this.sliceEventRanges(s.bg,i),fg:this.sliceEventRanges(s.fg,i)}}return{bg:[],fg:[]}}_sliceInteraction(e,t,n,r,...i){if(!e)return
null;let s=ii(e.mutatedEvents,t,Os(n,Boolean(r)),r);return{segs:this.sliceEventRanges(s.fg,i),affectedInstances:e.affectedEvents.instances,isEvent:e.isEvent}}_sliceDateSpan(e
,t,n,r,i,...s){if(!e)return[];let o=Os(t,Boolean(n)),a=ir(e.range,o);if(a){let t=function(e,t,n){let r=Ar({editable:!1},n),i=wr(r.refined,r.extra,"",e.allDay,!0,n);return{def
:i,ui:ci(i,t),instance:pr(i.defId,e.range),range:e.range,isStart:!0,isEnd:!0}}(e=Object.assign(Object.assign({},e),{range:a}),r,i),n=this.sliceRange(e.range,...s);for(let e
of n)e.eventRange=t;return n}return[]}sliceEventRanges(e,t){let n=[];for(let r of e)n.push(...this.sliceEventRange(r,t));return n}sliceEventRange(e,t){let
n=e.range;this.forceDayIfListItem&&"list-item"===e.ui.display&&(n={start:n.start,end:wt(n.start,1)});let r=this.sliceRange(n,...t);for(let t of
r)t.eventRange=e,t.isStart=e.isStart&&t.isStart,t.isEnd=e.isEnd&&t.isEnd;return r}}function Os(e,t){let n=e.activeRange;return
t?n:{start:Ct(n.start,e.slotMinTime.milliseconds),end:Ct(n.end,e.slotMaxTime.milliseconds-864e5)}}function Ns(e,t,n){let{instances:r}=e.mutatedEvents;for(let e in
r)if(!ar(t.validRange,r[e].range))return!1;return Hs({eventDrag:e},n)}function Ps(e,t,n){return!!ar(t.validRange,e.range)&&Hs({dateSelection:e},n)}function Hs(e,t){let n=t.ge
tCurrentData(),r=Object.assign({businessHours:n.businessHours,dateSelection:"",eventStore:n.eventStore,eventUiBases:n.eventUiBases,eventSelection:"",eventDrag:null,eventResiz
e:null},e);return(t.pluginHooks.isPropsValid||Bs)(r,t)}function Bs(e,t,n={},r){return!(e.eventDrag&&!function(e,t,n,r){let
i=t.getCurrentData(),s=e.eventDrag,o=s.mutatedEvents,a=o.defs,l=o.instances,c=li(a,s.isEvent?e.eventUiBases:{"":i.selectionConfig});r&&(c=An(c,r));let
d=(g=e.eventStore,p=s.affectedEvents.instances,{defs:g.defs,instances:Sn(g.instances,e=>!p[e.instanceId])}),u=d.defs,h=d.instances,f=li(u,e.eventUiBases);var g,p;for(let r in
 l){let o=l[r],g=o.range,p=c[o.defId],m=a[o.defId];if(!js(p.constraints,g,d,e.businessHours,t))return!1;let{eventOverlap:v}=t.options,y="function"==typeof v?v:null;for(let e
in h){let n=h[e];if(or(g,n.range)){if(!1===f[n.defId].overlap&&s.isEvent)return!1;if(!1===p.overlap)return!1;if(y&&!y(new ti(t,u[n.defId],n),new ti(t,m,o)))return!1}}let
b=i.eventStore;for(let e of p.allows){let i,s=Object.assign(Object.assign({},n),{range:o.range,allDay:m.allDay}),a=b.defs[m.defId],l=b.instances[r];if(i=a?new ti(t,a,l):new
ti(t,m),!e(Zr(s,t),i))return!1}}return!0}(e,t,n,r))&&!(e.dateSelection&&!function(e,t,n,r){let i=e.eventStore,s=i.defs,o=i.instances,a=e.dateSelection,l=a.range,{selectionCon
fig:c}=t.getCurrentData();r&&(c=r(c));if(!js(c.constraints,l,i,e.businessHours,t))return!1;let{selectOverlap:d}=t.options,u="function"==typeof d?d:null;for(let e in o){let
n=o[e];if(or(l,n.range)){if(!1===c.overlap)return!1;if(u&&!u(new ti(t,s[n.defId],n),null))return!1}}for(let e of c.allows){let
r=Object.assign(Object.assign({},n),a);if(!e(Zr(r,t),null))return!1}return!0}(e,t,n,r))}function js(e,t,n,r,i){for(let s of
e)if(!Ws(zs(s,t,n,r,i),t))return!1;return!0}function zs(e,t,n,r,i){return"businessHours"===e?Us(mr(r,t,i)):"string"==typeof e?Us(kr(n,t=>t.groupId===e)):"object"==typeof
```

```
e&&e?Us(mr(e,t,i)):[]}function Us(e){let{instances:t}=e,n=[];for(let e in t)n.push(t[e].range);return n}function Ws(e,t){for(let n of e)if(ar(n,t))return!0;return!1}const
Ls=/^(visible|hidden)$/;class Fs extends Gn{constructor(){super(...arguments),this.handleEl=e=>{this.el=e,Qn(this.props.elRef,e)}}render(){let{props:e}=this,{liquid:t,liquidI
sAbsolute:n}=e,r=t&&n,i=["fc-scroller"];return t&&(n?i.push("fc-scroller-liquid-absolute"):i.push("fc-scroller-liquid")),p("div",{ref:this.handleEl,className:i.join(" "),styl
e:{overflowX:e.overflowX,overflowY:e.overflowY,left:r&&-(e.overcomeLeft||0)||"",right:r&&-(e.overcomeRight||0)||"",bottom:r&&-(e.overcomeBottom||0)||"",marginLeft:!r&&-(e.ove
rcomeLeft||0)||"",marginRight:!r&&-(e.overcomeRight||0)||"",marginBottom:!r&&-(e.overcomeBottom||0)||"",maxHeight:e.maxHeight||""}},e.children)}needsXScrolling(){if(Ls.test(t
his.props.overflowX))return!1;let{el:e}=this,t=this.el.getBoundingClientRect().width-this.getYScrollbarWidth(),{children:n}=e;for(let e=0;e<n.length;e+=1){if(n[e].getBounding
ClientRect().width>t)return!0}return!1}needsYScrolling(){if(Ls.test(this.props.overflowY))return!1;let{el:e}=this,t=this.el.getBoundingClientRect().height-this.getXScrollbarW
idth(),{children:n}=e;for(let e=0;e<n.length;e+=1){if(n[e].getBoundingClientRect().height>t)return!0}return!1}getXScrollbarWidth(){return
Ls.test(this.props.overflowX)?0:this.el.offsetHeight-this.el.clientHeight}getYScrollbarWidth(){return
Ls.test(this.props.overflowY)?0:this.el.offsetWidth-this.el.clientWidth}}class
Vs{constructor(e){this.masterCallback=e,this.currentMap={},this.depths={},this.callbackMap={},this.handleValue=(e,t)=>{let{depths:n,currentMap:r}=this,i=!1,s=!1;null!==e?(i=t
 in r,r[t]=e,n[t]=(n[t]||0)+1,s=!0):(n[t]-=1,n[t]||(delete r[t],delete
this.callbackMap[t],i=!0)),this.masterCallback&&(i&&this.masterCallback(null,String(t)),s&&this.masterCallback(e,String(t)))}}createRef(e){let t=this.callbackMap[e];return
t||(t=this.callbackMap[e]=t=>{this.handleValue(t,String(e))},t}collect(e,t,n){return kn(this.currentMap,e,t,n)}getAll(){return wn(this.currentMap)}}function Gs(e){let
t=Pe(e,".fc-scrollgrid-shrink"),n=0;for(let e of t)n=Math.max(n,dt(e));return Math.ceil(n)}function Qs(e,t){return e.liquid&&t.liquid}function qs(e,t){return
null!=t.maxHeight||Qs(e,t)}function Ys(e,t,n,r){let{expandRows:i}=n;return"function"==typeof
t.content?t.content(n):p("table",{role:"presentation",className:[t.tableClassName,e.syncRowHeights?"fc-scrollgrid-sync-table":""].join("
"),style:{minWidth:n.tableMinWidth,width:n.clientWidth,height:i?n.clientHeight:""}},n.tableColGroupNode,p(r?"thead":"tbody",{role:"presentation"},"function"==typeof
t.rowContent?t.rowContent(n):t.rowContent))}function Zs(e,t){return St(e,t,Cn)}function Xs(e,t){let n=[];for(let r of e){let e=r.span||1;for(let
i=0;i<e;i+=1)n.push(p("col",{style:{width:"shrink"===r.width?$s(t):r.width||"",minWidth:r.minWidth||""}}))}return p("colgroup",{},...n)}function $s(e){return
null==e?4:e}function Js(e){for(let t of e)if("shrink"===t.width)return!0;return!1}function Ks(e,t){let n=["fc-scrollgrid",t.theme.getClass("table")];return
e&&n.push("fc-scrollgrid-liquid"),n}function eo(e,t){let n=["fc-scrollgrid-section","fc-scrollgrid-section-"+e.type,e.className];return
t&&e.liquid&&null==e.maxHeight&&n.push("fc-scrollgrid-section-liquid"),e.isSticky&&n.push("fc-scrollgrid-section-sticky"),n}function to(e){return
p("div",{className:"fc-scrollgrid-sticky-shim",style:{width:e.clientWidth,minWidth:e.tableMinWidth}})}function no(e){let{stickyHeaderDates:t}=e;return
null!=t&&"auto"!==t||(t="auto"===e.height||"auto"===e.viewHeight),t}function ro(e){let{stickyFooterScrollbar:t}=e;return
null!=t&&"auto"!==t||(t="auto"===e.height||"auto"===e.viewHeight),t}class io extends
Gn{constructor(){super(...arguments),this.processCols=Gt(e=>e,Zs),this.renderMicroColGroup=Gt(Xs),this.scrollerRefs=new Vs,this.scrollerElRefs=new Vs(this._handleScrollerEl.b
ind(this)),this.state={shrinkWidth:null,forceYScrollbars:!1,scrollerClientWidths:{},scrollerClientHeights:{}},this.handleSizing=()=>{this.safeSetState(Object.assign({shrinkWi
dth:this.computeShrinkWidth()},this.computeScrollerDims()))}}render(){let{props:e,state:t,context:n}=this,r=e.sections||[],i=this.processCols(e.cols),s=this.renderMicroColGro
up(i,t.shrinkWidth),o=Ks(e.liquid,n);e.collapsibleWidth&&o.push("fc-scrollgrid-collapsible");let a,l=r.length,c=0,d=[],u=[],h=[];for(;c<l&&"header"===(a=r[c]).type;)d.push(th
is.renderSection(a,s,!0)),c+=1;for(;c<l&&"body"===(a=r[c]).type;)u.push(this.renderSection(a,s,!1)),c+=1;for(;c<l&&"footer"===(a=r[c]).type;)h.push(this.renderSection(a,s,!0)
),c+=1;let f=!Ti();const g={role:"rowgroup"};return p("table",{role:"grid",className:o.join(" "),style:{height:e.height}},Boolean(!f&&d.length)&&p("thead",g,...d),Boolean(!f&
&u.length)&&p("tbody",g,...u),Boolean(!f&&h.length)&&p("tfoot",g,...h),f&&p("tbody",g,...d,...u,...h))}renderSection(e,t,n){return"outerContent"in
e?p(y,{key:e.key},e.outerContent):p("tr",{key:e.key,role:"presentation",className:eo(e,this.props.liquid).join("
")},this.renderChunkTd(e,t,e.chunk,n))}renderChunkTd(e,t,n,r){if("outerContent"in n)return n.outerContent;let{props:i}=this,{forceYScrollbars:s,scrollerClientWidths:o,scrolle
rClientHeights:a}=this.state,l=qs(i,e),c=Qs(i,e),d=i.liquid?s?"scroll":l?"auto":"hidden":"visible",u=e.key,h=Ys(e,n,{tableColGroupNode:t,tableMinWidth:"",clientWidth:i.collap
sibleWidth||void 0===o[u]?null:o[u],clientHeight:void 0!==a[u]?a[u]:null,expandRows:e.expandRows,syncRowHeights:!1,rowSyncHeights:[],reportRowHeightChange:()=>{}},r);return
p(r?"th":"td",{ref:n.elRef,role:"presentation"},p("div",{className:"fc-scroller-harness"+(c?" fc-scroller-harness-liquid":"")},p(Fs,{ref:this.scrollerRefs.createRef(u),elRef:
this.scrollerElRefs.createRef(u),overflowY:d,overflowX:i.liquid?"hidden":"visible",maxHeight:e.maxHeight,liquid:c,liquidIsAbsolute:!0},h)))}_handleScrollerEl(e,t){let
n=function(e,t){for(let n of e)if(n.key===t)return n;return null}(this.props.sections,t);n&&Qn(n.chunk.scrollerElRef,e)}componentDidMount(){this.handleSizing(),this.context.a
ddResizeHandler(this.handleSizing)}componentDidUpdate(){this.handleSizing()}componentWillUnmount(){this.context.removeResizeHandler(this.handleSizing)}computeShrinkWidth(){re
turn Js(this.props.cols)?Gs(this.scrollerElRefs.getAll()):0}computeScrollerDims(){let e=$i(),{scrollerRefs:t,scrollerElRefs:n}=this,r=!1,i={},s={};for(let e in
t.currentMap){let n=t.currentMap[e];if(n&&n.needsYScrolling()){r=!0;break}}for(let t of this.props.sections){let o=t.key,a=n.currentMap[o];if(a){let t=a.parentNode;i[o]=Math.
floor(t.getBoundingClientRect().width-(r?e.y:0)),s[o]=Math.floor(t.getBoundingClientRect().height)}}return{forceYScrollbars:r,scrollerClientWidths:i,scrollerClientHeights:s}}
}io.addStateEquality({scrollerClientWidths:Cn,scrollerClientHeights:Cn});class so extends Gn{constructor(){super(...arguments),this.handleEl=e=>{this.el=e,e&&oi(e,this.props.
seg)}}render(){const{props:e,context:t}=this,{options:n}=t,{seg:r}=e,{eventRange:i}=r,{ui:s}=i,o={event:new ti(t,i.def,i.instance),view:t.viewApi,timeText:e.timeText,textColo
r:s.textColor,backgroundColor:s.backgroundColor,borderColor:s.borderColor,isDraggable:!e.disableDragging&&hi(r,t),isStartResizable:!e.disableResizing&&fi(r,t),isEndResizable:
!e.disableResizing&&gi(r),isMirror:Boolean(e.isDragging||e.isResizing||e.isDateSelecting),isStart:Boolean(r.isStart),isEnd:Boolean(r.isEnd),isPast:Boolean(e.isPast),isFuture:
Boolean(e.isFuture),isToday:Boolean(e.isToday),isSelected:Boolean(e.isSelected),isDragging:Boolean(e.isDragging),isResizing:Boolean(e.isResizing)};return p(Jn,Object.assign({
},e,{elRef:this.handleEl,elClasses:[...vi(o),...r.eventRange.ui.classNames,...e.elClasses||[]],renderProps:o,generatorName:"eventContent",customGenerator:n.eventContent,defau
ltGenerator:e.defaultGenerator,classNameGenerator:n.eventClassNames,didMount:n.eventDidMount,willUnmount:n.eventWillUnmount}))}componentDidUpdate(e){this.el&&this.props.seg!=
=e.seg&&oi(this.el,this.props.seg)}}class oo extends Gn{render(){let{props:e,context:t}=this,{options:n}=t,{seg:r}=e,{ui:i}=r.eventRange,s=pi(r,n.eventTimeFormat||e.defaultTi
meFormat,t,e.defaultDisplayEventTime,e.defaultDisplayEventEnd);return p(so,Object.assign({},e,{elTag:"a",elStyle:{borderColor:i.borderColor,backgroundColor:i.backgroundColor}
,elAttrs:bi(r,t),defaultGenerator:ao,timeText:s}),(e,t)=>p(y,null,p(e,{elTag:"div",elClasses:["fc-event-main"],elStyle:{color:t.textColor}}),Boolean(t.isStartResizable)&&p("d
iv",{className:"fc-event-resizer fc-event-resizer-start"}),Boolean(t.isEndResizable)&&p("div",{className:"fc-event-resizer fc-event-resizer-end"})))}}function ao(e){return p(
"div",{className:"fc-event-main-frame"},e.timeText&&p("div",{className:"fc-event-time"},e.timeText),p("div",{className:"fc-event-title-container"},p("div",{className:"fc-even
t-title fc-sticky"},e.event.title||p(y,null," "))))}const
lo=e=>p(Ln.Consumer,null,t=>{let{options:n}=t,r={isAxis:e.isAxis,date:t.dateEnv.toDate(e.date),view:t.viewApi};return p(Jn,Object.assign({},e,{elTag:e.elTag||"div",renderProp
s:r,generatorName:"nowIndicatorContent",customGenerator:n.nowIndicatorContent,classNameGenerator:n.nowIndicatorClassNames,didMount:n.nowIndicatorDidMount,willUnmount:n.nowInd
icatorWillUnmount})})),co=an({day:"numeric"});class uo extends Gn{constructor(){super(...arguments),this.refineRenderProps=Qt(fo)}render(){let{props:e,context:t}=this,{option
s:n}=t,r=this.refineRenderProps({date:e.date,dateProfile:e.dateProfile,todayRange:e.todayRange,isMonthStart:e.isMonthStart||!1,showDayNumber:e.showDayNumber,extraRenderProps:
e.extraRenderProps,viewApi:t.viewApi,dateEnv:t.dateEnv,monthStartFormat:n.monthStartFormat});return p(Jn,Object.assign({},e,{elClasses:[...Vi(r,t.theme),...e.elClasses||[]],e
lAttrs:Object.assign(Object.assign({},e.elAttrs),r.isDisabled?{}:{"data-date":Wt(e.date)}),renderProps:r,generatorName:"dayCellContent",customGenerator:n.dayCellContent,defau
ltGenerator:e.defaultGenerator,classNameGenerator:r.isDisabled?void 0:n.dayCellClassNames,didMount:n.dayCellDidMount,willUnmount:n.dayCellWillUnmount}))}}function
ho(e){return Boolean(e.dayCellContent||Yn("dayCellContent",e))}function
fo(e){let{date:t,dateEnv:n,dateProfile:r,isMonthStart:i}=e,s=Fi(t,e.todayRange,null,r),o=e.showDayNumber?n.format(t,i?e.monthStartFormat:co):"";return
Object.assign(Object.assign(Object.assign({date:n.toDate(t),view:e.viewApi},s),{isMonthStart:i,dayNumberText:o}),e.extraRenderProps)}class go extends
Gn{render(){let{props:e}=this,{seg:t}=e;return p(so,{elTag:"div",elClasses:["fc-bg-event"],elStyle:{backgroundColor:t.eventRange.ui.backgroundColor},defaultGenerator:po,seg:t
,timeText:"",isDragging:!1,isResizing:!1,isDateSelecting:!1,isSelected:!1,isPast:e.isPast,isFuture:e.isFuture,isToday:e.isToday,disableDragging:!0,disableResizing:!0})}}funct
ion po(e){let{title:t}=e.event;return t&&p("div",{className:"fc-event-title"},e.event.title)}function mo(e){return p("div",{className:"fc-"+e})}const
vo=e=>p(Ln.Consumer,null,t=>{let{dateEnv:n,options:r}=t,{date:i}=e,s=r.weekNumberFormat||e.defaultFormat,o={num:n.computeWeekNumber(i),text:n.format(i,s),date:i};return p(Jn,
Object.assign({},e,{renderProps:o,generatorName:"weekNumberContent",customGenerator:r.weekNumberContent,defaultGenerator:yo,classNameGenerator:r.weekNumberClassNames,didMount
:r.weekNumberDidMount,willUnmount:r.weekNumberWillUnmount})}));function yo(e){return e.text}class bo extends Gn{constructor(){super(...arguments),this.state={titleId:We()},th
is.handleRootEl=e=>{this.rootEl=e,this.props.elRef&&Qn(this.props.elRef,e)},this.handleDocumentMouseDown=e=>{const t=ze(e);this.rootEl.contains(t)||this.handleCloseClick()},t
```

```
his.handleDocumentKeyDown=e=>{"Escape"===e.key&&this.handleCloseClick()},this.handleCloseClick=()=>{let{onClose:e}=this.props;e&&e()}}render(){let{theme:e,options:t}=this.con
text,{props:n,state:r}=this,i=["fc-popover",e.getClass("popover")].concat(n.extraClassNames||[]);return fe(p("div",Object.assign({},n.extraAttrs,{id:n.id,className:i.join("
"),"aria-labelledby":r.titleId,ref:this.handleRootEl}),p("div",{className:"fc-popover-header
"+e.getClass("popoverHeader")},p("span",{className:"fc-popover-title",id:r.titleId},n.title),p("span",{className:"fc-popover-close
"+e.getIconClass("close"),title:t.closeHint,onClick:this.handleCloseClick})),p("div",{className:"fc-popover-body "+e.getClass("popoverContent")},n.children)),n.parentEl)}comp
onentDidMount(){document.addEventListener("mousedown",this.handleDocumentMouseDown),document.addEventListener("keydown",this.handleDocumentKeyDown),this.updateSize()}componen
tWillUnmount(){document.removeEventListener("mousedown",this.handleDocumentMouseDown),document.removeEventListener("keydown",this.handleDocumentKeyDown)}updateSize(){let{isRt
l:e}=this.context,{alignmentEl:t,alignGridTop:n}=this.props,{rootEl:r}=this,i=function(e){let t=ns(e),n=e.getBoundingClientRect();for(let e of t){let
t=Hi(n,e.getBoundingClientRect());if(!t)return null;n=t}return n}(t);if(i){let s=r.getBoundingClientRect(),o=n?Oe(t,".fc-scrollgrid").getBoundingClientRect().top:i.top,a=e?i.
right-s.width:i.left;o=Math.max(o,10),a=Math.min(a,document.documentElement.clientWidth-10-s.width),a=Math.max(a,10);let
l=r.offsetParent.getBoundingClientRect();Be(r,{top:o-l.top,left:a-l.left})}}}class Eo extends ls{constructor(){super(...arguments),this.handleRootEl=e=>{this.rootEl=e,e?this.
context.registerInteractiveComponent(this,{el:e,useEventCenter:!1}):this.context.unregisterInteractiveComponent(this)}}render(){let{options:e,dateEnv:t}=this.context,{props:n
}=this,{startDate:r,todayRange:i,dateProfile:s}=n,o=t.format(r,e.dayPopoverFormat);return p(uo,{elRef:this.handleRootEl,date:r,dateProfile:s,todayRange:i},(t,r,i)=>p(bo,{elRe
f:i.ref,id:n.id,title:o,extraClassNames:["fc-more-popover"].concat(i.className||[]),extraAttrs:i,parentEl:n.parentEl,alignmentEl:n.alignmentEl,alignGridTop:n.alignGridTop,onC
lose:n.onClose},ho(e)&&p(t,{elTag:"div",elClasses:["fc-more-popover-misc"]}),n.children))}queryHit(e,t,n,r){let{rootEl:i,props:s}=this;return e>=0&&e<n&&t>=0&&t<r?{dateProfil
e:s.dateProfile,dateSpan:Object.assign({allDay:!s.forceTimed,range:{start:s.startDate,end:s.endDate}},s.extraDateSpan),dayEl:i,rect:{left:0,top:0,right:n,bottom:r},layer:1}:n
ull}}class So extends Gn{constructor(){super(...arguments),this.state={isPopoverOpen:!1,popoverId:We()},this.handleLinkEl=e=>{this.linkEl=e,this.props.elRef&&Qn(this.props.el
Ref,e)},this.handleClick=e=>{let{props:t,context:n}=this,{moreLinkClick:r}=n.options,i=Do(t).start;function s(e){let{def:t,instance:r,range:i}=e.eventRange;return{event:new
ti(n,t,r),start:n.dateEnv.toDate(i.start),end:n.dateEnv.toDate(i.end),isStart:e.isStart,isEnd:e.isEnd}}"function"==typeof
r&&(r=r({date:i,allDay:Boolean(t.allDayDate),allSegs:t.allSegs.map(s),hiddenSegs:t.hiddenSegs.map(s),jsEvent:e,view:n.viewApi})),r&&"popover"!==r?"string"==typeof
r&&n.calendarApi.zoomTo(i,r):this.setState({isPopoverOpen:!0})},this.handlePopoverClose=()=>{this.setState({isPopoverOpen:!1})}}render(){let{props:e,state:t}=this;return
p(Ln.Consumer,null,n=>{let{viewApi:r,options:i,calendarApi:s}=n,{moreLinkText:o}=i,{moreCnt:a}=e,l=Do(e),c="function"==typeof o?o.call(s,a):`+${a}
${o}`,d=at(i.moreLinkHint,[a],c),u={num:a,shortText:"+"+a,text:c,view:r};return p(y,null,Boolean(e.moreCnt)&&p(Jn,{elTag:e.elTag||"a",elRef:this.handleLinkEl,elClasses:[...e.
elClasses||[],"fc-more-link"],elStyle:e.elStyle,elAttrs:Object.assign(Object.assign({},e.elAttrs),Qe(this.handleClick)),{title:d,"aria-expanded":t.isPopoverOpen
,"aria-controls":t.isPopoverOpen?t.popoverId:""}),renderProps:u,generatorName:"moreLinkContent",customGenerator:i.moreLinkContent,defaultGenerator:e.defaultGenerator||Ao,clas
sNameGenerator:i.moreLinkClassNames,didMount:i.moreLinkDidMount,willUnmount:i.moreLinkWillUnmount},e.children),t.isPopoverOpen&&p(Eo,{id:t.popoverId,startDate:l.start,endDate
:l.end,dateProfile:e.dateProfile,todayRange:e.todayRange,extraDateSpan:e.extraDateSpan,parentEl:this.parentEl,alignmentEl:e.alignmentElRef?e.alignmentElRef.current:this.linkE
l,alignGridTop:e.alignGridTop,forceTimed:e.forceTimed,onClose:this.handlePopoverClose},e.popoverContent()))})}componentDidMount(){this.updateParentEl()}componentDidUpdate(){t
his.updateParentEl()}updateParentEl(){this.linkEl&&(this.parentEl=Oe(this.linkEl,".fc-view-harness"))}}function Ao(e){return e.text}function
Do(e){if(e.allDayDate)return{start:e.allDayDate,end:wt(e.allDayDate,1)};let{hiddenSegs:t}=e;return{start:wo(t),end:(n=t,n.reduce(Ro).eventRange.range.end)};var n}function
wo(e){return e.reduce(Co).eventRange.range.start}function Co(e,t){return e.eventRange.range.start<t.eventRange.range.start?e:t}function Ro(e,t){return
e.eventRange.range.end>t.eventRange.range.end?e:t}var
xo={__proto__:null,BASE_OPTION_DEFAULTS:cn,BaseComponent:Gn,BgEvent:go,CalendarImpl:Ni,CalendarRoot:ki,ContentContainer:Jn,CustomRenderingStore:class extends
class{constructor(){this.handlers=[]}set(e){this.currentValue=e;for(let t of this.handlers)t(e)}subscribe(e){this.handlers.push(e),void
0!==this.currentValue&&e(this.currentValue)}}{constructor(){super(...arguments),this.map=new Map}handle(e){const{map:t}=this;let n=!1;e.isActive?(t.set(e.id,e),n=!0):t.has(e.
id)&&(t.delete(e.id),n=!0),n&&this.set(t)}},DateComponent:ls,DateEnv:Hn,DateProfileGenerator:gr,DayCellContainer:uo,DayHeader:_s,DaySeriesModel:ks,DayTableModel:Ms,DelayedRun
ner:Me,ElementDragging:ms,ElementScrollController:os,Emitter:Gr,EventContainer:so,EventImpl:ti,Interaction:Mi,MoreLinkContainer:So,NamedTimeZoneImpl:class{constructor(e){this
.timeZoneName=e}},NowIndicatorContainer:lo,NowTimer:Rs,PositionCache:rs,RefMap:Vs,ScrollController:ss,ScrollResponder:Wn,Scroller:Fs,SegHierarchy:cs,SimpleScrollGrid:io,Slice
r:Is,Splitter:Wi,StandardEvent:oo,TableDateCell:Ds,TableDowCell:Cs,Theme:Bn,ViewContainer:er,ViewContextType:Ln,WeekNumberContainer:vo,WindowScrollController:as,addDays:wt,ad
dDurations:pt,addMs:Ct,addWeeks:Dt,allowContextMenu:tt,allowSelection:Ke,applyMutationToEventStore:$r,applyStyle:Be,asCleanDays:function(e){return
e.years||e.months||e.milliseconds?0:e.days},asRoughMinutes:function(e){return yt(e)/6e4},asRoughMs:yt,asRoughSeconds:function(e){return yt(e)/1e3},binarySearch:ps,buildElAttr
s:Zn,buildEntryKey:us,buildEventApis:ri,buildEventRangeKey:yi,buildIsoString:Ut,buildNavLinkAttrs:qi,buildSegTimeText:pi,collectFromHash:kn,combineEventUis:Pr,compareByFieldS
pecs:rt,compareNumbers:lt,compareObjs:_n,computeEarliestSegStart:wo,computeEdges:Ki,computeFallbackHeaderFormat:Es,computeInnerRect:es,computeRect:ts,computeShrinkWidth:Gs,co
mputeVisibleDayRange:dr,config:vs,constrainPoint:Bi,createDuration:ft,createEmptyEventStore:_r,createEventInstance:pr,createEventUi:Nr,createFormatter:an,diffDates:hr,diffDay
AndTime:_t,diffDays:xt,diffPoints:zi,diffWeeks:Rt,diffWholeDays:kt,diffWholeWeeks:Tt,disableCursor:Xe,elementClosest:Oe,elementMatches:Ne,enableCursor:$e,eventTupleToStore:Rr
,filterHash:Sn,findDirectChildren:function(e,t){let n=e instanceof HTMLElement?[e]:e,r=[];for(let e=0;e<n.length;e+=1){let i=n[e].children;for(let e=0;e<i.length;e+=1){let
n=i[e];t&&!Ne(n,t)||r.push(n)}}return r},findElements:Pe,flexibleCompare:st,formatDayString:Wt,formatIsoMonthStr:Lt,formatIsoTimeString:Ft,getAllowYScrolling:qs,getCanVGrowWi
thinCell:Ti,getClippingParents:ns,getDateMeta:Fi,getDayClassNames:Vi,getDefaultEventEnd:Xr,getElSeg:ai,getEntrySpanEnd:ds,getEventTargetViaRoot:ze,getIsRtlScrollbarOnLeft:Xi,
getRectCenter:ji,getRelevantEvents:xr,getScrollGridClassNames:Ks,getScrollbarWidths:$i,getSectionClassNames:eo,getSectionHasLiquidHeight:Qs,getSegAnchorAttrs:bi,getSegMeta:mi
,getSlotClassNames:function(e,t){let n=["fc-slot","fc-slot-"+At[e.dow]];return e.isDisabled?n.push("fc-slot-disabled"):(e.isToday&&(n.push("fc-slot-today"),n.push(t.getClass(
"today"))),e.isPast&&n.push("fc-slot-past"),e.isFuture&&n.push("fc-slot-future")),n},getStickyFooterScrollbar:ro,getStickyHeaderDates:no,getUniqueDomId:We,greatestDurationDen
ominator:Et,groupIntersectingEntries:hs,guid:Ze,hasBgRendering:si,hasCustomDayCellContent:ho,hasShrinkWidth:Js,identity:yn,injectStyles:xe,interactionSettingsStore:Oi,interac
tionSettingsToStore:Ii,intersectRanges:ir,intersectRects:Hi,intersectSpans:fs,isArraysEqual:St,isColPropsEqual:Zs,isDateSelectionValid:Ps,isDateSpansEqual:Ai,isInt:ct,isInter
actionValid:Ns,isMultiDayRange:ur,isPropsEqual:Cn,isPropsValid:Bs,isValidDate:jt,mapHash:An,memoize:Gt,memoizeArraylike:function(e,t,n){let r=[],i=[];return s=>{let
o=r.length,a=s.length,l=0;for(;l<o;l+=1)if(s[l]){if(!St(r[l],s[l]))n&&n(i[l]);let r=e.apply(this,s[l]);t&&t(r,i[l])||(i[l]=r)}else
n&&n(i[l]);for(;l<a;l+=1)i[l]=e.apply(this,s[l]);return r=s,i.splice(a),i}},memoizeHashlike:function(e,t,n){let r={},i={};return s=>{let o={};for(let a in
s)if(i[a])if(St(r[a],s[a]))o[a]=i[a];else{n&&n(i[a]);let r=e.apply(this,s[a]);o[a]=t&&t(r,i[a])?i[a]:r}else o[a]=e.apply(this,s[a]);return r=s,i=o,o}},memoizeObjArg:Qt,mergeE
ventStores:Tr,multiplyDuration:mt,padStart:ot,parseBusinessHours:qr,parseClassNames:Mr,parseDragMeta:bs,parseEventDef:wr,parseFieldSpecs:nt,parseMarker:Pn,pointInsideRect:Pi,
preventContextMenu:et,preventDefault:Le,preventSelection:Je,rangeContainsMarker:lr,rangeContainsRange:ar,rangesEqual:sr,rangesIntersect:or,refineEventDef:Ar,refineProps:vn,re
moveElement:Ie,removeExact:function(e,t){let n=0,r=0;for(;r<e.length;)e[r]===t?(e.splice(r,1),n+=1):r+=1;return n},renderChunkContent:Ys,renderFill:mo,renderMicroColGroup:Xs,
renderScrollShim:to,requestJson:xi,sanitizeShrinkWidth:$s,setRef:Qn,sliceEventStore:ii,sortEventSegs:di,startOfDay:Mt,translateRect:function(e,t,n){return{left:e.left+t,right
:e.right+t,top:e.top+n,bottom:e.bottom+n}},triggerDateSelect:Yr,unpromisify:Ci,whenTransitionDone:Ge,wholeDivideDurations:bt},_o={__proto__:null,createPortal:fe,createContext
:Un,flushSync:jn,Component:D,Fragment:y,cloneElement:function(e,n,r){var i,s,o,a=f({},e.props);for(o in n)"key"==o?i=n[o]:"ref"==o?s=n[o]:a[o]=n[o];return
arguments.length>2&&(a.children=arguments.length>3?t.call(arguments,2):r),m(e.type,a,i||e.key,s||e.ref,null)},createElement:p,createRef:v,h:p,hydrate:function
e(t,n){U(t,n,e)},get isValidElement(){return i},get options(){return n},render:U,toChildArray:M};const
To=[],ko={code:"en",week:{dow:0,doy:4},direction:"ltr",buttonText:{prev:"prev",next:"next",prevYear:"prev year",nextYear:"next year",year:"year",today:"today",month:"month",w
eek:"week",day:"day",list:"list"},weekText:"W",weekTextLong:"Week",closeHint:"Close",timeHint:"Time",eventHint:"Event",allDayText:"all-day",moreLinkText:"more",noEventsText:"
No events to display"},Mo=Object.assign(Object.assign({},ko),{buttonHints:{prev:"Previous $0",next:"Next $0",today:(e,t)=>"day"===t?"Today":"This "+e},viewHint:"$0
view",navLinkHint:"Go to $0",moreLinkHint:e=>`Show ${e} more event${1===e?"":"s"}`});function Io(e){let t=e.length>0?e[0].code:"en",n=To.concat(e),r={en:Mo};for(let e of
n)r[e.code]=e;return{map:r,defaultCode:t}}function Oo(e,t){return"object"!=typeof e||Array.isArray(e)?function(e,t){let n=[].concat(e||[]),r=function(e,t){for(let
n=0;n<e.length;n+=1){let r=e[n].toLocaleLowerCase().split("-");for(let e=r.length;e>0;e-=1){let n=r.slice(0,e).join("-");if(t[n])return t[n]}}return null}(n,t)||Mo;return
No(e,n,r)}(e,t):No(e.code,[e.code],e)}function No(e,t,n){let r=En([ko,n],["buttonText"]);delete r.code;let{week:i}=r;return delete
r.week,{codeArg:e,codes:t,week:i,simpleNumberFormat:new Intl.NumberFormat(e),options:r}}function Po(e){return{id:Ze(),name:e.name,premiumReleaseDate:e.premiumReleaseDate?new
Date(e.premiumReleaseDate):void 0,deps:e.deps||[],reducers:e.reducers||[],isLoadingFuncs:e.isLoadingFuncs||[],contextInit:[].concat(e.contextInit||[]),eventRefiners:e.eventRe
```

```
finers||{},eventDefMemberAdders:e.eventDefMemberAdders||[],eventSourceRefiners:e.eventSourceRefiners||{},isDraggableTransformers:e.isDraggableTransformers||[],eventDragMutati
onMassagers:e.eventDragMutationMassagers||[],eventDefMutationAppliers:e.eventDefMutationAppliers||[],dateSelectionTransformers:e.dateSelectionTransformers||[],datePointTransf
orms:e.datePointTransforms||[],dateSpanTransforms:e.dateSpanTransforms||[],views:e.views||{},viewPropsTransformers:e.viewPropsTransformers||[],isPropsValid:e.isPropsValid||nu
ll,externalDefTransforms:e.externalDefTransforms||[],viewContainerAppends:e.viewContainerAppends||[],eventDropTransformers:e.eventDropTransformers||[],componentInteractions:e
.componentInteractions||[],calendarInteractions:e.calendarInteractions||[],themeClasses:e.themeClasses||{},eventSourceDefs:e.eventSourceDefs||[],cmdFormatter:e.cmdFormatter,r
ecurringTypes:e.recurringTypes||[],namedTimeZonedImpl:e.namedTimeZonedImpl,initialView:e.initialView||"",elementDraggingImpl:e.elementDraggingImpl,optionChangeHandlers:e.opti
onChangeHandlers||{},scrollGridImpl:e.scrollGridImpl||null,listenerRefiners:e.listenerRefiners||{},optionRefiners:e.optionRefiners||{},propSetHandlers:e.propSetHandlers||{}}}
function Ho(){let e,t=[],n=[];return(r,i)=>(e&&St(r,t)&&St(i,n)||(e=function(e,t){let n={},r={premiumReleaseDate:void 0,reducers:[],isLoadingFuncs:[],contextInit:[],eventRefi
ners:{},eventDefMemberAdders:[],eventSourceRefiners:{},isDraggableTransformers:[],eventDragMutationMassagers:[],eventDefMutationAppliers:[],dateSelectionTransformers:[],dateP
ointTransforms:[],dateSpanTransforms:[],views:{},viewPropsTransformers:[],isPropsValid:null,externalDefTransforms:[],viewContainerAppends:[],eventDropTransformers:[],componen
tInteractions:[],calendarInteractions:[],themeClasses:{},eventSourceDefs:[],cmdFormatter:null,recurringTypes:[],namedTimeZonedImpl:null,initialView:"",elementDraggingImpl:nul
l,optionChangeHandlers:{},scrollGridImpl:null,listenerRefiners:{},optionRefiners:{},propSetHandlers:{}};function i(e){for(let o of e){const e=o.name,a=n[e];void 0===a?(n[e]=o
.id,i(o.deps),s=o,r={premiumReleaseDate:Bo((t=r).premiumReleaseDate,s.premiumReleaseDate),reducers:t.reducers.concat(s.reducers),isLoadingFuncs:t.isLoadingFuncs.concat(s.isLo
adingFuncs),contextInit:t.contextInit.concat(s.contextInit),eventRefiners:Object.assign(Object.assign({},t.eventRefiners),s.eventRefiners),eventDefMemberAdders:t.eventDefMemb
erAdders.concat(s.eventDefMemberAdders),eventSourceRefiners:Object.assign(Object.assign({},t.eventSourceRefiners),s.eventSourceRefiners),isDraggableTransformers:t.isDraggable
Transformers.concat(s.isDraggableTransformers),eventDragMutationMassagers:t.eventDragMutationMassagers.concat(s.eventDragMutationMassagers),eventDefMutationAppliers:t.eventDe
fMutationAppliers.concat(s.eventDefMutationAppliers),dateSelectionTransformers:t.dateSelectionTransformers.concat(s.dateSelectionTransformers),datePointTransforms:t.datePoint
Transforms.concat(s.datePointTransforms),dateSpanTransforms:t.dateSpanTransforms.concat(s.dateSpanTransforms),views:Object.assign(Object.assign({},t.views),s.views),viewProps
Transformers:t.viewPropsTransformers.concat(s.viewPropsTransformers),isPropsValid:s.isPropsValid||t.isPropsValid,externalDefTransforms:t.externalDefTransforms.concat(s.extern
alDefTransforms),viewContainerAppends:t.viewContainerAppends.concat(s.viewContainerAppends),eventDropTransformers:t.eventDropTransformers.concat(s.eventDropTransformers),cale
ndarInteractions:t.calendarInteractions.concat(s.calendarInteractions),componentInteractions:t.componentInteractions.concat(s.componentInteractions),themeClasses:Object.assig
n(Object.assign({},t.themeClasses),s.themeClasses),eventSourceDefs:t.eventSourceDefs.concat(s.eventSourceDefs),cmdFormatter:s.cmdFormatter||t.cmdFormatter,recurringTypes:t.re
curringTypes.concat(s.recurringTypes),namedTimeZonedImpl:s.namedTimeZonedImpl||t.namedTimeZonedImpl,initialView:t.initialView||s.initialView,elementDraggingImpl:t.elementDrag
gingImpl||s.elementDraggingImpl,optionChangeHandlers:Object.assign(Object.assign({},t.optionChangeHandlers),s.optionChangeHandlers),scrollGridImpl:s.scrollGridImpl||t.scrollG
ridImpl,listenerRefiners:Object.assign(Object.assign({},t.listenerRefiners),s.listenerRefiners),optionRefiners:Object.assign(Object.assign({},t.optionRefiners),s.optionRefine
rs),propSetHandlers:Object.assign(Object.assign({},t.propSetHandlers),s.propSetHandlers)}):a!==o.id&&console.warn(`Duplicate plugin '${e}'`)}var t,s}return
e&&i(e),i(t),r}(r,i)),t=r,n=i,e)}function Bo(e,t){return void 0===e?t:void 0===t?e:new Date(Math.max(e.valueOf(),t.valueOf()))}class jo extends Bn{}function
zo(e,t,n,r){if(t[e])return t[e];let i=function(e,t,n,r){let
i=n[e],s=r[e],o=e=>i&&null!==i[e]?i[e]:s&&null!==s[e]?s[e]:null,a=o("component"),l=o("superType"),c=null;if(l){if(l===e)throw new Error("Can't have a custom view type that
references itself");c=zo(l,t,n,r)}!a&&c&&(a=c.component);if(!a)return null;return{type:e,component:a,defaults:Object.assign(Object.assign({},c?c.defaults:{}),i?i.rawOptions:{
}),overrides:Object.assign(Object.assign({},c?c.overrides:{}),s?s.rawOptions:{})}}(e,t,n,r);return i&&(t[e]=i),i}function Uo(e){return An(e,Wo)}function Wo(e){let
t="function"==typeof e?{component:e}:e,{component:n}=t;return t.content?n=Lo(t):!n||n.prototype instanceof
Gn||(n=Lo(Object.assign(Object.assign({},t),{content:n}))),{superType:t.type,component:n,rawOptions:t}}function Lo(e){return t=>p(Ln.Consumer,null,n=>p(Jn,{elTag:"div",elClas
ses:tr(n.viewSpec),renderProps:Object.assign(Object.assign({},t),{nextDayThreshold:n.options.nextDayThreshold}),generatorName:void
0,customGenerator:e.content,classNameGenerator:e.classNames,didMount:e.didMount,willUnmount:e.willUnmount}))}function Fo(e,t,n,r){let i=Uo(e),s=Uo(t.views);return
An(function(e,t){let n,r={};for(n in e)zo(n,r,e,t);for(n in t)zo(n,r,e,t);return r}(i,s),e=>function(e,t,n,r,i){let
s=e.overrides.duration||e.defaults.duration||r.duration||n.duration,o=null,a="",l="",c={};if(s&&(o=function(e){let t=JSON.stringify(e),n=Vo[t];void
0===n&&(n=ft(e),Vo[t]=n);return n}(s),o)){let e=Et(o);a=e.unit,1===e.value&&(l=a,c=t[a]?t[a].rawOptions:{})}let d=t=>{let n=t.buttonText||{},r=e.defaults.buttonTextKey;return
 null!=r&&null!=n[r]?n[r]:null!=n[e.type]?n[e.type]:null!=n[l]?n[l]:null},u=t=>{let n=t.buttonHints||{},r=e.defaults.buttonTextKey;return null!=r&&null!=n[r]?n[r]:null!=n[e.t
ype]?n[e.type]:null!=n[l]?n[l]:null};return{type:e.type,component:e.component,duration:o,durationUnit:a,singleUnit:l,optionDefaults:e.defaults,optionOverrides:Object.assign(O
bject.assign({},c),e.overrides),buttonTextOverride:d(r)||d(n)||e.overrides.buttonText,buttonTextDefault:d(i)||e.defaults.buttonText||d(cn)||e.type,buttonTitleOverride:u(r)||u
(n)||e.overrides.buttonHint,buttonTitleDefault:u(i)||e.defaults.buttonHint||u(cn)}}(e,s,t,n,r))}jo.prototype.classes={root:"fc-theme-standard",tableCellShaded:"fc-cell-shaded
",buttonGroup:"fc-button-group",button:"fc-button fc-button-primary",buttonActive:"fc-button-active"},jo.prototype.baseIconClass="fc-icon",jo.prototype.iconClasses={close:"fc
-icon-x",prev:"fc-icon-chevron-left",next:"fc-icon-chevron-right",prevYear:"fc-icon-chevrons-left",nextYear:"fc-icon-chevrons-right"},jo.prototype.rtlIconClasses={prev:"fc-ic
on-chevron-right",next:"fc-icon-chevron-left",prevYear:"fc-icon-chevrons-right",nextYear:"fc-icon-chevrons-left"},jo.prototype.iconOverrideOption="buttonIcons",jo.prototype.i
conOverrideCustomButtonOption="icon",jo.prototype.iconOverridePrefix="fc-icon-";let Vo={};function Go(e,t,n){let r=t?t.activeRange:null;return Yo({},function(e,t){let
n=zr(t),r=[].concat(e.eventSources||[]),i=[];e.initialEvents&&r.unshift(e.initialEvents);e.events&&r.unshift(e.events);for(let e of r){let r=jr(e,t,n);r&&i.push(r)}return
i}(e,n),r,n)}function Qo(e,t,n,r){let i=n?n.activeRange:null;switch(t.type){case"ADD_EVENT_SOURCES":return Yo(e,t.sources,i,r);case"REMOVE_EVENT_SOURCE":return
s=e,o=t.sourceId,Sn(s,e=>e.sourceId!==o);case"PREV":case"NEXT":case"CHANGE_DATE":case"CHANGE_VIEW_TYPE":return n?Zo(e,i,r):e;case"FETCH_EVENT_SOURCES":return
Xo(e,t.sourceIds?Dn(t.sourceIds):Jo(e,r),i,t.isRefetch||!1,r);case"RECEIVE_EVENTS":case"RECEIVE_EVENT_ERROR":return function(e,t,n,r){let
i=e[t];if(i&&n===i.latestFetchId)return Object.assign(Object.assign({},e),{[t]:Object.assign(Object.assign({},i),{isFetching:!1,fetchRange:r})});return
e}(e,t.sourceId,t.fetchId,t.fetchRange);case"REMOVE_ALL_EVENT_SOURCES":return{};default:return e}var s,o}function qo(e){for(let t in
e)if(e[t].isFetching)return!0;return!1}function Yo(e,t,n,r){let i={};for(let e of t)i[e.sourceId]=e;return n&&(i=Zo(i,n,r)),Object.assign(Object.assign({},e),i)}function
Zo(e,t,n){return Xo(e,Sn(e,e=>function(e,t,n){if(!Ko(e,n))return!e.latestFetchId;return!n.options.lazyFetching||!e.fetchRange||e.isFetching||t.start<e.fetchRange.start||t.end
>e.fetchRange.end}(e,t,n)),t,!1,n)}function Xo(e,t,n,r,i){let s={};for(let o in e){let a=e[o];t[o]?s[o]=$o(a,n,r,i):s[o]=a}return s}function
$o(e,t,n,r){let{options:i,calendarApi:s}=r,o=r.pluginHooks.eventSourceDefs[e.sourceDefId],a=Ze();return o.fetch({eventSource:e,range:t,isRefetch:n,context:r},n=>{let{rawEvent
s:o}=n;i.eventSourceSuccess&&(o=i.eventSourceSuccess.call(s,o,n.response)||o),e.success&&(o=e.success.call(s,o,n.response)||o),r.dispatch({type:"RECEIVE_EVENTS",sourceId:e.so
urceId,fetchId:a,fetchRange:t,rawEvents:o})},n=>{let o=!1;i.eventSourceFailure&&(i.eventSourceFailure.call(s,n),o=!0),e.failure&&(e.failure(n),o=!0),o||console.warn(n.message
,n),r.dispatch({type:"RECEIVE_EVENT_ERROR",sourceId:e.sourceId,fetchId:a,fetchRange:t,error:n})}),Object.assign(Object.assign({},e),{isFetching:!0,latestFetchId:a})}function
Jo(e,t){return Sn(e,e=>Ko(e,t))}function Ko(e,t){return!t.pluginHooks.eventSourceDefs[e.sourceDefId].ignoreRange}function ea(e,t){switch(t.type){case"UNSELECT_DATES":return
null;case"SELECT_DATES":return t.selection;default:return e}}function ta(e,t){switch(t.type){case"UNSELECT_EVENT":return"";case"SELECT_EVENT":return
t.eventInstanceId;default:return e}}function na(e,t){let n;switch(t.type){case"UNSET_EVENT_DRAG":return null;case"SET_EVENT_DRAG":return
n=t.state,{affectedEvents:n.affectedEvents,mutatedEvents:n.mutatedEvents,isEvent:n.isEvent};default:return e}}function ra(e,t){let
n;switch(t.type){case"UNSET_EVENT_RESIZE":return null;case"SET_EVENT_RESIZE":return
n=t.state,{affectedEvents:n.affectedEvents,mutatedEvents:n.mutatedEvents,isEvent:n.isEvent};default:return e}}function
ia(e,t,n,r,i){return{header:e.headerToolbar?sa(e.headerToolbar,e,t,n,r,i):null,footer:e.footerToolbar?sa(e.footerToolbar,e,t,n,r,i):null}}function sa(e,t,n,r,i,s){let
o={},a=[],l=!1;for(let c in e){let
d=oa(e[c],t,n,r,i,s);o[c]=d.widgets,a.push(...d.viewsWithButtons),l=l||d.hasTitle}return{sectionWidgets:o,viewsWithButtons:a,hasTitle:l}}function oa(e,t,n,r,i,s){let
o="rtl"===t.direction,a=t.customButtons||{},l=n.buttonText||{},c=t.buttonText||{},d=n.buttonHints||{},u=t.buttonHints||{},h=e?e.split("
"):[],f=[],g=!1;return{widgets:h.map(e=>e.split(",").map(e=>{if("title"===e)return g=!0,{buttonName:e};let
n,h,p,m,v,y;if(n=a[e])p=e=>{n.click&&n.click.call(e.target,e,e.target)},(m=r.getCustomButtonIconClass(n))||(m=r.getIconClass(e,o))||(v=n.text),y=n.hint||n.text;else
if(h=i[e]){f.push(e),p=()=>{s.changeView(e)},(v=h.buttonTextOverride)||(m=r.getIconClass(e,o))||(v=h.buttonTextDefault);let
n=h.buttonTextOverride||h.buttonTextDefault;y=at(h.buttonTitleOverride||h.buttonTitleDefault||t.viewHint,[n,e],n)}else
if(s[e])if(p=()=>{s[e]()},(v=l[e])||(m=r.getIconClass(e,o))||(v=c[e]),"prevYear"===e||"nextYear"===e){let
```

```
t="prevYear"===e?"prev":"next";y=at(d[t]||u[t],[c.year||"year","year"],c[e])}else
y=t=>at(d[e]||u[e],[c[t]||t,t],c[e]);return{buttonName:e,buttonClick:p,buttonIcon:m,buttonText:v,buttonHint:y}})),viewsWithButtons:f,hasTitle:g}}class
aa{constructor(e,t,n){this.type=e,this.getCurrentData=t,this.dateEnv=n}get calendar(){return this.getCurrentData().calendarApi}get title(){return
this.getCurrentData().viewTitle}get activeStart(){return this.dateEnv.toDate(this.getCurrentData().dateProfile.activeRange.start)}get activeEnd(){return
this.dateEnv.toDate(this.getCurrentData().dateProfile.activeRange.end)}get currentStart(){return this.dateEnv.toDate(this.getCurrentData().dateProfile.currentRange.start)}get
 currentEnd(){return this.dateEnv.toDate(this.getCurrentData().dateProfile.currentRange.end)}getOption(e){return this.getCurrentData().options[e]}}function la(e,t){let
n=wn(t.getCurrentData().eventSources);if(1===n.length&&1===e.length&&Array.isArray(n[0]._raw)&&Array.isArray(e[0]))return void
t.dispatch({type:"RESET_RAW_EVENTS",sourceId:n[0].sourceId,rawEvents:e[0]});let r=[];for(let t of e){let e=!1;for(let
r=0;r<n.length;r+=1)if(n[r]._raw===t){n.splice(r,1),e=!0;break}e||r.push(t)}for(let e of n)t.dispatch({type:"REMOVE_EVENT_SOURCE",sourceId:e.sourceId});for(let e of
r)t.calendarApi.addEventSource(e)}const ca=[Po({name:"array-event-source",eventSourceDefs:[{ignoreRange:!0,parseMeta:e=>Array.isArray(e.events)?e.events:null,fetch(e,t){t({ra
wEvents:e.eventSource.meta})}}]}),Po({name:"func-event-source",eventSourceDefs:[{parseMeta:e=>"function"==typeof e.events?e.events:null,fetch(e,t,n){const{dateEnv:r}=e.contex
t;Ci(e.eventSource.meta.bind(null,Di(e.range,r)),e=>t({rawEvents:e}),n)}}]}),Po({name:"json-event-source",eventSourceRefiners:{method:String,extraParams:yn,startParam:String,
endParam:String,timeZoneParam:String},eventSourceDefs:[{parseMeta:e=>!e.url||"json"!==e.format&&e.format?null:{url:e.url,format:"json",method:(e.method||"GET").toUpperCase(),
extraParams:e.extraParams,startParam:e.startParam,endParam:e.endParam,timeZoneParam:e.timeZoneParam},fetch(e,t,n){const{meta:r}=e.eventSource,i=function(e,t,n){let r,i,s,o,{d
ateEnv:a,options:l}=n,c={};r=e.startParam,null==r&&(r=l.startParam);i=e.endParam,null==i&&(i=l.endParam);s=e.timeZoneParam,null==s&&(s=l.timeZoneParam);o="function"==typeof
e.extraParams?e.extraParams():e.extraParams||{};Object.assign(c,o),c[r]=a.formatIso(t.start),c[i]=a.formatIso(t.end),"local"!==a.timeZone&&(c[s]=a.timeZone);return c}(r,e.ran
ge,e.context);xi(r.method,r.url,i).then(([e,n])=>{t({rawEvents:e,response:n})},n)}}]}),Po({name:"simple-recurring-event",recurringTypes:[{parse(e,t){if(e.daysOfWeek||e.startT
ime||e.endTime||e.startRecur||e.endRecur){let i,s={daysOfWeek:e.daysOfWeek||null,startTime:e.startTime||null,endTime:e.endTime||null,startRecur:e.startRecur?t.createMarker(e.
startRecur):null,endRecur:e.endRecur?t.createMarker(e.endRecur):null};return e.duration&&(i=e.duration),!i&&e.startTime&&e.endTime&&(n=e.endTime,r=e.startTime,i={years:n.year
s-r.years,months:n.months-r.months,days:n.days-r.days,milliseconds:n.milliseconds-r.milliseconds}),{allDayGuess:Boolean(!e.startTime&&!e.endTime),duration:i,typeData:s}}var
n,r;return null},expand(e,t,n){let r=ir(t,{start:e.startRecur,end:e.endRecur});return r?function(e,t,n,r){let i=e?Dn(e):null,s=Mt(n.start),o=n.end,a=[];for(;s<o;){let
e;i&&!i[s.getUTCDay()]||(e=t?r.add(s,t):s,a.push(e)),s=wt(s,1)}return a}(e.daysOfWeek,e.startTime,r,n):[]}}],eventRefiners:{daysOfWeek:yn,startTime:ft,endTime:ft,duration:ft,
startRecur:yn,endRecur:yn}}),Po({name:"change-handler",optionChangeHandlers:{events(e,t){la([e],t)},eventSources:la}}),Po({name:"misc",isLoadingFuncs:[e=>qo(e.eventSources)],
propSetHandlers:{dateProfile:function(e,t){t.emitter.trigger("datesSet",Object.assign(Object.assign({},Di(e.activeRange,t.dateEnv)),{view:t.viewApi}))},eventStore:function(e,
t){let{emitter:n}=t;n.hasHandlers("eventsSet")&&n.trigger("eventsSet",ri(e,t))}}})];class
da{constructor(e,t){this.runTaskOption=e,this.drainedOption=t,this.queue=[],this.delayedRunner=new Me(this.drain.bind(this))}request(e,t){this.queue.push(e),this.delayedRunne
r.request(t)}pause(e){this.delayedRunner.pause(e)}resume(e,t){this.delayedRunner.resume(e,t)}drain(){let{queue:e}=this;for(;e.length;){let t,n=[];for(;t=e.shift();)this.runTa
sk(t),n.push(t);this.drained(n)}}runTask(e){this.runTaskOption&&this.runTaskOption(e)}drained(e){this.drainedOption&&this.drainedOption(e)}}function ua(e,t,n){let r;return r=
/^(year|month)$/.test(e.currentRangeUnit)?e.currentRange:e.activeRange,n.formatRange(r.start,r.end,an(t.titleFormat||function(e){let{currentRangeUnit:t}=e;if("year"===t)retur
n{year:"numeric"};if("month"===t)return{year:"numeric",month:"long"};let n=kt(e.currentRange.start,e.currentRange.end);if(null!==n&&n>1)return{year:"numeric",month:"short",da
y:"numeric"};return{year:"numeric",month:"long",day:"numeric"}}(e)),{isEndExclusive:e.isRangeAllDay,defaultSeparator:t.titleRangeSeparator})}class ha{constructor(e){this.comp
uteCurrentViewData=Gt(this._computeCurrentViewData),this.organizeRawLocales=Gt(Io),this.buildLocale=Gt(Oo),this.buildPluginHooks=Ho(),this.buildDateEnv=Gt(fa),this.buildTheme
=Gt(ga),this.parseToolbars=Gt(ia),this.buildViewSpecs=Gt(Fo),this.buildDateProfileGenerator=Qt(pa),this.buildViewApi=Gt(ma),this.buildViewUiProps=Qt(ba),this.buildEventUiBySo
urce=Gt(va,Cn),this.buildEventUiBases=Gt(ya),this.parseContextBusinessHours=Qt(Sa),this.buildTitle=Gt(ua),this.emitter=new Gr,this.actionRunner=new da(this._handleAction.bind
(this),this.updateData.bind(this)),this.currentCalendarOptionsInput={},this.currentCalendarOptionsRefined={},this.currentViewOptionsInput={},this.currentViewOptionsRefined={}
,this.currentCalendarOptionsRefiners={},this.optionsForRefining=[],this.optionsForHandling=[],this.getCurrentData=()=>this.data,this.dispatch=e=>{this.actionRunner.request(e)
},this.props=e,this.actionRunner.pause();let t={},n=this.computeOptionsData(e.optionOverrides,t,e.calendarApi),r=n.calendarOptions.initialView||n.pluginHooks.initialView,i=th
is.computeCurrentViewData(r,n,e.optionOverrides,t);e.calendarApi.currentDataManager=this,this.emitter.setThisContext(e.calendarApi),this.emitter.setOptions(i.options);let
s=function(e,t){let n=e.initialDate;return
null!=n?t.createMarker(n):fr(e.now,t)}(n.calendarOptions,n.dateEnv),o=i.dateProfileGenerator.build(s);lr(o.activeRange,s)||(s=o.currentRange.start);let a={dateEnv:n.dateEnv,o
ptions:n.calendarOptions,pluginHooks:n.pluginHooks,calendarApi:e.calendarApi,dispatch:this.dispatch,emitter:this.emitter,getCurrentData:this.getCurrentData};for(let e of
n.pluginHooks.contextInit)e(a);let l=Go(n.calendarOptions,o,a),c={dynamicOptionOverrides:t,currentViewType:r,currentDate:s,dateProfile:o,businessHours:this.parseContextBusine
ssHours(a),eventSources:l,eventUiBases:{},eventStore:{defs:{},instances:{}},renderableEventStore:{defs:{},instances:{}},dateSelection:null,eventSelection:"",eventDrag:null,ev
entResize:null,selectionConfig:this.buildViewUiProps(a).selectionConfig},d=Object.assign(Object.assign({},a),c);for(let e of n.pluginHooks.reducers)Object.assign(c,e(null,nul
l,d));Ea(c,a)&&this.emitter.trigger("loading",!0),this.state=c,this.updateData(),this.actionRunner.resume()}resetOptions(e,t){let{props:n}=this;void
0===t?n.optionOverrides=e:(n.optionOverrides=Object.assign(Object.assign({},n.optionOverrides||{}),e),this.optionsForRefining.push(...t)),(void
0===t||t.length)&&this.actionRunner.request({type:"NOTHING"})}_handleAction(e){let{props:t,state:n,emitter:r}=this,i=function(e,t){switch(t.type){case"SET_OPTION":return
Object.assign(Object.assign({},e),{[t.optionName]:t.rawOptionValue});default:return
e}}(n.dynamicOptionOverrides,e),s=this.computeOptionsData(t.optionOverrides,i,t.calendarApi),o=function(e,t){switch(t.type){case"CHANGE_VIEW_TYPE":e=t.viewType}return e}(n.cu
rrentViewType,e),a=this.computeCurrentViewData(o,s,t.optionOverrides,i);t.calendarApi.currentDataManager=this,r.setThisContext(t.calendarApi),r.setOptions(a.options);let l={d
ateEnv:s.dateEnv,options:s.calendarOptions,pluginHooks:s.pluginHooks,calendarApi:t.calendarApi,dispatch:this.dispatch,emitter:r,getCurrentData:this.getCurrentData},{currentDa
te:c,dateProfile:d}=n;this.data&&this.data.dateProfileGenerator!==a.dateProfileGenerator&&(d=a.dateProfileGenerator.build(c)),c=function(e,t){switch(t.type){case"CHANGE_DATE"
:return t.dateMarker;default:return e}}(c,e),d=function(e,t,n,r){let i;switch(t.type){case"CHANGE_VIEW_TYPE":return r.build(t.dateMarker||n);case"CHANGE_DATE":return
r.build(t.dateMarker);case"PREV":if(i=r.buildPrev(e,n),i.isValid)return i;break;case"NEXT":if(i=r.buildNext(e,n),i.isValid)return i}return
e}(d,e,c,a.dateProfileGenerator),"PREV"!==e.type&&"NEXT"!==e.type&&lr(d.currentRange,c)||(c=d.currentRange.start);let u=Qo(n.eventSources,e,d,l),h=Ur(n.eventStore,e,u,d,l),f=
qo(u)&&!a.options.progressiveEventRendering&&n.renderableEventStore||h,{eventUiSingleBase:g,selectionConfig:p}=this.buildViewUiProps(l),m=this.buildEventUiBySource(u),v={dyna
micOptionOverrides:i,currentViewType:o,currentDate:c,dateProfile:d,eventSources:u,eventStore:h,renderableEventStore:f,selectionConfig:p,eventUiBases:this.buildEventUiBases(f.
defs,g,m),businessHours:this.parseContextBusinessHours(l),dateSelection:ea(n.dateSelection,e),eventSelection:ta(n.eventSelection,e),eventDrag:na(n.eventDrag,e),eventResize:ra
(n.eventResize,e)},y=Object.assign(Object.assign({},l),v);for(let t of s.pluginHooks.reducers)Object.assign(v,t(n,e,y));let b=Ea(n,l),E=Ea(v,l);!b&&E?r.trigger("loading",!0):
b&&!E&&r.trigger("loading",!1),this.state=v,t.onAction&&t.onAction(e)}updateData(){let{props:e,state:t}=this,n=this.data,r=this.computeOptionsData(e.optionOverrides,t.dynamic
OptionOverrides,e.calendarApi),i=this.computeCurrentViewData(t.currentViewType,r,e.optionOverrides,t.dynamicOptionOverrides),s=this.data=Object.assign(Object.assign(Object.as
sign({viewTitle:this.buildTitle(t.dateProfile,i.options,r.dateEnv),calendarApi:e.calendarApi,dispatch:this.dispatch,emitter:this.emitter,getCurrentData:this.getCurrentData},r
),i),t),o=r.pluginHooks.optionChangeHandlers,a=n&&n.calendarOptions,l=r.calendarOptions;if(a&&a!==l){a.timeZone!==l.timeZone&&(t.eventSources=s.eventSources=function(e,t,n){l
et r=t?t.activeRange:null;return Xo(e,Jo(e,n),r,!0,n)}(s.eventSources,t.dateProfile,s),t.eventStore=s.eventStore=Fr(s.eventStore,n.dateEnv,s.dateEnv),t.renderableEventStore=s
.renderableEventStore=Fr(s.renderableEventStore,n.dateEnv,s.dateEnv));for(let e in o)-1===this.optionsForHandling.indexOf(e)&&a[e]===l[e]||o[e](l[e],s)}this.optionsForHandlin
g=[],e.onData&&e.onData(s)}computeOptionsData(e,t,n){if(!this.optionsForRefining.length&&e===this.stableOptionOverrides&&t===this.stableDynamicOptionOverrides)return
this.stableCalendarOptionsData;let{refinedOptions:r,pluginHooks:i,localeDefaults:s,availableLocaleData:o,extra:a}=this.processRawCalendarOptions(e,t);Aa(a);let l=this.buildDa
teEnv(r.timeZone,r.locale,r.weekNumberCalculation,r.firstDay,r.weekText,i,o,r.defaultRangeSeparator),c=this.buildViewSpecs(i.views,this.stableOptionOverrides,this.stableDynam
icOptionOverrides,s),d=this.buildTheme(r,i),u=this.parseToolbars(r,this.stableOptionOverrides,d,c,n);return this.stableCalendarOptionsData={calendarOptions:r,pluginHooks:i,da
teEnv:l,viewSpecs:c,theme:d,toolbarConfig:u,localeDefaults:s,availableRawLocales:o.map}}processRawCalendarOptions(e,t){let{locales:n,locale:r}=mn([cn,e,t]),i=this.organizeRaw
Locales(n),s=i.map,o=this.buildLocale(r||i.defaultCode,s).options,a=this.buildPluginHooks(e.plugins||[],ca),l=this.currentCalendarOptionsRefiners=Object.assign(Object.assign(
Object.assign(Object.assign(Object.assign({},ln),dn),un),a.listenerRefiners),a.optionRefiners),c={},d=mn([cn,o,e,t]),u={},h=this.currentCalendarOptionsInput,f=this.currentCal
endarOptionsRefined,g=!1;for(let e in d)-1===this.optionsForRefining.indexOf(e)&&(d[e]===h[e]||hn[e]&&e in
h&&hn[e](h[e],d[e]))?u[e]=f[e]:l[e]?(u[e]=l[e](d[e]),g=!0):c[e]=h[e];return g&&(this.currentCalendarOptionsInput=d,this.currentCalendarOptionsRefined=u,this.stableOptionOverr
```

```
ides=e,this.stableDynamicOptionOverrides=t),this.optionsForHandling.push(...this.optionsForRefining),this.optionsForRefining=[],{rawOptions:this.currentCalendarOptionsInput,r
efinedOptions:this.currentCalendarOptionsRefined,pluginHooks:a,availableLocaleData:i,localeDefaults:o,extra:c}}_computeCurrentViewData(e,t,n,r){let
i=t.viewSpecs[e];if(!i)throw new Error(`viewType "${e}" is not available. Please make sure you've loaded all neccessary
plugins`);let{refinedOptions:s,extra:o}=this.processRawViewOptions(i,t.pluginHooks,t.localeDefaults,n,r);return Aa(o),{viewSpec:i,options:s,dateProfileGenerator:this.buildDat
eProfileGenerator({dateProfileGeneratorClass:i.optionDefaults.dateProfileGeneratorClass,duration:i.duration,durationUnit:i.durationUnit,usesMinMaxTime:i.optionDefaults.usesMi
nMaxTime,dateEnv:t.dateEnv,calendarApi:this.props.calendarApi,slotMinTime:s.slotMinTime,slotMaxTime:s.slotMaxTime,showNonCurrentDates:s.showNonCurrentDates,dayCount:s.dayCoun
t,dateAlignment:s.dateAlignment,dateIncrement:s.dateIncrement,hiddenDays:s.hiddenDays,weekends:s.weekends,nowInput:s.now,validRangeInput:s.validRange,visibleRangeInput:s.visi
bleRange,fixedWeekCount:s.fixedWeekCount}),viewApi:this.buildViewApi(e,this.getCurrentData,t.dateEnv)}}processRawViewOptions(e,t,n,r,i){let s=mn([cn,e.optionDefaults,n,r,e.op
tionOverrides,i]),o=Object.assign(Object.assign(Object.assign(Object.assign(Object.assign(Object.assign({},ln),dn),un),pn),t.listenerRefiners),t.optionRefiners),a={},l=this.c
urrentViewOptionsInput,c=this.currentViewOptionsRefined,d=!1,u={};for(let e in
s)s[e]===l[e]||hn[e]&&hn[e](s[e],l[e])?a[e]=c[e]:(s[e]===this.currentCalendarOptionsInput[e]||hn[e]&&hn[e](s[e],this.currentCalendarOptionsInput[e])?e in
this.currentCalendarOptionsRefined&&(a[e]=this.currentCalendarOptionsRefined[e]):o[e]?a[e]=o[e](s[e]):u[e]=s[e],d=!0);return
d&&(this.currentViewOptionsInput=s,this.currentViewOptionsRefined=a),{rawOptions:this.currentViewOptionsInput,refinedOptions:this.currentViewOptionsRefined,extra:u}}}function
 fa(e,t,n,r,i,s,o,a){let l=Oo(t||o.defaultCode,o.map);return new Hn({calendarSystem:"gregory",timeZone:e,namedTimeZoneImpl:s.namedTimeZonedImpl,locale:l,weekNumberCalculation
:n,firstDay:r,weekText:i,cmdFormatter:s.cmdFormatter,defaultSeparator:a})}function ga(e,t){return new(t.themeClasses[e.themeSystem]||jo)(e)}function pa(e){return
new(e.dateProfileGeneratorClass||gr)(e)}function ma(e,t,n){return new aa(e,t,n)}function va(e){return An(e,e=>e.ui)}function ya(e,t,n){let r={"":t};for(let t in e){let
i=e[t];i.sourceId&&n[i.sourceId]&&(r[t]=n[i.sourceId])}return r}function ba(e){let{options:t}=e;return{eventUiSingleBase:Nr({display:t.eventDisplay,editable:t.editable,startE
ditable:t.eventStartEditable,durationEditable:t.eventDurationEditable,constraint:t.eventConstraint,overlap:"boolean"==typeof t.eventOverlap?t.eventOverlap:void 0,allow:t.even
tAllow,backgroundColor:t.eventBackgroundColor,borderColor:t.eventBorderColor,textColor:t.eventTextColor,color:t.eventColor},e),selectionConfig:Nr({constraint:t.selectConstrai
nt,overlap:"boolean"==typeof t.selectOverlap?t.selectOverlap:void 0,allow:t.selectAllow},e)}}function Ea(e,t){for(let n of
t.pluginHooks.isLoadingFuncs)if(n(e))return!0;return!1}function Sa(e){return qr(e.options.businessHours,e)}function Aa(e,t){for(let n in e)console.warn(`Unknown option
'${n}'`+(t?` for view '${t}'`:""))}class Da extends Gn{render(){return p("div",{className:"fc-toolbar-chunk"},...this.props.widgetGroups.map(e=>this.renderWidgetGroup(e)))}re
nderWidgetGroup(e){let{props:t}=this,{theme:n}=this.context,r=[],i=!0;for(let s of
e){let{buttonName:e,buttonClick:o,buttonText:a,buttonIcon:l,buttonHint:c}=s;if("title"===e)i=!1,r.push(p("h2",{className:"fc-toolbar-title",id:t.titleId},t.title));else{let i
=e===t.activeButton,s=!t.isTodayEnabled&&"today"===e||!t.isPrevEnabled&&"prev"===e||!t.isNextEnabled&&"next"===e,d=[`fc-${e}-button`,n.getClass("button")];i&&d.push(n.getClas
s("buttonActive")),r.push(p("button",{type:"button",title:"function"==typeof c?c(t.navUnit):c,disabled:s,"aria-pressed":i,className:d.join("
"),onClick:o},a||(l?p("span",{className:l,role:"img"}):""))}}if(r.length>1){return p("div",{className:i&&n.getClass("buttonGroup")||""},...r)}return r[0]}}class wa extends
Gn{render(){let e,t,{model:n,extraClassName:r}=this.props,i=!1,s=n.sectionWidgets,o=s.center;return
s.left?(i=!0,e=s.left):e=s.start,s.right?(i=!0,t=s.right):t=s.end,p("div",{className:[r||"","fc-toolbar",i?"fc-toolbar-ltr":""].join("
")},this.renderSection("start",e||[]),this.renderSection("center",o||[]),this.renderSection("end",t||[]))}renderSection(e,t){let{props:n}=this;return p(Da,{key:e,widgetGroups
:t,title:n.title,navUnit:n.navUnit,activeButton:n.activeButton,isTodayEnabled:n.isTodayEnabled,isPrevEnabled:n.isPrevEnabled,isNextEnabled:n.isNextEnabled,titleId:n.titleId})
}}class Ca extends Gn{constructor(){super(...arguments),this.state={availableWidth:null},this.handleEl=e=>{this.el=e,Qn(this.props.elRef,e),this.updateAvailableWidth()},this.
handleResize=()=>{this.updateAvailableWidth()}}render(){let{props:e,state:t}=this,{aspectRatio:n}=e,r=["fc-view-harness",n||e.liquid||e.height?"fc-view-harness-active":"fc-vi
ew-harness-passive"],i="",s="";return
n?null!==t.availableWidth?i=t.availableWidth/n:s=1/n*100+"%":i=e.height||"",p("div",{"aria-labelledby":e.labeledById,ref:this.handleEl,className:r.join(" "),style:{height:i,p
addingBottom:s}},e.children)}componentDidMount(){this.context.addResizeHandler(this.handleResize)}componentWillUnmount(){this.context.removeResizeHandler(this.handleResize)}u
pdateAvailableWidth(){this.el&&this.props.aspectRatio&&this.setState({availableWidth:this.el.offsetWidth})}}class Ra extends
Mi{constructor(e){super(e),this.handleSegClick=(e,t)=>{let{component:n}=this,{context:r}=n,i=ai(t);if(i&&n.isValidSegDownEl(e.target)){let
s=Oe(e.target,".fc-event-forced-url"),o=s?s.querySelector("a[href]").href:"";r.emitter.trigger("eventClick",{el:t,event:new ti(n.context,i.eventRange.def,i.eventRange.instanc
e),jsEvent:e,view:r.viewApi}),o&&!e.defaultPrevented&&(window.location.href=o)}},this.destroy=Fe(e.el,"click",".fc-event",this.handleSegClick)}}class xa extends Mi{constructo
r(e){super(e),this.handleEventElRemove=e=>{e===this.currentSegEl&&this.handleSegLeave(null,this.currentSegEl)},this.handleSegEnter=(e,t)=>{ai(t)&&(this.currentSegEl=t,this.tr
iggerEvent("eventMouseEnter",e,t))},this.handleSegLeave=(e,t)=>{this.currentSegEl&&(this.currentSegEl=null,this.triggerEvent("eventMouseLeave",e,t))},this.removeHoverListener
s=function(e,t,n,r){let i;return Fe(e,"mouseover",t,(e,t)=>{if(t!==i){i=t,n(e,t);let s=e=>{i=null,r(e,t),t.removeEventListener("mouseleave",s)};t.addEventListener("mouseleave
",s)}})}(e.el,".fc-event",this.handleSegEnter,this.handleSegLeave)}destroy(){this.removeHoverListeners()}triggerEvent(e,t,n){let{component:r}=this,{context:i}=r,s=ai(n);t&&!r
.isValidSegDownEl(t.target)||i.emitter.trigger(e,{el:n,event:new ti(i,s.eventRange.def,s.eventRange.instance),jsEvent:t,view:i.viewApi})}}class _a extends Vn{constructor(){su
per(...arguments),this.buildViewContext=Gt(Fn),this.buildViewPropTransformers=Gt(ka),this.buildToolbarProps=Gt(Ta),this.headerRef={current:null},this.footerRef={current:null}
,this.interactionsStore={},this.state={viewLabelId:We()},this.registerInteractiveComponent=(e,t)=>{let n=function(e,t){return{component:e,el:t.el,useEventCenter:null==t.useEv
entCenter||t.useEventCenter,isHitComboAllowed:t.isHitComboAllowed||null}}(e,t),r=[Ra,xa].concat(this.props.pluginHooks.componentInteractions).map(e=>new
e(n));this.interactionsStore[e.uid]=r,Oi[e.uid]=n},this.unregisterInteractiveComponent=e=>{let t=this.interactionsStore[e.uid];if(t){for(let e of t)e.destroy();delete
this.interactionsStore[e.uid]}delete Oi[e.uid]},this.resizeRunner=new Me(()=>{this.props.emitter.trigger("_resize",!0),this.props.emitter.trigger("windowResize",{view:this.pr
ops.viewApi})}),this.handleWindowResize=e=>{let{options:t}=this.props;t.handleWindowResize&&e.target===window&&this.resizeRunner.request(t.windowResizeDelay)}}render(){let e,
{props:t}=this,{toolbarConfig:n,options:r}=t,i=this.buildToolbarProps(t.viewSpec,t.dateProfile,t.dateProfileGenerator,t.currentDate,fr(t.options.now,t.dateEnv),t.viewTitle),s
=!1,o="";t.isHeightAuto||t.forPrint?o="":null!=r.height?s=!0:null!=r.contentHeight?o=r.contentHeight:e=Math.max(r.aspectRatio,.5);let a=this.buildViewContext(t.viewSpec,t.vie
wApi,t.options,t.dateProfileGenerator,t.dateEnv,t.theme,t.pluginHooks,t.dispatch,t.getCurrentData,t.emitter,t.calendarApi,this.registerInteractiveComponent,this.unregisterInt
eractiveComponent),l=n.header&&n.header.hasTitle?this.state.viewLabelId:void 0;return p(Ln.Provider,{value:a},n.header&&p(wa,Object.assign({ref:this.headerRef,extraClassName:
"fc-header-toolbar",model:n.header,titleId:l},i)),p(Ca,{liquid:s,height:o,aspectRatio:e,labeledById:l},this.renderView(t),this.buildAppendContent()),n.footer&&p(wa,Object.ass
ign({ref:this.footerRef,extraClassName:"fc-footer-toolbar",model:n.footer,titleId:""},i))}componentDidMount(){let{props:e}=this;this.calendarInteractions=e.pluginHooks.calen
darInteractions.map(t=>new t(e)),window.addEventListener("resize",this.handleWindowResize);let{propSetHandlers:t}=e.pluginHooks;for(let n in
t)t[n](e[n],e)}componentDidUpdate(e){let{props:t}=this,{propSetHandlers:n}=t.pluginHooks;for(let r in
n)t[r]!==e[r]&&n[r](t[r],t)}componentWillUnmount(){window.removeEventListener("resize",this.handleWindowResize),this.resizeRunner.clear();for(let e of
this.calendarInteractions)e.destroy();this.props.emitter.trigger("_unmount")}buildAppendContent(){let{props:e}=this;return p(y,{},...e.pluginHooks.viewContainerAppends.map(t=
>t(e)))}renderView(e){let{pluginHooks:t}=e,{viewSpec:n}=e,r={dateProfile:e.dateProfile,businessHours:e.businessHours,eventStore:e.renderableEventStore,eventUiBases:e.eventUiB
ases,dateSelection:e.dateSelection,eventSelection:e.eventSelection,eventDrag:e.eventDrag,eventResize:e.eventResize,isHeightAuto:e.isHeightAuto,forPrint:e.forPrint},i=this.bui
ldViewPropTransformers(t.viewPropsTransformers);for(let t of i)Object.assign(r,t.transform(r,e));return p(n.component,Object.assign({},r))}}function Ta(e,t,n,r,i,s){let
o=n.build(i,void 0,!1),a=n.buildPrev(t,r,!1),l=n.buildNext(t,r,!1);return{title:s,activeButton:e.type,navUnit:e.singleUnit,isTodayEnabled:o.isValid&&!lr(t.currentRange,i),isP
revEnabled:a.isValid,isNextEnabled:l.isValid}}function ka(e){return e.map(e=>new e)}function Ma(e){let t=Oo(e.locale||"en",Io([]).map);return new
Hn(Object.assign(Object.assign({timeZone:cn.timeZone,calendarSystem:"gregory"},e),{locale:t}))}vs.touchMouseIgnoreWait=500;let Ia=0,Oa=0,Na=!1;class Pa{constructor(e){this.su
bjectEl=null,this.selector="",this.handleSelector="",this.shouldIgnoreMove=!1,this.shouldWatchScroll=!0,this.isDragging=!1,this.isTouchDragging=!1,this.wasTouchScroll=!1,this
.handleMouseDown=e=>{if(!this.shouldIgnoreMouse()&&function(e){return 0===e.button&&!e.ctrlKey}(e)&&this.tryStart(e)){let t=this.createEventFromMouse(e,!0);this.emitter.trigg
er("pointerdown",t),this.initScrollWatch(t),this.shouldIgnoreMove||document.addEventListener("mousemove",this.handleMouseMove),document.addEventListener("mouseup",this.handle
MouseUp)}},this.handleMouseMove=e=>{let t=this.createEventFromMouse(e);this.recordCoords(t),this.emitter.trigger("pointermove",t)},this.handleMouseUp=e=>{document.removeEvent
Listener("mousemove",this.handleMouseMove),document.removeEventListener("mouseup",this.handleMouseUp),this.emitter.trigger("pointerup",this.createEventFromMouse(e)),this.clea
nup()},this.handleTouchStart=e=>{if(this.tryStart(e)){this.isTouchDragging=!0;let
t=this.createEventFromTouch(e,!0);this.emitter.trigger("pointerdown",t),this.initScrollWatch(t);let n=e.target;this.shouldIgnoreMove||n.addEventListener("touchmove",this.hand
```

```
leTouchMove),n.addEventListener("touchend",this.handleTouchEnd),n.addEventListener("touchcancel",this.handleTouchEnd),window.addEventListener("scroll",this.handleTouchScroll,
!0)}},this.handleTouchMove=e=>{let t=this.createEventFromTouch(e);this.recordCoords(t),this.emitter.trigger("pointermove",t)},this.handleTouchEnd=e=>{if(this.isDragging){let
t=e.target;t.removeEventListener("touchmove",this.handleTouchMove),t.removeEventListener("touchend",this.handleTouchEnd),t.removeEventListener("touchcancel",this.handleTouchE
nd),window.removeEventListener("scroll",this.handleTouchScroll,!0),this.emitter.trigger("pointerup",this.createEventFromTouch(e)),this.cleanup(),this.isTouchDragging=!1,Ia+=1
,setTimeout(()=>{Ia-=1},vs.touchMouseIgnoreWait)}},this.handleTouchScroll=()=>{this.wasTouchScroll=!0},this.handleScroll=e=>{if(!this.shouldIgnoreMove){let t=window.pageXOffs
et-this.prevScrollX+this.prevPageX,n=window.pageYOffset-this.prevScrollY+this.prevPageY;this.emitter.trigger("pointermove",{origEvent:e,isTouch:this.isTouchDragging,subjectEl
:this.subjectEl,pageX:t,pageY:n,deltaX:t-this.origPageX,deltaY:n-this.origPageY})}},this.containerEl=e,this.emitter=new Gr,e.addEventListener("mousedown",this.handleMouseDown
),e.addEventListener("touchstart",this.handleTouchStart,{passive:!0}),Oa+=1,1===Oa&&window.addEventListener("touchmove",Ha,{passive:!1})}destroy(){this.containerEl.removeEven
tListener("mousedown",this.handleMouseDown),this.containerEl.removeEventListener("touchstart",this.handleTouchStart,{passive:!0}),Oa-=1,Oa||window.removeEventListener("touchm
ove",Ha,{passive:!1})}tryStart(e){let t=this.querySubjectEl(e),n=e.target;return!(!t||this.handleSelector&&!Oe(n,this.handleSelector))&&(this.subjectEl=t,this.isDragging=!0,t
his.wasTouchScroll=!1,!0)}cleanup(){Na=!1,this.isDragging=!1,this.subjectEl=null,this.destroyScrollWatch()}querySubjectEl(e){return
this.selector?Oe(e.target,this.selector):this.containerEl}shouldIgnoreMouse(){return Ia||this.isTouchDragging}cancelTouchScroll(){this.isDragging&&(Na=!0)}initScrollWatch(e){
this.shouldWatchScroll&&(this.recordCoords(e),window.addEventListener("scroll",this.handleScroll,!0))}recordCoords(e){this.shouldWatchScroll&&(this.prevPageX=e.pageX,this.pre
vPageY=e.pageY,this.prevScrollX=window.pageXOffset,this.prevScrollY=window.pageYOffset)}destroyScrollWatch(){this.shouldWatchScroll&&window.removeEventListener("scroll",this.
handleScroll,!0)}createEventFromMouse(e,t){let n=0,r=0;return t?(this.origPageX=e.pageX,this.origPageY=e.pageY):(n=e.pageX-this.origPageX,r=e.pageY-this.origPageY),{origEvent
:e,isTouch:!1,subjectEl:this.subjectEl,pageX:e.pageX,pageY:e.pageY,deltaX:n,deltaY:r}}createEventFromTouch(e,t){let n,r,i=e.touches,s=0,o=0;return i&&i.length?(n=i[0].pageX,r
=i[0].pageY):(n=e.pageX,r=e.pageY),t?(this.origPageX=n,this.origPageY=r):(s=n-this.origPageX,o=r-this.origPageY),{origEvent:e,isTouch:!0,subjectEl:this.subjectEl,pageX:n,page
Y:r,deltaX:s,deltaY:o}}}function Ha(e){Na&&e.preventDefault()}class Ba{constructor(){this.isVisible=!1,this.sourceEl=null,this.mirrorEl=null,this.sourceElRect=null,this.paren
tNode=document.body,this.zIndex=9999,this.revertDuration=0}start(e,t,n){this.sourceEl=e,this.sourceElRect=this.sourceEl.getBoundingClientRect(),this.origScreenX=t-window.page
XOffset,this.origScreenY=n-window.pageYOffset,this.deltaX=0,this.deltaY=0,this.updateElPosition()}handleMove(e,t){this.deltaX=e-window.pageXOffset-this.origScreenX,this.delta
Y=t-window.pageYOffset-this.origScreenY,this.updateElPosition()}setIsVisible(e){e?this.isVisible||(this.mirrorEl&&(this.mirrorEl.style.display=""),this.isVisible=e,this.updat
eElPosition()):this.isVisible&&(this.mirrorEl&&(this.mirrorEl.style.display="none"),this.isVisible=e)}stop(e,t){let n=()=>{this.cleanup(),t()};e&&this.mirrorEl&&this.isVisibl
e&&this.revertDuration&&(this.deltaX||this.deltaY)?this.doRevertAnimation(n,this.revertDuration):setTimeout(n,0)}doRevertAnimation(e,t){let
n=this.mirrorEl,r=this.sourceEl.getBoundingClientRect();n.style.transition="top "+t+"ms,left "+t+"ms",Be(n,{left:r.left,top:r.top}),Ge(n,()=>{n.style.transition="",e()})}clea
nup(){this.mirrorEl&&(Ie(this.mirrorEl),this.mirrorEl=null),this.sourceEl=null}updateElPosition(){this.sourceEl&&this.isVisible&&Be(this.getMirrorEl(),{left:this.sourceElRect
.left+this.deltaX,top:this.sourceElRect.top+this.deltaY})}getMirrorEl(){let e=this.sourceElRect,t=this.mirrorEl;return t||(t=this.mirrorEl=this.sourceEl.cloneNode(!0),t.style
.userSelect="none",t.style.webkitUserSelect="none",t.classList.add("fc-event-dragging"),Be(t,{position:"fixed",zIndex:this.zIndex,visibility:"",boxSizing:"border-box",width:e
.right-e.left,height:e.bottom-e.top,right:"auto",bottom:"auto",margin:0}),this.parentNode.appendChild(t)),t}}class ja extends ss{constructor(e,t){super(),this.handleScroll=()
=>{this.scrollTop=this.scrollController.getScrollTop(),this.scrollLeft=this.scrollController.getScrollLeft(),this.handleScrollChange()},this.scrollController=e,this.doesListe
ning=t,this.scrollTop=this.origScrollTop=e.getScrollTop(),this.scrollLeft=this.origScrollLeft=e.getScrollLeft(),this.scrollWidth=e.getScrollWidth(),this.scrollHeight=e.getScr
ollHeight(),this.clientWidth=e.getClientWidth(),this.clientHeight=e.getClientHeight(),this.clientRect=this.computeClientRect(),this.doesListening&&this.getEventTarget().addEv
entListener("scroll",this.handleScroll)}destroy(){this.doesListening&&this.getEventTarget().removeEventListener("scroll",this.handleScroll)}getScrollTop(){return
this.scrollTop}getScrollLeft(){return this.scrollLeft}setScrollTop(e){this.scrollController.setScrollTop(e),this.doesListening||(this.scrollTop=Math.max(Math.min(e,this.getMa
xScrollTop()),0),this.handleScrollChange())}setScrollLeft(e){this.scrollController.setScrollLeft(e),this.doesListening||(this.scrollLeft=Math.max(Math.min(e,this.getMaxScroll
Left()),0),this.handleScrollChange())}getClientWidth(){return this.clientWidth}getClientHeight(){return this.clientHeight}getScrollWidth(){return
this.scrollWidth}getScrollHeight(){return this.scrollHeight}handleScrollChange(){}}class za extends ja{constructor(e,t){super(new os(e),t)}getEventTarget(){return
this.scrollController.el}computeClientRect(){return es(this.scrollController.el)}}class Ua extends ja{constructor(e){super(new as,e)}getEventTarget(){return window}computeCli
entRect(){return{left:this.scrollLeft,right:this.scrollLeft+this.clientWidth,top:this.scrollTop,bottom:this.scrollTop+this.clientHeight}}handleScrollChange(){this.clientRect=
this.computeClientRect()}}const Wa="function"==typeof performance?performance.now:Date.now;class La{constructor(){this.isEnabled=!0,this.scrollQuery=[window,".fc-scroller"],t
his.edgeThreshold=50,this.maxVelocity=300,this.pointerScreenX=null,this.pointerScreenY=null,this.isAnimating=!1,this.scrollCaches=null,this.everMovedUp=!1,this.everMovedDown=
!1,this.everMovedLeft=!1,this.everMovedRight=!1,this.animate=()=>{if(this.isAnimating){let
e=this.computeBestEdge(this.pointerScreenX+window.pageXOffset,this.pointerScreenY+window.pageYOffset);if(e){let
t=Wa();this.handleSide(e,(t-this.msSinceRequest)/1e3),this.requestAnimation(t)}else this.isAnimating=!1}}}start(e,t,n){this.isEnabled&&(this.scrollCaches=this.buildCaches(n),
this.pointerScreenX=null,this.pointerScreenY=null,this.everMovedUp=!1,this.everMovedDown=!1,this.everMovedLeft=!1,this.everMovedRight=!1,this.handleMove(e,t))}handleMove(e,t)
{if(this.isEnabled){let n=e-window.pageXOffset,r=t-window.pageYOffset,i=null===this.pointerScreenY?0:r-this.pointerScreenY,s=null===this.pointerScreenX?0:n-this.pointerScreen
X;i<0?this.everMovedUp=!0:i>0&&(this.everMovedDown=!0),s<0?this.everMovedLeft=!0:s>0&&(this.everMovedRight=!0),this.pointerScreenX=n,this.pointerScreenY=r,this.isAnimating||(
this.isAnimating=!0,this.requestAnimation(Wa()))}}stop(){if(this.isEnabled){this.isAnimating=!1;for(let e of this.scrollCaches)e.destroy();this.scrollCaches=null}}requestAnim
ation(e){this.msSinceRequest=e,requestAnimationFrame(this.animate)}handleSide(e,t){let{scrollCache:n}=e,{edgeThreshold:r}=this,i=r-e.distance,s=i*i/(r*r)*this.maxVelocity*t,o
=1;switch(e.name){case"left":o=-1;case"right":n.setScrollLeft(n.getScrollLeft()+s*o);break;case"top":o=-1;case"bottom":n.setScrollTop(n.getScrollTop()+s*o)}}computeBestEdge(e
,t){let{edgeThreshold:n}=this,r=null,i=this.scrollCaches||[];for(let s of i){let i=s.clientRect,o=e-i.left,a=i.right-e,l=t-i.top,c=i.bottom-t;o>=0&&a>=0&&l>=0&&c>=0&&(l<=n&&t
his.everMovedUp&&s.canScrollUp()&&(!r||r.distance>l)&&(r={scrollCache:s,name:"top",distance:l}),c<=n&&this.everMovedDown&&s.canScrollDown()&&(!r||r.distance>c)&&(r={scrollCac
he:s,name:"bottom",distance:c}),o<=n&&this.everMovedLeft&&s.canScrollLeft()&&(!r||r.distance>o)&&(r={scrollCache:s,name:"left",distance:o}),a<=n&&this.everMovedRight&&s.canSc
rollRight()&&(!r||r.distance>a)&&(r={scrollCache:s,name:"right",distance:a}))}return r}buildCaches(e){return this.queryScrollEls(e).map(e=>e===window?new Ua(!1):new
za(e,!1))}queryScrollEls(e){let t=[];for(let n of this.scrollQuery)"object"==typeof
n?t.push(n):t.push(...Array.prototype.slice.call(e.getRootNode().querySelectorAll(n)));return t}}class Fa extends ms{constructor(e,t){super(e),this.containerEl=e,this.delay=n
ull,this.minDistance=0,this.touchScrollAllowed=!0,this.mirrorNeedsRevert=!1,this.isInteracting=!1,this.isDragging=!1,this.isDelayEnded=!1,this.isDistanceSurpassed=!1,this.del
ayTimeoutId=null,this.onPointerDown=e=>{this.isDragging||(this.isInteracting=!0,this.isDelayEnded=!1,this.isDistanceSurpassed=!1,Je(document.body),et(document.body),e.isTouch
||e.origEvent.preventDefault(),this.emitter.trigger("pointerdown",e),this.isInteracting&&!this.pointer.shouldIgnoreMove&&(this.mirror.setIsVisible(!1),this.mirror.start(e.sub
jectEl,e.pageX,e.pageY),this.startDelay(e),this.minDistance||this.handleDistanceSurpassed(e)))},this.onPointerMove=e=>{if(this.isInteracting){if(this.emitter.trigger("pointer
move",e),!this.isDistanceSurpassed){let t,n=this.minDistance,{deltaX:r,deltaY:i}=e;t=r*r+i*i,t>=n*n&&this.handleDistanceSurpassed(e)}this.isDragging&&("scroll"!==e.origEvent.
type&&(this.mirror.handleMove(e.pageX,e.pageY),this.autoScroller.handleMove(e.pageX,e.pageY)),this.emitter.trigger("dragmove",e))}},this.onPointerUp=e=>{this.isInteracting&&(
this.isInteracting=!1,Ke(document.body),tt(document.body),this.emitter.trigger("pointerup",e),this.isDragging&&(this.autoScroller.stop(),this.tryStopDrag(e)),this.delayTimeou
tId&&(clearTimeout(this.delayTimeoutId),this.delayTimeoutId=null))};let n=this.pointer=new Pa(e);n.emitter.on("pointerdown",this.onPointerDown),n.emitter.on("pointermove",thi
s.onPointerMove),n.emitter.on("pointerup",this.onPointerUp),t&&(n.selector=t),this.mirror=new Ba,this.autoScroller=new
La}destroy(){this.pointer.destroy(),this.onPointerUp({})}startDelay(e){"number"==typeof this.delay?this.delayTimeoutId=setTimeout(()=>{this.delayTimeoutId=null,this.handleDel
ayEnd(e)},this.delay):this.handleDelayEnd(e)}handleDelayEnd(e){this.isDelayEnded=!0,this.tryStartDrag(e)}handleDistanceSurpassed(e){this.isDistanceSurpassed=!0,this.tryStartD
rag(e)}tryStartDrag(e){this.isDelayEnded&&this.isDistanceSurpassed&&(this.pointer.wasTouchScroll&&!this.touchScrollAllowed||(this.isDragging=!0,this.mirrorNeedsRevert=!1,this
.autoScroller.start(e.pageX,e.pageY,this.containerEl),this.emitter.trigger("dragstart",e),!1===this.touchScrollAllowed&&this.pointer.cancelTouchScroll()))}tryStopDrag(e){this
.mirror.stop(this.mirrorNeedsRevert,this.stopDrag.bind(this,e))}stopDrag(e){this.isDragging=!1,this.emitter.trigger("dragend",e)}setIgnoreMove(e){this.pointer.shouldIgnoreMov
e=e}setMirrorIsVisible(e){this.mirror.setIsVisible(e)}setMirrorNeedsRevert(e){this.mirrorNeedsRevert=e}setAutoScrollEnabled(e){this.autoScroller.isEnabled=e}}class
Va{constructor(e){this.origRect=ts(e),this.scrollCaches=ns(e).map(e=>new za(e,!0))}destroy(){for(let e of this.scrollCaches)e.destroy()}computeLeft(){let
e=this.origRect.left;for(let t of this.scrollCaches)e+=t.origScrollLeft-t.getScrollLeft();return e}computeTop(){let e=this.origRect.top;for(let t of
this.scrollCaches)e+=t.origScrollTop-t.getScrollTop();return e}isWithinClipping(e,t){let n={left:e,top:t};for(let e of
this.scrollCaches)if(!Ga(e.getEventTarget())&&!Pi(n,e.clientRect))return!1;return!0}}function Ga(e){let t=e.tagName;return"HTML"===t||"BODY"===t}class Qa{constructor(e,t){thi
```

```
s.useSubjectCenter=!1,this.requireInitial=!0,this.initialHit=null,this.movingHit=null,this.finalHit=null,this.handlePointerDown=e=>{let{dragging:t}=this;this.initialHit=null,
this.movingHit=null,this.finalHit=null,this.prepareHits(),this.processFirstCoord(e),this.initialHit||!this.requireInitial?(t.setIgnoreMove(!1),this.emitter.trigger("pointerdo
wn",e)):t.setIgnoreMove(!0)},this.handleDragStart=e=>{this.emitter.trigger("dragstart",e),this.handleMove(e,!0)},this.handleDragMove=e=>{this.emitter.trigger("dragmove",e),th
is.handleMove(e)},this.handlePointerUp=e=>{this.releaseHits(),this.emitter.trigger("pointerup",e)},this.handleDragEnd=e=>{this.movingHit&&this.emitter.trigger("hitupdate",nul
l,!0,e),this.finalHit=this.movingHit,this.movingHit=null,this.emitter.trigger("dragend",e)},this.droppableStore=t,e.emitter.on("pointerdown",this.handlePointerDown),e.emitter
.on("dragstart",this.handleDragStart),e.emitter.on("dragmove",this.handleDragMove),e.emitter.on("pointerup",this.handlePointerUp),e.emitter.on("dragend",this.handleDragEnd),t
his.dragging=e,this.emitter=new Gr}processFirstCoord(e){let t,n={left:e.pageX,top:e.pageY},r=n,i=e.subjectEl;i instanceof HTMLElement&&(t=ts(i),r=Bi(r,t));let
s=this.initialHit=this.queryHitForOffset(r.left,r.top);if(s){if(this.useSubjectCenter&&t){let e=Hi(t,s.rect);e&&(r=ji(e))}this.coordAdjust=zi(r,n)}else
this.coordAdjust={left:0,top:0}}handleMove(e,t){let n=this.queryHitForOffset(e.pageX+this.coordAdjust.left,e.pageY+this.coordAdjust.top);!t&&qa(this.movingHit,n)||(this.movin
gHit=n,this.emitter.trigger("hitupdate",n,!1,e))}prepareHits(){this.offsetTrackers=An(this.droppableStore,e=>(e.component.prepareHits(),new
Va(e.el)))}releaseHits(){let{offsetTrackers:e}=this;for(let t in
e)e[t].destroy();this.offsetTrackers={}}queryHitForOffset(e,t){let{droppableStore:n,offsetTrackers:r}=this,i=null;for(let s in n){let
o=n[s].component,a=r[s];if(a&&a.isWithinClipping(e,t)){let
n=a.computeLeft(),r=a.computeTop(),l=e-n,c=t-r,{origRect:d}=a,u=d.right-d.left,h=d.bottom-d.top;if(l>=0&&l<u&&c>=0&&c<h){let e=o.queryHit(l,c,u,h);e&&ar(e.dateProfile.activeR
ange,e.dateSpan.range)&&(!i||e.layer>i.layer)&&(e.componentId=s,e.context=o.context,e.rect.left+=n,e.rect.right+=n,e.rect.top+=r,e.rect.bottom+=r,i=e)}}}return i}}function
qa(e,t){return!e&&!t||Boolean(e)===Boolean(t)&&Ai(e.dateSpan,t.dateSpan)}function Ya(e,t){let n={};for(let r of t.pluginHooks.datePointTransforms)Object.assign(n,r(e,t));var
r,i;return Object.assign(n,(r=e,{date:(i=t.dateEnv).toDate(r.range.start),dateStr:i.formatIso(r.range.start,{omitTime:r.allDay}),allDay:r.allDay}))},n}class Za extends Mi{cons
tructor(e){super(e),this.subjectEl=null,this.subjectSeg=null,this.isDragging=!1,this.eventRange=null,this.relevantEvents=null,this.receivingContext=null,this.validMutation=nu
ll,this.mutatedRelevantEvents=null,this.handlePointerDown=e=>{let
t=e.origEvent.target,{component:n,dragging:r}=this,{mirror:i}=r,{options:s}=n.context,o=n.context;this.subjectEl=e.subjectEl;let a=this.subjectSeg=ai(e.subjectEl),l=(this.eve
ntRange=a.eventRange).instance.instanceId;this.relevantEvents=xr(o.getCurrentData().eventStore,l),r.minDistance=e.isTouch?0:s.eventDragMinDistance,r.delay=e.isTouch&&l!==n.pr
ops.eventSelection?function(e){let{options:t}=e.context,n=t.eventLongPressDelay;null==n&&(n=t.longPressDelay);return
n}(n):null,s.fixedMirrorParent?i.parentNode=s.fixedMirrorParent:i.parentNode=Oe(t,".fc"),i.revertDuration=s.dragRevertDuration;let
c=n.isValidSegDownEl(t)&&!Oe(t,".fc-event-resizer");r.setIgnoreMove(!c),this.isDragging=c&&e.subjectEl.classList.contains("fc-event-draggable")},this.handleDragStart=e=>{let
t=this.component.context,n=this.eventRange,r=n.instance.instanceId;e.isTouch?r!==this.component.props.eventSelection&&t.dispatch({type:"SELECT_EVENT",eventInstanceId:r}):t.di
spatch({type:"UNSELECT_EVENT"}),this.isDragging&&(t.calendarApi.unselect(e),t.emitter.trigger("eventDragStart",{el:this.subjectEl,event:new
ti(t,n.def,n.instance),jsEvent:e.origEvent,view:t.viewApi}))},this.handleHitUpdate=(e,t)=>{if(!this.isDragging)return;let n=this.relevantEvents,r=this.hitDragging.initialHit,
i=this.component.context,s=null,o=null,a=null,l=!1,c={affectedEvents:n,mutatedEvents:{defs:{},instances:{}},isEvent:!0};if(e){s=e.context;let
t=s.options;i===s||t.editable&&t.droppable?(o=function(e,t,n){let r=e.dateSpan,i=t.dateSpan,s=r.range.start,o=i.range.start,a={};r.allDay!==i.allDay&&(a.allDay=i.allDay,a.has
End=t.context.options.allDayMaintainDuration,i.allDay&&(s=Mt(s)));let
l=hr(s,o,e.context.dateEnv,e.componentId===t.componentId?e.largeUnit:null);l.milliseconds&&(a.allDay=!1);let c={datesDelta:l,standardProps:a};for(let r of n)r(c,e,t);return c
}(r,e,s.getCurrentData().pluginHooks.eventDragMutationMassagers),o&&(a=$r(n,s.getCurrentData().eventUiBases,o,s),c.mutatedEvents=a,Ns(c,e.dateProfile,s)||(l=!0,o=null,a=null,
c.mutatedEvents={defs:{},instances:{}}))):s=null}this.displayDrag(s,c),l?Xe():$e(),t||(i===s&&qa(r,e)&&(o=null),this.dragging.setMirrorNeedsRevert(!o),this.dragging.setMirror
IsVisible(!e||!this.subjectEl.getRootNode().querySelector(".fc-event-mirror")),this.receivingContext=s,this.validMutation=o,this.mutatedRelevantEvents=a)},this.handlePointerU
p=()=>{this.isDragging||this.cleanup()},this.handleDragEnd=e=>{if(this.isDragging){let
t=this.component.context,n=t.viewApi,{receivingContext:r,validMutation:i}=this,s=this.eventRange.def,o=this.eventRange.instance,a=new ti(t,s,o),l=this.relevantEvents,c=this.m
utatedRelevantEvents,{finalHit:d}=this.hitDragging;if(this.clearDrag(),t.emitter.trigger("eventDragStop",{el:this.subjectEl,event:a,jsEvent:e.origEvent,view:n}),i){if(r===t){
let r=new ti(t,c.defs[s.defId],o?c.instances[o.instanceId]:null);t.dispatch({type:"MERGE_EVENTS",eventStore:c});let
d={oldEvent:a,event:r,relatedEvents:ri(c,t,o),revert(){t.dispatch({type:"MERGE_EVENTS",eventStore:l})}},u={};for(let e of t.getCurrentData().pluginHooks.eventDropTransformers
)Object.assign(u,e(i,t));t.emitter.trigger("eventDrop",Object.assign(Object.assign(Object.assign({},d),u),{el:e.subjectEl,delta:i.datesDelta,jsEvent:e.origEvent,view:n})),t.e
mitter.trigger("eventChange",d)}else if(r){let i={event:a,relatedEvents:ri(l,t,o),revert(){t.dispatch({type:"MERGE_EVENTS",eventStore:l})}};t.emitter.trigger("eventLeave",Obj
ect.assign(Object.assign(Object.assign({},i),{draggedEl:e.subjectEl,view:n})),t.dispatch({type:"REMOVE_EVENTS",eventStore:l}),t.emitter.trigger("eventRemove",i);let
u=c.defs[s.defId],h=c.instances[o.instanceId],f=new ti(r,u,h);r.dispatch({type:"MERGE_EVENTS",eventStore:c});let g={event:f,relatedEvents:ri(c,r,h),revert(){r.dispatch({type:
"REMOVE_EVENTS",eventStore:c})}};r.emitter.trigger("eventAdd",g),e.isTouch&&r.dispatch({type:"SELECT_EVENT",eventInstanceId:o.instanceId}),r.emitter.trigger("drop",Object.ass
ign(Object.assign({},Ya(d.dateSpan,r)),{draggedEl:e.subjectEl,jsEvent:e.origEvent,view:d.context.viewApi})),r.emitter.trigger("eventReceive",Object.assign(Object.assign({},g)
,{draggedEl:e.subjectEl,view:d.context.viewApi}))}else t.emitter.trigger("_noEventDrop")}this.cleanup()};let{component:t}=this,{options:n}=t.context,r=this.dragging=new
Fa(e.el);r.pointer.selector=Za.SELECTOR,r.touchScrollAllowed=!1,r.autoScroller.isEnabled=n.dragScroll;let i=this.hitDragging=new Qa(this.dragging,Oi);i.useSubjectCenter=e.use
EventCenter,i.emitter.on("pointerdown",this.handlePointerDown),i.emitter.on("dragstart",this.handleDragStart),i.emitter.on("hitupdate",this.handleHitUpdate),i.emitter.on("poi
nterup",this.handlePointerUp),i.emitter.on("dragend",this.handleDragEnd)}destroy(){this.dragging.destroy()}displayDrag(e,t){let n=this.component.context,r=this.receivingConte
xt;r&&r!==e&&(r===n?r.dispatch({type:"SET_EVENT_DRAG",state:{affectedEvents:t.affectedEvents,mutatedEvents:{defs:{},instances:{}},isEvent:!0}}):r.dispatch({type:"UNSET_EVENT_
DRAG"})),e&&e.dispatch({type:"SET_EVENT_DRAG",state:t})}clearDrag(){let e=this.component.context,{receivingContext:t}=this;t&&t.dispatch({type:"UNSET_EVENT_DRAG"}),e!==t&&e.d
ispatch({type:"UNSET_EVENT_DRAG"})}cleanup(){this.subjectSeg=null,this.isDragging=!1,this.eventRange=null,this.relevantEvents=null,this.receivingContext=null,this.validMutati
on=null,this.mutatedRelevantEvents=null}}Za.SELECTOR=".fc-event-draggable, .fc-event-resizable";const Xa={fixedMirrorParent:yn},$a={dateClick:yn,eventDragStart:yn,eventDragSt
op:yn,eventDrop:yn,eventResizeStart:yn,eventResizeStop:yn,eventResize:yn,drop:yn,eventReceive:yn,eventLeave:yn};class Ja{constructor(e,t){this.receivingContext=null,this.drop
pableEvent=null,this.suppliedDragMeta=null,this.dragMeta=null,this.handleDragStart=e=>{this.dragMeta=this.buildDragMeta(e.subjectEl)},this.handleHitUpdate=(e,t,n)=>{let{dragg
ing:r}=this.hitDragging,i=null,s=null,o=!1,a={affectedEvents:{defs:{},instances:{}},mutatedEvents:{defs:{},instances:{}},isEvent:this.dragMeta.create};e&&(i=e.context,this.ca
nDropElOnCalendar(n.subjectEl,i)&&(s=function(e,t,n){let r=Object.assign({},t.leftoverProps);for(let i of n.pluginHooks.externalDefTransforms)Object.assign(r,i(e,t));let{refi
ned:i,extra:s}=Ar(r,n),o=wr(i,s,t.sourceId,e.allDay,n.options.forceEventDuration||Boolean(t.duration),n),a=e.range.start;e.allDay&&t.startTime&&(a=n.dateEnv.add(a,t.startTime
));let l=t.duration?n.dateEnv.add(a,t.duration):Xr(e.allDay,a,n),c=pr(o.defId,{start:a,end:l});return{def:o,instance:c}}(e.dateSpan,this.dragMeta,i),a.mutatedEvents=Rr(s),o=!
Ns(a,e.dateProfile,i),o&&(a.mutatedEvents={defs:{},instances:{}},s=null))),this.displayDrag(i,a),r.setMirrorIsVisible(t||!s||!document.querySelector(".fc-event-mirror")),o?Xe
():$e(),t||(r.setMirrorNeedsRevert(!s),this.receivingContext=i,this.droppableEvent=s)},this.handleDragEnd=e=>{let{receivingContext:t,droppableEvent:n}=this;if(this.clearDrag(
),t&&n){let r=this.hitDragging.finalHit,i=r.context.viewApi,s=this.dragMeta;if(t.emitter.trigger("drop",Object.assign(Object.assign({},Ya(r.dateSpan,t)),{draggedEl:e.subjectE
l,jsEvent:e.origEvent,view:i})),s.create){let r=Rr(n);t.dispatch({type:"MERGE_EVENTS",eventStore:r}),e.isTouch&&t.dispatch({type:"SELECT_EVENT",eventInstanceId:n.instance.ins
tanceId}),t.emitter.trigger("eventReceive",{event:new ti(t,n.def,n.instance),relatedEvents:[],revert(){t.dispatch({type:"REMOVE_EVENTS",eventStore:r})}},draggedEl:e.subjectEl,
view:i})}}this.receivingContext=null,this.droppableEvent=null};let n=this.hitDragging=new Qa(e,Oi);n.requireInitial=!1,n.emitter.on("dragstart",this.handleDragStart),n.emitte
r.on("hitupdate",this.handleHitUpdate),n.emitter.on("dragend",this.handleDragEnd),this.suppliedDragMeta=t}buildDragMeta(e){return"object"==typeof
this.suppliedDragMeta?bs(this.suppliedDragMeta):"function"==typeof this.suppliedDragMeta?bs(this.suppliedDragMeta(e)):function(e){let t=function(e,t){let
n=vs.dataAttrPrefix,r=(n?n+"-":"")+t;return e.getAttribute("data-"+r)||""}(e,"event");return bs(t?JSON.parse(t):{create:!1})}(e)}displayDrag(e,t){let n=this.receivingContext;
n&&n!==e&&n.dispatch({type:"UNSET_EVENT_DRAG"}),e&&e.dispatch({type:"SET_EVENT_DRAG",state:t})}clearDrag(){this.receivingContext&&this.receivingContext.dispatch({type:"UNSET_
EVENT_DRAG"})}canDropElOnCalendar(e,t){let n=t.options.dropAccept;return"function"==typeof n?n.call(t.calendarApi,e):"string"!=typeof
n||!n||Boolean(Ne(e,n))}}vs.dataAttrPrefix="";class Ka extends ms{constructor(e){super(e),this.shouldIgnoreMove=!1,this.mirrorSelector="",this.currentMirrorEl=null,this.handl
ePointerDown=e=>{this.emitter.trigger("pointerdown",e),this.shouldIgnoreMove||this.emitter.trigger("dragstart",e)},this.handlePointerMove=e=>{this.shouldIgnoreMove||this.emit
ter.trigger("dragmove",e)},this.handlePointerUp=e=>{this.emitter.trigger("pointerup",e),this.shouldIgnoreMove||this.emitter.trigger("dragend",e)};let t=this.pointer=new Pa(e)
;t.emitter.on("pointerdown",this.handlePointerDown),t.emitter.on("pointermove",this.handlePointerMove),t.emitter.on("pointerup",this.handlePointerUp)}destroy(){this.pointer.d
```

```
estroy()}setIgnoreMove(e){this.shouldIgnoreMove=e}setMirrorIsVisible(e){if(e)this.currentMirrorEl&&(this.currentMirrorEl.style.visibility="",this.currentMirrorEl=null);else{l
et e=this.mirrorSelector?document.querySelector(this.mirrorSelector):null;e&&(this.currentMirrorEl=e,e.style.visibility="hidden")}}}var
el=Po({name:"@fullcalendar/interaction",componentInteractions:[class extends Mi{constructor(e){super(e),this.handlePointerDown=e=>{let{dragging:t}=this,n=e.origEvent.target;t
.setIgnoreMove(!this.component.isValidDateDownEl(n))},this.handleDragEnd=e=>{let{component:t}=this,{pointer:n}=this.dragging;if(!n.wasTouchScroll){let{initialHit:n,finalHit:r
}=this.hitDragging;if(n&&r&&qa(n,r)){let{context:r}=t,i=Object.assign(Object.assign({},Ya(n.dateSpan,r)),{dayEl:n.dayEl,jsEvent:e.origEvent,view:r.viewApi||r.calendarApi.view
});r.emitter.trigger("dateClick",i)}}},this.dragging=new Fa(e.el),this.dragging.autoScroller.isEnabled=!1;let t=this.hitDragging=new
Qa(this.dragging,Ii(e));t.emitter.on("pointerdown",this.handlePointerDown),t.emitter.on("dragend",this.handleDragEnd)}destroy(){this.dragging.destroy()}},class extends Mi{con
structor(e){super(e),this.dragSelection=null,this.handlePointerDown=e=>{let{component:t,dragging:n}=this,{options:r}=t.context,i=r.selectable&&t.isValidDateDownEl(e.origEvent
.target);n.setIgnoreMove(!i),n.delay=e.isTouch?function(e){let{options:t}=e.context,n=t.selectLongPressDelay;null==n&&(n=t.longPressDelay);return
n}(t):null},this.handleDragStart=e=>{this.component.context.calendarApi.unselect(e)},this.handleHitUpdate=(e,t)=>{let{context:n}=this.component,r=null,i=!1;if(e){let
t=this.hitDragging.initialHit;e.componentId===t.componentId&&this.isHitComboAllowed&&!this.isHitComboAllowed(t,e)||(r=function(e,t,n){let
r=e.dateSpan,i=t.dateSpan,s=[r.range.start,r.range.end,i.range.start,i.range.end];s.sort(lt);let o={};for(let r of n){let n=r(e,t);if(!1===n)return
null;n&&Object.assign(o,n)}return o.range={start:s[0],end:s[3]},o.allDay=r.allDay,o}(t,e,n.pluginHooks.dateSelectionTransformers)),r&&Ps(r,e.dateProfile,n)||(i=!0,r=null)}r?n
.dispatch({type:"SELECT_DATES",selection:r}):t||n.dispatch({type:"UNSELECT_DATES"}),i?Xe():$e(),t||(this.dragSelection=r)},this.handlePointerUp=e=>{this.dragSelection&&(Yr(th
is.dragSelection,e,this.component.context),this.dragSelection=null)};let{component:t}=e,{options:n}=t.context,r=this.dragging=new
Fa(e.el);r.touchScrollAllowed=!1,r.minDistance=n.selectMinDistance||0,r.autoScroller.isEnabled=n.dragScroll;let i=this.hitDragging=new Qa(this.dragging,Ii(e));i.emitter.on("p
ointerdown",this.handlePointerDown),i.emitter.on("dragstart",this.handleDragStart),i.emitter.on("hitupdate",this.handleHitUpdate),i.emitter.on("pointerup",this.handlePointerU
p)}destroy(){this.dragging.destroy()}},Za,class extends Mi{constructor(e){super(e),this.draggingSegEl=null,this.draggingSeg=null,this.eventRange=null,this.relevantEvents=null
,this.validMutation=null,this.mutatedRelevantEvents=null,this.handlePointerDown=e=>{let{component:t}=this,n=ai(this.querySegEl(e)),r=this.eventRange=n.eventRange;this.draggin
g.minDistance=t.context.options.eventDragMinDistance,this.dragging.setIgnoreMove(!this.component.isValidSegDownEl(e.origEvent.target)||e.isTouch&&this.component.props.eventSe
lection!==r.instance.instanceId)},this.handleDragStart=e=>{let{context:t}=this.component,n=this.eventRange;this.relevantEvents=xr(t.getCurrentData().eventStore,this.eventRang
e.instance.instanceId);let r=this.querySegEl(e);this.draggingSegEl=r,this.draggingSeg=ai(r),t.calendarApi.unselect(),t.emitter.trigger("eventResizeStart",{el:r,event:new ti(t
,n.def,n.instance),jsEvent:e.origEvent,view:t.viewApi})},this.handleHitUpdate=(e,t,n)=>{let{context:r}=this.component,i=this.relevantEvents,s=this.hitDragging.initialHit,o=th
is.eventRange.instance,a=null,l=null,c=!1,d={affectedEvents:i,mutatedEvents:{defs:{},instances:{}},isEvent:!0};if(e){e.componentId===s.componentId&&this.isHitComboAllowed&&!t
his.isHitComboAllowed(s,e)||(a=function(e,t,n,r){let
i=e.context.dateEnv,s=e.dateSpan.range.start,o=t.dateSpan.range.start,a=hr(s,o,i,e.largeUnit);if(n){if(i.add(r.start,a)<r.end)return{startDelta:a}}else
if(i.add(r.end,a)>r.start)return{endDelta:a};return null}(s,e,n.subjectEl.classList.contains("fc-event-resizer-start"),o.range))}a&&(l=$r(i,r.getCurrentData().eventUiBases,a,
r),d.mutatedEvents=l,Ns(d,e.dateProfile,r)||(c=!0,a=null,l=null,d.mutatedEvents=null)),l?r.dispatch({type:"SET_EVENT_RESIZE",state:d}):r.dispatch({type:"UNSET_EVENT_RESIZE"})
,c?Xe():$e(),t||(a&&qa(s,e)&&(a=null),this.validMutation=a,this.mutatedRelevantEvents=l)},this.handleDragEnd=e=>{let{context:t}=this.component,n=this.eventRange.def,r=this.ev
entRange.instance,i=new ti(t,n,r),s=this.relevantEvents,o=this.mutatedRelevantEvents;if(t.emitter.trigger("eventResizeStop",{el:this.draggingSegEl,event:i,jsEvent:e.origEvent
,view:t.viewApi}),this.validMutation){let a=new ti(t,o.defs[n.defId],r?o.instances[r.instanceId]:null);t.dispatch({type:"MERGE_EVENTS",eventStore:o});let l={oldEvent:i,event:
a,relatedEvents:ri(o,t,r),revert(){t.dispatch({type:"MERGE_EVENTS",eventStore:s})}};t.emitter.trigger("eventResize",Object.assign(Object.assign({},l),{el:this.draggingSegEl,s
tartDelta:this.validMutation.startDelta||ft(0),endDelta:this.validMutation.endDelta||ft(0),jsEvent:e.origEvent,view:t.viewApi})),t.emitter.trigger("eventChange",l)}else
t.emitter.trigger("_noEventResize");this.draggingSeg=null,this.relevantEvents=null,this.validMutation=null};let{component:t}=e,n=this.dragging=new
Fa(e.el);n.pointer.selector=".fc-event-resizer",n.touchScrollAllowed=!1,n.autoScroller.isEnabled=t.context.options.dragScroll;let r=this.hitDragging=new Qa(this.dragging,Ii(e
));r.emitter.on("pointerdown",this.handlePointerDown),r.emitter.on("dragstart",this.handleDragStart),r.emitter.on("hitupdate",this.handleHitUpdate),r.emitter.on("dragend",thi
s.handleDragEnd)}destroy(){this.dragging.destroy()}querySegEl(e){return Oe(e.subjectEl,".fc-event")}}],calendarInteractions:[class{constructor(e){this.context=e,this.isRecent
PointerDateSelect=!1,this.matchesCancel=!1,this.matchesEvent=!1,this.onSelect=e=>{e.jsEvent&&(this.isRecentPointerDateSelect=!0)},this.onDocumentPointerDown=e=>{let t=this.co
ntext.options.unselectCancel,n=ze(e.origEvent);this.matchesCancel=!!Oe(n,t),this.matchesEvent=!!Oe(n,Za.SELECTOR)},this.onDocumentPointerUp=e=>{let{context:t}=this,{documentP
ointer:n}=this,r=t.getCurrentData();if(!n.wasTouchScroll){if(r.dateSelection&&!this.isRecentPointerDateSelect){let n=t.options.unselectAuto;!n||n&&this.matchesCancel||t.calen
darApi.unselect(e)}r.eventSelection&&!this.matchesEvent&&t.dispatch({type:"UNSELECT_EVENT"})}this.isRecentPointerDateSelect=!1};let t=this.documentPointer=new Pa(document);t.
shouldIgnoreMove=!0,t.shouldWatchScroll=!1,t.emitter.on("pointerdown",this.onDocumentPointerDown),t.emitter.on("pointerup",this.onDocumentPointerUp),e.emitter.on("select",thi
s.onSelect)}destroy(){this.context.emitter.off("select",this.onSelect),this.documentPointer.destroy()}}],elementDraggingImpl:Fa,optionRefiners:Xa,listenerRefiners:$a});class
tl extends ls{constructor(){super(...arguments),this.headerElRef={current:null}}renderSimpleLayout(e,t){let{props:n,context:r}=this,i=[],s=no(r.options);return e&&i.push({typ
e:"header",key:"header",isSticky:s,chunk:{elRef:this.headerElRef,tableClassName:"fc-col-header",rowContent:e}}),i.push({type:"body",key:"body",liquid:!0,chunk:{content:t}}),p
(er,{elClasses:["fc-daygrid"],viewSpec:r.viewSpec},p(io,{liquid:!n.isHeightAuto&&!n.forPrint,collapsibleWidth:n.forPrint,cols:[],sections:i}))}renderHScrollLayout(e,t,n,r){le
t i=this.context.pluginHooks.scrollGridImpl;if(!i)throw new Error("No ScrollGrid
implementation");let{props:s,context:o}=this,a=!s.forPrint&&no(o.options),l=!s.forPrint&&ro(o.options),c=[];return e&&c.push({type:"header",key:"header",isSticky:a,chunks:[{k
ey:"main",elRef:this.headerElRef,tableClassName:"fc-col-header",rowContent:e}]}),c.push({type:"body",key:"body",liquid:!0,chunks:[{key:"main",content:t}]}),l&&c.push({type:"f
ooter",key:"footer",isSticky:!0,chunks:[{key:"main",content:to}]}),p(er,{elClasses:["fc-daygrid"],viewSpec:o.viewSpec},p(i,{liquid:!s.isHeightAuto&&!s.forPrint,forPrint:s.for
Print,collapsibleWidth:s.forPrint,colGroups:[{cols:[{span:n,minWidth:r}]}],sections:c}))}}function nl(e,t){let n=[];for(let e=0;e<t;e+=1)n[e]=[];for(let t of
e)n[t.row].push(t);return n}function rl(e,t){let n=[];for(let e=0;e<t;e+=1)n[e]=[];for(let t of e)n[t.firstCol].push(t);return n}function il(e,t){let n=[];if(e){for(let
r=0;r<t;r+=1)n[r]={affectedInstances:e.affectedInstances,isEvent:e.isEvent,segs:[]};for(let t of e.segs)n[t.row].segs.push(t)}else for(let e=0;e<t;e+=1)n[e]=null;return
n}const sl=an({hour:"numeric",minute:"2-digit",omitZeroMinute:!0,meridiem:"narrow"});function
ol(e){let{display:t}=e.eventRange.ui;return"list-item"===t||"auto"===t&&!e.eventRange.def.allDay&&e.firstCol===e.lastCol&&e.isStart&&e.isEnd}class al extends
Gn{render(){let{props:e}=this;return p(oo,Object.assign({},e,{elClasses:["fc-daygrid-event","fc-daygrid-block-event","fc-h-event"],defaultTimeFormat:sl,defaultDisplayEventEnd
:e.defaultDisplayEventEnd,disableResizing:!e.seg.eventRange.def.allDay}))}}class ll extends
Gn{render(){let{props:e,context:t}=this,{options:n}=t,{seg:r}=e,i=pi(r,n.eventTimeFormat||sl,t,!0,e.defaultDisplayEventEnd);return p(so,Object.assign({},e,{elTag:"a",elClasse
s:["fc-daygrid-event","fc-daygrid-dot-event"],elAttrs:bi(e.seg,t),defaultGenerator:cl,timeText:i,isResizing:!1,isDateSelecting:!1}))}}function cl(e){return p(y,null,p("div",{
className:"fc-daygrid-event-dot",style:{borderColor:e.borderColor||e.backgroundColor}}),e.timeText&&p("div",{className:"fc-event-time"},e.timeText),p("div",{className:"fc-eve
nt-title"},e.event.title||p(y,null," "))}class dl extends
Gn{constructor(){super(...arguments),this.compileSegs=Gt(ul)}render(){let{props:e}=this,{allSegs:t,invisibleSegs:n}=this.compileSegs(e.singlePlacements);return p(So,{elClasse
s:["fc-daygrid-more-link"],dateProfile:e.dateProfile,todayRange:e.todayRange,allDayDate:e.allDayDate,moreCnt:e.moreCnt,allSegs:t,hiddenSegs:n,alignmentElRef:e.alignmentElRef,
alignGridTop:e.alignGridTop,extraDateSpan:e.extraDateSpan,popoverContent:()=>{let
n=(e.eventDrag?e.eventDrag.affectedInstances:null)||(e.eventResize?e.eventResize.affectedInstances:null)||{};return p(y,null,t.map(t=>{let
r=t.eventRange.instance.instanceId;return p("div",{className:"fc-daygrid-event-harness",key:r,style:{visibility:n[r]?"hidden":""}},ol(t)?p(ll,Object.assign({seg:t,isDragging:
!1,isSelected:r===e.eventSelection,defaultDisplayEventEnd:!1},mi(t,e.todayRange))):p(al,Object.assign({seg:t,isDragging:!1,isResizing:!1,isDateSelecting:!1,isSelected:r===e.e
ventSelection,defaultDisplayEventEnd:!1},mi(t,e.todayRange))))}))}}}function ul(e){let t=[],n=[];for(let r of
e)t.push(r.seg),r.isVisible||n.push(r.seg);return{allSegs:t,invisibleSegs:n}}const hl=an({week:"narrow"});class fl extends ls{constructor(){super(...arguments),this.rootElRef
={current:null},this.state={dayNumberId:We()},this.handleRootEl=e=>{Qn(this.rootElRef,e),Qn(this.props.elRef,e)}}render(){let{context:e,props:t,state:n,rootElRef:r}=this,{opt
ions:i,dateEnv:s}=e,{date:o,dateProfile:a}=t;const l=t.showDayNumber&&function(e,t,n){const{start:r,end:i}=t,s=Ct(i,-1),o=n.getYear(r),a=n.getMonth(r),l=n.getYear(s),c=n.getM
onth(s);return!(o===l&&a===c)&&Boolean(e.valueOf()===r.valueOf()||1===n.getDay(e)&&e.valueOf()<i.valueOf())}(o,a.currentRange,s);return p(uo,{elTag:"td",elRef:this.handleRoot
El,elClasses:["fc-daygrid-day",...t.extraClassNames||[]],elAttrs:Object.assign(Object.assign(Object.assign({},t.extraDataAttrs),t.showDayNumber?{"aria-labelledby":n.dayNumber
Id}:{}),{role:"gridcell"}),defaultGenerator:gl,date:o,dateProfile:a,todayRange:t.todayRange,showDayNumber:t.showDayNumber,isMonthStart:l,extraRenderProps:t.extraRenderProps},
```

```
(s,a)=>p("div",{ref:t.innerElRef,className:"fc-daygrid-day-frame fc-scrollgrid-sync-inner",style:{minHeight:t.minHeight}},t.showWeekNumber&&p(vo,{elTag:"a",elClasses:["fc-day
grid-week-number"],elAttrs:qi(e,o,"week"),date:o,defaultFormat:hl}),!a.isDisabled&&(t.showDayNumber||ho(i)||t.forceDayTop)?p("div",{className:"fc-daygrid-day-top"},p(s,{elTag
:"a",elClasses:["fc-daygrid-day-number",l&&"fc-daygrid-month-start"],elAttrs:Object.assign(Object.assign({},qi(e,o)),{id:n.dayNumberId})})):t.showDayNumber?p("div",{className
:"fc-daygrid-day-top",style:{visibility:"hidden"}},p("a",{className:"fc-daygrid-day-number"},"  ")):void 0,p("div",{className:"fc-daygrid-day-events",ref:t.fgContentElRef},t.f
gContent,p("div",{className:"fc-daygrid-day-bottom",style:{marginTop:t.moreMarginTop}},p(dl,{allDayDate:o,singlePlacements:t.singlePlacements,moreCnt:t.moreCnt,alignmentElRef
:r,alignGridTop:!t.showDayNumber,extraDateSpan:t.extraDateSpan,dateProfile:t.dateProfile,eventSelection:t.eventSelection,eventDrag:t.eventDrag,eventResize:t.eventResize,today
Range:t.todayRange}))),p("div",{className:"fc-daygrid-day-bg"},t.bgContent))))}function gl(e){return e.dayNumberText||p(y,null,"  ")}function pl(e){return
e.eventRange.instance.instanceId+":"+e.firstCol}function ml(e){return pl(e)+":"+e.lastCol}function vl(e,t,n,r,i,s,o){let a=new bl(t=>{let
n=e[t.index].eventRange.instance.instanceId+":"+t.span.start+":"+(t.span.end-1);return
i[n]||1});a.allowReslicing=!0,a.strictOrder=r,!0===t||!0===n?(a.maxCoord=s,a.hiddenConsumes=!0):"number"==typeof t?a.maxStackCnt=t:"number"==typeof
n&&(a.maxStackCnt=n,a.hiddenConsumes=!0);let l=[],c=[];for(let t=0;t<e.length;t+=1){let
n=e[t],r=ml(n);null!=i[r]?l.push({index:t,span:{start:n.firstCol,end:n.lastCol+1}}):c.push(n)}let
d=a.addSegs(l),u=a.toRects(),{singleColPlacements:h,multiColPlacements:f,leftoverMargins:g}=function(e,t,n){let r=function(e,t){let n=[];for(let
e=0;e<t;e+=1)n.push([]);for(let t of e)for(let e=t.span.start;e<t.span.end;e+=1)n[e].push(t);return n}(e,n.length),i=[],s=[],o=[];for(let e=0;e<n.length;e+=1){let
a=r[e],l=[],c=0,d=0;for(let r of a){let
i=t[r.index];l.push({seg:yl(i,e,e+1,n),isVisible:!0,isAbsolute:!1,absoluteTop:r.levelCoord,marginTop:r.levelCoord-c}),c=r.levelCoord+r.thickness}let u=[];c=0,d=0;for(let r of
 a){let i=t[r.index],s=r.span.end-r.span.start>1,o=r.span.start===e;d+=r.levelCoord-c,c=r.levelCoord+r.thickness,s?(d+=r.thickness,o&&u.push({seg:yl(i,r.span.start,r.span.end
,n),isVisible:!0,isAbsolute:!0,absoluteTop:r.levelCoord,marginTop:0})):o&&(u.push({seg:yl(i,r.span.start,r.span.end,n),isVisible:!0,isAbsolute:!1,absoluteTop:r.levelCoord,mar
ginTop:d}),d=0)}i.push(l),s.push(u),o.push(d)}return{singleColPlacements:i,multiColPlacements:s,leftoverMargins:o}}(u,e,o),p=[],m=[];for(let e of
c){f[e.firstCol].push({seg:e,isVisible:!1,isAbsolute:!0,absoluteTop:0,marginTop:0});for(let
t=e.firstCol;t<=e.lastCol;t+=1)h[t].push({seg:yl(e,t,t+1,o),isVisible:!1,isAbsolute:!1,absoluteTop:0,marginTop:0})}for(let e=0;e<o.length;e+=1)p.push(0);for(let t of d){let
n=e[t.index],r=t.span;f[r.start].push({seg:yl(n,r.start,r.end,o),isVisible:!1,isAbsolute:!0,absoluteTop:0,marginTop:0});for(let
e=r.start;e<r.end;e+=1)p[e]+=1,h[e].push({seg:yl(n,e,e+1,o),isVisible:!1,isAbsolute:!1,absoluteTop:0,marginTop:0})}for(let
e=0;e<o.length;e+=1)m.push(g[e]);return{singleColPlacements:h,multiColPlacements:f,moreCnts:p,moreMarginTops:m}}function yl(e,t,n,r){if(e.firstCol===t&&e.lastCol===n-1)return
 e;let i=e.eventRange,s=i.range,o=ir(s,{start:r[t].date,end:wt(r[n-1].date,1)});return Object.assign(Object.assign({},e),{firstCol:t,lastCol:n-1,eventRange:{def:i.def,ui:Obje
ct.assign(Object.assign({},i.ui),{durationEditable:!1}),instance:i.instance,range:o},isStart:e.isStart&&o.start.valueOf()===s.start.valueOf(),isEnd:e.isEnd&&o.end.valueOf()==
=s.end.valueOf()})}class bl extends cs{constructor(){super(...arguments),this.hiddenConsumes=!1,this.forceHidden={}}addSegs(e){const
t=super.addSegs(e),{entriesByLevel:n}=this,r=e=>!this.forceHidden[us(e)];for(let e=0;e<n.length;e+=1)n[e]=n[e].filter(r);return
t}handleInvalidInsertion(e,t,n){const{entriesByLevel:r,forceHidden:i}=this,{touchingEntry:s,touchingLevel:o,touchingLateral:a}=e;if(this.hiddenConsumes&&s){const
e=us(s);if(!i[e])if(this.allowReslicing){const e=Object.assign(Object.assign({},s),{span:fs(s.span,t.span)});i[us(e)]=!0,r[o][a]=e,n.push(e),this.splitEntry(s,t,n)}else
i[e]=!0,n.push(s)}super.handleInvalidInsertion(e,t,n)}}class El extends ls{constructor(){super(...arguments),this.cellElRefs=new Vs,this.frameElRefs=new
Vs,this.fgElRefs=new Vs,this.segHarnessRefs=new Vs,this.rootElRef={current:null},this.state={framePositions:null,maxContentHeight:null,segHeights:{}},this.handleResize=e=>{e&&this.updateSizing(!0
)}}render(){let{props:e,state:t,context:n}=this,{options:r}=n,i=e.cells.length,s=rl(e.businessHourSegs,i),o=rl(e.bgEventSegs,i),a=rl(this.getHighlightSegs(),i),l=rl(this.getM
irrorSegs(),i),{singleColPlacements:c,multiColPlacements:d,moreCnts:u,moreMarginTops:h}=vl(di(e.fgEventSegs,r.eventOrder),e.dayMaxEvents,e.dayMaxEventRows,r.eventOrderStrict,
t.segHeights,t.maxContentHeight,e.cells),f=e.eventDrag&&e.eventDrag.affectedInstances||e.eventResize&&e.eventResize.affectedInstances||{};return
p("tr",{ref:this.rootElRef,role:"row"},e.renderIntro&&e.renderIntro(),e.cells.map((t,n)=>{let
r=this.renderFgSegs(n,e.forPrint?c[n]:d[n],e.todayRange,f),i=this.renderFgSegs(n,function(e,t){if(!e.length)return[];let n=function(e){let t={};for(let n of e)for(let e of
n)t[e.seg.eventRange.instance.instanceId]=e.absoluteTop;return t}(t);return e.map(e=>({seg:e,isVisible:!0,isAbsolute:!0,absoluteTop:n[e.eventRange.instance.instanceId],margin
Top:0}))}(l[n],d),e.todayRange,{},Boolean(e.eventDrag),Boolean(e.eventResize),!1);return p(fl,{key:t.key,elRef:this.cellElRefs.createRef(t.key),innerElRef:this.frameElRefs.cr
eateRef(t.key),dateProfile:e.dateProfile,date:t.date,showDayNumber:e.showDayNumbers,showWeekNumber:e.showWeekNumbers&&0===n,forceDayTop:e.showWeekNumbers,todayRange:e.todayRa
nge,eventSelection:e.eventSelection,eventDrag:e.eventDrag,eventResize:e.eventResize,extraRenderProps:t.extraRenderProps,extraDataAttrs:t.extraDataAttrs,extraClassNames:t.extr
aClassNames,extraDateSpan:t.extraDateSpan,moreCnt:u[n],moreMarginTop:h[n],singlePlacements:c[n],fgContentElRef:this.fgElRefs.createRef(t.key),fgContent:p(y,null,p(y,null,r),p
(y,null,i)),bgContent:p(y,null,this.renderFillSegs(a[n],"highlight"),this.renderFillSegs(s[n],"non-business"),this.renderFillSegs(o[n],"bg-event")),minHeight:e.cellMinHeight}
)}))}componentDidMount(){this.updateSizing(!0),this.context.addResizeHandler(this.handleResize)}componentDidUpdate(e,t){let
n=this.props;this.updateSizing(!Cn(e,n))}componentWillUnmount(){this.context.removeResizeHandler(this.handleResize)}getHighlightSegs(){let{props:e}=this;return
e.eventDrag&&e.eventDrag.segs.length?e.eventDrag.segs:e.eventResize&&e.eventResize.segs.length?e.eventResize.segs:e.dateSelectionSegs}getMirrorSegs(){let{props:e}=this;return
 e.eventResize&&e.eventResize.segs.length?e.eventResize.segs:[]}renderFgSegs(e,t,n,r,i,s,o){let{context:a}=this,{eventSelection:l}=this.props,{framePositions:c}=this.state,d=
1===this.props.cells.length,u=i||s||o,h=[];if(c)for(let e of t){let{seg:t}=e,{instanceId:f}=t.eventRange.instance,g=e.isVisible&&!r[f],m=e.isAbsolute,v="",y="";m&&(a.isRtl?(y
=0,v=c.lefts[t.lastCol]-c.lefts[t.firstCol]):(v=0,y=c.rights[t.firstCol]-c.rights[t.lastCol])),h.push(p("div",{className:"fc-daygrid-event-harness"+(m?" fc-daygrid-event-harn
ess-abs":""),key:pl(t),ref:u?null:this.segHarnessRefs.createRef(ml(t)),style:{visibility:g?"":"hidden",marginTop:m?"":e.marginTop,top:m?e.absoluteTop:"",left:v,right:y}},ol(t
)?p(ll,Object.assign({seg:t,isDragging:i,isSelected:f===l,defaultDisplayEventEnd:d},mi(t,n))):p(al,Object.assign({seg:t,isDragging:i,isResizing:s,isDateSelecting:o,isSelected
:f===l,defaultDisplayEventEnd:d},mi(t,n)))))}return h}renderFillSegs(e,t){let{isRtl:n}=this.context,{todayRange:r}=this.props,{framePositions:i}=this.state,s=[];if(i)for(let
o of e){let e=n?{right:0,left:i.lefts[o.lastCol]-i.lefts[o.firstCol]}:{left:0,right:i.rights[o.firstCol]-i.rights[o.lastCol]};s.push(p("div",{key:yi(o.eventRange),className:"
fc-daygrid-bg-harness",style:e},"bg-event"===t?p(go,Object.assign({seg:o},mi(o,r))):mo(t)))}return
p(y,{},...s)}updateSizing(e){let{props:t,state:n,frameElRefs:r}=this;if(!t.forPrint&&null!==t.clientWidth){if(e){let e=t.cells.map(e=>r.currentMap[e.key]);if(e.length){let
t=this.rootElRef.current,r=new rs(t,e,!0,!1);n.framePositions&&n.framePositions.similarTo(r)||this.setState({framePositions:new rs(t,e,!0,!1)})}}const i=this.state.segHeights
,s=this.querySegHeights(),o=!0===t.dayMaxEvents||!0===t.dayMaxEventRows;this.safeSetState({segHeights:Object.assign(Object.assign({},i),s),maxContentHeight:o?this.computeMaxC
ontentHeight():null})}}querySegHeights(){let e=this.segHarnessRefs.currentMap,t={};for(let n in e){let
r=Math.round(e[n].getBoundingClientRect().height);t[n]=Math.max(t[n]||0,r)}return t}computeMaxContentHeight(){let
e=this.props.cells[0].key,t=this.cellElRefs.currentMap[e],n=this.fgElRefs.currentMap[e];return t.getBoundingClientRect().bottom-n.getBoundingClientRect().top}getCellEls(){let
 e=this.cellElRefs.currentMap;return this.props.cells.map(t=>e[t.key])}}El.addStateEquality({segHeights:Cn});class Sl extends ls{constructor(){super(...arguments),this.splitB
usinessHourSegs=Gt(nl),this.splitBgEventSegs=Gt(nl),this.splitFgEventSegs=Gt(nl),this.splitDateSelectionSegs=Gt(nl),this.splitEventDrag=Gt(il),this.splitEventResize=Gt(il),th
is.rowRefs=new Vs}render(){let{props:e,context:t}=this,n=e.cells.length,r=this.splitBusinessHourSegs(e.businessHourSegs,n),i=this.splitBgEventSegs(e.bgEventSegs,n),s=this.spl
itFgEventSegs(e.fgEventSegs,n),o=this.splitDateSelectionSegs(e.dateSelectionSegs,n),a=this.splitEventDrag(e.eventDrag,n),l=this.splitEventResize(e.eventResize,n),c=n>=7&&e.cl
ientWidth?e.clientWidth/t.options.aspectRatio/6:null;return p(Rs,{unit:"day"},(t,d)=>p(y,null,e.cells.map((t,u)=>p(El,{ref:this.rowRefs.createRef(u),key:t.length?t[0].date.to
ISOString():u,showDayNumbers:n>1,showWeekNumbers:e.showWeekNumbers,todayRange:d,dateProfile:e.dateProfile,cells:t,renderIntro:e.renderRowIntro,businessHourSegs:r[u],eventSele
ction:e.eventSelection,bgEventSegs:i[u].filter(Al),fgEventSegs:s[u],dateSelectionSegs:o[u],eventDrag:a[u],eventResize:l[u],dayMaxEvents:e.dayMaxEvents,dayMaxEventRows:e.dayMa
xEventRows,clientWidth:e.clientWidth,clientHeight:e.clientHeight,cellMinHeight:c,forPrint:e.forPrint}))))}componentDidMount(){this.registerInteractiveComponent()}componentDid
Update(){this.registerInteractiveComponent()}registerInteractiveComponent(){if(!this.rootEl){const e=this.rowRefs.currentMap[0].getCellEls()[0],t=e?e.closest(".fc-daygrid-bod
y"):null;t&&(this.rootEl=t,this.context.registerInteractiveComponent(this,{el:t,isHitComboAllowed:this.props.isHitComboAllowed}))}}componentWillUnmount(){this.rootEl&&(this.c
ontext.unregisterInteractiveComponent(this),this.rootEl=null)}prepareHits(){this.rowPositions=new
rs(this.rootEl,this.rowRefs.collect().map(e=>e.getCellEls()[0]),!1,!0),this.colPositions=new rs(this.rootEl,this.rowRefs.currentMap[0].getCellEls(),!0,!1)}queryHit(e,t){let{c
olPositions:n,rowPositions:r}=this,i=n.leftToIndex(e),s=r.topToIndex(t);if(null!=s&&null!=i){let e=this.props.cells[s][i];return{dateProfile:this.props.dateProfile,dateSpan:O
bject.assign({range:this.getCellRange(s,i),allDay:!0},e.extraDateSpan),dayEl:this.getCellEl(s,i),rect:{left:n.lefts[i],right:n.rights[i],top:r.tops[s],bottom:r.bottoms[s]},la
```

```
yer:0}}return null}getCellEl(e,t){return this.rowRefs.currentMap[e].getCellEls()[t]}getCellRange(e,t){let n=this.props.cells[e][t].date;return{start:n,end:wt(n,1)}}}function
Al(e){return e.eventRange.def.allDay}class Dl extends ls{constructor(){super(...arguments),this.elRef={current:null},this.needsScrollReset=!1}render(){let{props:e}=this,{dayM
axEventRows:t,dayMaxEvents:n,expandRows:r}=e,i=!0===n||!0===t;i&&!r&&(i=!1,t=null,n=null);let
s=["fc-daygrid-body",i?"fc-daygrid-body-balanced":"fc-daygrid-body-unbalanced",r?"":"fc-daygrid-body-natural"];return p("div",{ref:this.elRef,className:s.join(" "),style:{wid
th:e.clientWidth,minWidth:e.tableMinWidth}},p("table",{role:"presentation",className:"fc-scrollgrid-sync-table",style:{width:e.clientWidth,minWidth:e.tableMinWidth,height:r?e
.clientHeight:""}},e.colGroupNode,p("tbody",{role:"presentation"},p(Sl,{dateProfile:e.dateProfile,cells:e.cells,renderRowIntro:e.renderRowIntro,showWeekNumbers:e.showWeekNumb
ers,clientWidth:e.clientWidth,clientHeight:e.clientHeight,businessHourSegs:e.businessHourSegs,bgEventSegs:e.bgEventSegs,fgEventSegs:e.fgEventSegs,dateSelectionSegs:e.dateSele
ctionSegs,eventSelection:e.eventSelection,eventDrag:e.eventDrag,eventResize:e.eventResize,dayMaxEvents:n,dayMaxEventRows:t,forPrint:e.forPrint,isHitComboAllowed:e.isHitComboA
llowed})))))componentDidMount(){this.requestScrollReset()}componentDidUpdate(e){e.dateProfile!==this.props.dateProfile?this.requestScrollReset():this.flushScrollReset()}reque
stScrollReset(){this.needsScrollReset=!0,this.flushScrollReset()}flushScrollReset(){if(this.needsScrollReset&&this.props.clientWidth){const e=function(e,t){let
n;t.currentRangeUnit.match(/year|month/)&&(n=e.querySelector(`[data-date="${Lt(t.currentDate)}-01"]`));n||(n=e.querySelector(`[data-date="${Wt(t.currentDate)}"]`));return
n}(this.elRef.current,this.props.dateProfile);if(e){const
t=e.closest(".fc-daygrid-body"),n=t.closest(".fc-scroller"),r=e.getBoundingClientRect().top-t.getBoundingClientRect().top;n.scrollTop=r?r+1:0}this.needsScrollReset=!1}}}class
 wl extends Is{constructor(){super(...arguments),this.forceDayIfListItem=!0}sliceRange(e,t){return t.sliceRange(e)}}class Cl extends
ls{constructor(){super(...arguments),this.slicer=new wl,this.tableRef={current:null}}render(){let{props:e,context:t}=this;return p(Dl,Object.assign({ref:this.tableRef},this.s
licer.sliceProps(e,e.dateProfile,e.nextDayThreshold,t,e.dayTableModel),{dateProfile:e.dateProfile,cells:e.dayTableModel.cells,colGroupNode:e.colGroupNode,tableMinWidth:e.tabl
eMinWidth,renderRowIntro:e.renderRowIntro,dayMaxEvents:e.dayMaxEvents,dayMaxEventRows:e.dayMaxEventRows,showWeekNumbers:e.showWeekNumbers,expandRows:e.expandRows,headerAlignE
lRef:e.headerAlignElRef,clientWidth:e.clientWidth,clientHeight:e.clientHeight,forPrint:e.forPrint}))}}function Rl(e,t){let n=new ks(e.renderRange,t);return new
Ms(n,/year|month|week/.test(e.currentRangeUnit))}class xl extends gr{buildRenderRange(e,t,n){let r=super.buildRenderRange(e,t,n),{props:i}=this;return
_l({currentRange:r,snapToWeek:/^(year|month)$/.test(t),fixedWeekCount:i.fixedWeekCount,dateEnv:i.dateEnv})}}function _l(e){let
t,{dateEnv:n,currentRange:r}=e,{start:i,end:s}=r;if(e.snapToWeek&&(i=n.startOfWeek(i),t=n.startOfWeek(s),t.valueOf()!==s.valueOf()&&(s=Dt(t,1))),e.fixedWeekCount){let e=n.sta
rtOfWeek(n.startOfMonth(wt(r.end,-1)));s=Dt(s,6-Math.ceil(Rt(e,s)))}return{start:i,end:s}}xe(':root{--fc-daygrid-event-dot-width:8px}.fc-daygrid-day-events:after,.fc-daygrid-
day-events:before,.fc-daygrid-day-frame:after,.fc-daygrid-day-frame:before,.fc-daygrid-event-harness:after,.fc-daygrid-event-harness:before{clear:both;content:"";display:tabl
e}.fc .fc-daygrid-body{position:relative;z-index:1}.fc .fc-daygrid-day.fc-day-today{background-color:var(--fc-today-bg-color)}.fc
.fc-daygrid-day-frame{min-height:100%;position:relative}.fc .fc-daygrid-day-top{display:flex;flex-direction:row-reverse}.fc .fc-day-other .fc-daygrid-day-top{opacity:.3}.fc
.fc-daygrid-day-number{padding:4px;position:relative;z-index:4}.fc .fc-daygrid-month-start{font-size:1.1em;font-weight:700}.fc .fc-daygrid-day-events{margin-top:1px}.fc
.fc-daygrid-body-balanced .fc-daygrid-day-events{left:0;position:absolute;right:0}.fc .fc-daygrid-body-unbalanced .fc-daygrid-day-events{min-height:2em;position:relative}.fc
.fc-daygrid-body-natural .fc-daygrid-day-events{margin-bottom:1em}.fc .fc-daygrid-event-harness{position:relative}.fc
.fc-daygrid-event-harness-abs{left:0;position:absolute;right:0;top:0}.fc .fc-daygrid-bg-harness{bottom:0;position:absolute;top:0}.fc .fc-daygrid-day-bg
.fc-non-business{z-index:1}.fc .fc-daygrid-day-bg .fc-bg-event{z-index:2}.fc .fc-daygrid-day-bg .fc-highlight{z-index:3}.fc .fc-daygrid-event{margin-top:1px;z-index:6}.fc
.fc-daygrid-event.fc-event-mirror{z-index:7}.fc .fc-daygrid-day-bottom{font-size:.85em;margin:0 2px}.fc .fc-daygrid-day-bottom:after,.fc
.fc-daygrid-day-bottom:before{clear:both;content:"";display:table}.fc .fc-daygrid-more-link{border-radius:3px;cursor:pointer;line-height:1;margin-top:1px;max-width:100%;overf
low:hidden;padding:2px;position:relative;white-space:nowrap;z-index:4}.fc .fc-daygrid-more-link:hover{background-color:rgba(0,0,0,.1)}.fc .fc-daygrid-week-number{background-c
olor:var(--fc-neutral-bg-color);color:var(--fc-neutral-text-color);min-width:1.5em;padding:2px;position:absolute;text-align:center;top:0;z-index:5}.fc .fc-more-popover
.fc-popover-body{min-width:220px;padding:10px}.fc-direction-ltr .fc-daygrid-event.fc-event-start,.fc-direction-rtl
.fc-daygrid-event.fc-event-end{margin-left:2px}.fc-direction-ltr .fc-daygrid-event.fc-event-end,.fc-direction-rtl
.fc-daygrid-event.fc-event-start{margin-right:2px}.fc-direction-ltr .fc-daygrid-more-link{float:left}.fc-direction-ltr .fc-daygrid-week-number{border-radius:0 0 3px
0;left:0}.fc-direction-rtl .fc-daygrid-more-link{float:right}.fc-direction-rtl .fc-daygrid-week-number{border-radius:0 0 0 3px;right:0}.fc-liquid-hack
.fc-daygrid-day-frame{position:static}.fc-daygrid-event{border-radius:3px;font-size:var(--fc-small-font-size);position:relative;white-space:nowrap}.fc-daygrid-block-event
.fc-event-time{font-weight:700}.fc-daygrid-block-event .fc-event-time,.fc-daygrid-block-event
.fc-event-title{padding:1px}.fc-daygrid-dot-event{align-items:center;display:flex;padding:2px 0}.fc-daygrid-dot-event .fc-event-title{flex-grow:1;flex-shrink:1;font-weight:70
0;min-width:0;overflow:hidden}.fc-daygrid-dot-event.fc-event-mirror,.fc-daygrid-dot-event:hover{background:rgba(0,0,0,.1)}.fc-daygrid-dot-event.fc-event-selected:before{botto
m:-10px;top:-10px}.fc-daygrid-event-dot{border:calc(var(--fc-daygrid-event-dot-width)/2) solid
var(--fc-event-border-color);border-radius:calc(var(--fc-daygrid-event-dot-width)/2);box-sizing:content-box;height:0;margin:0 4px;width:0}.fc-direction-ltr .fc-daygrid-event
.fc-event-time{margin-right:3px}.fc-direction-rtl .fc-daygrid-event .fc-event-time{margin-left:3px}');var
Tl=Po({name:"@fullcalendar/daygrid",initialView:"dayGridMonth",views:{dayGrid:{component:class extends tl{constructor(){super(...arguments),this.buildDayTableModel=Gt(Rl),thi
s.headerRef={current:null},this.tableRef={current:null}}render(){let{options:e,dateProfileGenerator:t}=this.context,{props:n}=this,r=this.buildDayTableModel(n.dateProfile,t),
i=e.dayHeaders&&p(_s,{ref:this.headerRef,dateProfile:n.dateProfile,dates:r.headerDates,datesRepDistinctDays:1===r.rowCnt}),s=t=>p(Cl,{ref:this.tableRef,dateProfile:n.dateProf
ile,dayTableModel:r,businessHours:n.businessHours,dateSelection:n.dateSelection,eventStore:n.eventStore,eventUiBases:n.eventUiBases,eventSelection:n.eventSelection,eventDrag:
n.eventDrag,eventResize:n.eventResize,nextDayThreshold:e.nextDayThreshold,colGroupNode:t.tableColGroupNode,tableMinWidth:t.tableMinWidth,dayMaxEvents:e.dayMaxEvents,dayMaxEve
ntRows:e.dayMaxEventRows,showWeekNumbers:e.weekNumbers,expandRows:!n.isHeightAuto,headerAlignElRef:this.headerElRef,clientWidth:t.clientWidth,clientHeight:t.clientHeight,forP
rint:n.forPrint});return e.dayMinWidth?this.renderHScrollLayout(i,s,r.colCnt,e.dayMinWidth):this.renderSimpleLayout(i,s)}},dateProfileGeneratorClass:xl},dayGridDay:{type:"day
Grid",duration:{days:1}},dayGridWeek:{type:"dayGrid",duration:{weeks:1}},dayGridMonth:{type:"dayGrid",duration:{months:1},fixedWeekCount:!0},dayGridYear:{type:"dayGrid",durat
ion:{years:1}}}});class kl extends Wi{getKeyInfo(){return{allDay:{},timed:{}}}getKeysForDateSpan(e){return e.allDay?["allDay"]:["timed"]}getKeysForEventDef(e){return
e.allDay?si(e)?["timed","allDay"]:["allDay"]:["timed"]}}const Ml=an({hour:"numeric",minute:"2-digit",omitZeroMinute:!0,meridiem:"short"});function Il(e){let
t=["fc-timegrid-slot","fc-timegrid-slot-label",e.isLabeled?"fc-scrollgrid-shrink":"fc-timegrid-slot-minor"];return p(Ln.Consumer,null,n=>{if(!e.isLabeled)return
p("td",{className:t.join(" "),"data-time":e.isoTimeStr});let{dateEnv:r,options:i,viewApi:s}=n,o=null==i.slotLabelFormat?Ml:Array.isArray(i.slotLabelFormat)?an(i.slotLabelForm
at[0]):an(i.slotLabelFormat),a={level:0,time:e.time,date:r.toDate(e.date),view:s,text:r.format(e.date,o)};return p(Jn,{elTag:"td",elClasses:t,elAttrs:{"data-time":e.isoTimeSt
r},renderProps:a,generatorName:"slotLabelContent",customGenerator:i.slotLabelContent,defaultGenerator:Ol,classNameGenerator:i.slotLabelClassNames,didMount:i.slotLabelDidMount
,willUnmount:i.slotLabelWillUnmount},e=>p("div",{className:"fc-timegrid-slot-label-frame
fc-scrollgrid-shrink-frame"},p(e,{elTag:"div",elClasses:["fc-timegrid-slot-label-cushion","fc-scrollgrid-shrink-cushion"]})))})}function Ol(e){return e.text}class Nl extends
Gn{render(){return this.props.slatMetas.map(e=>p("tr",{key:e.key},p(Il,Object.assign({},e))))}}const Pl=an({week:"short"});class Hl extends
ls{constructor(){super(...arguments),this.allDaySplitter=new
kl,this.headerElRef={current:null},this.rootElRef={current:null},this.scrollerElRef={current:null},this.state={slatCoords:null},this.handleScrollTopRequest=e=>{let t=this.scr
ollerElRef.current;t&&(t.scrollTop=e)},this.renderHeadAxis=(e,t="")=>{let{options:n}=this.context,{dateProfile:r}=this.props,i=r.renderRange,s=1===xt(i.start,i.end)?qi(this.c
ontext,i.start,"week"):{};return n.weekNumbers&&"day"===e?p(vo,{elTag:"th",elClasses:["fc-timegrid-axis","fc-scrollgrid-shrink"],elAttrs:{"aria-hidden":!0},date:i.start,defau
ltFormat:Pl},e=>p("div",{className:["fc-timegrid-axis-frame","fc-scrollgrid-shrink-frame","fc-timegrid-axis-frame-liquid"].join(" "),style:{height:t}},p(e,{elTag:"a",elClasse
s:["fc-timegrid-axis-cushion","fc-scrollgrid-shrink-cushion","fc-scrollgrid-sync-inner"],elAttrs:s}))):p("th",{"aria-hidden":!0,className:"fc-timegrid-axis"},p("div",{className:"fc-timegrid-axis-frame",style:{height:t}}))},this.renderTableRowAxis=e=>{let{options:t,viewApi:n}=this.context,r={text:t.allDayText,view:n};return p(Jn,{elTag:"td",elClas
ses:["fc-timegrid-axis","fc-scrollgrid-shrink"],elAttrs:{"aria-hidden":!0},renderProps:r,generatorName:"allDayContent",customGenerator:t.allDayContent,defaultGenerator:Bl,cla
ssNameGenerator:t.allDayClassNames,didMount:t.allDayDidMount,willUnmount:t.allDayWillUnmount},t=>p("div",{className:["fc-timegrid-axis-frame","fc-scrollgrid-shrink-frame",nul
l==e?" fc-timegrid-axis-frame-liquid":""].join(" "),style:{height:e}},p(t,{elTag:"span",elClasses:["fc-timegrid-axis-cushion","fc-scrollgrid-shrink-cushion","fc-scrollgrid-sy
nc-inner"]})))},this.handleSlatCoords=e=>{this.setState({slatCoords:e})}}renderSimpleLayout(e,t,n){let{context:r,props:i}=this,s=[],o=no(r.options);return e&&s.push({type:"he
ader",key:"header",isSticky:o,chunk:{elRef:this.headerElRef,tableClassName:"fc-col-header",rowContent:e}}),t&&(s.push({type:"body",key:"all-day",chunk:{content:t}}),s.push({t
```

```
ype:"body",key:"all-day-divider",outerContent:p("tr",{role:"presentation",className:"fc-scrollgrid-section"},p("td",{className:"fc-timegrid-divider "+r.theme.getClass("tableC
ellShaded")})))))),s.push({type:"body",key:"body",liquid:!0,expandRows:Boolean(r.options.expandRows),chunk:{scrollerElRef:this.scrollerElRef,content:n}}),p(er,{elRef:this.root
ElRef,elClasses:["fc-timegrid"],viewSpec:r.viewSpec},p(io,{liquid:!i.isHeightAuto&&!i.forPrint,collapsibleWidth:i.forPrint,cols:[{width:"shrink"}],sections:s})))renderHScroll
Layout(e,t,n,r,i,s,o){let a=this.context.pluginHooks.scrollGridImpl;if(!a)throw new Error("No ScrollGrid implementation");let{context:l,props:c}=this,d=!c.forPrint&&no(l.opti
ons),u=!c.forPrint&&ro(l.options),h=[];e&&h.push({type:"header",key:"header",isSticky:d,syncRowHeights:!0,chunks:[{key:"axis",rowContent:e=>p("tr",{role:"presentation"},this.
renderHeadAxis("day",e.rowSyncHeights[0]))},{key:"cols",elRef:this.headerElRef,tableClassName:"fc-col-header",rowContent:e}]}),t&&(h.push({type:"body",key:"all-day",syncRowHe
ights:!0,chunks:[{key:"axis",rowContent:e=>p("tr",{role:"presentation"},this.renderTableRowAxis(e.rowSyncHeights[0]))},{key:"cols",content:t}]}),h.push({key:"all-day-divider"
,type:"body",outerContent:p("tr",{role:"presentation",className:"fc-scrollgrid-section"},p("td",{colSpan:2,className:"fc-timegrid-divider
"+l.theme.getClass("tableCellShaded")})))));let f=l.options.nowIndicator;return h.push({type:"body",key:"body",liquid:!0,expandRows:Boolean(l.options.expandRows),chunks:[{key
:"axis",content:e=>p("div",{className:"fc-timegrid-axis-chunk"},p("table",{"aria-hidden":!0,style:{height:e.expandRows?e.clientHeight:""}},e.tableColGroupNode,p("tbody",null,
p(Nl,{slatMetas:s}))),p("div",{className:"fc-timegrid-now-indicator-container"},p(Rs,{unit:f?"minute":"day"},e=>{let t=f&&o&&o.safeComputeTop(e);return"number"==typeof t?p(lo
,{elClasses:["fc-timegrid-now-indicator-arrow"],elStyle:{top:t},isAxis:!0,date:e}):null})))},{key:"cols",scrollerElRef:this.scrollerElRef,content:n}]}),u&&h.push({key:"footer
",type:"footer",isSticky:!0,chunks:[{key:"axis",content:to},{key:"cols",content:to}]}),p(er,{elRef:this.rootElRef,elClasses:["fc-timegrid"],viewSpec:l.viewSpec},p(a,{liquid:!
c.isHeightAuto&&!c.forPrint,forPrint:c.forPrint,collapsibleWidth:!1,colGroups:[{width:"shrink",cols:[{width:"shrink"}]},{cols:[{span:r,minWidth:i}]}],sections:h}))}getAllDayM
axEventProps(){let{dayMaxEvents:e,dayMaxEventRows:t}=this.context.options;return!0!==e&&!0!==t||(e=void 0,t=5),{dayMaxEvents:e,dayMaxEventRows:t}}}function Bl(e){return
e.text}class jl{constructor(e,t,n){this.positions=e,this.dateProfile=t,this.slotDuration=n}safeComputeTop(e){let{dateProfile:t}=this;if(lr(t.currentRange,e)){let
n=Mt(e),r=e.valueOf()-n.valueOf();if(r>=yt(t.slotMinTime)&&r<yt(t.slotMaxTime))return this.computeTimeTop(ft(r))}return null}computeDateTop(e,t){return
t||(t=Mt(e)),this.computeTimeTop(ft(e.valueOf()-t.valueOf()))}computeTimeTop(e){let
t,n,{positions:r,dateProfile:i}=this,s=r.els.length,o=(e.milliseconds-yt(i.slotMinTime))/yt(this.slotDuration);return
o=Math.max(0,o),o=Math.min(s,o),t=Math.floor(o),t=Math.min(t,s-1),n=o-t,r.tops[t]+r.getHeight(t)*n}}class zl extends
Gn{render(){let{props:e,context:t}=this,{options:n}=t,{slatElRefs:r}=e;return p("tbody",null,e.slatMetas.map((i,s)=>{let
o={time:i.time,date:t.dateEnv.toDate(i.date),view:t.viewApi};return p("tr",{key:i.key,ref:r.createRef(i.key)},e.axis&&p(Il,Object.assign({},i)),p(Jn,{elTag:"td",elClasses:["f
c-timegrid-slot","fc-timegrid-slot-lane",!i.isLabeled&&"fc-timegrid-slot-minor"],elAttrs:{"data-time":i.isoTimeStr},renderProps:o,generatorName:"slotLaneContent",customGenera
tor:n.slotLaneContent,classNameGenerator:n.slotLaneClassNames,didMount:n.slotLaneDidMount,willUnmount:n.slotLaneWillUnmount})})))}class Ul extends
Gn{constructor(){super(...arguments),this.rootElRef={current:null},this.slatElRefs=new Vs}render(){let{props:e,context:t}=this;return p("div",{ref:this.rootElRef,className:"f
c-timegrid-slots"},p("table",{"aria-hidden":!0,className:t.theme.getClass("table"),style:{minWidth:e.tableMinWidth,width:e.clientWidth,height:e.minHeight}},e.tableColGroupNod
e,p(zl,{slatElRefs:this.slatElRefs,axis:e.axis,slatMetas:e.slatMetas})))}componentDidMount(){this.updateSizing()}componentDidUpdate(){this.updateSizing()}componentWillUnmount
(){this.props.onCoords&&this.props.onCoords(null)}updateSizing(){let{context:e,props:t}=this;if(t.onCoords&&null!==t.clientWidth){this.rootElRef.current.offsetHeight&&t.onCoo
rds(new jl(new rs(this.rootElRef.current,(n=this.slatElRefs.currentMap,t.slatMetas.map(e=>n[e.key])),!1,!0),this.props.dateProfile,e.options.slotDuration))}var n}}function
Wl(e,t){let n,r=[];for(n=0;n<t;n+=1)r.push([]);if(e)for(n=0;n<e.length;n+=1)r[e[n].col].push(e[n]);return r}function Ll(e,t){let n=[];if(e){for(let
r=0;r<t;r+=1)n[r]={affectedInstances:e.affectedInstances,isEvent:e.isEvent,segs:[]};for(let t of e.segs)n[t.col].segs.push(t)}else for(let e=0;e<t;e+=1)n[e]=null;return
n}class Fl extends Gn{render(){let{props:e}=this;return p(So,{elClasses:["fc-timegrid-more-link"],elStyle:{top:e.top,bottom:e.bottom},allDayDate:null,moreCnt:e.hiddenSegs.len
gth,allSegs:e.hiddenSegs,hiddenSegs:e.hiddenSegs,extraDateSpan:e.extraDateSpan,dateProfile:e.dateProfile,todayRange:e.todayRange,popoverContent:()=>ec(e.hiddenSegs,e),default
Generator:Vl,forceTimed:!0},e=>p(e,{elTag:"div",elClasses:["fc-timegrid-more-link-inner","fc-sticky"]}))}}function Vl(e){return e.shortText}function Gl(e,t,n){let r=new
cs;null!=t&&(r.strictOrder=t),null!=n&&(r.maxStackCnt=n);let i=hs(r.addSegs(e)),s=function(e){const{entriesByLevel:t}=e,n=Zl((e,t)=>e+":"+t,(r,i)=>{let
s=Ql(function(e,t,n){let{levelCoords:r,entriesByLevel:i}=e,s=i[t][n],o=r[t]+s.thickness,a=r.length,l=t;for(;l<a&&r[l]<o;l+=1);for(;l<a;l+=1){let
e,t=i[l],n=ps(t,s.span.start,ds),r=n[0]+n[1],o=r;for(;(e=t[o])&&e.span.start<s.span.end;)o+=1;if(r<o)return{level:l,lateralStart:r,lateralEnd:o}}return
null}(e,r,i),n),o=t[r][i];return[Object.assign(Object.assign({},o),{nextLevelNodes:s[0]}),o.thickness+s[1]]});return
Ql(t.length?{level:0,lateralStart:0,lateralEnd:t[0].length}:null,n)[0]}(r);return s=function(e,t){const n=Zl((e,t,n)=>us(e),(e,r,i)=>{let
s,{nextLevelNodes:o,thickness:a}=e,l=a+i,c=a/l,d=[];if(o.length)for(let e of o)if(void 0===s){let t=n(e,r,l);s=t[0],d.push(t[1])}else{let t=n(e,s,0);d.push(t[1])}else s=t;let
 u=(s-r)*c;return[s-u,Object.assign(Object.assign({},e),{thickness:u,nextLevelNodes:d})]});return e.map(e=>n(e,0,0)[1])}(s,1),{segRects:function(e){let t=[];const
n=Zl((e,t,n)=>us(e),(e,n,i)=>{let s=Object.assign(Object.assign({},e),{levelCoord:n,stackDepth:i,stackForward:0});return
t.push(s),s.stackForward=r(e.nextLevelNodes,n+e.thickness,i+1)+1});function r(e,t,r){let i=0;for(let s of e)i=Math.max(n(s,t,r),i);return i}return
r(e,0,0),t}(s),hiddenGroups:i}}function Ql(e,t){if(!e)return[[],0];let{level:n,lateralStart:r,lateralEnd:i}=e,s=r,o=[];for(;s<i;)o.push(t(n,s)),s+=1;return
o.sort(ql),[o.map(Yl),o[0][1]]}function ql(e,t){return t[1]-e[1]}function Yl(e){return e[0]}function Zl(e,t){const n={};return(...r)=>{let i=e(...r);return i in
n?n[i]:n[i]=t(...r)}}function Xl(e,t,n=null,r=0){let i=[];if(n)for(let s=0;s<e.length;s+=1){let
o=e[s],a=n.computeDateTop(o.start,t),l=Math.max(a+(r||0),n.computeDateTop(o.end,t));i.push({start:Math.round(a),end:Math.round(l)})}return i}const
$l=an({hour:"numeric",minute:"2-digit",meridiem:!1});class Jl extends Gn{render(){return
p(oo,Object.assign({},this.props,{elClasses:["fc-timegrid-event","fc-v-event",this.props.isShort&&"fc-timegrid-event-short"],defaultTimeFormat:$l}))}}class Kl extends Gn{cons
tructor(){super(...arguments),this.sortEventSegs=Gt(di)}render(){let{props:e,context:t}=this,{options:n}=t,r=n.selectMirror,i=e.eventDrag&&e.eventDrag.segs||e.eventResize&&e.
eventResize.segs||r&&e.dateSelectionSegs||[],s=e.eventDrag&&e.eventDrag.affectedInstances||e.eventResize&&e.eventResize.affectedInstances||{},o=this.sortEventSegs(e.fgEventSe
gs,n.eventOrder);return p(uo,{elTag:"td",elRef:e.elRef,elClasses:["fc-timegrid-col",...e.extraClassNames||[]],elAttrs:Object.assign({role:"gridcell"},e.extraDataAttrs),date:e
.date,dateProfile:e.dateProfile,todayRange:e.todayRange,extraRenderProps:e.extraRenderProps},t=>p("div",{className:"fc-timegrid-col-frame"},p("div",{className:"fc-timegrid-co
l-bg"},this.renderFillSegs(e.businessHourSegs,"non-business"),this.renderFillSegs(e.bgEventSegs,"bg-event"),this.renderFillSegs(e.dateSelectionSegs,"highlight")),p("div",{cla
ssName:"fc-timegrid-col-events"},this.renderFgSegs(o,s,!1,!1,!1)),p("div",{className:"fc-timegrid-col-events"},this.renderFgSegs(i,{},Boolean(e.eventDrag),Boolean(e.eventResi
ze),Boolean(r),"mirror")),p("div",{className:"fc-timegrid-now-indicator-container"},this.renderNowIndicator(e.nowIndicatorSegs)),ho(n)&&p(t,{elTag:"div",elClasses:["fc-timegr
id-col-misc"]})))}renderFgSegs(e,t,n,r,i,s){let{props:o}=this;return o.forPrint?ec(e,o):this.renderPositionedFgSegs(e,t,n,r,i,s)}renderPositionedFgSegs(e,t,n,r,i,s){let{event
MaxStack:o,eventShortHeight:a,eventOrderStrict:l,eventMinHeight:c}=this.context.options,{date:d,slatCoords:u,eventSelection:h,todayRange:f,nowDate:g}=this.props,m=n||r||i,v=X
l(e,d,u,c),{segPlacements:b,hiddenGroups:E}=function(e,t,n,r){let i=[],s=[];for(let n=0;n<e.length;n+=1){let
r=t[n];r?i.push({index:n,thickness:1,span:r}):s.push(e[n])}let{segRects:o,hiddenGroups:a}=Gl(i,n,r),l=[];for(let t of o)l.push({seg:e[t.index],rect:t});for(let e of
s)l.push({seg:e,rect:null});return{segPlacements:l,hiddenGroups:a}}(e,v,l,o);return p(y,null,this.renderHiddenGroups(E,e),b.map(e=>{let{seg:o,rect:l}=e,c=o.eventRange.instanc
e.instanceId,d=m||Boolean(!t[c]&&l),u=tc(l&&l.span),v=!m&&l?this.computeSegHStyle(l):{left:0,right:0},y=Boolean(l)&&l.stackForward>0,b=Boolean(l)&&l.span.end-l.span.start<a;r
eturn p("div",{className:"fc-timegrid-event-harness"+(y?" fc-timegrid-event-harness-inset":""),key:s||c,style:Object.assign(Object.assign({visibility:d?"":"hidden"},u),v)},p(
Jl,Object.assign({seg:o,isDragging:n,isResizing:r,isDateSelecting:i,isSelected:c===h,isShort:b},mi(o,f,g)))))}renderHiddenGroups(e,t){let{extraDateSpan:n,dateProfile:r,toda
yRange:i,nowDate:s,eventSelection:o,eventDrag:a,eventResize:l}=this.props;return p(y,null,e.map(e=>{let c=tc(e.span),d=(u=e.entries,h=t,u.map(e=>h[e.index]));var u,h;return p
(Fl,{key:Ut(wo(d)),hiddenSegs:d,top:c.top,bottom:c.bottom,extraDateSpan:n,dateProfile:r,todayRange:i,nowDate:s,eventSelection:o,eventDrag:a,eventResize:l})}))}renderFillSegs(
e,t){let{props:n,context:r}=this,i=Xl(e,n.date,n.slatCoords,r.options.eventMinHeight).map((r,i)=>{let s=e[i];return
p("div",{key:yi(s.eventRange),className:"fc-timegrid-bg-harness",style:tc(r)},"bg-event"===t?p(go,Object.assign({seg:s},mi(s,n.todayRange,n.nowDate))):mo(t))});return
p(y,null,i)}renderNowIndicator(e){let{slatCoords:t,date:n}=this.props;return
t?e.map((e,r)=>p(lo,{key:r,elClasses:["fc-timegrid-now-indicator-line"],elStyle:{top:t.computeDateTop(e.start,n)},isAxis:!1,date:n})):null}computeSegHStyle(e){let
t,n,{isRtl:r,options:i}=this.context,s=i.slotEventOverlap,o=e.levelCoord,a=e.levelCoord+e.thickness;s&&(a=Math.min(1,o+2*(a-o))),r?(t=1-a,n=o):(t=o,n=1-a);let
l={zIndex:e.stackDepth+1,left:100*t+"%",right:100*n+"%"};return s&&!e.stackForward&&(l[r?"marginLeft":"marginRight"]=20),l}}function
ec(e,{todayRange:t,nowDate:n,eventSelection:r,eventDrag:i,eventResize:s}){let o=(i?i.affectedInstances:null)||(s?s.affectedInstances:null)||{};return p(y,null,e.map(e=>{let
i=e.eventRange.instance.instanceId;return p("div",{key:i,style:{visibility:o[i]?"hidden":""}},p(Jl,Object.assign({seg:e,isDragging:!1,isResizing:!1,isDateSelecting:!1,isSelec
```

```
ted:i===r,isShort:!1},mi(e,t,n)))))}}function tc(e){return e?{top:e.start,bottom:-e.end}:{top:"",bottom:""}}class nc extends Gn{constructor(){super(...arguments),this.splitF
gEventSegs=Gt(Wl),this.splitBgEventSegs=Gt(Wl),this.splitBusinessHourSegs=Gt(Wl),this.splitNowIndicatorSegs=Gt(Wl),this.splitDateSelectionSegs=Gt(Wl),this.splitEventDrag=Gt(L
l),this.splitEventResize=Gt(Ll),this.rootElRef={current:null},this.cellElRefs=new Vs}render(){let{props:e,context:t}=this,n=t.options.nowIndicator&&e.slatCoords&&e.slatCoords
.safeComputeTop(e.nowDate),r=e.cells.length,i=this.splitFgEventSegs(e.fgEventSegs,r),s=this.splitBgEventSegs(e.bgEventSegs,r),o=this.splitBusinessHourSegs(e.businessHourSegs,
r),a=this.splitNowIndicatorSegs(e.nowIndicatorSegs,r),l=this.splitDateSelectionSegs(e.dateSelectionSegs,r),c=this.splitEventDrag(e.eventDrag,r),d=this.splitEventResize(e.even
tResize,r);return p("div",{className:"fc-timegrid-cols",ref:this.rootElRef},p("table",{role:"presentation",style:{minWidth:e.tableMinWidth,width:e.clientWidth}},e.tableColGro
upNode,p("tbody",{role:"presentation"},p("tr",{role:"row"},e.axis&&p("td",{"aria-hidden":!0,className:"fc-timegrid-col
fc-timegrid-axis"},p("div",{className:"fc-timegrid-col-frame"},p("div",{className:"fc-timegrid-now-indicator-container"},"number"==typeof n&&p(lo,{elClasses:["fc-timegrid-now
-indicator-arrow"],elStyle:{top:n},isAxis:!0,date:e.nowDate})))),e.cells.map((t,n)=>p(Kl,{key:t.key,elRef:this.cellElRefs.createRef(t.key),dateProfile:e.dateProfile,date:t.da
te,nowDate:e.nowDate,todayRange:e.todayRange,extraRenderProps:t.extraRenderProps,extraDataAttrs:t.extraDataAttrs,extraClassNames:t.extraClassNames,extraDateSpan:t.extraDateSp
an,fgEventSegs:i[n],bgEventSegs:s[n],businessHourSegs:o[n],nowIndicatorSegs:a[n],dateSelectionSegs:l[n],eventDrag:c[n],eventResize:d[n],slatCoords:e.slatCoords,eventSelection
:e.eventSelection,forPrint:e.forPrint})))))}componentDidMount(){this.updateCoords()}componentDidUpdate(){this.updateCoords()}updateCoords(){let{props:e}=this;var
t;e.onColCoords&&null!==e.clientWidth&&e.onColCoords(new rs(this.rootElRef.current,(t=this.cellElRefs.currentMap,e.cells.map(e=>t[e.key])),!0,!1))}}class rc extends ls{constr
uctor(){super(...arguments),this.processSlotOptions=Gt(ic),this.state={slatCoords:null},this.handleRootEl=e=>{e?this.context.registerInteractiveComponent(this,{el:e,isHitComb
oAllowed:this.props.isHitComboAllowed}):this.context.unregisterInteractiveComponent(this)},this.handleScrollRequest=e=>{let{onScrollTopRequest:t}=this.props,{slatCoords:n}=th
is.state;if(t&&n){if(e.time){let r=n.computeTimeTop(e.time);r=Math.ceil(r),r&&(r+=1),t(r)}return!0}return!1},this.handleColCoords=e=>{this.colCoords=e},this.handleSlatCoords=
e=>{this.setState({slatCoords:e}),this.props.onSlatCoords&&this.props.onSlatCoords(e)}}render(){let{props:e,state:t}=this;return p("div",{className:"fc-timegrid-body",ref:thi
s.handleRootEl,style:{width:e.clientWidth,minWidth:e.tableMinWidth}},p(Ul,{axis:e.axis,dateProfile:e.dateProfile,slatMetas:e.slatMetas,clientWidth:e.clientWidth,minHeight:e.e
xpandRows?e.clientHeight:"",tableMinWidth:e.tableMinWidth,tableColGroupNode:e.axis?e.tableColGroupNode:null,onCoords:this.handleSlatCoords}),p(nc,{cells:e.cells,axis:e.axis,d
ateProfile:e.dateProfile,businessHourSegs:e.businessHourSegs,bgEventSegs:e.bgEventSegs,fgEventSegs:e.fgEventSegs,dateSelectionSegs:e.dateSelectionSegs,eventSelection:e.eventS
election,eventDrag:e.eventDrag,eventResize:e.eventResize,todayRange:e.todayRange,nowDate:e.nowDate,nowIndicatorSegs:e.nowIndicatorSegs,clientWidth:e.clientWidth,tableMinWidth
:e.tableMinWidth,tableColGroupNode:e.tableColGroupNode,slatCoords:t.slatCoords,onColCoords:this.handleColCoords,forPrint:e.forPrint}))}componentDidMount(){this.scrollResponde
r=this.context.createScrollResponder(this.handleScrollRequest)}componentDidUpdate(e){this.scrollResponder.update(e.dateProfile!==this.props.dateProfile)}componentWillUnmount(
){this.scrollResponder.detach()}queryHit(e,t){let{dateEnv:n,options:r}=this.context,{colCoords:i}=this,{dateProfile:s}=this.props,{slatCoords:o}=this.state,{snapDuration:a,sn
apsPerSlot:l}=this.processSlotOptions(this.props.slotDuration,r.snapDuration),c=i.leftToIndex(e),d=o.positions.topToIndex(t);if(null!=c&&null!=d){let e=this.props.cells[c],r=
o.positions.tops[d],u=o.positions.getHeight(d),h=(t-r)/u,f=d*l+Math.floor(h*l),g=this.props.cells[c].date,p=pt(s.slotMinTime,mt(a,f)),m=n.add(g,p),v=n.add(m,a);return{datePro
file:s,dateSpan:Object.assign({range:{start:m,end:v},allDay:!1},e.extraDateSpan),dayEl:i.els[c],rect:{left:i.lefts[c],right:i.rights[c],top:r,bottom:r+u},layer:0}}return
null}}function ic(e,t){let n=t||e,r=bt(e,n);return null===r&&(n=e,r=1),{snapDuration:n,snapsPerSlot:r}}class sc extends Is{sliceRange(e,t){let n=[];for(let
r=0;r<t.length;r+=1){let i=ir(e,t[r]);i&&n.push({start:i.start,end:i.end,isStart:i.start.valueOf()===e.start.valueOf(),isEnd:i.end.valueOf()===e.end.valueOf(),col:r})}return
n}}class oc extends ls{constructor(){super(...arguments),this.buildDayRanges=Gt(ac),this.slicer=new sc,this.timeColsRef=[current:null}}render(){let{props:e,context:t}=this,{d
ateProfile:n,dayTableModel:r}=e,{nowIndicator:i,nextDayThreshold:s}=t.options,o=this.buildDayRanges(r,n,t.dateEnv);return p(Rs,{unit:i?"minute":"day"},(a,l)=>p(rc,Object.assi
gn({ref:this.timeColsRef},this.slicer.sliceProps(e,n,null,t,o),{forPrint:e.forPrint,axis:e.axis,dateProfile:n,slatMetas:e.slatMetas,slotDuration:e.slotDuration,cells:r.cells[
0],tableColGroupNode:e.tableColGroupNode,tableMinWidth:e.tableMinWidth,clientWidth:e.clientWidth,clientHeight:e.clientHeight,expandRows:e.expandRows,nowDate:a,nowIndicatorSeg
s:i&&this.slicer.sliceNowDate(a,n,s,t,o),todayRange:l,onScrollTopRequest:e.onScrollTopRequest,onSlatCoords:e.onSlatCoords}))))}function ac(e,t,n){let r=[];for(let i of
e.headerDates)r.push({start:n.add(i,t.slotMinTime),end:n.add(i,t.slotMaxTime)});return r}const lc=[{hours:1},{minutes:30},{minutes:15},{seconds:30},{seconds:15}];function
cc(e,t,n,r,i){let s=new Date(0),o=e,a=ft(0),l=n||function(e){let t,n,r;for(t=lc.length-1;t>=0;t-=1)if(n=ft(lc[t]),r=bt(n,e),null!==r&&r>1)return n;return
e}(r),c=[];for(;yt(o)<yt(t);){let e=i.add(s,o),t=null!==bt(a,l);c.push({date:e,time:o,key:e.toISOString(),isoTimeStr:Ft(e),isLabeled:t}),o=pt(o,r),a=pt(a,r)}return c}function
 dc(e,t){let n=new ks(e.renderRange,t);return new Ms(n,!1)}xe('.fc-v-event{background-color:var(--fc-event-bg-color);border:1px solid
var(--fc-event-border-color);display:block}.fc-v-event .fc-event-main{color:var(--fc-event-text-color);height:100%}.fc-v-event
.fc-event-main-frame{display:flex;flex-direction:column;height:100%}.fc-v-event .fc-event-time{flex-grow:0;flex-shrink:0;max-height:100%;overflow:hidden}.fc-v-event
.fc-event-title-container{flex-grow:1;flex-shrink:1;min-height:0}.fc-v-event .fc-event-title{bottom:0;max-height:100%;overflow:hidden;top:0}.fc-v-event:not(.fc-event-start){b
order-top-left-radius:0;border-top-right-radius:0;border-top-width:0}.fc-v-event:not(.fc-event-end){border-bottom-left-radius:0;border-bottom-right-radius:0;border-bottom-wid
th:0}.fc-v-event.fc-event-selected:before{left:-10px;right:-10px}.fc-v-event .fc-event-resizer-start{cursor:n-resize}.fc-v-event
.fc-event-resizer-end{cursor:s-resize}.fc-v-event:not(.fc-event-selected)
.fc-event-resizer{height:var(--fc-event-resizer-thickness);left:0;right:0}.fc-v-event:not(.fc-event-selected)
.fc-event-resizer-start{top:calc(var(--fc-event-resizer-thickness)/-2)}.fc-v-event:not(.fc-event-selected)
.fc-event-resizer-end{bottom:calc(var(--fc-event-resizer-thickness)/-2)}.fc-v-event.fc-event-selected
.fc-event-resizer{left:50%;margin-left:calc(var(--fc-event-resizer-dot-total-width)/-2)}.fc-v-event.fc-event-selected
.fc-event-resizer-start{top:calc(var(--fc-event-resizer-dot-total-width)/-2)}.fc-v-event.fc-event-selected
.fc-event-resizer-end{bottom:calc(var(--fc-event-resizer-dot-total-width)/-2)}.fc .fc-timegrid .fc-daygrid-body{z-index:2}.fc .fc-timegrid-divider{padding:0 0 2px}.fc
.fc-timegrid-body{min-height:100%;position:relative;z-index:1}.fc .fc-timegrid-axis-chunk{position:relative}.fc .fc-timegrid-axis-chunk>table,.fc
.fc-timegrid-slots{position:relative;z-index:1}.fc .fc-timegrid-slot{border-bottom:0;height:1.5em}.fc .fc-timegrid-slot:empty:before{content:"\\00a0"}.fc
.fc-timegrid-slot-minor{border-top-style:dotted}.fc .fc-timegrid-slot-label-cushion{display:inline-block;white-space:nowrap}.fc
.fc-timegrid-slot-label{vertical-align:middle}.fc .fc-timegrid-axis-cushion,.fc .fc-timegrid-slot-label-cushion{padding:0 4px}.fc
.fc-timegrid-axis-frame-liquid{height:100%}.fc .fc-timegrid-axis-frame{align-items:center;display:flex;justify-content:flex-end;overflow:hidden}.fc
.fc-timegrid-axis-cushion{flex-shrink:0;max-width:60px}.fc-direction-ltr .fc-timegrid-slot-label-frame{text-align:right}.fc-direction-rtl
.fc-timegrid-slot-label-frame{text-align:left}.fc-liquid-hack .fc-timegrid-axis-frame-liquid{bottom:0;height:auto;left:0;position:absolute;right:0;top:0}.fc
.fc-timegrid-col.fc-day-today{background-color:var(--fc-today-bg-color)}.fc .fc-timegrid-col-frame{min-height:100%;position:relative}.fc-media-screen.fc-liquid-hack
.fc-timegrid-col-frame{bottom:0;height:auto;left:0;position:absolute;right:0;top:0}.fc-media-screen
.fc-timegrid-cols{bottom:0;left:0;position:absolute;right:0;top:0}.fc-media-screen .fc-timegrid-cols>table{height:100%}.fc-media-screen .fc-timegrid-col-bg,.fc-media-screen
.fc-timegrid-col-events,.fc-media-screen .fc-timegrid-now-indicator-container{left:0;position:absolute;right:0;top:0}.fc .fc-timegrid-col-bg{z-index:2}.fc .fc-timegrid-col-bg
 .fc-non-business{z-index:1}.fc .fc-timegrid-col-bg .fc-bg-event{z-index:2}.fc .fc-timegrid-col-bg .fc-highlight{z-index:3}.fc
.fc-timegrid-bg-harness{left:0;position:absolute;right:0}.fc .fc-timegrid-col-events{z-index:3}.fc
.fc-timegrid-now-indicator-container{bottom:0;overflow:hidden}.fc-direction-ltr .fc-timegrid-col-events{margin:0 2.5% 0 2px}.fc-direction-rtl .fc-timegrid-col-events{margin:0
 2px 0 2.5%}.fc-timegrid-event-harness{position:absolute}.fc-timegrid-event-harness>.fc-timegrid-event{bottom:0;left:0;position:absolute;right:0;top:0}.fc-timegrid-event-harn
ess-inset .fc-timegrid-event,.fc-timegrid-event.fc-event-mirror,.fc-timegrid-more-link{box-shadow:0 0 0 1px var(--fc-page-bg-color)}.fc-timegrid-event,.fc-timegrid-more-link{
border-radius:3px;font-size:var(--fc-small-font-size)}.fc-timegrid-event{margin-bottom:1px}.fc-timegrid-event .fc-event-main{padding:1px 1px 0}.fc-timegrid-event
.fc-event-time{font-size:var(--fc-small-font-size);margin-bottom:1px;white-space:nowrap}.fc-timegrid-event-short
.fc-event-main-frame{flex-direction:row;overflow:hidden}.fc-timegrid-event-short .fc-event-time:after{content:"\\00a0-\\00a0"}.fc-timegrid-event-short .fc-event-title{font-si
ze:var(--fc-small-font-size)}.fc-timegrid-more-link{background:var(--fc-more-link-bg-color);color:var(--fc-more-link-text-color);cursor:pointer;margin-bottom:1px;position:abs
olute;z-index:9999}.fc-timegrid-more-link-inner{padding:3px 2px;top:0}.fc-direction-ltr .fc-timegrid-more-link{right:0}.fc-direction-rtl .fc-timegrid-more-link{left:0}.fc
.fc-timegrid-now-indicator-line{border-color:var(--fc-now-indicator-color);border-style:solid;border-width:1px 0 0;left:0;position:absolute;right:0;z-index:4}.fc
.fc-timegrid-now-indicator-arrow{border-color:var(--fc-now-indicator-color);border-style:solid;margin-top:-5px;position:absolute;z-index:4}.fc-direction-ltr
.fc-timegrid-now-indicator-arrow{border-bottom-color:transparent;border-top-color:transparent;border-width:5px 0 5px 6px;left:0}.fc-direction-rtl
```

```
.fc-timegrid-now-indicator-arrow{border-bottom-color:transparent;border-top-color:transparent;border-width:5px 6px 5px 0;right:0}');var
uc=Po({name:"@fullcalendar/timegrid",initialView:"timeGridWeek",optionRefiners:{allDaySlot:Boolean},views:{timeGrid:{component:class extends Hl{constructor(){super(...argumen
ts),this.buildTimeColsModel=Gt(dc),this.buildSlatMetas=Gt(cc)}render(){let{options:e,dateEnv:t,dateProfileGenerator:n}=this.context,{props:r}=this,{dateProfile:i}=r,s=this.bu
ildTimeColsModel(i,n),o=this.allDaySplitter.splitProps(r),a=this.buildSlatMetas(i.slotMinTime,i.slotMaxTime,e.slotLabelInterval,e.slotDuration,t),{dayMinWidth:l}=e,c=!l,d=l,u
=e.dayHeaders&&p(_s,{dates:s.headerDates,dateProfile:i,datesRepDistinctDays:!0,renderIntro:c?this.renderHeadAxis:null}),h=!1!==e.allDaySlot&&(t=>p(Cl,Object.assign({},o.allDa
y,{dateProfile:i,dayTableModel:s,nextDayThreshold:e.nextDayThreshold,tableMinWidth:t.tableMinWidth,colGroupNode:t.tableColGroupNode,renderRowIntro:c?this.renderTableRowAxis:n
ull,showWeekNumbers:!1,expandRows:!1,headerAlignElRef:this.headerElRef,clientWidth:t.clientWidth,clientHeight:t.clientHeight,forPrint:r.forPrint},this.getAllDayMaxEventProps(
))),f=t=>p(oc,Object.assign({},o.timed,{dayTableModel:s,dateProfile:i,axis:c,slotDuration:e.slotDuration,slatMetas:a,forPrint:r.forPrint,tableColGroupNode:t.tableColGroupNod
e,tableMinWidth:t.tableMinWidth,clientWidth:t.clientWidth,clientHeight:t.clientHeight,onSlatCoords:this.handleSlatCoords,expandRows:t.expandRows,onScrollTopRequest:this.handl
eScrollTopRequest}));return d?this.renderHScrollLayout(u,h,f,s.colCnt,l,a,this.state.slatCoords):this.renderSimpleLayout(u,h,f)}},usesMinMaxTime:!0,allDaySlot:!0,slotDuration
:"00:30:00",slotEventOverlap:!0},timeGridDay:{type:"timeGrid",duration:{days:1}},timeGridWeek:{type:"timeGrid",duration:{weeks:1}}}});class hc extends Gn{constructor(){super(
...arguments),this.state={textId:We()}}render(){let{theme:e,dateEnv:t,options:n,viewApi:r}=this.context,{cellId:i,dayDate:s,todayRange:o}=this.props,{textId:a}=this.state,l=F
i(s,o),c=n.listDayFormat?t.format(s,n.listDayFormat):"",d=n.listDaySideFormat?t.format(s,n.listDaySideFormat):"",u=Object.assign({date:t.toDate(s),view:r,textId:a,text:c,side
Text:d,navLinkAttrs:qi(this.context,s),sideNavLinkAttrs:qi(this.context,s,"day",!1)},l);return p(Jn,{elTag:"tr",elClasses:["fc-list-day",...Vi(l,e)],elAttrs:{"data-date":Wt(s
)},renderProps:u,generatorName:"dayHeaderContent",customGenerator:n.dayHeaderContent,defaultGenerator:fc,classNameGenerator:n.dayHeaderClassNames,didMount:n.dayHeaderDidMount
,willUnmount:n.dayHeaderWillUnmount},t=>p("th",{scope:"colgroup",colSpan:3,id:i,"aria-labelledby":a},p(t,{elTag:"div",elClasses:["fc-list-day-cushion",e.getClass("tableCellSh
aded")]}))))}}function fc(e){return p(y,null,e.text&&p("a",Object.assign({id:e.textId,className:"fc-list-day-text"},e.navLinkAttrs),e.text),e.sideText&&p("a",Object.assign({"a
ria-hidden":!0,className:"fc-list-day-side-text"},e.sideNavLinkAttrs),e.sideText))}const gc=an({hour:"numeric",minute:"2-digit",meridiem:"short"});class pc extends
Gn{render(){let{props:e,context:t}=this,{options:n}=t,{seg:r,timeHeaderId:i,eventHeaderId:s,dateHeaderId:o}=e,a=n.eventTimeFormat||gc;return
p(so,Object.assign({},e,{elTag:"tr",elClasses:["fc-list-event",r.eventRange.def.url&&"fc-event-forced-url"],defaultGenerator:()=>function(e,t){let n=bi(e,t);return p("a",Objec
t.assign({},n),e.eventRange.def.title)}(r,t),seg:r,timeText:"",disableDragging:!0,disableResizing:!0}),(e,n)=>p(y,null,function(e,t,n,r,i){let{options:s}=n;if(!1!==s.display
EventTime){let o,a=e.eventRange.def,l=e.eventRange.instance,c=!1;if(a.allDay?c=!0:ur(e.eventRange.range)?e.isStart?o=pi(e,t,n,null,null,l.range.start,e.end):e.isEnd?o=pi(e,t,
n,null,null,e.start,l.range.end):c=!0:o=pi(e,t,n),c){let e={text:n.options.allDayText,view:n.viewApi};return
p(Jn,{elTag:"td",elClasses:["fc-list-event-time"],elAttrs:{headers:`${r} ${i}`},renderProps:e,generatorName:"allDayContent",customGenerator:s.allDayContent,defaultGenerator:m
c,classNameGenerator:s.allDayClassNames,didMount:s.allDayDidMount,willUnmount:s.allDayWillUnmount})}return p("td",{className:"fc-list-event-time"},o)}return null}(r,a,t,i,o),
p("td",{"aria-hidden":!0,className:"fc-list-event-graphic"},p("span",{className:"fc-list-event-dot",style:{borderColor:n.borderColor||n.backgroundColor}})),p(e,{elTag:"td",el
Classes:["fc-list-event-title"],elAttrs:{headers:`${s} ${o}`}})))}function mc(e){return e.text}function vc(e){return e.text}function yc(e){let t=Mt(e.renderRange.start),n=e.
renderRange.end,r=[],i=[];for(;t<n;)r.push(t),i.push({start:t,end:wt(t,1)}),t=wt(t,1);return{dayDates:r,dayRanges:i}}xe(':root{--fc-list-event-dot-width:10px;--fc-list-event-
hover-bg-color:#f5f5f5}.fc-theme-standard .fc-list{border:1px solid var(--fc-border-color)}.fc
.fc-list-empty{align-items:center;background-color:var(--fc-neutral-bg-color);display:flex;height:100%;justify-content:center}.fc .fc-list-empty-cushion{margin:5em 0}.fc
.fc-list-table{border-style:hidden;width:100%}.fc .fc-list-table tr>*{border-left:0;border-right:0}.fc .fc-list-sticky
.fc-list-day>*{background:var(--fc-page-bg-color);position:sticky;top:0}.fc .fc-list-table thead{left:-10000px;position:absolute}.fc .fc-list-table tbody>tr:first-child
th{border-top:0}.fc .fc-list-table th{padding:0}.fc .fc-list-day-cushion,.fc .fc-list-table td{padding:8px 14px}.fc
.fc-list-day-cushion:after{clear:both;content:"";display:table}.fc-theme-standard .fc-list-day-cushion{background-color:var(--fc-neutral-bg-color)}.fc-direction-ltr
.fc-list-day-text,.fc-direction-rtl .fc-list-day-side-text{float:left}.fc-direction-ltr .fc-list-day-side-text,.fc-direction-rtl
.fc-list-day-text{float:right}.fc-direction-ltr .fc-list-table .fc-list-event-graphic{padding-right:0}.fc-direction-rtl .fc-list-table
.fc-list-event-graphic{padding-left:0}.fc .fc-list-event.fc-event-forced-url{cursor:pointer}.fc .fc-list-event:hover
td{background-color:var(--fc-list-event-hover-bg-color)}.fc .fc-list-event-graphic,.fc .fc-list-event-time{white-space:nowrap;width:1px}.fc
.fc-list-event-dot{border:calc(var(--fc-list-event-dot-width)/2) solid
var(--fc-event-border-color);border-radius:calc(var(--fc-list-event-dot-width)/2);box-sizing:content-box;display:inline-block;height:0;width:0}.fc .fc-list-event-title
a{color:inherit;text-decoration:none}.fc .fc-list-event.fc-event-forced-url:hover a{text-decoration:underline}');function bc(e){return!1===e?null:an(e)}var Ec=Po({name:"@full
calendar/list",optionRefiners:{listDayFormat:bc,listDaySideFormat:bc,noEventsClassNames:yn,noEventsContent:yn,noEventsDidMount:yn,noEventsWillUnmount:yn},views:{list:{compone
nt:class extends ls{constructor(){super(...arguments),this.computeDateVars=Gt(yc),this.eventStoreToSegs=Gt(this._eventStoreToSegs),this.state={timeHeaderId:We(),eventHeaderId
:We(),dateHeaderIdRoot:We()},this.setRootEl=e=>{e?this.context.registerInteractiveComponent(this,{el:e}):this.context.unregisterInteractiveComponent(this)}}render(){let{props
:e,context:t}=this,{dayDates:n,dayRanges:r}=this.computeDateVars(e.dateProfile),i=this.eventStoreToSegs(e.eventStore,e.eventUiBases,r);return p(er,{elRef:this.setRootEl,elCla
sses:["fc-list",t.theme.getClass("table"),!1!==t.options.stickyHeaderDates?"fc-list-sticky":""],viewSpec:t.viewSpec},p(Fs,{liquid:!e.isHeightAuto,overflowX:e.isHeightAuto?"vi
sible":"hidden",overflowY:e.isHeightAuto?"visible":"auto"},i.length>0?this.renderSegList(i,n):this.renderEmptyMessage()))}renderEmptyMessage(){let{options:e,viewApi:t}=this.c
ontext,n={text:e.noEventsText,view:t};return p(Jn,{elTag:"div",elClasses:["fc-list-empty"],renderProps:n,generatorName:"noEventsContent",customGenerator:e.noEventsContent,def
aultGenerator:vc,classNameGenerator:e.noEventsClassNames,didMount:e.noEventsDidMount,willUnmount:e.noEventsWillUnmount},e=>p(e,{elTag:"div",elClasses:["fc-list-empty-cushion"
]}))}renderSegList(e,t){let{theme:n,options:r}=this.context,{timeHeaderId:i,eventHeaderId:s,dateHeaderIdRoot:o}=this.state,a=function(e){let
t,n,r=[];for(t=0;t<e.length;t+=1)n=e[t],(r[n.dayIndex]||(r[n.dayIndex]=[])).push(n);return r}(e);return p(Rs,{unit:"day"},(e,l)=>{let c=[];for(let n=0;n<a.length;n+=1){let
d=a[n];if(d){let a=Wt(t[n]),u=o+"-"+a;c.push(p(hc,{key:a,cellId:u,dayDate:t[n],todayRange:l}));d=di(d,r.eventOrder);for(let t of d)c.push(p(pc,Object.assign({key:a+":"+t.even
tRange.instance.instanceId,seg:t,isDragging:!1,isResizing:!1,isDateSelecting:!1,isSelected:!1,timeHeaderId:i,eventHeaderId:s,dateHeaderId:u},mi(t,l,e))))}return
p("table",{className:"fc-list-table "+n.getClass("table")},p("thead",null,p("tr",null,p("th",{scope:"col",id:i},r.timeHint),p("th",{scope:"col","aria-hidden":!0}),p("th",{sco
pe:"col",id:s},r.eventHint))),p("tbody",null,c))})}}_eventStoreToSegs(e,t,n){return
this.eventRangesToSegs(ii(e,t,this.props.dateProfile.activeRange,this.context.options.nextDayThreshold).fg,n)}eventRangesToSegs(e,t){let n=[];for(let r of
e)n.push(...this.eventRangeToSegs(r,t));return n}eventRangeToSegs(e,t){let n,r,i,{dateEnv:s}=this.context,{nextDayThreshold:o}=this.context.options,a=e.range,l=e.def.allDay,c
=[];for(n=0;n<t.length;n+=1)if(r=ir(a,t[n]),r&&(i={component:this,eventRange:e,start:r.start,end:r.end,isStart:e.isStart&&r.start.valueOf()===a.start.valueOf(),isEnd:e.isEnd&
&r.end.valueOf()===a.end.valueOf(),dayIndex:n},c.push(i),!i.isEnd&&!l&&n+1<t.length&&a.end<s.add(t[n+1].start,o))){i.end=a.end,i.isEnd=!0;break}return c}},buttonTextKey:"list
",listDayFormat:{month:"long",day:"numeric",year:"numeric"}},listDay:{type:"list",duration:{days:1},listDayFormat:{weekday:"long"}},listWeek:{type:"list",duration:{weeks:1},l
istDayFormat:{weekday:"long"},listDaySideFormat:{month:"long",day:"numeric",year:"numeric"}},listMonth:{type:"list",duration:{month:1},listDaySideFormat:{weekday:"long"}},lis
tYear:{type:"list",duration:{year:1},listDaySideFormat:{weekday:"long"}}}});class Sc extends
ls{constructor(){super(...arguments),this.buildDayTableModel=Gt(Rl),this.slicer=new wl,this.state={labelId:We()}}render(){const{props:e,state:t,context:n}=this,{dateProfile:r
,forPrint:i}=e,{options:s}=n,o=this.buildDayTableModel(r,n.dateProfileGenerator),a=this.slicer.sliceProps(e,r,s.nextDayThreshold,n,o),l=null!=e.tableWidth?e.tableWidth/s.aspe
ctRatio:null,c=o.cells.length,d=null!=l?l/c:null;return p("div",{ref:e.elRef,"data-date":e.isoDateStr,className:"fc-multimonth-month",style:{width:e.width},role:"grid","aria-
labelledby":t.labelId},p("div",{className:"fc-multimonth-header",style:{marginBottom:d},role:"presentation"},p("div",{className:"fc-multimonth-title",id:t.labelId},n.dateEnv.
format(e.dateProfile.currentRange.start,e.titleFormat)),p("table",{className:["fc-multimonth-header-table",n.theme.getClass("table")].join(" "),role:"presentation"},p("thead"
,{role:"rowgroup"},p(_s,{dateProfile:e.dateProfile,dates:o.headerDates,datesRepDistinctDays:!1})))),p("div",{className:["fc-multimonth-daygrid","fc-daygrid","fc-daygrid-body"
,!i&&"fc-daygrid-body-balanced",i&&"fc-daygrid-body-unbalanced",i&&"fc-daygrid-body-natural"].join("
"),style:{marginTop:-d}},p("table",{className:["fc-multimonth-daygrid-table",n.theme.getClass("table")].join(" "),style:{height:i?"":l},role:"presentation"},p("tbody",{role:"
rowgroup"},p(Sl,Object.assign({},a,{dateProfile:r,cells:o.cells,eventSelection:e.eventSelection,dayMaxEvents:!i,dayMaxEventRows:!i,showWeekNumbers:s.weekNumbers,clientWidth:e
.clientWidth,clientHeight:e.clientHeight,forPrint:i})))))))}}const Ac=ft(1,"month");function Dc(e,t,n,r,i){const{start:s,end:o}=t.currentRange;let a=s;const
l=[];for(;a.valueOf()<o.valueOf();){const s=n.add(a,Ac),o={start:e.skipHiddenDays(a),end:e.skipHiddenDays(s,-1,!0)};let
c=_l({currentRange:o,snapToWeek:!0,fixedWeekCount:r,dateEnv:n});c={start:e.skipHiddenDays(c.start),end:e.skipHiddenDays(c.end,-1,!0)};const d=t.activeRange?ir(t.activeRange,i
```

```
?c:o):null;l.push({currentDate:t.currentDate,isValid:t.isValid,validRange:t.validRange,renderRange:c,activeRange:d,currentRange:o,currentRangeUnit:"month",isRangeAllDay:!0,da
teIncrement:t.dateIncrement,slotMinTime:t.slotMaxTime,slotMaxTime:t.slotMinTime}),a=s}return l}const wc=an({year:"numeric",month:"long"}),Cc=an({month:"long"});function
Rc(e,t){return e||(t[0].currentRange.start.getUTCFullYear()!==t[t.length-1].currentRange.start.getUTCFullYear()?wc:Cc)}const
xc={multiMonthTitleFormat:an,multiMonthMaxColumns:Number,multiMonthMinWidth:Number};xe(".fc .fc-multimonth{border:1px solid
var(--fc-border-color);display:flex;flex-wrap:wrap;overflow-x:hidden;overflow-y:auto}.fc .fc-multimonth-title{font-size:1.2em;font-weight:700;padding:1em
0;text-align:center}.fc .fc-multimonth-daygrid{background:var(--fc-page-bg-color)}.fc .fc-multimonth-daygrid-table,.fc
.fc-multimonth-header-table{table-layout:fixed;width:100%}.fc .fc-multimonth-daygrid-table{border-top-style:hidden!important}.fc .fc-multimonth-singlecol
.fc-multimonth{position:relative}.fc .fc-multimonth-singlecol .fc-multimonth-header{background:var(--fc-page-bg-color);position:relative;top:0;z-index:2}.fc
.fc-multimonth-singlecol .fc-multimonth-daygrid{position:relative;z-index:1}.fc .fc-multimonth-singlecol .fc-multimonth-daygrid-table,.fc .fc-multimonth-singlecol
.fc-multimonth-header-table{border-left-style:hidden;border-right-style:hidden}.fc .fc-multimonth-singlecol .fc-multimonth-month:last-child
.fc-multimonth-daygrid-table{border-bottom-style:hidden}.fc .fc-multimonth-multicol{line-height:1}.fc .fc-multimonth-multicol .fc-multimonth-month{padding:0 1.2em 1.2em}.fc
.fc-multimonth-multicol .fc-daygrid-more-link{border:1px solid var(--fc-event-border-color);display:block;float:none;padding:1px}.fc .fc-multimonth-compact{line-height:1}.fc
.fc-multimonth-compact .fc-multimonth-daygrid-table,.fc .fc-multimonth-compact .fc-multimonth-header-table{font-size:.9em}.fc-media-screen .fc-multimonth-singlecol
.fc-multimonth-header{position:sticky}.fc-media-print .fc-multimonth{overflow:visible}");var
_c=Po({name:"@fullcalendar/multimonth",initialView:"multiMonthYear",optionRefiners:xc,views:{multiMonth:{component:class extends ls{constructor(){super(...arguments),this.spl
itDateProfileByMonth=Gt(Dc),this.buildMonthFormat=Gt(Rc),this.scrollElRef={current:null},this.firstMonthElRef={current:null},this.needsScrollReset=!1,this.handleSizing=e=>{e&
&this.updateSize()}}render(){const{context:e,props:t,state:n}=this,{options:r}=e,{clientWidth:i,clientHeight:s}=n,o=n.monthHPadding||0,a=Math.min(null!=i?Math.floor(i/(r.mult
iMonthMinWidth+o)):1,r.multiMonthMaxColumns)||1,l=100/a+"%",c=null==i?null:i/a-o,d=null!=i&&1===a,u=this.splitDateProfileByMonth(e.dateProfileGenerator,t.dateProfile,e.dateEn
v,!d&&r.fixedWeekCount,r.showNonCurrentDates),h=this.buildMonthFormat(r.multiMonthTitleFormat,u),f=["fc-multimonth",d?"fc-multimonth-singlecol":"fc-multimonth-multicol",null!
=c&&c<400?"fc-multimonth-compact":""];return p(er,{elRef:this.scrollElRef,elClasses:f,viewSpec:e.viewSpec},u.map((e,n)=>{const r=Lt(e.currentRange.start);return
p(Sc,Object.assign({},t,{key:r,isoDateStr:r,elRef:0===n?this.firstMonthElRef:void 0,titleFormat:h,dateProfile:e,width:l,tableWidth:c,clientWidth:i,clientHeight:s}))}))}componen
tDidMount(){this.updateSize(),this.context.addResizeHandler(this.handleSizing),this.requestScrollReset()}componentDidUpdate(e){Cn(e,this.props)||this.handleSizing(!1),e.dat
eProfile!==this.props.dateProfile?this.requestScrollReset():this.flushScrollReset()}componentWillUnmount(){this.context.removeResizeHandler(this.handleSizing)}updateSize(){co
nst e=this.scrollElRef.current,t=this.firstMonthElRef.current;e&&this.setState({clientWidth:e.clientWidth,clientHeight:e.clientHeight}),t&&e&&null==this.state.monthHPadding&&
this.setState({monthHPadding:e.clientWidth-t.firstChild.offsetWidth})}requestScrollReset(){this.needsScrollReset=!0,this.flushScrollReset()}flushScrollReset(){if(this.needsSc
rollReset&&null!=this.state.monthHPadding){const{currentDate:e}=this.props.dateProfile,t=this.scrollElRef.current,n=t.querySelector(`[data-date="${Lt(e)}"]`);t.scrollTop=n.ge
tBoundingClientRect().top-this.firstMonthElRef.current.getBoundingClientRect().top,this.needsScrollReset=!1}}shouldComponentUpdate(){return!0}},dateProfileGeneratorClass:xl,m
ultiMonthMinWidth:350,multiMonthMaxColumns:3},multiMonthYear:{type:"multiMonth",duration:{years:1},fixedWeekCount:!0,showNonCurrentDates:!1}}});return
ca.push(el,Tl,uc,Ec,_c),e.Calendar=class extends Ni{constructor(e,t={}){super(),this.isRendering=!1,this.isRendered=!1,this.currentClassNames=[],this.customContentRenderId=0,
this.handleAction=e=>{switch(e.type){case"SET_EVENT_DRAG":case"SET_EVENT_RESIZE":this.renderRunner.tryDrain()}},this.handleData=e=>{this.currentData=e,this.renderRunner.reque
st(e.calendarOptions.rerenderDelay)},this.handleRenderRequest=()=>{if(this.isRendering){this.isRendered=!0;let{currentData:e}=this;jn(()=>{U(p(ki,{options:e.calendarOptions,t
heme:e.theme,emitter:e.emitter},(t,n,r,i)=>(this.setClassNames(t),this.setHeight(n),p($n.Provider,{value:this.customContentRenderId},p(_a,Object.assign({isHeightAuto:r,forPri
nt:i},e)))),this.el)})}else this.isRendered&&(this.isRendered=!1,U(null,this.el),this.setClassNames([]),this.setHeight(""))},function(e){e.isConnected&&e.getRootNode&&_e(e.g
etRootNode())}(e),this.el=e,this.renderRunner=new Me(this.handleRenderRequest),new
ha({optionOverrides:t,calendarApi:this,onAction:this.handleAction,onData:this.handleData})}render(){let e=this.isRendering;e?this.customContentRenderId+=1:this.isRendering=!0
,this.renderRunner.request(),e&&this.updateSize()}destroy(){this.isRendering&&(this.isRendering=!1,this.renderRunner.request())}updateSize(){jn(()=>{super.updateSize()})}batc
hRendering(e){this.renderRunner.pause("batchRendering"),e(),this.renderRunner.resume("batchRendering")}pauseRendering(){this.renderRunner.pause("pauseRendering")}resumeRender
ing(){this.renderRunner.resume("pauseRendering",!0)}resetOptions(e,t){this.currentDataManager.resetOptions(e,t)}setClassNames(e){if(!St(e,this.currentClassNames)){let{classLi
st:t}=this.el;for(let e of this.currentClassNames)t.remove(e);for(let n of e)t.add(n);this.currentClassNames=e}}setHeight(e){je(this.el,"height",e)}},e.Draggable=class{constr
uctor(e,t={}){this.handlePointerDown=e=>{let{dragging:t}=this,{minDistance:n,longPressDelay:r}=this.settings;t.minDistance=null!=n?n:e.isTouch?0:cn.eventDragMinDistance,t.del
ay=e.isTouch?null!=r?r:cn.longPressDelay:0},this.handleDragStart=e=>{e.isTouch&&this.dragging.delay&&e.subjectEl.classList.contains("fc-event")&&this.dragging.mirror.getMirro
rEl().classList.add("fc-event-selected")},this.settings=t;let n=this.dragging=new Fa(e);n.touchScrollAllowed=!1,null!=t.itemSelector&&(n.pointer.selector=t.itemSelector),null
!=t.appendTo&&(n.mirror.parentNode=t.appendTo),n.emitter.on("pointerdown",this.handlePointerDown),n.emitter.on("dragstart",this.handleDragStart),new
Ja(n,t.eventData)}destroy(){this.dragging.destroy()}},e.Internal=xo,e.JsonRequestError=Ri,e.Preact=_o,e.ThirdPartyDraggable=class{constructor(e,t){let
n=document;e===document||e instanceof Element?(n=e,t=t||{}):t=e||{};let r=this.dragging=new Ka(n);"string"==typeof
t.itemSelector?r.pointer.selector=t.itemSelector:n===document&&(r.pointer.selector="[data-event]"),"string"==typeof t.mirrorSelector&&(r.mirrorSelector=t.mirrorSelector),new
Ja(r,t.eventData)}destroy(){this.dragging.destroy()}},e.createPlugin=Po,e.formatDate=function(e,t={}){let n=Ma(t),r=an(t),i=n.createMarkerMeta(e);return
i?n.format(i.marker,r,{forcedTzo:i.forcedTzo}):""},e.formatRange=function(e,t,n){let r=Ma("object"==typeof
n&&n?n:{}),i=an(n),s=r.createMarkerMeta(e),o=r.createMarkerMeta(t);return s&&o?r.formatRange(s.marker,o.marker,i,{forcedStartTzo:s.forcedTzo,forcedEndTzo:o.forcedTzo,isEndExc
lusive:n.isEndExclusive,defaultSeparator:cn.defaultRangeSeparator}):""},e.globalLocales=To,e.globalPlugins=ca,e.sliceEvents=function(e,t){return
ii(e.eventStore,e.eventUiBases,e.dateProfile.activeRange,t?e.nextDayThreshold:null).fg},e.version="6.1.11",Object.defineProperty(e,"__esModule",{value:!0}),e}({});
```

public/fullcalendar/locales-all.global.min.js

```
/*!
FullCalendar Core v6.1.11
Docs & License: https://fullcalendar.io
(c) 2023 Adam Shaw
*/
!function(e){"use strict";function t(e){return"Tag"===e||"Monat"===e?"r":"Jahr"===e?"s":""}function n(e){return"Tag"===e||"Monat"===e?"r":"Jahr"===e?"s":""}var o=[{code:"af",
week:{dow:1,doy:4},buttonText:{prev:"Vorige",next:"Volgende",today:"Vandag",year:"Jaar",month:"Maand",week:"Week",day:"Dag",list:"Agenda"},allDayText:"Heeldag",moreLinkText:"
Addisionele",noEventsText:"Daar is geen gebeurtenisse nie"},{code:"ar-dz",week:{dow:0,doy:4},direction:"rtl",buttonText:{prev:"      ",next:"      ",today:"     ",year:"   ",
month:"   ",week:"     ",day:"   ",list:"     "},weekText:"     ",allDayText:"          "},{code:"ar-kw",week:{dow:0,doy:12},d
irection:"rtl",buttonText:{prev:"      ",next:"      ",today:"     ",year:"   ",month:"   ",week:"     ",day:"   ",list:"     "},weekText:"     ",allDayText:"
   ",moreLinkText:"    ",noEventsText:"            "},{code:"ar-ly",week:{dow:6,doy:12},direction:"rtl",buttonText:{prev:"      ",next:"      ",today:"     ",year:"   ",mont
h:"   ",week:"     ",day:"   ",list:"     "},weekText:"     ",allDayText:"          "},{code:"ar-ma",week:{dow:6,doy:12},direc
tion:"rtl",buttonText:{prev:"      ",next:"      ",today:"     ",year:"   ",month:"   ",week:"     ",day:"   ",list:"     "},weekText:"     ",allDayText:"
   ",moreLinkText:"    ",noEventsText:"            "},{code:"ar-sa",week:{dow:0,doy:6},direction:"rtl",buttonText:{prev:"      ",next:"      ",today:"     ",year:"   ",month
:"   ",week:"     ",day:"   ",list:"     "},weekText:"     ",allDayText:"          ",moreLinkText:"    ",noEventsText:"            "},{code:"ar-tn",week:{dow:1,doy:4},directi
on:"rtl",buttonText:{prev:"      ",next:"      ",today:"     ",year:"   ",month:"   ",week:"     ",day:"   ",list:"     "},weekText:"     ",allDayText:"
   ",moreLinkText:"    ",noEventsText:"            "},{code:"ar",week:{dow:6,doy:12},direction:"rtl",buttonText:{prev:"      ",next:"      ",today:"     ",year:"   ",month:"
   ",week:"     ",day:"   ",list:"     "},weekText:"     ",allDayText:"          ",moreLinkText:"    ",noEventsText:"
    "},{code:"az",week:{dow:1,doy:4},buttonText:{prev:"Əvvəl",next:"Sonra",today:"Bu
Gün",year:"Il",month:"Ay",week:"Həftə",day:"Gün",list:"Gündəm"},weekText:"Həftə",allDayText:"Bütün Gün",moreLinkText:e=>"+ daha çox "+e,noEventsText:"Göstərmək üçün hadisə
```

```
yoxdur"},{code:"bg",week:{dow:1,doy:4},buttonText:{prev:"назад",next:"напред",today:"днес",year:"година",month:"Месец",week:"Седмица",day:"Ден",list:"График"},allDayText:"Цял
 ден",moreLinkText:e=>"+още "+e,noEventsText:"Няма събития за показване"},{code:"bn",week:{dow:0,doy:6},buttonText:{prev:"     ",next:"     ",today:"  ",year:"   ",month:"
",week:"      ",day:"   ",list:"      "},weekText:"      ",allDayText:"       ",moreLinkText:e=>"+         "+e,noEventsText:"              "},{code:"bs",week:{dow:1,doy:7},b
uttonText:{prev:"Prošli",next:"Sljedeći",today:"Danas",year:"Godina",month:"Mjesec",week:"Sedmica",day:"Dan",list:"Raspored"},weekText:"Sed",allDayText:"Cijeli
dan",moreLinkText:e=>"+ još "+e,noEventsText:"Nema događaja za prikazivanje"},{code:"ca",week:{dow:1,doy:4},buttonText:{prev:"Anterior",next:"Següent",today:"Avui",year:"Any"
,month:"Mes",week:"Setmana",day:"Dia",list:"Agenda"},weekText:"Set",allDayText:"Tot el dia",moreLinkText:"més",noEventsText:"No hi ha esdeveniments per mostrar"},{code:"cs",w
eek:{dow:1,doy:4},buttonText:{prev:"Dříve",next:"Později",today:"Nyní",year:"Rok",month:"Měsíc",week:"Týden",day:"Den",list:"Agenda"},weekText:"Týd",allDayText:"Celý
den",moreLinkText:e=>"+další: "+e,noEventsText:"Žádné akce k zobrazení"},{code:"cy",week:{dow:1,doy:4},buttonText:{prev:"Blaenorol",next:"Nesaf",today:"Heddiw",year:"Blwyddyn
",month:"Mis",week:"Wythnos",day:"Dydd",list:"Rhestr"},weekText:"Wythnos",allDayText:"Trwy'r dydd",moreLinkText:"Mwy",noEventsText:"Dim
digwyddiadau"},{code:"da",week:{dow:1,doy:4},buttonText:{prev:"Forrige",next:"Næste",today:"I
dag",year:"År",month:"Måned",week:"Uge",day:"Dag",list:"Agenda"},weekText:"Uge",allDayText:"Hele dagen",moreLinkText:"flere",noEventsText:"Ingen arrangementer at vise"},{code
:"de-at",week:{dow:1,doy:4},buttonText:{prev:"Zurück",next:"Vor",today:"Heute",year:"Jahr",month:"Monat",week:"Woche",day:"Tag",list:"Terminübersicht"},weekText:"KW",weekText
Long:"Woche",allDayText:"Ganztägig",moreLinkText:e=>"+ weitere "+e,noEventsText:"Keine Ereignisse anzuzeigen",buttonHints:{prev:e=>`Vorherige${t(e)}
${e}`,next:e=>`Nächste${t(e)} ${e}`,today:e=>"Tag"===e?"Heute":`Diese${t(e)} ${e}`},viewHint:e=>e+("Woche"===e?"n":"Monat"===e?"s":"es")+"ansicht",navLinkHint:"Gehe zu
$0",moreLinkHint:e=>"Zeige "+(1===e?"ein weiteres Ereignis":e+" weitere Ereignisse"),closeHint:"Schließen",timeHint:"Uhrzeit",eventHint:"Ereignis"},{code:"de",week:{dow:1,doy
:4},buttonText:{prev:"Zurück",next:"Vor",today:"Heute",year:"Jahr",month:"Monat",week:"Woche",day:"Tag",list:"Terminübersicht"},weekText:"KW",weekTextLong:"Woche",allDayText:
"Ganztägig",moreLinkText:e=>"+ weitere "+e,noEventsText:"Keine Ereignisse anzuzeigen",buttonHints:{prev:e=>`Vorherige${n(e)} ${e}`,next:e=>`Nächste${n(e)}
${e}`,today:e=>"Tag"===e?"Heute":`Diese${n(e)} ${e}`},viewHint:e=>e+("Woche"===e?"n":"Monat"===e?"s":"es")+"ansicht",navLinkHint:"Gehe zu $0",moreLinkHint:e=>"Zeige
"+(1===e?"ein weiteres Ereignis":e+" weitere Ereignisse"),closeHint:"Schließen",timeHint:"Uhrzeit",eventHint:"Ereignis"},{code:"el",week:{dow:1,doy:4},buttonText:{prev:"Προη
ούμενος",next:"Επόμενος",today:"Σήμερα",year:"Ετος",month:"Μήνας",week:"Εβδομάδα",day:"Ημέρα",list:"Ατζέντα"},weekText:"Εβδ",allDayText:"Ολοήμερο",moreLinkText:"περισσότερα",
noEventsText:"Δεν υπάρχουν γεγονότα προς εμφάνιση"},{code:"en-au",week:{dow:1,doy:4},buttonHints:{prev:"Previous $0",next:"Next $0",today:"This $0"},viewHint:"$0
view",navLinkHint:"Go to $0",moreLinkHint:e=>`Show ${e} more event${1===e?"":"s"}`},{code:"en-gb",week:{dow:1,doy:4},buttonHints:{prev:"Previous $0",next:"Next
$0",today:"This $0"},viewHint:"$0 view",navLinkHint:"Go to $0",moreLinkHint:e=>`Show ${e} more
event${1===e?"":"s"}`},{code:"en-nz",week:{dow:1,doy:4},buttonHints:{prev:"Previous $0",next:"Next $0",today:"This $0"},viewHint:"$0 view",navLinkHint:"Go to
$0",moreLinkHint:e=>`Show ${e} more event${1===e?"":"s"}`},{code:"eo",week:{dow:1,doy:4},buttonText:{prev:"Antaŭa",next:"Sekva",today:"Hodiaŭ",year:"Jaro",month:"Monato",week
:"Semajno",day:"Tago",list:"Tagordo"},weekText:"Sm",allDayText:"Tuta tago",moreLinkText:"pli",noEventsText:"Neniuj eventoj por montri"},{code:"es",week:{dow:0,doy:6},buttonTe
xt:{prev:"Ant",next:"Sig",today:"Hoy",year:"Año",month:"Mes",week:"Semana",day:"Día",list:"Agenda"},weekText:"Sm",allDayText:"Todo el día",moreLinkText:"más",noEventsText:"No
 hay eventos para
mostrar"},{code:"es",week:{dow:1,doy:4},buttonText:{prev:"Ant",next:"Sig",today:"Hoy",year:"Año",month:"Mes",week:"Semana",day:"Día",list:"Agenda"},buttonHints:{prev:"$0
antes",next:"$0 siguiente",today:e=>"Día"===e?"Hoy":("Semana"===e?"Esta":"Este")+" "+e.toLocaleLowerCase()},viewHint:e=>"Vista "+("Semana"===e?"de la":"del")+"
"+e.toLocaleLowerCase(),weekText:"Sm",weekTextLong:"Semana",allDayText:"Todo el día",moreLinkText:"más",moreLinkHint:e=>`Mostrar ${e} eventos más`,noEventsText:"No hay
eventos para mostrar",navLinkHint:"Ir al $0",closeHint:"Cerrar",timeHint:"La hora",eventHint:"Evento"},{code:"et",week:{dow:1,doy:4},buttonText:{prev:"Eelnev",next:"Järgnev",
today:"Täna",year:"Aasta",month:"Kuu",week:"Nädal",day:"Päev",list:"Päevakord"},weekText:"näd",allDayText:"Kogu päev",moreLinkText:e=>"+ veel "+e,noEventsText:"Kuvamiseks
puuduvad sündmused"},{code:"eu",week:{dow:1,doy:7},buttonText:{prev:"Aur",next:"Hur",today:"Gaur",year:"Urtea",month:"Hilabetea",week:"Astea",day:"Eguna",list:"Agenda"},weekT
ext:"As",allDayText:"Egun osoa",moreLinkText:"gehiago",noEventsText:"Ez dago ekitaldirik erakusteko"},{code:"fa",week:{dow:6,doy:12},direction:"rtl",buttonText:{prev:"    ",n
ext:"    ",today:"     ",year:"   ",month:"   ",week:"    ",day:"   ",list:"      "},weekText:"  ",allDayText:"       ",moreLinkText:e=>"      "+e,noEventsText:"
        "},{code:"fi",week:{dow:1,doy:4},buttonText:{prev:"Edellinen",next:"Seuraava",today:"Tänään",year:"Vuosi",month:"Kuukausi",week:"Viikko",day:"Päivä",list:"Tapahtumat
"},weekText:"Vk",allDayText:"Koko päivä",moreLinkText:"lisää",noEventsText:"Ei näytettäviä
tapahtumia"},{code:"fr",buttonText:{prev:"Précédent",next:"Suivant",today:"Aujourd'hui",year:"Année",month:"Mois",week:"Semaine",day:"Jour",list:"Mon
planning"},weekText:"Sem.",allDayText:"Toute la journée",moreLinkText:"en plus",noEventsText:"Aucun évènement à
afficher"},{code:"fr-ch",week:{dow:1,doy:4},buttonText:{prev:"Précédent",next:"Suivant",today:"Courant",year:"Année",month:"Mois",week:"Semaine",day:"Jour",list:"Mon
planning"},weekText:"Sm",allDayText:"Toute la journée",moreLinkText:"en plus",noEventsText:"Aucun évènement à afficher"},{code:"fr",week:{dow:1,doy:4},buttonText:{prev:"Précé
dent",next:"Suivant",today:"Aujourd'hui",year:"Année",month:"Mois",week:"Semaine",day:"Jour",list:"Planning"},weekText:"Sem.",weekTextLong:"Semaine",allDayText:"Toute la
journée",moreLinkText:"en plus",noEventsText:"Aucun évènement à
afficher"},{code:"gl",week:{dow:1,doy:4},buttonText:{prev:"Ant",next:"Seg",today:"Hoxe",year:"Ano",month:"Mes",week:"Semana",day:"Día",list:"Axenda"},buttonHints:{prev:"$0
antes",next:"$0 seguinte",today:e=>"Día"===e?"Hoxe":("Semana"===e?"Esta":"Este")+" "+e.toLocaleLowerCase()},viewHint:e=>"Vista "+("Semana"===e?"da":"do")+"
"+e.toLocaleLowerCase(),weekText:"Sm",weekTextLong:"Semana",allDayText:"Todo o día",moreLinkText:"máis",moreLinkHint:e=>`Amosar ${e} eventos máis`,noEventsText:"Non hai
eventos para amosar",navLinkHint:"Ir ao $0",closeHint:"Pechar",timeHint:"A
hora",eventHint:"Evento"},{code:"he",direction:"rtl",buttonText:{prev:"     ",next:"   ",today:"    ",year:"   ",month:"    ",week:"    ",day:"   ",list:"
   "},allDayText:"       ",moreLinkText:"      ",noEventsText:"                ",weekText:"    "},{code:"hi",week:{dow:0,doy:6},buttonText:{prev:"     ",next:"    ",today:"
 ",year:"    ",month:"     ",week:"      ",day:"   ",list:"         "},weekText:"     ",allDayText:"       ",moreLinkText:e=>"+     "+e,noEventsText:"
          "},{code:"hr",week:{dow:1,doy:7},buttonText:{prev:"Prijašnji",next:"Sljedeći",today:"Danas",year:"Godina",month:"Mjesec",week:"Tjedan",day:"Dan",list:"Raspored"},w
eekText:"Tje",allDayText:"Cijeli dan",moreLinkText:e=>"+ još "+e,noEventsText:"Nema događaja za prikaz"},{code:"hu",week:{dow:1,doy:4},buttonText:{prev:"vissza",next:"előre",
today:"ma",year:"Év",month:"Hónap",week:"Hét",day:"Nap",list:"Lista"},weekText:"Hét",allDayText:"Egész nap",moreLinkText:"további",noEventsText:"Nincs megjeleníthető
esemény"},{code:"hy-am",week:{dow:1,doy:4},buttonText:{prev:"Նախորդ",next:"Հաջորդ",today:"Այսօր",year:"Տարի",month:"Ամիս",week:"Շաբաթ",day:"Օր",list:"Օրվա
gnւցաuկ"},weekText:"Շաբ",allDayText:"Ամբողջ օր",moreLinkText:e=>"+ ևս "+e,noEventsText:"Բացակայում է իրադարձություն
gnւցաղրեւnւ"},{code:"id",week:{dow:1,doy:7},buttonText:{prev:"mundur",next:"maju",today:"hari
ini",year:"Tahun",month:"Bulan",week:"Minggu",day:"Hari",list:"Agenda"},weekText:"Mg",allDayText:"Sehari penuh",moreLinkText:"lebih",noEventsText:"Tidak ada acara untuk
ditampilkan"},{code:"is",week:{dow:1,doy:4},buttonText:{prev:"Fyrri",next:"Næsti",today:"Í
dag",year:"Ár",month:"Mánuður",week:"Vika",day:"Dagur",list:"Dagskrá"},weekText:"Vika",allDayText:"Allan daginn",moreLinkText:"meira",noEventsText:"Engir viðburðir til að sýn
a"},{code:"it",week:{dow:1,doy:4},buttonText:{prev:"Prec",next:"Succ",today:"Oggi",year:"Anno",month:"Mese",week:"Settimana",day:"Giorno",list:"Agenda"},weekText:"Sm",allDayT
ext:"Tutto il giorno",moreLinkText:e=>"+altri "+e,noEventsText:"Non ci sono eventi da
visualizzare"},{code:"ja",buttonText:{prev:" ",next:" ",today:"  ",year:" ",month:" ",week:" ",day:" ",list:"     "},weekText:" ",allDayText:"  ",moreLinkText:e=>"  "+e+"  ",
noEventsText:"            "},{code:"ka",week:{dow:1,doy:7},buttonText:{prev:"    ",next:"       ",today:"    ",year:"        ",month:"   ",week:"     ",day:"   ",list:"
       "},weekText:"  ",allDayText:"         ",moreLinkText:e=>"+       "+e,noEventsText:"
    "},{code:"kk",week:{dow:1,doy:7},buttonText:{prev:"Алдыңғы",next:"Келесі",today:"Бүгін",year:"Жыл",month:"Ай",week:"Апта",day:"Күн",list:"Күн
тәртібі"},weekText:"Не",allDayText:"Күні бойы",moreLinkText:e=>"+ тағы "+e,noEventsText:"Көрсету үшін оқиғалар жоқ"},{code:"km",week:{dow:1,doy:4},buttonText:{prev:"   ",next
:"       ",today:"       ",year:"     ",month:"  ",week:"       ",day:"     ",list:"     "},weekText:"       ",allDayText:"          ",moreLinkText:"        ",noEventsText:"
          "},{code:"ko",buttonText:{prev:"   ",next:"   ",today:"  ",year:"  ",month:" ",week:" ",day:" ",list:"    "},weekText:" ",allDayText:"  ",moreLinkT
ext:" ",noEventsText:"        "},{code:"ku",week:{dow:6,doy:12},direction:"rtl",buttonText:{prev:"     ",next:"     ",today:"     ",year:"   ",month:"    ",week:"     ",day:"
   ",list:"       "},weekText:"     ",allDayText:"            ",moreLinkText:"     ",noEventsText:"             "},{code:"lb",week:{dow:1,doy:4},buttonText:{prev:"Zréck",n
ext:"Weider",today:"Haut",year:"Joer",month:"Mount",week:"Woch",day:"Dag",list:"Terminiwwersiicht"},weekText:"W",allDayText:"Ganzen Dag",moreLinkText:"méi",noEventsText:"Nee
Evenementer ze affichéieren"},{code:"lt",week:{dow:1,doy:4},buttonText:{prev:"Atgal",next:"Pirmyn",today:"Šiandien",year:"Metai",month:"Mėnuo",week:"Savaitė",day:"Diena",list
:"Darbotvarkė"},weekText:"SAV",allDayText:"Visą dieną",moreLinkText:"daugiau",noEventsText:"Nėra įvykių
```

```
rodyti"},{code:"lv",week:{dow:1,doy:4},buttonText:{prev:"Iepr.",next:"Nāk.",today:"Šodien",year:"Gads",month:"Mēnesis",week:"Nedēļa",day:"Diena",list:"Dienas kārtība"},weekText:"Ned.",allDayText:"Visu dienu",moreLinkText:e=>"+vēl "+e,noEventsText:"Nav notikumu"},{code:"mk",buttonText:{prev:"претходно",next:"следно",today:"Денес",year:"година",month:"Месец",week:"Недела",day:"Ден",list:"График"},weekText:"Сед",allDayText:"Цел ден",moreLinkText:e=>"+повеќе "+e,noEventsText:"Нема настани за прикажување"},{code:"ms",week:{dow:1,doy:7},buttonText:{prev:"Sebelum",next:"Selepas",today:"hari ini",year:"Tahun",month:"Bulan",week:"Minggu",day:"Hari",list:"Agenda"},weekText:"Mg",allDayText:"Sepanjang hari",moreLinkText:e=>"masih ada "+e+" acara",noEventsText:"Tiada peristiwa untuk dipaparkan"},{code:"nb",week:{dow:1,doy:4},buttonText:{prev:"Forrige",next:"Neste",today:"I dag",year:"År",month:"Måned",week:"Uke",day:"Dag",list:"Agenda"},weekText:"Uke",weekTextLong:"Uke",allDayText:"Hele dagen",moreLinkText:"til",noEventsText:"Ingen hendelser å vise",buttonHints:{prev:"Forrige $0",next:"Neste $0",today:"Nåværende $0"},viewHint:"$0 visning",navLinkHint:"Gå til $0",moreLinkHint:e=>`Vis ${e} flere hendelse${1===e?"":"r"}`},{code:"ne",week:{dow:7,doy:1},buttonText:{prev:"       ",next:"     ",today:"  ",year:"    ",month:"     ",week:"     ",day:"   ",list:"    "},weekText:"     ",allDayText:"      ",moreLinkText:"       ",noEventsText:"
     "},{code:"nl",week:{dow:1,doy:4},buttonText:{prev:"Vorige",next:"Volgende",today:"Vandaag",year:"Jaar",month:"Maand",week:"Week",day:"Dag",list:"Lijst"},allDayText:"Hele dag",moreLinkText:"extra",noEventsText:"Geen evenementen om te laten zien"},{code:"nn",week:{dow:1,doy:4},buttonText:{prev:"Førre",next:"Neste",today:"I dag",year:"År",month:"Månad",week:"Veke",day:"Dag",list:"Agenda"},weekText:"Veke",allDayText:"Heile dagen",moreLinkText:"til",noEventsText:"Ingen hendelser å vise"},{code:"pl",week:{dow:1,doy:4},buttonText:{prev:"Poprzedni",next:"Następny",today:"Dziś",year:"Rok",month:"Miesiąc",week:"Tydzień",day:"Dzień",list:"Plan dnia"},weekText:"Tydz",allDayText:"Cały dzień",moreLinkText:"więcej",noEventsText:"Brak wydarzeń do wyświetlenia"},{code:"pt-br",buttonText:{prev:"Anterior",next:"Próximo",prevYear:"Ano anterior",nextYear:"Próximo ano",year:"Ano",today:"Hoje",month:"Mês",week:"Semana",day:"Dia",list:"Lista"},buttonHints:{prev:"$0 Anterior",next:"Próximo $0",today:e=>"Dia"===e?"Hoje":("Semana"===e?"Esta":"Este")+" "+e.toLocaleLowerCase()},viewHint:e=>"Visualizar "+("Semana"===e?"a":"o")+" "+e.toLocaleLowerCase(),weekText:"Sm",weekTextLong:"Semana",allDayText:"dia inteiro",moreLinkText:e=>"mais +"+e,moreLinkHint:e=>`Mostrar mais ${e} eventos`,noEventsText:"Não há eventos para mostrar",navLinkHint:"Ir para $0",closeHint:"Fechar",timeHint:"A hora",eventHint:"Evento"},{code:"pt",week:{dow:1,doy:4},buttonText:{prev:"Anterior",next:"Seguinte",today:"Hoje",year:"Ano",month:"Mês",week:"Semana",day:"Dia",list:"Agenda"},weekText:"Sem",allDayText:"Todo o dia",moreLinkText:"mais",noEventsText:"Não há eventos para mostrar"},{code:"ro",week:{dow:1,doy:7},buttonText:{prev:"precedentă",next:"următoare",today:"Azi",year:"An",month:"Lună",week:"Săptămână",day:"Zi",list:"Agendă"},weekText:"Săpt",allDayText:"Toată ziua",moreLinkText:e=>"+alte "+e,noEventsText:"Nu există evenimente de afișat"},{code:"ru",week:{dow:1,doy:4},buttonText:{prev:"Пред",next:"След",today:"Сегодня",year:"Год",month:"Месяц",week:"Неделя",day:"День",list:"Повестка дня"},weekText:"Нед",allDayText:"Весь день",moreLinkText:e=>"+ ещё "+e,noEventsText:"Нет событий для отображения"},{code:"si-lk",week:{dow:1,doy:4},buttonText:{prev:"   ",next:"   ",today:"  ",year:"       ",month:"    ",week:"    ",day:"   ",list:"         "},weekText:"   ",allDayText:"     ",moreLinkText:"    ",noEventsText:"          "},{code:"sk",week:{dow:1,doy:4},buttonText:{prev:"Predchádzajúci",next:"Nasledujúci",today:"Dnes",year:"Rok",month:"Mesiac",week:"Týždeň",day:"Deň",list:"Rozvrh"},weekText:"Ty",allDayText:"Celý deň",moreLinkText:e=>"+ďalšie: "+e,noEventsText:"Žiadne akcie na zobrazenie"},{code:"sl",week:{dow:1,doy:7},buttonText:{prev:"Prejšnji",next:"Naslednji",today:"Trenutni",year:"Leto",month:"Mesec",week:"Teden",day:"Dan",list:"Dnevni red"},weekText:"Teden",allDayText:"Ves dan",moreLinkText:"več",noEventsText:"Ni dogodkov za prikaz"},{code:"sm",buttonText:{prev:"Talu ai",next:"Mulimuli atu",today:"Aso nei",year:"Tausaga",month:"Masina",week:"Vaiaso",day:"Aso",list:"Faasologa"},weekText:"Vaiaso",allDayText:"Aso atoa",moreLinkText:"sili atu",noEventsText:"Leai ni mea na tutupu"},{code:"sq",week:{dow:1,doy:4},buttonText:{prev:"mbrapa",next:"Përpara",today:"Sot",year:"Viti",month:"Muaj",week:"Javë",day:"Ditë",list:"Listë"},weekText:"Ja",allDayText:"Gjithë ditën",moreLinkText:e=>"+më tepër "+e,noEventsText:"Nuk ka evente për të shfaqur"},{code:"sr-cyrl",week:{dow:1,doy:7},buttonText:{prev:"Претходна",next:"следећи",today:"Данас",year:"Година",month:"Месец",week:"Недеља",day:"Дан",list:"Планер"},weekText:"Сед",allDayText:"Цео дан",moreLinkText:e=>"+ још "+e,noEventsText:"Нема догађаја за приказ"},{code:"sr",week:{dow:1,doy:7},buttonText:{prev:"Prethodna",next:"Sledec´i",today:"Danas",year:"Godina",month:"Mesec",week:"Nedelja",day:"Dan",list:"Planer"},weekText:"Sed",allDayText:"Ceo dan",moreLinkText:e=>"+ još "+e,noEventsText:"Nema događaja za prikaz"},{code:"sv",week:{dow:1,doy:4},buttonText:{prev:"Förra",next:"Nästa",today:"Idag",year:"År",month:"Månad",week:"Vecka",day:"Dag",list:"Program"},buttonHints:{prev:e=>"Föregående "+e.toLocaleLowerCase(),next:e=>"Nästa "+e.toLocaleLowerCase(),today:e=>("Program"===e?"Detta":"Denna")+" "+e.toLocaleLowerCase()},viewHint:"$0 vy",navLinkHint:"Gå till $0",moreLinkHint:e=>`Visa ytterligare ${e} händelse${1===e?"":"r"}`,weekText:"v.",weekTextLong:"Vecka",allDayText:"Heldag",moreLinkText:"till",noEventsText:"Inga händelser att visa",closeHint:"Stäng",timeHint:"Klockan",eventHint:"Händelse"},{code:"ta-in",week:{dow:1,doy:4},buttonText:{prev:"       ",next:"        ",today:"     ",year:"     ",month:"     ",week:"     ",day:"    ",list:"          "},weekText:"     ",allDayText:"             ",moreLinkText:e=>"+        "+e,noEventsText:"                        "},{code:"th",week:{dow:1,doy:4},buttonText:{prev:"        ",next:"     ",prevYear:"         ",nextYear:"       ",year:"  ",today:"      ",month:"     ",week:"       ",day:"   ",list:"        "},weekText:"       ",allDayText:"       ",moreLinkText:"         ",noEventsText:"                    "},{code:"tr",week:{dow:1,doy:7},buttonText:{prev:"geri",next:"ileri",today:"bugün",year:"Yıl",month:"Ay",week:"Hafta",day:"Gün",list:"Ajanda"},weekText:"Hf",allDayText:"Tüm gün",moreLinkText:"daha fazla",noEventsText:"Gösterilecek etkinlik yok"},{code:"ug",buttonText:{prev:"        ",next:"       ",today:"     ",year:"   ",month:"   ",week:"     ",day:"   ",list:"         "},allDayText:"
   "},{code:"uk",week:{dow:1,doy:7},buttonText:{prev:"Попередній",next:"далі",today:"Сьогодні",year:"рік",month:"Місяць",week:"Тиждень",day:"День",list:"Порядок денний"},weekText:"Тиж",allDayText:"Увесь день",moreLinkText:e=>"+ще "+e+"...",noEventsText:"Немає подій для відображення"},{code:"uz-cy",week:{dow:1,doy:4},buttonText:{prev:"Олин",next:"Кейин",today:"Бугун",month:"Ой",week:"Ҳафта",day:"Кун",list:"Кун тартиби"},weekText:"Ҳафта",allDayText:"Кун бўйича",moreLinkText:e=>"+ яна "+e,noEventsText:"Кўрсатиш учун воқеалар йўқ"},{code:"uz",week:{dow:1,doy:4},buttonText:{prev:"Oldingi",next:"Keyingi",today:"Bugun",year:"Yil",month:"Oy",week:"Xafta",day:"Kun",list:"Kun tartibi"},allDayText:"Kun bo'yi",moreLinkText:e=>"+ yana "+e,noEventsText:"Ko'rsatish uchun voqealar yo'q"},{code:"vi",week:{dow:1,doy:4},buttonText:{prev:"Trước",next:"Tiếp",today:"Hôm nay",year:"Năm",month:"Tháng",week:"Tuần",day:"Ngày",list:"Lịch biểu"},weekText:"Tu",allDayText:"Cả ngày",moreLinkText:e=>"+ thêm "+e,noEventsText:"Không có sự kiện để hiển thị"},{code:"zh-cn",week:{dow:1,doy:4},buttonText:{prev:"  ",next:"  ",today:"  ",year:" ",month:" ",week:" ",day:" ",list:"  "},weekText:" ",allDayText:"  ",moreLinkText:e=>"    "+e+"  ",noEventsText:"      "},{code:"zh-tw",buttonText:{prev:"  ",next:"  ",today:"  ",year:" ",month:" ",week:" ",day:" ",list:"    "},weekText:" ",allDayText:"  ",moreLinkText:"    ",noEventsText:"      "}];FullCalendar.globalLocales.push(...o)}();
```

public/index.php

```php
<?php

use Illuminate\Foundation\Application;
use Illuminate\Http\Request;

define('LARAVEL_START', microtime(true));

// Determine if the application is in maintenance mode...
if (file_exists($maintenance = __DIR__.'/../storage/framework/maintenance.php')) {
    require $maintenance;
}

// Register the Composer autoloader...
require __DIR__.'/../vendor/autoload.php';

// Bootstrap Laravel and handle the request...
```

```php
/** @var Application $app */
$app = require_once __DIR__.'/../bootstrap/app.php';

$app->handleRequest(Request::capture());
```

public/js/app.js

```javascript
if (typeof window !== 'undefined' && window.axios) {
    window.axios.defaults = window.axios.defaults || {};
    window.axios.defaults.headers = window.axios.defaults.headers || {};
    window.axios.defaults.headers.common = window.axios.defaults.headers.common || {};
    window.axios.defaults.headers.common['X-Requested-With'] = 'XMLHttpRequest';
}
```

resources

resources/css/app.css

```css
/*
    Primary Tailwind CSS entrypoint for the Laravel frontend.
    This file is compiled by Vite into the application stylesheet.
*/
@import 'tailwindcss';

@source '../../vendor/laravel/framework/src/Illuminate/Pagination/resources/views/*.blade.php';
@source '../../storage/framework/views/*.php';
@source '../**/*.blade.php';
@source '../**/*.js';

@theme {
    --font-sans: 'Instrument Sans', ui-sans-serif, system-ui, sans-serif, 'Apple Color Emoji', 'Segoe UI Emoji',
        'Segoe UI Symbol', 'Noto Color Emoji';
}

:root {
    --color-black: #14532d;
    --color-brand-green-50: #f0fdf4;
    --color-brand-green-100: #dcfce7;
    --color-brand-green-200: #bbf7d0;
    --color-brand-green-300: #86efac;
    --color-brand-green-400: #4ade80;
    --color-brand-green-500: #22c55e;
    --color-brand-green-600: #16a34a;
    --color-brand-green-700: #15803d;
    --color-brand-green-800: #166534;
    --color-brand-green-900: #14532d;
    --color-brand-green-950: #052e16;
    --color-brand-gold-50: #fefce8;
    --color-brand-gold-100: #fef9c3;
    --color-brand-gold-200: #fef08a;
    --color-brand-gold-300: #fde047;
    --color-brand-gold-400: #facc15;
    --color-brand-gold-500: #eab308;
    --color-brand-gold-600: #ca8a04;
    --color-brand-gold-700: #a16207;
    --color-brand-gold-800: #854d0e;
    --color-brand-gold-900: #713f12;
    --color-brand-gold-950: #422006;

    --color-indigo-50: var(--color-brand-green-50);
    --color-indigo-100: var(--color-brand-green-100);
    --color-indigo-200: var(--color-brand-green-200);
    --color-indigo-300: var(--color-brand-green-300);
    --color-indigo-400: var(--color-brand-green-400);
    --color-indigo-500: var(--color-brand-green-500);
    --color-indigo-600: var(--color-brand-green-600);
    --color-indigo-700: var(--color-brand-green-700);
    --color-indigo-800: var(--color-brand-green-800);
    --color-indigo-900: var(--color-brand-green-900);
    --color-indigo-950: var(--color-brand-green-950);
    --color-violet-50: var(--color-brand-green-50);
```

```
    --color-violet-100: var(--color-brand-green-100);
    --color-violet-200: var(--color-brand-green-200);
    --color-violet-300: var(--color-brand-green-300);
    --color-violet-400: var(--color-brand-green-400);
    --color-violet-500: var(--color-brand-green-500);
    --color-violet-600: var(--color-brand-green-600);
    --color-violet-700: var(--color-brand-green-700);
    --color-violet-800: var(--color-brand-green-800);
    --color-violet-900: var(--color-brand-green-900);
    --color-violet-950: var(--color-brand-green-950);
    --color-purple-50: var(--color-brand-green-50);
    --color-purple-100: var(--color-brand-green-100);
    --color-purple-200: var(--color-brand-green-200);
    --color-purple-300: var(--color-brand-green-300);
    --color-purple-400: var(--color-brand-green-400);
    --color-purple-500: var(--color-brand-green-500);
    --color-purple-600: var(--color-brand-green-600);
    --color-purple-700: var(--color-brand-green-700);
    --color-purple-800: var(--color-brand-green-800);
    --color-purple-900: var(--color-brand-green-900);
    --color-purple-950: var(--color-brand-green-950);
    --color-blue-50: var(--color-brand-green-50);
    --color-blue-100: var(--color-brand-green-100);
    --color-blue-200: var(--color-brand-green-200);
    --color-blue-300: var(--color-brand-green-300);
    --color-blue-400: var(--color-brand-green-400);
    --color-blue-500: var(--color-brand-green-500);
    --color-blue-600: var(--color-brand-green-600);
    --color-blue-700: var(--color-brand-green-700);
    --color-blue-800: var(--color-brand-green-800);
    --color-blue-900: var(--color-brand-green-900);
    --color-blue-950: var(--color-brand-green-950);
    --color-teal-50: var(--color-brand-green-50);
    --color-teal-100: var(--color-brand-green-100);
    --color-teal-200: var(--color-brand-green-200);
    --color-teal-300: var(--color-brand-green-300);
    --color-teal-400: var(--color-brand-green-400);
    --color-teal-500: var(--color-brand-green-500);
    --color-teal-600: var(--color-brand-green-600);
    --color-teal-700: var(--color-brand-green-700);
    --color-teal-800: var(--color-brand-green-800);
    --color-teal-900: var(--color-brand-green-900);
    --color-teal-950: var(--color-brand-green-950);
    --color-cyan-50: var(--color-brand-gold-50);
    --color-cyan-100: var(--color-brand-gold-100);
    --color-cyan-200: var(--color-brand-gold-200);
    --color-cyan-300: var(--color-brand-gold-300);
    --color-cyan-400: var(--color-brand-gold-400);
    --color-cyan-500: var(--color-brand-gold-500);
    --color-cyan-600: var(--color-brand-gold-600);
    --color-cyan-700: var(--color-brand-gold-700);
    --color-cyan-800: var(--color-brand-gold-800);
    --color-cyan-900: var(--color-brand-gold-900);
    --color-cyan-950: var(--color-brand-gold-950);
    --color-pink-50: var(--color-brand-gold-50);
    --color-pink-100: var(--color-brand-gold-100);
    --color-pink-200: var(--color-brand-gold-200);
    --color-pink-300: var(--color-brand-gold-300);
    --color-pink-400: var(--color-brand-gold-400);
    --color-pink-500: var(--color-brand-gold-500);
    --color-pink-600: var(--color-brand-gold-600);
    --color-pink-700: var(--color-brand-gold-700);
    --color-pink-800: var(--color-brand-gold-800);
    --color-pink-900: var(--color-brand-gold-900);
    --color-pink-950: var(--color-brand-gold-950);
    --color-sky-50: var(--color-brand-gold-50);
    --color-sky-100: var(--color-brand-gold-100);
    --color-sky-200: var(--color-brand-gold-200);
    --color-sky-300: var(--color-brand-gold-300);
    --color-sky-400: var(--color-brand-gold-400);
    --color-sky-500: var(--color-brand-gold-500);
    --color-sky-600: var(--color-brand-gold-600);
    --color-sky-700: var(--color-brand-gold-700);
    --color-sky-800: var(--color-brand-gold-800);
```

```
    --color-sky-900: var(--color-brand-gold-900);
    --color-sky-950: var(--color-brand-gold-950);
}
```

resources/js/app.js

```
// Main frontend entrypoint for the Laravel Vite build.
// This file loads shared bootstrap behavior and can be extended with component scripts.
import './bootstrap';

const eyeIcon = `
    <svg data-password-eye-open class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 12a3 3 0 11-6 0 3 3 0 016 0z"></path>
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M2.458 12C3.732 7.943 7.523 5 12 5c4.478 0 8.268 2.943 9.542 7-1.274 4.057-5.064 7-9.542 7-4.477 0-8.268-2.943-9.542-7z"></path>
    </svg>
    <svg data-password-eye-closed class="hidden h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M13.875 18.825A10.05 10.05 0 0112 19c-4.478 0-8.268-2.943-9.543-7a9.97 9.97 0 011.563-3.029m5.858.908a3 3 0 114.243 4.243M9.878 9.878l4.242 4.242M9.88 9.88l-3.29-3.29m7.532 7.532l3.29 3.29M3 3l3.59 3.59m0 0A9.953 9.953 0 0112 5c4.478 0 8.268 2.943 9.543 7a10.025 10.025 0 01-4.132 5.411m0 0L21 21"></path>
    </svg>`;

function initializePasswordVisibilityToggles() {
    document.querySelectorAll('input[type="password"]').forEach((input) => {
        if (!(input instanceof HTMLInputElement) || input.dataset.passwordToggleInitialized) return;

        let wrapper = input.parentElement;
        if (!wrapper?.classList.contains('relative')) {
            wrapper = document.createElement('div');
            wrapper.className = 'relative';
            input.before(wrapper);
            wrapper.append(input);
        }

        input.classList.add('pr-12');

        const label = input.labels?.[0];
        const labelText = label?.querySelector('span')?.textContent?.trim()
            || label?.textContent?.trim()
            || 'password';
        const button = document.createElement('button');
        button.type = 'button';
        button.className = 'absolute right-3 top-1/2 -translate-y-1/2 rounded p-1 text-gray-500 transition hover:text-gray-800 focus:outline-none focus:ring-2 focus:ring-indigo-500';
        button.setAttribute('aria-label', `Show ${labelText.toLowerCase()}`);
        button.setAttribute('aria-pressed', 'false');
        button.title = `Show ${labelText.toLowerCase()}`;
        button.innerHTML = eyeIcon;
        button.addEventListener('mousedown', (event) => event.preventDefault());
        button.addEventListener('click', () => {
            input.type = input.type === 'password' ? 'text' : 'password';
            const isVisible = input.type === 'text';
            const action = isVisible ? 'Hide' : 'Show';
            button.setAttribute('aria-label', `${action} ${labelText.toLowerCase()}`);
            button.setAttribute('aria-pressed', String(isVisible));
            button.title = `${action} ${labelText.toLowerCase()}`;
            button.querySelector('[data-password-eye-open]')?.classList.toggle('hidden', isVisible);
            button.querySelector('[data-password-eye-closed]')?.classList.toggle('hidden', !isVisible);
            input.focus();
        });
        wrapper.append(button);
        input.dataset.passwordToggleInitialized = 'true';
    });
}

if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', initializePasswordVisibilityToggles);
} else {
    initializePasswordVisibilityToggles();
}
```

resources/js/bootstrap.js

```
// Standard HTTP client used by the frontend for AJAX requests.
import axios from 'axios';
window.axios = axios;

// Ensure Laravel recognizes the requests as AJAX and can handle CSRF/headers correctly.
window.axios.defaults.headers.common['X-Requested-With'] = 'XMLHttpRequest';
```

resources/views/about.blade.php

```
@extends('layouts.app')

@section('title', 'About Us')

@section('content')
<section class="relative overflow-hidden bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50 py-16 sm:py-20">
    <div aria-hidden="true" class="pointer-events-none absolute -left-24 top-10 h-80 w-80 rounded-full bg-indigo-200/60 blur-3xl"></div>
    <div aria-hidden="true" class="pointer-events-none absolute -right-20 top-48 h-96 w-96 rounded-full bg-pink-200/50 blur-3xl"></div>
    <div class="relative mx-auto max-w-6xl px-4 sm:px-6">
        <div class="grid gap-10 lg:grid-cols-[1.2fr_0.8fr] items-center">
            <div>
                <p class="mb-4 text-sm font-semibold uppercase tracking-[0.2em] text-indigo-700">About Campus Reserve</p>
                <h1 class="mb-6 text-4xl font-semibold leading-tight tracking-tight text-slate-950 sm:text-5xl">Smart campus booking made simple for students, staff, and administrators.</h1>
                <p class="text-lg leading-8 text-slate-600 mb-6">Campus Reserve is a secure reservation platform designed to bring campus spaces, classrooms, labs, and event venues together in one intuitive calendar. We help schools manage bookings efficiently, prevent conflicts, and keep everyone aligned with real-time availability.</p>
                <div class="grid gap-5 sm:grid-cols-2">
                    <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-6 shadow-sm">
                        <p class="mb-3 text-sm font-semibold uppercase tracking-[0.2em] text-indigo-700">Our mission</p>
                        <p class="text-slate-700 leading-7">Create a dependable campus scheduling experience that reduces manual work and keeps facilities running smoothly.</p>
                    </div>
                    <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-6 shadow-sm">
                        <p class="mb-3 text-sm font-semibold uppercase tracking-[0.2em] text-pink-700">Our promise</p>
                        <p class="text-slate-700 leading-7">Deliver fast, secure access to room reservations while protecting student and staff data.</p>
                    </div>
                </div>
            </div>

            <div class="rounded-4xl border border-white/70 bg-gradient-to-br from-indigo-700 via-violet-700 to-purple-700 p-8 text-white shadow-2xl shadow-indigo-950/15">
                <div class="mb-8">
                    <p class="text-sm font-semibold uppercase tracking-[0.2em] text-indigo-200">Why customers love it</p>
                    <h2 class="text-3xl font-semibold mt-4 text-white">Built for organized campuses.</h2>
                </div>
                <div class="space-y-5">
                    <div class="rounded-3xl border border-white/10 bg-white/10 p-5 shadow-sm backdrop-blur">
                        <p class="text-2xl font-semibold text-white">24/7 availability</p>
                        <p class="text-slate-300 mt-2">Book rooms at any hour with instant calendar visibility and no waiting.</p>
                    </div>
                    <div class="rounded-3xl border border-white/10 bg-white/10 p-5 shadow-sm backdrop-blur">
                        <p class="text-2xl font-semibold text-white">Conflict-free scheduling</p>
                        <p class="text-slate-300 mt-2">Automatically prevent double bookings by seeing real-time availability across campus spaces.</p>
                    </div>
                    <div class="rounded-3xl border border-white/10 bg-white/10 p-5 shadow-sm backdrop-blur">
                        <p class="text-2xl font-semibold text-white">Secure sign-in</p>
                        <p class="text-slate-300 mt-2">Protect access with robust authentication and two-factor verification when it matters most.</p>
                    </div>
                </div>
            </div>
        </div>

        <div class="mt-16 grid gap-8 lg:grid-cols-3">
            <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-8 shadow-sm transition hover:-translate-y-1 hover:border-indigo-200 hover:shadow-lg">
                <p class="mb-3 text-sm font-semibold uppercase tracking-[0.2em] text-indigo-700">Fast adoption</p>
                <p class="text-3xl font-semibold text-slate-900">Easy setup</p>
                <p class="mt-4 text-slate-600 leading-7">Add facilities, manage users, and start booking in minutes with a clean interface that feels natural for every campus role.</p>
            </div>
            <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-8 shadow-sm transition hover:-translate-y-1 hover:border-pink-200 hover:shadow-lg">
                <p class="mb-3 text-sm font-semibold uppercase tracking-[0.2em] text-pink-700">Reliable operations</p>
                <p class="text-3xl font-semibold text-slate-900">Fewer conflicts</p>
                <p class="mt-4 text-slate-600 leading-7">Keep schedules aligned and reduce manual coordination with automated reservation tracking and approval workflows.</p>
            </div>
```

```
            <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-8 shadow-sm transition hover:-translate-y-1 hover:border-cyan-200 hover:shadow-lg">
                <p class="mb-3 text-sm font-semibold uppercase tracking-[0.2em] text-cyan-700">User-first design</p>
                <p class="text-3xl font-semibold text-slate-900">Clear visibility</p>
                <p class="mt-4 text-slate-600 leading-7">See every booking at a glance using a modern calendar, helpful reservations panel, and easy search tools.</p>
            </div>
        </div>
    </div>
</section>
@endsection
```

resources/views/admin/dashboard.blade.php

```
@extends('layouts.app')

@section('title', 'Staff Dashboard - Campus Reserve')

@section('content')
<!-- Admin overview groups existing reservation, member, and audit tools beside contextual navigation. -->
<div class="min-h-screen bg-[#f5f3f7]">
    <div class="flex min-h-screen flex-col md:flex-row">
        @include('admin.partials.sidebar')

        <main class="min-w-0 flex-1 px-3 py-5 sm:px-6 sm:py-8 xl:px-10">
            <div class="mx-auto max-w-[1600px]">
                <section id="overview" data-admin-section="overview">
                <div class="mb-7 flex flex-col gap-4 sm:flex-row sm:items-end sm:justify-between">
                    <div>
                        <p class="text-sm font-semibold text-green-800">Campus Reserve / Workspace</p>
                        <h1 class="mt-1 text-3xl font-semibold tracking-tight text-slate-950">Dashboard overview</h1>
                        <p class="mt-2 text-sm text-slate-500 sm:text-base">Reservations, members, and administration activity in one place.</p>
                    </div>
                    <a href="/reserve" class="inline-flex w-fit items-center gap-2 rounded-xl border border-slate-200 bg-white px-4 py-2.5 text-sm font-semibold
text-slate-700 shadow-sm transition hover:border-green-300 hover:text-green-900 md:hidden">
                        Open user portal
                    </a>
                </div>

        @if(session('status'))
            <div class="mb-6 bg-green-100 border border-green-300 text-green-800 px-4 py-3 rounded">{{ session('status') }}</div>
        @endif
        @if(session('error'))
            <div class="mb-6 bg-red-100 border border-red-300 text-red-800 px-4 py-3 rounded">{{ session('error') }}</div>
        @endif
        @if($errors->has('current_password'))
            <div class="mb-6 rounded-xl border border-red-300 bg-red-50 px-4 py-3 text-red-800" role="alert">{{ $errors->first('current_password') }}</div>
        @elseif($errors->any())
            <div class="mb-6 rounded-xl border border-red-300 bg-red-50 px-4 py-3 text-red-800" role="alert">
                {{ $errors->first() }}
            </div>
        @endif

        <div class="mb-6 grid grid-cols-2 gap-3 sm:mb-8 sm:gap-5 xl:grid-cols-4">
            <div class="rounded-2xl border border-orange-100 bg-white p-4 shadow-sm sm:p-5">
                <p class="text-sm font-medium text-gray-500">Pending Reservations</p>
                <p id="pending-reservations-total" class="mt-2 text-2xl font-bold text-gray-900 sm:mt-4 sm:text-3xl">{{ $pendingReservations->total() }}</p>
            </div>
            <div class="rounded-2xl border border-green-100 bg-white p-4 shadow-sm sm:p-5">
                <p class="text-sm font-medium text-gray-500">Recent Decisions</p>
                <p id="recent-decisions-total" class="mt-2 text-2xl font-bold text-gray-900 sm:mt-4 sm:text-3xl">{{ $recentReservations->count() }}</p>
            </div>
            <div class="rounded-2xl border border-amber-100 bg-white p-4 shadow-sm sm:p-5">
                <p class="text-sm font-medium text-gray-500">Audit Log Entries</p>
                <p id="audit-log-total" class="mt-2 text-2xl font-bold text-gray-900 sm:mt-4 sm:text-3xl">{{ $auditLogs->count() }}</p>
            </div>
            <div class="rounded-2xl border border-slate-200 bg-white p-4 shadow-sm sm:p-5">
                <p class="text-sm font-medium text-gray-500">Members</p>
                <p class="mt-2 text-2xl font-bold text-gray-900 sm:mt-4 sm:text-3xl">{{ $users->total() ?? 0 }}</p>
            </div>
        </div>

                <div class="grid gap-3 sm:grid-cols-3">
                    <a href="#reservations" data-admin-section-link="reservations" class="rounded-2xl border border-slate-200 bg-white p-4 text-sm font-semibold
text-slate-700 shadow-sm transition hover:border-green-300 hover:text-green-900">Review reservations <span class="ml-1 text-slate-400">→</span></a>
                    <a href="#members" data-admin-section-link="members" class="rounded-2xl border border-slate-200 bg-white p-4 text-sm font-semibold text-slate-700
```

```
    shadow-sm transition hover:border-green-300 hover:text-green-900">Manage members <span class="ml-1 text-slate-400">→</span></a>
61                  <a href="#audit" data-admin-section-link="audit" class="rounded-2xl border border-slate-200 bg-white p-4 text-sm font-semibold text-slate-700 shadow-sm
    transition hover:border-green-300 hover:text-green-900">View audit activity <span class="ml-1 text-slate-400">→</span></a>
62              </div>
63
64              @php
65                  $maxDecisionCount = max($approvedReservationCount, $rejectedReservationCount, 1);
66              @endphp
67              <section class="mt-6 rounded-2xl border border-slate-200 bg-white p-5 shadow-sm sm:p-6" aria-labelledby="reservation-outcomes-title">
68                  <div class="mb-6 flex flex-col gap-2 sm:flex-row sm:items-end sm:justify-between">
69                      <div>
70                          <h2 id="reservation-outcomes-title" class="text-xl font-semibold text-slate-900">Reservation outcomes</h2>
71                          <p class="mt-1 text-sm text-slate-500">Total approved and rejected reservations.</p>
72                      </div>
73                      <span class="text-xs font-medium text-slate-400">All time</span>
74                  </div>
75
76                  <div class="space-y-5" role="img" aria-label="Bar chart comparing total approved and rejected reservations">
77                      <div data-reservation-outcome="approved" data-total="{{ $approvedReservationCount }}">
78                          <div class="mb-2 flex items-center justify-between gap-4">
79                              <span class="inline-flex items-center gap-2 text-sm font-medium text-slate-700">
80                                  <span class="h-2.5 w-2.5 rounded-full bg-green-600" aria-hidden="true"></span>
81                                  Approved
82                              </span>
83                              <span class="text-sm font-semibold tabular-nums text-slate-900">{{ number_format($approvedReservationCount) }}</span>
84                          </div>
85                          <div class="h-3 overflow-hidden rounded-full bg-slate-100">
86                              <div class="h-full rounded-full bg-green-600 transition-[width]" style="width: {{ round(($approvedReservationCount / $maxDecisionCount) * 100,
     1) }}%"></div>
87                          </div>
88                      </div>
89
90                      <div data-reservation-outcome="rejected" data-total="{{ $rejectedReservationCount }}">
91                          <div class="mb-2 flex items-center justify-between gap-4">
92                              <span class="inline-flex items-center gap-2 text-sm font-medium text-slate-700">
93                                  <span class="h-2.5 w-2.5 rounded-full bg-rose-500" aria-hidden="true"></span>
94                                  Rejected
95                              </span>
96                              <span class="text-sm font-semibold tabular-nums text-slate-900">{{ number_format($rejectedReservationCount) }}</span>
97                          </div>
98                          <div class="h-3 overflow-hidden rounded-full bg-slate-100">
99                              <div class="h-full rounded-full bg-rose-500 transition-[width]" style="width: {{ round(($rejectedReservationCount / $maxDecisionCount) * 100,
    1) }}%"></div>
100                         </div>
101                     </div>
102                 </div>
103             </section>
104
105                 </section>
106
107             <section id="reservations" data-admin-section="reservations" class="hidden">
108         <section id="pending-reservations-section" class="scroll-mt-6 rounded-2xl border border-slate-200/80 bg-white p-4 shadow-sm sm:p-6">
109             <div class="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4 mb-4">
110                 <div>
111                     <h2 class="text-xl font-semibold text-gray-900">Pending Reservations</h2>
112                     <p class="text-sm text-gray-600">Review and approve or reject requests.</p>
113                 </div>
114                 <span id="pending-reservations-label" class="text-sm text-gray-500">{{ $pendingReservations->total() }} pending</span>
115             </div>
116             <p id="dashboard-action-status" class="mb-3 hidden text-sm text-emerald-700" role="status" aria-live="polite"></p>
117             <p id="dashboard-action-error" class="mb-3 hidden text-sm text-rose-700" role="alert"></p>
118
119             @if($pendingReservations->isEmpty())
120                 <p class="text-gray-500">No pending reservations at the moment.</p>
121             @else
122                 <div id="pending-reservations-list" class="space-y-4 pr-2" style="height: 32rem; overflow-y: scroll; scrollbar-width: auto; scrollbar-color: #6b7280
    #e5e7eb;" tabindex="0" aria-label="Scrollable pending reservations">
123                     @foreach($pendingReservations as $reservation)
124                         <div data-reservation-card class="rounded-3xl border border-slate-200 bg-white p-4 transition hover:border-indigo-200">
125                             <div class="flex flex-col gap-3 sm:flex-row sm:justify-between sm:items-start">
126                                 <div>
127                                     <p class="text-sm text-gray-500">Reservation #{{ $reservation->id }}</p>
128                                     <h3 class="text-lg font-semibold text-gray-900">{{ $reservation->items->isNotEmpty() ? 'Property items' :
    ($reservation->facility->name ?? 'Facility') }} — {{ $reservation->reservation_date->format('M d, Y') }}</h3>
```

```
                                    <p class="text-sm text-gray-600">
                                        @if($reservation->items->isNotEmpty())
                                            Use: {{ $reservation->start_time->format('H:i') }} · Return: {{ $reservation->end_time->format('H:i') }}
                                        @else
                                            {{ $reservation->start_time->format('H:i') }} – {{ $reservation->end_time->format('H:i') }}
                                        @endif
                                    </p>
                                    <p class="text-sm text-gray-600 mt-2">Requested by {{ $reservation->user->name }} ({{ $reservation->user->email }})</p>
                                    <p class="text-sm text-gray-600 mt-1">Purpose: {{ $reservation->purpose }}</p>
                                    @if($reservation->items->isNotEmpty())
                                        <p class="mt-2 text-sm text-gray-700">
                                            Items:
                                            @foreach($reservation->items as $item)
                                                {{ $item->name }} × {{ $item->pivot->quantity }}@if(!$loop->last), @endif
                                            @endforeach
                                        </p>
                                    @endif
                                </div>
                                <div class="flex gap-2">
                                    <form method="POST" action="{{ route('admin.reservations.approve', $reservation) }}" data-pending-action>
                                        @csrf
                                        <button type="submit" class="px-4 py-2 rounded-full bg-green-600 text-white hover:bg-green-700 transition">Approve</button>
                                    </form>
                                    <form method="POST" action="{{ route('admin.reservations.reject', $reservation) }}" data-pending-action>
                                        @csrf
                                        <button type="submit" class="px-4 py-2 rounded-full bg-red-600 text-white hover:bg-red-700 transition">Reject</button>
                                    </form>
                                    <form method="POST" action="{{ route('admin.reservations.destroy', $reservation) }}" data-pending-action data-confirm="Delete this reservation? This will remove it from the calendar and user view.">
                                        @csrf
                                        @method('DELETE')
                                        <button type="submit" class="px-4 py-2 rounded-full bg-gray-600 text-white hover:bg-gray-700 transition">Delete</button>
                                    </form>
                                </div>
                            </div>
                        </div>
                    @endforeach
                </div>
                <div class="mt-4">
                    {{ $pendingReservations->links() }}
                </div>
            @endif
        </section>

        <section id="edit-reservations-section" class="mt-6 scroll-mt-6 rounded-2xl border border-slate-200/80 bg-white p-4 shadow-sm sm:p-6">
            <div class="mb-4">
                <h2 class="text-xl font-semibold text-gray-900">Edit Reservations</h2>
                <p class="text-sm text-gray-600">Adjust approved reservation dates, schedules, or purpose. Deleting a reservation removes it from the client’s list and sends them a notification.</p>
            </div>
            <form id="approved-reservation-filter-form" method="GET" action="{{ route('admin.dashboard') }}#reservations" class="mb-4 flex flex-col gap-2 sm:w-64">
                <input type="hidden" name="role" value="{{ $role ?? 'all' }}">
                <input type="hidden" name="search" value="{{ $search ?? '' }}">
                <input type="hidden" name="member_sort" value="{{ $memberSort ?? 'joined_newest' }}">
                <label for="reservation-sort" class="text-sm font-medium text-slate-700">Sort approved reservations</label>
                <select id="reservation-sort" name="reservation_sort" class="rounded-full border border-slate-300 bg-white px-4 py-2 text-sm text-slate-700 focus:border-green-500 focus:outline-none focus:ring-2 focus:ring-green-100">
                    <option value="recent" {{ ($reservationSort ?? 'recent') === 'recent' ? 'selected' : '' }}>Most recent date</option>
                    <option value="oldest" {{ ($reservationSort ?? '') === 'oldest' ? 'selected' : '' }}>Oldest date</option>
                    <option value="alphabetical" {{ ($reservationSort ?? '') === 'alphabetical' ? 'selected' : '' }}>Alphabetical (room / item)</option>
                </select>
            </form>

            @if($approvedReservations->isEmpty())
                <p class="text-gray-500">No approved reservations to edit.</p>
            @else
                <div class="overflow-hidden rounded-2xl border border-slate-200">
                    <div class="h-[32rem] overflow-y-scroll overflow-x-auto" style="scrollbar-width: auto; scrollbar-color: #fbbf24 #fffbeb;" tabindex="0" aria-label="Scrollable approved reservations">
                        <table class="min-w-full divide-y divide-slate-200 bg-white">
                            <thead class="sticky top-0 z-10 bg-green-50">
                                <tr>
                                    <th class="whitespace-nowrap px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-slate-500">Reservation</th>
                                    <th class="whitespace-nowrap px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-slate-500">Room / Items</th>
```

```
                                    <th class="whitespace-nowrap px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-slate-500">Requested by</th>
                                    <th class="whitespace-nowrap px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-slate-500">Date</th>
                                    <th class="whitespace-nowrap px-4 py-3 text-right text-xs font-semibold uppercase tracking-wide text-slate-500">Actions</th>
                                </tr>
                            </thead>
                            <tbody class="divide-y divide-slate-200">
                                @foreach($approvedReservations as $reservation)
                                    <tr class="hover:bg-green-50/40">
                                        <td class="whitespace-nowrap px-4 py-3 text-sm font-semibold text-slate-800">#{{ $reservation->id }}</td>
                                        <td class="px-4 py-3 text-sm text-slate-700">
                                            {{ $reservation->facility?->name ?? ($reservation->items->isNotEmpty() ? 'Property items' : 'Facility') }}
                                            @if($reservation->items->isNotEmpty())
                                                <span class="block max-w-xs truncate text-xs text-slate-500">
                                                    @foreach($reservation->items as $item)
                                                        {{ $item->name }} × {{ $item->pivot->quantity }}@if(!$loop->last), @endif
                                                    @endforeach
                                                </span>
                                            @endif
                                        </td>
                                        <td class="whitespace-nowrap px-4 py-3 text-sm text-slate-600">{{ $reservation->user?->name ?? 'Unknown user' }}</td>
                                        <td class="whitespace-nowrap px-4 py-3 text-sm text-slate-600">{{ $reservation->reservation_date->format('M d, Y') }}</td>
                                        <td class="whitespace-nowrap px-4 py-3 text-right">
                                            <div class="inline-flex items-center gap-2">
                                                <button
                                                    type="button"
                                                    data-edit-reservation
                                                    data-edit-action="{{ route('admin.reservations.update', $reservation) }}"
                                                    data-reservation-id="{{ $reservation->id }}"
                                                    data-reservation-room="{{ $reservation->facility?->name ?? ($reservation->items->isNotEmpty() ? 'Property items' :
'Facility') }}"
                                                    data-reservation-date="{{ $reservation->reservation_date->format('Y-m-d') }}"
                                                    data-reservation-start="{{ $reservation->start_time->format('H:i') }}"
                                                    data-reservation-end="{{ $reservation->end_time->format('H:i') }}"
                                                    data-reservation-purpose="{{ $reservation->purpose }}"
                                                    class="rounded-full bg-green-700 px-4 py-2 text-xs font-semibold text-white transition hover:bg-green-800">
                                                    Edit
                                                </button>
                                                <form method="POST" action="{{ route('admin.reservations.destroy', $reservation) }}" data-confirm="Delete this approved
reservation? The client will be notified.">
                                                    @csrf
                                                    @method('DELETE')
                                                    <button type="submit" class="rounded-full bg-red-600 px-4 py-2 text-xs font-semibold text-white transition
hover:bg-red-700">Delete</button>
                                                </form>
                                            </div>
                                        </td>
                                    </tr>
                                @endforeach
                            </tbody>
                        </table>
                    </div>
                </div>
                <div class="mt-4">
                    {{ $approvedReservations->links() }}
                </div>
            @endif
        </section>

        <div id="approved-reservation-edit-modal" class="fixed inset-0 z-[90] hidden items-center justify-center overflow-y-auto bg-slate-950/60 p-4 backdrop-blur-sm"
role="dialog" aria-modal="true" aria-labelledby="approved-reservation-edit-title">
            <div class="my-auto w-full max-w-3xl rounded-3xl border border-slate-200 bg-white p-5 shadow-2xl sm:p-7">
                <div class="mb-6 flex items-start justify-between gap-4">
                    <div>
                        <p class="text-sm font-medium text-slate-500">Approved reservation</p>
                        <h2 id="approved-reservation-edit-title" class="mt-1 text-2xl font-semibold text-slate-900">Edit reservation</h2>
                        <p id="approved-reservation-edit-summary" class="mt-1 text-sm text-slate-600"></p>
                    </div>
                    <button id="approved-reservation-edit-close" type="button" class="rounded-full p-2 text-xl leading-none text-slate-500 hover:bg-slate-100
hover:text-slate-900" aria-label="Close edit reservation dialog">&times;</button>
                </div>

                <form id="approved-reservation-edit-form" method="POST" action="#" data-confirm="Save these changes to the approved reservation?" class="space-y-5">
                    @csrf
                    @method('PUT')
```

```
                        <div class="grid gap-4 sm:grid-cols-2">
                            <label class="block text-sm font-medium text-slate-700">
                                Date
                                <input id="approved-reservation-edit-date" name="reservation_date" type="date" required min="{{ now()->toDateString() }}" max="{{ $maximumReservationDate }}" class="mt-1 w-full rounded-xl border border-slate-300 bg-white px-3 py-3 text-sm focus:border-green-700 focus:outline-none focus:ring-2 focus:ring-green-100">
                            </label>
                            <label class="block text-sm font-medium text-slate-700">
                                Start time
                                <input id="approved-reservation-edit-start" name="start_time" type="time" required class="mt-1 w-full rounded-xl border border-slate-300 bg-white px-3 py-3 text-sm focus:border-green-700 focus:outline-none focus:ring-2 focus:ring-green-100">
                            </label>
                            <label class="block text-sm font-medium text-slate-700">
                                End time
                                <input id="approved-reservation-edit-end" name="end_time" type="time" required class="mt-1 w-full rounded-xl border border-slate-300 bg-white px-3 py-3 text-sm focus:border-green-700 focus:outline-none focus:ring-2 focus:ring-green-100">
                            </label>
                            <label class="block text-sm font-medium text-slate-700">
                                Purpose
                                <input id="approved-reservation-edit-purpose" name="purpose" type="text" required maxlength="255" class="mt-1 w-full rounded-xl border border-slate-300 bg-white px-3 py-3 text-sm focus:border-green-700 focus:outline-none focus:ring-2 focus:ring-green-100">
                            </label>
                        </div>
                        <div class="flex flex-col-reverse gap-3 border-t border-slate-100 pt-5 sm:flex-row sm:justify-end">
                            <button id="approved-reservation-edit-cancel" type="button" class="rounded-xl border border-slate-300 px-5 py-3 text-sm font-semibold text-slate-700 transition hover:bg-slate-50">Cancel</button>
                            <button type="submit" class="rounded-xl bg-green-800 px-6 py-3 text-sm font-semibold text-white transition hover:bg-green-900">Save changes</button>
                        </div>
                    </form>
                </div>
            </div>
                </section>

                <section id="audit" data-admin-section="audit" class="hidden">
            <section id="audit-activity-section" class="scroll-mt-6 rounded-2xl border border-slate-200/80 bg-white p-4 shadow-sm sm:p-6">
                <div class="mb-4">
                    <h2 class="text-xl font-semibold text-gray-900">Audit Log</h2>
                    <p class="text-sm text-gray-600">Track administrator actions and reservation activity.</p>
                </div>

                <div id="audit-log-list" class="space-y-4 pr-2" style="height: 32rem; overflow-y: scroll; scrollbar-width: auto; scrollbar-color: #6b7280 #e5e7eb;" tabindex="0" aria-label="Scrollable audit log entries">
                    @if($auditLogs->isEmpty())
                        <p data-empty-audit class="text-gray-500">No audit log entries yet.</p>
                    @else
                        @foreach($auditLogs as $log)
                            <div class="rounded-3xl border border-slate-200 bg-indigo-50/40 p-4">
                                <div class="flex items-start justify-between gap-4">
                                    <div>
                                        <p class="text-sm text-gray-500">{{ $log->created_at->format('M d, Y H:i:s') }}</p>
                                        <p class="mt-1 text-base font-semibold text-gray-900 capitalize">{{ $log->action }}</p>
                                        <p class="text-sm text-gray-700 mt-1">{{ $log->details }}</p>
                                        @if($log->reservation)
                                            <p class="text-xs text-gray-500 mt-2">Reservation #{{ $log->reservation->id }}{{ $log->reservation->facility ? ' — '.$log->reservation->facility->name : ' — Property items' }}</p>
                                        @endif
                                    </div>
                                    <span class="px-3 py-1 rounded-full bg-gray-200 text-xs font-semibold uppercase text-gray-700">{{ $log->user?->name ?? 'Admin' }}</span>
                                </div>
                            </div>
                        @endforeach
                    @endif
                </div>
            </section>
                </section>

                <section id="members" data-admin-section="members" class="hidden">
        <p id="member-management-status" class="mt-3 hidden text-sm text-emerald-700" role="status" aria-live="polite"></p>
        <p id="member-management-error" class="mt-3 hidden text-sm text-rose-700" role="alert"></p>
        <div id="member-management-content" data-endpoint="{{ route('admin.dashboard') }}" aria-busy="false">
            @include('admin.partials.members')
        </div>
                </section>
```

```
            </div>
        </main>
    </div>
</div>
@include('admin.partials.password-confirmation')
@endsection

@push('scripts')
<script>
    document.addEventListener('DOMContentLoaded', function() {
        const memberContent = document.getElementById('member-management-content');
        const reservationEditModal = document.getElementById('approved-reservation-edit-modal');
        const reservationEditForm = document.getElementById('approved-reservation-edit-form');
        const reservationEditSummary = document.getElementById('approved-reservation-edit-summary');
        const reservationEditDate = document.getElementById('approved-reservation-edit-date');
        const reservationEditStart = document.getElementById('approved-reservation-edit-start');
        const reservationEditEnd = document.getElementById('approved-reservation-edit-end');
        const reservationEditPurpose = document.getElementById('approved-reservation-edit-purpose');
        let reservationEditTrigger = null;
        const errorMessage = document.getElementById('member-management-error');
        const memberStatus = document.getElementById('member-management-status');
        const dashboardStatus = document.getElementById('dashboard-action-status');
        const dashboardError = document.getElementById('dashboard-action-error');
        let activeRequest = null;
        let searchTimer = null;
        const dashboardSections = Array.from(document.querySelectorAll('[data-admin-section]'));
        const dashboardSectionLinks = Array.from(document.querySelectorAll('[data-admin-section-link]'));
        const legacySectionNames = {
            'pending-reservations-section': 'reservations',
            'member-management-section': 'members',
            'audit-activity-section': 'audit',
        };

        function closeReservationEditModal() {
            reservationEditModal.classList.add('hidden');
            reservationEditModal.classList.remove('flex');
            reservationEditTrigger?.focus();
            reservationEditTrigger = null;
        }

        document.addEventListener('click', function(event) {
            if (!(event.target instanceof Element)) return;

            const editButton = event.target.closest('[data-edit-reservation]');
            if (editButton) {
                reservationEditTrigger = editButton;
                reservationEditForm.action = editButton.dataset.editAction;
                reservationEditSummary.textContent = `Reservation #${editButton.dataset.reservationId} — ${editButton.dataset.reservationRoom}`;
                reservationEditDate.value = editButton.dataset.reservationDate;
                reservationEditStart.value = editButton.dataset.reservationStart;
                reservationEditEnd.value = editButton.dataset.reservationEnd;
                reservationEditPurpose.value = editButton.dataset.reservationPurpose;
                reservationEditModal.classList.remove('hidden');
                reservationEditModal.classList.add('flex');
                reservationEditDate.focus();
                return;
            }

            if (event.target === reservationEditModal
                || event.target.closest('#approved-reservation-edit-close, #approved-reservation-edit-cancel')) {
                closeReservationEditModal();
            }
        });

        document.addEventListener('keydown', function(event) {
            const passwordDialog = document.getElementById('admin-password-confirmation');
            if (event.key === 'Escape'
                && !reservationEditModal.classList.contains('hidden')
                && (!passwordDialog || passwordDialog.classList.contains('hidden'))) {
                closeReservationEditModal();
            }
        });

        function activateDashboardSection(sectionName, updateHistory) {
```

```
            const section = dashboardSections.find(function(item) {
                return item.dataset.adminSection === sectionName;
            });
            if (!section) return;

            dashboardSections.forEach(function(item) {
                item.classList.toggle('hidden', item !== section);
            });
            dashboardSectionLinks.forEach(function(link) {
                const isActive = link.dataset.adminSectionLink === sectionName;
                link.classList.toggle('bg-green-50', isActive);
                link.classList.toggle('text-green-900', isActive);
                link.classList.toggle('font-semibold', isActive);
                link.classList.toggle('text-slate-600', !isActive);
                link.classList.toggle('font-medium', !isActive);
                link.setAttribute('aria-current', isActive ? 'page' : 'false');
            });

            if (updateHistory) {
                window.history.pushState({}, '', `#${sectionName}`);
            }
            window.scrollTo(0, 0);
        }

        function loadApprovedReservations(url, updateHistory) {
            url.hash = '#reservations';
            fetch(url, {
                headers: { 'Accept': 'text/html' },
                credentials: 'same-origin',
            })
                .then(function(response) {
                    if (!response.ok) throw new Error('Approved reservations could not be updated. Please try again.');
                    return response.text();
                })
                .then(function(html) {
                    const updatedDocument = new DOMParser().parseFromString(html, 'text/html');
                    const updatedSection = updatedDocument.querySelector('#edit-reservations-section');
                    const currentSection = document.querySelector('#edit-reservations-section');
                    if (!updatedSection || !currentSection) {
                        throw new Error('The updated approved reservation list could not be displayed.');
                    }
                    currentSection.replaceWith(updatedSection);
                    if (updateHistory) {
                        window.history.pushState({}, '', url);
                    }
                    activateDashboardSection('reservations', false);
                })
                .catch(function(error) {
                    dashboardError.textContent = error.message;
                    dashboardError.classList.remove('hidden');
                });
        }

        function approvedReservationsUrl(form) {
            const url = new URL(form.action, window.location.origin);
            const params = new URLSearchParams(new FormData(form));
            url.search = params.toString();
            url.hash = '#reservations';
            return url;
        }

        document.addEventListener('change', function(event) {
            if (!(event.target instanceof HTMLSelectElement) || event.target.id !== 'reservation-sort') return;
            const form = event.target.form;
            if (!form) return;
            const url = approvedReservationsUrl(form);
            url.searchParams.delete('approvedPage');
            loadApprovedReservations(url, true);
        });

        document.addEventListener('click', function(event) {
            if (!(event.target instanceof Element)) return;
            const link = event.target.closest('#edit-reservations-section nav[aria-label="Pagination Navigation"] a');
            if (!link || event.ctrlKey || event.metaKey || event.shiftKey || event.altKey) return;
```

```
            event.preventDefault();
            const url = new URL(link.href);
            url.hash = '#reservations';
            loadApprovedReservations(url, true);
        });

        function activateDashboardSectionFromLocation() {
            const hash = window.location.hash.slice(1);
            const sectionName = legacySectionNames[hash] || hash || 'overview';
            activateDashboardSection(sectionName, false);
        }

        dashboardSectionLinks.forEach(function(link) {
            link.addEventListener('click', function(event) {
                event.preventDefault();
                activateDashboardSection(link.dataset.adminSectionLink, true);
            });
        });
        window.addEventListener('popstate', activateDashboardSectionFromLocation);
        activateDashboardSectionFromLocation();

        function showMessage(element, message) {
            element.textContent = message;
            element.classList.remove('hidden');
        }

        function setLoading(loading) {
            memberContent.setAttribute('aria-busy', String(loading));
            memberContent.querySelector('#member-search-loading')?.classList.toggle('hidden', !loading);
            memberContent.querySelector('#member-search-loading')?.classList.toggle('inline-flex', loading);
        }

        function loadMembers(url, historyMode) {
            if (activeRequest) activeRequest.abort();
            const controller = new AbortController();
            activeRequest = controller;
            setLoading(true);
            errorMessage.classList.add('hidden');

            fetch(url, {
                headers: {
                    'Accept': 'text/html',
                    'X-Requested-With': 'XMLHttpRequest',
                },
                credentials: 'same-origin',
                signal: controller.signal,
            })
                .then(function(response) {
                    if (!response.ok) {
                        throw new Error('Member list could not be updated. Please try again.');
                    }
                    return response.text();
                })
                .then(function(html) {
                    const previousScrollTop = memberContent.querySelector('[aria-label="Scrollable member list"]')?.scrollTop || 0;
                    const activeElement = document.activeElement;
                    const preserveSearchFocus = activeElement instanceof HTMLInputElement
                        && activeElement.name === 'search'
                        && memberContent.contains(activeElement);
                    const currentSearch = preserveSearchFocus
                        ? activeElement.value
                        : memberContent.querySelector('#search')?.value;
                    const selectionStart = preserveSearchFocus ? activeElement.selectionStart : null;
                    const selectionEnd = preserveSearchFocus ? activeElement.selectionEnd : null;

                    memberContent.innerHTML = html;
                    const updatedSearch = memberContent.querySelector('#search');
                    if (updatedSearch && currentSearch !== undefined) {
                        updatedSearch.value = currentSearch;
                    }
                    if (preserveSearchFocus && updatedSearch) {
                        updatedSearch.focus();
                        if (selectionStart !== null && selectionEnd !== null) {
                            updatedSearch.setSelectionRange(selectionStart, selectionEnd);
```

```
                    }
                }
                const updatedList = memberContent.querySelector('[aria-label="Scrollable member list"]');
                if (updatedList) updatedList.scrollTop = previousScrollTop;
                if (historyMode === 'push') {
                    window.history.pushState({}, '', url);
                } else if (historyMode === 'replace') {
                    window.history.replaceState({}, '', url);
                }
            })
            .catch(function(error) {
                if (error.name !== 'AbortError') {
                    errorMessage.textContent = error.message;
                    errorMessage.classList.remove('hidden');
                }
            })
            .finally(function() {
                if (activeRequest === controller) {
                    activeRequest = null;
                    setLoading(false);
                }
            });
    }

    function membersUrl(pageUrl) {
        const url = new URL(pageUrl || memberContent.dataset.endpoint, window.location.origin);
        const form = memberContent.querySelector('#member-filter-form');
        if (form) {
            const formData = new FormData(form);
            url.searchParams.set('search', String(formData.get('search') || '').trim());
            url.searchParams.set('role', String(formData.get('role') || 'all'));
            url.searchParams.set('member_sort', String(formData.get('member_sort') || 'joined_newest'));
            const currentUrl = new URL(window.location.href);
            if (currentUrl.searchParams.has('reservation_sort')) {
                url.searchParams.set('reservation_sort', currentUrl.searchParams.get('reservation_sort'));
            }
            if (!pageUrl) url.searchParams.delete('page');
        }
        url.hash = '#members';
        return url;
    }

    memberContent.addEventListener('submit', function(event) {
        const form = event.target;
        if (event.defaultPrevented) return;
        if (!(form instanceof HTMLFormElement)) return;
        if (form.id !== 'member-filter-form') {
            if (!form.action.includes('/users/')) return;
            event.preventDefault();
            memberStatus.classList.add('hidden');
            errorMessage.classList.add('hidden');
            const button = form.querySelector('button[type="submit"]');
            const originalText = button?.textContent;
            if (button) {
                button.disabled = true;
                button.textContent = 'Saving...';
            }
            const requestBody = new FormData(form);
            form.querySelector('input[name="current_password"]')?.remove();
            delete form.dataset.adminPasswordConfirmed;

            fetch(form.action, {
                method: form.method.toUpperCase(),
                headers: {
                    'Accept': 'application/json',
                    'X-Requested-With': 'XMLHttpRequest',
                },
                body: requestBody,
                credentials: 'same-origin',
            })
                .then(async function(response) {
                    if (response.status === 419) throw new Error('Your session expired. Refresh this page and try again.');
                    const data = await response.json();
                    if (!response.ok) {
```

```
                            throw new Error(Object.values(data.errors || {}).flat()[0] || data.message || 'Member update failed.');
                        }
                        return data;
                    })
                    .then(function(data) {
                        showMessage(memberStatus, data.status || 'Member updated.');
                        loadMembers(membersUrl().toString(), false);
                    })
                    .catch(function(error) {
                        showMessage(errorMessage, error.message);
                    })
                    .finally(function() {
                        if (button) {
                            button.disabled = false;
                            button.textContent = originalText;
                        }
                    });
                return;
            }
            event.preventDefault();
            window.clearTimeout(searchTimer);
            loadMembers(membersUrl().toString(), 'push');
        });

        memberContent.addEventListener('input', function(event) {
            if (!(event.target instanceof HTMLInputElement) || event.target.name !== 'search') return;
            window.clearTimeout(searchTimer);
            searchTimer = window.setTimeout(function() {
                loadMembers(membersUrl().toString(), 'replace');
            }, 200);
        });

        memberContent.addEventListener('change', function(event) {
            if (!(event.target instanceof HTMLSelectElement)
                || ![‘role', 'member_sort'].includes(event.target.name)
                || event.target.form?.id !== 'member-filter-form') return;
            loadMembers(membersUrl().toString(), 'push');
        });

        memberContent.addEventListener('click', function(event) {
            if (!(event.target instanceof Element)) return;
            const link = event.target.closest('nav[aria-label="Pagination Navigation"] a');
            if (!link || event.ctrlKey || event.metaKey || event.shiftKey || event.altKey) return;
            event.preventDefault();
            loadMembers(link.href, true);
        });

        window.addEventListener('popstate', function() {
            loadMembers(window.location.href, false);
        });

        document.addEventListener('submit', function(event) {
            const form = event.target;
            if (event.defaultPrevented) return;
            if (!(form instanceof HTMLFormElement) || !form.hasAttribute('data-pending-action')) return;
            event.preventDefault();
            dashboardStatus.classList.add('hidden');
            dashboardError.classList.add('hidden');
            const button = form.querySelector('button[type="submit"]');
            const originalText = button?.textContent;
            if (button) {
                button.disabled = true;
                button.textContent = 'Saving...';
            }
            const requestBody = new FormData(form);
            form.querySelector('input[name="current_password"]')?.remove();
            delete form.dataset.adminPasswordConfirmed;

            fetch(form.action, {
                method: form.method.toUpperCase(),
                headers: {
                    'Accept': 'application/json',
                    'X-Requested-With': 'XMLHttpRequest',
                },
```

```
                body: requestBody,
                credentials: 'same-origin',
            })
                .then(async function(response) {
                    if (response.status === 419) throw new Error('Your session expired. Refresh this page and try again.');
                    const data = await response.json();
                    if (!response.ok) throw new Error(data.message || 'Reservation update failed.');
                    return data;
                })
                .then(function(data) {
                    const list = document.getElementById('pending-reservations-list');
                    const scrollTop = list?.scrollTop || 0;
                    form.closest('[data-reservation-card]')?.remove();
                    const updatedList = document.getElementById('pending-reservations-list');
                    if (updatedList) {
                        updatedList.scrollTop = scrollTop;
                        if (!updatedList.querySelector('[data-reservation-card]')) {
                            const emptyMessage = document.createElement('p');
                            emptyMessage.className = 'text-gray-500';
                            emptyMessage.textContent = data.pending_count === 0
                                ? 'No pending reservations at the moment.'
                                : 'No pending reservations on this page.';
                            updatedList.appendChild(emptyMessage);
                        }
                    }
                    document.getElementById('pending-reservations-total').textContent = data.pending_count;
                    document.getElementById('pending-reservations-label').textContent = `${data.pending_count} pending`;
                    document.getElementById('recent-decisions-total').textContent = data.recent_decision_count;
                    document.getElementById('audit-log-total').textContent = data.audit_log_count;

                    if (data.audit_log) {
                        const auditList = document.getElementById('audit-log-list');
                        const auditScrollTop = auditList.scrollTop;
                        auditList.querySelector('[data-empty-audit]')?.remove();
                        const entry = document.createElement('div');
                        entry.className = 'rounded-3xl border border-slate-200 bg-indigo-50/40 p-4';
                        const content = document.createElement('div');
                        content.className = 'flex items-start justify-between gap-4';
                        const details = document.createElement('div');
                        const timestamp = document.createElement('p');
                        timestamp.className = 'text-sm text-gray-500';
                        timestamp.textContent = data.audit_log.created_at;
                        const action = document.createElement('p');
                        action.className = 'mt-1 text-base font-semibold text-gray-900 capitalize';
                        action.textContent = data.audit_log.action;
                        const description = document.createElement('p');
                        description.className = 'mt-1 text-sm text-gray-700';
                        description.textContent = data.audit_log.details;
                        const actor = document.createElement('span');
                        actor.className = 'rounded-full bg-gray-200 px-3 py-1 text-xs font-semibold uppercase text-gray-700';
                        actor.textContent = data.audit_log.user;
                        details.append(timestamp, action, description);
                        content.append(details, actor);
                        entry.appendChild(content);
                        auditList.prepend(entry);
                        auditList.querySelectorAll(':scope > div').forEach(function(oldEntry, index) {
                            if (index >= 25) oldEntry.remove();
                        });
                        auditList.scrollTop = auditScrollTop;
                    }

                    showMessage(dashboardStatus, data.status || 'Reservation updated.');
                })
                .catch(function(error) {
                    showMessage(dashboardError, error.message);
                })
                .finally(function() {
                    if (button) {
                        button.disabled = false;
                        button.textContent = originalText;
                    }
                });
        });
    });
```

```
</script>
@endpush
```

resources/views/admin/facilities/create.blade.php

```
@extends('layouts.app')

@section('title', 'Facility Management - Campus Reserve')

@section('content')
<div class="min-h-screen bg-[#f5f3f7]">
    <div class="flex min-h-screen flex-col md:flex-row">
        @include('admin.partials.sidebar')
        <main class="min-w-0 flex-1 px-3 py-5 sm:px-6 sm:py-8 xl:px-10">
    <div class="mx-auto max-w-[1600px]">
        <div class="mb-8">
            <h1 class="text-3xl font-bold text-gray-900">Facility Management</h1>
            <p class="mt-2 text-gray-600">Add facilities and control the website cards shown to visitors. Manage facility items and quantities in Property Management.</p>
        </div>

        @if(session('status'))
            <div class="mb-6 bg-green-100 border border-green-300 text-green-800 px-4 py-3 rounded-xl">{{ session('status') }}</div>
        @endif
        @if(session('error'))
            <div class="mb-6 bg-red-100 border border-red-300 text-red-800 px-4 py-3 rounded-xl">{{ session('error') }}</div>
        @endif
        @if($errors->has('current_password'))
            <div class="mb-6 rounded-xl border border-red-300 bg-red-50 px-4 py-3 text-red-800" role="alert">{{ $errors->first('current_password') }}</div>
        @endif

        <div class="grid grid-cols-1 gap-6">
            <section class="rounded-3xl border border-gray-200 bg-white p-6 shadow-sm">
                <div class="mb-6">
                    <h2 class="text-2xl font-bold text-gray-900">Add Facility</h2>
                    <p class="mt-2 text-sm text-gray-600">Add a room or space to Campus Reserve.</p>
                </div>

                <form id="facility-form" method="POST" action="{{ route('admin.facilities.store') }}" data-confirm-message="Save this facility change?" class="space-y-6">
                    @csrf
                    <input type="hidden" name="_method" id="facility-form-method" value="POST">

                    <div>
                        <label for="name" class="mb-2 block text-sm font-medium text-gray-700">Facility name</label>
                        <input id="name" name="name" type="text" value="{{ old('name') }}" required maxlength="100" class="w-full rounded-xl border border-gray-300 px-4 py-3
focus:border-black focus:outline-none focus:ring-1 focus:ring-black">
                        @error('name')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                    </div>

                    <div>
                        <label for="building" class="mb-2 block text-sm font-medium text-gray-700">Building</label>
                        <input id="building" name="building" type="text" value="{{ old('building') }}" required maxlength="100" class="w-full rounded-xl border
border-gray-300 px-4 py-3 focus:border-black focus:outline-none focus:ring-1 focus:ring-black">
                        @error('building')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                    </div>

                    <div class="grid grid-cols-1 gap-6 sm:grid-cols-2">
                        <div>
                            <label for="type" class="mb-2 block text-sm font-medium text-gray-700">Facility type</label>
                            <input id="type" name="type" type="text" value="{{ old('type') }}" required maxlength="50" placeholder="e.g. Classroom, Laboratory" class="w-full
rounded-xl border border-gray-300 px-4 py-3 focus:border-black focus:outline-none focus:ring-1 focus:ring-black">
                            @error('type')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                        </div>
                        <div>
                            <label for="capacity" class="mb-2 block text-sm font-medium text-gray-700">Capacity</label>
                            <input id="capacity" name="capacity" type="number" value="{{ old('capacity') }}" required min="1" step="1" class="w-full rounded-xl border
border-gray-300 px-4 py-3 focus:border-black focus:outline-none focus:ring-1 focus:ring-black">
                            @error('capacity')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                        </div>
                    </div>

                    <div class="flex justify-end gap-3 border-t border-gray-100 pt-4">
                        <button type="button" id="facility-reset-form" class="rounded-full px-5 py-2.5 text-sm font-medium text-gray-700 hover:bg-gray-100">Cancel</button>
                        <button id="facility-submit-button" type="submit" class="rounded-full bg-black px-5 py-2.5 text-sm font-semibold text-white hover:bg-gray-800">Save
Facility</button>
```

```
                    </div>
                </form>
            </section>

            <aside class="rounded-3xl border border-gray-200 bg-white p-6 shadow-sm">
                <div class="mb-6 flex items-start justify-between gap-4">
                    <div>
                        <h2 class="text-2xl font-bold text-gray-900">Edit/Remove</h2>
                        <p class="mt-1 text-sm text-gray-600">Update or remove cards shown on the public site.</p>
                    </div>

                    <div class="flex w-52 items-center gap-2 rounded-xl border border-gray-200 bg-gray-50 px-3 py-2.5 shadow-sm">
                        <svg xmlns="http://www.w3.org/2000/svg" class="h-4 w-4 text-gray-400" fill="none" viewBox="0 0 24 24" stroke="currentColor" stroke-width="2">
                            <path stroke-linecap="round" stroke-linejoin="round" d="M21 21l-4.35-4.35m1.85-5.15a7 7 0 11-14 0 7 7 0 0114 0z" />
                        </svg>
                        <input id="facility-search" type="text" placeholder="Search" class="w-full border-0 bg-transparent p-0 text-sm text-gray-700 placeholder:text-gray-400
 focus:outline-none focus:ring-0">
                    </div>
                </div>

                <div id="facility-scroll-area" class="space-y-3 rounded-2xl border border-gray-200 bg-gray-50/40 p-2 pr-3" style="height: 420px; max-height: 420px;
overflow-y: auto; scrollbar-width: thin; scrollbar-color: #5f5f5f #f3f4f6;">
                    @forelse($facilities as $facility)
                        <div class="rounded-2xl border border-gray-200 bg-gray-50 p-3 facility-card" data-name="{{ strtolower($facility->name) }}" data-description="{{
strtolower($facility->description ?? '') }}">
                            <div class="flex items-start justify-between gap-3">
                                <div class="min-w-0 flex-1">
                                    <h3 class="text-lg font-semibold text-gray-900">{{ $facility->name }}</h3>
                                    <p class="mt-1 text-sm text-gray-600">{{ $facility->description }}</p>
                                </div>
                                <div class="flex shrink-0 gap-2">
                                    <button type="button"
                                        data-edit-id="{{ $facility->id }}"
                                        data-name="{{ $facility->name }}"
                                        data-building="{{ $facility->building }}"
                                        data-type="{{ $facility->type }}"
                                        data-capacity="{{ $facility->capacity }}"
                                        class="rounded-full border border-gray-300 px-3 py-1.5 text-xs font-medium text-gray-700 hover:bg-gray-100">
                                        Edit
                                    </button>
                                    <button type="submit" form="delete-facility-form-{{ $facility->id }}" class="rounded-full border border-red-200 bg-red-50 px-3 py-1.5
text-xs font-medium text-red-700 hover:bg-red-100">Remove</button>
                                </div>
                            </div>
                        </div>
                        <form id="delete-facility-form-{{ $facility->id }}" method="POST" action="{{ route('admin.facilities.destroy', $facility) }}"
data-confirm-message="Remove this facility?" class="hidden">
                            @csrf
                            @method('DELETE')
                        </form>
                    @empty
                        <div class="rounded-2xl border border-dashed border-gray-300 bg-gray-50 p-6 text-center text-sm text-gray-500">
                            No facilities found yet.
                        </div>
                    @endforelse
                    <div id="no-facilities-match" class="hidden rounded-2xl border border-dashed border-gray-300 bg-gray-50 p-6 text-center text-sm text-gray-500">
                        No matching facilities found.
                    </div>
                </div>
            </aside>
        </div>
    </div>
        </main>
    </div>
</div>

<script>
    function resetFacilityForm() {
        const form = document.getElementById('facility-form');
        if (!form) {
            return;
        }

        form.action = '{{ route('admin.facilities.store') }}';
```

```
        document.getElementById('facility-form-method').value = 'POST';
        document.getElementById('facility-submit-button').textContent = 'Save Facility';
        form.dataset.confirmMessage = 'Add this facility?';
        form.reset();
    }

    function openFacilityEditForm(button) {
        const form = document.getElementById('facility-form');
        if (!form) {
            return;
        }

        const facilityId = button.dataset.editId;
        if (!facilityId) {
            return;
        }

        form.action = `/admin/facilities/${facilityId}`;
        document.getElementById('facility-form-method').value = 'PUT';
        document.getElementById('facility-submit-button').textContent = 'Update Facility';
        form.dataset.confirmMessage = `Update ${button.dataset.name || 'this facility'}?`;

        document.getElementById('name').value = button.dataset.name || '';
        document.getElementById('building').value = button.dataset.building || '';
        document.getElementById('type').value = button.dataset.type || '';
        document.getElementById('capacity').value = button.dataset.capacity || '';

        const formTop = form.getBoundingClientRect().top + window.scrollY - 80;
        window.scrollTo({ top: formTop, behavior: 'smooth' });
    }

    document.getElementById('facility-reset-form')?.addEventListener('click', resetFacilityForm);
    document.querySelectorAll('[data-edit-id]').forEach((button) => {
        button.addEventListener('click', function () {
            openFacilityEditForm(this);
        });
    });

    document.addEventListener('DOMContentLoaded', function () {
        const searchInput = document.getElementById('facility-search');
        const cards = Array.from(document.querySelectorAll('.facility-card'));
        const noMatch = document.getElementById('no-facilities-match');

        if (!searchInput || cards.length === 0) {
            return;
        }

        searchInput.addEventListener('input', function () {
            const query = this.value.trim().toLowerCase();
            let visibleCount = 0;

            cards.forEach((card) => {
                const text = (card.dataset.name || '') + ' ' + (card.dataset.description || '');
                const visible = text.includes(query);
                card.style.display = visible ? '' : 'none';

                if (visible) {
                    visibleCount += 1;
                }
            });

            if (noMatch) {
                noMatch.classList.toggle('hidden', visibleCount > 0 || query === '');
            }
        });
    });
</script>

<style>
    #facility-scroll-area::-webkit-scrollbar {
        width: 10px;
    }

    #facility-scroll-area::-webkit-scrollbar-track {
```

```
        background: #f3f4f6;
        border-radius: 9999px;
    }

    #facility-scroll-area::-webkit-scrollbar-thumb {
        background: #5f5f5f;
        border: 2px solid #f3f4f6;
        border-radius: 9999px;
    }

    #facility-scroll-area::-webkit-scrollbar-thumb:hover {
        background: #4b5563;
    }
</style>
@include('admin.partials.password-confirmation')
@endsection
```

resources/views/admin/partials/members.blade.php

```
<section id="member-management-section" class="scroll-mt-6 rounded-2xl border border-slate-200/80 bg-white p-4 shadow-sm sm:p-6">
    <div class="mb-4 flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-between">
        <div>
            <h2 class="text-xl font-semibold text-gray-900">Member Management</h2>
            <p class="text-sm text-gray-600">Search members and update staff access.</p>
        </div>
        <form id="member-filter-form" method="GET" action="{{ route('admin.dashboard') }}" class="flex w-full flex-col gap-3 sm:w-auto sm:flex-row">
            <label class="sr-only" for="role-filter">Filter members by role</label>
            <select id="role-filter" name="role" class="rounded-full border border-slate-300 bg-white px-4 py-2 text-sm text-slate-700 focus:border-indigo-400 focus:outline-none focus:ring-2 focus:ring-indigo-100">
                <option value="all" {{ ($role ?? 'all') === 'all' ? 'selected' : '' }}>All roles</option>
                <option value="user" {{ ($role ?? '') === 'user' ? 'selected' : '' }}>Members only</option>
                <option value="admin" {{ ($role ?? '') === 'admin' ? 'selected' : '' }}>Staff only</option>
                <option value="super_admin" {{ ($role ?? '') === 'super_admin' ? 'selected' : '' }}>Admins only</option>
            </select>
            <label class="sr-only" for="member-sort">Sort members</label>
            <select id="member-sort" name="member_sort" class="rounded-full border border-slate-300 bg-white px-4 py-2 text-sm text-slate-700 focus:border-indigo-400 focus:outline-none focus:ring-2 focus:ring-indigo-100">
                <option value="name" {{ ($memberSort ?? 'joined_newest') === 'name' ? 'selected' : '' }}>Name (A–Z)</option>
                <option value="joined_newest" {{ ($memberSort ?? 'joined_newest') === 'joined_newest' ? 'selected' : '' }}>Joined (newest)</option>
                <option value="joined_oldest" {{ ($memberSort ?? '') === 'joined_oldest' ? 'selected' : '' }}>Joined (oldest)</option>
            </select>
            <div class="relative">
                <label class="sr-only" for="search">Search members</label>
                <input id="search" name="search" type="search" value="{{ $search ?? '' }}" placeholder="Search name or email" class="w-full rounded-full border border-slate-300 bg-slate-50 py-2 pl-4 pr-20 text-sm focus:border-indigo-400 focus:ring-indigo-200 sm:w-80" autocomplete="off">
                <span id="member-search-loading" class="pointer-events-none absolute inset-y-0 right-16 hidden items-center text-xs text-slate-500" role="status">Searching</span>
                <button type="submit" class="absolute inset-y-0 right-0 inline-flex items-center pr-4 text-slate-500 hover:text-indigo-800">Search</button>
            </div>
        </form>
    </div>

    @if($users->isEmpty())
        <p class="text-gray-500">No users found.</p>
    @else
        <div class="rounded-3xl border border-slate-200" style="height: 32rem; overflow: auto; scrollbar-width: auto; scrollbar-color: #fbbf24 #fffbeb;" tabindex="0" aria-label="Scrollable member list">
            <table class="min-w-full divide-y divide-slate-200 bg-white">
                <thead class="sticky top-0 z-10 bg-indigo-50/80">
                    <tr>
                        <th class="px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-gray-500">Name</th>
                        <th class="px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-gray-500">Email</th>
                        <th class="px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-gray-500">Role</th>
                        <th class="px-4 py-3 text-left text-xs font-semibold uppercase tracking-wide text-gray-500">Joined</th>
                        <th class="px-4 py-3 text-right text-xs font-semibold uppercase tracking-wide text-gray-500">Actions</th>
                    </tr>
                </thead>
                <tbody class="divide-y divide-slate-200">
                    @foreach($users as $user)
                        <tr class="hover:bg-indigo-50/50">
                            <td class="px-4 py-4 text-sm text-gray-700">
                                <div class="flex min-w-0 items-center gap-3">
                                    <img
                                        src="{{ $user->profilePictureUrl() }}"
```

```
                                        onerror="this.onerror=null;this.src='{{ asset('images/guest-avatar.svg') }}';"
                                        alt=""
                                        aria-hidden="true"
                                        class="h-9 w-9 shrink-0 rounded-full border border-slate-200 object-cover"
                                    >
                                    <span class="truncate">{{ $user->name }}</span>
                                </div>
                            </td>
                            <td class="px-4 py-4 text-sm text-gray-700">{{ $user->email }}</td>
                            <td class="px-4 py-4 text-sm text-gray-700">
                                @if($user->isSuperAdmin())
                                    <span class="inline-flex items-center rounded-full bg-indigo-800 px-3 py-1 text-xs font-semibold uppercase tracking-wide
text-white">Admin</span>
                                @elseif($user->is_admin)
                                    <span class="inline-flex items-center rounded-full bg-green-100 px-3 py-1 text-xs font-semibold uppercase tracking-wide
text-green-800">Staff</span>
                                @else
                                    <span class="inline-flex items-center rounded-full bg-gray-100 px-3 py-1 text-xs font-semibold uppercase tracking-wide
text-gray-700">Member</span>
                                @endif
                            </td>
                            <td class="px-4 py-4 text-sm text-gray-500">{{ $user->created_at->format('M d, Y') }}</td>
                            <td class="min-w-[210px] whitespace-nowrap px-4 py-4 text-right text-sm font-medium">
                                @if(auth()->user()->isSuperAdmin())
                                    @if(! $user->isSuperAdmin())
                                        <form method="POST" action="{{ route('admin.users.role', $user) }}" class="inline-flex shrink-0 items-center gap-2 whitespace-nowrap
align-middle">
                                            @csrf
                                            <input type="hidden" name="return_search" value="{{ $search ?? '' }}">
                                            <input type="hidden" name="return_role" value="{{ $role ?? 'all' }}">
                                            <input type="hidden" name="return_member_sort" value="{{ $memberSort ?? 'joined_newest' }}">
                                            <select name="role" class="w-24 shrink-0 rounded-full border border-slate-300 bg-white px-3 py-2 text-sm text-slate-700
focus:outline-none focus:ring-2 focus:ring-indigo-200">
                                                <option value="user" {{ ! $user->is_admin ? 'selected' : '' }}>Member</option>
                                                <option value="admin" {{ $user->is_admin ? 'selected' : '' }}>Staff</option>
                                            </select>
                                            <button type="submit" class="shrink-0 whitespace-nowrap rounded-full bg-indigo-700 px-3 py-2 text-xs text-white transition
hover:bg-indigo-800">Save</button>
                                        </form>
                                        <form method="POST" action="{{ route('admin.users.destroy', $user) }}" class="inline-flex shrink-0 items-center whitespace-nowrap
align-middle" data-confirm="Delete this user? This cannot be undone.">
                                            @csrf
                                            @method('DELETE')
                                            <input type="hidden" name="return_search" value="{{ $search ?? '' }}">
                                            <input type="hidden" name="return_role" value="{{ $role ?? 'all' }}">
                                            <input type="hidden" name="return_member_sort" value="{{ $memberSort ?? 'joined_newest' }}">
                                            <button type="submit" class="shrink-0 whitespace-nowrap rounded-full bg-red-600 px-3 py-2 text-xs text-white transition
hover:bg-red-700">Delete</button>
                                        </form>
                                    @else
                                        <span class="text-xs text-gray-500">Protected</span>
                                    @endif
                                @else
                                    <span class="text-xs text-gray-500">Admin only</span>
                                @endif
                            </td>
                        </tr>
                    @endforeach
                </tbody>
            </table>
        </div>

        <div class="mt-4">
            {{ $users->links('vendor.pagination.members') }}
        </div>
    @endif
</section>
```

resources/views/admin/partials/password-confirmation.blade.php

```
<div id="admin-password-confirmation" class="fixed inset-0 z-[100] hidden items-center justify-center bg-slate-950/60 p-4 backdrop-blur-sm" style="-webkit-backdrop-filter:
blur(8px); backdrop-filter: blur(8px);" role="dialog" aria-modal="true" aria-labelledby="admin-password-confirmation-title"
aria-describedby="admin-password-confirmation-message">
    <div class="w-full max-w-md rounded-3xl border border-slate-200 bg-white p-6 shadow-2xl">
```

```
        <h2 id="admin-password-confirmation-title" class="text-xl font-semibold text-slate-900">Confirm admin action</h2>
        <p id="admin-password-confirmation-message" class="mt-2 text-sm text-slate-600">Enter your password to continue.</p>
        <label class="mt-5 block">
            <span class="text-sm font-semibold text-slate-800">Current password</span>
            <input id="admin-action-password" type="password" autocomplete="current-password" class="mt-2 w-full rounded-xl border border-slate-300 px-4 py-3 text-sm focus:border-green-700 focus:outline-none focus:ring-2 focus:ring-green-100">
        </label>
        <p id="admin-password-confirmation-error" class="mt-3 hidden text-sm text-rose-700" role="alert"></p>
        <div class="mt-6 flex justify-end gap-3">
            <button id="admin-password-confirmation-cancel" type="button" class="rounded-full border border-slate-300 px-5 py-2.5 text-sm font-semibold text-slate-700 hover:bg-slate-50">Cancel</button>
            <button id="admin-password-confirmation-submit" type="button" class="rounded-full bg-green-800 px-5 py-2.5 text-sm font-semibold text-white hover:bg-green-900">Confirm action</button>
        </div>
    </div>
</div>

@push('scripts')
<script>
    (() => {
        const modal = document.getElementById('admin-password-confirmation');
        const message = document.getElementById('admin-password-confirmation-message');
        const passwordInput = document.getElementById('admin-action-password');
        const error = document.getElementById('admin-password-confirmation-error');
        const cancelButton = document.getElementById('admin-password-confirmation-cancel');
        const confirmButton = document.getElementById('admin-password-confirmation-submit');
        let pendingForm = null;
        let pendingSubmitter = null;

        function closeModal() {
            modal.classList.add('hidden');
            modal.classList.remove('flex');
            passwordInput.value = '';
            error.classList.add('hidden');
            pendingForm = null;
            pendingSubmitter = null;
        }

        document.addEventListener('submit', function(event) {
            const form = event.target;
            if (!(form instanceof HTMLFormElement)) return;

            const action = new URL(form.action, window.location.origin);
            if (action.origin !== window.location.origin
                || !action.pathname.startsWith('/admin/')
                || form.method.toUpperCase() !== 'POST'
                || form.dataset.adminPasswordConfirmed === 'true') {
                return;
            }

            event.preventDefault();
            event.stopImmediatePropagation();
            pendingForm = form;
            pendingSubmitter = event.submitter;
            const actionLabel = pendingSubmitter?.textContent?.trim() || 'Continue';
            message.textContent = form.dataset.confirm
                || form.dataset.confirmMessage
                || `${actionLabel}: enter your password to authorize this admin action.`;
            modal.classList.remove('hidden');
            modal.classList.add('flex');
            passwordInput.focus();
        }, true);

        cancelButton.addEventListener('click', closeModal);
        modal.addEventListener('click', function(event) {
            if (event.target === modal) closeModal();
        });
        document.addEventListener('keydown', function(event) {
            if (event.key === 'Escape' && !modal.classList.contains('hidden')) closeModal();
        });

        confirmButton.addEventListener('click', function() {
            if (!pendingForm) return;
            if (!passwordInput.value) {
```

```
                error.textContent = 'Enter your current password to continue.';
                error.classList.remove('hidden');
                passwordInput.focus();
                return;
            }

            const form = pendingForm;
            const submitter = pendingSubmitter;
            let passwordField = form.querySelector('input[name="current_password"]');
            if (!passwordField) {
                passwordField = document.createElement('input');
                passwordField.type = 'hidden';
                passwordField.name = 'current_password';
                form.appendChild(passwordField);
            }
            passwordField.value = passwordInput.value;
            form.dataset.adminPasswordConfirmed = 'true';
            closeModal();

            if (submitter instanceof HTMLElement && submitter.form === form) {
                form.requestSubmit(submitter);
            } else {
                form.requestSubmit();
            }
        });

        passwordInput.addEventListener('keydown', function(event) {
            if (event.key === 'Enter') {
                event.preventDefault();
                confirmButton.click();
            }
        });
    })();
</script>
@endpush
```

resources/views/admin/partials/sidebar.blade.php

```
@php
    $isDashboardPage = request()->routeIs('admin.dashboard');
    $dashboardUrl = route('admin.dashboard');
    $activeAdminPage = request()->routeIs('admin.facilities.*')
        ? 'facilities'
        : (request()->routeIs('admin.property-management.*')
            ? 'property'
            : (request()->routeIs('admin.reservation-settings*') ? 'reservation-settings' : 'overview'));
    $baseLinkClasses = 'inline-flex shrink-0 items-center gap-3 rounded-xl px-3 py-2.5 text-sm transition md:w-full';
    $inactiveLinkClasses = 'font-medium text-slate-600 hover:bg-slate-100 hover:text-slate-900';
    $activeLinkClasses = 'bg-green-50 font-semibold text-green-900 hover:bg-green-100';
@endphp

<aside class="z-20 flex w-full shrink-0 flex-col border-b border-slate-200 bg-white px-4 py-4 md:sticky md:top-0 md:h-screen md:w-64 md:border-b-0 md:border-r md:px-5 md:py-6">
    <a href="{{ $dashboardUrl }}" class="flex items-center gap-3 px-2">
        <span class="flex h-10 w-10 items-center justify-center rounded-xl bg-green-800 text-sm font-bold text-white shadow-sm">CR</span>
        <span>
            <span class="block text-sm font-bold tracking-wide text-slate-900">CAMPUS RESERVE</span>
            <span class="mt-0.5 block text-xs text-slate-500">Administration</span>
        </span>
    </a>

    <div class="mt-6 hidden items-center gap-3 rounded-2xl bg-slate-50 p-3 md:flex">
        <img
            src="{{ auth()->user()->profilePictureUrl() }}"
            onerror="this.onerror=null;this.src='{{ asset('images/guest-avatar.svg') }}';"
            alt="{{ auth()->user()->name }} profile picture"
            class="h-10 w-10 shrink-0 rounded-full object-cover"
        >
        <span class="min-w-0">
            <span class="block truncate text-sm font-semibold text-slate-800">{{ auth()->user()->name }}</span>
            <span class="mt-0.5 block text-xs text-slate-500">{{ auth()->user()->isSuperAdmin() ? 'Admin' : 'Staff' }}</span>
        </span>
    </div>

```

```
    <p class="mb-2 mt-7 hidden px-3 text-[11px] font-semibold uppercase tracking-[0.16em] text-slate-400 md:block">Workspace</p>
    <nav aria-label="Admin dashboard navigation" class="mt-5 flex gap-2 overflow-x-auto md:mt-0 md:flex-col md:overflow-visible">
        <a href="{{ $isDashboardPage ? '#overview' : $dashboardUrl . '#overview' }}"
            @if($isDashboardPage) data-admin-section-link="overview" @endif
            @if($isDashboardPage && $activeAdminPage === 'overview') aria-current="page" @endif
            class="{{ $baseLinkClasses }} {{ $isDashboardPage && $activeAdminPage === 'overview' ? $activeLinkClasses : $inactiveLinkClasses }}">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true"><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="1.8" d="M3 10.5 12 3l9 7.5v9a1.5 1.5 0 0 1-1.5 1.5h-15A1.5 1.5 0 0 1 3 19.5v-9ZM9 21v-7.5h6V21"/></svg>
            Overview
        </a>
        <a href="{{ $isDashboardPage ? '#reservations' : $dashboardUrl . '#reservations' }}"
            @if($isDashboardPage) data-admin-section-link="reservations" @endif
            class="{{ $baseLinkClasses }} {{ $isDashboardPage && request()->get('section') === 'reservations' ? $activeLinkClasses : $inactiveLinkClasses }}">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true"><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="1.8" d="M8 2v4m8-4v4M4 9h16M5 4h14a1 1 0 0 1 1 1v15H4V5a1 1 0 0 1 1-1Z"/><path stroke-linecap="round" stroke-linejoin="round" stroke-width="1.8" d="m9 14 2 2
4-4"/></svg>
            Reservations
        </a>
        <a href="{{ $isDashboardPage ? '#members' : $dashboardUrl . '#members' }}"
            @if($isDashboardPage) data-admin-section-link="members" @endif
            class="{{ $baseLinkClasses }} {{ $isDashboardPage && request()->get('section') === 'members' ? $activeLinkClasses : $inactiveLinkClasses }}">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true"><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="1.8" d="M16 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2m16 0v-2a4 4 0 0 0-3-3.87M14 3.13a4 4 0 0 1 0 7.75M10 7a4 4 0 1 1-8 0 4 4 0 0 1 8 0Z"/></svg>
            Members
        </a>
        <a href="{{ $isDashboardPage ? '#audit' : $dashboardUrl . '#audit' }}"
            @if($isDashboardPage) data-admin-section-link="audit" @endif
            class="{{ $baseLinkClasses }} {{ $isDashboardPage && request()->get('section') === 'audit' ? $activeLinkClasses : $inactiveLinkClasses }}">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true"><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="1.8" d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2Z"/><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="1.8" d="M8 7h8m-8 4h8"/></svg>
            Audit activity
        </a>
        <a href="{{ route('admin.facilities.create') }}"
            @if($activeAdminPage === 'facilities') aria-current="page" @endif
            class="{{ $baseLinkClasses }} {{ $activeAdminPage === 'facilities' ? $activeLinkClasses : $inactiveLinkClasses }}">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true"><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="1.8" d="M3 21h18M5 21V5l7-3v19m7 0V9l-7-4M8 8v.01M8 12v.01M8 16v.01m8-4v.01m0 4v.01"/></svg>
            Facilities
        </a>
        <a href="{{ route('admin.property-management.index') }}"
            @if($activeAdminPage === 'property') aria-current="page" @endif
            class="{{ $baseLinkClasses }} {{ $activeAdminPage === 'property' ? $activeLinkClasses : $inactiveLinkClasses }}">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true"><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="1.8" d="M4 7.5 12 3l8 4.5v9L12 21l-8-4.5v-9Z"/><path stroke-linecap="round" stroke-linejoin="round" stroke-width="1.8" d="m4.3 7.7 7.7 4.4 7.7-4.4M12
21v-8.9m-4-6.8 8 4.5"/></svg>
            Property items
        </a>
        <a href="{{ route('admin.reservation-settings') }}"
            @if($activeAdminPage === 'reservation-settings') aria-current="page" @endif
            class="{{ $baseLinkClasses }} {{ $activeAdminPage === 'reservation-settings' ? $activeLinkClasses : $inactiveLinkClasses }}">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true"><circle cx="12" cy="12" r="9" stroke-width="1.8"/><path
stroke-linecap="round" stroke-width="1.8" d="M12 7v5l3 2"/></svg>
            Reservation settings
        </a>
    </nav>

    <div class="mt-auto hidden border-t border-slate-100 pt-5 md:block">
        <a href="/reserve" class="inline-flex w-full items-center gap-3 rounded-xl px-3 py-2.5 text-sm font-medium text-slate-600 transition hover:bg-slate-100
hover:text-slate-900">
            <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true"><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="1.8" d="M14 3h7v7m0-7-9 9m6-2v9a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h9"/></svg>
            Back to user portal
        </a>
    </div>
</aside>
```

resources/views/admin/property-management.blade.php

```
@extends('layouts.app')

@section('title', 'Property Management - Campus Reserve')

@section('content')
```

```
<div class="min-h-screen bg-[#f5f3f7]">
    <div class="flex min-h-screen flex-col md:flex-row">
        @include('admin.partials.sidebar')
        <main class="min-w-0 flex-1 px-3 py-5 sm:px-6 sm:py-8 xl:px-10">
    <div class="mx-auto max-w-[1600px]">
        <div class="mb-8">
            <h1 class="text-3xl font-bold text-gray-900">Property Management</h1>
            <p class="mt-2 text-gray-600">Manage property items and inventory independently of rooms and facilities.</p>
        </div>

        @if(session('status'))
            <div class="mb-6 rounded-xl border border-green-300 bg-green-100 px-4 py-3 text-green-800">{{ session('status') }}</div>
        @endif
        @if($errors->has('current_password'))
            <div class="mb-6 rounded-xl border border-red-300 bg-red-50 px-4 py-3 text-red-800" role="alert">{{ $errors->first('current_password') }}</div>
        @endif

        <section class="mb-8 rounded-3xl border border-gray-200 bg-white p-6 shadow-sm">
            <h2 class="text-xl font-semibold text-gray-900">Add an Item</h2>
            <form method="POST" action="{{ route('admin.property-management.items.store') }}" data-confirm-message="Add this property item?" class="mt-5 grid grid-cols-1
gap-5 md:grid-cols-3">
                @csrf
                <div>
                    <label for="name" class="mb-2 block text-sm font-medium text-gray-700">Item name</label>
                    <input id="name" name="name" type="text" value="{{ old('name') }}" required maxlength="100" placeholder="e.g. Projector" class="w-full rounded-xl border
border-gray-300 px-4 py-3 focus:border-black focus:outline-none focus:ring-1 focus:ring-black">
                    @error('name')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                </div>
                <div>
                    <label for="quantity" class="mb-2 block text-sm font-medium text-gray-700">Quantity</label>
                    <input id="quantity" name="quantity" type="number" value="{{ old('quantity', 1) }}" required min="1" step="1" class="w-full rounded-xl border
border-gray-300 px-4 py-3 focus:border-black focus:outline-none focus:ring-1 focus:ring-black">
                    @error('quantity')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                </div>
                <button type="submit" class="w-fit rounded-xl bg-black px-5 py-3 text-sm font-semibold text-white transition-colors hover:bg-gray-800">Add Item</button>
            </form>
        </section>

        <section class="rounded-3xl border border-gray-200 bg-white p-6 shadow-sm">
            <div class="mb-5">
                <h2 class="text-xl font-semibold text-gray-900">Property Inventory</h2>
                <p class="mt-1 text-sm text-gray-600">Edit item names and quantities, or remove an item.</p>
            </div>

            @if($items->isEmpty())
                <p class="rounded-xl bg-gray-50 px-4 py-6 text-center text-gray-500">No property items have been added yet.</p>
            @else
                <div class="space-y-4">
                    @foreach($items as $item)
                        <div class="rounded-2xl border border-gray-200 bg-gray-50 p-4">
                            <form id="update-item-{{ $item->id }}" method="POST" action="{{ route('admin.property-management.items.update', $item) }}"
data-confirm-message="Save these property item changes?" class="grid grid-cols-1 items-end gap-4 md:grid-cols-4">
                                @csrf
                                @method('PUT')
                                <div>
                                    <label for="item-name-{{ $item->id }}" class="mb-2 block text-sm font-medium text-gray-700">Item name</label>
                                    <input id="item-name-{{ $item->id }}" name="name" type="text" value="{{ $item->name }}" required maxlength="100" class="w-full rounded-xl
border border-gray-300 bg-white px-4 py-3 focus:border-black focus:outline-none focus:ring-1 focus:ring-black">
                                </div>
                                <div>
                                    <label for="item-quantity-{{ $item->id }}" class="mb-2 block text-sm font-medium text-gray-700">Quantity</label>
                                    <input id="item-quantity-{{ $item->id }}" name="quantity" type="number" value="{{ $item->quantity }}" required min="1" step="1"
class="w-full rounded-xl border border-gray-300 bg-white px-4 py-3 focus:border-black focus:outline-none focus:ring-1 focus:ring-black">
                                </div>
                                <div class="flex gap-2">
                                    <button type="submit" class="rounded-full bg-blue-700 px-4 py-2.5 text-sm font-medium text-white hover:bg-blue-800">Save</button>
                                    <button form="delete-item-{{ $item->id }}" type="submit" class="rounded-full border border-red-200 px-4 py-2.5 text-sm font-medium
text-red-700 hover:bg-red-50">Remove</button>
                                </div>
                            </form>
                            <form id="delete-item-{{ $item->id }}" method="POST" action="{{ route('admin.property-management.items.destroy', $item) }}"
data-confirm-message="Remove this property item?">
                                @csrf
                                @method('DELETE')
```

```
                            </form>
                        </div>
                    @endforeach
                </div>
            @endif
        </section>
    </div>
        </main>
    </div>
</div>

@include('admin.partials.password-confirmation')
@endsection
```

resources/views/admin/reservation-settings.blade.php

```
@extends('layouts.app')

@section('title', 'Reservation Settings - Campus Reserve')

@push('styles')
<style>
    #maintenance-edit-dialog,
    #property-maintenance-edit-dialog,
    #password_confirmation_dialog {
        position: fixed;
        inset: 0;
        width: calc(100% - 2rem);
        max-width: 42rem;
        max-height: calc(100dvh - 2rem);
        height: fit-content;
        margin: auto;
        overflow-y: auto;
        background: white;
    }

    #maintenance-edit-dialog:not([open]),
    #property-maintenance-edit-dialog:not([open]),
    #password_confirmation_dialog:not([open]) {
        display: none;
    }

    #maintenance-edit-dialog::backdrop,
    #property-maintenance-edit-dialog::backdrop,
    #password_confirmation_dialog::backdrop {
        background: rgb(15 23 42 / 55%);
        -webkit-backdrop-filter: blur(8px);
        backdrop-filter: blur(8px);
    }
</style>
@endpush

@section('content')
<div class="min-h-screen bg-[#f5f3f7]">
    <div class="flex min-h-screen flex-col md:flex-row">
        @include('admin.partials.sidebar')
        <main class="min-w-0 flex-1 px-3 py-5 sm:px-6 sm:py-8 xl:px-10">
            <div class="mx-auto max-w-5xl">
                <div class="mb-8">
                    <h1 class="text-3xl font-bold text-gray-900">Reservation Settings</h1>
                    <p class="mt-2 text-gray-600">Set the daily time window when facility and property reservations are allowed.</p>
                </div>

                @if(session('status'))
                    <div class="mb-6 rounded-xl border border-green-300 bg-green-100 px-4 py-3 text-green-800" role="status">{{ session('status') }}</div>
                @endif
                @if(session('error'))
                    <div class="mb-6 rounded-xl border border-red-300 bg-red-50 px-4 py-3 text-red-800" role="alert">{{ session('error') }}</div>
                @endif
                @unless($settingsStorageAvailable)
                    <div class="mb-6 rounded-xl border border-amber-300 bg-amber-50 px-4 py-3 text-amber-900" role="alert">
                        Saving these settings is not available until the reservation settings database migration has been applied.
                    </div>
                @endunless
```

```
                @if($errors->has('current_password'))
                    <div class="mb-6 rounded-xl border border-red-300 bg-red-50 px-4 py-3 text-red-800" role="alert">{{ $errors->first('current_password') }}</div>
                @endif
                @if($errors->has('facility_id') || $errors->has('facility_item_id') || $errors->has('maintenance_date') || $errors->has('is_all_day') || $errors->has('start_time') || $errors->has('end_time') || $errors->has('reason'))
                    <div class="mb-6 rounded-xl border border-red-300 bg-red-50 px-4 py-3 text-red-800" role="alert">
                        Please check the maintenance details below.
                    </div>
                @endif

                <section class="rounded-3xl border border-gray-200 bg-white p-6 shadow-sm sm:p-8">
                    <div class="mb-6">
                        <h2 class="text-xl font-semibold text-gray-900">Allowed reservation hours</h2>
                        <p class="mt-1 text-sm text-gray-600">Reservations must start at or after opening and finish by closing. This applies to new requests, edits, and approvals.</p>
                    </div>

                    @if($settingsStorageAvailable)
                        <form method="POST" action="{{ route('admin.reservation-settings.update') }}" data-password-confirm class="space-y-6">
                            @csrf
                            @method('PUT')
                            <div class="grid gap-5 sm:grid-cols-2">
                                <div>
                                    <label for="start_time" class="mb-2 block text-sm font-medium text-gray-700">Opening time</label>
                                    <input id="start_time" name="start_time" type="time" value="{{ old('start_time', $reservationHours['start_time']) }}" required class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @error('start_time')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                                <div>
                                    <label for="end_time" class="mb-2 block text-sm font-medium text-gray-700">Closing time</label>
                                    <input id="end_time" name="end_time" type="time" value="{{ old('end_time', $reservationHours['end_time']) }}" required class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @error('end_time')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                            </div>
                            <input type="hidden" name="current_password">
                            <div class="flex justify-end border-t border-gray-100 pt-5">
                                <button type="submit" class="rounded-full bg-green-800 px-6 py-3 text-sm font-semibold text-white transition hover:bg-green-900">Save reservation hours</button>
                            </div>
                        </form>
                    @else
                        <p class="rounded-xl bg-slate-50 p-4 text-sm text-slate-600">The current default reservation window is midnight to 11:59 PM. You can edit it after the database migration is applied.</p>
                    @endif
                </section>

                <section class="mt-6 rounded-3xl border border-gray-200 bg-white p-6 shadow-sm sm:p-8">
                    <div class="mb-6">
                        <h2 class="text-xl font-semibold text-gray-900">Facility maintenance</h2>
                        <p class="mt-1 text-sm text-gray-600">Mark a room unavailable for a day or specific time. Overlapping pending and approved reservations will be rejected and their owners notified.</p>
                    </div>

                    @unless($maintenanceStorageAvailable)
                        <p class="mb-6 rounded-xl border border-amber-300 bg-amber-50 p-4 text-sm text-amber-900" role="alert">
                            Maintenance scheduling is unavailable until its database migration has been applied.
                        </p>
                    @endunless

                    @if($maintenanceStorageAvailable)
                        <form method="POST" action="{{ route('admin.reservation-settings.maintenance.store') }}" data-password-confirm class="space-y-5">
                            @csrf
                            <input type="hidden" name="current_password">
                            <div class="grid gap-5 sm:grid-cols-2">
                                <div>
                                    <label for="maintenance_facility_id" class="mb-2 block text-sm font-medium text-gray-700">Room or facility</label>
                                    <select id="maintenance_facility_id" name="facility_id" required class="w-full rounded-xl border border-gray-300 bg-white px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                        <option value="">Select a facility</option>
                                        @foreach($facilities as $facility)
                                            <option value="{{ $facility->id }}" @selected(old('facility_id') == $facility->id)>{{ $facility->name }}{{ $facility->building ? ' — '.$facility->building : '' }}</option>
                                        @endforeach
```

```
                                    </select>
                                    @error('facility_id')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                                <div>
                                    <label for="maintenance_date" class="mb-2 block text-sm font-medium text-gray-700">Date</label>
                                    <input id="maintenance_date" name="maintenance_date" type="date" min="{{ now()->toDateString() }}" value="{{ old('maintenance_date') }}"
    required class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @error('maintenance_date')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                            </div>
                            <div>
                                <input type="hidden" name="is_all_day" value="0">
                                <label class="inline-flex items-center gap-3 text-sm font-medium text-gray-700">
                                    <input id="maintenance_all_day" name="is_all_day" type="checkbox" value="1" @checked(old('is_all_day')) class="rounded border-gray-300
    text-green-800 focus:ring-green-700">
                                    Unavailable all day
                                </label>
                                @error('is_all_day')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                            </div>
                            <div id="maintenance_time_fields" class="grid gap-5 sm:grid-cols-2">
                                <div>
                                    <label for="maintenance_start_time" class="mb-2 block text-sm font-medium text-gray-700">Unavailable from</label>
                                    <input id="maintenance_start_time" name="start_time" type="time" value="{{ old('start_time', $reservationHours['start_time']) }}" required
     class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @error('start_time')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                                <div>
                                    <label for="maintenance_end_time" class="mb-2 block text-sm font-medium text-gray-700">Unavailable until</label>
                                    <input id="maintenance_end_time" name="end_time" type="time" value="{{ old('end_time', $reservationHours['end_time']) }}" required
    class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @error('end_time')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                            </div>
                            <div>
                                <label for="maintenance_reason" class="mb-2 block text-sm font-medium text-gray-700">Reason</label>
                                <input id="maintenance_reason" name="reason" type="text" value="{{ old('reason') }}" maxlength="255" required placeholder="e.g. Electrical
    maintenance" class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                @error('reason')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                            </div>
                            <div class="flex justify-end border-t border-gray-100 pt-5">
                                <button type="submit" class="rounded-full bg-green-800 px-6 py-3 text-sm font-semibold text-white transition hover:bg-green-900">Block
    availability</button>
                            </div>
                        </form>
                    @endif

                    <div class="mt-8 border-t border-gray-100 pt-6">
                        <h3 class="text-lg font-semibold text-gray-900">Scheduled maintenance</h3>
                        <div class="mt-4 space-y-3">
                            @forelse($maintenanceBlocks as $block)
                                <div class="flex flex-col gap-4 rounded-2xl border border-slate-200 bg-slate-50 p-4 sm:flex-row sm:items-center sm:justify-between">
                                    <div>
                                        <p class="font-semibold text-gray-900">{{ $block->facility->name }} — {{ $block->maintenance_date->format('M j, Y') }}</p>
                                        <p class="mt-1 text-sm text-gray-600">
                                            {{ $block->is_all_day ? 'All day' : date('g:i A', strtotime($block->start_time)).'–'.date('g:i A', strtotime($block->end_time)) }}
                                            · {{ $block->reason }}
                                        </p>
                                    </div>
                                    <div class="flex items-center gap-3">
                                        <button
                                            type="button"
                                            data-edit-maintenance
                                            data-edit-action="{{ route('admin.reservation-settings.maintenance.update', $block) }}"
                                            data-facility-id="{{ $block->facility_id }}"
                                            data-maintenance-date="{{ $block->maintenance_date->toDateString() }}"
                                            data-all-day="{{ $block->is_all_day ? '1' : '0' }}"
                                            data-start-time="{{ $block->start_time ? substr($block->start_time, 0, 5) : '' }}"
                                            data-end-time="{{ $block->end_time ? substr($block->end_time, 0, 5) : '' }}"
                                            data-reason="{{ $block->reason }}"
                                            class="rounded-xl border border-green-200 bg-white px-4 py-2 text-sm font-semibold text-green-800 hover:bg-green-50">
                                            Edit
                                        </button>
                                        <form method="POST" action="{{ route('admin.reservation-settings.maintenance.destroy', $block) }}" class="flex items-center gap-3"
    data-password-confirm>
```

```
                                        @csrf
                                        @method('DELETE')
                                        <input type="hidden" name="current_password">
                                        <button type="submit" class="rounded-xl border border-rose-200 bg-white px-4 py-2 text-sm font-semibold text-rose-700 hover:bg-rose-50">Remove</button>
                                    </form>
                                </div>
                            </div>
                        @empty
                            <p class="rounded-xl bg-slate-50 p-4 text-sm text-slate-600">No upcoming maintenance blocks.</p>
                        @endforelse
                    </div>
                </div>

                <dialog id="maintenance-edit-dialog" class="w-full max-w-2xl rounded-3xl border border-slate-200 p-0 shadow-2xl backdrop:bg-slate-950/60">
                    <form id="maintenance-edit-form" method="POST" action="#" data-password-confirm class="space-y-5 p-6 sm:p-8">
                        @csrf
                        @method('PUT')
                        <input type="hidden" name="current_password">
                        <div class="flex items-start justify-between gap-4">
                            <div>
                                <h3 class="text-xl font-semibold text-slate-900">Edit scheduled maintenance</h3>
                                <p class="mt-1 text-sm text-slate-600">Update the facility, date, time, or reason.</p>
                            </div>
                            <button id="maintenance-edit-close" type="button" class="rounded-full p-2 text-xl leading-none text-slate-500 hover:bg-slate-100" aria-label="Close maintenance editor">&times;</button>
                        </div>
                        <div class="grid gap-5 sm:grid-cols-2">
                            <label class="block text-sm font-medium text-gray-700">
                                Room or facility
                                <select id="maintenance-edit-facility" name="facility_id" required class="mt-2 w-full rounded-xl border border-gray-300 bg-white px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @foreach($facilities as $facility)
                                        <option value="{{ $facility->id }}">{{ $facility->name }}{{ $facility->building ? ' — '.$facility->building : '' }}</option>
                                    @endforeach
                                </select>
                            </label>
                            <label class="block text-sm font-medium text-gray-700">
                                Date
                                <input id="maintenance-edit-date" name="maintenance_date" type="date" min="{{ now()->toDateString() }}" required class="mt-2 w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                            </label>
                        </div>
                        <div>
                            <input type="hidden" name="is_all_day" value="0">
                            <label class="inline-flex items-center gap-3 text-sm font-medium text-gray-700">
                                <input id="maintenance-edit-all-day" name="is_all_day" type="checkbox" value="1" class="rounded border-gray-300 text-green-800 focus:ring-green-700">
                                Unavailable all day
                            </label>
                        </div>
                        <div id="maintenance-edit-time-fields" class="grid gap-5 sm:grid-cols-2">
                            <label class="block text-sm font-medium text-gray-700">
                                Unavailable from
                                <input id="maintenance-edit-start" name="start_time" type="time" required class="mt-2 w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                            </label>
                            <label class="block text-sm font-medium text-gray-700">
                                Unavailable until
                                <input id="maintenance-edit-end" name="end_time" type="time" required class="mt-2 w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                            </label>
                        </div>
                        <label class="block text-sm font-medium text-gray-700">
                            Reason
                            <input id="maintenance-edit-reason" name="reason" type="text" maxlength="255" required class="mt-2 w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                        </label>
                        <div class="flex flex-col-reverse gap-3 border-t border-slate-100 pt-5 sm:flex-row sm:justify-end">
                            <button id="maintenance-edit-cancel" type="button" class="rounded-xl border border-slate-300 px-5 py-3 text-sm font-semibold text-slate-700 hover:bg-slate-50">Cancel</button>
                            <button type="submit" class="rounded-xl bg-green-800 px-6 py-3 text-sm font-semibold text-white hover:bg-green-900">Save changes</button>
                        </div>
                    </form>
```

```
                    </dialog>
                </section>

                <section class="mt-6 rounded-3xl border border-gray-200 bg-white p-6 shadow-sm sm:p-8">
                    <div class="mb-6">
                        <h2 class="text-xl font-semibold text-gray-900">Property item maintenance</h2>
                        <p class="mt-1 text-sm text-gray-600">Schedule an item outage for a day or time. It disappears from Available Property Items only while maintenance is active; future reservations overlapping the block are unavailable.</p>
                    </div>

                    @unless($propertyMaintenanceStorageAvailable)
                        <p class="mb-6 rounded-xl border border-amber-300 bg-amber-50 p-4 text-sm text-amber-900" role="alert">
                            Property item maintenance is unavailable until its database migration has been applied.
                        </p>
                    @endunless

                    @if($propertyMaintenanceStorageAvailable)
                        <form method="POST" action="{{ route('admin.reservation-settings.property-maintenance.store') }}" data-password-confirm class="space-y-5">
                            @csrf
                            <input type="hidden" name="current_password">
                            <div class="grid gap-5 sm:grid-cols-2">
                                <div>
                                    <label for="property_maintenance_item_id" class="mb-2 block text-sm font-medium text-gray-700">Property item</label>
                                    <select id="property_maintenance_item_id" name="facility_item_id" required class="w-full rounded-xl border border-gray-300 bg-white px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                        <option value="">Select an item</option>
                                        @foreach($propertyItems as $item)
                                            <option value="{{ $item->id }}" @selected(old('facility_item_id') == $item->id)>{{ $item->name }}{{ $item->facility ? ' — '.$item->facility->name : '' }}</option>
                                        @endforeach
                                    </select>
                                    @error('facility_item_id')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                                <div>
                                    <label for="property_maintenance_date" class="mb-2 block text-sm font-medium text-gray-700">Date</label>
                                    <input id="property_maintenance_date" name="maintenance_date" type="date" min="{{ now()->toDateString() }}" value="{{ old('maintenance_date') }}" required class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @error('maintenance_date')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                            </div>
                            <div>
                                <input type="hidden" name="is_all_day" value="0">
                                <label class="inline-flex items-center gap-3 text-sm font-medium text-gray-700">
                                    <input id="property_maintenance_all_day" name="is_all_day" type="checkbox" value="1" @checked(old('is_all_day')) class="rounded border-gray-300 text-green-800 focus:ring-green-700">
                                    Unavailable all day
                                </label>
                                @error('is_all_day')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                            </div>
                            <div id="property_maintenance_time_fields" class="grid gap-5 sm:grid-cols-2">
                                <div>
                                    <label for="property_maintenance_start_time" class="mb-2 block text-sm font-medium text-gray-700">Unavailable from</label>
                                    <input id="property_maintenance_start_time" name="start_time" type="time" value="{{ old('start_time', $reservationHours['start_time']) }}" required class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @error('start_time')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                                <div>
                                    <label for="property_maintenance_end_time" class="mb-2 block text-sm font-medium text-gray-700">Unavailable until</label>
                                    <input id="property_maintenance_end_time" name="end_time" type="time" value="{{ old('end_time', $reservationHours['end_time']) }}" required class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                    @error('end_time')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                                </div>
                            </div>
                            <div>
                                <label for="property_maintenance_reason" class="mb-2 block text-sm font-medium text-gray-700">Reason</label>
                                <input id="property_maintenance_reason" name="reason" type="text" value="{{ old('reason') }}" maxlength="255" required placeholder="e.g. Repair or inspection" class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                @error('reason')<p class="mt-1 text-sm text-red-600">{{ $message }}</p>@enderror
                            </div>
                            <div class="flex justify-end border-t border-gray-100 pt-5">
                                <button type="submit" class="rounded-full bg-green-800 px-6 py-3 text-sm font-semibold text-white transition hover:bg-green-900">Schedule item maintenance</button>
                            </div>
                        </form>
```

```
                    @endif

                    <div class="mt-8 border-t border-gray-100 pt-6">
                        <h3 class="text-lg font-semibold text-gray-900">Scheduled item maintenance</h3>
                        <div class="mt-4 space-y-3">
                            @forelse($propertyMaintenanceBlocks as $block)
                                <div class="flex flex-col gap-4 rounded-2xl border border-slate-200 bg-slate-50 p-4 sm:flex-row sm:items-center sm:justify-between">
                                    <div>
                                        <p class="font-semibold text-gray-900">{{ $block->item->name }} — {{ $block->maintenance_date->format('M j, Y') }}</p>
                                        <p class="mt-1 text-sm text-gray-600">
                                            {{ $block->is_all_day ? 'All day' : date('g:i A', strtotime($block->start_time)).'–'.date('g:i A', strtotime($block->end_time)) }}
                                            · {{ $block->reason }}
                                        </p>
                                    </div>
                                    <div class="flex items-center gap-3">
                                        <button
                                            type="button"
                                            data-edit-property-maintenance
                                            data-edit-action="{{ route('admin.reservation-settings.property-maintenance.update', $block) }}"
                                            data-item-id="{{ $block->facility_item_id }}"
                                            data-maintenance-date="{{ $block->maintenance_date->toDateString() }}"
                                            data-all-day="{{ $block->is_all_day ? '1' : '0' }}"
                                            data-start-time="{{ $block->start_time ? substr($block->start_time, 0, 5) : '' }}"
                                            data-end-time="{{ $block->end_time ? substr($block->end_time, 0, 5) : '' }}"
                                            data-reason="{{ $block->reason }}"
                                            class="rounded-xl border border-green-200 bg-white px-4 py-2 text-sm font-semibold text-green-800 hover:bg-green-50">
                                            Edit
                                        </button>
                                        <form method="POST" action="{{ route('admin.reservation-settings.property-maintenance.destroy', $block) }}" class="flex items-center
gap-3" data-password-confirm>
                                            @csrf
                                            @method('DELETE')
                                            <input type="hidden" name="current_password">
                                            <button type="submit" class="rounded-xl border border-rose-200 bg-white px-4 py-2 text-sm font-semibold text-rose-700
hover:bg-rose-50">Remove</button>
                                        </form>
                                    </div>
                                </div>
                            @empty
                                <p class="rounded-xl bg-slate-50 p-4 text-sm text-slate-600">No upcoming property item maintenance blocks.</p>
                            @endforelse
                        </div>
                    </div>

                    <dialog id="property-maintenance-edit-dialog" class="w-full max-w-2xl rounded-3xl border border-slate-200 p-0 shadow-2xl backdrop:bg-slate-950/60">
                        <form id="property-maintenance-edit-form" method="POST" action="#" data-password-confirm class="space-y-5 p-6 sm:p-8">
                            @csrf
                            @method('PUT')
                            <input type="hidden" name="current_password">
                            <div class="flex items-start justify-between gap-4">
                                <div>
                                    <h3 class="text-xl font-semibold text-slate-900">Edit item maintenance</h3>
                                    <p class="mt-1 text-sm text-slate-600">Update the item, date, time, or reason.</p>
                                </div>
                                <button id="property-maintenance-edit-close" type="button" class="rounded-full p-2 text-xl leading-none text-slate-500 hover:bg-slate-100"
aria-label="Close property maintenance editor">&times;</button>
                            </div>
                            <div class="grid gap-5 sm:grid-cols-2">
                                <label class="block text-sm font-medium text-gray-700">
                                    Property item
                                    <select id="property-maintenance-edit-item" name="facility_item_id" required class="mt-2 w-full rounded-xl border border-gray-300 bg-white
 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                        @foreach($propertyItems as $item)
                                            <option value="{{ $item->id }}">{{ $item->name }}{{ $item->facility ? ' — '.$item->facility->name : '' }}</option>
                                        @endforeach
                                    </select>
                                </label>
                                <label class="block text-sm font-medium text-gray-700">
                                    Date
                                    <input id="property-maintenance-edit-date" name="maintenance_date" type="date" min="{{ now()->toDateString() }}" required class="mt-2
w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                </label>
                            </div>
                            <div>
```

```
                                <input type="hidden" name="is_all_day" value="0">
                                <label class="inline-flex items-center gap-3 text-sm font-medium text-gray-700">
                                    <input id="property-maintenance-edit-all-day" name="is_all_day" type="checkbox" value="1" class="rounded border-gray-300 text-green-800 focus:ring-green-700">
                                    Unavailable all day
                                </label>
                            </div>
                            <div id="property-maintenance-edit-time-fields" class="grid gap-5 sm:grid-cols-2">
                                <label class="block text-sm font-medium text-gray-700">
                                    Unavailable from
                                    <input id="property-maintenance-edit-start" name="start_time" type="time" required class="mt-2 w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                </label>
                                <label class="block text-sm font-medium text-gray-700">
                                    Unavailable until
                                    <input id="property-maintenance-edit-end" name="end_time" type="time" required class="mt-2 w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                                </label>
                            </div>
                            <label class="block text-sm font-medium text-gray-700">
                                Reason
                                <input id="property-maintenance-edit-reason" name="reason" type="text" maxlength="255" required class="mt-2 w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
                            </label>
                            <div class="flex flex-col-reverse gap-3 border-t border-slate-100 pt-5 sm:flex-row sm:justify-end">
                                <button id="property-maintenance-edit-cancel" type="button" class="rounded-xl border border-slate-300 px-5 py-3 text-sm font-semibold text-slate-700 hover:bg-slate-50">Cancel</button>
                                <button type="submit" class="rounded-xl bg-green-800 px-6 py-3 text-sm font-semibold text-white hover:bg-green-900">Save changes</button>
                            </div>
                        </form>
                    </dialog>
                </section>
            </div>
        </main>
    </div>
</div>
<dialog id="password_confirmation_dialog" class="fixed inset-0 m-auto w-[calc(100%-2rem)] max-w-md rounded-2xl p-0 shadow-2xl backdrop:bg-slate-950/50">
    <form id="password_confirmation_form" class="space-y-5 p-6">
        <div>
            <h2 class="text-lg font-semibold text-gray-900">Confirm password</h2>
            <p class="mt-1 text-sm text-gray-600">Enter your password to continue with this change.</p>
        </div>
        <div>
            <label for="password_confirmation_input" class="mb-2 block text-sm font-medium text-gray-700">Password</label>
            <input id="password_confirmation_input" type="password" autocomplete="current-password" required class="w-full rounded-xl border border-gray-300 px-4 py-3 focus:border-green-700 focus:outline-none focus:ring-1 focus:ring-green-700">
        </div>
        <div class="flex justify-end gap-3">
            <button id="password_confirmation_cancel" type="button" class="rounded-full border border-gray-300 px-5 py-2.5 text-sm font-semibold text-gray-700 hover:bg-gray-50">Cancel</button>
            <button type="submit" class="rounded-full bg-green-800 px-5 py-2.5 text-sm font-semibold text-white transition hover:bg-green-900">Confirm</button>
        </div>
    </form>
</dialog>
@endsection

@push('scripts')
<script>
    document.addEventListener('DOMContentLoaded', function() {
        var passwordDialog = document.getElementById('password_confirmation_dialog');
        var passwordForm = document.getElementById('password_confirmation_form');
        var passwordInput = document.getElementById('password_confirmation_input');
        var cancelPasswordButton = document.getElementById('password_confirmation_cancel');
        var pendingForm = null;

        document.querySelectorAll('form[data-password-confirm]').forEach(function(form) {
            form.addEventListener('submit', function(event) {
                if (form.dataset.passwordConfirmed === 'true') {
                    delete form.dataset.passwordConfirmed;
                    return;
                }

                event.preventDefault();
                pendingForm = form;
```

```
            passwordInput.value = '';
            passwordDialog.showModal();
            passwordInput.focus();
        });
    });

    passwordForm.addEventListener('submit', function(event) {
        event.preventDefault();
        if (!pendingForm || !passwordForm.reportValidity()) return;

        pendingForm.querySelector('input[name="current_password"]').value = passwordInput.value;
        pendingForm.dataset.passwordConfirmed = 'true';
        passwordDialog.close();
        pendingForm.requestSubmit();
        passwordInput.value = '';
        pendingForm = null;
    });

    cancelPasswordButton.addEventListener('click', function() {
        passwordDialog.close();
    });

    passwordDialog.addEventListener('close', function() {
        passwordInput.value = '';
        pendingForm = null;
    });

    function bindMaintenanceTimeFields(allDayInput, timeFields, startInput, endInput) {
        if (!allDayInput || !timeFields || !startInput || !endInput) return;

        function updateTimeFields() {
            timeFields.classList.toggle('hidden', allDayInput.checked);
            startInput.disabled = allDayInput.checked;
            endInput.disabled = allDayInput.checked;
            startInput.required = !allDayInput.checked;
            endInput.required = !allDayInput.checked;
        }

        allDayInput.addEventListener('change', updateTimeFields);
        updateTimeFields();
    }

    bindMaintenanceTimeFields(
        document.getElementById('maintenance_all_day'),
        document.getElementById('maintenance_time_fields'),
        document.getElementById('maintenance_start_time'),
        document.getElementById('maintenance_end_time')
    );

    var editDialog = document.getElementById('maintenance-edit-dialog');
    var editAllDayInput = document.getElementById('maintenance-edit-all-day');
    bindMaintenanceTimeFields(
        editAllDayInput,
        document.getElementById('maintenance-edit-time-fields'),
        document.getElementById('maintenance-edit-start'),
        document.getElementById('maintenance-edit-end')
    );

    document.querySelectorAll('[data-edit-maintenance]').forEach(function(button) {
        button.addEventListener('click', function() {
            document.getElementById('maintenance-edit-form').action = button.dataset.editAction;
            document.getElementById('maintenance-edit-facility').value = button.dataset.facilityId;
            document.getElementById('maintenance-edit-date').value = button.dataset.maintenanceDate;
            editAllDayInput.checked = button.dataset.allDay === '1';
            document.getElementById('maintenance-edit-start').value = button.dataset.startTime;
            document.getElementById('maintenance-edit-end').value = button.dataset.endTime;
            document.getElementById('maintenance-edit-reason').value = button.dataset.reason;
            editAllDayInput.dispatchEvent(new Event('change'));
            editDialog.showModal();
        });
    });

    document.getElementById('maintenance-edit-close').addEventListener('click', function() {
        editDialog.close();
```

```
        });
        document.getElementById('maintenance-edit-cancel').addEventListener('click', function() {
            editDialog.close();
        });

        bindMaintenanceTimeFields(
            document.getElementById('property_maintenance_all_day'),
            document.getElementById('property_maintenance_time_fields'),
            document.getElementById('property_maintenance_start_time'),
            document.getElementById('property_maintenance_end_time')
        );

        var propertyEditDialog = document.getElementById('property-maintenance-edit-dialog');
        var propertyEditAllDayInput = document.getElementById('property-maintenance-edit-all-day');
        bindMaintenanceTimeFields(
            propertyEditAllDayInput,
            document.getElementById('property-maintenance-edit-time-fields'),
            document.getElementById('property-maintenance-edit-start'),
            document.getElementById('property-maintenance-edit-end')
        );

        document.querySelectorAll('[data-edit-property-maintenance]').forEach(function(button) {
            button.addEventListener('click', function() {
                document.getElementById('property-maintenance-edit-form').action = button.dataset.editAction;
                document.getElementById('property-maintenance-edit-item').value = button.dataset.itemId;
                document.getElementById('property-maintenance-edit-date').value = button.dataset.maintenanceDate;
                propertyEditAllDayInput.checked = button.dataset.allDay === '1';
                document.getElementById('property-maintenance-edit-start').value = button.dataset.startTime;
                document.getElementById('property-maintenance-edit-end').value = button.dataset.endTime;
                document.getElementById('property-maintenance-edit-reason').value = button.dataset.reason;
                propertyEditAllDayInput.dispatchEvent(new Event('change'));
                propertyEditDialog.showModal();
            });
        });

        document.getElementById('property-maintenance-edit-close').addEventListener('click', function() {
            propertyEditDialog.close();
        });
        document.getElementById('property-maintenance-edit-cancel').addEventListener('click', function() {
            propertyEditDialog.close();
        });
    });
</script>
@endpush
```

resources/views/auth/2fa-verify.blade.php

```
@extends('layouts.app')

@section('title', '2FA Verification - Campus Reserve')

@section('content')
<!-- Standalone 2FA verification page for the one-time code validation step. -->
<main class="flex min-h-[calc(100vh-73px)] bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50">
    <div class="flex w-full flex-col items-center justify-center px-5 py-10 sm:px-8 md:w-1/2 md:py-12">
        <div class="w-full max-w-md rounded-[2rem] border border-white/80 bg-white/90 p-6 shadow-xl shadow-indigo-950/10 backdrop-blur sm:p-10">
            <div class="flex justify-center mb-6">
                <div class="flex items-center rounded-xl border border-indigo-200 bg-white px-2 py-1 shadow-sm">
                    <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                    <span class="ml-1 rounded-md bg-indigo-700 px-1 text-xs font-medium text-white">RESERVE</span>
                </div>
            </div>

            <h1 class="mb-8 text-center text-3xl font-bold tracking-tight text-slate-900">
                VERIFY YOUR LOGIN
            </h1>

            <div class="text-center mb-8">
                @if($method === 'authenticator')
                    <p class="mb-2 text-slate-600">
                        @if($provider === 'google')
                            Google sign-in succeeded. Enter the current code from your Campus Reserve authenticator app.
                        @else
                            Enter the current code from your authenticator app.
```

```
                        @endif
                    </p>
                @else
                    <p class="mb-2 text-slate-600">
                        @if($provider === 'google')
                            Google sign-in succeeded. We sent a 6-digit verification code to your email.
                        @else
                            We sent a 6-digit verification code to your email.
                        @endif
                    </p>
                @endif
            </div>

            @if (session('status'))
                <div class="mb-6 rounded-2xl border border-emerald-200 bg-emerald-50 px-4 py-3 text-emerald-800">
                    {{ session('status') }}
                </div>
            @endif

            <p class="mb-6 rounded-xl bg-indigo-50 px-4 py-3 text-center text-sm font-medium text-indigo-700">
                You have {{ $attemptsRemaining }} attempts remaining.
            </p>

            <form method="POST" action="{{ route('2fa.verify') }}" class="space-y-6">
                @csrf

                <div class="space-y-2">
                    <label for="otp" class="block text-sm font-medium text-slate-700">Verification Code</label>
                    <input
                        id="otp"
                        name="otp"
                        type="text"
                        inputmode="numeric"
                        pattern="[0-9]{6}"
                        maxlength="6"
                        autocomplete="one-time-code"
                        required
                        class="w-full rounded-2xl border-2 border-indigo-100 bg-indigo-50 px-4 py-4 text-center text-2xl font-bold tracking-[0.4em] text-indigo-950 transition focus:border-indigo-500 focus:bg-white focus:outline-none focus:ring-4 focus:ring-indigo-100"
                        placeholder="000000"
                    >
                    @error('otp')
                        <p class="mt-1 text-xs text-rose-600">{{ $message }}</p>
                    @enderror
                </div>

                <div class="flex flex-col sm:flex-row gap-3">
                    <button
                        type="submit"
                        class="flex-1 rounded-xl bg-gradient-to-r from-indigo-700 via-violet-700 to-amber-400 px-8 py-3 font-semibold text-white shadow-lg shadow-indigo-900/15 transition hover:-translate-y-0.5 hover:from-indigo-800 hover:via-violet-800 hover:to-amber-500"
                    >
                        Verify
                    </button>
                    <button
                        type="button"
                        onclick="document.querySelector('input[name=otp]').value = ''"
                        class="flex-1 rounded-xl border border-indigo-100 bg-indigo-50 px-8 py-3 font-semibold text-indigo-900 transition hover:bg-indigo-100"
                    >
                        Clear
                    </button>
                </div>
            </form>

            <div class="mt-6 text-center">
                @if($method !== 'authenticator')
                    <form method="POST" action="{{ route('2fa.resend') }}" class="inline">
                        @csrf
                        <button type="submit" class="text-sm font-semibold text-indigo-700 hover:text-violet-700 hover:underline">
                            Resend Code
                        </button>
                    </form>
                    <p class="mt-2 text-xs text-slate-500">
                        Didn't receive it? Check your spam folder.
```

```
                    </p>
                @endif
                <p class="mt-4">
                    <a href="/login" class="text-sm font-medium text-indigo-700 hover:text-violet-700 hover:underline">Back to Login</a>
                </p>
            </div>
        </div>
    </div>

    <div aria-hidden="true" class="relative hidden min-h-[calc(100vh-73px)] w-1/2 overflow-hidden bg-gradient-to-br from-indigo-700 via-violet-600 to-amber-400 md:block">
        <div class="absolute -right-32 -top-28 h-[32rem] w-[32rem] rounded-full bg-pink-300/35 blur-3xl"></div>
        <div class="absolute -bottom-40 -left-32 h-[34rem] w-[34rem] rounded-full bg-cyan-200/35 blur-3xl"></div>
        <div class="absolute inset-0 opacity-20" style="background-image: radial-gradient(#fff 1px, transparent 1px); background-size: 28px 28px;"></div>
        <div class="absolute inset-x-12 top-1/2 h-px -translate-y-1/2 bg-gradient-to-r from-transparent via-white/60 to-transparent"></div>
        <div class="absolute left-1/4 top-1/3 h-24 w-24 rotate-12 rounded-[2rem] border border-white/30 bg-white/10 backdrop-blur-sm"></div>
        <div class="absolute bottom-1/3 right-1/4 h-36 w-36 rounded-full border border-white/30 bg-white/10 backdrop-blur-sm"></div>
    </div>
</main>

@endsection
```

resources/views/auth/admin-login.blade.php

```
@extends('layouts.app')

@section('title', 'Admin Login - Campus Reserve')

@section('content')
<!-- Hidden admin login page with optional 2FA modal flow for admin authentication. -->
<main class="flex min-h-[calc(100vh-73px)] items-stretch">
    <div class="w-full md:w-1/2 flex flex-col items-center justify-center px-8 py-12">
        <div class="w-full max-w-md">
            <div class="flex justify-center mb-6">
                <div class="flex items-center border border-black px-2 py-1">
                    <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                    <span class="bg-black text-white text-xs font-medium px-1 ml-1">ADMIN</span>
                </div>
            </div>

            <h1 class="text-3xl font-bold text-center text-gray-900 mb-8">Admin Portal</h1>

            @if (session('error'))
                <div class="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded mb-6">
                    {{ session('error') }}
                </div>
            @endif

            @if (!session('show_2fa_modal'))
            <form method="POST" action="{{ route('admin.login') }}" class="space-y-6" id="adminLoginForm">
                @csrf
                <div class="space-y-2">
                    <label for="email" class="block text-sm text-gray-700">Email</label>
                    <input id="email" name="email" type="email" required class="w-full px-4 py-2 rounded-full border border-gray-300 focus:outline-none focus:ring-2
focus:ring-black focus:border-transparent" value="{{ old('email') }}">
                    @error('email')
                        <p class="text-red-500 text-xs mt-1">{{ $message }}</p>
                    @enderror
                </div>

                <div class="space-y-2">
                    <label for="password" class="block text-sm text-gray-700">Password</label>
                    <input id="password" name="password" type="password" required class="w-full px-4 py-2 rounded-full border border-gray-300 focus:outline-none focus:ring-2
focus:ring-black focus:border-transparent">
                    @error('password')
                        <p class="text-red-500 text-xs mt-1">{{ $message }}</p>
                    @enderror
                </div>

                <div class="space-y-3">
                    <label class="flex items-center">
                        <input type="checkbox" name="remember_device" class="rounded border-gray-300 text-black focus:ring-black">
                        <span class="ml-2 text-sm text-gray-600">Remember this device for 30 days</span>
                    </label>
                </div>
```

```
50
51                  <div class="flex justify-center pt-4">
52                      <button type="submit" class="px-8 py-2 bg-black text-white rounded-md hover:bg-gray-800 transition-colors w-full">Admin Sign In</button>
53                  </div>
54              </form>
55              @endif
56
57              @if (session('show_2fa_modal') && session('pending_2fa_user_id'))
58                  <div id="2faModal" class="fixed inset-0 bg-gray-900 bg-opacity-75 backdrop-blur-sm flex items-center justify-center z-50 p-4 animate-in fade-in zoom-in
    duration-200">
59                      <div class="bg-white/95 backdrop-blur-xl rounded-3xl shadow-2xl p-8 max-w-sm w-full border border-gray-200 animate-in slide-in-from-bottom-4
    duration-300">
60                          <div class="flex justify-between items-center mb-6">
61                              <h2 class="text-2xl font-bold text-gray-900">Verify Your Login</h2>
62                              <button onclick="close2faModal()" class="text-gray-500 hover:text-gray-900 text-xl p-1 rounded-full hover:bg-gray-200
    transition-all">&amp;times;</button>
63                          </div>
64                          <p class="text-gray-700 mb-2 text-lg font-medium">An OTP has been generated for your account.</p>
65                          <p class="text-sm text-gray-500 mb-2">Enter the 6-digit code from the server log.</p>
66                          <p class="text-xs text-gray-500 mb-4">This code expires in 30 seconds and is logged for development.</p>
67                          <div class="text-center mb-8">
68                              <div class="text-lg font-mono bg-gray-100 rounded-lg p-4 mb-2">{{ session('pending_2fa_email') }}</div>
69                              <p class="text-xs text-gray-500">Time remaining: <span id="otpTimer">30s</span></p>
70                          </div>
71                          <form method="POST" action="{{ route('2fa.verify') }}" class="space-y-6">
72                              @csrf
73                              <div>
74                                  <input id="verify_otp" name="otp" type="text" inputmode="numeric" pattern="[0-9]{6}" maxlength="6" autocomplete="one-time-code" required
    class="w-full px-6 py-4 rounded-2xl border-2 border-gray-200 focus:border-black focus:outline-none text-center text-2xl font-bold tracking-widest uppercase bg-gray-50
    hover:bg-white transition-all" placeholder="000000">
75                                  @error('otp')
76                                      <p class="text-red-500 text-xs mt-2 text-center">{{ $message }}</p>
77                                  @enderror
78                              </div>
79                              <div class="flex gap-3 pt-2">
80                                  <button type="button" onclick="clearOtp()" class="flex-1 px-6 py-3 bg-gray-100 text-gray-700 rounded-xl hover:bg-gray-200 transition-all
    font-medium border border-gray-200">Clear</button>
81                                  <button type="submit" class="flex-1 px-6 py-3 bg-black text-white rounded-xl hover:bg-gray-900 transition-all font-bold shadow-lg
    hover:shadow-xl">Verify</button>
82                              </div>
83                          </form>
84                          <div class="flex items-center justify-center gap-2 mt-6 pt-6 border-t border-gray-200">
85                              <form id="adminResendOtpForm" method="POST" action="{{ route('2fa.resend') }}" class="inline-flex items-center">
86                                  @csrf
87                                  <button id="adminResendOtpButton" type="submit" class="text-sm text-blue-600 hover:text-blue-700 font-medium hover:underline
    transition-colors">Resend Code</button>
88                              </form>
89                              <span class="text-xs text-gray-500">| The code is logged for development and expires quickly.</span>
90                          </div>
91                          <div id="adminOtpStatusMessage" class="text-center text-sm text-green-700 mt-3"></div>
92                          </div>
93                          <button onclick="location.href='{{ route('login') }}'" class="w-full mt-6 py-2 px-4 bg-gray-100 text-gray-700 rounded-xl hover:bg-gray-200
    transition-all font-medium text-sm">← Back to User Login</button>
94                      </div>
95                  </div>
96              @endif
97
98              <div class="mt-6 text-center">
99                  <a href="/login" class="text-sm text-gray-600 hover:text-gray-900 hover:underline">Return to normal user login</a>
100             </div>
101         </div>
102     </div>
103
104     <div class="hidden md:block w-1/2 relative overflow-hidden h-full min-h-[calc(100vh-73px)]">
105         <div class="absolute inset-0" style="background: linear-gradient(135deg, #14532d 0%, #15803d 55%, #facc15 100%);"></div>
106     </div>
107 </main>
108
109 <script>
110 var timeLeft = {{ session('time_left', 30) }};
111 var timerInterval = null;
112 const timerEl = document.getElementById('otpTimer');
113
114 function startTimer() {
```

```
    // Clear any existing timer
    if (timerInterval) {
        clearInterval(timerInterval);
    }

    if (timerEl && timeLeft > 0) {
        timerEl.innerHTML = timeLeft + 's';

        timerInterval = setInterval(() => {
            timeLeft--;
            if (timeLeft > 0) {
                timerEl.textContent = timeLeft + 's';
            } else {
                clearInterval(timerInterval);
                timerEl.innerHTML = '<span class="text-red-500">Expired — click Resend Code</span>';
            }
        }, 1000);
    } else if (timerEl && timeLeft <= 0) {
        timerEl.innerHTML = '<span class="text-red-500">Expired — click Resend Code</span>';
    }
}

// Initialize timer on page load
if (timerEl) {
    startTimer();
}

function close2faModal() {
    const modal = document.getElementById('2faModal');
    if (modal) {
        modal.style.display = 'none';
    }
}

function clearOtp() {
    const otp = document.getElementById('verify_otp');
    if (otp) otp.value = '';
}

async function handleAdminResendOtp(event) {
    event.preventDefault();
    const form = document.getElementById('adminResendOtpForm');
    const status = document.getElementById('adminOtpStatusMessage');
    const timerEl = document.getElementById('otpTimer');

    if (! form || ! status || ! timerEl) {
        return;
    }

    status.textContent = 'Resending code...';
    status.classList.remove('text-red-700');
    status.classList.add('text-green-700');

    const token = form.querySelector('input[name="_token"]').value;

    try {
        const response = await fetch(form.action, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'Accept': 'application/json',
                'X-CSRF-TOKEN': token,
            },
            body: JSON.stringify({}),
            credentials: 'same-origin',
        });

        const data = await response.json();
        if (! response.ok) {
            status.classList.remove('text-green-700');
            status.classList.add('text-red-700');
            status.textContent = data.error || 'Unable to resend code. Try again.';
            return;
        }
```

```

        status.classList.remove('text-red-700');
        status.classList.add('text-green-700');
        status.textContent = data.status || 'A new code has been generated and logged.';

        // Reset timer with new time
        timeLeft = data.time_left || 30;
        startTimer();
    } catch (error) {
        status.classList.remove('text-green-700');
        status.classList.add('text-red-700');
        status.textContent = 'Unable to resend code. Try again.';
    }
}

document.addEventListener('DOMContentLoaded', function () {
    const form = document.getElementById('adminResendOtpForm');
    if (form) {
        form.addEventListener('submit', handleAdminResendOtp);
    }
});
</script>
@endsection
```

resources/views/auth/forgot-password.blade.php

```
@extends('layouts.app')

@section('title', 'Forgot Password - Campus Reserve')

@section('content')
<main class="flex min-h-[calc(100vh-73px)] items-center justify-center bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50 px-5 py-12 sm:px-6">
    <div class="w-full max-w-md rounded-[2rem] border border-white/80 bg-white/90 p-6 shadow-xl shadow-indigo-950/10 backdrop-blur sm:p-10">
        <div class="mb-6 flex justify-center">
            <div class="flex items-center rounded-xl border border-indigo-200 bg-white px-2 py-1 shadow-sm">
                <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                <span class="ml-1 rounded-md bg-indigo-700 px-1 text-xs font-medium text-white">RESERVE</span>
            </div>
        </div>
        <h1 class="text-3xl font-bold tracking-tight text-slate-900">Forgot password?</h1>
        <p class="mt-2 text-sm leading-6 text-slate-600">Enter your Campus Reserve email and we will send you a password reset link.</p>

        @if (session('status'))
            <div class="mt-6 rounded-2xl border border-emerald-200 bg-emerald-50 px-4 py-3 text-sm text-emerald-800">{{ session('status') }}</div>
        @endif

        <form method="POST" action="{{ route('password.email') }}" class="mt-8 space-y-5">
            @csrf
            <label class="block">
                <span class="text-sm font-medium text-slate-700">Email</span>
                <input type="email" name="email" value="{{ old('email') }}" required autofocus class="mt-2 w-full rounded-xl border border-slate-300 bg-white px-4 py-3
text-sm text-slate-900 transition focus:border-indigo-500 focus:outline-none focus:ring-4 focus:ring-indigo-100">
                @error('email')<p class="mt-2 text-sm text-rose-600">{{ $message }}</p>@enderror
            </label>
            <button type="submit" class="w-full rounded-xl bg-gradient-to-r from-indigo-700 via-violet-700 to-amber-400 px-8 py-3 text-sm font-semibold text-white shadow-lg
shadow-indigo-900/15 transition hover:-translate-y-0.5 hover:from-indigo-800 hover:via-violet-800 hover:to-amber-500 focus:outline-none focus:ring-4
focus:ring-indigo-200">Send reset link</button>
        </form>

        <p class="mt-6 text-center text-sm text-slate-600"><a href="{{ route('login') }}" class="font-semibold text-indigo-700 hover:text-violet-700 hover:underline">Back to
login</a></p>
    </div>
</main>
@endsection
```

resources/views/auth/login.blade.php

```
@extends('layouts.app')

@section('title', 'Login - Campus Reserve')

@section('content')
<main class="flex min-h-[calc(100vh-73px)] items-stretch bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50">
```

```
    <!-- Left side - Form -->
    <!-- Login form area with email/password input and optional 2FA modal. -->
    <div class="flex w-full flex-col items-center justify-center px-5 py-10 sm:px-8 md:w-1/2 md:py-12">
        <div class="w-full max-w-md rounded-[2rem] border border-white/80 bg-white/90 p-6 shadow-xl shadow-indigo-950/10 backdrop-blur sm:p-10">
            <!-- Logo -->
            <div class="flex justify-center mb-6">
                <div class="flex items-center rounded-xl border border-indigo-200 bg-white px-2 py-1 shadow-sm">
                    <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                    <span class="ml-1 rounded-md bg-indigo-700 px-1 text-xs font-medium text-white">RESERVE</span>
                </div>
            </div>

            <h1 class="mb-8 text-center text-3xl font-bold tracking-tight text-slate-900">
                WELCOME BACK!
            </h1>

            <!-- Status Messages -->
            @if (session('status'))
                <div class="mb-6 rounded-2xl border border-emerald-200 bg-emerald-50 px-4 py-3 text-emerald-800">
                    {{ session('status') }}
                </div>
            @endif

            @if (session('error'))
                <div class="mb-6 rounded-2xl border border-rose-200 bg-rose-50 px-4 py-3 text-rose-700">
                    {{ session('error') }}
                </div>
            @endif

            <!-- Login Form -->
            <form method="POST" action="{{ route('login') }}" class="space-y-6" id="loginForm">
                @csrf
                <div id="loginFormError" class="hidden rounded-2xl border border-rose-200 bg-rose-50 px-4 py-3 text-sm text-rose-700">
                    <p id="loginFormErrorMessage"></p>
                    <button id="refreshLoginPage" type="button" class="mt-2 hidden font-semibold underline hover:text-rose-900">Refresh login page</button>
                </div>

                <div class="space-y-2">
                    <label for="email" class="block text-sm font-medium text-slate-700">Email</label>
                    <input
                        id="email"
                        name="email"
                        type="email"
                        required
                        class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-slate-900 transition placeholder:text-slate-400 focus:border-indigo-500
focus:outline-none focus:ring-4 focus:ring-indigo-100"
                        value="{{ old('email') }}"
                    >
                    @error('email')
                        <p class="text-red-500 text-xs mt-1">{{ $message }}</p>
                    @enderror
                </div>

                <div class="space-y-2">
                    <label for="password" class="block text-sm font-medium text-slate-700">Password</label>
                    <input
                        id="password"
                        name="password"
                        type="password"
                        required
                        class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-slate-900 transition placeholder:text-slate-400 focus:border-indigo-500
focus:outline-none focus:ring-4 focus:ring-indigo-100"
                        >
                    @error('password')
                        <p class="text-red-500 text-xs mt-1">{{ $message }}</p>
                    @enderror
                    <div class="text-right">
                        <a href="{{ route('password.request') }}" class="text-sm font-medium text-indigo-700 transition hover:text-violet-700 hover:underline">Forgot
password?</a>
                    </div>
                </div>

                <!-- Device Remember & Submit -->
                <div class="space-y-3">
```

```
                    <label class="flex items-center">
                        <input type="checkbox" name="remember_device" class="rounded border-slate-300 text-indigo-700 focus:ring-indigo-500">
                        <span class="ml-2 text-sm text-slate-600">Remember this device for 30 days</span>
                    </label>
                </div>

                <div class="flex justify-center pt-4">
                    <button
                        type="submit"
                        class="w-full rounded-xl bg-gradient-to-r from-indigo-700 via-violet-700 to-amber-400 px-8 py-3 font-semibold text-white shadow-lg
shadow-indigo-900/15 transition hover:-translate-y-0.5 hover:from-indigo-800 hover:via-violet-800 hover:to-amber-500 focus:outline-none focus:ring-4 focus:ring-indigo-200"
                    >
                        Log In
                    </button>
                </div>
            </form>

            <!-- 2FA Modal -->
<div id="2faModal" class="fixed inset-0 {{ session('show_2fa_modal') && session('pending_2fa_user_id') ? 'flex' : 'hidden' }} items-center justify-center z-50 p-4 animate-in
fade-in zoom-in duration-200" style="background: rgba(0, 0, 0, 0.18);">
                <div class="w-full max-w-sm rounded-3xl border border-indigo-100 bg-white/95 p-8 shadow-2xl shadow-indigo-950/20 animate-in slide-in-from-bottom-4
duration-300">
                    <div class="flex justify-between items-center mb-6">
                        <h2 class="text-2xl font-bold text-slate-900">Verify Your Login</h2>
                        <button onclick="close2faModal()" class="rounded-full p-1 text-xl text-slate-500 transition-all hover:bg-indigo-50
hover:text-indigo-900">&times;</button>
                    </div>

                    <div id="two-factor-instructions">
                        @if (session('pending_2fa_method') === 'authenticator')
                            <p class="mb-2 text-lg font-medium text-slate-800">Enter your authenticator code.</p>
                            <p class="mb-4 text-sm text-slate-500">Open Google Authenticator or Microsoft Authenticator and enter the current 6-digit code.</p>
                        @else
                            <p class="mb-2 text-lg font-medium text-slate-800">Enter the code sent to your email.</p>
                            <p class="mb-4 text-sm text-slate-500">This code expires in 30 seconds.</p>
                        @endif
                    </div>

                    <div class="text-center mb-8">
                        <div class="mb-2 rounded-xl border border-indigo-100 bg-indigo-50 p-4 font-mono text-lg text-indigo-950">{{ session('pending_2fa_email') }}</div>
                        <p id="two-factor-attempts-warning" class="text-sm font-medium text-indigo-700">You have {{ session('two_fa_attempts_remaining', 3) }} attempts
remaining.</p>
                    </div>

                    <form id="verify2faForm" method="POST" action="{{ route('2fa.verify') }}" class="space-y-6">
                        @csrf

                        <div>
                            <input
                                id="verify_otp"
                                name="otp"
                                type="text"
                                inputmode="numeric"
                                pattern="[0-9]{6}"
                                maxlength="6"
                                autocomplete="one-time-code"
                                required
                                class="w-full rounded-2xl border-2 border-indigo-100 bg-indigo-50 px-6 py-4 text-center text-2xl font-bold uppercase tracking-widest
transition-all hover:bg-white focus:border-indigo-500 focus:outline-none focus:ring-4 focus:ring-indigo-100"
                                placeholder="000000"
                            >
                            <p id="two-factor-code-error" class="{{ $errors->has('otp') ? '' : 'hidden' }} text-red-500 text-xs mt-2 text-center">{{ $errors->first('otp')
}}</p>
                        </div>

                        <div class="flex gap-3 pt-2">
                            <button
                                type="button"
                                onclick="clearOtp()"
                                class="flex-1 rounded-xl border border-indigo-100 bg-indigo-50 px-6 py-3 font-medium text-indigo-900 transition-all hover:bg-indigo-100"
                            >
                                Clear
                            </button>
                            <button
```

```
                                type="submit"
                                class="flex-1 rounded-xl bg-gradient-to-r from-indigo-700 to-violet-700 px-6 py-3 font-bold text-white shadow-lg shadow-indigo-900/15 transition-all hover:from-indigo-800 hover:to-violet-800 hover:shadow-xl"
                            >
                                Confirm
                            </button>
                        </div>
                    </form>

                    <div class="mt-6 flex items-center justify-center gap-2 border-t border-indigo-100 pt-6">
                        <div id="two-factor-resend" class="{{ session('pending_2fa_method') === 'authenticator' ? 'hidden' : '' }}">
                            <form id="resendOtpForm" method="POST" action="{{ route('2fa.resend') }}" class="inline-flex items-center">
                                @csrf
                                <button type="submit" class="text-sm font-medium text-indigo-700 transition-colors hover:text-violet-700 hover:underline">
                                    Resend Code
                                </button>
                            </form>
                            <span class="text-xs text-slate-500">| Code expires quickly.</span>
                        </div>
                        <span id="authenticator-refresh-message" class="{{ session('pending_2fa_method') === 'authenticator' ? '' : 'hidden' }} text-xs text-slate-500">Your authenticator code refreshes automatically.</span>
                    </div>
                    <div id="otpStatusMessage" class="mt-3 text-center text-sm text-emerald-700"></div>

                    <button onclick="location.href='{{ route('login') }}'" class="mt-6 w-full rounded-xl border border-indigo-100 bg-indigo-50 px-4 py-2 font-medium text-indigo-900 transition-all hover:bg-indigo-100">
                        Cancel
                    </button>
                </div>
            </div>

            <div class="mt-6 text-center">
                <p class="mb-4 text-sm text-slate-600">Or use below to login to your account</p>

                <div class="flex justify-center gap-4 mb-4">
                    <!-- Google Login -->
                    <a href="{{ route('auth.google') }}" class="flex h-11 w-11 items-center justify-center rounded-2xl border border-indigo-200 bg-white shadow-sm transition hover:-translate-y-0.5 hover:border-indigo-300 hover:bg-indigo-50" title="Login with Google">
                        <svg class="w-5 h-5" viewBox="0 0 24 24">
                            <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
                            <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
                            <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>
                            <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
                        </svg>
                    </a>
                    <!-- Facebook and LinkedIn similar -->
                </div>

                <div id="adminHint" class="hidden mt-6 text-center bg-gray-900 text-white rounded-3xl p-4 border border-gray-800">
                    <p class="text-sm font-semibold">Hidden admin portal available.</p>
                    <a href="{{ route('admin.login') }}" class="inline-flex mt-3 items-center justify-center rounded-full border border-white px-4 py-2 text-sm font-medium text-white hover:bg-white hover:text-black transition-colors">Go to Admin Login</a>
                </div>

                <p class="mx-auto mt-6 inline-block text-sm text-slate-600">
                    Don't have an account?
                    <a href="/signup" class="font-semibold text-indigo-700 hover:text-violet-700 hover:underline">Sign up here</a>
                </p>
            </div>
        </div>
    </div>

    <!-- Right side - Gradient -->
    <div aria-hidden="true" class="relative hidden min-h-[calc(100vh-73px)] w-1/2 overflow-hidden bg-gradient-to-br from-indigo-700 via-violet-600 to-amber-400 md:block">
        <div class="absolute -right-32 -top-28 h-[32rem] w-[32rem] rounded-full bg-pink-300/35 blur-3xl"></div>
        <div class="absolute -bottom-40 -left-32 h-[34rem] w-[34rem] rounded-full bg-cyan-200/35 blur-3xl"></div>
        <div class="absolute inset-0 opacity-20" style="background-image: radial-gradient(#fff 1px, transparent 1px); background-size: 28px 28px;"></div>
        <div class="absolute inset-x-12 top-1/2 h-px -translate-y-1/2 bg-gradient-to-r from-transparent via-white/60 to-transparent"></div>
        <div class="absolute left-1/4 top-1/3 h-24 w-24 rounded-[2rem] border border-white/30 bg-white/10 rotate-12 backdrop-blur-sm"></div>
        <div class="absolute bottom-1/3 right-1/4 h-36 w-36 rounded-full border border-white/30 bg-white/10 backdrop-blur-sm"></div>
    </div>
```

```
</main>

<script>
document.getElementById('loginForm')?.addEventListener('submit', async function(event) {
    event.preventDefault();
    const form = this;
    const submitButton = form.querySelector('button[type="submit"]');
    const error = document.getElementById('loginFormError');
    const errorMessage = document.getElementById('loginFormErrorMessage');
    const refreshButton = document.getElementById('refreshLoginPage');
    submitButton.disabled = true;
    submitButton.textContent = 'Checking...';
    error.classList.add('hidden');
    refreshButton.classList.add('hidden');

    try {
        const response = await fetch(form.action, {
            method: 'POST',
            headers: {
                'Accept': 'application/json',
                'X-Requested-With': 'XMLHttpRequest',
                'X-CSRF-TOKEN': form.querySelector('input[name="_token"]').value,
            },
            body: new FormData(form),
            credentials: 'same-origin',
        });

        const contentType = response.headers.get('content-type') || '';
        const data = contentType.includes('application/json')
            ? await response.json()
            : {};
        if (!response.ok) {
            if (response.status === 419) {
                errorMessage.textContent = 'Your login session expired or changed. Refresh this page and try again.';
                refreshButton.classList.remove('hidden');
                error.classList.remove('hidden');
                return;
            }
            throw new Error(data.message || Object.values(data.errors || {}).flat()[0] || 'The provided credentials do not match our records.');
        }

        if (data.redirect) {
            window.location.href = data.redirect;
            return;
        }

        if (data.two_factor_required) {
            const modal = document.getElementById('2faModal');
            const instructions = document.getElementById('two-factor-instructions');
            const resend = document.getElementById('two-factor-resend');
            const refreshMessage = document.getElementById('authenticator-refresh-message');
            document.getElementById('2faModal').querySelector('.text-lg.font-mono').textContent = data.email;
            instructions.innerHTML = data.method === 'authenticator'
                ? '<p class="mb-2 text-lg font-medium text-slate-800">Enter your authenticator code.</p><p class="mb-4 text-sm text-slate-500">Open Google Authenticator or Microsoft Authenticator and enter the current 6-digit code.</p>'
                : '<p class="mb-2 text-lg font-medium text-slate-800">Enter the code sent to your email.</p><p class="mb-4 text-sm text-slate-500">This code expires in 30 seconds.</p>';
            resend.classList.toggle('hidden', data.method === 'authenticator');
            refreshMessage.classList.toggle('hidden', data.method !== 'authenticator');
            document.getElementById('two-factor-attempts-warning').textContent = 'You have 3 attempts remaining.';
            modal.classList.remove('hidden');
            modal.classList.add('flex');
            document.getElementById('verify_otp').focus();
        }
    } catch (requestError) {
        errorMessage.textContent = requestError.message;
        error.classList.remove('hidden');
    } finally {
        submitButton.disabled = false;
        submitButton.textContent = 'Log In';
    }
});

document.getElementById('refreshLoginPage')?.addEventListener('click', function() {
```

```
    window.location.reload();
});

document.getElementById('verify2faForm')?.addEventListener('submit', async function(event) {
    event.preventDefault();
    const form = this;
    const button = form.querySelector('button[type="submit"]');
    const error = document.getElementById('two-factor-code-error');
    const warning = document.getElementById('two-factor-attempts-warning');
    button.disabled = true;
    button.textContent = 'Checking...';
    error.classList.add('hidden');

    try {
        const response = await fetch(form.action, {
            method: 'POST',
            headers: {
                'Accept': 'application/json',
                'X-CSRF-TOKEN': form.querySelector('input[name="_token"]').value,
            },
            body: new FormData(form),
            credentials: 'same-origin',
        });
        const data = await response.json();

        if (response.status === 429) {
            window.location.href = data.redirect || '{{ route('login') }}';
            return;
        }

        if (!response.ok) {
            error.textContent = data.message || 'Invalid Authenticator Code';
            error.classList.remove('hidden');
            if (data.attempts_remaining) {
                warning.textContent = `You have ${data.attempts_remaining} attempts remaining.`;
            }
            document.getElementById('verify_otp').value = '';
            document.getElementById('verify_otp').focus();
            return;
        }

        if (data.redirect) {
            window.location.href = data.redirect;
        }
    } catch (requestError) {
        error.textContent = 'Unable to verify the code. Please try again.';
        error.classList.remove('hidden');
    } finally {
        button.disabled = false;
        button.textContent = 'Confirm';
    }
});

function close2faModal() {
    document.getElementById('2faModal').style.display = 'none';
}

function clearOtp() {
    document.getElementById('verify_otp').value = '';
}

async function handleResendOtp(event) {
    event.preventDefault();
    const form = document.getElementById('resendOtpForm');
    const status = document.getElementById('otpStatusMessage');

    if (! form || ! status) {
        return;
    }

    status.textContent = 'Resending code...';
    status.classList.remove('text-red-700');
    status.classList.add('text-green-700');

```

```
    const token = form.querySelector('input[name="_token"]').value;

    try {
        const response = await fetch(form.action, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'Accept': 'application/json',
                'X-CSRF-TOKEN': token,
            },
            body: JSON.stringify({}),
            credentials: 'same-origin',
        });

        const data = await response.json();
        if (! response.ok) {
            status.classList.remove('text-green-700');
            status.classList.add('text-red-700');
            status.textContent = data.error || 'Unable to resend code. Try again.';
            return;
        }

        status.classList.remove('text-red-700');
        status.classList.add('text-green-700');
        status.textContent = data.status || 'A new code has been generated and logged.';

    } catch (error) {
        status.classList.remove('text-green-700');
        status.classList.add('text-red-700');
        status.textContent = 'Unable to resend code. Try again.';
    }
}

document.addEventListener('DOMContentLoaded', function () {
    const resendForm = document.getElementById('resendOtpForm');
    if (resendForm) {
        resendForm.addEventListener('submit', handleResendOtp);
    }
});

window.addEventListener('keydown', function (event) {
    if (event.ctrlKey && event.shiftKey && event.key.toLowerCase() === 'a') {
        const adminHint = document.getElementById('adminHint');
        if (adminHint) {
            adminHint.classList.remove('hidden');
            adminHint.classList.add('block');
        }
    }
});
</script>
@endsection
```

resources/views/auth/register.blade.php

```
@extends('layouts.app')

@section('title', 'Sign Up - Campus Reserve')

@section('content')
<!-- User registration form with validation error display and account creation flow. -->
<main class="flex min-h-[calc(100vh-73px)] items-stretch bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50">
    <!-- Left side - Form -->
    <div class="flex w-full flex-col items-center justify-center px-5 py-10 sm:px-8 md:w-1/2 md:py-12">
        <div class="w-full max-w-md rounded-[2rem] border border-white/80 bg-white/90 p-6 shadow-xl shadow-indigo-950/10 backdrop-blur sm:p-10">
            <!-- Logo -->
            <div class="flex justify-center mb-6">
                <div class="flex items-center rounded-xl border border-indigo-200 bg-white px-2 py-1 shadow-sm">
                    <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                    <span class="ml-1 rounded-md bg-indigo-700 px-1 text-xs font-medium text-white">RESERVE</span>
                </div>
            </div>

            <h1 class="mb-8 text-center text-3xl font-bold tracking-tight text-slate-900">
                CREATE ACCOUNT
```

```
            </h1>

            <form method="POST" action="{{ route('register') }}" class="space-y-6">
                @csrf

                @if(!empty($googleSignup))
                    <div class="rounded-2xl border border-indigo-200 bg-gradient-to-r from-indigo-50 via-pink-50 to-cyan-50 px-4 py-3 text-sm text-indigo-900">
                        Finish creating your account with a password of your choice.
                    </div>
                @endif

                <div class="space-y-2">
                    <label for="name" class="block text-sm font-medium text-slate-700">Full Name</label>
                    <input
                        id="name"
                        name="name"
                        type="text"
                        required
                        class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-slate-900 transition focus:border-indigo-500 focus:outline-none focus:ring-4
focus:ring-indigo-100"
                        value="{{ $googleSignup ? $googleName : old('name', '') }}"
                    >
                    @error('name')
                        <p class="text-red-500 text-xs mt-1">{{ $message }}</p>
                    @enderror
                </div>

                <div class="space-y-2">
                    <label for="email" class="block text-sm font-medium text-slate-700">Email</label>
                    <input
                        id="email"
                        name="email"
                        type="email"
                        required
                        @if($googleSignup) readonly @endif
                        class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-slate-900 transition focus:border-indigo-500 focus:outline-none focus:ring-4
focus:ring-indigo-100 {{ $googleSignup ? 'bg-slate-100 text-slate-600' : '' }}"
                        value="{{ $googleSignup ? $googleEmail : old('email', '') }}"
                    >
                    @error('email')
                        <p class="text-red-500 text-xs mt-1">{{ $message }}</p>
                    @enderror
                </div>

                <div class="space-y-2">
                    <label for="password" class="block text-sm font-medium text-slate-700">Password</label>
                    <input
                        id="password"
                        name="password"
                        type="password"
                        required
                        class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-slate-900 transition focus:border-indigo-500 focus:outline-none focus:ring-4
focus:ring-indigo-100"
                    >

                    <!-- Password Strength Meter -->
                    <div class="mt-2">
                        <div class="flex items-center gap-2 mb-1">
                            <span class="text-xs text-gray-600">Strength:</span>
                            <div class="h-2 flex-1 overflow-hidden rounded-full bg-slate-200">
                                <div
                                    id="passwordStrengthBar"
                                    class="h-full w-0 transition-all duration-300 rounded-full"
                                    style="background-color: #ef4444;"
                                ></div>
                            </div>
                            <span id="passwordStrengthText" class="text-xs font-semibold text-red-600 w-16">Weak</span>
                        </div>
                    </div>

                    @error('password')
                        <p class="text-red-500 text-xs mt-1">{{ $message }}</p>
                    @enderror
                </div>
```

```
92
93                  <div class="space-y-2">
94                      <label for="password_confirmation" class="block text-sm font-medium text-slate-700">Confirm Password</label>
95                      <input
96                          id="password_confirmation"
97                          name="password_confirmation"
98                          type="password"
99                          required
100                         class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-slate-900 transition focus:border-indigo-500 focus:outline-none focus:ring-4
    focus:ring-indigo-100"
101                         >
102                         <p class="mb-2 mt-1 text-xs text-slate-500">Use at least 5 characters. Longer passwords are easier to keep secure.</p>
103                 </div>
104
105                 <div class="flex justify-center pt-4">
106                     <button
107                         type="submit"
108                         id="submitBtn"
109                         class="w-full rounded-xl bg-gradient-to-r from-indigo-700 via-violet-700 to-amber-400 px-8 py-3 font-semibold text-white shadow-lg
    shadow-indigo-900/15 transition hover:-translate-y-0.5 hover:from-indigo-800 hover:via-violet-800 hover:to-amber-500 focus:outline-none focus:ring-4 focus:ring-indigo-200"
110                         >
111                         Sign Up
112                     </button>
113                 </div>
114             </form>
115
116             <script>
117                 // Password strength checker
118                 const passwordInput = document.getElementById('password');
119                 const passwordConfirmInput = document.getElementById('password_confirmation');
120                 const submitBtn = document.getElementById('submitBtn');
121                 const strengthBar = document.getElementById('passwordStrengthBar');
122                 const strengthText = document.getElementById('passwordStrengthText');
123
124                 function checkPasswordStrength(password) {
125                     let strength = 0;
126
127                     if (password.length >= 5) strength++;
128                     if (password.length >= 8) strength++;
129                     if (password.length >= 12) strength++;
130                     if (/[a-z]/.test(password)) strength++;
131                     if (/[A-Z]/.test(password)) strength++;
132                     if (/\d/.test(password)) strength++;
133                     if (/[!@#$%^&*()_+]/.test(password)) strength++;
134
135                     return { strength };
136                 }
137
138                 function updatePasswordUI() {
139                     const password = passwordInput.value;
140                     const { strength } = checkPasswordStrength(password);
141
142                     let level = 'Start';
143                     let color = '#d1d5db';
144                     let percentage = 0;
145
146                     if (!password.length) {
147                         level = 'Start';
148                         color = '#d1d5db';
149                         percentage = 0;
150                     } else if (password.length < 5) {
151                         level = 'Too short';
152                         color = '#f59e0b';
153                         percentage = 25;
154                     } else if (password.length >= 5 && strength < 6) {
155                         level = 'Good';
156                         color = '#10b981';
157                         percentage = 70;
158                     } else {
159                         level = 'Strong';
160                         color = '#10b981';
161                         percentage = 100;
162                     }
163
```

```
                    strengthBar.style.backgroundColor = color;
                    strengthBar.style.width = percentage + '%';

                    strengthText.textContent = level;
                    strengthText.style.color = color;

                    const isValidLength = password.length >= 5;
                    submitBtn.disabled = !isValidLength;
                    submitBtn.style.opacity = isValidLength ? '1' : '0.5';
                    submitBtn.style.cursor = isValidLength ? 'pointer' : 'not-allowed';
                }

                // Add event listener
                passwordInput.addEventListener('input', updatePasswordUI);

                // Initial check
                updatePasswordUI();
            </script>

            <div class="mt-6 text-center">
                <p class="text-sm text-gray-600 mb-4">Or continue with Google</p>

                <div class="flex justify-center">
                    <a href="{{ route('auth.google') }}" class="flex h-11 w-11 items-center justify-center rounded-2xl border border-indigo-200 bg-white shadow-sm transition hover:-translate-y-0.5 hover:border-indigo-300 hover:bg-indigo-50" title="Sign up with Google">
                        <svg class="w-5 h-5" viewBox="0 0 24 24">
                            <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
                            <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
                            <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>
                            <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
                        </svg>
                    </a>
                </div>

                <p class="mt-6 text-sm text-slate-600">
                    Already have an account?
                    <a href="/login" class="font-semibold text-indigo-700 hover:text-violet-700 hover:underline">Sign in here</a>
                </p>
            </div>
        </div>
    </div>

    <!-- Right side - Gradient -->
    <div aria-hidden="true" class="relative hidden min-h-[calc(100vh-73px)] w-1/2 overflow-hidden bg-gradient-to-br from-indigo-700 via-violet-600 to-amber-400 md:block">
        <div class="absolute -right-32 -top-28 h-[32rem] w-[32rem] rounded-full bg-pink-300/35 blur-3xl"></div>
        <div class="absolute -bottom-40 -left-32 h-[34rem] w-[34rem] rounded-full bg-cyan-200/35 blur-3xl"></div>
        <div class="absolute inset-0 opacity-20" style="background-image: radial-gradient(#fff 1px, transparent 1px); background-size: 28px 28px;"></div>
        <div class="absolute inset-x-12 top-1/2 h-px -translate-y-1/2 bg-gradient-to-r from-transparent via-white/60 to-transparent"></div>
        <div class="absolute left-1/4 top-1/3 h-24 w-24 rotate-12 rounded-[2rem] border border-white/30 bg-white/10 backdrop-blur-sm"></div>
        <div class="absolute bottom-1/3 right-1/4 h-36 w-36 rounded-full border border-white/30 bg-white/10 backdrop-blur-sm"></div>
    </div>
</main>
@endsection
```

resources/views/auth/reset-password.blade.php

```
@extends('layouts.app')

@section('title', 'Reset Password - Campus Reserve')

@section('content')
<main class="flex min-h-[calc(100vh-73px)] items-center justify-center bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50 px-5 py-12 sm:px-6">
    <div class="w-full max-w-md rounded-[2rem] border border-white/80 bg-white/90 p-6 shadow-xl shadow-indigo-950/10 backdrop-blur sm:p-10">
        <div class="mb-6 flex justify-center">
            <div class="flex items-center rounded-xl border border-indigo-200 bg-white px-2 py-1 shadow-sm">
                <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                <span class="ml-1 rounded-md bg-indigo-700 px-1 text-xs font-medium text-white">RESERVE</span>
            </div>
        </div>
        <h1 class="text-3xl font-bold tracking-tight text-slate-900">Reset password</h1>
        <p class="mt-2 text-sm text-slate-600">Choose a new password for your Campus Reserve account.</p>
```

```

        <form method="POST" action="{{ route('password.update') }}" class="mt-8 space-y-5">
            @csrf
            <input type="hidden" name="token" value="{{ $token }}">
            <label class="block">
                <span class="text-sm font-medium text-slate-700">Email</span>
                <input type="email" name="email" value="{{ old('email', $email) }}" required class="mt-2 w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-sm text-slate-900 transition focus:border-indigo-500 focus:outline-none focus:ring-4 focus:ring-indigo-100">
                @error('email')<p class="mt-2 text-sm text-rose-600">{{ $message }}</p>@enderror
            </label>
            <label class="block">
                <span class="text-sm font-medium text-slate-700">New password</span>
                <input type="password" name="password" required class="mt-2 w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-sm text-slate-900 transition focus:border-indigo-500 focus:outline-none focus:ring-4 focus:ring-indigo-100">
                <p class="mt-2 text-xs leading-5 text-slate-500">Use 5-30 characters for a secure password. Longer passwords are easier to keep safe.</p>
            </label>
            <label class="block">
                <span class="text-sm font-medium text-slate-700">Confirm new password</span>
                <input type="password" name="password_confirmation" required class="mt-2 w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-sm text-slate-900 transition focus:border-indigo-500 focus:outline-none focus:ring-4 focus:ring-indigo-100">
                @error('password')<p class="mt-2 text-sm text-rose-600">{{ $message }}</p>@enderror
            </label>
            <button type="submit" class="w-full rounded-xl bg-gradient-to-r from-indigo-700 via-violet-700 to-amber-400 px-8 py-3 text-sm font-semibold text-white shadow-lg shadow-indigo-900/15 transition hover:-translate-y-0.5 hover:from-indigo-800 hover:via-violet-800 hover:to-amber-500 focus:outline-none focus:ring-4 focus:ring-indigo-200">Reset password</button>
        </form>
    </div>
</main>
@endsection
```

resources/views/calendar-manage.blade.php

```
@extends('layouts.embedded')

@section('title', 'Reserve a Facility')

@section('content')
<div class="min-h-screen bg-[#f3f7f4]">
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <div class="flex flex-col gap-4 md:gap-6 mb-6">
            <div class="flex flex-col sm:flex-row sm:items-end sm:justify-between gap-4">
                <div>
                    <p class="text-sm font-semibold uppercase tracking-[0.16em] text-green-800">Campus Reserve</p>
                    <h1 class="mt-1 text-3xl font-semibold tracking-tight text-slate-950">Reserve a Facility</h1>
                </div>
            </div>

            <div class="calendar-manage-layout grid gap-4 lg:grid-cols-[260px_minmax(0,1fr)]">
                <aside class="calendar-manage-sidebar order-1 space-y-4 lg:order-1">
                    <div class="rounded-3xl border border-slate-200 bg-white p-5 shadow-sm">
                        <div class="flex items-center justify-between mb-4">
                            <h2 class="text-lg font-semibold text-slate-900">Choose a date</h2>
                        </div>
                        <div class="space-y-3">
                            <div class="flex items-center justify-between">
                                <button id="miniCalendarPrev" type="button" aria-label="Previous month" class="rounded-lg px-2 py-1 text-lg leading-none text-slate-500 hover:bg-slate-100">&lsaquo;</button>
                                <h3 id="miniCalendarTitle" class="text-sm font-semibold text-slate-800"></h3>
                                <button id="miniCalendarNext" type="button" aria-label="Next month" class="rounded-lg px-2 py-1 text-lg leading-none text-slate-500 hover:bg-slate-100">&rsaquo;</button>
                            </div>
                            <div class="hidden">
                                <label for="monthDropdown" class="sr-only">Month</label>
                                <select id="monthDropdown">
                                    <option value="0">January</option>
                                    <option value="1">February</option>
                                    <option value="2">March</option>
                                    <option value="3">April</option>
                                    <option value="4">May</option>
                                    <option value="5">June</option>
                                    <option value="6">July</option>
                                    <option value="7">August</option>
                                    <option value="8">September</option>
                                    <option value="9">October</option>
```

```
                                    <option value="10">November</option>
                                    <option value="11">December</option>
                                </select>
                                <label for="yearDropdown" class="sr-only">Year</label>
                                <select id="yearDropdown"></select>
                            </div>
                            <div class="rounded-2xl border border-slate-100 bg-white p-3">
                                <div class="mb-3 grid grid-cols-7 gap-1 text-center text-[10px] font-medium uppercase tracking-wide text-slate-400">
                                    <span>Su</span><span>Mo</span><span>Tu</span><span>We</span><span>Th</span><span>Fr</span><span>Sa</span>
                                </div>
                                <div id="miniCalendarDays" class="grid grid-cols-7 gap-1 text-center"></div>
                            </div>
                            <div class="rounded-2xl border border-slate-100 bg-slate-50 p-4">
                                <h3 class="text-sm font-semibold text-slate-900 mb-3">Ready to reserve</h3>
                                <p class="text-sm text-slate-600 mb-2">Select a date or time on the calendar to start your booking.</p>
                                <p class="mb-4 text-xs font-medium text-green-900">Reservations allowed {{ date('g:i A', strtotime($reservationHours['start_time'])) }}–{{ date('g:i A', strtotime($reservationHours['end_time'])) }}.</p>
                            </div>
                        </div>
                    </div>
                </aside>

                <main class="calendar-manage-main order-2 min-w-0 rounded-3xl border border-slate-200 bg-white p-4 shadow-sm sm:p-6 lg:order-2">
                    <div class="mb-4 flex flex-col gap-2 sm:max-w-sm">
                        <label for="calendarFacilityFilter" class="text-sm font-medium text-slate-700">Show reservations for</label>
                        <select id="calendarFacilityFilter" class="w-full rounded-xl border border-slate-200 bg-white px-4 py-3 text-sm text-slate-700 shadow-sm focus:border-green-700 focus:outline-none focus:ring-2 focus:ring-green-100">
                            <option value="">All facilities</option>
                            @foreach($calendarFacilities as $facility)
                                <option value="{{ $facility->id }}">{{ $facility->name }}</option>
                            @endforeach
                        </select>
                    </div>
                    <div class="mb-5 flex flex-col gap-4 border-b border-slate-100 pb-4 sm:flex-row sm:items-center sm:justify-between">
                        <div class="flex items-center gap-3">
                            <button id="calendarPrevBtn" type="button" aria-label="Previous dates" class="rounded-full border border-slate-200 bg-white px-3 py-2 text-slate-600 transition hover:bg-slate-50">&lsaquo;</button>
                            <h2 id="calendarCurrentTitle" class="min-w-0 text-lg font-semibold text-slate-900">Reservation calendar</h2>
                            <button id="calendarNextBtn" type="button" aria-label="Next dates" class="rounded-full border border-slate-200 bg-white px-3 py-2 text-slate-600 transition hover:bg-slate-50">&rsaquo;</button>
                        </div>
                        <div class="flex flex-wrap items-center gap-2">
                            <div class="hidden rounded-xl border border-slate-200 bg-slate-50 p-1 md:flex">
                                <button data-view="timeGridWeek" class="hidden rounded-lg bg-white px-3 py-1.5 text-sm font-medium text-green-900 shadow-sm md:inline-flex">Week</button>
                                <button data-view="dayGridMonth" class="hidden rounded-lg px-3 py-1.5 text-sm text-slate-600 transition hover:bg-white md:inline-flex">Month</button>
                                <button data-view="listWeek" class="hidden rounded-lg px-3 py-1.5 text-sm text-slate-600 transition hover:bg-white md:inline-flex">List</button>
                            </div>
                        </div>
                    </div>
                    <p class="mb-3 flex items-center gap-2 text-xs text-slate-500">
                        <span class="h-2.5 w-2.5 rounded-sm bg-green-100 ring-1 ring-green-200" aria-hidden="true"></span>
                        Approved reservations show the room and booked time.
                    </p>
                    <div id="calendar" class="rounded-2xl border border-slate-100 bg-white"></div>
                </main>
            </div>
        </div>
    </div>
</div>

<div id="reservationModal" class="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/10 backdrop-blur-sm p-4 opacity-0 pointer-events-none transition-opacity duration-200" aria-hidden="true">
    <div class="max-h-[90vh] w-full max-w-2xl transform overflow-y-auto rounded-3xl border border-slate-200 bg-white p-8 shadow-2xl transition-transform duration-200">
        <h2 class="text-2xl font-semibold text-slate-900 mb-3">Reserve a facility on <span id="modalDate"></span></h2>
        <form id="reservationForm" action="/api/calendar/reserve" method="POST" class="space-y-5">
            @csrf
            <input type="hidden" name="date" id="reservationDate" />
            <div>
                <label for="facilitySelectModal" class="block text-sm font-medium text-slate-700">Facility</label>
                <select id="facilitySelectModal" name="facility" class="mt-2 w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 py-3 text-sm text-slate-700" required></select>
```

```
            </div>
            <div>
                <label for="purpose" class="block text-sm font-medium text-slate-700">Purpose</label>
                <input type="text" id="purpose" name="purpose" placeholder="e.g. club meeting" class="mt-2 w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 py-3 text-sm text-slate-700" required />
            </div>
            <div class="grid gap-4 md:grid-cols-2">
                <div>
                    <label for="startTime" class="block text-sm font-medium text-slate-700">Start time</label>
                    <input type="time" id="startTime" name="startTime" value="{{ $reservationHours['start_time'] }}" min="{{ $reservationHours['start_time'] }}" max="{{ $reservationHours['end_time'] }}" class="mt-2 w-full h-12 rounded-2xl border border-slate-200 bg-slate-50 px-4 py-2.5 text-sm leading-6 text-slate-700" required />
                </div>
                <div>
                    <label for="endTime" class="block text-sm font-medium text-slate-700">End time</label>
                    <input type="time" id="endTime" name="endTime" value="{{ $reservationHours['end_time'] }}" min="{{ $reservationHours['start_time'] }}" max="{{ $reservationHours['end_time'] }}" class="mt-2 w-full h-12 rounded-2xl border border-slate-200 bg-slate-50 px-4 py-2.5 text-sm leading-6 text-slate-700" required />
                </div>
            </div>
            <p class="rounded-2xl bg-slate-50 p-4 text-sm text-slate-600">Property items are reserved separately from facilities on the Reserve Property tab.</p>
            <div class="flex flex-col gap-3 sm:flex-row sm:justify-between sm:items-center">
                <div id="reservationValidationMessage" class="min-h-5 text-sm font-semibold text-rose-600"></div>
                <div class="flex gap-3">
                    <button type="button" id="closeModalBtn" class="rounded-2xl border border-slate-200 bg-white px-5 py-3 text-sm font-semibold text-slate-700 hover:bg-slate-100 transition">Cancel</button>
                    <button type="submit" class="rounded-2xl bg-green-800 px-5 py-3 text-sm font-semibold text-white transition hover:bg-green-900">Reserve</button>
                </div>
            </div>
        </form>
    </div>
</div>

<div id="errorModal" class="fixed inset-0 z-50 hidden items-center justify-center bg-slate-900/10 backdrop-blur-sm p-4 min-h-screen">
    <div class="w-full max-w-md rounded-3xl bg-white p-8 shadow-2xl">
        <div class="text-center">
            <p id="errorLabel" class="text-sm font-semibold text-red-600">Error</p>
            <h2 id="errorTitle" class="mt-3 text-2xl font-semibold text-slate-900">Reservation failed</h2>
            <p id="errorMessage" class="mt-4 whitespace-pre-line text-sm text-slate-600">An error occurred.</p>
        </div>
        <div class="mt-8 flex justify-center gap-3">
            <button id="closeErrorBtn" type="button" class="rounded-2xl bg-green-800 px-5 py-3 text-sm font-semibold text-white transition hover:bg-green-900">Close</button>
        </div>
    </div>
</div>

<div id="successModal" class="fixed inset-0 z-50 hidden items-center justify-center bg-slate-900/10 backdrop-blur-sm p-4 min-h-screen">
    <div class="w-full max-w-md rounded-3xl bg-white p-8 shadow-2xl">
        <div class="text-center">
            <p class="text-sm font-semibold text-emerald-600">Success</p>
            <h2 class="mt-3 text-2xl font-semibold text-slate-900">Reservation is created and now waiting for approval!</h2>
            <p class="mt-4 text-sm text-slate-600">Your booking has been submitted. You can return to the main calendar to review the reservation.</p>
        </div>
        <div class="mt-8 flex justify-center gap-3">
            <button id="redirectToCalendarBtn" type="button" class="rounded-2xl bg-green-800 px-5 py-3 text-sm font-semibold text-white transition hover:bg-green-900">Go to Calendar</button>
            <button id="closeSuccessBtn" type="button" class="rounded-2xl border border-slate-200 bg-white px-5 py-3 text-sm font-semibold text-slate-700 hover:bg-slate-50 transition">Close</button>
        </div>
    </div>
</div>
@endsection

@push('styles')
<style>
    #calendar {
        min-height: 620px;
    }

    @media (max-width: 1023px) {
        #calendar {
            min-height: 480px;
        }

        .calendar-manage-layout {
            grid-template-columns: minmax(0, 1fr);
```

```
        }

    }

    @media (max-width: 767px) {
        #reservationModal > div {
            max-height: calc(100dvh - 2rem);
            padding: 1.25rem;
        }
    }

    @media (max-width: 767px) {
        .calendar-manage-layout {
            gap: 0;
        }

        .calendar-manage-main {
            border-radius: 1.25rem;
            padding: 0.875rem;
        }

        .calendar-manage-main > div:first-child {
            gap: 0.75rem;
            margin-bottom: 0.75rem;
            padding-bottom: 0.75rem;
        }

        .calendar-manage-main > div:first-child > div:first-child {
            gap: 0.5rem;
        }

        #calendar .fc-timegrid-slot {
            height: 2.75rem;
        }
    }

    #calendar {
        --fc-border-color: #e8eeea;
        --fc-page-bg-color: transparent;
        --fc-neutral-bg-color: #f8fafc;
        --fc-today-bg-color: #f0f7f1;
        --fc-button-bg-color: #166534;
        --fc-button-border-color: #166534;
        --fc-button-hover-bg-color: #14532d;
        --fc-button-hover-border-color: #14532d;
        --fc-button-active-bg-color: #052e16;
        --fc-button-active-border-color: #052e16;
        --fc-event-border-color: #166534;
        --fc-event-bg-color: #166534;
        --fc-event-text-color: #fff;
    }

    #calendar .fc-toolbar-title {
        color: #1f2937;
        font-weight: 600;
    }

    #calendar .fc-col-header-cell-cushion,
    #calendar .fc-daygrid-day-number,
    #calendar .fc-timegrid-slot-label-cushion {
        color: #475569;
        text-decoration: none;
    }

    #calendar .fc-timegrid-now-indicator-line {
        border-color: #b45309;
    }

    #calendar .fc-scrollgrid {
        border: 0;
    }

    #calendar .fc-scrollgrid-section > td,
    #calendar .fc-scrollgrid-section > th {
```

```
        border: 0;
    }

    #calendar .fc-scrollgrid-section-header .fc-scroller {
        scrollbar-width: none;
    }

    #calendar .fc-scrollgrid-section-header .fc-scroller::-webkit-scrollbar {
        display: none;
    }

    #calendar .fc-col-header-cell {
        border: 0;
        padding: 0.35rem 0.25rem 0.75rem;
    }

    #calendar .fc-col-header-cell-cushion {
        display: block;
        min-height: 3.5rem;
        border: 1px solid #e6ece7;
        border-radius: 0.9rem;
        background: #f8faf8;
    }

    #calendar .fc-day-today .fc-col-header-cell-cushion,
    #calendar th.fc-day-today .fc-col-header-cell-cushion {
        border-color: #c8ddce;
        background: #edf5ef;
    }

    #calendar .fc-timegrid-col {
        background: #fff;
    }

    #calendar .fc-timegrid-col.fc-day-today {
        background: #f7faf7;
    }

    #calendar .fc-timegrid-slot {
        height: 3.25rem;
    }

    #calendar .fc-v-event {
        border: 0;
        border-left: 3px solid #5d8868;
        border-radius: 0.75rem;
        box-shadow: 0 2px 6px rgb(31 41 55 / 8%);
        padding: 0.15rem;
    }

    #calendar .fc-event-main {
        color: #26372c;
        padding: 0.3rem 0.4rem;
    }

    #calendar .fc-daygrid-event {
        border: 0;
        border-left: 3px solid #5d8868;
        border-radius: 0.6rem;
        padding: 0.15rem 0.25rem;
    }

    #calendar .fc-daygrid-event .fc-event-main {
        padding: 0.15rem 0.25rem;
    }

    #miniCalendarDays button {
        min-width: 0;
        border-radius: 9999px;
        padding: 0.45rem 0;
        font-size: 0.75rem;
        transition: background-color 150ms ease, color 150ms ease;
    }

```

```
    #miniCalendarDays button:hover {
        background: #edf5ef;
    }
</style>
@endpush

@push('scripts')
<script defer src="{{ asset('fullcalendar/index.global.min.js') }}"></script>
<script>
    document.addEventListener('DOMContentLoaded', function() {
        var calendarEl = document.getElementById('calendar');
        var monthDropdown = document.getElementById('monthDropdown');
        var yearDropdown = document.getElementById('yearDropdown');
        var reservationModal = document.getElementById('reservationModal');
        var successModal = document.getElementById('successModal');
        var closeSuccessBtn = document.getElementById('closeSuccessBtn');
        var redirectToCalendarBtn = document.getElementById('redirectToCalendarBtn');
        var errorModal = document.getElementById('errorModal');
        var closeErrorBtn = document.getElementById('closeErrorBtn');
        var errorLabel = document.getElementById('errorLabel');
        var errorTitle = document.getElementById('errorTitle');
        var errorMessage = document.getElementById('errorMessage');
        var modalDate = document.getElementById('modalDate');
        var facilitySelectModal = document.getElementById('facilitySelectModal');
        var selectedDate = null;
        var facilityFilter = document.getElementById('calendarFacilityFilter');
        var reservationOpenTime = @json($reservationHours['start_time']);
        var reservationCloseTime = @json($reservationHours['end_time']);
        var viewButtons = document.querySelectorAll('[data-view]');
        var calendarTitle = document.getElementById('calendarCurrentTitle');
        var previousCalendarButton = document.getElementById('calendarPrevBtn');
        var nextCalendarButton = document.getElementById('calendarNextBtn');
        var miniCalendarDays = document.getElementById('miniCalendarDays');
        var miniCalendarTitle = document.getElementById('miniCalendarTitle');
        var miniCalendarPreviousButton = document.getElementById('miniCalendarPrev');
        var miniCalendarNextButton = document.getElementById('miniCalendarNext');

        function updateActiveViewButton(viewType) {
            viewButtons.forEach(function(button) {
                var isActive = button.getAttribute('data-view') === viewType;
                button.classList.toggle('bg-white', isActive);
                button.classList.toggle('font-medium', isActive);
                button.classList.toggle('text-green-900', isActive);
                button.classList.toggle('shadow-sm', isActive);
                button.classList.toggle('text-slate-600', !isActive);
                button.setAttribute('aria-pressed', String(isActive));
            });
        }

        function toggleModal(show) {
            if (show) {
                reservationModal.classList.remove('opacity-0', 'pointer-events-none');
                reservationModal.classList.add('opacity-100');
            } else {
                reservationModal.classList.add('opacity-0', 'pointer-events-none');
                reservationModal.classList.remove('opacity-100');
            }
        }

        function populateFacilities(selected) {
            fetch('/api/facilities', { credentials: 'include' })
                .then(function(response) {
                    if (!response.ok) {
                        throw new Error('Could not load available facilities.');
                    }
                    return response.json();
                })
                .then(function(facilities) {
                    facilitySelectModal.innerHTML = '';

                    facilities.forEach(function(fac) {
                        var option = document.createElement('option');
                        option.value = fac.id;
                        option.textContent = fac.name;
```

```
                        if (selected && fac.name === selected) option.selected = true;
                        facilitySelectModal.appendChild(option);
                    });
                })
                .catch(function() {
                    toggleErrorModal(true, 'Facilities unavailable', 'Could not load available facilities. Please close and try again.', 'Error');
                });
        }

        var calendar = new window.FullCalendar.Calendar(calendarEl, {
            initialView: 'timeGridWeek',
            headerToolbar: false,
            selectable: true,
            editable: false,
            allDaySlot: false,
            slotMinTime: reservationOpenTime + ':00',
            slotMaxTime: reservationCloseTime + ':00',
            dayHeaderContent: function(info) {
                var weekday = info.date.toLocaleDateString('en-US', { weekday: 'short' });
                var container = document.createElement('div');
                container.className = 'flex flex-col items-center gap-0.5 py-1';

                var weekdayLabel = document.createElement('span');
                weekdayLabel.className = 'text-[10px] font-medium uppercase tracking-wide text-slate-500';
                weekdayLabel.textContent = weekday;

                if (info.view.type === 'dayGridMonth') {
                    return { domNodes: [weekdayLabel] };
                }

                var dateLabel = document.createElement('span');
                dateLabel.className = 'text-sm font-semibold text-slate-800';
                dateLabel.textContent = String(info.date.getDate());

                container.append(weekdayLabel, dateLabel);

                return { domNodes: [container] };
            },
            events: function(info, successCallback, failureCallback) {
                var query = new URLSearchParams({
                    start: info.startStr.slice(0, 10),
                    end: info.endStr.slice(0, 10)
                });
                if (facilityFilter && facilityFilter.value) {
                    query.set('facility_id', facilityFilter.value);
                }

                fetch('/api/calendar/events?' + query, { credentials: 'include' })
                    .then(function(response) {
                        if (!response.ok) {
                            throw new Error('Calendar events could not be loaded.');
                        }
                        return response.json();
                    })
                    .then(successCallback)
                    .catch(function(error) {
                        console.error(error);
                        failureCallback(error);
                    });
            },
            dateClick: function(info) {
                selectedDate = info.dateStr.split('T')[0];
                modalDate.textContent = selectedDate;
                reservationForm.reset();
                if (reservationDateInput) {
                    reservationDateInput.value = selectedDate;
                }
                var clickedTime = info.allDay ? null : (info.dateStr.match(/T(\d{2}:\d{2})/) || [])[1];
                setDefaultReservationTimes(clickedTime);
                resetReservationValidation();
                populateFacilities();
                toggleModal(true);
            }
        });
```

```

        calendar.render();
        window.addEventListener('message', function(event) {
            if (event.origin !== window.location.origin || event.data !== 'facility-calendar:resize') {
                return;
            }
            calendar.updateSize();
        });
        if (facilityFilter) {
            facilityFilter.addEventListener('change', function() {
                calendar.refetchEvents();
            });
        }
        var today = new Date();
        var currentYear = today.getFullYear();

        for (var y = currentYear - 5; y <= currentYear + 5; y++) {
            var opt = document.createElement('option');
            opt.value = y;
            opt.textContent = y;
            if (y === currentYear) opt.selected = true;
            yearDropdown.appendChild(opt);
        }

        // Initialize dropdowns with today's date (not calendar.getDate() which might be week start)
        monthDropdown.value = today.getMonth();
        yearDropdown.value = today.getFullYear();

        function syncDropdowns() {
            // Always sync dropdowns based on calendar's current view date
            var viewDate = calendar.view.currentStart || calendar.getDate();
            if (viewDate) {
                monthDropdown.value = viewDate.getMonth();
                yearDropdown.value = viewDate.getFullYear();
            }
            if (calendarTitle && calendar.view) {
                calendarTitle.textContent = calendar.view.title;
            }
            if (miniCalendarTitle && viewDate) {
                miniCalendarTitle.textContent = viewDate.toLocaleDateString('en-US', {
                    month: 'long',
                    year: 'numeric'
                });
            }
            renderMiniCalendar(viewDate || calendar.getDate());
        }

        function renderMiniCalendar(date) {
            if (!miniCalendarDays) return;

            var firstDay = new Date(date.getFullYear(), date.getMonth(), 1);
            var startDate = new Date(firstDay);
            startDate.setDate(startDate.getDate() - firstDay.getDay());
            var today = new Date();
            today.setHours(0, 0, 0, 0);
            miniCalendarDays.replaceChildren();

            for (var dayIndex = 0; dayIndex < 42; dayIndex++) {
                var dayDate = new Date(startDate);
                dayDate.setDate(startDate.getDate() + dayIndex);

                var button = document.createElement('button');
                button.type = 'button';
                button.textContent = String(dayDate.getDate());
                button.dataset.calendarDate = [
                    dayDate.getFullYear(),
                    String(dayDate.getMonth() + 1).padStart(2, '0'),
                    String(dayDate.getDate()).padStart(2, '0')
                ].join('-');
                button.setAttribute('aria-label', dayDate.toLocaleDateString());
                button.className = 'text-slate-700';

                if (dayDate.toDateString() === today.toDateString()) {
                    button.className = 'bg-green-800 text-white hover:bg-green-900';
```

```
                } else if (dayDate.toDateString() === calendar.getDate().toDateString()) {
                    button.className = 'bg-green-100 font-semibold text-green-900';
                }
                if (dayDate.getMonth() !== date.getMonth()) {
                    button.classList.add('opacity-40');
                }
                miniCalendarDays.appendChild(button);
            }
        }

        function syncCalendarToDropdowns() {
            var year = parseInt(yearDropdown.value, 10);
            var month = parseInt(monthDropdown.value, 10);
            calendar.gotoDate(new Date(year, month, 1));
        }

        calendar.on('datesSet', function() {
            syncDropdowns();
            updateActiveViewButton(calendar.view.type);
        });
        syncDropdowns();
        updateActiveViewButton(calendar.view.type);

        monthDropdown.addEventListener('change', function() {
            syncCalendarToDropdowns();
        });

        yearDropdown.addEventListener('change', function() {
            syncCalendarToDropdowns();
        });
        miniCalendarDays.addEventListener('click', function(event) {
            var dayButton = event.target.closest('button[data-calendar-date]');
            if (dayButton) {
                calendar.gotoDate(dayButton.dataset.calendarDate);
            }
        });
        [miniCalendarPreviousButton, miniCalendarNextButton].forEach(function(button, index) {
            button.addEventListener('click', function() {
                var monthOffset = index === 0 ? -1 : 1;
                calendar.gotoDate(new Date(calendar.getDate().getFullYear(), calendar.getDate().getMonth() + monthOffset, 1));
            });
        });

        if (previousCalendarButton) {
            previousCalendarButton.addEventListener('click', function() {
                calendar.prev();
            });
        }

        if (nextCalendarButton) {
            nextCalendarButton.addEventListener('click', function() {
                calendar.next();
            });
        }

        viewButtons.forEach(function(button) {
            button.addEventListener('click', function() {
                calendar.changeView(button.getAttribute('data-view'));
            });
        });

        function toggleSuccessModal(show) {
            if (!successModal) return;
            if (show) {
                successModal.classList.remove('hidden');
                successModal.classList.add('flex');
            } else {
                successModal.classList.add('hidden');
                successModal.classList.remove('flex');
            }
        }

        function toggleErrorModal(show, title, message, label) {
            if (!errorModal) return;
```

```
            if (show) {
                errorLabel.textContent = label || 'Error';
                if (title) errorTitle.textContent = title;
                if (message) errorMessage.textContent = message;
                errorModal.classList.remove('hidden');
                errorModal.classList.add('flex');
            } else {
                errorModal.classList.add('hidden');
                errorModal.classList.remove('flex');
            }
        }

        if (redirectToCalendarBtn) {
            redirectToCalendarBtn.addEventListener('click', function() {
                window.top.location.href = '/calendar';
            });
        }

        if (closeSuccessBtn) {
            closeSuccessBtn.addEventListener('click', function() {
                toggleSuccessModal(false);
            });
        }

        if (closeErrorBtn) {
            closeErrorBtn.addEventListener('click', function() {
                toggleErrorModal(false);
            });
        }

        var reservationDateInput = document.getElementById('reservationDate');
        var reservationForm = document.getElementById('reservationForm');
        var reservationValidationMessage = document.getElementById('reservationValidationMessage');

        function setDefaultReservationTimes(startTime) {
            var startInput = document.getElementById('startTime');
            var endInput = document.getElementById('endTime');
            var selectedStart = startTime || reservationOpenTime;
            var parts = selectedStart.split(':').map(Number);
            var endHour = (parts[0] + 1) % 24;
            var endTime = String(endHour).padStart(2, '0') + ':' + String(parts[1]).padStart(2, '0');
            if (endTime <= selectedStart || endTime > reservationCloseTime) {
                endTime = reservationCloseTime;
            }
            if (startInput) startInput.value = selectedStart;
            if (endInput) endInput.value = endTime;
        }

        function resetReservationValidation() {
            if (reservationValidationMessage) reservationValidationMessage.textContent = '';
        }

        function setReservationValidationError(message) {
            if (reservationValidationMessage) reservationValidationMessage.textContent = message;
        }

        document.getElementById('closeModalBtn').addEventListener('click', function() {
            resetReservationValidation();
            toggleModal(false);
        });

        reservationForm.addEventListener('submit', function(e) {
            e.preventDefault();
            if (reservationDateInput) {
                reservationDateInput.value = selectedDate;
            }
            var facility = facilitySelectModal.value;
            var purpose = document.getElementById('purpose').value;
            var startTime = document.getElementById('startTime').value;
            var endTime = document.getElementById('endTime').value;
            resetReservationValidation();
            if (startTime && endTime && endTime <= startTime) {
                setReservationValidationError("End time can't be earlier than start");
                return;
```

```
            }
            if (startTime < reservationOpenTime || endTime > reservationCloseTime) {
                setReservationValidationError('Reservations are allowed from ' + reservationOpenTime + ' to ' + reservationCloseTime + '.');
                return;
            }
            var reservationPayload = {
                facility: facility,
                purpose: purpose,
                date: selectedDate,
                start_time: startTime,
                end_time: endTime
            };
            fetch('/api/calendar/reserve', {
                method: 'POST',
                credentials: 'include',
                headers: {
                    'Content-Type': 'application/json',
                    'Accept': 'application/json',
                    'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').getAttribute('content')
                },
                body: JSON.stringify(reservationPayload)
            })
            .then(function(response) {
                return response.text().then(function(text) {
                    var data = {};
                    if (text) {
                        try {
                            data = JSON.parse(text);
                        } catch (parseError) {
                            if (response.ok) {
                                throw parseError;
                            }
                        }
                    }
                    if (!response.ok) {
                        var errorMsg = data.error || data.message || (data.errors ? Object.values(data.errors).flat()[0] : null) || 'Reservation failed';
                        throw { error: errorMsg, status: response.status, code: data.code };
                    }
                    return data;
                });
            })
            .then(function() {
                calendar.refetchEvents();
                toggleModal(false);
                document.getElementById('reservationForm').reset();
                resetReservationValidation();
                setDefaultReservationTimes();
                toggleSuccessModal(true);
            })
            .catch(function(err) {
                var title = 'Reservation failed';
                var message = 'An error occurred while processing your reservation.';
                var label = 'Error';

                if (err && err.code === 'schedule_occupied') {
                    label = 'Sorry!';
                    title = 'This schedule and room is already occupied!';
                    message = 'Please try another time or room!';
                } else if (err && err.error) {
                    if (err.status === 409) {
                        title = 'Reservation unavailable';
                        message = err.error;
                    } else {
                        message = err.error;
                    }
                } else if (err && err.message) {
                    message = err.message;
                }

                toggleErrorModal(true, title, message, label);
            });
        });
    });
</script>
```

```
@endpush
```

resources/views/calendar.blade.php

```
@extends('layouts.app')

@section('title', 'Reserve Calendar - Campus Reserve')

@section('content')
<!-- Calendar page showing reservation events and interaction controls for event details. -->
<div class="min-h-screen bg-[#f3f7f4]">
    <div class="max-w-6xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <div class="flex flex-col gap-4 md:gap-6 mb-6">
            <div class="flex flex-col sm:flex-row sm:items-end sm:justify-between gap-4">
                <div>
                    <p class="text-sm font-semibold uppercase tracking-[0.16em] text-green-800">Campus Reserve</p>
                    <h1 class="mt-1 text-3xl font-semibold tracking-tight text-slate-950">Calendar</h1>
                </div>
                <div class="flex flex-col sm:flex-row sm:items-center gap-3">
                    <div class="relative w-full sm:w-80">
                        <label for="eventSearch" class="sr-only">Search events</label>
                        <input id="eventSearch" type="search" placeholder="Search events, rooms, details..." class="w-full rounded-2xl border border-slate-200 bg-white py-3 px-4 text-sm text-slate-700 shadow-sm focus:border-green-700 focus:outline-none focus:ring-2 focus:ring-green-100" autocomplete="off" />
                        <div id="searchSuggestions" class="absolute left-0 right-0 mt-2 hidden max-h-72 overflow-auto rounded-2xl border border-slate-200 bg-white shadow-lg z-20"></div>
                    </div>
                </div>
            </div>
            <div class="calendar-page-layout grid gap-4 lg:grid-cols-[260px_minmax(0,1fr)]" data-calendar-grid-layout>
                <div class="calendar-sidebar space-y-6">
                    <aside class="calendar-mini rounded-3xl border border-slate-200 bg-white p-5 shadow-sm">
                        <div class="rounded-2xl border border-slate-100 bg-white p-3 text-slate-900">
                            <div class="mb-4 flex items-center justify-between gap-2 text-sm text-slate-600">
                                <button id="miniCalendarPrev" type="button" aria-label="Previous month" class="rounded-lg px-2 py-1 text-lg leading-none text-slate-500 hover:bg-slate-100">&lsaquo;</button>
                                <h3 id="miniCalendarTitle" class="text-sm font-semibold text-slate-800"></h3>
                                <button id="miniCalendarNext" type="button" aria-label="Next month" class="rounded-lg px-2 py-1 text-lg leading-none text-slate-500 hover:bg-slate-100">&rsaquo;</button>
                                <div class="hidden">
                                    <label class="sr-only" for="miniCalendarMonth">Calendar month</label>
                                    <select id="miniCalendarMonth" class="min-w-0 rounded-lg border-0 bg-white py-1 pl-1 pr-5 text-sm font-medium text-slate-800 focus:ring-1 focus:ring-green-700"></select>
                                    <label class="sr-only" for="miniCalendarYear">Calendar year</label>
                                    <select id="miniCalendarYear" class="rounded-lg border-0 bg-white py-1 pl-1 pr-5 text-sm font-medium text-slate-800 focus:ring-1 focus:ring-green-700"></select>
                                </div>
                            </div>
                            <div class="grid grid-cols-7 gap-2 text-center text-[11px] uppercase text-slate-500 mb-3">
                                <span>Su</span>
                                <span>Mo</span>
                                <span>Tu</span>
                                <span>We</span>
                                <span>Th</span>
                                <span>Fr</span>
                                <span>Sa</span>
                            </div>
                            <div id="miniCalendarDays" class="grid grid-cols-7 gap-1.5 text-sm text-slate-700">
                                <!-- Filled by JavaScript -->
                            </div>
                        </div>
                    </aside>
                    <aside class="rounded-3xl border border-slate-200 bg-white p-5 shadow-sm">
                        <div class="flex items-center justify-between mb-4">
                            <div>
                                <p class="text-sm font-medium text-slate-500">Pending</p>
                                <h2 class="text-xl font-semibold text-slate-900">Pending reservations</h2>
                            </div>
                            <span class="rounded-full bg-amber-100 px-3 py-1 text-xs font-semibold text-amber-800">Waiting approval</span>
                        </div>
                        <div id="pendingReservationsList" class="space-y-4">
                            <div class="rounded-2xl bg-slate-50 p-4 text-sm text-slate-600">
                                Pending reservation requests will appear here.
                            </div>
                        </div>
```

```
                    </aside>
                </div>
                <main class="calendar-main min-w-0 rounded-3xl border border-slate-200 bg-white p-4 shadow-sm sm:p-6">
                    <div class="mb-4 flex flex-col gap-2 sm:max-w-sm">
                        <label for="calendarFacilityFilter" class="text-sm font-medium text-slate-700">Show reservations for</label>
                        <select id="calendarFacilityFilter" class="w-full rounded-xl border border-slate-200 bg-white px-4 py-3 text-sm text-slate-700 shadow-sm
focus:border-green-700 focus:outline-none focus:ring-2 focus:ring-green-100">
                            <option value="">All facilities</option>
                            @foreach($calendarFacilities as $facility)
                                <option value="{{ $facility->id }}">{{ $facility->name }}</option>
                            @endforeach
                        </select>
                    </div>
                    <div class="calendar-mobile-date-bar mb-4 hidden items-center justify-between gap-3 rounded-2xl border border-slate-200 bg-slate-50 p-3">
                        <button id="calendarMobilePrevBtn" type="button" aria-label="Previous date" class="rounded-xl border border-slate-200 bg-white px-3 py-2
text-slate-700">&lsaquo;</button>
                        <span id="calendarMobileDateTitle" class="text-sm font-semibold text-slate-900"></span>
                        <button id="calendarMobileNextBtn" type="button" aria-label="Next date" class="rounded-xl border border-slate-200 bg-white px-3 py-2
text-slate-700">&rsaquo;</button>
                    </div>
                    <div class="mb-5 flex flex-col gap-4 border-b border-slate-100 pb-4 sm:flex-row sm:items-center sm:justify-between">
                        <div class="calendar-desktop-date-nav flex items-center gap-3">
                            <button id="calendarPrevBtn" type="button" aria-label="Previous dates" class="rounded-full border border-slate-200 bg-white px-3 py-2
text-slate-600 transition hover:bg-slate-50">&lsaquo;</button>
                            <h2 id="calendarCurrentTitle" class="min-w-0 text-lg font-semibold text-slate-900">Calendar</h2>
                            <button id="calendarNextBtn" type="button" aria-label="Next dates" class="rounded-full border border-slate-200 bg-white px-3 py-2 text-slate-600
transition hover:bg-slate-50">&rsaquo;</button>
                        </div>
                        <div class="calendar-desktop-view-controls flex flex-wrap items-center gap-2">
                            <div class="hidden rounded-xl border border-slate-200 bg-slate-50 p-1 md:flex">
                                <button data-view="timeGridWeek" class="hidden rounded-lg px-3 py-1.5 text-sm text-slate-600 transition hover:bg-white
md:inline-flex">Week</button>
                                <button data-view="dayGridMonth" class="hidden rounded-lg px-3 py-1.5 text-sm text-slate-600 transition hover:bg-white
md:inline-flex">Month</button>
                                <button data-view="listWeek" class="hidden rounded-lg px-3 py-1.5 text-sm text-slate-600 transition hover:bg-white
md:inline-flex">List</button>
                            </div>
                        </div>
                    </div>

                    <p class="mb-3 flex items-center gap-2 text-xs text-slate-500">
                        <span class="h-2.5 w-2.5 rounded-sm bg-green-100 ring-1 ring-green-200" aria-hidden="true"></span>
                        Approved reservations show the room and booked time.
                    </p>
                    <div id="calendar" class="w-full rounded-2xl border border-slate-100 bg-white"></div>
                </main>
            </div>
        </div>
    </div>
</div>

<div id="eventInfoModal" class="fixed inset-0 z-50 hidden items-center justify-center overflow-y-auto bg-slate-900/30 p-3 backdrop-blur-sm sm:p-4">
    <div class="my-auto w-full max-w-xl rounded-3xl border border-slate-200 bg-white p-5 shadow-2xl sm:p-8">
        <div class="flex items-start justify-between gap-4 mb-6">
            <div>
                <p class="text-sm text-slate-500">Reservation details</p>
                <h2 id="eventInfoTitle" class="text-2xl font-semibold text-slate-900 mt-2">Reservation</h2>
            </div>
            <button id="closeEventInfoBtn" type="button" class="rounded-full bg-slate-100 p-3 text-slate-600 hover:bg-slate-200">×</button>
        </div>
        <div class="grid gap-5 sm:grid-cols-2">
            <div class="space-y-2">
                <p class="text-sm font-semibold text-slate-700">Room</p>
                <p id="eventInfoRoom" class="text-slate-900">—</p>
            </div>
            <div class="space-y-2">
                <p class="text-sm font-semibold text-slate-700">Status</p>
                <p id="eventInfoStatus" class="inline-flex rounded-full bg-amber-100 px-3 py-1 text-sm font-semibold text-amber-800">Waiting Approval</p>
            </div>
            <div class="space-y-2">
                <p class="text-sm font-semibold text-slate-700">Date</p>
                <p id="eventInfoDate" class="text-slate-900">—</p>
            </div>
            <div class="space-y-2">
```

```
                <p class="text-sm font-semibold text-slate-700">Time</p>
                <p id="eventInfoTime" class="text-slate-900">—</p>
            </div>
            <div class="sm:col-span-2 space-y-2">
                <p class="text-sm font-semibold text-slate-700">Purpose</p>
                <p id="eventInfoPurpose" class="text-slate-900">—</p>
            </div>
        </div>
        <div class="mt-8 flex flex-wrap items-center gap-3">
            <a id="eventInfoEditBtn" href="#" class="hidden items-center justify-center rounded-full border border-slate-200 bg-slate-100 px-5 py-3 text-sm font-semibold text-slate-700 hover:bg-slate-200 transition">Edit reservation</a>
            <form id="eventInfoDeleteForm" method="POST" action="#" class="hidden" onsubmit="return confirm('Remove this reservation? This cannot be undone.');">
                @csrf
                @method('DELETE')
                <button type="submit" class="inline-flex items-center justify-center rounded-full bg-rose-600 px-5 py-3 text-sm font-semibold text-white hover:bg-rose-700 transition">Remove</button>
            </form>
        </div>
    </div>
</div>
@endsection

@push('styles')
<style>
    #calendar {
        min-height: 640px;
    }

    @media (max-width: 767px) {
        .calendar-page-layout {
            display: flex;
            flex-direction: column;
        }

        .calendar-main {
            order: 1;
        }

        .calendar-sidebar {
            order: 2;
        }

        .calendar-mini {
            display: none;
        }

        #pendingReservationsList {
            max-height: 16rem;
            overflow-y: auto;
        }

        #calendar .fc-toolbar {
            flex-direction: column;
            align-items: stretch;
            gap: 0.5rem;
        }

        #calendar .fc-toolbar-chunk {
            display: flex;
            justify-content: center;
            flex-wrap: wrap;
            gap: 0.25rem;
        }

        #calendar .fc-button {
            padding: 0.35rem 0.5rem;
            font-size: 0.8rem;
        }

        .calendar-main {
            padding: 0.875rem;
        }

        .calendar-desktop-date-nav,
```

```
        .calendar-desktop-view-controls,
        .calendar-main > p {
            display: none;
        }

        .calendar-mobile-date-bar {
            display: flex;
        }

        #calendar {
            min-height: 480px;
        }

        #calendar .fc-timegrid-slot {
            height: 2.75rem;
        }
    }

    #calendar {
        --fc-border-color: #e8eeea;
        --fc-page-bg-color: transparent;
        --fc-neutral-bg-color: #f8fafc;
        --fc-today-bg-color: #f0f7f1;
        --fc-button-bg-color: #166534;
        --fc-button-border-color: #166534;
        --fc-button-hover-bg-color: #14532d;
        --fc-button-hover-border-color: #14532d;
        --fc-button-active-bg-color: #052e16;
        --fc-button-active-border-color: #052e16;
        --fc-event-border-color: #166534;
        --fc-event-bg-color: #166534;
        --fc-event-text-color: #fff;
    }

    #calendar .fc-toolbar-title {
        color: #1f2937;
        font-weight: 600;
    }

    #calendar .fc-col-header-cell-cushion,
    #calendar .fc-daygrid-day-number,
    #calendar .fc-timegrid-slot-label-cushion {
        color: #475569;
        text-decoration: none;
    }

    #calendar .fc-timegrid-now-indicator-line {
        border-color: #b45309;
    }

    #calendar .fc-scrollgrid {
        border: 0;
    }

    #calendar .fc-scrollgrid-section > td,
    #calendar .fc-scrollgrid-section > th {
        border: 0;
    }

    #calendar .fc-scrollgrid-section-header .fc-scroller {
        scrollbar-width: none;
    }

    #calendar .fc-scrollgrid-section-header .fc-scroller::-webkit-scrollbar {
        display: none;
    }

    #calendar .fc-col-header-cell {
        border: 0;
        padding: 0.35rem 0.25rem 0.75rem;
    }

    #calendar .fc-col-header-cell-cushion {
        display: block;
```

```
        min-height: 3.5rem;
        border: 1px solid #e6ece7;
        border-radius: 0.9rem;
        background: #f8faf8;
    }

    #calendar .fc-day-today .fc-col-header-cell-cushion,
    #calendar th.fc-day-today .fc-col-header-cell-cushion {
        border-color: #c8ddce;
        background: #edf5ef;
    }

    #calendar .fc-timegrid-col {
        background: #fff;
    }

    #calendar .fc-timegrid-col.fc-day-today {
        background: #f7faf7;
    }

    #calendar .fc-timegrid-slot {
        height: 3.25rem;
    }

    #calendar .fc-v-event {
        border: 0;
        border-left: 3px solid #5d8868;
        border-radius: 0.75rem;
        box-shadow: 0 2px 6px rgb(31 41 55 / 8%);
        padding: 0.15rem;
    }

    #calendar .fc-event-main {
        color: #26372c;
        padding: 0.3rem 0.4rem;
    }

    #calendar .fc-daygrid-event {
        border: 0;
        border-left: 3px solid #5d8868;
        border-radius: 0.6rem;
        padding: 0.15rem 0.25rem;
    }

    #calendar .fc-daygrid-event .fc-event-main {
        padding: 0.15rem 0.25rem;
    }

    #miniCalendarDays button {
        min-width: 0;
        border-radius: 9999px;
        padding: 0.45rem 0;
        font-size: 0.75rem;
        transition: background-color 150ms ease, color 150ms ease;
    }

    #miniCalendarDays button:hover {
        background: #edf5ef;
    }

    #pendingReservationsList {
        max-height: 18rem;
        overflow-y: auto;
        padding-right: 0.5rem;
        scrollbar-color: #d6a629 #f8f7ef;
        scrollbar-width: thin;
    }

    #pendingReservationsList::-webkit-scrollbar {
        width: 8px;
    }

    #pendingReservationsList::-webkit-scrollbar-track {
        background: #f8f7ef;
```

```
        border-radius: 9999px;
    }

    #pendingReservationsList::-webkit-scrollbar-thumb {
        background: #fbbf24;
        border-radius: 9999px;
    }
</style>
@endpush

@push('scripts')
<!-- FullCalendar scripts and page event handling logic for loading events and opening modals. -->
<script defer src="{{ asset('fullcalendar/index.global.min.js') }}"></script>
<script>
    document.addEventListener('DOMContentLoaded', function() {
        var calendarEl = document.getElementById('calendar');
        var searchInput = document.getElementById('eventSearch');
        var facilityFilter = document.getElementById('calendarFacilityFilter');
        var pendingReservationsList = document.getElementById('pendingReservationsList');
        var viewButtons = document.querySelectorAll('[data-view]');
        var eventInfoModal = document.getElementById('eventInfoModal');
        var eventInfoTitle = document.getElementById('eventInfoTitle');
        var eventInfoRoom = document.getElementById('eventInfoRoom');
        var eventInfoStatus = document.getElementById('eventInfoStatus');
        var eventInfoDate = document.getElementById('eventInfoDate');
        var eventInfoTime = document.getElementById('eventInfoTime');
        var eventInfoPurpose = document.getElementById('eventInfoPurpose');
        var eventInfoEditBtn = document.getElementById('eventInfoEditBtn');
        var eventInfoDeleteForm = document.getElementById('eventInfoDeleteForm');
        var eventInfoClose = document.getElementById('closeEventInfoBtn');
        var currentSearch = '';
        var cachedEvents = [];
            var suggestionsEl = document.getElementById('searchSuggestions');
        var reservationOpenTime = @json($reservationHours['start_time']);
        var reservationCloseTime = @json($reservationHours['end_time']);
        var minimumCalendarYear = new Date().getFullYear() - 1;
        var calendarTitle = document.getElementById('calendarCurrentTitle');
        var previousCalendarButton = document.getElementById('calendarPrevBtn');
        var nextCalendarButton = document.getElementById('calendarNextBtn');
        var mobilePreviousButton = document.getElementById('calendarMobilePrevBtn');
        var mobileNextButton = document.getElementById('calendarMobileNextBtn');
        var mobileDateTitle = document.getElementById('calendarMobileDateTitle');
        var miniCalendarTitle = document.getElementById('miniCalendarTitle');
        var miniCalendarPreviousButton = document.getElementById('miniCalendarPrev');
        var miniCalendarNextButton = document.getElementById('miniCalendarNext');

        function updateActiveViewButton(viewType) {
            viewButtons.forEach(function(button) {
                var isActive = button.getAttribute('data-view') === viewType;
                button.classList.toggle('bg-white', isActive);
                button.classList.toggle('font-medium', isActive);
                button.classList.toggle('text-green-900', isActive);
                button.classList.toggle('shadow-sm', isActive);
                button.classList.toggle('text-slate-600', !isActive);
                button.setAttribute('aria-pressed', String(isActive));
            });
        }

            function closeEventInfoModal() {
                if (eventInfoModal) {
                    eventInfoModal.classList.add('hidden');
                    eventInfoModal.classList.remove('flex');
                }
            }

            function renderSearchSuggestions(filteredEvents) {
                if (!suggestionsEl) return;
                suggestionsEl.replaceChildren();
                if (!filteredEvents.length || !currentSearch) {
                    suggestionsEl.classList.add('hidden');
                    return;
                }

                filteredEvents.forEach(function(item) {
```

```
                    var date = new Date(item.start);
                    var time = date.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
                    var button = document.createElement('button');
                    button.type = 'button';
                    button.className = 'w-full text-left px-4 py-3 hover:bg-slate-100 transition-colors';
                    button.dataset.eventId = item.id;

                    var title = document.createElement('div');
                    title.className = 'font-semibold text-slate-900';
                    title.textContent = item.title;

                    var details = document.createElement('div');
                    details.className = 'text-xs text-slate-500';
                    details.textContent = (item.extendedProps.room || '') + (
                        item.extendedProps.purpose ? ' • ' + item.extendedProps.purpose : ''
                    );

                    var schedule = document.createElement('div');
                    schedule.className = 'mt-1 text-xs text-slate-500';
                    schedule.textContent = date.toLocaleDateString() + ' • ' + time;

                    button.append(title, details, schedule);
                    suggestionsEl.appendChild(button);
                });
                suggestionsEl.classList.remove('hidden');
            }

            function getSearchMatches(query) {
                if (!query) return [];
                query = query.toLowerCase();

                return cachedEvents.filter(function(event) {
                    var title = (event.title || '').toLowerCase();
                    var room = (event.extendedProps.room || '').toLowerCase();
                    var purpose = (event.extendedProps.purpose || '').toLowerCase();
                    return title.includes(query) || room.includes(query) || purpose.includes(query);
                }).slice(0, 6);
            }

            function openEventFromSuggestion(eventId) {
                var event = calendar.getEventById(eventId);
                var sourceEvent = cachedEvents.find(function(item) {
                    return String(item.id) === String(eventId);
                });

                if (!sourceEvent) return;
                if (sourceEvent.start) {
                    calendar.gotoDate(new Date(sourceEvent.start));
                }

                if (event) {
                    event.setProp('backgroundColor', event.backgroundColor || event.color);
                }

                // Show details modal for the selected suggestion
                var props = sourceEvent.extendedProps || {};
                var date = new Date(sourceEvent.start);
                var startTime = date.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
                var endTime = sourceEvent.end ? new Date(sourceEvent.end).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }) : '';
                var status = props.status || 'pending';
                var statusLabel = props.is_maintenance ? 'Unavailable' : status === 'approved' ? 'Approved' : status === 'rejected' ? 'Rejected' : 'Waiting Approval';
                var statusClasses = props.is_maintenance ? 'inline-flex rounded-full px-3 py-1 text-sm font-semibold bg-rose-100 text-rose-700' :
                                    status === 'approved' ? 'inline-flex rounded-full px-3 py-1 text-sm font-semibold bg-emerald-100 text-emerald-700' :
                                    status === 'rejected' ? 'inline-flex rounded-full px-3 py-1 text-sm font-semibold bg-red-100 text-red-700' :
                                    'inline-flex rounded-full bg-amber-100 px-3 py-1 text-sm font-semibold text-amber-800';

                eventInfoTitle.textContent = props.room || sourceEvent.title || 'Reservation';
                eventInfoRoom.textContent = props.room || sourceEvent.title || '—';
                eventInfoStatus.textContent = statusLabel;
                eventInfoStatus.className = statusClasses;
                eventInfoDate.textContent = props.date || date.toLocaleDateString();
                eventInfoTime.textContent = startTime + ' - ' + endTime;
                eventInfoPurpose.textContent = props.purpose || (
                    props.is_owner ? '—' : 'Details are only visible to the reservation owner.'
```

```
                );
                if (eventInfoEditBtn) {
                    eventInfoEditBtn.href = '/reserve/reservations/' + sourceEvent.id + '/edit';
                    if (props.is_owner && ['approved', 'rejected', 'pending'].includes(props.status)) {
                        eventInfoEditBtn.classList.remove('hidden');
                    } else {
                        eventInfoEditBtn.classList.add('hidden');
                    }
                }
                if (eventInfoDeleteForm) {
                    eventInfoDeleteForm.action = '/reserve/reservations/' + sourceEvent.id;
                    if (props.is_owner && (status === 'rejected' || status === 'pending')) {
                        eventInfoDeleteForm.classList.remove('hidden');
                    } else {
                        eventInfoDeleteForm.classList.add('hidden');
                    }
                }
                eventInfoModal.classList.remove('hidden');
                eventInfoModal.classList.add('flex');
            }

        var calendar = new window.FullCalendar.Calendar(calendarEl, {
            initialView: 'timeGridWeek',
            headerToolbar: false,
            navLinks: true,
            selectable: false,
            editable: false,
            allDaySlot: false,
            slotMinTime: reservationOpenTime + ':00',
            slotMaxTime: reservationCloseTime + ':00',
            dayHeaderContent: function(info) {
                var weekday = info.date.toLocaleDateString('en-US', { weekday: 'short' });
                var container = document.createElement('div');
                container.className = 'flex flex-col items-center gap-0.5 py-1';

                var weekdayLabel = document.createElement('span');
                weekdayLabel.className = 'text-[10px] font-medium uppercase tracking-wide text-slate-500';
                weekdayLabel.textContent = weekday;

                if (info.view.type === 'dayGridMonth') {
                    return { domNodes: [weekdayLabel] };
                }

                var dateLabel = document.createElement('span');
                dateLabel.className = 'text-sm font-semibold text-slate-800';
                dateLabel.textContent = String(info.date.getDate());

                container.append(weekdayLabel, dateLabel);

                return { domNodes: [container] };
            },
            datesSet: function(info) {
                if (calendarTitle) {
                    calendarTitle.textContent = info.view.title;
                }
                if (mobileDateTitle) {
                    mobileDateTitle.textContent = info.view.title;
                }
                updateActiveViewButton(info.view.type);
                if (calendar.getDate().getFullYear() < minimumCalendarYear) {
                    calendar.gotoDate(new Date(minimumCalendarYear, 0, 1));
                    return;
                }

                updateMiniCalendar();
            },
            events: function(info, successCallback, failureCallback) {
                var query = new URLSearchParams({
                    start: info.startStr.slice(0, 10),
                    end: info.endStr.slice(0, 10)
                });
                if (facilityFilter && facilityFilter.value) {
                    query.set('facility_id', facilityFilter.value);
                }
```

```
                fetch('/api/calendar/events?' + query, { credentials: 'include' })
                    .then(function(response) {
                        if (!response.ok) {
                            throw new Error('Calendar events could not be loaded.');
                        }
                        return response.json();
                    })
                    .then(function(events) {
                        cachedEvents = events.map(function(item) {
                            return {
                                id: item.id,
                                title: item.title,
                                start: item.start,
                                end: item.end,
                                extendedProps: item.extendedProps || {},
                                color: item.color || null
                            };
                        });
                        successCallback(events);
                    })
                    .catch(function(err) {
                        console.error(err);
                        failureCallback(err);
                    });
            },
            eventClick: function(info) {
                var event = info.event;
                var props = event.extendedProps || {};
                var startTime = event.start ? event.start.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }) : '';
                var endTime = event.end ? event.end.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }) : '';
                var status = props.status || 'pending';
                var statusLabel = props.is_maintenance ? 'Unavailable' : status === 'approved' ? 'Approved' : status === 'rejected' ? 'Rejected' : 'Waiting Approval';
                var statusClasses = props.is_maintenance ? 'inline-flex rounded-full px-3 py-1 text-sm font-semibold bg-rose-100 text-rose-700' :
                                    status === 'approved' ? 'inline-flex rounded-full px-3 py-1 text-sm font-semibold bg-emerald-100 text-emerald-700' :
                                    status === 'rejected' ? 'inline-flex rounded-full px-3 py-1 text-sm font-semibold bg-red-100 text-red-700' :
                                    'inline-flex rounded-full bg-amber-100 px-3 py-1 text-sm font-semibold text-amber-800';

                eventInfoTitle.textContent = props.room || event.title || 'Reservation';
                eventInfoRoom.textContent = props.room || event.title || '—';
                eventInfoStatus.textContent = statusLabel;
                eventInfoStatus.className = statusClasses;
                eventInfoDate.textContent = props.date || (event.start ? event.start.toLocaleDateString() : '—');
                eventInfoTime.textContent = startTime + ' - ' + endTime;
                eventInfoPurpose.textContent = props.purpose || (
                    props.is_owner ? '—' : 'Details are only visible to the reservation owner.'
                );
                if (eventInfoEditBtn) {
                    eventInfoEditBtn.href = '/reserve/reservations/' + event.id + '/edit';
                    if (props.is_owner && ['approved', 'rejected', 'pending'].includes(props.status)) {
                        eventInfoEditBtn.classList.remove('hidden');
                    } else {
                        eventInfoEditBtn.classList.add('hidden');
                    }
                }
                if (eventInfoDeleteForm) {
                    eventInfoDeleteForm.action = '/reserve/reservations/' + event.id;
                    if (props.is_owner && (status === 'rejected' || status === 'pending')) {
                        eventInfoDeleteForm.classList.remove('hidden');
                    } else {
                        eventInfoDeleteForm.classList.add('hidden');
                    }
                }
                eventInfoModal.classList.remove('hidden');
                eventInfoModal.classList.add('flex');
            }
        });

        calendar.render();
        if (facilityFilter) {
        facilityFilter.addEventListener('change', function() {
            calendar.refetchEvents();
        });
        }
        fetchPendingReservations();
```

```
647
648         function fetchPendingReservations() {
649             if (!pendingReservationsList) return;
650             fetch('/api/calendar/pending', { credentials: 'include' })
651                 .then(function(response) { return response.json(); })
652                 .then(function(pending) {
653                     if (!pending.length) {
654                         pendingReservationsList.innerHTML = '<div class="rounded-3xl bg-slate-50 p-4 text-sm text-slate-500">No pending reservations at the moment.</div>';
655                         return;
656                     }
657
658                     pendingReservationsList.innerHTML = pending.map(function(item) {
659                         return '<div class="rounded-3xl bg-slate-50 p-4">'
660                             + '<div class="flex items-center justify-between">'
661                             + '<div>'
662                             + '<p class="text-sm font-semibold text-slate-900">' + item.facility + '</p>'
663                             + '<p class="text-sm text-slate-500">' + item.date + ' • ' + item.start_time + ' - ' + item.end_time + '</p>'
664                             + '</div>'
665                             + '<span class="rounded-full bg-amber-100 px-3 py-1 text-xs font-semibold text-amber-800">Pending</span>'
666                             + '</div>'
667                             + '<p class="mt-3 text-sm text-slate-600">' + (item.purpose || 'No purpose provided.') + '</p>'
668                             + '</div>';
669                     }).join('');
670                 })
671                 .catch(function() {
672                     if (pendingReservationsList) {
673                         pendingReservationsList.innerHTML = '<div class="rounded-3xl bg-slate-50 p-4 text-sm text-slate-500">Unable to load pending reservations.</div>';
674                     }
675                 });
676         }
677
678         // Mini calendar update function
679         function updateMiniCalendar() {
680             var currentDate = calendar.getDate();
681             var miniMonth = document.getElementById('miniCalendarMonth');
682             var miniYear = document.getElementById('miniCalendarYear');
683             var miniView = document.getElementById('miniCalendarView');
684             var miniDays = document.getElementById('miniCalendarDays');
685
686             if (!miniMonth || !miniYear || !miniDays) return;
687
688             var monthNames = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
689                               'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
690             if (miniMonth.options.length === 0) {
691                 monthNames.forEach(function(monthName, monthIndex) {
692                     var option = document.createElement('option');
693                     option.value = String(monthIndex);
694                     option.textContent = monthName;
695                     miniMonth.appendChild(option);
696                 });
697             }
698
699             var selectedYear = currentDate.getFullYear();
700             if (miniYear.options.length === 0) {
701                 var lastYear = Math.max(new Date().getFullYear() + 20, selectedYear);
702                 for (var year = minimumCalendarYear; year <= lastYear; year++) {
703                     var yearOption = document.createElement('option');
704                     yearOption.value = String(year);
705                     yearOption.textContent = String(year);
706                     miniYear.appendChild(yearOption);
707                 }
708             }
709             miniMonth.value = String(currentDate.getMonth());
710             miniYear.value = String(selectedYear);
711             if (miniCalendarTitle) {
712                 miniCalendarTitle.textContent = currentDate.toLocaleDateString('en-US', {
713                     month: 'long',
714                     year: 'numeric'
715                 });
716             }
717
718             // Get first day of month and number of days
719             var firstDay = new Date(currentDate.getFullYear(), currentDate.getMonth(), 1);
720             var lastDay = new Date(currentDate.getFullYear(), currentDate.getMonth() + 1, 0);
```

```
            var startDate = new Date(firstDay);
            startDate.setDate(startDate.getDate() - firstDay.getDay());

            // Clear and populate days
            miniDays.innerHTML = '';
            var today = new Date();
            today.setHours(0, 0, 0, 0);

            for (var i = 0; i < 42; i++) {
                var dayDate = new Date(startDate);
                dayDate.setDate(dayDate.getDate() + i);
                var dayButton = document.createElement('button');
                dayButton.type = 'button';
                dayButton.textContent = dayDate.getDate();
                dayButton.dataset.calendarDate = [
                    dayDate.getFullYear(),
                    String(dayDate.getMonth() + 1).padStart(2, '0'),
                    String(dayDate.getDate()).padStart(2, '0')
                ].join('-');
                dayButton.setAttribute('aria-label', dayDate.toLocaleDateString());
                dayButton.className = 'text-slate-700';

                // Highlight today
                if (dayDate.toDateString() === today.toDateString()) {
                    dayButton.className = 'bg-green-800 text-white hover:bg-green-900';
                } else if (dayDate.toDateString() === currentDate.toDateString()) {
                    dayButton.className = 'bg-green-100 font-semibold text-green-900';
                }

                // Dim days outside current month
                if (dayDate.getMonth() !== currentDate.getMonth()) {
                    dayButton.classList.add('opacity-40');
                }

                miniDays.appendChild(dayButton);
            }

        }

        updateMiniCalendar();
        document.getElementById('miniCalendarDays').addEventListener('click', function(event) {
            var dayButton = event.target.closest('button[data-calendar-date]');
            if (dayButton) {
                calendar.gotoDate(dayButton.dataset.calendarDate);
            }
        });

        var miniMonth = document.getElementById('miniCalendarMonth');
        var miniYear = document.getElementById('miniCalendarYear');
        function navigateCalendarFromSelectors(monthSelect, yearSelect) {
            var month = Number(monthSelect.value);
            var year = Number(yearSelect.value);

            if (Number.isInteger(month) && month >= 0 && month <= 11
                && Number.isInteger(year) && year >= minimumCalendarYear) {
                calendar.gotoDate(new Date(year, month, 1));
            }
        }

        function navigateFromMiniCalendar() {
            navigateCalendarFromSelectors(miniMonth, miniYear);
        }

        miniMonth.addEventListener('change', navigateFromMiniCalendar);
        miniYear.addEventListener('change', navigateFromMiniCalendar);
        [miniCalendarPreviousButton, miniCalendarNextButton].forEach(function(button, index) {
            button.addEventListener('click', function() {
                var monthOffset = index === 0 ? -1 : 1;
                calendar.gotoDate(new Date(calendar.getDate().getFullYear(), calendar.getDate().getMonth() + monthOffset, 1));
            });
        });

        if (searchInput) {
            searchInput.addEventListener('input', function() {
```

```
                currentSearch = this.value.trim();
                var matches = getSearchMatches(currentSearch);
                renderSearchSuggestions(matches);
            });

            searchInput.addEventListener('keydown', function(event) {
                if (event.key === 'Escape') {
                    currentSearch = '';
                    this.value = '';
                    renderSearchSuggestions([]);
                }
            });

            searchInput.addEventListener('blur', function() {
                setTimeout(function() {
                    renderSearchSuggestions([]);
                }, 150);
            });
        }

        if (suggestionsEl) {
            suggestionsEl.addEventListener('click', function(event) {
                var button = event.target.closest('button[data-event-id]');
                if (!button) return;
                var eventId = button.getAttribute('data-event-id');
                openEventFromSuggestion(eventId);
                renderSearchSuggestions([]);
                searchInput.value = '';
                currentSearch = '';
            });
        }

        if (previousCalendarButton) {
            previousCalendarButton.addEventListener('click', function() {
                calendar.prev();
            });
        }

        if (nextCalendarButton) {
            nextCalendarButton.addEventListener('click', function() {
                calendar.next();
            });
        }

        if (mobilePreviousButton) {
            mobilePreviousButton.addEventListener('click', function() {
                calendar.prev();
            });
        }

        if (mobileNextButton) {
            mobileNextButton.addEventListener('click', function() {
                calendar.next();
            });
        }

        viewButtons.forEach(function(button) {
            button.addEventListener('click', function() {
                var viewName = button.getAttribute('data-view');
                if (viewName) {
                    calendar.changeView(viewName);
                    updateMiniCalendar();
                }
            });
        });

        if (eventInfoClose) {
            eventInfoClose.addEventListener('click', closeEventInfoModal);
        }

        if (eventInfoModal) {
            eventInfoModal.addEventListener('click', function(event) {
                if (event.target === eventInfoModal) {
                    closeEventInfoModal();
```

```
                }
            });
        }
    });
</script>
@endpush
```

resources/views/components/cta-section.blade.php

```
<section class="px-6 pb-20 sm:px-8 md:pb-24">
    <div class="relative isolate mx-auto max-w-7xl overflow-hidden rounded-[2rem] bg-indigo-700 px-6 py-14 text-center shadow-2xl shadow-indigo-950/15 sm:px-12 sm:py-16">
        <div aria-hidden="true" class="absolute inset-0 -z-10">
            <div class="absolute -right-20 -top-44 h-96 w-96 rounded-full bg-indigo-500/70 blur-3xl"></div>
            <div class="absolute -bottom-48 -left-20 h-96 w-96 rounded-full bg-cyan-300/20 blur-3xl"></div>
        </div>
        <p class="mx-auto max-w-3xl text-lg font-medium leading-8 text-indigo-100 sm:text-xl">Ready to streamline your campus reservations? Explore the calendar and take control of your scheduling with Campus Reserve – efficient, reliable, and easy.</p>
        <div class="mt-8 flex flex-wrap justify-center gap-3">
            <a href="/reserve" class="inline-flex items-center gap-2 rounded-full bg-white px-6 py-3.5 text-sm font-semibold text-indigo-800 shadow-sm transition hover:-translate-y-0.5 hover:bg-indigo-50">
                Explore Calendar
                <svg aria-hidden="true" class="h-4 w-4" viewBox="0 0 20 20" fill="none">
                    <path d="M4 10h12m-5-5 5 5-5 5" stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"/>
                </svg>
            </a>
            <a href="/about" class="inline-flex items-center rounded-full border border-white/30 px-6 py-3.5 text-sm font-semibold text-white transition hover:bg-white/10">Learn More</a>
        </div>
    </div>
</section>
```

resources/views/components/features-section.blade.php

```
<section class="bg-slate-50 px-6 py-20 sm:px-8 md:py-24">
    <div class="mx-auto max-w-7xl">
        <div class="mx-auto max-w-2xl text-center">
            <p class="text-sm font-semibold uppercase tracking-[0.18em] text-indigo-600">Campus Reserve</p>
            <h2 class="mt-3 text-3xl font-semibold tracking-tight text-slate-950 sm:text-4xl">With Campus Reserve You Can!</h2>
        </div>

        <div class="mt-12 grid gap-4 sm:grid-cols-2 lg:grid-cols-4">
            <article class="group rounded-3xl border border-slate-200 bg-white p-6 shadow-sm transition duration-200 hover:-translate-y-1 hover:border-indigo-200 hover:shadow-lg hover:shadow-indigo-950/5">
                <span class="flex h-12 w-12 items-center justify-center rounded-2xl bg-indigo-50 text-indigo-700 transition group-hover:bg-indigo-100">
                    <svg aria-hidden="true" class="h-6 w-6" viewBox="0 0 24 24" fill="none">
                        <rect x="3.5" y="5" width="17" height="15.5" rx="2.5" stroke="currentColor" stroke-width="1.7"/>
                        <path d="M7.5 3.5v3M16.5 3.5v3M4 9h16M8 12.5h2m4 0h2M8 16h2" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"/>
                    </svg>
                </span>
                <h3 class="mt-5 text-lg font-semibold text-slate-950">Centralize reservations</h3>
                <p class="mt-2 text-sm leading-6 text-slate-600">Bring all room and facility bookings into one platform. Campus Reserve eliminates scattered logs and manual coordination by providing a single, organized system for managing schedules.</p>
            </article>

            <article class="group rounded-3xl border border-slate-200 bg-white p-6 shadow-sm transition duration-200 hover:-translate-y-1 hover:border-cyan-200 hover:shadow-lg hover:shadow-cyan-950/5">
                <span class="flex h-12 w-12 items-center justify-center rounded-2xl bg-cyan-50 text-cyan-700 transition group-hover:bg-cyan-100">
                    <svg aria-hidden="true" class="h-6 w-6" viewBox="0 0 24 24" fill="none">
                        <path d="M12 3.75v16.5M3.75 12h16.5" stroke="currentColor" stroke-width="1.8" stroke-linecap="round"/>
                        <circle cx="12" cy="12" r="9" stroke="currentColor" stroke-width="1.6"/>
                    </svg>
                </span>
                <h3 class="mt-5 text-lg font-semibold text-slate-950">Streamline booking</h3>
                <p class="mt-2 text-sm leading-6 text-slate-600">Make reservations fast and intuitive. With just a few clicks, students, faculty, and staff can book shared spaces without unnecessary steps or delays.</p>
            </article>

            <article class="group rounded-3xl border border-slate-200 bg-white p-6 shadow-sm transition duration-200 hover:-translate-y-1 hover:border-emerald-200 hover:shadow-lg hover:shadow-emerald-950/5">
                <span class="flex h-12 w-12 items-center justify-center rounded-2xl bg-emerald-50 text-emerald-700 transition group-hover:bg-emerald-100">
                    <svg aria-hidden="true" class="h-6 w-6" viewBox="0 0 24 24" fill="none">
                        <path d="M7 4.5h10M7 8h10M7 11.5h6M6 3.5h12a2 2 0 0 1 2 2v13a2 2 0 0 1-2 2H6a2 2 0 0 1-2-2v-13a2 2 0 0 1 2-2Z" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"/>
```

```
                        <path d="m14 16 1.5 1.5L19 14" stroke="currentColor" stroke-width="1.7" stroke-linecap="round" stroke-linejoin="round"/>
                    </svg>
                </span>
                <h3 class="mt-5 text-lg font-semibold text-slate-950">Real-time availability</h3>
                <p class="mt-2 text-sm leading-6 text-slate-600">View up-to-date availability through an integrated calendar. Users can instantly see open time slots, helping schools prevent double-bookings and scheduling conflicts.</p>
            </article>

            <article class="group rounded-3xl border border-slate-200 bg-white p-6 shadow-sm transition duration-200 hover:-translate-y-1 hover:border-amber-200 hover:shadow-lg hover:shadow-amber-950/5">
                <span class="flex h-12 w-12 items-center justify-center rounded-2xl bg-amber-50 text-amber-700 transition group-hover:bg-amber-100">
                    <svg aria-hidden="true" class="h-6 w-6" viewBox="0 0 24 24" fill="none">
                        <path d="M5 20V9l7-5 7 5v11M9 20v-6h6v6M3 20h18" stroke="currentColor" stroke-width="1.7" stroke-linecap="round" stroke-linejoin="round"/>
                    </svg>
                </span>
                <h3 class="mt-5 text-lg font-semibold text-slate-950">Improve administrative efficiency</h3>
                <p class="mt-2 text-sm leading-6 text-slate-600">Reduce the workload on administrators by automating approvals, tracking reservations, and maintaining clear records. Campus Reserve helps schools save time while keeping operations organized and transparent.</p>
            </article>
        </div>

        <div class="mt-10 flex justify-center">
            <a href="/reserve" class="inline-flex items-center gap-2 rounded-full border border-indigo-700 px-6 py-3 text-sm font-semibold text-indigo-800 transition hover:bg-indigo-50">
                Reserve Now
                <svg aria-hidden="true" class="h-4 w-4" viewBox="0 0 20 20" fill="none">
                    <path d="M4 10h12m-5-5 5 5-5 5" stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"/>
                </svg>
            </a>
        </div>
    </div>
</section>
```

resources/views/components/header.blade.php

```
<header class="flex items-center justify-between px-6 py-4 bg-white border-b border-gray-200">
    <a href="/" class="flex items-center">
        <div class="flex items-center border border-black px-2 py-1">
            <span class="text-xs font-medium tracking-wide">CAMPUS</span>
            <span class="bg-black text-white text-xs font-medium px-1 ml-1">RESERVE</span>
        </div>
    </a>

    <nav class="hidden md:flex items-center gap-8">
        <a href="/" class="text-sm font-medium text-black hover:text-slate-700 transition-colors">HOME</a>
        <a href="/reserve" class="text-sm font-medium text-black hover:text-slate-700 transition-colors">RESERVE</a>
        <a href="/calendar" class="text-sm font-medium text-black hover:text-slate-700 transition-colors">CALENDAR</a>
        <a href="/about" class="text-sm font-medium text-black hover:text-slate-700 transition-colors">ABOUT US</a>
        <a href="/contact" class="text-sm font-medium text-black hover:text-slate-700 transition-colors">CONTACT</a>
    </nav>

    <div class="flex items-center gap-3">
        @auth
            <a href="{{ route('credentials.index') }}" class="text-sm font-medium text-black hover:text-slate-700">Passwords</a>
            <form method="POST" action="{{ route('logout') }}" class="inline">
                @csrf
                <button type="submit" class="px-4 py-1.5 text-sm font-medium bg-black text-white rounded-full hover:bg-gray-800 transition-colors">
                    Log Out
                </button>
            </form>
        @else
            <a href="/signup" class="px-4 py-1.5 text-sm font-medium bg-black text-white rounded-full hover:bg-gray-800 transition-colors">Sign Up</a>
            <a href="/login" class="px-4 py-1.5 text-sm font-medium bg-black text-white rounded-full hover:bg-gray-800 transition-colors">Log In</a>
        @endauth
    </div>
</header>
```

resources/views/components/hero-section.blade.php

```
<section class="relative isolate overflow-hidden bg-white text-slate-900">
    <div aria-hidden="true" class="absolute inset-0 overflow-hidden">
        <div class="absolute inset-0 bg-gradient-to-br from-purple-100 via-pink-100 to-cyan-100 opacity-70"></div>
        <div class="absolute -left-40 -top-48 h-[34rem] w-[34rem] rounded-full bg-indigo-300/30 blur-3xl"></div>
```

```
        <div class="absolute -bottom-64 right-0 h-[38rem] w-[38rem] rounded-full bg-cyan-200/40 blur-3xl"></div>
        <div class="absolute inset-0 opacity-[0.16]" style="background-image: radial-gradient(#64748b 1px, transparent 1px); background-size: 28px 28px;"></div>
    </div>

    <div class="relative mx-auto grid max-w-7xl items-center gap-14 px-6 py-20 sm:px-8 md:py-28 lg:grid-cols-[0.92fr_1.08fr] lg:gap-16 lg:px-12">
        <div class="max-w-2xl">
            <div class="inline-flex items-center gap-2 rounded-full border border-indigo-200 bg-white/70 px-3.5 py-2 text-sm text-indigo-900 shadow-sm backdrop-blur">
                <span class="h-2 w-2 rounded-full bg-indigo-500"></span>
                Reservation and scheduling platform for schools
            </div>

            <h1 class="mt-7 text-5xl font-semibold tracking-tight text-slate-900 sm:text-6xl lg:text-7xl">
                Welcome to
                <span class="block text-indigo-700">Campus Reserve!</span>
            </h1>
            <p class="mt-6 max-w-xl text-lg leading-8 text-slate-700">
                Campus Reserve is a reservation and scheduling platform designed specifically for schools to manage shared spaces and facilities efficiently. It provides a centralized system where students, faculty, and staff can view availability, coordinate schedules, and book rooms with ease.
            </p>

            <div class="mt-9 flex flex-wrap items-center gap-3">
                <a href="/reserve" class="inline-flex items-center gap-2 rounded-full bg-slate-900 px-6 py-3.5 text-sm font-semibold text-white shadow-lg shadow-indigo-950/15 transition hover:-translate-y-0.5 hover:bg-indigo-800">
                    Reserve Now
                    <svg aria-hidden="true" class="h-4 w-4" viewBox="0 0 20 20" fill="none">
                        <path d="M4 10h12m-5-5 5 5-5 5" stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"/>
                    </svg>
                </a>
                <a href="/calendar" class="inline-flex items-center rounded-full border border-slate-300 bg-white/60 px-6 py-3.5 text-sm font-semibold text-slate-800 transition hover:border-indigo-300 hover:bg-white">
                    Explore Calendar
                </a>
            </div>

            <div class="mt-10 flex items-center gap-3 text-sm text-slate-600">
                <div class="flex -space-x-2" aria-hidden="true">
                    <span class="flex h-8 w-8 items-center justify-center rounded-full border-2 border-white bg-indigo-200 text-xs font-semibold text-indigo-950">S</span>
                    <span class="flex h-8 w-8 items-center justify-center rounded-full border-2 border-white bg-pink-200 text-xs font-semibold text-pink-950">F</span>
                    <span class="flex h-8 w-8 items-center justify-center rounded-full border-2 border-white bg-cyan-200 text-xs font-semibold text-cyan-950">A</span>
                </div>
                <span>For students, faculty, and staff</span>
            </div>
        </div>

        <div class="relative mx-auto w-full max-w-2xl" aria-label="Preview of the Campus Reserve booking calendar">
            <div aria-hidden="true" class="absolute -inset-5 rounded-[2.5rem] bg-gradient-to-br from-indigo-400/25 via-pink-300/20 to-cyan-300/30 blur-2xl"></div>
            <div class="relative overflow-hidden rounded-[1.75rem] border border-white/80 bg-white text-slate-900 shadow-2xl shadow-indigo-900/15">
                <div class="flex items-center justify-between border-b border-slate-100 px-5 py-4 sm:px-7">
                    <div class="flex items-center gap-3">
                        <span class="flex h-10 w-10 items-center justify-center rounded-2xl bg-indigo-700 text-white">
                            <svg aria-hidden="true" class="h-5 w-5" viewBox="0 0 24 24" fill="none">
                                <rect x="3.5" y="5" width="17" height="15.5" rx="2.5" stroke="currentColor" stroke-width="1.7"/>
                                <path d="M7.5 3.5v3M16.5 3.5v3M4 9h16" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"/>
                                <path d="m9.5 14 1.7 1.7 3.8-4" stroke="currentColor" stroke-width="1.7" stroke-linecap="round" stroke-linejoin="round"/>
                            </svg>
                        </span>
                        <div>
                            <p class="text-sm font-semibold">Campus Reserve</p>
                            <p class="text-xs text-slate-500">Centralized scheduling</p>
                        </div>
                    </div>
                    <span class="rounded-full bg-emerald-50 px-3 py-1.5 text-xs font-semibold text-emerald-700">Real-time availability</span>
                </div>

                <div class="p-5 sm:p-7">
                    <div class="flex items-end justify-between gap-3">
                        <div>
                            <p class="text-xs font-medium uppercase tracking-[0.16em] text-slate-400">Integrated calendar</p>
                            <h2 class="mt-1 text-xl font-semibold tracking-tight sm:text-2xl">Real-Time Calendar Visibility</h2>
                        </div>
                        <span class="rounded-xl border border-slate-200 px-3 py-2 text-xs font-medium text-slate-600">October 2026</span>
                    </div>

                    <div class="mt-6 grid grid-cols-[3.2rem_repeat(5,minmax(0,1fr))] gap-1.5 sm:gap-2">
```

```
                        <div></div>
                        <div class="pb-2 text-center text-[10px] font-semibold uppercase tracking-wide text-slate-400 sm:text-xs">Mon<span class="mt-1 block text-sm text-slate-700">19</span></div>
                        <div class="pb-2 text-center text-[10px] font-semibold uppercase tracking-wide text-slate-400 sm:text-xs">Tue<span class="mt-1 block text-sm text-slate-700">20</span></div>
                        <div class="pb-2 text-center text-[10px] font-semibold uppercase tracking-wide text-slate-400 sm:text-xs">Wed<span class="mt-1 block text-sm text-slate-700">21</span></div>
                        <div class="pb-2 text-center text-[10px] font-semibold uppercase tracking-wide text-slate-400 sm:text-xs">Thu<span class="mt-1 block text-sm text-slate-700">22</span></div>
                        <div class="pb-2 text-center text-[10px] font-semibold uppercase tracking-wide text-slate-400 sm:text-xs">Fri<span class="mt-1 block text-sm text-slate-700">23</span></div>

                        <div class="flex h-14 items-start pt-1 text-[10px] font-medium text-slate-400 sm:h-16 sm:text-xs">9 AM</div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="row-span-2 rounded-xl border border-indigo-100 bg-indigo-50 p-2 sm:p-3">
                            <span class="block text-[9px] font-semibold text-indigo-500 sm:text-[10px]">9:00–10:30</span>
                            <span class="mt-1 block text-[10px] font-semibold leading-tight text-indigo-950 sm:text-xs">Study room</span>
                        </div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>

                        <div class="flex h-14 items-start pt-1 text-[10px] font-medium text-slate-400 sm:h-16 sm:text-xs">10 AM</div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="row-span-2 rounded-xl border border-cyan-200 bg-cyan-50 p-2 sm:p-3">
                            <span class="block text-[9px] font-semibold text-cyan-700 sm:text-[10px]">10:00–11:00</span>
                            <span class="mt-1 block text-[10px] font-semibold leading-tight text-cyan-950 sm:text-xs">Project room</span>
                        </div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>

                        <div class="flex h-14 items-start pt-1 text-[10px] font-medium text-slate-400 sm:h-16 sm:text-xs">11 AM</div>
                        <div class="row-span-2 rounded-xl border border-emerald-100 bg-emerald-50 p-2 sm:p-3">
                            <span class="block text-[9px] font-semibold text-emerald-600 sm:text-[10px]">11:00–12:00</span>
                            <span class="mt-1 block text-[10px] font-semibold leading-tight text-emerald-950 sm:text-xs">Lecture hall</span>
                        </div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                        <div class="h-14 rounded-lg border border-slate-100 bg-slate-50 sm:h-16"></div>
                    </div>

                    <div class="mt-5 flex items-center justify-between rounded-2xl bg-slate-50 px-4 py-3">
                        <div class="flex items-center gap-2 text-xs font-medium text-slate-600">
                            <span class="h-2 w-2 rounded-full bg-emerald-500"></span>
                            View up-to-date availability
                        </div>
                        <span class="text-xs font-semibold text-slate-900">Explore spaces <span aria-hidden="true">→</span></span>
                    </div>
                </div>
            </div>
            <div aria-hidden="true" class="absolute -bottom-5 -left-5 hidden items-center gap-3 rounded-2xl border border-slate-200/80 bg-white p-3 shadow-xl sm:flex">
                <span class="flex h-10 w-10 items-center justify-center rounded-xl bg-emerald-100 text-emerald-700">
                    <svg class="h-5 w-5" viewBox="0 0 24 24" fill="none">
                        <path d="m5 12 4 4L19 6" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/>
                    </svg>
                </span>
                <span><span class="block text-xs font-semibold text-slate-900">Efficient booking process</span><span class="mt-0.5 block text-[11px] text-slate-500">Make reservations fast and intuitive</span></span>
            </div>
        </div>
    </div>
</section>
```

resources/views/components/why-choose-section.blade.php

```
<section class="overflow-hidden bg-white px-6 py-20 sm:px-8 md:py-24">
    <div class="mx-auto grid max-w-7xl items-center gap-12 lg:grid-cols-[0.95fr_1.05fr] lg:gap-20">
        <div>
            <p class="text-sm font-semibold uppercase tracking-[0.18em] text-indigo-600">Campus Reserve</p>
            <h2 class="mt-3 max-w-xl text-3xl font-semibold tracking-tight text-slate-950 sm:text-4xl">Why Choose Campus Reserve?</h2>

            <div class="mt-8 space-y-5">
```

```
                <div class="flex gap-4">
                    <span class="mt-1 inline-flex h-5 w-5 shrink-0 items-center justify-center rounded-full bg-indigo-100 text-xs font-bold text-indigo-700"> </span>
                    <div>
                        <h3 class="font-semibold text-slate-950">Centralized Scheduling</h3>
                        <p class="mt-1 text-sm leading-6 text-slate-600">Manage all room and facility reservations in one organized platform.</p>
                    </div>
                </div>
                <div class="flex gap-4">
                    <span class="mt-1 inline-flex h-5 w-5 shrink-0 items-center justify-center rounded-full bg-pink-100 text-xs font-bold text-pink-800"> </span>
                    <div>
                        <h3 class="font-semibold text-slate-950">Real-Time Calendar Visibility</h3>
                        <p class="mt-1 text-sm leading-6 text-slate-600">Instantly view availability to prevent double-bookings and scheduling conflicts.</p>
                    </div>
                </div>
                <div class="flex gap-4">
                    <span class="mt-1 inline-flex h-5 w-5 shrink-0 items-center justify-center rounded-full bg-cyan-100 text-xs font-bold text-cyan-800"> </span>
                    <div>
                        <h3 class="font-semibold text-slate-950">Efficient Booking Process</h3>
                        <p class="mt-1 text-sm leading-6 text-slate-600">Reserve spaces quickly with an intuitive, step-by-step booking system.</p>
                    </div>
                </div>
                <div class="flex gap-4">
                    <span class="mt-1 inline-flex h-5 w-5 shrink-0 items-center justify-center rounded-full bg-indigo-100 text-xs font-bold text-indigo-700"> </span>
                    <div>
                        <h3 class="font-semibold text-slate-950">Reduced Administrative Workload</h3>
                        <p class="mt-1 text-sm leading-6 text-slate-600">Automate scheduling tasks and minimize manual coordination.</p>
                    </div>
                </div>
                <div class="flex gap-4">
                    <span class="mt-1 inline-flex h-5 w-5 shrink-0 items-center justify-center rounded-full bg-pink-100 text-xs font-bold text-pink-800"> </span>
                    <div>
                        <h3 class="font-semibold text-slate-950">Improved Space Utilization</h3>
                        <p class="mt-1 text-sm leading-6 text-slate-600">Gain better insight into how facilities are used and optimize resource allocation.</p>
                    </div>
                </div>
                <div class="flex gap-4">
                    <span class="mt-1 inline-flex h-5 w-5 shrink-0 items-center justify-center rounded-full bg-cyan-100 text-xs font-bold text-cyan-800"> </span>
                    <div>
                        <h3 class="font-semibold text-slate-950">Accessible Anytime, Anywhere</h3>
                        <p class="mt-1 text-sm leading-6 text-slate-600">Use the system through a web-based platform that supports flexible access.</p>
                    </div>
                </div>
                <div class="flex gap-4">
                    <span class="mt-1 inline-flex h-5 w-5 shrink-0 items-center justify-center rounded-full bg-indigo-100 text-xs font-bold text-indigo-700"> </span>
                    <div>
                        <h3 class="font-semibold text-slate-950">Reliable Third-Party Solution</h3>
                        <p class="mt-1 text-sm leading-6 text-slate-600">Operates independently, allowing schools to adopt the system without institutional dependency.</p>
                    </div>
                </div>
            </div>

            <a href="/about" class="mt-8 inline-flex items-center gap-2 text-sm font-semibold text-slate-900 transition hover:text-indigo-700">
                Learn More
                <svg aria-hidden="true" class="h-4 w-4" viewBox="0 0 20 20" fill="none">
                    <path d="M4 10h12m-5-5 5 5-5 5" stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"/>
                </svg>
            </a>
        </div>

        <div class="relative">
            <div aria-hidden="true" class="absolute -inset-8 rounded-[3rem] bg-gradient-to-br from-indigo-100 via-pink-50 to-cyan-100 blur-2xl"></div>
            <div class="relative rounded-[2rem] border border-slate-200 bg-slate-50 p-5 shadow-xl shadow-slate-900/5 sm:p-8">
                <div class="flex items-center justify-between gap-3">
                    <div>
                        <p class="text-xs font-semibold uppercase tracking-[0.16em] text-slate-400">Your reservations</p>
                        <h3 class="mt-1 text-xl font-semibold text-slate-950">Centralized Scheduling</h3>
                    </div>
                    <span class="flex h-11 w-11 items-center justify-center rounded-2xl bg-indigo-700 text-white">
                        <svg aria-hidden="true" class="h-5 w-5" viewBox="0 0 24 24" fill="none">
                            <path d="M5 12h14m-6-6 6 6-6 6" stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"/>
                        </svg>
                    </span>
                </div>
```

```
                <div class="mt-7 space-y-3">
                    <div class="rounded-2xl border border-slate-200 bg-white p-4">
                        <div class="flex items-start justify-between gap-3">
                            <div class="flex items-start gap-3">
                                <span class="flex h-10 w-10 shrink-0 items-center justify-center rounded-xl bg-indigo-50 text-indigo-700">
                                    <svg aria-hidden="true" class="h-5 w-5" viewBox="0 0 24 24" fill="none">
                                        <path d="M4 20V7.5L12 4l8 3.5V20M8 20v-5h8v5M2.5 20h19M8 9h.01M12 9h.01M16 9h.01" stroke="currentColor" stroke-width="1.7"
stroke-linecap="round" stroke-linejoin="round"/>
                                    </svg>
                                </span>
                                <div>
                                    <p class="font-semibold text-slate-900">Study room</p>
                                    <p class="mt-1 text-xs text-slate-500">Today · 9:00–10:00 AM</p>
                                </div>
                            </div>
                            <span class="rounded-full bg-indigo-50 px-2.5 py-1 text-[11px] font-semibold text-indigo-800">Pending</span>
                        </div>
                    </div>

                    <div class="rounded-2xl border border-slate-200 bg-white p-4">
                        <div class="flex items-start justify-between gap-3">
                            <div class="flex items-start gap-3">
                                <span class="flex h-10 w-10 shrink-0 items-center justify-center rounded-xl bg-emerald-50 text-emerald-700">
                                    <svg aria-hidden="true" class="h-5 w-5" viewBox="0 0 24 24" fill="none">
                                        <path d="M4 20V7.5L12 4l8 3.5V20M8 20v-5h8v5M2.5 20h19M8 9h.01M12 9h.01M16 9h.01" stroke="currentColor" stroke-width="1.7"
stroke-linecap="round" stroke-linejoin="round"/>
                                    </svg>
                                </span>
                                <div>
                                    <p class="font-semibold text-slate-900">Conference room</p>
                                    <p class="mt-1 text-xs text-slate-500">Thursday · 1:00–2:30 PM</p>
                                </div>
                            </div>
                            <span class="rounded-full bg-emerald-50 px-2.5 py-1 text-[11px] font-semibold text-emerald-800">Approved</span>
                        </div>
                    </div>

                    <div class="flex items-center gap-3 rounded-2xl border border-dashed border-slate-300 bg-white/70 p-4">
                        <span class="flex h-9 w-9 items-center justify-center rounded-xl bg-slate-100 text-slate-500">
                            <svg aria-hidden="true" class="h-5 w-5" viewBox="0 0 24 24" fill="none">
                                <path d="M12 5v14M5 12h14" stroke="currentColor" stroke-width="1.8" stroke-linecap="round"/>
                            </svg>
                        </span>
                        <span class="text-sm font-medium text-slate-600">Keep operations organized and transparent.</span>
                    </div>
                </div>

                <div class="mt-5 flex items-center gap-2 text-xs text-slate-500">
                    <span class="h-2 w-2 rounded-full bg-indigo-500"></span>
                    Improved Space Utilization
                </div>
            </div>
        </div>
    </div>
</section>
```

resources/views/contact.blade.php

```
@extends('layouts.app')

@section('title', 'Contact Us')

@section('content')
<section class="relative overflow-hidden bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50 py-16 sm:py-20">
    <div aria-hidden="true" class="pointer-events-none absolute -left-20 top-12 h-96 w-96 rounded-full bg-gradient-to-br from-pink-300 via-violet-200 to-cyan-200 opacity-70
blur-3xl"></div>
    <div aria-hidden="true" class="pointer-events-none absolute right-10 top-28 h-64 w-64 rounded-full bg-gradient-to-br from-indigo-300 via-cyan-200 to-slate-100 opacity-70
blur-2xl"></div>
    <div class="relative mx-auto max-w-6xl px-4 sm:px-6">
        <div class="grid gap-10 lg:grid-cols-[1.2fr_0.8fr] items-start">
            <div>
                <p class="mb-4 text-sm font-semibold uppercase tracking-[0.2em] text-indigo-700">Contact Us</p>
                <h1 class="mb-6 text-4xl font-semibold leading-tight tracking-tight text-slate-950 sm:text-5xl">Get in touch with the Campus Reserve team.</h1>
                <p class="text-lg leading-8 text-slate-600 mb-8">Have questions about reservations, account access, or campus scheduling? Find our campus location below or
```

```
    prepare a message for the Campus Reserve team.</p>

                <div class="space-y-6">
                    <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-6 shadow-sm">
                        <h2 class="mb-3 text-xl font-semibold text-slate-950">Campus location</h2>
                        <p class="text-slate-600 leading-7">FEU Roosevelt Marikina</p>
                        <p class="text-slate-600 leading-7">J.P. Rizal St., Lamuan, Marikina City</p>
                        <p class="text-slate-600 leading-7">Philippines</p>
                    </div>
                </div>
            </div>

            <div class="rounded-4xl border border-slate-200/80 bg-white/95 p-8 shadow-xl shadow-indigo-950/10">
                <div class="mb-8">
                    <p class="text-sm font-semibold uppercase tracking-[0.2em] text-indigo-700">Send a message</p>
                    <h2 class="mt-4 text-3xl font-semibold tracking-tight text-slate-950">How can we help?</h2>
                </div>

                <form class="space-y-6">
                    <div>
                        <label for="name" class="block text-sm font-medium text-slate-700 mb-2">Your name</label>
                        <input id="name" name="name" type="text" placeholder="Jane Doe" class="w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 py-3 text-slate-700
     focus:border-indigo-400 focus:outline-none focus:ring-2 focus:ring-indigo-100" />
                    </div>
                    <div>
                        <label for="email" class="block text-sm font-medium text-slate-700 mb-2">Email address</label>
                        <input id="email" name="email" type="email" placeholder="jane.doe@example.com" class="w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 py-3
     text-slate-700 focus:border-indigo-400 focus:outline-none focus:ring-2 focus:ring-indigo-100" />
                    </div>
                    <div>
                        <label for="subject" class="block text-sm font-medium text-slate-700 mb-2">Subject</label>
                        <input id="subject" name="subject" type="text" placeholder="Schedule request, account access, other" class="w-full rounded-2xl border border-slate-200
     bg-slate-50 px-4 py-3 text-slate-700 focus:border-indigo-400 focus:outline-none focus:ring-2 focus:ring-indigo-100" />
                    </div>
                    <div>
                        <label for="message" class="block text-sm font-medium text-slate-700 mb-2">Message</label>
                        <textarea id="message" name="message" rows="5" placeholder="Write your question here..." class="w-full rounded-2xl border border-slate-200 bg-slate-50
     px-4 py-3 text-slate-700 focus:border-indigo-400 focus:outline-none focus:ring-2 focus:ring-indigo-100"></textarea>
                    </div>
                    <button type="submit" class="w-full rounded-2xl bg-indigo-700 px-6 py-3 text-sm font-semibold text-white shadow transition hover:bg-indigo-800">Submit
    message</button>
                    <p class="text-sm text-slate-500">This form is interactive for demo purposes. Submission is not yet active.</p>
                </form>
            </div>
        </div>
    </div>
</section>
@endsection
```

resources/views/credentials/create.blade.php

```
@extends('layouts.app')

@section('title', 'Add Credential - Campus Reserve')

@section('content')
<!-- Create credential form: user can submit site login details to be encrypted and stored. -->
<div class="min-h-screen bg-gray-50">
    <div class="max-w-2xl mx-auto px-4 sm:px-6 lg:px-8 py-12">
        <!-- Header -->
        <div class="mb-8">
            <a href="{{ route('credentials.index') }}" class="inline-flex items-center text-sm text-gray-600 hover:text-gray-900 mb-4">
                <svg class="w-4 h-4 mr-1" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 19l-7-7 7-7"></path>
                </svg>
                Back to Credentials
            </a>
            <h1 class="text-3xl font-bold text-gray-900">Add New Credential</h1>
            <p class="text-gray-600 mt-2">Your credentials will be securely encrypted with AES-256.</p>
        </div>

        <!-- Form -->
        <div class="bg-white rounded-xl shadow-sm border border-gray-200 p-6">
            <form method="POST" action="{{ route('credentials.store') }}">
```

```
                @csrf

                <!-- Site Name -->
                <div class="mb-6">
                    <label for="site_name" class="block text-sm font-medium text-gray-700 mb-2">Site Name *</label>
                    <input type="text" name="site_name" id="site_name" value="{{ old('site_name') }}" required
                        class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all @error('site_name') border-red-500 @enderror"
                        placeholder="e.g., Gmail, GitHub, Netflix">
                    @error('site_name')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Site URL -->
                <div class="mb-6">
                    <label for="site_url" class="block text-sm font-medium text-gray-700 mb-2">Site URL (Optional)</label>
                    <input type="text" name="site_url" id="site_url" value="{{ old('site_url') }}"
                        class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all @error('site_url') border-red-500 @enderror"
                        placeholder="e.g., https://gmail.com">
                    @error('site_url')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Username / Email -->
                <div class="mb-6">
                    <label for="username" class="block text-sm font-medium text-gray-700 mb-2">Username / Email *</label>
                    <input type="text" name="username" id="username" value="{{ old('username') }}" required
                        class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all @error('username') border-red-500 @enderror"
                        placeholder="Your login username or email">
                    @error('username')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Password -->
                <div class="mb-6">
                    <label for="password" class="block text-sm font-medium text-gray-700 mb-2">Password *</label>
                    <div class="relative">
                        <input type="password" name="password" id="password" required
                            class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all pr-12 @error('password') border-red-500 @enderror"
                            placeholder="Enter the password for this site">
                    </div>
                    @error('password')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Notes -->
                <div class="mb-8">
                    <label for="notes" class="block text-sm font-medium text-gray-700 mb-2">Notes (Optional)</label>
                    <textarea name="notes" id="notes" rows="3"
                        class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all resize-none @error('notes') border-red-500 @enderror"
                        placeholder="Any additional notes about this credential">{{ old('notes') }}</textarea>
                    @error('notes')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Submit -->
                <div class="flex items-center justify-end gap-4">
                    <a href="{{ route('credentials.index') }}" class="px-6 py-3 text-gray-700 hover:text-gray-900 transition-colors">
                        Cancel
                    </a>
                    <button type="submit" class="px-6 py-3 bg-black text-white rounded-lg hover:bg-gray-800 transition-colors">
                        Save Credential
                    </button>
                </div>
            </form>
```

```
        </div>

        <!-- Security Notice -->
        <div class="mt-6 p-4 bg-green-50 border border-green-200 rounded-lg">
            <div class="flex items-start">
                <svg class="w-5 h-5 text-green-600 mt-0.5 mr-3" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9 12l2 2 4-4m5.618-4.016A11.955 11.955 0 0112 2.944a11.955 11.955 0 01-8.618 3.04A12.02 12.02 0 003 9c0 5.591 3.824 10.29 9 11.622 5.176-1.332 9-6.03 9-11.622 0-1.042-.133-2.052-.382-3.016z"></path>
                </svg>
                <div>
                    <h4 class="text-sm font-semibold text-green-900">Your data is secure</h4>
                    <p class="text-sm text-green-700 mt-1">All fields are encrypted using AES-256-CBC before being stored in the database.</p>
                </div>
            </div>
        </div>
    </div>
</div>

@endsection
```

resources/views/credentials/edit.blade.php

```
@extends('layouts.app')

@section('title', 'Edit Credential - Campus Reserve')

@section('content')
<!-- Edit credential form: user can update stored credential data with encryption on save. -->
<div class="min-h-screen bg-gray-50">
    <div class="max-w-2xl mx-auto px-4 sm:px-6 lg:px-8 py-12">
        <!-- Header -->
        <div class="mb-8">
            <a href="{{ route('credentials.index') }}" class="inline-flex items-center text-sm text-gray-600 hover:text-gray-900 mb-4">
                <svg class="w-4 h-4 mr-1" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 19l-7-7 7-7"></path>
                </svg>
                Back to Credentials
            </a>
            <h1 class="text-3xl font-bold text-gray-900">Edit Credential</h1>
            <p class="text-gray-600 mt-2">Update your saved credential for {{ $credential->site_name }}.</p>
        </div>

        <!-- Form -->
        <div class="bg-white rounded-xl shadow-sm border border-gray-200 p-6">
            <form method="POST" action="{{ route('credentials.update', $credential) }}">
                @csrf
                @method('PUT')

                <!-- Site Name -->
                <div class="mb-6">
                    <label for="site_name" class="block text-sm font-medium text-gray-700 mb-2">Site Name *</label>
                    <input type="text" name="site_name" id="site_name" value="{{ old('site_name', $credential->site_name) }}" required
                        class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all @error('site_name') border-red-500 @enderror"
                        placeholder="e.g., Gmail, GitHub, Netflix">
                    @error('site_name')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Site URL -->
                <div class="mb-6">
                    <label for="site_url" class="block text-sm font-medium text-gray-700 mb-2">Site URL (Optional)</label>
                    <input type="text" name="site_url" id="site_url" value="{{ old('site_url', $credential->site_url) }}"
                        class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all @error('site_url') border-red-500 @enderror"
                        placeholder="e.g., https://gmail.com">
                    @error('site_url')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Username / Email -->
                <div class="mb-6">
```

```
                    <label for="username" class="block text-sm font-medium text-gray-700 mb-2">Username / Email *</label>
                    <input type="text" name="username" id="username" value="{{ old('username', $credential->username) }}" required
                        class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all @error('username') border-red-500 @enderror"
                        placeholder="Your login username or email">
                    @error('username')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Password -->
                <div class="mb-6">
                    <label for="password" class="block text-sm font-medium text-gray-700 mb-2">Password *</label>
                    <div class="relative">
                        <input type="password" name="password" id="password" value="{{ old('password', $credential->password) }}" required
                            class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all pr-12 @error('password') border-red-500 @enderror"
                            placeholder="Enter the password for this site">
                    </div>
                    <p class="mt-1 text-xs text-gray-500">Leave unchanged to keep the current password, or enter a new one to update it.</p>
                    @error('password')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Notes -->
                <div class="mb-8">
                    <label for="notes" class="block text-sm font-medium text-gray-700 mb-2">Notes (Optional)</label>
                    <textarea name="notes" id="notes" rows="3"
                        class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-black focus:border-transparent transition-all resize-none @error('notes') border-red-500 @enderror"
                        placeholder="Any additional notes about this credential">{{ old('notes', $credential->notes) }}</textarea>
                    @error('notes')
                    <p class="mt-1 text-sm text-red-500">{{ $message }}</p>
                    @enderror
                </div>

                <!-- Submit -->
                <div class="flex items-center justify-end gap-4">
                    <a href="{{ route('credentials.index') }}" class="px-6 py-3 text-gray-700 hover:text-gray-900 transition-colors">
                        Cancel
                    </a>
                    <button type="submit" class="px-6 py-3 bg-black text-white rounded-lg hover:bg-gray-800 transition-colors">
                        Update Credential
                    </button>
                </div>
            </form>
        </div>

        <!-- Danger Zone -->
        <div class="mt-8 p-6 bg-red-50 border border-red-200 rounded-xl">
            <h3 class="text-lg font-semibold text-red-900 mb-2">Danger Zone</h3>
            <p class="text-sm text-red-700 mb-4">Once you delete this credential, there is no going back. Please be certain.</p>
            <form method="POST" action="{{ route('credentials.destroy', $credential) }}" onsubmit="return confirm('Are you sure you want to delete this credential? This action cannot be undone.');">
                @csrf
                @method('DELETE')
                <button type="submit" class="px-4 py-2 bg-red-600 text-white rounded-lg hover:bg-red-700 transition-colors">
                    Delete This Credential
                </button>
            </form>
        </div>
    </div>
</div>

@endsection
```

resources/views/credentials/index.blade.php

```
@extends('layouts.app')

@section('title', 'Saved Credentials - Campus Reserve')

@section('content')
```

```
<!-- Saved credentials listing page, showing user-owned entries and actions for view/edit/delete. -->
<div class="min-h-screen bg-gray-50">
    <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-12">
        <!-- Header -->
        <div class="flex items-center justify-between mb-8">
            <div>
                <h1 class="text-3xl font-bold text-gray-900">Saved Credentials</h1>
                <p class="text-gray-600 mt-2">Securely store and manage your login credentials with AES encryption.</p>
            </div>
            <a href="{{ route('credentials.create') }}" class="inline-flex items-center px-4 py-2 bg-black text-white rounded-lg hover:bg-gray-800 transition-colors">
                <svg class="w-5 h-5 mr-2" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12 4v16m8-8H4"></path>
                </svg>
                Add New
            </a>
        </div>

        <!-- Success Message -->
        @if(session('success'))
        <div class="mb-6 p-4 bg-green-50 border border-green-200 rounded-lg">
            <div class="flex items-center">
                <svg class="w-5 h-5 text-green-600 mr-2" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9 12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z"></path>
                </svg>
                <span class="text-green-800">{{ session('success') }}</span>
            </div>
        </div>
        @endif

        <!-- Credentials List -->
        @if($credentials->count() > 0)
        <div class="bg-white rounded-xl shadow-sm border border-gray-200 overflow-hidden">
            <div class="divide-y divide-gray-200">
                @foreach($credentials as $credential)
                <div class="p-6 hover:bg-gray-50 transition-colors">
                    <div class="flex items-center justify-between">
                        <div class="flex items-center">
                            <div class="w-12 h-12 bg-gray-100 rounded-lg flex items-center justify-center mr-4">
                                <svg class="w-6 h-6 text-gray-500" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 7a2 2 0 012 2m4 0a6 6 0 01-7.743 5.743L11 17H9v2H7v2H4a1 1 0
01-1-1v-2.586a1 1 0 01.293-.707l5.964-5.964A6 6 0 1121 9z"></path>
                                </svg>
                            </div>
                            <div>
                                <h3 class="font-semibold text-gray-900">{{ $credential->site_name }}</h3>
                                <p class="text-sm text-gray-500">{{ $credential->username }}</p>
                                @if($credential->site_url)
                                <a href="{{ $credential->site_url }}" target="_blank" class="text-xs text-blue-600 hover:underline">{{ $credential->site_url }}</a>
                                @endif
                            </div>
                        </div>
                        <div class="flex items-center gap-2">
                            <!-- Copy Password Button -->
                            <button onclick="copyPassword({{ $credential->id }})" class="p-2 text-gray-500 hover:text-gray-700 hover:bg-gray-100 rounded-lg transition-colors"
 title="Copy Password">
                                <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M8 5H6a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2v-1M8 5a2 2 0
002 2h2a2 2 0 002-2M8 5a2 2 0 012-2h2a2 2 0 012 2m0 0h2a2 2 0 012 2v3m2 4H10m0 0l3-3m-3 3l3 3"></path>
                                </svg>
                            </button>
                            <!-- View Button -->
                            <a href="{{ route('credentials.show', $credential) }}" class="p-2 text-gray-500 hover:text-gray-700 hover:bg-gray-100 rounded-lg
transition-colors" title="View">
                                <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 12a3 3 0 11-6 0 3 3 0 016 0z"></path>
                                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M2.458 12C3.732 7.943 7.523 5 12 5c4.478 0 8.268 2.943 9.542
7-1.274 4.057-5.064 7-9.542 7-4.477 0-8.268-2.943-9.542-7z"></path>
                                </svg>
                            </a>
                            <!-- Edit Button -->
                            <a href="{{ route('credentials.edit', $credential) }}" class="p-2 text-gray-500 hover:text-gray-700 hover:bg-gray-100 rounded-lg
transition-colors" title="Edit">
                                <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M11 5H6a2 2 0 00-2 2v11a2 2 0 002 2h11a2 2 0
```

```
002-2v-5m-1.414-9.414a2 2 0 112.828 2.828L11.828 15H9v-2.828l8.586-8.586z"></path>
                                </svg>
                            </a>
                            <!-- Delete Button -->
                            <form method="POST" action="{{ route('credentials.destroy', $credential) }}" onsubmit="return confirm('Are you sure you want to delete this
credential?');" class="inline">
                                @csrf
                                @method('DELETE')
                                <button type="submit" class="p-2 text-red-500 hover:text-red-700 hover:bg-red-50 rounded-lg transition-colors" title="Delete">
                                    <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0
01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16"></path>
                                    </svg>
                                </button>
                            </form>
                        </div>
                    </div>
                </div>
                @endforeach
            </div>
        </div>
        @else
        <!-- Empty State -->
        <div class="bg-white rounded-xl shadow-sm border border-gray-200 p-12 text-center">
            <div class="w-16 h-16 bg-gray-100 rounded-full flex items-center justify-center mx-auto mb-4">
                <svg class="w-8 h-8 text-gray-400" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12 15v2m-6 4h12a2 2 0 002-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002
2zm10-10V7a4 4 0 00-8 0v4h8z"></path>
                </svg>
            </div>
            <h3 class="text-lg font-semibold text-gray-900 mb-2">No saved credentials</h3>
            <p class="text-gray-500 mb-6">Start by adding your first login credential. All data is AES encrypted.</p>
            <a href="{{ route('credentials.create') }}" class="inline-flex items-center px-4 py-2 bg-black text-white rounded-lg hover:bg-gray-800 transition-colors">
                <svg class="w-5 h-5 mr-2" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12 4v16m8-8H4"></path>
                </svg>
                Add Your First Credential
            </a>
        </div>
        @endif

        <!-- Security Notice -->
        <div class="mt-8 p-4 bg-blue-50 border border-blue-200 rounded-lg">
            <div class="flex items-start">
                <svg class="w-5 h-5 text-blue-600 mt-0.5 mr-3" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M13 16h-1v-4h-1m1-4h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z"></path>
                </svg>
                <div>
                    <h4 class="text-sm font-semibold text-blue-900">AES-256 Encryption</h4>
                    <p class="text-sm text-blue-700 mt-1">All your credentials are encrypted using AES-256-CBC encryption. Only you can access your saved passwords.</p>
                </div>
            </div>
        </div>
    </div>
</div>

<!-- Copy Password Script -->
<script>
async function copyPassword(credentialId) {
    try {
        const response = await fetch(`/credentials/${credentialId}/password`);
        const data = await response.json();

        if (data.password) {
            await navigator.clipboard.writeText(data.password);
            alert('Password copied to clipboard!');
        } else {
            alert('Failed to copy password');
        }
    } catch (error) {
        alert('Failed to copy password');
    }
}
</script>
```

```
@endsection
```

resources/views/credentials/show.blade.php

```
@extends('layouts.app')

@section('title', $credential->site_name . ' - Campus Reserve')

@section('content')
<!-- View credential details page. Password display and clipboard actions are handled with JavaScript. -->
<div class="min-h-screen bg-gray-50">
    <div class="max-w-2xl mx-auto px-4 sm:px-6 lg:px-8 py-12">
        <!-- Header -->
        <div class="mb-8">
            <a href="{{ route('credentials.index') }}" class="inline-flex items-center text-sm text-gray-600 hover:text-gray-900 mb-4">
                <svg class="w-4 h-4 mr-1" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 19l-7-7 7-7"></path>
                </svg>
                Back to Credentials
            </a>
            <div class="flex items-center justify-between">
                <h1 class="text-3xl font-bold text-gray-900">{{ $credential->site_name }}</h1>
                <div class="flex items-center gap-2">
                    <a href="{{ route('credentials.edit', $credential) }}" class="px-4 py-2 text-gray-700 border border-gray-300 rounded-lg hover:bg-gray-50
transition-colors">
                        Edit
                    </a>
                    <form method="POST" action="{{ route('credentials.destroy', $credential) }}" onsubmit="return confirm('Are you sure you want to delete this
credential?');" class="inline">
                        @csrf
                        @method('DELETE')
                        <button type="submit" class="px-4 py-2 text-red-600 border border-red-300 rounded-lg hover:bg-red-50 transition-colors">
                            Delete
                        </button>
                    </form>
                </div>
            </div>
        </div>

        <!-- Credential Details -->
        <div class="bg-white rounded-xl shadow-sm border border-gray-200 overflow-hidden">
            <!-- Site URL -->
            @if($credential->site_url)
            <div class="p-6 border-b border-gray-200">
                <label class="block text-sm font-medium text-gray-500 mb-1">Site URL</label>
                <a href="{{ $credential->site_url }}" target="_blank" class="text-blue-600 hover:underline flex items-center">
                    {{ $credential->site_url }}
                    <svg class="w-4 h-4 ml-1" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M10 6H6a2 2 0 00-2 2v10a2 2 0 002 2h10a2 2 0 002-2v-4M14 4h6m0 0v6m0-6L10
14"></path>
                    </svg>
                </a>
            </div>
            @endif

            <!-- Username -->
            <div class="p-6 border-b border-gray-200">
                <label class="block text-sm font-medium text-gray-500 mb-1">Username / Email</label>
                <div class="flex items-center justify-between">
                    <span class="text-gray-900 font-mono">{{ $credential->username }}</span>
                    <button onclick="copyToClipboard('{{ $credential->username }}')" class="p-2 text-gray-500 hover:text-gray-700 hover:bg-gray-100 rounded-lg
transition-colors" title="Copy">
                        <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M8 5H6a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2v-1M8 5a2 2 0 002
2 0 002-2M8 5a2 2 0 012-2h2a2 2 0 012 2m0 0h2a2 2 0 012 2v3m2 4H10m0 0l3-3m-3 3l3 3"></path>
                        </svg>
                    </button>
                </div>
            </div>

            <!-- Password -->
            <div class="p-6 border-b border-gray-200">
                <label class="block text-sm font-medium text-gray-500 mb-1">Password</label>
                <div class="flex items-center justify-between">
```

```
                    <div class="flex items-center gap-3">
                        <span id="password-display" class="text-gray-900 font-mono">••••••••••••</span>
                        <button type="button" onclick="togglePasswordVisibility()" class="rounded p-1 text-gray-500 transition hover:text-gray-800 focus:outline-none focus:ring-2 focus:ring-indigo-500" id="toggle-btn" aria-label="Show password" aria-pressed="false" title="Show password">
                            <svg data-password-eye-open class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 12a3 3 0 11-6 0 3 3 0 016 0z"></path>
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M2.458 12C3.732 7.943 7.523 5 12 5c4.478 0 8.268 2.943 9.542 7-1.274 4.057-5.064 7-9.542 7-4.477 0-8.268-2.943-9.542-7z"></path>
                            </svg>
                            <svg data-password-eye-closed class="hidden h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M13.875 18.825A10.05 10.05 0 0112 19c-4.478 0-8.268-2.943-9.543-7a9.97 9.97 0 011.563-3.029m5.858.908a3 3 0 114.243 4.243M9.878 9.878l4.242 4.242M9.88 9.88l-3.29-3.29m7.532 7.532l3.29 3.29M3 3l3.59 3.59m0 0A9.953 9.953 0 0112 5c4.478 0 8.268 2.943 9.543 7a10.025 10.025 0 01-4.132 5.411m0 0L21 21"></path>
                            </svg>
                        </button>
                    </div>
                    <button onclick="copyPassword()" class="p-2 text-gray-500 hover:text-gray-700 hover:bg-gray-100 rounded-lg transition-colors" title="Copy Password">
                        <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M8 5H6a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2v-1M8 5a2 2 0 002 2h2a2 2 0 002-2M8 5a2 2 0 012-2h2a2 2 0 012 2m0 0h2a2 2 0 012 2v3m2 4H10m0 0l3-3m-3 3l3 3"></path>
                        </svg>
                    </button>
                </div>
            </div>

            <!-- Notes -->
            @if($credential->notes)
            <div class="p-6">
                <label class="block text-sm font-medium text-gray-500 mb-1">Notes</label>
                <p class="text-gray-900 whitespace-pre-wrap">{{ $credential->notes }}</p>
            </div>
            @endif
        </div>

        <!-- Metadata -->
        <div class="mt-6 p-4 bg-gray-100 rounded-lg">
            <div class="flex items-center justify-between text-sm text-gray-500">
                <span>Created: {{ $credential->created_at->format('M d, Y h:i A') }}</span>
                <span>Updated: {{ $credential->updated_at->format('M d, Y h:i A') }}</span>
            </div>
        </div>

        <!-- Security Notice -->
        <div class="mt-6 p-4 bg-blue-50 border border-blue-200 rounded-lg">
            <div class="flex items-start">
                <svg class="w-5 h-5 text-blue-600 mt-0.5 mr-3" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12 15v2m-6 4h12a2 2 0 002-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002 2zm10-10V7a4 4 0 00-8 0v4h8z"></path>
                </svg>
                <div>
                    <h4 class="text-sm font-semibold text-blue-900">Encrypted Storage</h4>
                    <p class="text-sm text-blue-700 mt-1">This credential is stored with AES-256-CBC encryption. The password is decrypted only when you view or copy it.</p>
                </div>
            </div>
        </div>
    </div>
</div>

<script>
let passwordVisible = false;
let cachedPassword = null;

async function fetchPassword() {
    if (cachedPassword) return cachedPassword;

    try {
        const response = await fetch('/credentials/{{ $credential->id }}/password');
        const data = await response.json();
        cachedPassword = data.password;
        return cachedPassword;
    } catch (error) {
        return null;
    }
}
```

```

async function togglePasswordVisibility() {
    const display = document.getElementById('password-display');
    const btn = document.getElementById('toggle-btn');
    const isVisible = passwordVisible;

    if (!isVisible) {
        const password = await fetchPassword();
        if (password) {
            display.textContent = password;
            passwordVisible = true;
        }
    } else {
        display.textContent = '••••••••••••';
        passwordVisible = false;
    }

    const action = passwordVisible ? 'Hide' : 'Show';
    btn.setAttribute('aria-label', `${action} password`);
    btn.setAttribute('aria-pressed', String(passwordVisible));
    btn.title = `${action} password`;
    btn.querySelector('[data-password-eye-open]').classList.toggle('hidden', passwordVisible);
    btn.querySelector('[data-password-eye-closed]').classList.toggle('hidden', !passwordVisible);
}

async function copyPassword() {
    const password = await fetchPassword();
    if (password) {
        await navigator.clipboard.writeText(password);
        alert('Password copied to clipboard!');
    } else {
        alert('Failed to copy password');
    }
}

function copyToClipboard(text) {
    navigator.clipboard.writeText(text);
    alert('Copied to clipboard!');
}
</script>
@endsection
```

resources/views/debug/decrypt-otp-verify.blade.php

```
@extends('layouts.app')

@section('title', 'Verify OTP - Decrypt Utility')

@section('content')
<!-- OTP verification modal for decrypt utility - similar to login 2FA flow -->
<main class="flex min-h-[calc(100vh-73px)]">
    <div class="w-full md:w-1/2 flex flex-col items-center justify-center px-8 py-12">
        <div class="w-full max-w-md">
            <div class="flex justify-center mb-6">
                <div class="flex items-center border border-black px-2 py-1">
                    <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                    <span class="bg-black text-white text-xs font-medium px-1 ml-1">RESERVE</span>
                </div>
            </div>

            <h1 class="text-3xl font-bold text-center text-gray-900 mb-8">
                VERIFY YOUR ACCESS
            </h1>

            <div class="text-center mb-8">
                <p class="text-gray-600 mb-2">We sent a 6-digit code to your admin email.</p>
                <p class="text-sm text-gray-500">Code expires in: <span id="timer" class="font-semibold text-red-600">10s</span></p>
            </div>

            @if (session('error'))
                <div class="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded mb-6">
                    {{ session('error') }}
                </div>
            @endif
```

```

            <form method="POST" action="{{ route('debug.decrypt.otp.verify') }}" class="space-y-6">
                @csrf

                <div class="space-y-2">
                    <label for="otp" class="block text-sm text-gray-700">Verification Code</label>
                    <input
                        id="otp"
                        name="otp"
                        type="text"
                        inputmode="numeric"
                        pattern="[0-9]{6}"
                        maxlength="6"
                        autocomplete="one-time-code"
                        required
                        class="w-full px-4 py-2 rounded-full border border-gray-300 focus:outline-none focus:ring-2 focus:ring-black focus:border-transparent text-center
    text-lg tracking-widest"
                        placeholder="000000"
                    >
                    @error('otp')
                        <p class="text-red-500 text-xs mt-1">{{ $message }}</p>
                    @enderror
                </div>

                <div class="flex flex-col sm:flex-row gap-3">
                    <button
                        type="submit"
                        class="flex-1 px-8 py-2 bg-black text-white rounded-md hover:bg-gray-800 transition-colors"
                    >
                        Verify
                    </button>
                    <button
                        type="button"
                        onclick="document.querySelector('input[name=otp]').value = ''"
                        class="flex-1 px-8 py-2 bg-gray-200 text-gray-700 rounded-md hover:bg-gray-300 transition-colors"
                    >
                        Clear
                    </button>
                </div>
            </form>

            <div class="mt-6 text-center">
                <form method="POST" action="{{ route('debug.decrypt.otp.resend') }}" class="inline">
                    @csrf
                    <button type="submit" id="resendBtn" class="text-sm text-blue-600 hover:underline">
                        Resend Code
                    </button>
                </form>
                <p class="text-xs text-gray-500 mt-2">
                    Didn't receive the code? Check spam folder or resend above.
                </p>
                <p class="mt-4">
                    <a href="/admin/dashboard" class="text-sm text-gray-600 hover:underline">Back to Dashboard</a>
                </p>
            </div>
        </div>
    </div>

    <div class="hidden md:block w-1/2 relative overflow-hidden">
        <div class="absolute inset-0 bg-linear-to-br from-purple-300 via-pink-200 to-red-200"></div>
    </div>
</main>

<script>
    let timeLeft = 10; // 10 seconds
    const timerEl = document.getElementById('timer');
    const resendBtn = document.getElementById('resendBtn');

    const interval = setInterval(() => {
        timeLeft--;
        if (timerEl && timeLeft > 0) {
            timerEl.textContent = timeLeft + 's';
        } else {
            clearInterval(interval);
```

```
            if (timerEl) {
                timerEl.innerHTML = '<span class="text-red-600">Expired</span>';
                document.querySelector('button[type="submit"]').disabled = true;
            }
        }
    }, 1000);
</script>
@endsection
```

resources/views/debug/decrypt-otp.blade.php

```
@extends('layouts.app')

@section('title', 'Verify Decrypt Access - Campus Reserve')

@section('content')
<!-- OTP verification modal for decrypt utility access -->
<main class="flex min-h-[calc(100vh-73px)]">
    <div class="w-full md:w-1/2 flex flex-col items-center justify-center px-8 py-12">
        <div class="w-full max-w-md">
            <div class="flex justify-center mb-6">
                <div class="flex items-center border border-black px-2 py-1">
                    <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                    <span class="bg-black text-white text-xs font-medium px-1 ml-1">RESERVE</span>
                </div>
            </div>

            <h1 class="text-3xl font-bold text-center text-gray-900 mb-8">
                SECURE ACCESS
            </h1>

            <div class="text-center mb-8">
                <p class="text-gray-600 mb-2">You are accessing the admin decrypt utility.</p>
                <p class="text-sm text-gray-500">A verification code will be sent to your registered email.</p>
            </div>

            @if (session('error'))
                <div class="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded mb-6">
                    {{ session('error') }}
                </div>
            @endif

            <form method="POST" action="{{ route('debug.decrypt.otp') }}" class="space-y-6">
                @csrf

                <div class="text-center">
                    <p class="text-sm text-gray-600 mb-4">Ready to proceed?</p>
                    <button
                        type="submit"
                        class="w-full px-8 py-3 bg-black text-white rounded-full hover:bg-gray-800 transition-colors font-semibold"
                    >
                        Send OTP
                    </button>
                </div>
            </form>

            <div class="mt-6 text-center">
                <a href="/admin/dashboard" class="text-sm text-gray-600 hover:underline">Back to Dashboard</a>
            </div>
        </div>
    </div>

    <div class="hidden md:block w-1/2 relative overflow-hidden">
        <div class="absolute inset-0 bg-linear-to-br from-purple-300 via-pink-200 to-red-200"></div>
    </div>
</main>
@endsection
```

resources/views/debug/decrypt.blade.php

```
@extends('layouts.app')

@section('title', 'Decrypt User Data - Campus Reserve')
```

```
4
5  @section('content')
6  <main class="min-h-[calc(100vh-73px)] bg-slate-50 py-10 px-4 md:px-8 xl:px-16">
7      <div class="mx-auto max-w-3xl rounded-3xl bg-white p-8 shadow-lg">
8          <div class="flex items-center justify-between mb-4">
9              <div>
10                 <h1 class="text-3xl font-bold text-slate-900">Decrypt User Data</h1>
11                 <p class="mt-2 text-sm text-slate-600">Select a registered member to view and decrypt their stored information.</p>
12             </div>
13             <a href="/admin/dashboard" class="text-sm text-gray-600 hover:underline">← Back to Admin</a>
14         </div>
15
16         <form method="POST" action="{{ route('debug.decrypt') }}" class="mt-8 space-y-6">
17             @csrf
18
19             <!-- Member Selection Dropdown -->
20             <div>
21                 <label for="member" class="block text-sm font-medium text-slate-700">1. Select Registered Member</label>
22                 <select id="member" name="user_id" required class="mt-1 block w-full rounded-2xl border border-slate-300 bg-slate-50 px-4 py-3 text-sm text-slate-900
   focus:border-black focus:ring-black">
23                     <option value="">-- Choose a member --</option>
24                     @foreach($users as $user)
25                         <option value="{{ $user->id }}" {{ (isset($selectedUserId) && $selectedUserId == $user->id) ? 'selected' : '' }}>{{ $user->name }} ({{ $user->email
   }})</option>
26                     @endforeach
27                 </select>
28                 @error('user_id')
29                     <p class="mt-2 text-sm text-rose-600">{{ $message }}</p>
30                 @enderror
31             </div>
32
33             <!-- Email Field (Auto-filled, Read-only) -->
34             <div>
35                 <label for="email" class="block text-sm font-medium text-slate-700">2. Email</label>
36                 <input type="email" id="email" readonly class="mt-1 block w-full rounded-2xl border border-slate-300 bg-slate-100 px-4 py-3 text-sm text-slate-600
   cursor-not-allowed">
37             </div>
38
39             <!-- Password Field (Auto-filled, Read-only) -->
40             <div>
41                 <label for="password" class="block text-sm font-medium text-slate-700">3. Password (Encrypted)</label>
42                 <input type="text" id="password" readonly class="mt-1 block w-full rounded-2xl border border-slate-300 bg-slate-100 px-4 py-3 text-sm text-slate-600
   cursor-not-allowed" placeholder="(Hashed - Not encrypted)">
43             </div>
44
45             <!-- Phone Number Field (Auto-filled, Read-only) -->
46             <div>
47                 <label for="phone" class="block text-sm font-medium text-slate-700">4. Phone Number</label>
48                 <input type="text" id="phone" readonly class="mt-1 block w-full rounded-2xl border border-slate-300 bg-slate-100 px-4 py-3 text-sm text-slate-600
   cursor-not-allowed">
49             </div>
50
51             <button type="submit" class="inline-flex items-center justify-center rounded-full bg-black px-6 py-3 text-sm font-semibold text-white hover:bg-slate-800">Decrypt
   & View</button>
52         </form>
53
54         <!-- Decrypted Results Display -->
55         @if(isset($decrypted))
56             <div class="mt-10 rounded-3xl border-2 border-green-200 bg-green-50 p-6">
57                 <h2 class="text-lg font-semibold text-green-900 mb-4">  Decrypted User Information</h2>
58                 <pre class="text-sm text-slate-800 whitespace-pre-wrap wrap-break-word leading-relaxed">User: {{ $decrypted['user'] }}
59 Email: {{ $decrypted['email'] }}
60 Password: (Hashed - Not decryptable)
61 Phone Number: {{ $decrypted['phone'] }}</pre>
62             </div>
63         @endif
64
65         <div class="mt-6 text-xs text-slate-500">
66             <p><strong>  Session Status:</strong> OTP verified for this session. Safe to decrypt multiple users.</p>
67             <p class="mt-2"><strong>Note:</strong> Password fields are bcrypt-hashed and cannot be decrypted. Email and phone are AES-encrypted and shown above when
   decrypted.</p>
68         </div>
69     </div>
70 </main>
```

```

<script>
    // Auto-fill fields when member is selected
    document.getElementById('member').addEventListener('change', async function(e) {
        const userId = e.target.value;
        if (!userId) {
            document.getElementById('email').value = '';
            document.getElementById('password').value = '';
            document.getElementById('phone').value = '';
            return;
        }

        try {
            const response = await fetch(`{{ route('debug.decrypt.user-details') }}?user_id=${userId}`);
            const data = await response.json();
            document.getElementById('email').value = data.email;
            document.getElementById('password').value = data.email_encrypted ? '[Encrypted]' : '(None)';
            document.getElementById('phone').value = data.phone;
        } catch (error) {
            console.error('Error fetching user details:', error);
        }
    });

    // Trigger auto-fill on page load if user is already selected
    if (document.getElementById('member').value) {
        document.getElementById('member').dispatchEvent(new Event('change'));
    }
</script>
@endsection
```

resources/views/errors/403.blade.php

```
@extends('layouts.app')

@section('title', 'Access Denied')

@section('content')
<div class="min-h-screen flex items-center justify-center px-4">
    <div class="text-center">
        <h1 class="text-6xl font-bold text-gray-800 mb-4">403</h1>
        <h2 class="text-2xl font-semibold text-gray-600 mb-4">Access Denied</h2>
        <p class="text-gray-500 mb-8">You don't have permission to access this resource.</p>
        <a href="/" class="inline-block px-6 py-3 bg-black text-white font-semibold rounded hover:bg-gray-800">
            Go Home
        </a>
    </div>
</div>
@endsection
```

resources/views/errors/404.blade.php

```
@extends('layouts.app')

@section('title', 'Page Not Found')

@section('content')
<div class="min-h-screen flex items-center justify-center px-4">
    <div class="text-center">
        <h1 class="text-6xl font-bold text-gray-800 mb-4">404</h1>
        <h2 class="text-2xl font-semibold text-gray-600 mb-4">Page Not Found</h2>
        <p class="text-gray-500 mb-8">The page you're looking for doesn't exist.</p>
        <a href="/" class="inline-block px-6 py-3 bg-black text-white font-semibold rounded hover:bg-gray-800">
            Go Home
        </a>
    </div>
</div>
@endsection
```

resources/views/errors/429.blade.php

```
@extends('layouts.app')

@section('title', 'Too Many Requests')
```

```

@section('content')
<div class="min-h-screen flex items-center justify-center px-4">
    <div class="text-center">
        <h1 class="text-6xl font-bold text-gray-800 mb-4">429</h1>
        <h2 class="text-2xl font-semibold text-gray-600 mb-4">Too Many Requests</h2>
        <p class="text-gray-500 mb-8">You're making requests too quickly. Please wait before trying again.</p>
        <a href="/" class="inline-block px-6 py-3 bg-black text-white font-semibold rounded hover:bg-gray-800">
            Go Home
        </a>
    </div>
</div>
@endsection
```

resources/views/errors/500.blade.php

```
@extends('layouts.app')

@section('title', 'Server Error')

@section('content')
<div class="min-h-screen flex items-center justify-center px-4">
    <div class="text-center">
        <h1 class="text-6xl font-bold text-gray-800 mb-4">500</h1>
        <h2 class="text-2xl font-semibold text-gray-600 mb-4">Internal Server Error</h2>
        <p class="text-gray-500 mb-8">An unexpected error occurred. Our team has been notified.</p>
        <a href="/" class="inline-block px-6 py-3 bg-black text-white font-semibold rounded hover:bg-gray-800">
            Go Home
        </a>
    </div>
</div>
@endsection
```

resources/views/layouts/app.blade.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="csrf-token" content="{{ csrf_token() }}">
    <title>@yield('title', 'Campus Reserve')</title>
    <link rel="icon" type="image/svg+xml" href="{{ asset('favicon.svg') }}">

    @vite(['resources/css/app.css', 'resources/js/app.js'])
    @stack('styles')
</head>
<body class="min-h-screen bg-white text-slate-900 transition-colors duration-200">
    <!-- Application header and global navigation bar -->
    <header class="relative z-40 flex flex-wrap items-center justify-between gap-x-4 gap-y-3 border-b border-gray-200 bg-white px-4 py-3 transition-colors duration-200 sm:px-6 sm:py-4">
        <a href="/" class="flex items-center">
            <div class="flex items-center rounded-xl border border-amber-300 bg-white px-2 py-1 shadow-sm">
                <span class="text-xs font-medium tracking-wide">CAMPUS</span>
                <span class="ml-1 rounded-md bg-green-800 px-1 text-xs font-medium text-white">RESERVE</span>
            </div>
        </a>

        <nav class="hidden items-center gap-8 md:flex">
            <!-- Main site navigation links. The active route is highlighted dynamically. -->
            <a href="/" @if(request()->path() === '/') aria-current="page" @endif class="relative text-sm font-medium transition-colors hover:text-green-900 {{ request()->path() === '/' ? 'text-green-900 after:absolute after:-bottom-2 after:left-0 after:h-0.5 after:w-full after:rounded-full after:bg-green-900' : 'text-gray-700' }}">HOME</a>
            <a href="/reserve" @if(request()->is('reserve')) aria-current="page" @endif class="relative text-sm font-medium transition-colors hover:text-green-900 {{ request()->is('reserve') ? 'text-green-900 after:absolute after:-bottom-2 after:left-0 after:h-0.5 after:w-full after:rounded-full after:bg-green-900' : 'text-gray-700' }}">RESERVE</a>
            <a href="/calendar" @if(request()->is('calendar')) aria-current="page" @endif class="relative text-sm font-medium transition-colors hover:text-green-900 {{ request()->is('calendar') ? 'text-green-900 after:absolute after:-bottom-2 after:left-0 after:h-0.5 after:w-full after:rounded-full after:bg-green-900' : 'text-gray-700' }}">CALENDAR</a>
            <a href="/about" @if(request()->is('about')) aria-current="page" @endif class="relative text-sm font-medium transition-colors hover:text-green-900 {{ request()->is('about') ? 'text-green-900 after:absolute after:-bottom-2 after:left-0 after:h-0.5 after:w-full after:rounded-full after:bg-green-900' : 'text-gray-700' }}">ABOUT US</a>
            <a href="/contact" @if(request()->is('contact')) aria-current="page" @endif class="relative text-sm font-medium transition-colors hover:text-green-900 {{
```

```
    request()->is('contact') ? 'text-green-900 after:absolute after:-bottom-2 after:left-0 after:h-0.5 after:w-full after:rounded-full after:bg-green-900' : 'text-gray-700'
    }}">CONTACT US</a>
30          </nav>
31
32          <div class="hidden items-center gap-3 md:flex">
33              @auth
34                  <a id="notificationLink" href="{{ route('notifications.index') }}" class="relative text-gray-700 hover:text-gray-900">
35                      <svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5" fill="none" viewBox="0 0 24 24" stroke="currentColor">
36                          <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 17h5l-1.405-1.405A2.032 2.032 0 0118 14.158V11a6.002 6.002 0
    00-4-5.659V5a2 2 0 10-4 0v.341C7.67 6.165 6 8.388 6 11v3.159c0 .538-.214 1.055-.595 1.436L4 17h5m6 0v1a3 3 0 11-6 0v-1m6 0H9" />
37                      </svg>
38                  </a>
39                  <a href="/profile" class="flex items-center gap-2 text-sm font-medium text-gray-700 hover:text-gray-900">
40                      <img
41                          src="{{ auth()->user()->profilePictureUrl() }}"
42                          onerror="this.onerror=null;this.src='{{ asset('images/guest-avatar.svg') }}';"
43                          alt="Profile Picture"
44                          class="w-8 h-8 rounded-full object-cover"
45                      >
46                      Profile
47                  </a>
48                  @if(auth()->user()->isAdmin())
49                      <a href="{{ route('admin.dashboard') }}" class="text-sm font-medium text-gray-700 hover:text-gray-900">Admin</a>
50                  @endif
51                  <form method="POST" action="{{ route('logout') }}" class="inline">
52                      @csrf
53                      <button type="submit" class="px-4 py-1.5 text-sm font-medium bg-black text-white rounded-full hover:bg-gray-800 transition-colors">
54                          Log Out
55                      </button>
56                  </form>
57              @else
58                  <a href="/signup" class="px-4 py-1.5 text-sm font-medium bg-black text-white rounded-full hover:bg-gray-800 transition-colors">Sign Up</a>
59                  <a href="/login" class="px-4 py-1.5 text-sm font-medium bg-black text-white rounded-full hover:bg-gray-800 transition-colors">Log In</a>
60              @endauth
61          </div>
62          <button
63              id="mobile-menu-toggle"
64              type="button"
65              class="relative z-[60] inline-flex h-11 w-11 items-center justify-center rounded-xl border border-slate-200 text-slate-700 transition hover:bg-slate-50
    focus:outline-none focus:ring-2 focus:ring-green-700 md:hidden"
66              aria-expanded="false"
67              aria-controls="mobile-navigation"
68              aria-label="Open navigation menu"
69          >
70              <svg id="mobile-menu-open-icon" class="h-6 w-6" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true">
71                  <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 6h16M4 12h16M4 18h16"></path>
72              </svg>
73              <svg id="mobile-menu-close-icon" class="hidden h-6 w-6" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true">
74                  <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="m6 6 12 12M18 6 6 18"></path>
75              </svg>
76          </button>
77
78          <button id="mobile-navigation-backdrop" type="button" class="pointer-events-none fixed inset-x-0 bottom-0 z-40 bg-slate-950/20 opacity-0 backdrop-blur-sm
    transition-opacity duration-300 md:hidden" aria-label="Close navigation menu" aria-hidden="true" tabindex="-1"></button>
79          <nav id="mobile-navigation" class="pointer-events-none fixed bottom-0 left-0 z-50 flex w-[min(84vw,20rem)] -translate-x-full flex-col gap-1 overflow-y-auto border-r
    border-slate-200 bg-white px-3 py-4 shadow-2xl transition-transform duration-300 ease-out md:hidden" aria-label="Mobile navigation" aria-hidden="true" inert>
80              <a href="/" @if(request()->path() === '/') aria-current="page" @endif class="rounded-xl px-4 py-3 text-sm font-semibold transition-colors hover:bg-green-50
    hover:text-green-900 {{ request()->path() === '/' ? 'bg-green-50 text-green-900 underline decoration-2 underline-offset-4' : 'text-slate-700' }}">Home</a>
81              <a href="/reserve" @if(request()->is('reserve')) aria-current="page" @endif class="rounded-xl px-4 py-3 text-sm font-semibold transition-colors hover:bg-green-50
    hover:text-green-900 {{ request()->is('reserve') ? 'bg-green-50 text-green-900 underline decoration-2 underline-offset-4' : 'text-slate-700' }}">Reserve</a>
82              <a href="/calendar" @if(request()->is('calendar')) aria-current="page" @endif class="rounded-xl px-4 py-3 text-sm font-semibold transition-colors
    hover:bg-green-50 hover:text-green-900 {{ request()->is('calendar') ? 'bg-green-50 text-green-900 underline decoration-2 underline-offset-4' : 'text-slate-700'
    }}">Calendar</a>
83              <a href="/about" @if(request()->is('about')) aria-current="page" @endif class="rounded-xl px-4 py-3 text-sm font-semibold transition-colors hover:bg-green-50
    hover:text-green-900 {{ request()->is('about') ? 'bg-green-50 text-green-900 underline decoration-2 underline-offset-4' : 'text-slate-700' }}">About us</a>
84              <a href="/contact" @if(request()->is('contact')) aria-current="page" @endif class="rounded-xl px-4 py-3 text-sm font-semibold transition-colors hover:bg-green-50
    hover:text-green-900 {{ request()->is('contact') ? 'bg-green-50 text-green-900 underline decoration-2 underline-offset-4' : 'text-slate-700' }}">Contact us</a>
85              @auth
86                  <a href="{{ route('notifications.index') }}" class="rounded-xl px-4 py-3 text-sm font-semibold text-slate-700 hover:bg-green-50">Notifications</a>
87                  <a href="/profile" class="rounded-xl px-4 py-3 text-sm font-semibold text-slate-700 hover:bg-green-50">Profile settings</a>
88                  @if(auth()->user()->isAdmin())
89                      <a href="{{ route('admin.dashboard') }}" class="rounded-xl px-4 py-3 text-sm font-semibold text-slate-700 hover:bg-green-50">Staff dashboard</a>
90                  @endif
91                  <form method="POST" action="{{ route('logout') }}" class="px-4 pb-2 pt-1">
```

```
                    @csrf
                    <button type="submit" class="w-full rounded-xl bg-green-800 px-4 py-3 text-left text-sm font-semibold text-white hover:bg-green-900">Log out</button>
                </form>
            @else
                <div class="flex gap-3 px-4 pb-2 pt-1">
                    <a href="/signup" class="flex-1 rounded-xl bg-green-800 px-4 py-3 text-center text-sm font-semibold text-white hover:bg-green-900">Sign up</a>
                    <a href="/login" class="flex-1 rounded-xl border border-green-800 px-4 py-3 text-center text-sm font-semibold text-green-900 hover:bg-green-50">Log in</a>
                </div>
            @endauth
        </nav>
    </header>

    <main>
        @yield('content')
    </main>

    @auth
        <script>
            document.addEventListener('DOMContentLoaded', function() {
                var audioContext = null;
                var lastCount = null;
                var lastNotificationTimestamp = null;
                var notificationCheckInProgress = false;
                var notificationLink = document.getElementById('notificationLink');
                var notificationEndpoint = @json(route('notifications.unreadSummary'));

                function unlockNotificationSound() {
                    var AudioContextConstructor = window.AudioContext || window.webkitAudioContext;
                    if (!AudioContextConstructor || audioContext) return;

                    audioContext = new AudioContextConstructor();
                    if (audioContext.state === 'suspended') {
                        audioContext.resume().catch(function(error) {
                            console.warn('Notification sound could not be enabled.', error);
                        });
                    }
                }

                function playNotificationSound() {
                    if (!audioContext || audioContext.state !== 'running') return;

                    var oscillator = audioContext.createOscillator();
                    var gain = audioContext.createGain();
                    var now = audioContext.currentTime;
                    oscillator.type = 'sine';
                    oscillator.frequency.setValueAtTime(880, now);
                    oscillator.frequency.setValueAtTime(660, now + 0.12);
                    gain.gain.setValueAtTime(0.0001, now);
                    gain.gain.exponentialRampToValueAtTime(0.18, now + 0.02);
                    gain.gain.exponentialRampToValueAtTime(0.0001, now + 0.3);
                    oscillator.connect(gain);
                    gain.connect(audioContext.destination);
                    oscillator.start(now);
                    oscillator.stop(now + 0.3);
                }

                document.addEventListener('pointerdown', unlockNotificationSound, { once: true });
                document.addEventListener('keydown', unlockNotificationSound, { once: true });

                function updateNotificationStatus() {
                    if (document.hidden || notificationCheckInProgress) return;

                    notificationCheckInProgress = true;
                    fetch(notificationEndpoint, {
                        headers: { 'Accept': 'application/json' },
                        credentials: 'same-origin'
                    })
                        .then(function(response) {
                            if (!response.ok) {
                                throw new Error('Notification status request failed: ' + response.status);
                            }
                            return response.json();
                        })
                        .then(function(summary) {
```

```
                            var count = Number(summary.count) || 0;
                            var notificationTimestamp = summary.latest_at || null;

                            if (lastCount !== null
                                && (count > lastCount
                                    || (count === lastCount && count > 0 && notificationTimestamp !== lastNotificationTimestamp))) {
                                playNotificationSound();
                            }

                            lastCount = count;
                            lastNotificationTimestamp = notificationTimestamp;

                            var badge = document.getElementById('notificationBadge');
                            if (count > 0) {
                                if (!badge && notificationLink) {
                                    badge = document.createElement('span');
                                    badge.id = 'notificationBadge';
                                    badge.className = 'absolute -top-1 -right-1 inline-flex h-5 min-w-5 items-center justify-center rounded-full bg-red-600 px-1.5 text-[10px] font-semibold text-white';
                                    notificationLink.appendChild(badge);
                                }
                                if (badge) badge.textContent = String(count);
                            } else if (badge) {
                                badge.remove();
                            }
                        })
                        .catch(function(error) {
                            console.error('Could not check for new notifications.', error);
                        })
                        .finally(function() {
                            notificationCheckInProgress = false;
                        });
                }

                updateNotificationStatus();
                window.setInterval(updateNotificationStatus, 30000);
                document.addEventListener('visibilitychange', function() {
                    if (!document.hidden) updateNotificationStatus();
                });
            });
        </script>
    @endauth

    @if(auth()->check() && !request()->routeIs('profile.show', 'security.compromised', 'login', 'register', 'password.*', '2fa.*', 'admin.*'))
        <div id="chatbot-widget" class="fixed inset-0 z-50 pointer-events-none" style="position: fixed; inset: 0; z-index: 9999; pointer-events: none;">
            <div id="chatbot-backdrop" class="invisible pointer-events-none absolute inset-0 bg-slate-950/20 opacity-0 transition-opacity duration-300" aria-hidden="true"></div>
            <button id="chatbot-toggle" type="button" class="pointer-events-auto fixed bottom-5 right-5 flex h-14 w-14 items-center justify-center rounded-full bg-gradient-to-br from-green-800 via-green-700 to-amber-400 text-white shadow-xl shadow-green-900/25 transition hover:-translate-y-0.5 hover:shadow-2xl" style="position: fixed; right: 20px; bottom: 20px; z-index: 10001; display: flex; width: 56px; height: 56px; border-radius: 9999px; background: linear-gradient(135deg, #166534, #15803d 58%, #facc15); color: #fff; pointer-events: auto; cursor: pointer;" aria-label="Open Arcee" title="Open Arcee">
                <svg class="h-6 w-6" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="1.8" d="M8 10h8m-8 4h5m7-2a8 8 0 01-8 8 8.4 8.4 0 01-3.7-.85L4 20l.85-3.3A8 8 0 1112 20" />
                </svg>
            </button>
            <section id="chatbot-panel" class="invisible pointer-events-none fixed right-0 top-0 flex h-full translate-x-full flex-col border-l border-indigo-100 bg-white opacity-0 shadow-2xl shadow-indigo-950/20 transition duration-300 ease-out sm:my-3 sm:mr-3 sm:h-[calc(100%-1.5rem)] sm:rounded-3xl sm:border" style="position: fixed; top: 0; right: 0; bottom: 0; z-index: 10000; display: flex; width: min(440px, 100vw); height: 100%;" aria-label="Arcee chatbot">
                <div class="flex items-center justify-between border-b border-white/15 bg-gradient-to-r from-green-900 via-green-800 to-amber-400 px-5 py-4 text-white sm:rounded-t-3xl" style="display: flex; align-items: center; justify-content: space-between; min-height: 78px; padding: 16px 20px; background: linear-gradient(110deg, #14532d, #166534 58%, #facc15); color: #fff;">
                    <div>
                        <div class="flex items-center gap-3" style="display: flex; align-items: center; gap: 12px;">
                            <span class="flex h-10 w-10 shrink-0 items-center justify-center rounded-2xl border border-white/30 bg-white/15 text-sm font-bold shadow-inner" style="display: flex; flex: 0 0 40px; width: 40px; height: 40px; align-items: center; justify-content: center; border-radius: 16px; border: 1px solid rgba(255,255,255,.3); background: rgba(255,255,255,.15); color: #fff; font-size: 13px; font-weight: 700;">AC</span>
                            <div>
                                <h2 class="text-sm font-semibold" style="margin: 0; color: #fff; font-size: 14px; font-weight: 700;">Arcee</h2>
                                <p class="text-xs text-slate-300" style="margin: 3px 0 0; color: #cbd5e1; font-size: 12px;">Rooms, items, schedules, and reservations</p>
                            </div>
                        </div>
                    </div>
                    <div class="flex items-center gap-3">
```

```
                        <button id="chatbot-new" type="button" class="rounded-full border border-white/25 bg-white/10 px-3 py-2 text-xs font-semibold text-white transition hover:bg-white/20" aria-label="Start a new chat">New chat</button>
                        <button id="chatbot-close" type="button" class="rounded-full px-2 py-1 text-xl leading-none text-white/80 transition hover:bg-white/15 hover:text-white" style="padding: 4px 8px; border: 0; border-radius: 9999px; background: transparent; color: rgba(255,255,255,.8); font-size: 22px; line-height: 1; cursor: pointer;" aria-label="Close Arcee">&times;</button>
                    </div>
                </div>
                <div id="chatbot-messages" class="min-h-0 flex-1 space-y-4 overflow-y-auto bg-gradient-to-br from-purple-50/80 via-slate-50 to-cyan-50/70 p-5 text-sm" style="min-height: 0; flex: 1 1 auto; overflow-y: auto; scrollbar-width: thin;" aria-live="polite">
                    <div class="flex gap-3">
                        <span class="flex h-8 w-8 shrink-0 items-center justify-center rounded-xl bg-indigo-700 text-[10px] font-bold text-white shadow-sm">AC</span>
                        <p class="w-fit max-w-[90%] rounded-2xl rounded-tl-sm border border-indigo-100 bg-white/95 px-3.5 py-2.5 text-slate-700 shadow-sm" style="width: fit-content; max-width: 90%;">Hi! Ask me about room and item availability, property reservations, schedules, or booking status.</p>
                    </div>
                </div>
                <form id="chatbot-form" class="pointer-events-auto border-t border-indigo-100 bg-white/95 p-4 sm:rounded-b-3xl" style="pointer-events: auto; position: relative; z-index: 10002;">
                    @csrf
                    <div class="flex items-center gap-2 rounded-2xl border border-slate-300 bg-white p-1.5 shadow-sm transition focus-within:border-indigo-400 focus-within:ring-2 focus-within:ring-indigo-100">
                        <label for="chatbot-input" class="sr-only">Ask Arcee</label>
                        <input id="chatbot-input" type="text" class="pointer-events-auto min-w-0 flex-1 border-0 bg-transparent px-3 py-2 text-sm text-slate-900 outline-none placeholder:text-slate-400" style="pointer-events: auto; position: relative; z-index: 10003; cursor: text;" placeholder="Ask about availability..." autocomplete="off">
                        <button id="chatbot-send" type="submit" class="rounded-xl bg-indigo-700 px-4 py-2 text-sm font-semibold text-white shadow-sm transition hover:bg-indigo-800 disabled:cursor-wait disabled:opacity-60">Send</button>
                    </div>
                    <p class="mt-2 px-1 text-[11px] text-slate-500">Check room or item availability, request a reservation, or view booking status.</p>
                </form>
            </section>
        </div>
        <script>
            document.addEventListener('DOMContentLoaded', function() {
                const toggle = document.getElementById('chatbot-toggle');
                const close = document.getElementById('chatbot-close');
                const newChat = document.getElementById('chatbot-new');
                const panel = document.getElementById('chatbot-panel');
                const backdrop = document.getElementById('chatbot-backdrop');
                const form = document.getElementById('chatbot-form');
                const input = document.getElementById('chatbot-input');
                const messages = document.getElementById('chatbot-messages');
                const sendButton = document.getElementById('chatbot-send');
                const csrfToken = form.querySelector('input[name="_token"]').value;
                let responsePending = false;

                input.addEventListener('contextmenu', function() {
                    window.setTimeout(function() {
                        input.focus();
                    }, 0);
                });

                function resetVisibleChat() {
                    messages.innerHTML = `
                        <div class="flex gap-3">
                            <span class="flex h-8 w-8 shrink-0 items-center justify-center rounded-xl bg-indigo-700 text-[10px] font-bold text-white shadow-sm">AC</span>
                            <p class="w-fit max-w-[90%] rounded-2xl rounded-tl-sm border border-indigo-100 bg-white/95 px-3.5 py-2.5 text-slate-700 shadow-sm">Hi! Ask me about room and item availability, property reservations, schedules, or booking status.</p>
                        </div>
                    `;
                    input.value = '';
                }

                function resetServerChat(keepalive = false) {
                    return fetch('{{ route('chatbot.reset') }}', {
                        method: 'POST',
                        headers: {
                            'Accept': 'application/json',
                            'Content-Type': 'application/json',
                            'X-CSRF-TOKEN': csrfToken,
                        },
                        body: JSON.stringify({}),
                        credentials: 'same-origin',
                        keepalive,
                    });
                }
```

```
292
293                 function setChatbotButtonVisibility(isOpen) {
294                     if (!toggle) return;
295
296                     const shouldHide = Boolean(isOpen);
297                     toggle.style.opacity = shouldHide ? '0' : '1';
298                     toggle.style.visibility = shouldHide ? 'hidden' : 'visible';
299                     toggle.style.pointerEvents = shouldHide ? 'none' : 'auto';
300                     toggle.setAttribute('aria-hidden', shouldHide ? 'true' : 'false');
301                 }
302
303                 toggle?.addEventListener('click', function() {
304                     const isClosed = panel.classList.contains('invisible');
305
306                     if (isClosed) {
307                         setChatbotButtonVisibility(true);
308                     } else {
309                         setChatbotButtonVisibility(false);
310                     }
311
312                     panel.classList.toggle('invisible', !isClosed);
313                     panel.classList.toggle('pointer-events-none', !isClosed);
314                     panel.classList.toggle('translate-x-full', !isClosed);
315                     panel.classList.toggle('opacity-0', !isClosed);
316                     panel.style.transform = isClosed ? 'translateX(0)' : 'translateX(100%)';
317                     backdrop.classList.toggle('invisible', !isClosed);
318                     backdrop.classList.toggle('pointer-events-none', !isClosed);
319                     backdrop.classList.toggle('opacity-0', !isClosed);
320                     document.getElementById('chatbot-widget').classList.toggle('pointer-events-none', !isClosed);
321                     panel.style.pointerEvents = isClosed ? 'auto' : 'none';
322                     if (isClosed) input.focus();
323                 });
324                 close?.addEventListener('click', function() {
325                     panel.classList.add('invisible', 'pointer-events-none', 'translate-x-full', 'opacity-0');
326                     panel.style.transform = 'translateX(100%)';
327                     backdrop.classList.add('invisible', 'pointer-events-none', 'opacity-0');
328                     document.getElementById('chatbot-widget').classList.add('pointer-events-none');
329                     panel.style.pointerEvents = 'none';
330                     setChatbotButtonVisibility(false);
331                 });
332                 backdrop?.addEventListener('click', function() { close.click(); });
333
334                 newChat?.addEventListener('click', async function() {
335                     try {
336                         await resetServerChat();
337                     } catch (error) {
338                         // The visible chat is still reset; the next message will establish fresh state.
339                     }
340
341                     resetVisibleChat();
342                     input.focus();
343                 });
344
345                 window.addEventListener('pagehide', function() {
346                     resetServerChat(true).catch(() => {});
347                 });
348
349                 form?.addEventListener('submit', async function(event) {
350                     event.preventDefault();
351                     const message = input.value.trim();
352                     if (!message || responsePending) return;
353                     responsePending = true;
354
355                     const escapedMessage = message.replace(/[&<>]/g, character => ({'&':'&amp;','<':'&lt;','>':'&gt;'}[character]));
356                     messages.insertAdjacentHTML('beforeend', `<p style="width: fit-content; max-width: 90%; margin-left: auto;" class="rounded-2xl rounded-tr-sm bg-indigo-700
    px-3.5 py-2.5 text-white shadow-sm">${escapedMessage}</p>`);
357                     input.value = '';
358                     sendButton.disabled = true;
359                     sendButton.textContent = 'Checking...';
360                     const pendingReply = document.createElement('p');
361                     pendingReply.className = 'w-fit rounded-2xl rounded-tl-sm border border-indigo-100 bg-white/95 px-3.5 py-2.5 text-slate-500 shadow-sm';
362                     pendingReply.textContent = 'Checking the latest room and reservation details...';
363                     pendingReply.setAttribute('role', 'status');
364                     messages.appendChild(pendingReply);
```

```
                    messages.scrollTop = messages.scrollHeight;

                    try {
                        const response = await fetch('{{ route('chatbot.respond') }}', {
                            method: 'POST',
                            headers: {
                                'Accept': 'application/json',
                                'Content-Type': 'application/json',
                                'X-CSRF-TOKEN': form.querySelector('input[name="_token"]').value,
                            },
                            body: JSON.stringify({ message }),
                            credentials: 'same-origin',
                        });
                        const data = await response.json();
                        const reply = document.createElement('p');
                        reply.className = 'rounded-2xl rounded-tl-sm border border-indigo-100 bg-white/95 px-3.5 py-2.5 text-slate-700 shadow-sm';
                        reply.style.width = 'fit-content';
                        reply.style.maxWidth = '90%';
                        reply.textContent = data.reply || 'I could not understand that request.';
                        messages.appendChild(reply);
                        if (data.moderated && data.user_message) {
                            const userBubble = messages.lastElementChild?.previousElementSibling;
                            if (userBubble) {
                                userBubble.textContent = data.user_message;
                            }
                        }
                        if (data.action_url) {
                            const action = document.createElement('a');
                            action.href = data.action_url;
                            action.textContent = data.action_label || 'Continue';
                            action.className = 'block rounded-xl bg-gradient-to-r from-indigo-700 to-violet-700 px-4 py-2.5 text-xs font-semibold text-white shadow-sm
transition hover:from-indigo-800 hover:to-violet-800';
                            action.style.width = 'fit-content';
                            messages.appendChild(action);
                        }
                        messages.scrollTop = messages.scrollHeight;
                    } catch (error) {
                        const reply = document.createElement('p');
                        reply.className = 'w-fit max-w-[90%] rounded-2xl border border-rose-200 bg-rose-50 px-3.5 py-2.5 text-rose-700 shadow-sm';
                        reply.style.width = 'fit-content';
                        reply.style.maxWidth = '90%';
                        reply.textContent = 'Arcee is temporarily unavailable.';
                        messages.appendChild(reply);
                    } finally {
                        pendingReply.remove();
                        responsePending = false;
                        sendButton.disabled = false;
                        sendButton.textContent = 'Send';
                    }
                });
            });
        </script>
    @endif
    @stack('scripts')
    <script>
        document.addEventListener('DOMContentLoaded', function() {
            const toggle = document.getElementById('mobile-menu-toggle');
            const navigation = document.getElementById('mobile-navigation');
            const backdrop = document.getElementById('mobile-navigation-backdrop');
            const openIcon = document.getElementById('mobile-menu-open-icon');
            const closeIcon = document.getElementById('mobile-menu-close-icon');
            const header = toggle.closest('header');

            function updateNavigationPosition() {
                document.documentElement.style.setProperty('--mobile-header-height', `${header.offsetHeight}px`);
                navigation.style.top = `${header.offsetHeight}px`;
                backdrop.style.top = `${header.offsetHeight}px`;
            }

            function setMenuOpen(open) {
                updateNavigationPosition();
                toggle.setAttribute('aria-expanded', String(open));
                toggle.setAttribute('aria-label', open ? 'Close navigation menu' : 'Open navigation menu');
                navigation.setAttribute('aria-hidden', String(!open));
```

```
                navigation.inert = !open;
                navigation.classList.toggle('pointer-events-none', !open);
                navigation.classList.toggle('pointer-events-auto', open);
                navigation.classList.toggle('translate-x-0', open);
                navigation.classList.toggle('-translate-x-full', !open);
                backdrop.setAttribute('aria-hidden', String(!open));
                backdrop.classList.toggle('pointer-events-none', !open);
                backdrop.classList.toggle('pointer-events-auto', open);
                backdrop.classList.toggle('opacity-100', open);
                backdrop.classList.toggle('opacity-0', !open);
                openIcon.classList.toggle('hidden', open);
                closeIcon.classList.toggle('hidden', !open);
            }

            updateNavigationPosition();
            window.addEventListener('resize', updateNavigationPosition);

            toggle.addEventListener('click', function() {
                setMenuOpen(toggle.getAttribute('aria-expanded') !== 'true');
            });

            backdrop.addEventListener('click', function() {
                setMenuOpen(false);
            });

            navigation.addEventListener('click', function(event) {
                if (event.target.closest('a')) setMenuOpen(false);
            });

            document.addEventListener('keydown', function(event) {
                if (event.key === 'Escape') setMenuOpen(false);
            });

            window.matchMedia('(min-width: 768px)').addEventListener('change', function(event) {
                if (event.matches) setMenuOpen(false);
            });
        });
    </script>
</body>
</html>
```

resources/views/layouts/embedded.blade.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="csrf-token" content="{{ csrf_token() }}">
    <title>@yield('title', 'Campus Reserve')</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
    @stack('styles')
</head>
<body class="min-h-screen bg-[#f3f7f4] text-slate-900">
    <main>
        @yield('content')
    </main>
    @stack('scripts')
</body>
</html>
```

resources/views/notifications/index.blade.php

```
@extends('layouts.app')

@section('title', 'Notifications - Campus Reserve')

@section('content')
<div class="min-h-[calc(100vh-73px)] bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50 px-4 py-10 md:px-8 xl:px-16">
    <div class="mx-auto max-w-5xl">
        <div class="mb-8 flex flex-col gap-4 rounded-3xl border border-white/80 bg-gradient-to-r from-indigo-100/90 via-pink-50 to-cyan-100/80 p-6 shadow-sm md:flex-row
md:items-center md:justify-between">
            <div>
```

```
                <h1 class="text-2xl font-semibold tracking-tight text-slate-900">Notifications</h1>
                <p class="mt-2 text-sm text-slate-600">Review your recent alerts and mark them as read or remove selected messages.</p>
            </div>
            <!-- Actions for notification batch processing: mark all read or remove selected. -->
            <div class="flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-end">
                <label class="inline-flex items-center text-sm text-slate-600">
                    <input id="selectAllNotifications" type="checkbox" class="mr-2 h-4 w-4 rounded border-slate-300 text-indigo-700 focus:ring-indigo-500">
                    Select all
                </label>
                <div class="flex flex-wrap items-center gap-3">
                    <form method="POST" action="{{ route('notifications.markRead') }}">
                        @csrf
                        <button type="submit" class="inline-flex items-center justify-center rounded-full bg-gradient-to-r from-indigo-700 to-violet-700 px-5 py-2.5 text-sm font-semibold text-white shadow-sm transition hover:from-indigo-800 hover:to-violet-800">
                            Mark all as read
                        </button>
                    </form>
                    <button type="submit" form="notificationsDeleteForm" class="inline-flex items-center justify-center rounded-full bg-rose-600 px-5 py-2.5 text-sm font-semibold text-white transition hover:bg-rose-700">
                        Remove selected
                    </button>
                </div>
            </div>
        </div>

        @if(session('success'))
            <div class="mb-6 rounded-2xl border border-emerald-200 bg-emerald-50 p-4 text-emerald-900" role="status">
                {{ session('success') }}
            </div>
        @elseif(session('error'))
            <div class="mb-6 rounded-2xl border border-rose-200 bg-rose-50 p-4 text-rose-900" role="alert">
                {{ session('error') }}
            </div>
        @elseif(session('info'))
            <div class="mb-6 rounded-2xl border border-sky-200 bg-sky-50 p-4 text-sky-900" role="status">
                {{ session('info') }}
            </div>
        @endif

        @if($notifications->isEmpty())
            <div class="rounded-3xl border border-dashed border-indigo-200 bg-white/90 p-10 text-center text-slate-600 shadow-sm">
                <p class="text-lg font-medium">No notifications yet.</p>
                <p class="mt-2 text-sm">You will receive reservation approval or rejection updates here.</p>
            </div>
        @else
            <form id="notificationsDeleteForm" method="POST" action="{{ route('notifications.destroySelected') }}" class="hidden">
                @csrf
            </form>
            <div class="space-y-4">
                @foreach($notifications as $notification)
                    <div class="rounded-3xl border p-5 shadow-sm transition {{ is_null($notification->read_at) ? 'border-indigo-200 bg-white shadow-indigo-950/5' : 'border-slate-200/80 bg-white/75' }}">
                        @php $isApprovalRequest = $notification->type === \App\Notifications\ReservationPendingApproval::class; @endphp
                        <div class="flex flex-col gap-3 md:flex-row md:items-start md:justify-between">
                            <div class="flex items-start gap-3">
                                <label class="inline-flex items-center">
                                    <input type="checkbox" form="notificationsDeleteForm" name="selected_notifications[]" value="{{ $notification->id }}" data-approval-request="{{ $isApprovalRequest ? '1' : '0' }}" class="notification-checkbox h-4 w-4 rounded border-slate-300 text-indigo-700 focus:ring-indigo-500">
                                </label>
                                <div>
                                    <div class="flex items-center gap-2">
                                        <p class="text-sm font-semibold text-slate-900">{{ $notification->data['title'] ?? 'Notification' }}</p>
                                        @if($isApprovalRequest)
                                            <span class="rounded-full bg-pink-100 px-2 py-1 text-[11px] uppercase tracking-[0.2em] text-pink-900">Request</span>
                                        @endif
                                    </div>
                                    <p class="mt-2 text-sm leading-6 text-slate-700">{{ $notification->data['message'] ?? '' }}</p>
                                </div>
                            </div>
                            <div class="flex flex-col items-start gap-2 text-right text-xs text-slate-500 md:items-end">
                                <span>{{ $notification->created_at->diffForHumans() }}</span>
                                @if(isset($notification->data['status']))
                                    <span class="rounded-full bg-slate-100 px-2 py-1 text-[11px] uppercase tracking-[0.2em] text-slate-700">
                                        {{ ucfirst($notification->data['status']) }}
```

```
                                    </span>
                                @endif
                            </div>
                        </div>
                        @if($notification->type === \App\Notifications\PropertyReturnDue::class && empty($notification->data['response']))
                            <div class="mt-4 flex flex-wrap gap-3">
                                <form method="POST" action="{{ route('notifications.propertyReturn.respond', $notification->data['reservation_id']) }}">
                                    @csrf
                                    <input type="hidden" name="notification_id" value="{{ $notification->id }}">
                                    <button type="submit" name="response" value="returned" class="rounded-full bg-gradient-to-r from-indigo-700 to-violet-700 px-4 py-2 text-sm font-semibold text-white transition hover:from-indigo-800 hover:to-violet-800">
                                        Yes, returned
                                    </button>
                                </form>
                                <form method="POST" action="{{ route('notifications.propertyReturn.respond', $notification->data['reservation_id']) }}">
                                    @csrf
                                    <input type="hidden" name="notification_id" value="{{ $notification->id }}">
                                    <button type="submit" name="response" value="not_returned" class="rounded-full border border-rose-200 bg-rose-50 px-4 py-2 text-sm font-semibold text-rose-700 transition hover:bg-rose-100">
                                        No, not returned
                                    </button>
                                </form>
                            </div>
                        @elseif($notification->type === \App\Notifications\PropertyReturnDue::class && !empty($notification->data['response']))
                            <p class="mt-4 text-sm font-medium text-slate-600">
                                Your response: {{ $notification->data['response'] === 'returned' ? 'Items returned' : 'Items not returned' }}.
                            </p>
                        @endif
                        @if(!empty($notification->data['action_url']))
                            <div class="mt-4">
                                <a href="{{ $notification->data['action_url'] }}" class="text-sm font-semibold text-indigo-600 transition hover:text-indigo-800">View details</a>
                            </div>
                        @endif
                    </div>
                @endforeach
            </div>

            <div class="mt-8">
                {{ $notifications->links() }}
            </div>
        @endif
    </div>
</div>

@push('scripts')
<script>
    // Notification page JavaScript to handle select-all, checkbox sync, and deletion confirmation.
    document.addEventListener('DOMContentLoaded', function() {
        var selectAll = document.getElementById('selectAllNotifications');
        var checkboxes = document.querySelectorAll('.notification-checkbox');
        var deleteForm = document.getElementById('notificationsDeleteForm');

        if (!deleteForm || checkboxes.length === 0) {
            return;
        }

        if (selectAll) {
            selectAll.addEventListener('change', function() {
                checkboxes.forEach(function(checkbox) {
                    checkbox.checked = selectAll.checked;
                });
            });
        }

        checkboxes.forEach(function(checkbox) {
            checkbox.addEventListener('change', function() {
                if (selectAll) {
                    selectAll.checked = Array.from(checkboxes).every(function(box) {
                        return box.checked;
                    });
                }
            });
        });
```

```

        deleteForm.addEventListener('submit', function(event) {
            var selectedCheckboxes = Array.from(checkboxes).filter(function(checkbox) {
                return checkbox.checked;
            });

            if (selectedCheckboxes.length === 0) {
                return;
            }

            var selectedRequestNotifications = selectedCheckboxes.filter(function(checkbox) {
                return checkbox.dataset.approvalRequest === '1';
            });

            var confirmationMessage = selectedRequestNotifications.length > 0
                ? 'You are deleting approval request notification(s). This may clear pending reservation requests. Do you want to continue?'
                : 'Delete selected notification(s)?';

            if (!confirm(confirmationMessage)) {
                event.preventDefault();
            }
        });
    });
</script>
@endpush
@endsection
```

resources/views/profile.blade.php

```
@extends('layouts.app')

@section('title', 'Profile Settings - Campus Reserve')

@push('styles')
<style>
    .profile-picture-input::file-selector-button {
        margin-right: 0.75rem;
        border: 0;
        border-radius: 9999px;
        background: #15803d;
        padding: 0.5rem 1rem;
        color: #fff;
        font-size: 0.875rem;
        font-weight: 600;
        cursor: pointer;
        transition: background-color 150ms ease;
    }

    .profile-picture-input::file-selector-button:hover {
        background: #14532d;
    }

    .profile-picture-input::-webkit-file-upload-button {
        margin-right: 0.75rem;
        border: 0;
        border-radius: 9999px;
        background: #15803d;
        padding: 0.5rem 1rem;
        color: #fff;
        font-size: 0.875rem;
        font-weight: 600;
        cursor: pointer;
    }
</style>
@endpush

@section('content')
<!-- Profile settings page for updating user account details, profile picture, and phone information. -->
<main class="min-h-[calc(100vh-73px)] bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50 px-3 py-6 sm:px-4 sm:py-10 md:px-8 xl:px-16">
    <div class="mx-auto w-full max-w-5xl">
        <section class="space-y-8">
            <div class="overflow-hidden rounded-3xl border border-slate-200/80 bg-white/95 shadow-sm shadow-indigo-950/5">
                <div class="bg-gradient-to-r from-indigo-700 via-violet-700 to-amber-400 px-8 py-8 text-white">
                    <h1 class="text-3xl font-semibold">Account</h1>
```

```
                    <p class="mt-2 max-w-2xl text-sm text-slate-300">Update your profile, security preferences, and manage how people see your account.</p>
                </div>

                <div class="space-y-6 p-4 sm:space-y-8 sm:p-8">
                    <div id="profile-status" class="{{ session('status') ? '' : 'hidden' }} rounded-3xl bg-emerald-50 border border-emerald-200 px-5 py-4 text-sm text-emerald-800">{{ session('status') }}</div>
                    <div id="password-change-alert" class="hidden rounded-3xl border border-amber-200 bg-amber-50 px-5 py-4 text-sm text-amber-900">
                        <span>Password changed successfully. If you did not make this change, </span><a href="{{ route('security.compromised') }}" class="font-semibold underline">secure your account immediately</a>.
                    </div>
                    @if(session('error'))
                        <div class="rounded-3xl bg-red-50 border border-red-200 px-5 py-4 text-sm text-red-800">{{ session('error') }}</div>
                    @endif

                    <form id="profile-settings-form" method="POST" action="{{ route('profile.update') }}" enctype="multipart/form-data" class="space-y-8">
                        @csrf
                        <input type="hidden" id="password-2fa-otp" name="password_2fa_otp">

                        <!-- Hidden inputs for phone and country code submission -->
                        <input type="hidden" name="phone" id="form_phone_input" value="{{ old('phone', auth()->user()->phone ?? '') }}" />
                        <input type="hidden" name="country_code" id="form_country_code_input" value="" />
                        <input type="hidden" name="email_notifications" value="0" />

                        <div class="grid gap-4 sm:gap-6 xl:grid-cols-[280px_1fr]">
                            <div class="rounded-3xl border border-indigo-100 bg-indigo-50/50 p-4 sm:p-6">
                                <p class="text-sm font-semibold text-gray-900">Profile</p>
                                <p class="mt-2 text-sm text-gray-500">Your avatar, name, and contact details.</p>
                            </div>

                            <div class="rounded-3xl border border-slate-200/80 bg-white/95 p-4 shadow-sm shadow-indigo-950/5 sm:p-6">
                                <div class="flex flex-col gap-4 sm:flex-row sm:items-center">
                                    <div class="relative h-24 w-24 overflow-hidden rounded-3xl border border-gray-200 bg-gray-100">
                                        <img
                                            src="{{ auth()->user()->profilePictureUrl() }}"
                                            onerror="this.onerror=null;this.src='{{ asset('images/guest-avatar.svg') }}';"
                                            alt="Profile picture"
                                            class="h-full w-full object-cover"
                                        >
                                    </div>
                                    <div class="min-w-0 flex-1">
                                        <label class="block text-sm font-semibold text-gray-900">Profile picture</label>
                                        <p class="mt-1 text-sm text-gray-500">Square image works best.</p>
                                        <input type="file" name="profile_picture" accept="image/png,image/jpeg,image/webp" class="profile-picture-input mt-3 block w-full text-sm text-gray-700" />
                                        <p id="profile-picture-upload-status" class="mt-2 text-sm text-gray-600" role="status" aria-live="polite"></p>
                                        @error('profile_picture')<p class="mt-2 text-sm text-red-600">{{ $message }}</p>@enderror
                                    </div>
                                </div>

                                <div class="grid gap-6 md:grid-cols-2">
                                    <label class="block">
                                        <span class="text-sm font-semibold text-gray-900">Full name</span>
                                        <input id="profile-name-input" type="text" name="name" value="{{ old('name', auth()->user()->name) }}" required class="mt-2 w-full rounded-3xl border border-gray-300 bg-white px-4 py-3 text-sm text-gray-900 focus:border-indigo-400 focus:outline-none focus:ring-2 focus:ring-indigo-200" />
                                        @error('name')<p class="mt-2 text-sm text-red-600">{{ $message }}</p>@enderror
                                    </label>

                                    <div class="block">
                                        <label for="profile-email-input" class="text-sm font-semibold text-gray-900">Email address</label>
                                        <div class="mt-2 flex items-center gap-2 rounded-3xl border border-gray-200 bg-slate-100 px-4 py-2">
                                            <input id="profile-email-input" type="email" value="{{ auth()->user()->email }}" readonly required autocomplete="email" class="min-w-0 flex-1 border-0 bg-transparent py-1 text-sm text-gray-600 outline-none focus:ring-0" />
                                            <button type="button" id="profile-email-edit" class="rounded-full p-2 text-gray-500 transition hover:bg-white hover:text-indigo-700 focus:outline-none focus:ring-2 focus:ring-indigo-300" aria-label="Edit email address" title="Edit email address">
                                                <svg class="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" aria-hidden="true">
                                                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="1.8" d="M16.862 4.487l2.651 2.651M4 20l4.25-.85L19.57.9a1.875 1.875 0 00-2.65-2.65L5.6 16.5 4 20z" />
                                                </svg>
                                            </button>
                                        </div>
                                        <div id="profile-email-edit-controls" class="mt-3 hidden space-y-3">
                                            <label class="block">
                                                <span class="text-xs font-semibold text-gray-700">Current password to confirm</span>
                                                <input id="profile-email-current-password" type="password" autocomplete="current-password" class="mt-1 w-full rounded-3xl
```

```
    border border-gray-300 bg-white px-4 py-2 text-sm text-gray-900 focus:border-indigo-400 focus:outline-none focus:ring-2 focus:ring-indigo-200" />
113                                         </label>
114                                         <div class="flex gap-2">
115                                             <button type="button" id="profile-email-save" class="flex-1 rounded-3xl bg-indigo-700 px-4 py-2 text-sm font-semibold
    text-white transition hover:bg-indigo-800">Save email</button>
116                                             <button type="button" id="profile-email-cancel" class="flex-1 rounded-3xl border border-gray-300 px-4 py-2 text-sm
    font-semibold text-gray-700 transition hover:bg-gray-100">Cancel</button>
117                                         </div>
118                                     </div>
119                                     <p id="profile-email-status" class="mt-2 text-sm text-gray-600" role="status" aria-live="polite"></p>
120                                 </div>
121                             </div>
122
123                             <label class="block">
124                                 <span class="text-sm font-semibold text-gray-900">Phone number</span>
125
126                                 <!-- Display Mode -->
127                                 <div id="phone_display_mode" class="mt-2 hidden">
128                                     <div class="flex items-center justify-between rounded-3xl border border-slate-300 bg-slate-50 px-4 py-3">
129                                         <span class="text-sm text-gray-900" id="phone_display_text">
130                                             @if(auth()->user()->phone)
131                                                 {{ auth()->user()->phone }}
132                                             @else
133                                                 <span class="text-gray-500">Not set</span>
134                                             @endif
135                                         </span>
136                                         <button type="button" id="phone_edit_btn" class="text-gray-500 hover:text-gray-900 transition">
137                                             <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
138                                                 <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M11 5H6a2 2 0 00-2 2v11a2 2 0 002 2h11a2 2 0
    002-2v-5m-7-10l7-7m0 0l-7 7m7-7v12"></path>
139                                             </svg>
140                                         </button>
141                                     </div>
142                                 </div>
143
144                                 <!-- Edit Mode -->
145                                 <div id="phone_edit_mode" class="mt-2 space-y-3">
146                                     <div class="grid gap-3 sm:grid-cols-[150px_1fr]">
147                                         <label class="block">
148                                             <span class="text-xs font-semibold text-gray-700">Country Code</span>
149                                             <select id="country_code_select" class="mt-1 w-full rounded-3xl border border-gray-300 bg-white px-3 py-2 text-sm
    text-gray-900 focus:outline-none focus:ring-2 focus:ring-black">
150                                                 <option value="+1">+1 (USA/Canada)</option>
151                                                 <option value="+44">+44 (UK)</option>
152                                                 <option value="+63">+63 (Philippines)</option>
153                                                 <option value="+81">+81 (Japan)</option>
154                                                 <option value="+91">+91 (India)</option>
155                                                 <option value="+86">+86 (China)</option>
156                                                 <option value="+33">+33 (France)</option>
157                                                 <option value="+49">+49 (Germany)</option>
158                                                 <option value="+39">+39 (Italy)</option>
159                                                 <option value="+61">+61 (Australia)</option>
160                                             </select>
161                                         </label>
162                                         <label class="block">
163                                             <span class="text-xs font-semibold text-gray-700">Phone Number</span>
164                                             <input type="tel" id="phone_number_input" placeholder="9123456789" class="mt-1 w-full rounded-3xl border border-gray-300
    bg-white px-4 py-2 text-sm text-gray-900 focus:outline-none focus:ring-2 focus:ring-black" />
165                                         </label>
166                                     </div>
167                                     <div class="flex gap-2">
168                                         <button type="button" id="phone_save_btn" class="flex-1 rounded-3xl bg-indigo-700 px-4 py-2 text-sm font-semibold text-white
    transition hover:bg-indigo-800">Save</button>
169                                         <button type="button" id="phone_cancel_btn" class="flex-1 rounded-3xl border border-gray-300 px-4 py-2 text-sm font-semibold
    text-gray-700 hover:bg-gray-100 transition">Cancel</button>
170                                     </div>
171                                 </div>
172                                 @error('phone')<p class="mt-2 text-sm text-red-600">{{ $message }}</p>@enderror
173                             </label>
174                         </div>
175                     </div>
176
177                     <div class="grid gap-6 xl:grid-cols-[280px_1fr]">
178                         <div class="rounded-3xl border border-gray-200 bg-slate-50 p-6">
```

```
                                <p class="text-sm font-semibold text-gray-900">Email notifications</p>
                                <p class="mt-2 text-sm text-gray-500">Choose whether to receive reservation updates by email.</p>
                            </div>
                            <div class="rounded-3xl border border-slate-200/80 bg-white/95 p-6 shadow-sm shadow-indigo-950/5">
                                <label class="flex items-start gap-3">
                                    <input id="email-notifications-toggle" type="checkbox" name="email_notifications" value="1" @checked(old('email_notifications',
    auth()->user()->email_notifications)) class="mt-1 rounded border-gray-300 text-indigo-700 focus:ring-indigo-500" />
                                    <span>
                                        <span class="block text-sm font-semibold text-gray-900">Email me about my reservations</span>
                                        <span class="mt-1 block text-sm text-gray-500">Receive an email when a reservation is submitted, approved, or denied. In-app
    notifications are always on.</span>
                                    </span>
                                </label>
                                <p id="email-notifications-status" class="mt-3 text-sm text-gray-600" role="status" aria-live="polite"></p>
                            </div>
                        </div>

                        <div class="grid gap-6 xl:grid-cols-[280px_1fr]">
                            <div class="rounded-3xl border border-gray-200 bg-slate-50 p-6">
                                <p class="text-sm font-semibold text-gray-900">Security</p>
                                <p class="mt-2 text-sm text-gray-500">Control password access and two-factor authentication.</p>
                            </div>

                            <div class="rounded-3xl border border-slate-200/80 bg-white/95 p-6 shadow-sm shadow-indigo-950/5">
                                <div class="pb-6">
                                    <p class="text-sm font-semibold text-gray-900">Password Reset</p>
                                    <p class="mt-2 text-sm text-gray-500">Change your account password securely.</p>
                                    <button type="button" id="open-password-modal" class="mt-4 rounded-full bg-indigo-700 px-5 py-3 text-sm font-semibold text-white
    transition hover:bg-indigo-800">Change password</button>
                                </div>

                                <div class="mt-6">
                                    <span class="text-sm font-semibold text-gray-900">Two-factor authentication</span>
                                    @if (!auth()->user()->two_fa_enabled)
                                        <div id="authenticator-setup-panel" class="mt-3 {{ session('authenticator_setup_qr') || old('two_fa_enabled') ? '' : 'hidden' }}
    rounded-3xl border border-blue-200 bg-blue-50 p-5">
                                            <p class="text-sm font-semibold text-blue-900">Use an authenticator app</p>
                                            <p class="mt-1 text-sm text-blue-800">Set up Google Authenticator or Microsoft Authenticator to generate login codes.</p>
                                            <div class="mt-4 flex flex-col gap-3 sm:flex-row">
                                                <input id="setup-current-password" type="password" name="setup_current_password" class="w-full rounded-3xl border
    border-blue-200 bg-white px-4 py-3 text-sm" placeholder="Current password" autocomplete="current-password">
                                                <button type="button" id="generate-authenticator-qr" class="rounded-full bg-indigo-700 px-5 py-3 text-sm font-semibold
    whitespace-nowrap text-white hover:bg-indigo-800">Generate QR code</button>
                                                <button type="button" id="cancel-authenticator-setup" class="rounded-md bg-red-600 px-5 py-3 text-sm font-semibold
    whitespace-nowrap text-white hover:bg-red-700">Cancel</button>
                                            </div>
                                            <p id="authenticator-setup-error" class="{{ $errors->has('setup_current_password') ? '' : 'hidden' }} mt-2 text-sm
    text-red-600">{{ $errors->first('setup_current_password') }}</p>
                                        </div>
                                    @endif
                                    <div id="authenticator-qr-result" class="{{ session('authenticator_setup_qr') ? '' : 'hidden' }} mt-4 rounded-3xl border
    border-emerald-200 bg-emerald-50 p-5">
                                        <p class="text-sm font-semibold text-emerald-900">Scan this QR code</p>
                                        <img id="authenticator-qr-image" src="{{ session('authenticator_setup_qr') }}" alt="Authenticator setup QR code" class="mt-3 h-48
    w-48 bg-white p-2">
                                        <p class="mt-3 text-sm text-emerald-900">Manual setup key: <strong id="authenticator-secret">{{
    session('authenticator_setup_secret') }}</strong></p>
                                        <div class="mt-4 flex flex-col gap-3 sm:flex-row">
                                            <input type="text" name="authenticator_otp" inputmode="numeric" maxlength="6" pattern="[0-9]{6}" class="w-full rounded-3xl
    border border-emerald-200 bg-white px-4 py-3 text-sm" placeholder="6-digit authenticator code" autocomplete="one-time-code">
                                            <button type="submit" formaction="{{ route('profile.2fa.confirm') }}" formmethod="POST" class="rounded-full bg-emerald-700
    px-5 py-3 text-sm font-semibold text-white">Save authenticator</button>
                                        </div>
                                        @error('authenticator_otp')<p class="mt-2 text-sm text-red-600">{{ $message }}</p>@enderror
                                    </div>
                                    <div class="mt-3 inline-flex items-center gap-4 rounded-3xl border border-gray-300 bg-slate-50 p-4">
                                        <input type="hidden" name="two_fa_enabled" value="0">
                                        <input type="checkbox" id="two_fa_enabled" name="two_fa_enabled" value="1" class="h-5 w-5 rounded border-gray-300 text-indigo-700
    focus:ring-indigo-500" {{ old('two_fa_enabled', auth()->user()->two_fa_enabled) ? 'checked' : '' }}>
                                        <span class="text-sm text-gray-700">Enable two-factor authentication</span>
                                    </div>
                                    @error('two_fa_enabled')<p class="mt-2 text-sm text-red-600">{{ $message }}</p>@enderror
                                    <div id="disable-2fa-confirmation" class="hidden"></div>
                                </div>
```

```
                            </div>
                        </div>

                    </form>
                </div>
            </div>
        </section>
    </div>
</main>

<div id="disable-2fa-modal" class="fixed inset-0 z-50 hidden items-center justify-center bg-slate-950/60 p-4 backdrop-blur-sm" role="dialog" aria-modal="true" aria-labelledby="disable-2fa-title">
    <div class="w-full max-w-md rounded-3xl border border-white/20 bg-white p-6 shadow-2xl">
        <h2 id="disable-2fa-title" class="text-xl font-semibold text-gray-900">Disable two-factor authentication?</h2>
        <p class="mt-2 text-sm text-gray-600">For your security, enter your current password and the code from your authenticator app.</p>
        <div class="mt-5 space-y-4">
            <label class="block">
                <span class="text-sm font-semibold text-gray-900">Current password</span>
                <input id="disable-current-password" form="profile-settings-form" type="password" class="mt-2 w-full rounded-3xl border border-gray-300 px-4 py-3 text-sm focus:outline-none focus:ring-2 focus:ring-black" autocomplete="current-password">
            </label>
            <label class="block">
                <span class="text-sm font-semibold text-gray-900">Authenticator code</span>
                <input id="disable-authenticator-code" form="profile-settings-form" type="text" inputmode="numeric" maxlength="6" pattern="[0-9]{6}" class="mt-2 w-full rounded-3xl border border-gray-300 px-4 py-3 text-sm tracking-widest focus:outline-none focus:ring-2 focus:ring-black" placeholder="000000" autocomplete="one-time-code">
            </label>
            <p id="disable-2fa-modal-error" class="{{ $errors->has('disable_current_password') || $errors->has('disable_2fa_otp') ? '' : 'hidden' }} text-sm font-medium text-red-600">{{ $errors->first('disable_current_password') ?: $errors->first('disable_2fa_otp') }}</p>
        </div>
        <div class="mt-6 flex justify-end gap-3">
            <button type="button" id="cancel-disable-2fa" class="rounded-full border border-gray-300 px-5 py-3 text-sm font-semibold text-gray-700">Cancel</button>
            <button type="button" id="continue-disable-2fa" class="rounded-full bg-black px-5 py-3 text-sm font-semibold text-white">Continue</button>
        </div>
    </div>
</div>

<div id="password-2fa-modal" class="fixed inset-0 z-50 hidden items-center justify-center bg-black/20 p-4" style="backdrop-filter: blur(10px); -webkit-backdrop-filter: blur(10px);" role="dialog" aria-modal="true" aria-labelledby="password-2fa-title">
    <div class="w-full max-w-md rounded-3xl border border-white/20 bg-white p-6 shadow-2xl">
        <h2 id="password-2fa-title" class="text-xl font-semibold text-gray-900">Confirm password change</h2>
        <p class="mt-2 text-sm text-gray-600">Enter the current six-digit code from your authenticator app to save your new password.</p>
        <p id="password-2fa-attempts" class="mt-4 text-sm font-medium text-amber-700">You have 3 attempts remaining.</p>
        <input id="password-authenticator-code" form="profile-settings-form" type="text" inputmode="numeric" maxlength="6" pattern="[0-9]{6}" class="mt-5 w-full rounded-3xl border border-gray-300 px-4 py-3 text-center text-lg tracking-widest focus:outline-none focus:ring-2 focus:ring-black" placeholder="000000" autocomplete="one-time-code">
        <p id="password-2fa-error" class="hidden mt-3 text-sm font-medium text-red-600"></p>
        <div class="mt-6 flex justify-end gap-3">
            <button type="button" id="cancel-password-2fa" class="rounded-md border border-gray-300 px-5 py-3 text-sm font-semibold text-gray-700">Cancel</button>
            <button type="button" id="confirm-password-2fa" class="rounded-md bg-black px-5 py-3 text-sm font-semibold text-white">Confirm</button>
        </div>
    </div>
</div>

<div id="password-modal" class="fixed inset-0 z-50 hidden items-center justify-center bg-black/20 p-4" style="backdrop-filter: blur(10px); -webkit-backdrop-filter: blur(10px);" role="dialog" aria-modal="true" aria-labelledby="password-title">
    <div class="w-full max-w-md rounded-3xl border border-white/20 bg-white p-6 shadow-2xl">
        <h2 id="password-title" class="text-xl font-semibold text-gray-900">Change password</h2>
        <p class="mt-2 text-sm text-gray-600">Enter your current password and choose a new password.</p>
        <div class="mt-5 space-y-4">
            <label class="block">
                <span class="text-sm font-semibold text-gray-900">Current password</span>
                <input id="password-modal-current" form="profile-settings-form" type="password" name="current_password" class="mt-2 w-full rounded-3xl border border-gray-300 px-4 py-3 text-sm focus:outline-none focus:ring-2 focus:ring-black" autocomplete="current-password">
                <button type="button" id="forgot-password-link" class="mt-2 text-sm font-medium text-blue-700 hover:underline">Forgot password?</button>
                <div class="mt-2 flex flex-wrap items-center gap-x-3 gap-y-1">
                    <p id="forgot-password-status" class="hidden flex-1 text-sm" role="status" aria-live="polite"></p>
                    <button type="button" id="resend-password-reset-link" class="hidden text-sm font-medium text-blue-700 hover:underline">Resend email</button>
                </div>
            </label>
            <label class="block">
                <span class="text-sm font-semibold text-gray-900">New password</span>
                <div class="relative mt-2">
                    <input id="password-modal-new" form="profile-settings-form" type="password" name="password" class="w-full rounded-3xl border border-gray-300 px-4 py-3 pr-16 text-sm focus:outline-none focus:ring-2 focus:ring-black" autocomplete="new-password">
                </div>
```

```
                <p class="mt-2 text-xs leading-5 text-gray-500">Use 5-30 characters, including uppercase and lowercase letters, a number, and a symbol.</p>
            </label>
            <label class="block">
                <span class="text-sm font-semibold text-gray-900">Confirm new password</span>
                <div class="relative mt-2">
                    <input id="password-modal-confirm" form="profile-settings-form" type="password" name="password_confirmation" class="w-full rounded-3xl border
border-gray-300 px-4 py-3 pr-16 text-sm focus:outline-none focus:ring-2 focus:ring-black" autocomplete="new-password">
                </div>
            </label>
            <p id="password-modal-error" class="hidden text-sm font-medium text-red-600"></p>
        </div>
        <div class="mt-6 flex justify-end gap-3">
            <button type="button" id="cancel-password-modal" class="rounded-md border border-gray-300 px-5 py-3 text-sm font-semibold text-gray-700">Cancel</button>
            <button type="button" id="save-password-modal" class="rounded-md bg-black px-5 py-3 text-sm font-semibold text-white">Save password</button>
        </div>
    </div>
</div>

<script>
    document.addEventListener('DOMContentLoaded', function() {
        const twoFactorToggle = document.getElementById('two_fa_enabled');
        const authenticatorSetupPanel = document.getElementById('authenticator-setup-panel');
        const setupCurrentPassword = document.getElementById('setup-current-password');
        const disableTwoFactorModal = document.getElementById('disable-2fa-modal');
        const profileSettingsForm = document.getElementById('profile-settings-form');
        const profileStatus = document.getElementById('profile-status');
        const emailNotificationsToggle = document.getElementById('email-notifications-toggle');
        const emailNotificationsStatus = document.getElementById('email-notifications-status');
        const profilePictureInput = profileSettingsForm?.querySelector('input[name="profile_picture"]');
        const profilePictureUploadStatus = document.getElementById('profile-picture-upload-status');
        const disableCurrentPassword = document.getElementById('disable-current-password');
        const disableAuthenticatorCode = document.getElementById('disable-authenticator-code');
        const profileNameInput = document.getElementById('profile-name-input');
        const profileEmailInput = document.getElementById('profile-email-input');
        const profileEmailEdit = document.getElementById('profile-email-edit');
        const profileEmailControls = document.getElementById('profile-email-edit-controls');
        const profileEmailPassword = document.getElementById('profile-email-current-password');
        const profileEmailSave = document.getElementById('profile-email-save');
        const profileEmailStatus = document.getElementById('profile-email-status');

        profileNameInput?.addEventListener('change', async function() {
            const name = this.value.trim();
            if (!name) {
                this.value = this.dataset.savedName || '';
                return;
            }

            if (name === this.dataset.savedName) return;

            this.disabled = true;
            profileStatus.textContent = 'Saving name...';
            profileStatus.className = 'rounded-3xl border border-slate-200 bg-slate-50 px-5 py-4 text-sm text-slate-700';
            profileStatus.classList.remove('hidden');

            try {
                const response = await fetch('{{ route('profile.name.update') }}', {
                    method: 'POST',
                    headers: {
                        'Accept': 'application/json',
                        'Content-Type': 'application/json',
                        'X-CSRF-TOKEN': profileSettingsForm.querySelector('input[name="_token"]').value,
                    },
                    body: JSON.stringify({ name }),
                    credentials: 'same-origin',
                });
                if (response.status === 419) {
                    throw new Error('Your session expired. Refresh this page and try again.');
                }

                const data = await response.json();
                if (!response.ok) {
                    throw new Error(data.errors?.name?.[0] || data.message || 'Unable to save name.');
                }

```

```
                this.value = data.name;
                this.dataset.savedName = data.name;
                profileStatus.textContent = data.status;
                profileStatus.className = 'rounded-3xl border border-emerald-200 bg-emerald-50 px-5 py-4 text-sm text-emerald-800';
            } catch (error) {
                this.value = this.dataset.savedName || this.value;
                profileStatus.textContent = error.message;
                profileStatus.className = 'rounded-3xl border border-red-200 bg-red-50 px-5 py-4 text-sm text-red-800';
            } finally {
                this.disabled = false;
            }
        });
        profileNameInput.dataset.savedName = profileNameInput.value;

        profileEmailEdit?.addEventListener('click', function() {
            profileEmailInput.readOnly = false;
            profileEmailInput.classList.remove('text-gray-600');
            profileEmailInput.classList.add('text-gray-900');
            profileEmailInput.closest('div').classList.replace('bg-slate-100', 'bg-white');
            profileEmailEdit.classList.add('hidden');
            profileEmailControls.classList.remove('hidden');
            profileEmailStatus.textContent = '';
            profileEmailInput.focus();
            profileEmailInput.select();
        });

        document.getElementById('profile-email-cancel')?.addEventListener('click', function() {
            profileEmailInput.value = profileEmailInput.dataset.savedEmail;
            profileEmailInput.readOnly = true;
            profileEmailInput.classList.remove('text-gray-900');
            profileEmailInput.classList.add('text-gray-600');
            profileEmailInput.closest('div').classList.replace('bg-white', 'bg-slate-100');
            profileEmailEdit.classList.remove('hidden');
            profileEmailControls.classList.add('hidden');
            profileEmailPassword.value = '';
            profileEmailStatus.textContent = '';
            profileEmailStatus.className = 'mt-2 text-sm text-gray-600';
        });

        profileEmailInput.dataset.savedEmail = profileEmailInput.value;
        profileEmailSave?.addEventListener('click', async function() {
            if (!profileEmailInput.reportValidity()) return;

            profileEmailSave.disabled = true;
            profileEmailSave.textContent = 'Saving...';
            profileEmailStatus.textContent = '';

            try {
                const response = await fetch('{{ route('profile.email.update') }}', {
                    method: 'POST',
                    headers: {
                        'Accept': 'application/json',
                        'Content-Type': 'application/json',
                        'X-CSRF-TOKEN': profileSettingsForm.querySelector('input[name="_token"]').value,
                    },
                    body: JSON.stringify({
                        email: profileEmailInput.value.trim(),
                        current_password: profileEmailPassword.value,
                    }),
                    credentials: 'same-origin',
                });
                if (response.status === 419) {
                    throw new Error('Your session expired. Refresh this page and try again.');
                }

                const data = await response.json();
                if (!response.ok) {
                    throw new Error(data.errors?.email?.[0] || data.errors?.current_password?.[0] || data.message || 'Unable to update email address.');
                }

                profileEmailInput.value = data.email;
                profileEmailInput.dataset.savedEmail = data.email;
                profileEmailInput.readOnly = true;
                profileEmailInput.classList.remove('text-gray-900');
```

```
                profileEmailInput.classList.add('text-gray-600');
                profileEmailInput.closest('div').classList.replace('bg-white', 'bg-slate-100');
                profileEmailEdit.classList.remove('hidden');
                profileEmailControls.classList.add('hidden');
                profileEmailPassword.value = '';
                profileEmailStatus.textContent = data.status;
                profileEmailStatus.className = 'mt-2 text-sm text-emerald-700';
            } catch (error) {
                profileEmailStatus.textContent = error.message;
                profileEmailStatus.className = 'mt-2 text-sm text-red-600';
            } finally {
                profileEmailSave.disabled = false;
                profileEmailSave.textContent = 'Save email';
            }
        });

        profilePictureInput?.addEventListener('change', async function() {
            const picture = this.files?.[0];
            if (!picture) return;

            this.disabled = true;
            profilePictureUploadStatus.textContent = 'Uploading profile picture...';
            profilePictureUploadStatus.className = 'mt-2 text-sm text-gray-600';

            const formData = new FormData();
            formData.append('profile_picture', picture);
            formData.append('_token', profileSettingsForm.querySelector('input[name="_token"]').value);

            try {
                const response = await fetch('{{ route('profile.picture.upload') }}', {
                    method: 'POST',
                    headers: { 'Accept': 'application/json' },
                    body: formData,
                    credentials: 'same-origin',
                });
                if (response.status === 419) {
                    throw new Error('Your session expired. Refresh this page and try uploading again.');
                }
                const data = await response.json();

                if (!response.ok) {
                    const validationMessage = data.errors?.profile_picture?.[0];
                    throw new Error(validationMessage || data.message || 'Unable to upload profile picture.');
                }

                document.querySelectorAll('img[alt="Profile Picture"], img[alt="Profile picture"]').forEach(function(image) {
                    image.src = data.picture_url;
                });
                profilePictureUploadStatus.textContent = data.status || 'Profile picture uploaded successfully.';
                profilePictureUploadStatus.className = 'mt-2 text-sm text-emerald-700';
                this.value = '';
                this.disabled = false;
            } catch (error) {
                profilePictureUploadStatus.textContent = error.message;
                profilePictureUploadStatus.className = 'mt-2 text-sm text-red-600';
                this.disabled = false;
                this.value = '';
            }
        });
        const disableModalError = document.getElementById('disable-2fa-modal-error');
        const passwordTwoFactorModal = document.getElementById('password-2fa-modal');
        const passwordAuthenticatorCode = document.getElementById('password-authenticator-code');
        const passwordTwoFactorOtp = document.getElementById('password-2fa-otp');
        const passwordTwoFactorError = document.getElementById('password-2fa-error');
        const passwordTwoFactorAttempts = document.getElementById('password-2fa-attempts');
        const passwordModal = document.getElementById('password-modal');
        const passwordModalCurrent = document.getElementById('password-modal-current');
        const passwordModalNew = document.getElementById('password-modal-new');
        const passwordModalConfirm = document.getElementById('password-modal-confirm');
        const passwordModalError = document.getElementById('password-modal-error');
        const passwordRequirement = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()_+])[A-Za-z\d!@#$%^&*()_+]{5,30}$/;
        let passwordTwoFactorConfirmed = false;
        let disableConfirmed = false;

```

```
        emailNotificationsToggle?.addEventListener('change', async function() {
            const toggle = this;
            const previousValue = !toggle.checked;
            toggle.disabled = true;
            emailNotificationsStatus.textContent = 'Saving preference...';
            emailNotificationsStatus.className = 'mt-3 text-sm text-gray-600';

            try {
                const response = await fetch('{{ route('profile.email-notifications.update') }}', {
                    method: 'POST',
                    headers: {
                        'Accept': 'application/json',
                        'Content-Type': 'application/json',
                        'X-CSRF-TOKEN': profileSettingsForm.querySelector('input[name="_token"]').value,
                    },
                    body: JSON.stringify({ email_notifications: toggle.checked }),
                    credentials: 'same-origin',
                });
                const data = await response.json();

                if (!response.ok) {
                    throw new Error(data.errors?.email_notifications?.[0] || data.message || 'Unable to save this preference.');
                }

                emailNotificationsStatus.textContent = data.status || 'Email notification preference saved.';
                emailNotificationsStatus.className = 'mt-3 text-sm text-emerald-700';
            } catch (error) {
                toggle.checked = previousValue;
                emailNotificationsStatus.textContent = `${error.message} Your previous setting has been restored.`;
                emailNotificationsStatus.className = 'mt-3 text-sm text-red-600';
            } finally {
                toggle.disabled = false;
            }
        });

        function resetPasswordVisibility() {
            [passwordModalNew, passwordModalConfirm].forEach(function(input) {
                if (input.type === 'text') {
                    input.nextElementSibling?.click();
                }
            });
        }

        twoFactorToggle?.addEventListener('change', function() {
            const isDisabling = !this.checked && {{ auth()->user()->two_fa_enabled ? 'true' : 'false' }};
            const isEnabling = this.checked && {{ auth()->user()->two_fa_enabled ? 'false' : 'true' }};
            authenticatorSetupPanel?.classList.toggle('hidden', !isEnabling);
            if (setupCurrentPassword) {
                setupCurrentPassword.required = false;
            }

            if (isEnabling) {
                setupCurrentPassword?.focus();
            }

            if (isDisabling) {
                disableConfirmed = false;
                disableTwoFactorModal?.classList.remove('hidden');
                disableTwoFactorModal?.classList.add('flex');
                disableCurrentPassword?.focus();
            }
        });

        document.getElementById('cancel-disable-2fa')?.addEventListener('click', function() {
            twoFactorToggle.checked = true;
            disableTwoFactorModal?.classList.add('hidden');
            disableTwoFactorModal?.classList.remove('flex');
        });

        document.getElementById('continue-disable-2fa')?.addEventListener('click', function() {
            disableModalError?.classList.add('hidden');
            if (!disableCurrentPassword?.value || !disableAuthenticatorCode?.value.match(/^\d{6}$/)) {
                disableModalError.textContent = 'Enter your current password and a valid 6-digit authenticator code.';
                disableModalError.classList.remove('hidden');
```

```
                return;
            }

            const passwordInput = profileSettingsForm.querySelector('input[name="current_password"]');
            passwordInput.value = disableCurrentPassword.value;
            disableCurrentPassword.name = 'disable_current_password';
            disableAuthenticatorCode.name = 'disable_2fa_otp';
            disableConfirmed = true;
            disableTwoFactorModal?.classList.add('hidden');
            disableTwoFactorModal?.classList.remove('flex');
            profileSettingsForm.requestSubmit();
        });

        document.getElementById('generate-authenticator-qr')?.addEventListener('click', async function() {
            const button = this;
            const setupError = document.getElementById('authenticator-setup-error');
            if (!setupCurrentPassword?.value) {
                setupError.textContent = 'Enter your current password.';
                setupError.classList.remove('hidden');
                setupCurrentPassword?.focus();
                return;
            }

            button.disabled = true;
            button.textContent = 'Generating...';
            setupError.classList.add('hidden');

            try {
                const response = await fetch('{{ route('profile.2fa.setup') }}', {
                    method: 'POST',
                    headers: {
                        'Accept': 'application/json',
                        'X-CSRF-TOKEN': document.querySelector('input[name="_token"]').value,
                    },
                    body: new FormData(profileSettingsForm),
                });
                const data = await response.json();
                if (!response.ok) {
                    throw new Error(data.message || 'Unable to generate the QR code.');
                }

                document.getElementById('authenticator-qr-image').src = data.authenticator_setup_qr;
                document.getElementById('authenticator-secret').textContent = data.authenticator_setup_secret;
                document.getElementById('authenticator-qr-result').classList.remove('hidden');
                const authenticatorOtpInput = document.querySelector('input[name="authenticator_otp"]');
                authenticatorOtpInput.required = false;
                setupCurrentPassword.value = '';
                authenticatorOtpInput.focus();
            } catch (error) {
                setupError.textContent = error.message;
                setupError.classList.remove('hidden');
            } finally {
                button.disabled = false;
                button.textContent = 'Generate QR code';
            }
        });

        document.getElementById('cancel-authenticator-setup')?.addEventListener('click', function() {
            twoFactorToggle.checked = false;
            authenticatorSetupPanel?.classList.add('hidden');
            setupCurrentPassword.required = false;
            setupCurrentPassword.value = '';
            const authenticatorOtpInput = document.querySelector('input[name="authenticator_otp"]');
            if (authenticatorOtpInput) {
                authenticatorOtpInput.value = '';
                authenticatorOtpInput.required = false;
            }
            document.getElementById('authenticator-setup-error')?.classList.add('hidden');
            document.getElementById('authenticator-qr-result')?.classList.add('hidden');
        });

        profileSettingsForm?.addEventListener('submit', async function(event) {
            syncPhoneFieldsToForm();
            const isAuthenticatorAction = event.submitter?.formAction?.includes('/profile/2fa/');
```

```
673
674             if (isAuthenticatorAction) {
675                 event.preventDefault();
676                 const button = event.submitter;
677                 const originalText = button?.textContent;
678                 if (button) {
679                     button.disabled = true;
680                     button.textContent = 'Saving...';
681                 }
682
683                 try {
684                     const response = await fetch(button.formAction, {
685                         method: 'POST',
686                         headers: {
687                             'Accept': 'application/json',
688                             'X-Requested-With': 'XMLHttpRequest',
689                             'X-CSRF-TOKEN': profileSettingsForm.querySelector('input[name="_token"]').value,
690                         },
691                         body: new FormData(profileSettingsForm),
692                         credentials: 'same-origin',
693                     });
694                     if (response.status === 419) {
695                         throw new Error('Your session expired. Refresh this page and try again.');
696                     }
697                     const data = await response.json();
698                     if (!response.ok) {
699                         throw new Error(data.errors?.authenticator_otp?.[0] || data.message || 'Unable to save authenticator settings.');
700                     }
701
702                     twoFactorToggle.checked = true;
703                     authenticatorSetupPanel?.classList.add('hidden');
704                     document.getElementById('authenticator-qr-result')?.classList.add('hidden');
705                     const status = document.getElementById('profile-status');
706                     status.textContent = data.status || 'Authenticator app 2FA is now enabled.';
707                     status.className = 'rounded-3xl border border-emerald-200 bg-emerald-50 px-5 py-4 text-sm text-emerald-800';
708                     status.classList.remove('hidden');
709                 } catch (error) {
710                     const setupError = document.getElementById('authenticator-setup-error');
711                     setupError.textContent = error.message;
712                     setupError.classList.remove('hidden');
713                 } finally {
714                     if (button) {
715                         button.disabled = false;
716                         button.textContent = originalText;
717                     }
718                 }
719                 return;
720             }
721
722             if (!isAuthenticatorAction && {{ auth()->user()->two_fa_enabled ? 'true' : 'false' }} && this.querySelector('input[name="password"]')?.value &&
    !passwordTwoFactorConfirmed) {
723                 event.preventDefault();
724                 passwordTwoFactorModal?.classList.remove('hidden');
725                 passwordTwoFactorModal?.classList.add('flex');
726                 passwordAuthenticatorCode?.focus();
727                 return;
728             }
729
730             if (!isAuthenticatorAction && !{{ auth()->user()->two_fa_enabled ? 'true' : 'false' }} && twoFactorToggle.checked) {
731                 event.preventDefault();
732                 authenticatorSetupPanel?.classList.remove('hidden');
733                 const setupError = document.getElementById('authenticator-setup-error');
734                 setupError.textContent = 'Finish authenticator setup and verify a code before enabling two-factor authentication. Your other profile settings have not been
    saved yet.';
735                 setupError.classList.remove('hidden');
736                 if (document.getElementById('authenticator-qr-result').classList.contains('hidden')) {
737                     setupCurrentPassword?.focus();
738                 } else {
739                     document.querySelector('input[name="authenticator_otp"]')?.focus();
740                 }
741                 return;
742             }
743
744             if ({{ auth()->user()->two_fa_enabled ? 'true' : 'false' }} && !twoFactorToggle.checked && !disableConfirmed) {
```

```
                event.preventDefault();
                disableTwoFactorModal?.classList.remove('hidden');
                disableTwoFactorModal?.classList.add('flex');
                disableCurrentPassword?.focus();
                return;
            }

            event.preventDefault();
            const submitButton = event.submitter || document.getElementById('profile-settings-save');
            const originalText = submitButton?.textContent;
            if (submitButton) {
                submitButton.disabled = true;
                submitButton.textContent = 'Saving...';
            }

            try {
                const response = await fetch(profileSettingsForm.action, {
                    method: 'POST',
                    headers: {
                        'Accept': 'application/json',
                        'X-Requested-With': 'XMLHttpRequest',
                        'X-CSRF-TOKEN': profileSettingsForm.querySelector('input[name="_token"]').value,
                    },
                    body: new FormData(profileSettingsForm),
                    credentials: 'same-origin',
                });
                if (response.status === 419) {
                    throw new Error('Your session expired. Refresh this page and try again.');
                }
                const data = await response.json();
                if (!response.ok) {
                    const validationMessage = Object.values(data.errors || {}).flat()[0];
                    throw new Error(validationMessage || data.message || 'Unable to save profile settings.');
                }

                const status = document.getElementById('profile-status');
                status.textContent = data.status || 'Profile updated successfully.';
                status.className = 'rounded-3xl border border-emerald-200 bg-emerald-50 px-5 py-4 text-sm text-emerald-800';
                status.classList.remove('hidden');

                if (data.user?.phone) {
                    const phone = data.user.phone;
                    document.getElementById('phone_display_text').textContent = phone;
                    document.getElementById('form_phone_input').value = phone;
                    displayMode.classList.remove('hidden');
                    editMode.classList.add('hidden');
                }
                if (data.user?.two_fa_enabled === false) {
                    disableCurrentPassword.value = '';
                    disableAuthenticatorCode.value = '';
                    disableConfirmed = false;
                }
            } catch (error) {
                const status = document.getElementById('profile-status');
                status.textContent = error.message;
                status.className = 'rounded-3xl border border-red-200 bg-red-50 px-5 py-4 text-sm text-red-800';
                status.classList.remove('hidden');
            } finally {
                if (submitButton) {
                    submitButton.disabled = false;
                    submitButton.textContent = originalText;
                }
            }
        });

        document.getElementById('cancel-password-2fa')?.addEventListener('click', function() {
            passwordAuthenticatorCode.value = '';
            passwordTwoFactorOtp.value = '';
            passwordTwoFactorConfirmed = false;
            passwordTwoFactorModal.classList.add('hidden');
            passwordTwoFactorModal.classList.remove('flex');
        });

        document.getElementById('open-password-modal')?.addEventListener('click', function() {
```

```
            passwordTwoFactorConfirmed = false;
            passwordModalError.classList.add('hidden');
            passwordModal.classList.remove('hidden');
            passwordModal.classList.add('flex');
            passwordModalCurrent.focus();
        });

        document.getElementById('cancel-password-modal')?.addEventListener('click', function() {
            passwordModal.classList.add('hidden');
            passwordModal.classList.remove('flex');
            passwordModalCurrent.value = '';
            passwordModalNew.value = '';
            passwordModalConfirm.value = '';
            resetPasswordVisibility();
        });

        const forgotPasswordButton = document.getElementById('forgot-password-link');
        const resendPasswordResetButton = document.getElementById('resend-password-reset-link');
        const forgotPasswordStatus = document.getElementById('forgot-password-status');

        async function requestPasswordResetEmail(isResend) {
            const activeButton = isResend ? resendPasswordResetButton : forgotPasswordButton;
            const status = forgotPasswordStatus;
            const originalText = activeButton.textContent;
            forgotPasswordButton.disabled = true;
            resendPasswordResetButton.disabled = true;
            activeButton.textContent = isResend ? 'Resending...' : 'Sending reset link...';
            status.classList.remove('hidden');
            status.textContent = isResend
                ? 'Queuing another password reset email...'
                : 'Queuing a secure password reset email to your account address...';
            status.className = 'flex-1 text-sm text-gray-600';

            try {
                const response = await fetch('{{ route('profile.password.reset-link') }}', {
                    method: 'POST',
                    headers: {
                        'Accept': 'application/json',
                        'X-CSRF-TOKEN': profileSettingsForm.querySelector('input[name="_token"]').value,
                    },
                    credentials: 'same-origin',
                    signal: AbortSignal.timeout(20000),
                });
                const data = await response.json();

                if (!response.ok) {
                    throw new Error(data.message || 'Unable to send the reset link. Please try again shortly.');
                }

                status.textContent = isResend
                    ? 'Another password reset email was queued. Check your inbox and spam folder.'
                    : data.status;
                status.className = 'flex-1 text-sm text-emerald-700';
                resendPasswordResetButton.classList.remove('hidden');
            } catch (error) {
                status.textContent = error.name === 'TimeoutError' || error.name === 'AbortError'
                    ? 'The request timed out. The server did not confirm that a reset email was queued. Please try again, or contact an administrator.'
                    : error.message;
                status.className = 'flex-1 text-sm text-red-600';
            } finally {
                forgotPasswordButton.disabled = false;
                resendPasswordResetButton.disabled = false;
                activeButton.textContent = originalText;
            }
        }

        forgotPasswordButton?.addEventListener('click', function() {
            requestPasswordResetEmail(false);
        });

        resendPasswordResetButton?.addEventListener('click', function() {
            requestPasswordResetEmail(true);
        });

```

```
        document.getElementById('save-password-modal')?.addEventListener('click', async function() {
            passwordModalError.classList.add('hidden');
            if (!passwordModalCurrent.value || !passwordModalNew.value || !passwordModalConfirm.value) {
                passwordModalError.textContent = 'Complete all password fields.';
                passwordModalError.classList.remove('hidden');
                return;
            }

            if (passwordModalNew.value !== passwordModalConfirm.value) {
                passwordModalError.textContent = 'The new password and confirmation do not match.';
                passwordModalError.classList.remove('hidden');
                return;
            }

            if (!passwordRequirement.test(passwordModalNew.value)) {
                passwordModalError.textContent = 'Password must be at least 5 characters.';
                passwordModalError.classList.remove('hidden');
                passwordModalNew.focus();
                return;
            }

            if ({{ auth()->user()->two_fa_enabled ? 'true' : 'false' }}) {
                passwordModal.classList.add('hidden');
                passwordModal.classList.remove('flex');
                passwordTwoFactorModal.classList.remove('hidden');
                passwordTwoFactorModal.classList.add('flex');
                passwordTwoFactorAttempts.textContent = 'You have 3 attempts remaining.';
                passwordAuthenticatorCode.focus();
                return;
            }

            this.disabled = true;
            this.textContent = 'Saving...';

            try {
                const response = await fetch(profileSettingsForm.action, {
                    method: 'POST',
                    headers: {
                        'Accept': 'application/json',
                        'X-Requested-With': 'XMLHttpRequest',
                        'X-CSRF-TOKEN': profileSettingsForm.querySelector('input[name="_token"]').value,
                    },
                    body: new FormData(profileSettingsForm),
                    credentials: 'same-origin',
                });
                const data = await response.json();

                if (!response.ok) {
                    if (data.errors?.current_password) {
                        passwordModalError.textContent = data.errors.current_password[0];
                        passwordModalError.classList.remove('hidden');
                        passwordModalCurrent.focus();
                        return;
                    }

                    throw new Error(data.errors?.password?.[0] || data.message || 'Unable to change password.');
                }

                passwordModal.classList.add('hidden');
                passwordModal.classList.remove('flex');
                passwordModalCurrent.value = '';
                passwordModalNew.value = '';
                passwordModalConfirm.value = '';
                resetPasswordVisibility();
                const profileStatus = document.getElementById('profile-status');
                if (profileStatus) {
                    profileStatus.textContent = data.status || 'Password changed successfully.';
                    profileStatus.classList.remove('hidden');
                }
                if (data.password_changed) {
                    document.getElementById('password-change-alert')?.classList.remove('hidden');
                }
            } catch (error) {
                passwordModalError.textContent = error.message;
```

```
                passwordModalError.classList.remove('hidden');
            } finally {
                this.disabled = false;
                this.textContent = 'Save password';
            }
        });

        document.getElementById('confirm-password-2fa')?.addEventListener('click', async function() {
            passwordTwoFactorError.classList.add('hidden');
            if (!/^\d{6}$/.test(passwordAuthenticatorCode.value)) {
                passwordTwoFactorError.textContent = 'Enter a valid 6-digit authenticator code.';
                passwordTwoFactorError.classList.remove('hidden');
                return;
            }

            passwordTwoFactorOtp.value = passwordAuthenticatorCode.value;
            this.disabled = true;
            this.textContent = 'Saving...';

            try {
                const response = await fetch(profileSettingsForm.action, {
                    method: 'POST',
                    headers: {
                        'Accept': 'application/json',
                        'X-Requested-With': 'XMLHttpRequest',
                        'X-CSRF-TOKEN': profileSettingsForm.querySelector('input[name="_token"]').value,
                    },
                    body: new FormData(profileSettingsForm),
                    credentials: 'same-origin',
                });
                const data = await response.json();
                if (response.status === 429) {
                    passwordTwoFactorError.textContent = data.message || 'Too many invalid authenticator codes. Try again.';
                    passwordTwoFactorError.classList.remove('hidden');
                    passwordTwoFactorAttempts.textContent = 'No attempts remaining.';
                    passwordAuthenticatorCode.value = '';
                    passwordAuthenticatorCode.focus();
                    return;
                }
                if (!response.ok) {
                    if (data.errors?.current_password) {
                        passwordTwoFactorModal.classList.add('hidden');
                        passwordTwoFactorModal.classList.remove('flex');
                        passwordModal.classList.remove('hidden');
                        passwordModal.classList.add('flex');
                        passwordModalError.textContent = data.errors.current_password[0];
                        passwordModalError.classList.remove('hidden');
                        passwordModalCurrent.focus();
                        return;
                    }
                    if (data.errors?.password) {
                        passwordTwoFactorModal.classList.add('hidden');
                        passwordTwoFactorModal.classList.remove('flex');
                        passwordModal.classList.remove('hidden');
                        passwordModal.classList.add('flex');
                        passwordModalError.textContent = data.errors.password[0];
                        passwordModalError.classList.remove('hidden');
                        passwordModalNew.focus();
                        return;
                    }
                    passwordTwoFactorError.textContent = data.message || 'Invalid Authenticator Code';
                    passwordTwoFactorError.classList.remove('hidden');
                    passwordTwoFactorAttempts.textContent = `You have ${data.attempts_remaining || 2} attempts remaining.`;
                    passwordAuthenticatorCode.value = '';
                    passwordAuthenticatorCode.focus();
                    return;
                }

                passwordTwoFactorConfirmed = true;
                passwordTwoFactorModal.classList.add('hidden');
                passwordTwoFactorModal.classList.remove('flex');
                passwordModalCurrent.value = '';
                passwordModalNew.value = '';
                passwordModalConfirm.value = '';
```

```
                passwordTwoFactorOtp.value = '';
                const profileStatus = document.getElementById('profile-status');
                if (profileStatus) {
                    profileStatus.textContent = data.status || 'Password updated successfully.';
                    profileStatus.classList.remove('hidden');
                }
                if (data.password_changed) {
                    document.getElementById('password-change-alert')?.classList.remove('hidden');
                }
            } catch (requestError) {
                passwordTwoFactorError.textContent = requestError.message;
                passwordTwoFactorError.classList.remove('hidden');
                passwordAuthenticatorCode.value = '';
                passwordAuthenticatorCode.focus();
            } finally {
                this.disabled = false;
                this.textContent = 'Confirm';
            }
        });

        @if (session('show_disable_2fa_otp') || $errors->has('disable_current_password') || $errors->has('disable_2fa_otp'))
            disableTwoFactorModal?.classList.remove('hidden');
            disableTwoFactorModal?.classList.add('flex');
            disableCurrentPassword?.focus();
        @endif

        // Phone number edit functionality
        const displayMode = document.getElementById('phone_display_mode');
        const editMode = document.getElementById('phone_edit_mode');
        const editBtn = document.getElementById('phone_edit_btn');
        const saveBtn = document.getElementById('phone_save_btn');
        const cancelBtn = document.getElementById('phone_cancel_btn');
        const countryCodeSelect = document.getElementById('country_code_select');
        const phoneNumberInput = document.getElementById('phone_number_input');
        const displayText = document.getElementById('phone_display_text');
        const phoneHiddenInput = document.getElementById('form_phone_input');
        const countryCodeHiddenInput = document.getElementById('form_country_code_input');

        function syncPhoneFieldsToForm() {
            const countryCode = countryCodeSelect.value;
            const phoneNumber = phoneNumberInput.value.trim();

            if (phoneNumber) {
                phoneHiddenInput.value = countryCode + phoneNumber;
                countryCodeHiddenInput.value = countryCode;
                displayText.textContent = formatPhoneDisplay(countryCode, phoneNumber);
                return;
            }

            phoneHiddenInput.value = '';
            countryCodeHiddenInput.value = '';
            displayText.textContent = 'Not set';
        }

        // Parse current phone value to extract country code and number
        function parsePhone(phone) {
            if (!phone) {
                return { countryCode: '+63', number: '' };
            }

            const countryCodes = ['+1', '+44', '+63', '+81', '+91', '+86', '+33', '+49', '+39', '+61'];

            for (const code of countryCodes) {
                if (phone.startsWith(code)) {
                    return {
                        countryCode: code,
                        number: phone.substring(code.length)
                    };
                }
            }

            return { countryCode: '+63', number: phone };
        }

```

```
        // Format phone for display
        function formatPhoneDisplay(countryCode, number) {
            if (!number) return 'Not set';
            return countryCode + ' ' + number;
        }

        // Initialize with current phone data
        const currentPhone = '{{ auth()->user()->phone }}';
        const parsed = parsePhone(currentPhone);
        countryCodeSelect.value = parsed.countryCode;
        phoneNumberInput.value = parsed.number;
        if (currentPhone) {
            phoneHiddenInput.value = currentPhone;
            countryCodeHiddenInput.value = parsed.countryCode;
            displayMode.classList.remove('hidden');
            editMode.classList.add('hidden');
        } else {
            phoneHiddenInput.value = '';
            countryCodeHiddenInput.value = '';
            displayMode.classList.add('hidden');
            editMode.classList.remove('hidden');
        }

        profileSettingsForm?.addEventListener('submit', function() {
            syncPhoneFieldsToForm();
        });

        // Edit button click
        editBtn.addEventListener('click', function(e) {
            e.preventDefault();
            displayMode.classList.add('hidden');
            editMode.classList.remove('hidden');
            phoneNumberInput.focus();
        });

        // Save button click
        saveBtn.addEventListener('click', function(e) {
            e.preventDefault();
            const countryCode = countryCodeSelect.value;
            const phoneNumber = phoneNumberInput.value.trim();

            if (!phoneNumber) {
                alert('Please enter a phone number');
                return;
            }

            syncPhoneFieldsToForm();

            if (profileSettingsForm) {
                profileSettingsForm.requestSubmit();
                return;
            }

            const formattedPhone = formatPhoneDisplay(countryCode, phoneNumber);
            displayText.textContent = formattedPhone;
            displayMode.classList.remove('hidden');
            editMode.classList.add('hidden');
        });

        // Cancel button click
        cancelBtn.addEventListener('click', function(e) {
            e.preventDefault();
            // Reset to current values
            countryCodeSelect.value = parsed.countryCode;
            phoneNumberInput.value = parsed.number;

            // Switch back to display mode
            displayMode.classList.remove('hidden');
            editMode.classList.add('hidden');
        });

        // Allow Enter key to save
        phoneNumberInput.addEventListener('keypress', function(e) {
            if (e.key === 'Enter') {
```

```
                e.preventDefault();
                saveBtn.click();
            }
        });
    });
</script>
@endsection
```

resources/views/reservations/edit.blade.php

```
@extends('layouts.app')

@section('title', 'Edit Reservation')

@section('content')
<div class="min-h-screen bg-slate-50 py-16">
    <div class="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8">
        <div class="bg-white shadow-sm rounded-3xl border border-slate-200 p-8">
            <div class="flex items-start justify-between gap-4 mb-6">
                <div>
                    <p class="text-sm uppercase tracking-[0.3em] text-slate-500">Edit reservation</p>
                    <h1 class="text-3xl font-semibold text-slate-900 mt-3">Update your booking request</h1>
                </div>
                <div class="text-right">
                    <p class="text-sm text-slate-500">Current status</p>
                    <span class="inline-flex items-center rounded-full px-3 py-1 text-sm font-medium {{ $reservation->statusBadge['class'] ?? 'bg-gray-100 text-gray-800' }}">
                        {{ $reservation->statusBadge['label'] ?? ucfirst($reservation->status) }}
                    </span>
                </div>
            </div>

            @if(session('error'))
                <div class="mb-4 rounded-2xl border border-rose-200 bg-rose-50 px-4 py-3 text-rose-700">
                    {{ session('error') }}
                </div>
            @endif

            @if ($errors->any())
                <div class="mb-4 rounded-2xl border border-rose-200 bg-rose-50 px-4 py-3 text-rose-700">
                    <p class="font-semibold">Please fix the following errors:</p>
                    <ul class="mt-2 list-disc pl-5 space-y-1">
                        @foreach ($errors->all() as $error)
                            <li>{{ $error }}</li>
                        @endforeach
                    </ul>
                </div>
            @endif

            <form method="POST" action="{{ route('reservations.update', $reservation) }}">
                @csrf
                @method('PUT')

                <div class="grid grid-cols-1 gap-6">
                    @if($reservation->items->isNotEmpty())
                        <div class="rounded-2xl bg-slate-50 p-4">
                            <p class="text-sm font-medium text-slate-700">Independent property reservation</p>
                            <p class="mt-1 text-sm text-slate-600">
                                Items:
                                @foreach($reservation->items as $item)
                                    {{ $item->name }} × {{ $item->pivot->quantity }}@if(!$loop->last), @endif
                                @endforeach
                            </p>
                        </div>
                    @else
                        <div>
                            <label for="facility_id" class="block text-sm font-medium text-slate-700 mb-2">Facility</label>
                            <select id="facility_id" name="facility_id" required class="w-full rounded-3xl border border-slate-200 bg-slate-50 px-4 py-3 text-slate-700
focus:border-slate-400 focus:outline-none">
                                @foreach($facilities as $facility)
                                    <option value="{{ $facility->id }}" {{ $reservation->facility_id === $facility->id ? 'selected' : '' }}>{{ $facility->name }}
@if($facility->building) - {{ $facility->building }}@endif</option>
                                @endforeach
                            </select>
                            @error('facility_id')
```

```
                                <p class="mt-2 text-sm text-rose-600">{{ $message }}</p>
                            @enderror
                        </div>
                    @endif

                    <div class="grid grid-cols-1 sm:grid-cols-2 gap-6">
                        <div>
                            <label for="reservation_date" class="block text-sm font-medium text-slate-700 mb-2">Date</label>
                            <input id="reservation_date" name="reservation_date" type="date" max="{{ $maximumReservationDate }}" value="{{ old('reservation_date', $reservation->reservation_date->toDateString()) }}" class="w-full rounded-3xl border border-slate-200 bg-slate-50 px-4 py-3 text-slate-700 focus:border-slate-400 focus:outline-none" />
                            @error('reservation_date')
                                <p class="mt-2 text-sm text-rose-600">{{ $message }}</p>
                            @enderror
                        </div>
                        <div>
                            <label for="start_time" class="block text-sm font-medium text-slate-700 mb-2">Start time</label>
                            <input id="start_time" name="start_time" type="time" value="{{ old('start_time', $reservation->start_time->format('H:i')) }}" class="w-full h-12 rounded-3xl border border-slate-200 bg-slate-50 px-4 py-2.5 text-sm leading-6 text-slate-700 focus:border-slate-400 focus:outline-none" />
                            @error('start_time')
                                <p class="mt-2 text-sm text-rose-600">{{ $message }}</p>
                            @enderror
                        </div>
                        <div>
                            <label for="end_time" class="block text-sm font-medium text-slate-700 mb-2">End time</label>
                            <input id="end_time" name="end_time" type="time" value="{{ old('end_time', $reservation->end_time->format('H:i')) }}" class="w-full h-12 rounded-3xl border border-slate-200 bg-slate-50 px-4 py-2.5 text-sm leading-6 text-slate-700 focus:border-slate-400 focus:outline-none" />
                            @error('end_time')
                                <p class="mt-2 text-sm text-rose-600">{{ $message }}</p>
                            @enderror
                        </div>
                    </div>

                    <div>
                        <label for="purpose" class="block text-sm font-medium text-slate-700 mb-2">Purpose</label>
                        <input id="purpose" name="purpose" type="text" value="{{ old('purpose', $reservation->purpose) }}" class="w-full rounded-3xl border border-slate-200 bg-slate-50 px-4 py-3 text-slate-700 focus:border-slate-400 focus:outline-none" />
                        @error('purpose')
                            <p class="mt-2 text-sm text-rose-600">{{ $message }}</p>
                        @enderror
                    </div>

                    <div>
                        <label for="notes" class="block text-sm font-medium text-slate-700 mb-2">Notes (optional)</label>
                        <textarea id="notes" name="notes" rows="4" class="w-full rounded-3xl border border-slate-200 bg-slate-50 px-4 py-3 text-slate-700 focus:border-slate-400 focus:outline-none">{{ old('notes', $reservation->notes) }}</textarea>
                        @error('notes')
                            <p class="mt-2 text-sm text-rose-600">{{ $message }}</p>
                        @enderror
                    </div>
                </div>

                <div class="mt-8 grid gap-4 sm:grid-cols-3 sm:items-center">
                    <a href="{{ route('reserve') }}" class="inline-flex items-center justify-center rounded-3xl border border-slate-200 bg-white px-6 py-3 text-sm font-semibold text-slate-700 hover:bg-slate-50 transition">Cancel</a>
                    <button type="submit" class="inline-flex items-center justify-center rounded-3xl bg-sky-700 px-6 py-3 text-sm font-semibold text-white hover:bg-sky-600 transition">Save and resend for approval</button>
                    <form method="POST" action="{{ route('reservations.destroy', $reservation) }}" onsubmit="return confirm('Delete this reservation?');" class="w-full">
                        @csrf
                        @method('DELETE')
                        <button type="submit" class="inline-flex w-full items-center justify-center rounded-3xl border border-rose-200 bg-white px-6 py-3 text-sm font-semibold text-rose-700 hover:bg-rose-50 transition">Delete reservation</button>
                    </form>
                </div>
            </form>

            <p class="mt-4 text-sm text-slate-500">Editing an approved reservation will revert it to a pending approval request.</p>
        </div>
    </div>
</div>
@endsection
```

resources/views/reserve.blade.php

```
@extends('layouts.app')

@section('title', 'Reserve - Campus Reserve')

@push('styles')
<style>
    #reservationList {
        max-height: 26rem;
        overflow-y: scroll;
        padding-right: 0.5rem;
        scrollbar-color: #94a3b8 #f1f5f9;
        scrollbar-width: thin;
    }

    #reservationList::-webkit-scrollbar {
        width: 8px;
    }

    #reservationList::-webkit-scrollbar-track {
        background: #f1f5f9;
        border-radius: 9999px;
    }

    #reservationList::-webkit-scrollbar-thumb {
        background: #94a3b8;
        border-radius: 9999px;
    }

    .reservation-options-scroll {
        max-height: 32rem;
        overflow-y: auto;
        padding-right: 0.5rem;
        scrollbar-color: #fbbf24 #fffbeb;
        scrollbar-width: thin;
    }

    .reservation-options-scroll::-webkit-scrollbar {
        width: 8px;
    }

    .reservation-options-scroll::-webkit-scrollbar-track {
        background: #fffbeb;
        border-radius: 9999px;
    }

    .reservation-options-scroll::-webkit-scrollbar-thumb {
        background: #fbbf24;
        border-radius: 9999px;
    }
</style>
@endpush

@section('content')
<div class="min-h-screen bg-gradient-to-br from-purple-50 via-slate-50 to-cyan-50">
    <div class="mx-auto max-w-7xl px-3 py-6 sm:px-6 sm:py-10 lg:px-8 lg:py-12">
        <!-- Welcome Section -->
        <!-- Dashboard greeting and summary for the current logged in user. -->
        <div class="relative mb-8 overflow-hidden rounded-3xl border border-white/80 bg-gradient-to-r from-indigo-100/90 via-pink-50 to-cyan-100/80 p-7 shadow-sm sm:p-9">
            <div aria-hidden="true" class="absolute -right-10 -top-20 h-56 w-56 rounded-full bg-white/50 blur-3xl"></div>
            <p class="relative text-sm font-semibold uppercase tracking-[0.16em] text-indigo-700">Campus Reserve</p>
            <h1 class="relative mt-2 text-3xl font-semibold tracking-tight text-slate-950">Welcome, {{ Auth::user()->name }}!</h1>
            <p class="relative mt-2 text-slate-600">Manage your reservations and explore available facilities.</p>
        </div>

        @if(session('success'))
            <div class="mb-6 rounded-3xl border border-emerald-200 bg-emerald-50 p-4 text-emerald-900">
                {{ session('success') }}
            </div>
        @elseif(session('error'))
            <div class="mb-6 rounded-3xl border border-rose-200 bg-rose-50 p-4 text-rose-900">
                {{ session('error') }}
            </div>
```

```
        @endif
        <div id="property-booking-feedback" class="mb-6 hidden rounded-3xl border border-emerald-200 bg-emerald-50 p-4 text-emerald-900" role="status"></div>

        <!-- Quick Stats -->
        <!-- Summary cards showing the user's reservation and facility counts. -->
        <div id="facility-reservation-statistics" data-reservation-statistics="facility" class="mb-6 grid grid-cols-2 gap-3 sm:mb-8 sm:gap-6 md:grid-cols-3">
            <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-4 shadow-sm sm:p-6">
                <div class="flex items-center">
                    <div class="rounded-2xl bg-indigo-100 p-3">
                        <svg class="h-6 w-6 text-indigo-700" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 002-2V7a2 2 0 00-2-2H5a2 2 0 00-2
    2v12a2 2 0 002 2z"></path>
                        </svg>
                    </div>
                    <div class="ml-4">
                        <p class="text-sm font-medium text-gray-600">Upcoming Reservations</p>
                        <p class="text-2xl font-bold text-gray-900">{{ $upcomingCount ?? 0 }}</p>
                    </div>
                </div>
            </div>

            <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-4 shadow-sm sm:p-6">
                <div class="flex items-center">
                    <div class="rounded-2xl bg-emerald-100 p-3">
                        <svg class="h-6 w-6 text-emerald-700" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9 12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z"></path>
                        </svg>
                    </div>
                    <div class="ml-4">
                        <p class="text-sm font-medium text-gray-600">Completed</p>
                        <p class="text-2xl font-bold text-gray-900">{{ $completedCount ?? 0 }}</p>
                    </div>
                </div>
            </div>

            <div class="col-span-2 rounded-3xl border border-slate-200/80 bg-white/90 p-4 shadow-sm sm:col-span-1 sm:p-6">
                <div class="flex items-center">
                    <div class="rounded-2xl bg-cyan-100 p-3">
                        <svg class="h-6 w-6 text-cyan-700" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 21V5a2 2 0 00-2-2H7a2 2 0 00-2 2v16m14 0h2m-2 0h-5m-9 0H3m2 0h5M9
    7h1m-1 4h1m4-4h1m-1 4h1m-5 10v-5a1 1 0 011-1h2a1 1 0 011 1v5m-4 0h4"></path>
                        </svg>
                    </div>
                    <div class="ml-4">
                        <p class="text-sm font-medium text-gray-600">Available Facilities</p>
                        <p class="text-2xl font-bold text-gray-900">{{ $facilities->count() }}</p>
                    </div>
                </div>
            </div>

        </div>
        <div id="property-reservation-statistics" data-reservation-statistics="property" class="mb-6 grid grid-cols-2 gap-3 sm:mb-8 sm:gap-6">
            <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-4 shadow-sm sm:p-6">
                <div class="flex items-center">
                    <div class="rounded-2xl bg-indigo-100 p-3">
                        <svg class="h-6 w-6 text-indigo-700" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M20 7l-8-4-8 4m16 0l-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4 7v10l8 4"></path>
                        </svg>
                    </div>
                    <div class="ml-4">
                        <p class="text-sm font-medium text-gray-600">Borrowed Items</p>
                        <p class="text-2xl font-bold text-gray-900">{{ $borrowedItemCount ?? 0 }}</p>
                    </div>
                </div>
            </div>
            <div class="rounded-3xl border border-slate-200/80 bg-white/90 p-4 shadow-sm sm:p-6">
                <div class="flex items-center">
                    <div class="rounded-2xl bg-cyan-100 p-3">
                        <svg class="h-6 w-6 text-cyan-700" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9 12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z"></path>
                        </svg>
                    </div>
                    <div class="ml-4">
                        <p class="text-sm font-medium text-gray-600">Returned Items</p>
```

```
                        <p class="text-2xl font-bold text-gray-900">{{ $returnedItemCount ?? 0 }}</p>
                    </div>
                </div>
            </div>
        </div>

        <div class="mb-8 grid grid-cols-1 gap-5 lg:grid-cols-2">
            <section id="facility-reservation-panel" class="min-w-0 overflow-hidden rounded-3xl border border-slate-200/80 bg-white/95 p-4 shadow-sm shadow-indigo-950/5 sm:p-6">
                <div class="mb-5 flex items-center justify-between gap-3">
                    <h2 class="text-xl font-semibold text-gray-900">Available Facilities</h2>
                    <button type="button" id="facility-reserve-now" class="inline-flex shrink-0 items-center justify-center rounded-xl bg-emerald-700 px-4 py-2.5 text-sm font-semibold text-white transition hover:bg-emerald-800 focus:outline-none focus:ring-2 focus:ring-emerald-300 focus:ring-offset-2">
                        Reserve Now
                    </button>
                </div>
                <div class="mb-5">
                    <label class="mt-4 block">
                        <span class="sr-only">Search available facilities</span>
                        <input id="available-facilities-search" type="search" placeholder="Search rooms or buildings" class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-sm focus:border-indigo-500 focus:outline-none focus:ring-2 focus:ring-indigo-100" autocomplete="off">
                    </label>
                </div>
                @if($facilities->isEmpty())
                    <div id="available-facilities-empty" class="py-12 text-center text-gray-500">No facilities are available right now.</div>
                @else
                    <div id="available-facilities-list" class="reservation-options-scroll space-y-3" tabindex="0" aria-label="Available facilities">
                        @foreach($facilities as $facility)
                            <div data-available-facility-card data-search="{{ strtolower($facility->name . ' ' . ($facility->building ?? '') . ' ' . $facility->typeLabel . ' ' . ($facility->capacity ?? '')) }}" class="flex items-center justify-between gap-4 rounded-full border border-slate-100 bg-white px-5 py-4 shadow-sm transition hover:border-indigo-100 hover:shadow-md">
                                <div class="min-w-0">
                                    <h3 class="text-lg font-semibold text-gray-900">{{ $facility->name }}</h3>
                                    @if($facility->building)
                                        <p class="mt-1 text-sm text-gray-500">{{ $facility->building }}</p>
                                    @endif
                                </div>
                                <div class="shrink-0 text-right text-sm text-gray-600">
                                    <p>{{ $facility->typeLabel }}</p>
                                    <p>Capacity: {{ $facility->capacity ?? 'N/A' }}</p>
                                </div>
                            </div>
                        @endforeach
                    </div>
                    <p id="available-facilities-no-results" class="hidden py-8 text-center text-sm text-slate-500" role="status">No rooms match your search.</p>
                @endif
            </section>

            <section id="property-reservation-panel" class="min-w-0 overflow-hidden rounded-3xl border border-slate-200/80 bg-white/95 p-4 shadow-sm shadow-indigo-950/5 sm:p-6">
                <div class="mb-5 flex items-center justify-between gap-3">
                    <h2 class="text-xl font-semibold text-gray-900">Available Property Items</h2>
                    <button type="button" id="property-reserve-now" class="inline-flex shrink-0 items-center justify-center rounded-xl bg-emerald-700 px-4 py-2.5 text-sm font-semibold text-white transition hover:bg-emerald-800 focus:outline-none focus:ring-2 focus:ring-emerald-300 focus:ring-offset-2 disabled:cursor-not-allowed disabled:opacity-50" disabled>
                        Reserve Now
                    </button>
                </div>
                <label class="mb-4 block">
                    <span class="sr-only">Search available property items</span>
                    <input id="available-properties-search" type="search" placeholder="Search property items" class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-sm focus:border-indigo-500 focus:outline-none focus:ring-2 focus:ring-indigo-100" autocomplete="off">
                </label>
                <div id="available-properties-list"
                    data-items-url="{{ route('api.facilities.property-items') }}"
                    class="reservation-options-scroll grid grid-cols-1 gap-5 lg:grid-cols-2"
                    tabindex="0"
                    aria-label="Available property items"
                    aria-live="polite">
                    <p class="py-8 text-center text-sm text-slate-500">Loading property items...</p>
                </div>
            </section>
        </div>

```

```
        <div id="facility-calendar-modal" class="fixed inset-0 z-[70] hidden items-center justify-center bg-slate-950/40 p-2 backdrop-blur-sm sm:p-4" role="dialog" aria-modal="true" aria-labelledby="facility-calendar-modal-title">
            <div class="flex h-[99dvh] max-h-[99dvh] w-full max-w-[96rem] flex-col overflow-hidden rounded-3xl border border-slate-200 bg-white p-3 shadow-2xl sm:p-5">
                <div class="mb-3 flex shrink-0 items-center justify-between gap-3">
                    <h2 id="facility-calendar-modal-title" class="text-lg font-semibold text-slate-900 sm:text-xl">Reserve a facility</h2>
                    <button type="button" id="facility-calendar-close" class="inline-flex h-10 w-10 shrink-0 items-center justify-center rounded-full text-2xl leading-none text-slate-500 transition hover:bg-slate-100 hover:text-slate-900 focus:outline-none focus:ring-2 focus:ring-emerald-600" aria-label="Close facility reservation calendar">&times;</button>
                </div>
                <iframe
                    id="facility-calendar-frame"
                    data-src="{{ route('calendar.manage', ['embedded' => 1]) }}"
                    title="Facility reservation calendar"
                    class="min-h-0 w-full flex-1 rounded-2xl border border-slate-100 bg-[#f3f7f4]"
                    loading="lazy">
                </iframe>
            </div>
        </div>

        <div id="property-booking-modal" class="fixed inset-0 z-50 hidden items-center justify-center overflow-y-auto bg-slate-900/40 p-4 backdrop-blur-sm" role="dialog" aria-modal="true" aria-labelledby="property-booking-title">
            <div class="my-auto w-full max-w-2xl rounded-3xl bg-white p-6 shadow-2xl sm:p-8">
                <div class="flex items-start justify-between gap-4">
                    <div>
                        <p class="text-sm font-medium text-slate-500">Property reservation</p>
                        <h2 id="property-booking-title" class="mt-1 text-2xl font-semibold text-slate-900">Reserve property items</h2>
                        <p id="property-booking-facility" class="mt-1 text-sm text-slate-600">Property inventory is independent of rooms and facilities.</p>
                    </div>
                    <button type="button" id="property-booking-close" class="rounded-full p-2 text-slate-500 hover:bg-slate-100 hover:text-slate-900" aria-label="Close reservation dialog">&times;</button>
                </div>

                <form id="property-booking-form" class="mt-6 space-y-5">
                    <div>
                        <label for="property-booking-purpose" class="block text-sm font-medium text-slate-700">Purpose</label>
                        <input id="property-booking-purpose" type="text" maxlength="255" required placeholder="e.g. club meeting" class="mt-2 w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 py-3 text-sm text-slate-700 focus:border-slate-500 focus:outline-none focus:ring-1 focus:ring-slate-500">
                    </div>
                    <div>
                        <label for="property-booking-item-select" class="block text-sm font-medium text-slate-700">Property item</label>
                        <select id="property-booking-item-select" class="mt-2 w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 py-3 text-sm text-slate-700 focus:border-slate-500 focus:outline-none focus:ring-1 focus:ring-slate-500"></select>
                    </div>
                    <div class="grid gap-4 sm:grid-cols-3">
                        <div>
                            <label for="property-booking-date" class="block text-sm font-medium text-slate-700">Date</label>
                            <input id="property-booking-date" type="date" max="{{ $maximumReservationDate }}" required class="mt-2 h-12 w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 text-sm text-slate-700 focus:border-slate-500 focus:outline-none focus:ring-1 focus:ring-slate-500">
                        </div>
                        <div>
                            <label for="property-booking-start" class="block text-sm font-medium text-slate-700">Time to use</label>
                            <input id="property-booking-start" type="time" value="{{ $reservationHours['start_time'] }}" min="{{ $reservationHours['start_time'] }}" max="{{ $reservationHours['end_time'] }}" required class="mt-2 h-12 w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 text-sm text-slate-700 focus:border-slate-500 focus:outline-none focus:ring-1 focus:ring-slate-500">
                        </div>
                        <div>
                            <label for="property-booking-end" class="block text-sm font-medium text-slate-700">Time to return</label>
                            <input id="property-booking-end" type="time" value="{{ $reservationHours['end_time'] }}" min="{{ $reservationHours['start_time'] }}" max="{{ $reservationHours['end_time'] }}" required class="mt-2 h-12 w-full rounded-2xl border border-slate-200 bg-slate-50 px-4 text-sm text-slate-700 focus:border-slate-500 focus:outline-none focus:ring-1 focus:ring-slate-500">
                        </div>
                    </div>
                    <p class="text-xs font-medium text-green-900">
                        Reservations allowed {{ date('g:i A', strtotime($reservationHours['start_time'])) }}–{{ date('g:i A', strtotime($reservationHours['end_time'])) }}.
                    </p>

                    <div>
                        <div class="flex flex-wrap items-baseline justify-between gap-2">
                            <h3 class="text-sm font-semibold text-slate-900">Selected item and quantity</h3>
                            <p class="text-xs text-slate-500">Available quantity updates for the selected time.</p>
                        </div>
                        <div id="property-booking-items" class="mt-3 max-h-56 space-y-2 overflow-y-auto rounded-2xl border border-slate-200 p-3"></div>
                    </div>

```

```
                    <p id="property-booking-error" class="hidden whitespace-pre-line rounded-xl border border-rose-200 bg-rose-50 px-4 py-3 text-sm text-rose-800" role="alert"></p>
                    <div class="flex justify-end gap-3 border-t border-slate-100 pt-5">
                        <button type="button" id="property-booking-cancel" class="rounded-2xl border border-slate-200 bg-white px-5 py-3 text-sm font-semibold text-slate-700 transition hover:bg-slate-50">Cancel</button>
                        <button type="submit" id="property-booking-submit" class="rounded-2xl bg-slate-900 px-5 py-3 text-sm font-semibold text-white transition hover:bg-slate-800">Reserve Item</button>
                    </div>
                </form>
            </div>
        </div>

        <!-- My Reservations -->
        <div class="rounded-3xl border border-slate-200/80 bg-white/95 p-6 shadow-sm shadow-indigo-950/5">
            <h2 class="mb-4 text-xl font-semibold tracking-tight text-slate-950">My Reservations</h2>
            <form method="GET" action="{{ route('reserve') }}" class="mb-5 flex flex-col gap-3 sm:flex-row">
                <input type="hidden" name="tab" value="{{ request('tab') }}">
                <label class="block min-w-0 flex-1">
                    <span class="sr-only">Search my reservations</span>
                    <input type="search" name="reservation_search" value="{{ $reservationSearch ?? '' }}" placeholder="Search by room, purpose, status, or item" class="w-full rounded-xl border border-slate-300 bg-white px-4 py-3 text-sm focus:border-indigo-500 focus:outline-none focus:ring-2 focus:ring-indigo-100" autocomplete="off">
                </label>
                <div class="flex gap-2">
                    <button type="submit" class="rounded-xl bg-indigo-700 px-5 py-3 text-sm font-semibold text-white transition hover:bg-indigo-800">Search</button>
                    @if(!empty($reservationSearch))
                        <a href="{{ route('reserve', request('tab') ? ['tab' => request('tab')] : []) }}" class="inline-flex items-center rounded-xl border border-slate-300 px-4 py-3 text-sm font-semibold text-slate-700 hover:bg-slate-50">Clear</a>
                    @endif
                </div>
            </form>
            @if(isset($reservations) && $reservations->isNotEmpty())
                <form id="delete-selected-form" method="POST" action="{{ route('reservations.destroySelected') }}">
                    @csrf
                    <div id="reservationList" class="space-y-4">
                        @foreach($reservations as $reservation)
                            <div class="rounded-2xl border border-slate-200 bg-white p-5 transition hover:border-indigo-200 hover:shadow-md">
                                <div class="flex flex-col md:flex-row md:items-center md:justify-between gap-4">
                                    <div>
                                        <p class="text-sm text-gray-500">{{ $reservation->items->isNotEmpty() ? 'Property items' : ($reservation->facility->name ?? 'Facility') }}</p>
                                        <h3 class="text-lg font-semibold text-gray-900">{{ $reservation->purpose }}</h3>
                                        <p class="text-sm text-gray-600 mt-2">
                                            {{ $reservation->reservation_date->format('M d, Y') }} ·
                                            @if($reservation->items->isNotEmpty())
                                                Use: {{ $reservation->start_time->format('h:i A') }} · Return: {{ $reservation->end_time->format('h:i A') }}
                                            @else
                                                {{ $reservation->start_time->format('h:i A') }} - {{ $reservation->end_time->format('h:i A') }}
                                            @endif
                                        </p>
                                    </div>
                                    <div class="flex flex-col items-start sm:items-end gap-3">
                                        <span class="inline-flex items-center rounded-full px-3 py-1 text-sm font-medium {{ $reservation->statusBadge['class'] ?? 'bg-gray-100 text-gray-800' }}">
                                            {{ $reservation->statusBadge['label'] ?? ucfirst($reservation->status) }}
                                        </span>
                                        <label class="inline-flex items-center gap-2 text-sm text-slate-600">
                                            <input type="checkbox" name="selected_reservations[]" value="{{ $reservation->id }}" class="h-4 w-4 rounded border-slate-300 text-sky-600 focus:ring-sky-500" />
                                            Delete when selected
                                        </label>
                                    </div>
                                </div>
                                @if($reservation->notes)
                                    <p class="mt-4 text-sm text-gray-600">Notes: {{ $reservation->notes }}</p>
                                @endif
                                @if($reservation->items->isNotEmpty())
                                    <p class="mt-3 text-sm text-gray-600">
                                        Items requested:
                                        @foreach($reservation->items as $item)
                                            {{ $item->name }} × {{ $item->pivot->quantity }}@if(!$loop->last), @endif
                                        @endforeach
                                    </p>
                                @endif

```

```
                                <div class="mt-4 flex flex-wrap gap-3">
                                    @if(in_array($reservation->status, ['approved', 'rejected', 'pending']))
                                        <a href="{{ route('reservations.edit', $reservation) }}" class="inline-flex items-center rounded-full border border-slate-200 bg-slate-50 px-4 py-2 text-sm font-medium text-slate-700 hover:bg-slate-100 transition">Edit reservation</a>
                                    @endif
                                    @if($reservation->status === 'rejected' || $reservation->status === 'pending')
                                        <button type="button" class="inline-flex items-center rounded-full bg-rose-600 px-4 py-2 text-sm font-medium text-white hover:bg-rose-700 transition" onclick="openDeleteModal({{ $reservation->id }}, @json($reservation->facility->name ?? 'this reservation'))">Remove</button>
                                    @endif
                                </div>
                            </div>
                        @endforeach
                    </div>
                    <div class="mt-6 flex justify-end">
                        <button type="button" onclick="openBulkDeleteModal()" class="inline-flex items-center rounded-full bg-rose-600 px-5 py-3 text-sm font-semibold text-white hover:bg-rose-700 transition">Delete selected</button>
                    </div>
                </form>
                @foreach($reservations as $reservation)
                    <form id="delete-reservation-{{ $reservation->id }}" method="POST" action="{{ route('reservations.destroy', $reservation) }}" class="hidden">
                        @csrf
                        @method('DELETE')
                    </form>
                @endforeach

                <div class="mt-6">
                    {{ $reservations->links() }}
                </div>

                <div id="delete-modal" class="fixed inset-0 z-50 hidden items-center justify-center bg-slate-900/30 px-4 py-8 backdrop-blur-sm">
                    <div class="w-full max-w-md rounded-3xl border border-slate-200 bg-white p-6 shadow-2xl">
                        <h3 class="text-xl font-semibold text-gray-900">Confirm delete</h3>
                        <p class="mt-3 text-sm text-gray-600" id="delete-modal-message">Are you sure you want to remove this reservation?</p>
                        <div class="mt-6 flex justify-end gap-3">
                            <button type="button" class="rounded-full border border-gray-300 px-4 py-2 text-sm font-medium text-slate-700 hover:bg-gray-50" onclick="closeDeleteModal()">No</button>
                            <button type="button" id="delete-modal-confirm" class="rounded-full bg-rose-600 px-4 py-2 text-sm font-medium text-white hover:bg-rose-700">Yes, delete</button>
                        </div>
                    </div>
                </div>
            @else
                <div class="text-center py-8 text-gray-500">
                    <svg class="w-12 h-12 mx-auto mb-4 text-gray-300" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2 2 0 00-2-2h-2M9 5a2 2 0 002 2h2a2 2 0 002-2M9 5a2 2 0 012-2h2a2 2 0 012 2"></path>
                    </svg>
                    @if(!empty($reservationSearch))
                        <p>No reservations match “{{ $reservationSearch }}”.</p>
                        <p class="text-sm">Try a different room, purpose, status, or item name.</p>
                    @else
                        <p>No reservations yet.</p>
                        <p class="text-sm">Start by reserving a facility above!</p>
                    @endif
                </div>
            @endif
        </div>
    </div>
</div>

<div id="feedback-survey-modal" class="fixed inset-0 z-50 hidden items-center justify-center bg-slate-900/30 px-4 py-8 backdrop-blur-sm">
    <div class="w-full max-w-md rounded-3xl border border-slate-200 bg-white p-6 shadow-2xl">
        <div class="flex items-start justify-between gap-3">
            <div>
                <p class="text-sm font-medium uppercase tracking-wide text-slate-500">Quick feedback</p>
                <h3 class="mt-2 text-2xl font-semibold text-slate-900">How was your experience?</h3>
            </div>
        </div>

        <p class="mt-3 text-sm text-slate-600">We’d love to hear how Campus Reserve worked for you.</p>

        <div class="mt-5 flex justify-center gap-2" id="feedback-stars" aria-label="Rate your experience">
            <button type="button" class="feedback-star text-3xl text-slate-300 transition hover:scale-105" data-star="1" aria-label="1 star"> </button>
            <button type="button" class="feedback-star text-3xl text-slate-300 transition hover:scale-105" data-star="2" aria-label="2 stars"> </button>
```

```
            <button type="button" class="feedback-star text-3xl text-slate-300 transition hover:scale-105" data-star="3" aria-label="3 stars"> </button>
            <button type="button" class="feedback-star text-3xl text-slate-300 transition hover:scale-105" data-star="4" aria-label="4 stars"> </button>
            <button type="button" class="feedback-star text-3xl text-slate-300 transition hover:scale-105" data-star="5" aria-label="5 stars"> </button>
        </div>

        <div class="mt-6 flex justify-end gap-3">
            <button type="button" id="feedback-no-thanks" class="rounded-full border border-slate-300 px-4 py-2 text-sm font-medium text-slate-700 hover:bg-slate-50">No
    thanks</button>
            <button type="button" id="feedback-send" class="rounded-full bg-slate-900 px-4 py-2 text-sm font-medium text-white hover:bg-slate-800">Send feedback</button>
        </div>
    </div>
</div>
<script>
    (() => {
        const reservationOpenTime = @json($reservationHours['start_time']);
        const reservationCloseTime = @json($reservationHours['end_time']);
        const propertiesList = document.getElementById('available-properties-list');
        const facilityCalendarModal = document.getElementById('facility-calendar-modal');
        const facilityCalendarFrame = document.getElementById('facility-calendar-frame');
        const facilityCalendarOpenButton = document.getElementById('facility-reserve-now');
        const facilityCalendarCloseButton = document.getElementById('facility-calendar-close');
        const propertyReserveButton = document.getElementById('property-reserve-now');
        const facilitiesSearch = document.getElementById('available-facilities-search');
        const propertiesSearch = document.getElementById('available-properties-search');
        const facilityCards = Array.from(document.querySelectorAll('[data-available-facility-card]'));
        const facilityNoResults = document.getElementById('available-facilities-no-results');
        let propertyItems = [];
        let propertyItemsLoaded = false;
        let propertyItemsRequest = null;

        function openFacilityCalendar() {
            facilityCalendarModal.classList.remove('hidden');
            facilityCalendarModal.classList.add('flex');
            facilityCalendarModal.setAttribute('aria-hidden', 'false');
            document.body.classList.add('overflow-hidden');

            if (!facilityCalendarFrame.hasAttribute('src')) {
                facilityCalendarFrame.src = facilityCalendarFrame.dataset.src;
            } else {
                facilityCalendarFrame.contentWindow?.postMessage('facility-calendar:resize', window.location.origin);
            }
            facilityCalendarCloseButton.focus();
        }

        function closeFacilityCalendar() {
            facilityCalendarModal.classList.add('hidden');
            facilityCalendarModal.classList.remove('flex');
            facilityCalendarModal.setAttribute('aria-hidden', 'true');
            document.body.classList.remove('overflow-hidden');
            facilityCalendarOpenButton.focus();
        }

        facilityCalendarOpenButton.addEventListener('click', openFacilityCalendar);
        facilityCalendarCloseButton.addEventListener('click', closeFacilityCalendar);
        facilityCalendarModal.addEventListener('click', (event) => {
            if (event.target === facilityCalendarModal) {
                closeFacilityCalendar();
            }
        });
        facilityCalendarFrame.addEventListener('load', () => {
            if (!facilityCalendarModal.classList.contains('hidden')) {
                facilityCalendarFrame.contentWindow?.postMessage('facility-calendar:resize', window.location.origin);
            }
        });
        document.addEventListener('keydown', (event) => {
            if (event.key === 'Escape' && !facilityCalendarModal.classList.contains('hidden')) {
                closeFacilityCalendar();
            }
        });

        if (new URLSearchParams(window.location.search).has('open_facility_calendar')) {
            openFacilityCalendar();
        }

```

```
        function filterFacilities() {
            if (!facilitiesSearch) return;
            const query = facilitiesSearch.value.trim().toLowerCase();
            let visibleCount = 0;

            facilityCards.forEach((card) => {
                const matches = (card.dataset.search || '').includes(query);
                card.classList.toggle('hidden', !matches);
                if (matches) visibleCount += 1;
            });

            facilityNoResults?.classList.toggle('hidden', visibleCount > 0 || facilityCards.length === 0);
        }

        function filterPropertyItems() {
            if (!propertiesSearch || !propertyItemsLoaded) return;
            const query = propertiesSearch.value.trim().toLowerCase();
            let visibleCount = 0;

            propertiesList.querySelectorAll('[data-property-item-card]').forEach((card) => {
                const matches = (card.dataset.itemSearch || '').includes(query);
                card.classList.toggle('hidden', !matches);
                if (matches) visibleCount += 1;
            });

            const noResults = propertiesList.querySelector('[data-property-no-results]');
            if (noResults) noResults.classList.toggle('hidden', visibleCount > 0);
        }

        facilitiesSearch?.addEventListener('input', filterFacilities);
        propertiesSearch?.addEventListener('input', filterPropertyItems);

        async function loadPropertyItems() {
            if (propertyItemsRequest) {
                return propertyItemsRequest;
            }

            propertiesList.replaceChildren();
            const loadingMessage = document.createElement('p');
            loadingMessage.className = 'py-8 text-center text-sm text-slate-500';
            loadingMessage.textContent = 'Loading property items...';
            propertiesList.appendChild(loadingMessage);

            propertyItemsRequest = fetch(propertiesList.dataset.itemsUrl, {
                headers: { Accept: 'application/json' },
                credentials: 'same-origin',
            })
                .then((response) => {
                    if (!response.ok) {
                        throw new Error('Property items could not be loaded.');
                    }
                    return response.json();
                })
                .then((items) => {
                    propertyItems = items;
                    propertyItemsLoaded = true;
                    propertyReserveButton.disabled = items.length === 0;
                    propertiesList.replaceChildren();

                    if (!items.length) {
                        const emptyMessage = document.createElement('p');
                        emptyMessage.className = 'col-span-full py-12 text-center text-gray-500';
                        emptyMessage.textContent = 'No property items are available right now.';
                        propertiesList.appendChild(emptyMessage);
                        return;
                    }

                    const noResultsMessage = document.createElement('p');
                    noResultsMessage.dataset.propertyNoResults = 'true';
                    noResultsMessage.className = 'col-span-full hidden py-8 text-center text-sm text-slate-500';
                    noResultsMessage.textContent = 'No property items match your search.';

                    items.forEach((item) => {
                        const card = document.createElement('article');
```

```
                        card.dataset.propertyItemCard = 'true';
                        card.dataset.itemSearch = item.name.toLowerCase();
                        card.className = 'rounded-3xl border border-slate-200 bg-white p-5 shadow-sm';

                        const name = document.createElement('h3');
                        name.className = 'text-lg font-semibold text-gray-900';
                        name.textContent = item.name;

                        const quantity = document.createElement('p');
                        quantity.className = 'mt-2 text-sm text-gray-500';
                        quantity.textContent = `${item.quantity} in stock`;
                        card.append(name, quantity);
                        propertiesList.appendChild(card);
                    });
                    propertiesList.appendChild(noResultsMessage);
                    filterPropertyItems();
                })
                .catch((error) => {
                    propertiesList.replaceChildren();
                    const errorMessage = document.createElement('p');
                    errorMessage.className = 'col-span-full py-8 text-center text-sm text-rose-700';
                    errorMessage.textContent = error.message || 'Property items could not be loaded.';
                    propertiesList.appendChild(errorMessage);

                    const retryButton = document.createElement('button');
                    retryButton.type = 'button';
                    retryButton.className = 'col-span-full mx-auto rounded-xl border border-slate-300 px-4 py-2 text-sm font-semibold text-slate-700 hover:bg-slate-50';
                    retryButton.textContent = 'Try again';
                    retryButton.addEventListener('click', () => {
                        propertyItemsRequest = null;
                        loadPropertyItems();
                    });
                    propertiesList.appendChild(retryButton);
                })
                .finally(() => {
                    propertyItemsRequest = null;
                });

            return propertyItemsRequest;
        }

        loadPropertyItems();

        const feedback = document.getElementById('property-booking-feedback');
        const savedFeedback = sessionStorage.getItem('propertyBookingFeedback');
        if (savedFeedback) {
            feedback.textContent = savedFeedback;
            feedback.classList.remove('hidden');
            sessionStorage.removeItem('propertyBookingFeedback');
        }

        const modal = document.getElementById('property-booking-modal');
        const form = document.getElementById('property-booking-form');
        const itemsContainer = document.getElementById('property-booking-items');
        const itemSelect = document.getElementById('property-booking-item-select');
        const facilityLabel = document.getElementById('property-booking-facility');
        const dateInput = document.getElementById('property-booking-date');
        const startInput = document.getElementById('property-booking-start');
        const endInput = document.getElementById('property-booking-end');
        const submitButton = document.getElementById('property-booking-submit');
        const errorBox = document.getElementById('property-booking-error');
        let availabilityRequest = null;
        let availabilityVersion = 0;
        let modalItems = [];
        let preferredItemId = null;

        function setBookingError(message) {
            errorBox.textContent = message;
            errorBox.classList.toggle('hidden', !message);
        }

        function setModalOpen(open) {
            modal.classList.toggle('hidden', !open);
            modal.classList.toggle('flex', open);
```

```
            document.body.classList.toggle('overflow-hidden', open);
            if (!open && availabilityRequest) {
                availabilityRequest.abort();
                availabilityRequest = null;
            }
        }

        function renderItems(availability = modalItems) {
            const previousQuantity = itemsContainer.querySelector(`[data-quantity-for="${preferredItemId}"]`)?.value;
            itemsContainer.replaceChildren();

            const selectedItem = availability.find((item) => String(item.id) === preferredItemId);
            if (!selectedItem) {
                const emptyMessage = document.createElement('p');
                emptyMessage.className = 'px-2 py-4 text-sm text-slate-500';
                emptyMessage.textContent = 'This property item is no longer available.';
                itemsContainer.appendChild(emptyMessage);
                return;
            }

            const row = document.createElement('div');
            row.className = 'flex flex-wrap items-center justify-between gap-3 rounded-xl bg-slate-50 px-3 py-2.5';

            const nameGroup = document.createElement('div');
            const name = document.createElement('p');
            name.className = 'text-sm font-medium text-slate-800';
            name.textContent = selectedItem.name;

            const available = document.createElement('p');
            available.className = 'text-xs text-slate-500';
            available.dataset.availableFor = String(selectedItem.id);
            available.textContent = selectedItem.maintenance_reason
                ? `Under maintenance: ${selectedItem.maintenance_reason}`
                : `${selectedItem.available} available`;
            nameGroup.append(name, available);

            const quantity = document.createElement('input');
            quantity.type = 'number';
            quantity.min = '1';
            quantity.max = String(selectedItem.available);
            quantity.value = String(previousQuantity
                ? Math.min(Number(previousQuantity), selectedItem.available)
                : 1);
            quantity.disabled = selectedItem.available < 1;
            quantity.setAttribute('aria-label', `Quantity of ${selectedItem.name}`);
            quantity.className = 'w-24 rounded-xl border border-slate-200 bg-white px-3 py-2 text-sm text-slate-700 disabled:bg-slate-100';
            quantity.dataset.quantityFor = String(selectedItem.id);

            row.append(nameGroup, quantity);
            itemsContainer.appendChild(row);
        }

        async function refreshAvailability() {
            availabilityVersion += 1;
            const version = availabilityVersion;
            if (availabilityRequest) {
                availabilityRequest.abort();
            }

            if (!dateInput.value || !startInput.value || !endInput.value || endInput.value <= startInput.value) {
                submitButton.disabled = true;
                setBookingError('Choose a valid date and time range to check item availability.');
                renderItems(modalItems.map((item) => ({ ...item, available: item.quantity })));
                return;
            }

            setBookingError('');
            submitButton.disabled = true;
            const controller = new AbortController();
            availabilityRequest = controller;
            const query = new URLSearchParams({
                date: dateInput.value,
                start_time: startInput.value,
                end_time: endInput.value,
```

```
            });

            try {
                const response = await fetch(`/api/property-items/availability?${query}`, {
                    credentials: 'include',
                    headers: { Accept: 'application/json' },
                    signal: controller.signal,
                });
                const data = await response.json();
                if (!response.ok) {
                    throw new Error(data.message || 'Could not check item availability.');
                }
                if (version !== availabilityVersion) {
                    return;
                }

                renderItems(data.items);
                submitButton.disabled = !data.items.some((item) =>
                    String(item.id) === preferredItemId && item.available > 0
                );
            } catch (error) {
                if (error.name !== 'AbortError' && version === availabilityVersion) {
                    setBookingError(error.message || 'Could not check item availability. Please try again.');
                }
            }
        }

        function openPropertyBooking() {
            if (!propertyItemsLoaded || propertyItems.length === 0) {
                return;
            }

            const firstAvailableItem = propertyItems.find((item) => item.quantity > 0) || propertyItems[0];
            preferredItemId = String(firstAvailableItem.id);
            facilityLabel.textContent = 'Property inventory is independent of rooms and facilities.';
            modalItems = propertyItems;
            form.reset();
            itemsContainer.replaceChildren();
            itemSelect.replaceChildren();
            propertyItems.forEach((item) => {
                const option = document.createElement('option');
                option.value = String(item.id);
                option.textContent = `${item.name} (${item.quantity} in stock)`;
                itemSelect.appendChild(option);
            });
            itemSelect.value = preferredItemId;
            const now = new Date();
            dateInput.value = `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, '0')}-${String(now.getDate()).padStart(2, '0')}`;
            startInput.value = reservationOpenTime;
            const [openHour, openMinute] = reservationOpenTime.split(':').map(Number);
            const defaultEnd = `${String(openHour + 1).padStart(2, '0')}:${String(openMinute).padStart(2, '0')}`;
            endInput.value = defaultEnd > reservationCloseTime ? reservationCloseTime : defaultEnd;
            setBookingError('');
            renderItems(modalItems.map((item) => ({ ...item, available: item.quantity })));
            setModalOpen(true);
            refreshAvailability();
            itemSelect.focus();
        }

        propertyReserveButton.addEventListener('click', openPropertyBooking);

        itemSelect.addEventListener('change', () => {
            preferredItemId = itemSelect.value;
            setBookingError('');
            renderItems(modalItems.map((item) => ({ ...item, available: item.quantity })));
            refreshAvailability();
        });

        [dateInput, startInput, endInput].forEach((input) => {
            input.addEventListener('change', refreshAvailability);
        });

        document.getElementById('property-booking-close').addEventListener('click', () => setModalOpen(false));
        document.getElementById('property-booking-cancel').addEventListener('click', () => setModalOpen(false));
```

```
        modal.addEventListener('click', (event) => {
            if (event.target === modal) {
                setModalOpen(false);
            }
        });
        document.addEventListener('keydown', (event) => {
            if (event.key === 'Escape' && !modal.classList.contains('hidden')) {
                setModalOpen(false);
            }
        });

        form.addEventListener('submit', async (event) => {
            event.preventDefault();
            setBookingError('');

            if (endInput.value <= startInput.value) {
                setBookingError('End time must be later than start time.');
                return;
            }

            const quantityInput = itemsContainer.querySelector(`[data-quantity-for="${preferredItemId}"]`);
            if (!preferredItemId || !quantityInput || quantityInput.disabled) {
                setBookingError('This item is currently unavailable for the selected time.');
                return;
            }

            const quantity = Number(quantityInput.value);
            if (!Number.isInteger(quantity) || quantity < 1 || quantity > Number(quantityInput.max)) {
                setBookingError(`Enter a quantity from 1 to ${quantityInput.max}.`);
                return;
            }
            const selectedItems = [{
                facility_item_id: Number(preferredItemId),
                quantity,
            }];

            submitButton.disabled = true;
            submitButton.textContent = 'Submitting...';

            try {
                const response = await fetch('/api/calendar/reserve', {
                    method: 'POST',
                    credentials: 'include',
                    headers: {
                        Accept: 'application/json',
                        'Content-Type': 'application/json',
                        'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').content,
                    },
                    body: JSON.stringify({
                            purpose: document.getElementById('property-booking-purpose').value,
                        date: dateInput.value,
                        start_time: startInput.value,
                        end_time: endInput.value,
                        items: selectedItems,
                    }),
                });
                const data = await response.json();
                if (!response.ok) {
                    const validationError = data.errors ? Object.values(data.errors).flat()[0] : null;
                    const error = new Error(data.error || validationError || data.message || 'Reservation could not be submitted.');
                    error.code = data.code;
                    throw error;
                }

                sessionStorage.setItem('propertyBookingFeedback', 'Property reservation submitted and is pending approval.');
                const url = new URL(window.location.href);
                url.searchParams.set('tab', 'property');
                window.location.assign(url);
            } catch (error) {
                setBookingError(error.code === 'schedule_occupied'
                    ? 'Sorry!\nThis schedule and room is already occupied!\nPlease try another time or room!'
                    : error.message || 'Reservation could not be submitted. Please try again.');
            } finally {
                submitButton.disabled = false;
```

```
                submitButton.textContent = 'Reserve Item';
            }
        });
    })();

    let deleteReservationFormId = null;
    let deleteReservationMode = null;

    function openDeleteModal(reservationId, facilityName) {
        deleteReservationMode = 'single';
        deleteReservationFormId = reservationId;
        const modal = document.getElementById('delete-modal');
        const message = document.getElementById('delete-modal-message');
        message.textContent = `Are you sure you want to remove the reservation for ${facilityName}? This action cannot be undone.`;
        modal.classList.remove('hidden');
    }

    function openBulkDeleteModal() {
        deleteReservationMode = 'bulk';
        deleteReservationFormId = null;
        const modal = document.getElementById('delete-modal');
        const message = document.getElementById('delete-modal-message');
        message.textContent = 'Are you sure you want to delete the selected reservations? This action cannot be undone.';
        modal.classList.remove('hidden');
    }

    function closeDeleteModal() {
        deleteReservationMode = null;
        deleteReservationFormId = null;
        document.getElementById('delete-modal').classList.add('hidden');
    }

    document.getElementById('delete-modal-confirm')?.addEventListener('click', function () {
        if (deleteReservationMode === 'single' && deleteReservationFormId) {
            const form = document.getElementById(`delete-reservation-${deleteReservationFormId}`);
            if (form) {
                form.submit();
            }
        }

        if (deleteReservationMode === 'bulk') {
            const bulkForm = document.getElementById('delete-selected-form');
            if (bulkForm) {
                bulkForm.submit();
            }
        }
    });

    (function () {
        const storageKey = 'campusreserve_feedback_survey';
        const now = Date.now();
        const lastDismissed = Number(localStorage.getItem(`${storageKey}_dismissed_until`) || 0);
        const wasSubmitted = localStorage.getItem(`${storageKey}_submitted`) === '1';

        if (lastDismissed > now || wasSubmitted) {
            return;
        }

        const hasReservation = @json($reservations->isNotEmpty());
        const randomChance = Math.random() < 0.18;

        if (!hasReservation && !randomChance) {
            return;
        }

        const modal = document.getElementById('feedback-survey-modal');
        if (!modal) {
            return;
        }

        let selectedRating = 0;
        const stars = Array.from(document.querySelectorAll('.feedback-star'));

        stars.forEach((star) => {
```

```
            star.addEventListener('click', function () {
                selectedRating = Number(this.dataset.star);
                stars.forEach((item) => {
                    const itemValue = Number(item.dataset.star);
                    item.classList.toggle('text-yellow-400', itemValue <= selectedRating);
                    item.classList.toggle('text-slate-300', itemValue > selectedRating);
                });
            });
        });

        const dismissSurvey = (isSubmit = false) => {
            const suppressUntil = Date.now() + (1000 * 60 * 60 * 24 * 30);
            localStorage.setItem(`${storageKey}_dismissed_until`, String(suppressUntil));
            if (isSubmit) {
                localStorage.setItem(`${storageKey}_submitted`, '1');
            }
            modal.classList.add('hidden');
        };

        document.getElementById('feedback-no-thanks')?.addEventListener('click', () => dismissSurvey(false));
        modal.addEventListener('click', (event) => {
            if (event.target === modal) {
                dismissSurvey(false);
            }
        });
        document.getElementById('feedback-send')?.addEventListener('click', () => {
            if (!selectedRating) {
                selectedRating = 5;
            }
            dismissSurvey(true);
        });

        setTimeout(() => {
            modal.classList.remove('hidden');
            modal.classList.add('flex');
        }, 1200);
    })();
</script>
@endsection
```

resources/views/security/compromised.blade.php

```
@extends('layouts.app')

@section('title', 'Secure Your Account - Campus Reserve')

@section('content')
<main class="min-h-[calc(100vh-73px)] bg-slate-50 px-4 py-12 md:px-8">
    <section class="mx-auto max-w-2xl rounded-3xl border border-red-200 bg-white p-8 shadow-sm">
        <div class="mb-6 inline-flex rounded-full bg-red-100 px-3 py-1 text-sm font-semibold text-red-800">Account security</div>
        <h1 class="text-3xl font-semibold text-gray-900">Secure your account</h1>
        <p class="mt-3 text-gray-600">If you did not change your Campus Reserve password, take these steps immediately.</p>
        <ol class="mt-8 space-y-4 text-sm text-gray-700">
            <li><strong>1.</strong> Change your password from Account Settings using a new password you do not use elsewhere.</li>
            <li><strong>2.</strong> Confirm that authenticator-based two-factor authentication is enabled.</li>
            <li><strong>3.</strong> Sign out of unfamiliar sessions and review your account activity.</li>
            <li><strong>4.</strong> Contact your campus administrator if you cannot access your account.</li>
        </ol>
        <div class="mt-8 flex flex-wrap gap-3">
            <a href="{{ route('profile.show') }}" class="rounded-md bg-black px-5 py-3 text-sm font-semibold text-white">Open account settings</a>
            <a href="{{ route('home') }}" class="rounded-md border border-gray-300 px-5 py-3 text-sm font-semibold text-gray-700">Return home</a>
        </div>
    </section>
</main>
@endsection
```

resources/views/welcome.blade.php

```
@extends('layouts.app')

@section('title', 'Campus Reserve - Home')

@section('content')
```

```
    @include('components.hero-section')
    @include('components.features-section')
    @include('components.why-choose-section')
    @include('components.cta-section')
@endsection
```

routes

routes/console.php

```php
<?php

use Illuminate\Foundation\Inspiring;
use Illuminate\Support\Facades\Artisan;
use Illuminate\Support\Facades\Schedule;

Artisan::command('inspire', function () {
    $this->comment(Inspiring::quote());
})->purpose('Display an inspiring quote');

Schedule::command('property-returns:notify-due')->everyMinute()->withoutOverlapping();
Schedule::command('reservations:reject-expired')->everyMinute()->withoutOverlapping();
Schedule::command('reservations:complete-expired')->everyMinute()->withoutOverlapping();
Schedule::command('reservations:delete-expired-rejected')->everyMinute()->withoutOverlapping();
Schedule::command('reservations:delete-expired-completed')->everyMinute()->withoutOverlapping();
```

routes/web.php

```php
<?php

// ...existing code...
use App\Models\Facility;
use App\Models\FacilityMaintenanceBlock;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationPendingApproval;
use App\Services\ReservationNotificationDispatcher;
use App\Services\FacilityMaintenanceSchedule;
use App\Services\ReservationTimeWindow;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Notification;

// Keep the former facility reservation URL as an in-page entry point.
Route::middleware('auth')->get('/calendar/manage', function (Request $request, ReservationTimeWindow $reservationTimeWindow) {
    if (!$request->boolean('embedded')) {
        return redirect()->route('reserve', ['open_facility_calendar' => 1]);
    }

    $reservationHours = $reservationTimeWindow->current();
    $calendarFacilities = Facility::active()
        ->orderBy('name')
        ->get(['id', 'name']);

    return view('calendar-manage', compact('reservationHours', 'calendarFacilities'));
})->name('calendar.manage');
use App\Http\Controllers\DebugController;
use App\Http\Controllers\FacilityController;

// API endpoint for facility dropdown
Route::get('/api/facilities', [FacilityController::class, 'list']);

use App\Http\Controllers\AdminController;
use App\Http\Controllers\AuthController;
use App\Http\Controllers\ChatbotController;
use App\Http\Controllers\CredentialController;
use App\Http\Controllers\NotificationController;
use Illuminate\Support\Facades\Route;

/*
|--------------------------------------------------------------------------
| Web Routes
```

```
|--------------------------------------------------------------------------
*/

// Home page
Route::get('/', function () {
    // Public landing page for visitors and signed-out users.
    return view('welcome');
})->name('home');

use App\Http\Controllers\CalendarController;

// Static pages and protected views that require authentication.
Route::middleware('auth')->group(function () {
    Route::post('/api/chatbot', [ChatbotController::class, 'respond'])->name('chatbot.respond');
    Route::post('/api/chatbot/reset', [ChatbotController::class, 'reset'])->name('chatbot.reset');
    Route::get('/api/property-facilities', [FacilityController::class, 'propertyItems'])
        ->name('api.facilities.property-items');
    Route::get('/calendar', function (ReservationTimeWindow $reservationTimeWindow) {
        $reservationHours = $reservationTimeWindow->current();
        $calendarFacilities = Facility::active()
            ->orderBy('name')
            ->get(['id', 'name']);

        return view('calendar', compact('reservationHours', 'calendarFacilities'));
    })->name('calendar');

    Route::get('/api/calendar/events', [CalendarController::class, 'events']);
    Route::get('/api/calendar/pending', [CalendarController::class, 'pending']);
    Route::get('/api/facilities/{facility}/item-availability', [CalendarController::class, 'itemAvailability'])
        ->name('api.facilities.items.availability');
    Route::get('/api/property-items/availability', [CalendarController::class, 'propertyItemAvailability'])
        ->name('api.property-items.availability');
});

Route::middleware('auth')->post('/api/calendar/reserve', [CalendarController::class, 'store']);

Route::get('/about', function () {
    return view('about');
})->name('about');

Route::get('/contact', function () {
    return view('contact');
})->name('contact');

/*
|--------------------------------------------------------------------------
| Authentication Routes (Guest only)
|--------------------------------------------------------------------------
*/

Route::middleware('guest')->group(function () {
    // Guest-only authentication routes are accessible only when the user is not logged in.
    // Login
    Route::get('/login', [AuthController::class, 'showLogin'])->name('login');
    Route::post('/login', [AuthController::class, 'login'])->middleware('throttle:10,1');
    Route::get('/forgot-password', [AuthController::class, 'showForgotPassword'])->name('password.request');
    Route::post('/forgot-password', [AuthController::class, 'sendPasswordResetLink'])->name('password.email')->middleware('throttle:5,1');

    // Register
    Route::get('/signup', [AuthController::class, 'showRegister'])->name('register');
    Route::post('/signup', [AuthController::class, 'register'])->middleware('throttle:5,1');

    // Google OAuth
    Route::get('/auth/google', [AuthController::class, 'redirectToGoogle'])->name('auth.google');
    Route::get('/auth/google/callback', [AuthController::class, 'handleGoogleCallback']);
});

Route::get('/reset-password/{token}', [AuthController::class, 'showResetPassword'])->name('password.reset');
Route::post('/reset-password', [AuthController::class, 'resetPassword'])->name('password.update')->middleware('throttle:5,1');

// Admin secret login
Route::middleware('guest')->group(function () {
    Route::get('/admin-secret', [AuthController::class, 'showAdminLogin'])->name('admin.login');
    Route::post('/admin-secret', [AuthController::class, 'loginAdmin'])->middleware('throttle:5,1');
```

```
});

// 2FA routes (after initial password validation)
Route::middleware('web')->group(function () {
    Route::get('/2fa', [AuthController::class, 'show2faVerify'])->name('2fa.show');
    Route::post('/2fa/verify', [AuthController::class, 'verify2fa'])->name('2fa.verify')->middleware('throttle:5,1');
    Route::post('/2fa/resend', [AuthController::class, 'resend2fa'])->name('2fa.resend')->middleware('throttle:3,1');
    Route::get('/admin-2fa', [AuthController::class, 'showAdmin2fa'])->name('admin.2fa');
});

/*
|--------------------------------------------------------------------------
| Protected Routes (Auth required)
|--------------------------------------------------------------------------
*/

Route::middleware('auth')->group(function () {
    // Authenticated user routes. All of these pages require the user to be signed in.
    // Reserve page (Dashboard) - requires login
    Route::get('/reserve', function (
        Request $request,
        ReservationTimeWindow $reservationTimeWindow,
        \App\Services\ReservationDateWindow $reservationDateWindow
    ) {
        $user = auth()->user();
        $reservationSearch = trim((string) $request->query('reservation_search', ''));
        $reservationsQuery = Reservation::with(['facility', 'items'])
            ->where('user_id', $user->id)
            ->when($reservationSearch !== '', function ($query) use ($reservationSearch) {
                $search = '%'.mb_strtolower($reservationSearch).'%';

                $query->where(function ($query) use ($search) {
                    $query->whereRaw('LOWER(purpose) LIKE ?', [$search])
                        ->orWhereRaw('LOWER(status) LIKE ?', [$search])
                        ->orWhereHas('facility', fn ($facilityQuery) => $facilityQuery->whereRaw('LOWER(name) LIKE ?', [$search]))
                        ->orWhereHas('items', fn ($itemQuery) => $itemQuery->whereRaw('LOWER(name) LIKE ?', [$search]));
                });
            });
        $reservations = $reservationsQuery
            ->orderBy('reservation_date')
            ->orderBy('start_time')
            ->paginate(20)
            ->withQueryString();

        $today = now()->toDateString();
        $reservationStats = Reservation::where('user_id', $user->id)
            ->selectRaw(
                'COUNT(CASE WHEN status IN (?, ?) AND reservation_date >= ? THEN 1 END) AS upcoming_count,
                COUNT(CASE WHEN status = ? OR reservation_date < ? THEN 1 END) AS completed_count',
                ['pending', 'approved', $today, 'completed', $today]
            )
            ->first();

        $itemStats = Reservation::query()
            ->leftJoin('reservation_items', 'reservations.id', '=', 'reservation_items.reservation_id')
            ->where('reservations.user_id', $user->id)
            ->selectRaw(
                'COALESCE(SUM(CASE WHEN reservations.status = ? THEN reservation_items.quantity ELSE 0 END), 0) AS borrowed_item_count,
                COALESCE(SUM(CASE WHEN reservations.status = ? THEN reservation_items.quantity ELSE 0 END), 0) AS returned_item_count',
                ['approved', 'completed']
            )
            ->first();

        $upcomingCount = (int) $reservationStats->upcoming_count;
        $completedCount = (int) $reservationStats->completed_count;
        $borrowedItemCount = (int) $itemStats->borrowed_item_count;
        $returnedItemCount = (int) $itemStats->returned_item_count;

        $facilities = Facility::active()
            ->orderBy('name')
            ->get(['id', 'name', 'building', 'capacity']);
        $reservationHours = $reservationTimeWindow->current();
        $maximumReservationDate = $reservationDateWindow->maximumDate();

```

```
        return view('reserve', compact(
            'reservations',
            'upcomingCount',
            'completedCount',
            'borrowedItemCount',
            'returnedItemCount',
            'facilities',
            'reservationHours',
            'reservationSearch',
            'maximumReservationDate'
        ));
    })->name('reserve');

    Route::get('/reserve/reservations/{reservation}/edit', function (
        Reservation $reservation,
        \App\Services\ReservationDateWindow $reservationDateWindow
    ) {
        $reservation = Reservation::with(['facility', 'items'])
            ->where('user_id', auth()->id())
            ->findOrFail($reservation->id);

        $facilities = $reservation->items->isEmpty()
            ? Facility::active()->orderBy('name')->get()
            : collect();

        $maximumReservationDate = $reservationDateWindow->maximumDate();

        return view('reservations.edit', compact('reservation', 'facilities', 'maximumReservationDate'));
    })->name('reservations.edit');

    Route::put('/reserve/reservations/{reservation}', function (
        Illuminate\Http\Request $request,
        Reservation $reservation,
        ReservationNotificationDispatcher $reservationNotifications,
        ReservationTimeWindow $reservationTimeWindow,
        FacilityMaintenanceSchedule $maintenanceSchedule,
        \App\Services\ReservationDateWindow $reservationDateWindow,
        \App\Services\FacilityItemAvailability $itemAvailability
    ) {
        $reservation = Reservation::where('user_id', auth()->id())->findOrFail($reservation->id);
        $isPropertyReservation = $reservation->items()->exists();

        $request->validate([
            'facility_id' => [$isPropertyReservation ? 'nullable' : 'required', 'exists:facilities,id'],
            'purpose' => 'required|string|max:255',
            'reservation_date' => $reservationDateWindow->rules(),
            'start_time' => 'required|date_format:H:i',
            'end_time' => 'required|date_format:H:i|after:start_time',
            'notes' => 'nullable|string|max:500',
        ]);

        if (! $reservationTimeWindow->allows($request->start_time, $request->end_time)) {
            return back()
                ->withInput()
                ->withErrors(['start_time' => $reservationTimeWindow->validationMessage()]);
        }

        if ($isPropertyReservation) {
            foreach ($reservation->items as $item) {
                $availableQuantity = $itemAvailability->availableQuantity(
                    $item,
                    $request->reservation_date,
                    $request->start_time,
                    $request->end_time,
                    $reservation->id
                );
                if ($item->pivot->quantity > $availableQuantity) {
                    return back()->withInput()->with('error', "{$item->name} has only {$availableQuantity} available for that date and time.");
                }
            }
        } else {
            $facility = Facility::findOrFail($request->facility_id);
            if (! $facility->isAvailable($request->reservation_date, $request->start_time, $request->end_time, $reservation->id)) {
                return back()->withInput()->with('error', 'Selected time slot is unavailable. Please choose another slot.');
```

```
            }

            $maintenanceBlock = $maintenanceSchedule->conflictingBlock(
                $facility->id,
                $request->reservation_date,
                $request->start_time,
                $request->end_time
            );
            if ($maintenanceBlock) {
                return back()->withInput()->with('error', $maintenanceSchedule->validationMessage($maintenanceBlock));
            }
        }

        $reservation->update([
            'facility_id' => $isPropertyReservation ? null : $facility->id,
            'reservation_date' => $request->reservation_date,
            'start_time' => $request->start_time,
            'end_time' => $request->end_time,
            'purpose' => $request->purpose,
            'notes' => $request->notes,
            'status' => 'pending',
        ]);

        $admins = User::where('is_admin', true)
            ->orWhere('is_super_admin', true)
            ->get();
        Notification::send($admins, new ReservationPendingApproval($reservation));
        $reservationNotifications->notifyBorrower($reservation);

        return redirect()->route('reserve')->with('success', 'Reservation updated and resubmitted for approval.');
    })->name('reservations.update');

    Route::delete('/reserve/reservations/{reservation}', function (Reservation $reservation) {
        $reservation = Reservation::where('user_id', auth()->id())->findOrFail($reservation->id);
        $reservation->delete();

        return redirect()->route('reserve')->with('success', 'Reservation removed successfully.');
    })->name('reservations.destroy');

    Route::post('/reserve/reservations/delete-selected', function (Illuminate\Http\Request $request) {
        $request->validate([
            'selected_reservations' => 'array',
            'selected_reservations.*' => 'integer|exists:reservations,id',
        ]);

        $selectedIds = $request->input('selected_reservations', []);
        $reservations = Reservation::whereIn('id', $selectedIds)
            ->where('user_id', auth()->id())
            ->get();

        $deletedCount = 0;
        foreach ($reservations as $reservation) {
            $reservation->delete();
            $deletedCount++;
        }

        return redirect()->route('reserve')->with('success', $deletedCount > 0 ? "{$deletedCount} reservation(s) deleted successfully." : 'No reservations were selected.');
    })->name('reservations.destroySelected');

    // Saved Credentials (Password Manager)
    Route::resource('credentials', CredentialController::class);
    Route::get('/credentials/{credential}/password', [CredentialController::class, 'getPassword'])
        ->name('credentials.password');

    // Logout
    Route::post('/logout', [AuthController::class, 'logout'])->name('logout');

    // Profile settings
    Route::get('/profile', [AuthController::class, 'showProfile'])->name('profile.show');
    Route::get('/profile/picture/{user}', [AuthController::class, 'showProfilePicture'])
        ->whereNumber('user')
        ->name('profile.picture.show');
    Route::post('/profile/name', [AuthController::class, 'updateProfileName'])
        ->name('profile.name.update')
```

```
        ->middleware('throttle:10,1');
    Route::post('/profile/email', [AuthController::class, 'updateEmail'])
        ->name('profile.email.update')
        ->middleware('throttle:5,1');
    Route::post('/profile/picture', [AuthController::class, 'uploadProfilePicture'])->name('profile.picture.upload');
    Route::post('/profile/password/verify', [AuthController::class, 'verifyCurrentPassword'])->name('profile.password.verify');
    Route::post('/profile/password/reset-link', [AuthController::class, 'sendAuthenticatedPasswordResetLink'])
        ->name('profile.password.reset-link')
        ->middleware('throttle:3,1');
    Route::post('/profile/email-notifications', [AuthController::class, 'updateEmailNotifications'])
        ->name('profile.email-notifications.update');
    Route::post('/profile', [AuthController::class, 'updateProfile'])->name('profile.update');
    Route::view('/security/account-compromised', 'security.compromised')->name('security.compromised');
    Route::post('/profile/2fa/setup', [AuthController::class, 'setupAuthenticator'])->name('profile.2fa.setup');
    Route::post('/profile/2fa/confirm', [AuthController::class, 'confirmAuthenticator'])->name('profile.2fa.confirm');

    Route::get('/notifications', [NotificationController::class, 'index'])->name('notifications.index');
    Route::get('/api/notifications/unread-summary', [NotificationController::class, 'unreadSummary'])
        ->name('notifications.unreadSummary');
    Route::post('/notifications/mark-read', [NotificationController::class, 'markAllRead'])->name('notifications.markRead');
    Route::post('/notifications/delete-selected', [NotificationController::class, 'destroySelected'])->name('notifications.destroySelected');
    Route::post('/notifications/property-return/{reservation}', [NotificationController::class, 'respondToPropertyReturn'])
        ->name('notifications.propertyReturn.respond');

    // Debug utility: simple developer decrypt page for Laravel encrypted strings.
    // Requires OTP verification for security.
    Route::get('/debug/decrypt', [DebugController::class, 'showDecryptForm'])->name('debug.decrypt.form');
    Route::post('/debug/decrypt/otp', [DebugController::class, 'issueDecryptOtp'])->name('debug.decrypt.otp');
    Route::post('/debug/decrypt/otp/resend', [DebugController::class, 'resendDecryptOtp'])->name('debug.decrypt.otp.resend');
    Route::post('/debug/decrypt/otp-verify', [DebugController::class, 'verifyDecryptOtp'])->name('debug.decrypt.otp.verify');
    Route::get('/debug/decrypt/user-details', [DebugController::class, 'getUserDetails'])->name('debug.decrypt.user-details');
    Route::post('/debug/decrypt', [DebugController::class, 'decrypt'])->name('debug.decrypt');

    // Admin portal routes (hidden / secret access only)
    Route::prefix('admin')
        ->middleware(\App\Http\Middleware\RequireAdminActionPassword::class)
        ->group(function () {
            Route::get('/dashboard', [AdminController::class, 'dashboard'])->name('admin.dashboard');
            Route::get('/facilities/create', [AdminController::class, 'createFacility'])->name('admin.facilities.create');
            Route::get('/property-management', [AdminController::class, 'propertyManagement'])->name('admin.property-management.index');
            Route::get('/reservation-settings', [AdminController::class, 'reservationSettings'])->name('admin.reservation-settings');
            Route::put('/reservation-settings', [AdminController::class, 'updateReservationSettings'])->name('admin.reservation-settings.update');
            Route::post('/reservation-settings/maintenance', [AdminController::class, 'storeMaintenanceBlock'])->name('admin.reservation-settings.maintenance.store');
            Route::put('/reservation-settings/maintenance/{block}', [AdminController::class,
'updateMaintenanceBlock'])->name('admin.reservation-settings.maintenance.update');
            Route::delete('/reservation-settings/maintenance/{block}', [AdminController::class,
'destroyMaintenanceBlock'])->name('admin.reservation-settings.maintenance.destroy');
            Route::post('/reservation-settings/property-maintenance', [\App\Http\Controllers\PropertyItemMaintenanceController::class,
'store'])->name('admin.reservation-settings.property-maintenance.store');
            Route::put('/reservation-settings/property-maintenance/{block}', [\App\Http\Controllers\PropertyItemMaintenanceController::class,
'update'])->name('admin.reservation-settings.property-maintenance.update');
            Route::delete('/reservation-settings/property-maintenance/{block}', [\App\Http\Controllers\PropertyItemMaintenanceController::class,
'destroy'])->name('admin.reservation-settings.property-maintenance.destroy');
            Route::post('/property-management/items', [AdminController::class, 'storeFacilityItem'])->name('admin.property-management.items.store');
            Route::put('/property-management/items/{item}', [AdminController::class, 'updateFacilityItem'])->name('admin.property-management.items.update');
            Route::delete('/property-management/items/{item}', [AdminController::class, 'destroyFacilityItem'])->name('admin.property-management.items.destroy');
            Route::post('/facilities', [AdminController::class, 'storeFacility'])->name('admin.facilities.store');
            Route::put('/facilities/{facility}', [AdminController::class, 'updateFacility'])->name('admin.facilities.update');
            Route::delete('/facilities/{facility}', [AdminController::class, 'destroyFacility'])->name('admin.facilities.destroy');
            Route::post('/reservations/{reservation}/approve', [AdminController::class, 'approve'])->name('admin.reservations.approve');
            Route::post('/reservations/{reservation}/reject', [AdminController::class, 'reject'])->name('admin.reservations.reject');
            Route::put('/reservations/{reservation}', [AdminController::class, 'updateApprovedReservation'])->name('admin.reservations.update');
            Route::delete('/reservations/{reservation}', [AdminController::class, 'destroyReservation'])->name('admin.reservations.destroy');
            Route::post('/users/{user}/role', [AdminController::class, 'updateUserRole'])->name('admin.users.role');
            Route::delete('/users/{user}', [AdminController::class, 'destroyUser'])->name('admin.users.destroy');
        });
});
```

tests

tests/Feature/AdminActionPasswordTest.php

```php
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class AdminActionPasswordTest extends TestCase
{
    use RefreshDatabase;

    public function test_facility_changes_require_the_admins_current_password(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $facility = [
            'name' => 'Computer Lab',
            'building' => 'Engineering Hall',
            'type' => 'Laboratory',
            'capacity' => 24,
        ];

        $this->actingAs($admin)
            ->postJson(route('admin.facilities.store'), $facility)
            ->assertUnprocessable()
            ->assertJsonPath('errors.current_password.0', 'Enter your current password to confirm this admin action.');

        $this->postJson(route('admin.facilities.store'), $facility + ['current_password' => 'incorrect'])
            ->assertUnprocessable()
            ->assertJsonPath('errors.current_password.0', 'Enter your current password to confirm this admin action.');

        $this->assertDatabaseCount('facilities', 0);

        $this->postJson(route('admin.facilities.store'), $facility + ['current_password' => 'password'])
            ->assertRedirect(route('admin.facilities.create'))
            ->assertSessionHas('status', 'Facility created successfully.');

        $this->assertDatabaseHas('facilities', ['name' => 'Computer Lab']);
    }

    public function test_login_and_signup_pages_do_not_render_password_visibility_controls(): void
    {
        $this->get(route('login'))
            ->assertOk()
            ->assertDontSee('data-password-toggle')
            ->assertDontSee('Show password')
            ->assertDontSee('Hide password');

        $this->get(route('register'))
            ->assertOk()
            ->assertDontSee('data-password-toggle')
            ->assertDontSee('Show password')
            ->assertDontSee('Hide password');
    }

    public function test_admin_management_pages_show_password_confirmation_for_actions(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee('admin-action-password')
            ->assertSee('Confirm admin action');

        $this->get(route('admin.facilities.create'))
            ->assertOk()
            ->assertSee('admin-action-password');

        $this->get(route('admin.property-management.index'))
            ->assertOk()
```

```
            ->assertSee('admin-action-password');
    }
}
```

tests/Feature/AdminDashboardPerformanceTest.php

```php
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\Reservation;
use App\Models\User;
use App\Services\UserNameSearchIndex;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Str;
use Tests\TestCase;

class AdminDashboardPerformanceTest extends TestCase
{
    use RefreshDatabase;

    public function test_dashboard_paginates_pending_reservations_and_keeps_total_count(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);

        for ($index = 1; $index <= 16; $index++) {
            Reservation::create([
                'user_id' => $admin->id,
                'facility_id' => $facility->id,
                'reservation_date' => now()->addDays($index)->toDateString(),
                'start_time' => '10:00',
                'end_time' => '11:00',
                'purpose' => "Pending request {$index}",
                'status' => 'pending',
            ]);
        }

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee('>16</p>', false)
            ->assertSee('16 pending')
            ->assertSee('Pending request 1')
            ->assertDontSee('Pending request 16')
            ->assertSee('pendingPage=2');
    }

    public function test_dashboard_management_areas_have_separate_navigation_sections(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee('data-admin-section="overview"', false)
            ->assertSee('data-admin-section="reservations"', false)
            ->assertSee('data-admin-section="members"', false)
            ->assertSee('data-admin-section="audit"', false)
            ->assertSee('data-admin-section-link="reservations"', false)
            ->assertSee('data-admin-section-link="members"', false)
            ->assertSee('data-admin-section-link="audit"', false)
            ->assertSee('Facilities')
            ->assertSee('Property items');
    }

    public function test_admin_sidebar_uses_the_profile_picture_saved_in_profile_settings(): void
```

```
    {
        $admin = User::factory()->create([
            'is_admin' => true,
            'profile_picture' => 'profile_pictures/admin-avatar.webp',
        ]);

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee('src="'.asset('storage/profile_pictures/admin-avatar.webp').'"', false)
            ->assertSee('alt="'.$admin->name.' profile picture"', false);
    }

    public function test_overview_graph_shows_total_approved_and_rejected_reservations(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);

        foreach (['approved', 'approved', 'rejected'] as $index => $status) {
            Reservation::create([
                'user_id' => $admin->id,
                'facility_id' => $facility->id,
                'reservation_date' => now()->addDays($index + 1)->toDateString(),
                'start_time' => '10:00',
                'end_time' => '11:00',
                'purpose' => "Decision {$index}",
                'status' => $status,
            ]);
        }

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee('Reservation outcomes')
            ->assertSee('data-reservation-outcome="approved" data-total="2"', false)
            ->assertSee('data-reservation-outcome="rejected" data-total="1"', false);
    }

    public function test_ajax_member_pagination_returns_only_the_member_section(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);

        for ($index = 1; $index <= 16; $index++) {
            User::factory()->create([
                'name' => "Member {$index}",
                'email' => Str::lower(Str::random(12)) . '@example.com',
            ]);
        }

        $response = $this->actingAs($admin)
            ->get(route('admin.dashboard', ['role' => 'user', 'page' => 2]), [
                'X-Requested-With' => 'XMLHttpRequest',
                'Accept' => 'text/html',
            ]);

        $response
            ->assertOk()
            ->assertSee('Member Management')
            ->assertSee('member-filter-form')
            ->assertSee('aria-label="Pagination Navigation"', false)
            ->assertSee('Showing')
            ->assertSee('Go to page 1')
            ->assertDontSee('Previous')
            ->assertDontSee('Next')
            ->assertDontSee('Admin Dashboard')
            ->assertDontSee('Pending Reservations')
            ->assertSee('page=1');

```

```
        $this->assertSame('text/html; charset=utf-8', strtolower($response->headers->get('Content-Type')));
    }

    public function test_ajax_member_search_returns_the_matching_email_without_full_dashboard(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        User::factory()->create([
            'name' => 'Search Target',
            'email' => 'target.member@example.com',
        ]);
        User::factory()->create([
            'name' => 'Other Member',
            'email' => 'other.member@example.com',
        ]);

        $this->actingAs($admin)
            ->get(route('admin.dashboard', ['search' => 'target.member@example.com']), [
                'X-Requested-With' => 'XMLHttpRequest',
                'Accept' => 'text/html',
            ])
            ->assertOk()
            ->assertSee('target.member@example.com')
            ->assertDontSee('other.member@example.com')
            ->assertDontSee('Admin Dashboard')
            ->assertDontSee('Pending Reservations');
    }

    public function test_ajax_member_search_matches_encrypted_names_across_all_roles(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $franz = User::factory()->create([
            'name' => 'Franz Example',
            'email' => 'franz.member@example.com',
        ]);
        User::factory()->create([
            'name' => 'Other Member',
            'email' => 'other.member@example.com',
        ]);
        User::factory()->create([
            'name' => 'fra ran anz decoy',
            'email' => 'decoy.member@example.com',
        ]);

        $this->assertDatabaseHas('user_name_search_tokens', ['user_id' => $franz->id]);
        $this->assertTrue(app(UserNameSearchIndex::class)->matchingUserIds('fRaNz')->contains($franz->id));

        $this->actingAs($admin)
            ->get(route('admin.dashboard', ['search' => 'fRaNz', 'role' => 'all']), [
                'X-Requested-With' => 'XMLHttpRequest',
                'Accept' => 'text/html',
            ])
            ->assertOk()
            ->assertSee('Franz Example')
            ->assertSee('franz.member@example.com')
            ->assertDontSee('Other Member')
            ->assertDontSee('fra ran anz decoy')
            ->assertDontSee('Admin Dashboard')
            ->assertDontSee('Pending Reservations');

        $franz->update(['name' => 'Updated Member']);

        $this->assertFalse(app(UserNameSearchIndex::class)->matchingUserIds('fRaNz')->contains($franz->id));
        $this->assertTrue(app(UserNameSearchIndex::class)->matchingUserIds('Updated')->contains($franz->id));
    }

    public function test_member_search_remains_interactive_while_results_load(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee("memberContent.setAttribute('aria-busy', String(loading));", false)
            ->assertSee("loadMembers(membersUrl().toString(), 'replace');", false)
```

```
            ->assertSee('updatedSearch.setSelectionRange(selectionStart, selectionEnd);', false)
            ->assertDontSee("memberContent.classList.toggle('pointer-events-none', loading);", false)
            ->assertDontSee("memberContent.classList.toggle('opacity-60', loading);", false);
    }

    public function test_ajax_member_list_can_be_sorted_by_name_and_joined_date(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        User::factory()->create([
            'name' => 'Zebra Member',
            'email' => 'zebra.member@example.com',
            'created_at' => now()->subDays(2),
        ]);
        User::factory()->create([
            'name' => 'Alpha Member',
            'email' => 'alpha.member@example.com',
            'created_at' => now()->subDay(),
        ]);

        $request = $this->actingAs($admin);
        $ajaxHeaders = [
            'X-Requested-With' => 'XMLHttpRequest',
            'Accept' => 'text/html',
        ];

        $request->get(route('admin.dashboard', ['role' => 'user', 'member_sort' => 'name']), $ajaxHeaders)
            ->assertOk()
            ->assertSeeInOrder(['Alpha Member', 'Zebra Member']);

        $request->get(route('admin.dashboard', ['role' => 'user', 'member_sort' => 'joined_oldest']), $ajaxHeaders)
            ->assertOk()
            ->assertSeeInOrder(['Zebra Member', 'Alpha Member']);

        $request->get(route('admin.dashboard', ['role' => 'user', 'member_sort' => 'joined_newest']), $ajaxHeaders)
            ->assertOk()
            ->assertSeeInOrder(['Alpha Member', 'Zebra Member']);
    }

    public function test_member_rows_display_each_profile_picture_before_the_member_name(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $member = User::factory()->create([
            'name' => 'Member With Picture',
            'profile_picture' => 'profile_pictures/member.webp',
        ]);

        $response = $this->actingAs($admin)
            ->get(route('admin.dashboard', ['role' => 'user']));

        $response
            ->assertOk()
            ->assertSee('src="'.asset('storage/profile_pictures/member.webp').'"', false)
            ->assertSee('alt=""', false)
            ->assertSee('Member With Picture');

        $html = $response->getContent();
        $imagePosition = strpos($html, 'src="'.asset('storage/profile_pictures/member.webp').'"');
        $namePosition = strpos($html, 'Member With Picture');
        $this->assertNotFalse($imagePosition);
        $this->assertNotFalse($namePosition);
        $this->assertLessThan($namePosition, $imagePosition);
    }
}
```

tests/Feature/AdminReservationManagementTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\FacilityItem;
use App\Models\Reservation;
use App\Models\User;
```

```php
use App\Notifications\ReservationStatusChanged;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Facades\Notification;
use Tests\TestCase;

class AdminReservationManagementTest extends TestCase
{
    use RefreshDatabase;

    public function test_dashboard_shows_an_edit_reservations_section_for_approved_bookings(): void
    {
        $admin = $this->createAdmin();
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $reservation = $this->createReservation($user, $facility);

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee('Edit Reservations')
            ->assertSee('#'.$reservation->id)
            ->assertSee('aria-label="Scrollable approved reservations"', false)
            ->assertSee('data-edit-reservation', false)
            ->assertSee('approved-reservation-edit-modal', false)
            ->assertSee('id="approved-reservation-filter-form"', false)
            ->assertSee('action="'.route('admin.dashboard').'#reservations"', false)
            ->assertDontSee('onchange="this.form.submit()"', false)
            ->assertSee('name="reservation_date"', false)
            ->assertSee('>Delete</button>', false)
            ->assertSee('Save changes');
    }

    public function test_approved_reservations_can_be_sorted_by_date_or_room_name(): void
    {
        $admin = $this->createAdmin();
        $user = User::factory()->create();
        $alphaRoom = $this->createFacility();
        $alphaRoom->update(['name' => 'Alpha Room']);
        $middleRoom = Facility::create([
            'name' => 'Middle Room',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
        $zuluRoom = Facility::create([
            'name' => 'Zulu Room',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);

        $this->createReservation($user, $alphaRoom)->update(['reservation_date' => now()->addDays(2)]);
        $this->createReservation($user, $middleRoom)->update(['reservation_date' => now()->addDays(3)]);
        $propertyReservation = $this->createReservation($user, $zuluRoom);
        $propertyReservation->update([
            'facility_id' => null,
            'reservation_date' => now()->addDay(),
        ]);
        $keyboard = FacilityItem::create([
            'facility_id' => null,
            'name' => 'Keyboard',
            'quantity' => 1,
        ]);
        $propertyReservation->items()->attach($keyboard, ['quantity' => 1]);

        $this->actingAs($admin)
            ->get(route('admin.dashboard', ['reservation_sort' => 'recent']))
            ->assertOk()
            ->assertSeeInOrder(['Middle Room', 'Alpha Room', 'Keyboard']);

        $this->get(route('admin.dashboard', ['reservation_sort' => 'oldest']))
            ->assertOk()
```

```
            ->assertSeeInOrder(['Keyboard', 'Alpha Room', 'Middle Room']);

        $this->get(route('admin.dashboard', ['reservation_sort' => 'alphabetical']))
            ->assertOk()
            ->assertSeeInOrder(['Alpha Room', 'Keyboard', 'Middle Room']);
    }

    public function test_admin_can_edit_an_approved_reservation_directly(): void
    {
        $admin = $this->createAdmin();
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $reservation = $this->createReservation($user, $facility);

        $this->actingAs($admin)
            ->put(route('admin.reservations.update', $reservation), [
                'current_password' => 'password',
                'reservation_date' => now()->addDays(2)->toDateString(),
                'start_time' => '13:00',
                'end_time' => '14:30',
                'purpose' => 'Updated presentation',
            ])
            ->assertRedirect(route('admin.dashboard').'#reservations')
            ->assertSessionHas('status', 'Approved reservation updated successfully.');

        $this->assertSame(now()->addDays(2)->toDateString(), $reservation->fresh()->reservation_date->format('Y-m-d'));
        $this->assertSame('13:00', $reservation->fresh()->start_time->format('H:i'));
        $this->assertSame('14:30', $reservation->fresh()->end_time->format('H:i'));
        $this->assertSame('Updated presentation', $reservation->fresh()->purpose);
        $this->assertSame('approved', $reservation->fresh()->status);
        $this->assertDatabaseHas('audit_logs', [
            'reservation_id' => $reservation->id,
            'action' => 'reservation_updated',
        ]);
    }

    public function test_admin_cannot_edit_an_approved_reservation_beyond_the_booking_window(): void
    {
        $admin = $this->createAdmin();
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $reservation = $this->createReservation($user, $facility);

        $this->actingAs($admin)
            ->putJson(route('admin.reservations.update', $reservation), [
                'current_password' => 'password',
                'reservation_date' => now()->addYears(2)->addDay()->toDateString(),
                'start_time' => '13:00',
                'end_time' => '14:00',
                'purpose' => 'Updated presentation',
            ])
            ->assertUnprocessable()
            ->assertJsonValidationErrors('reservation_date');
    }

    public function test_admin_cannot_edit_an_approved_reservation_into_an_occupied_schedule(): void
    {
        $admin = $this->createAdmin();
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $reservation = $this->createReservation($user, $facility);
        $conflictingReservation = $this->createReservation($user, $facility, '14:00', '15:00');

        $this->actingAs($admin)
            ->putJson(route('admin.reservations.update', $reservation), [
                'current_password' => 'password',
                'reservation_date' => $conflictingReservation->reservation_date->format('Y-m-d'),
                'start_time' => '14:30',
                'end_time' => '15:30',
                'purpose' => 'Updated presentation',
            ])
            ->assertUnprocessable()
            ->assertJsonValidationErrors('start_time');

```

```
157         $this->assertSame('10:00', $reservation->fresh()->start_time->format('H:i'));
158     }
159
160     public function test_admin_cannot_reschedule_approved_property_items_beyond_available_stock(): void
161     {
162         $admin = $this->createAdmin();
163         $user = User::factory()->create();
164         $otherUser = User::factory()->create();
165         $facility = $this->createFacility();
166         $item = FacilityItem::create([
167             'facility_id' => null,
168             'name' => 'Projector',
169             'quantity' => 2,
170         ]);
171         $reservation = $this->createReservation($user, $facility);
172         $reservation->update(['facility_id' => null]);
173         $reservation->items()->attach($item, ['quantity' => 2]);
174
175         $otherReservation = $this->createReservation($otherUser, $facility, '13:00', '14:00');
176         $otherReservation->update([
177             'facility_id' => null,
178             'reservation_date' => now()->addDays(2)->toDateString(),
179         ]);
180         $otherReservation->items()->attach($item, ['quantity' => 1]);
181
182         $this->actingAs($admin)
183             ->putJson(route('admin.reservations.update', $reservation), [
184                 'current_password' => 'password',
185                 'reservation_date' => now()->addDays(2)->toDateString(),
186                 'start_time' => '13:00',
187                 'end_time' => '14:00',
188                 'purpose' => 'Updated presentation',
189             ])
190             ->assertUnprocessable()
191             ->assertJsonValidationErrors('reservation_date');
192
193         $this->assertSame(
194             $reservation->reservation_date->format('Y-m-d'),
195             $reservation->fresh()->reservation_date->format('Y-m-d')
196         );
197     }
198
199     public function test_admin_deleting_an_approved_reservation_removes_it_and_notifies_the_owner(): void
200     {
201         Notification::fake();
202
203         $admin = $this->createAdmin();
204         $user = User::factory()->create();
205         $facility = $this->createFacility();
206         $reservation = $this->createReservation($user, $facility);
207
208         $this->actingAs($admin)
209             ->deleteJson(route('admin.reservations.destroy', $reservation), [
210                 'current_password' => 'password',
211             ])
212             ->assertJson([
213                 'status' => 'Reservation deleted successfully and the borrower was notified.',
214             ]);
215
216         $this->assertDatabaseMissing('reservations', ['id' => $reservation->id]);
217         $this->assertDatabaseHas('audit_logs', [
218             'action' => 'deleted',
219             'details' => 'Reservation cancelled and deleted by admin.',
220         ]);
221         Notification::assertSentTo($user, ReservationStatusChanged::class, function (ReservationStatusChanged $notification) use ($reservation) {
222             return $notification->toArray($reservation->user)['status'] === 'cancelled'
223                 && str_contains($notification->toArray($reservation->user)['message'], 'cancelled');
224         });
225     }
226
227     public function test_admin_edit_actions_require_current_password(): void
228     {
229         $admin = $this->createAdmin();
230         $user = User::factory()->create();
```

```
        $reservation = $this->createReservation($user, $this->createFacility());

        $this->actingAs($admin)
            ->putJson(route('admin.reservations.update', $reservation), [
                'reservation_date' => now()->addDays(2)->toDateString(),
                'start_time' => '13:00',
                'end_time' => '14:00',
                'purpose' => 'Updated presentation',
            ])
            ->assertUnprocessable()
            ->assertJsonValidationErrors('current_password');

        $this->deleteJson(route('admin.reservations.destroy', $reservation))
            ->assertUnprocessable()
            ->assertJsonValidationErrors('current_password');

        $this->assertSame('approved', $reservation->fresh()->status);
    }

    private function createAdmin(): User
    {
        return User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);
    }

    private function createFacility(): Facility
    {
        return Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
    }

    private function createReservation(
        User $user,
        Facility $facility,
        string $startTime = '10:00',
        string $endTime = '11:00'
    ): Reservation {
        return Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => now()->addDay()->toDateString(),
            'start_time' => $startTime,
            'end_time' => $endTime,
            'purpose' => 'Presentation',
            'status' => 'approved',
        ]);
    }
}
```

tests/Feature/AdminUserRoleTest.php

```
<?php

namespace Tests\Feature;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class AdminUserRoleTest extends TestCase
{
    use RefreshDatabase;

    public function test_regular_admin_cannot_change_roles_and_does_not_see_user_actions(): void
    {
        $admin = User::factory()->create([
            'is_admin' => true,
```

```
            'is_super_admin' => false,
        ]);
        $member = User::factory()->create([
            'is_admin' => false,
            'is_super_admin' => false,
        ]);

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertDontSee(route('admin.users.role', $member))
            ->assertDontSee(route('admin.users.destroy', $member));

        $this->actingAs($admin)
            ->post(route('admin.users.role', $member), ['role' => 'admin', 'current_password' => 'password'])
            ->assertForbidden();

        $this->assertFalse($member->fresh()->is_admin);
    }

    public function test_head_admin_can_promote_a_member_to_admin(): void
    {
        $headAdmin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => true,
        ]);
        $member = User::factory()->create([
            'is_admin' => false,
            'is_super_admin' => false,
        ]);

        $this->actingAs($headAdmin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee(route('admin.users.role', $member));

        $this->post(route('admin.users.role', $member), ['role' => 'admin', 'current_password' => 'password'])
            ->assertRedirect(route('admin.dashboard'))
            ->assertSessionHas('status', 'User role updated successfully.');

        $this->assertTrue($member->fresh()->is_admin);
    }

    public function test_head_admin_can_change_member_roles_without_a_redirect(): void
    {
        $headAdmin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => true,
        ]);
        $member = User::factory()->create([
            'is_admin' => false,
            'is_super_admin' => false,
        ]);

        $this->actingAs($headAdmin)
            ->postJson(route('admin.users.role', $member), ['role' => 'admin', 'current_password' => 'password'])
            ->assertOk()
            ->assertJsonPath('status', 'User role updated successfully.');

        $this->assertTrue($member->fresh()->is_admin);
    }

    public function test_member_management_uses_staff_and_admin_role_labels(): void
    {
        $headAdmin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => true,
        ]);
        User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);

        $this->actingAs($headAdmin)
```

```
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSeeText('Staff')
            ->assertSeeText('Admins only')
            ->assertSeeText('Staff only')
            ->assertDontSeeText('Head Admin')
            ->assertDontSeeText('Head admins only');
    }

    public function test_member_role_changes_require_the_admins_current_password(): void
    {
        $headAdmin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => true,
        ]);
        $member = User::factory()->create([
            'is_admin' => false,
            'is_super_admin' => false,
        ]);

        $this->actingAs($headAdmin)
            ->postJson(route('admin.users.role', $member), ['role' => 'admin'])
            ->assertUnprocessable()
            ->assertJsonPath('errors.current_password.0', 'Enter your current password to confirm this admin action.');

        $this->assertFalse($member->fresh()->is_admin);
    }
}
```

tests/Feature/CalendarFacilityFilterTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\FacilityMaintenanceBlock;
use App\Models\Reservation;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class CalendarFacilityFilterTest extends TestCase
{
    use RefreshDatabase;

    public function test_calendar_filter_shows_reservations_and_maintenance_for_selected_facility(): void
    {
        $user = User::factory()->create();
        $selectedFacility = $this->createFacility('Room 101');
        $otherFacility = $this->createFacility('Room 202');

        $selectedReservation = $this->createReservation($user, $selectedFacility);
        $this->createReservation($user, $otherFacility);

        $selectedMaintenance = FacilityMaintenanceBlock::create([
            'facility_id' => $selectedFacility->id,
            'maintenance_date' => '2030-05-02',
            'is_all_day' => true,
            'reason' => 'Repairs',
        ]);
        FacilityMaintenanceBlock::create([
            'facility_id' => $otherFacility->id,
            'maintenance_date' => '2030-05-02',
            'is_all_day' => true,
            'reason' => 'Cleaning',
        ]);

        $this->actingAs($user)
            ->get(route('calendar'))
            ->assertOk()
            ->assertSee('calendarFacilityFilter', false)
            ->assertSee("if (info.view.type === 'dayGridMonth')", false)
            ->assertSee('return { domNodes: [weekdayLabel] };', false)
```

```
            ->assertSee('All facilities')
            ->assertSee('Room 101')
            ->assertSee('Room 202');

        $this->get(route('reserve'))
            ->assertOk()
            ->assertSee('facility-calendar-frame', false)
            ->assertSee(route('calendar.manage', ['embedded' => 1]), false);

        $embeddedCalendar = $this->get(route('calendar.manage', ['embedded' => 1]))
            ->assertOk()
            ->assertHeader('X-Frame-Options', 'SAMEORIGIN')
            ->assertSee('calendarFacilityFilter', false)
            ->assertSee('All facilities')
            ->assertSee('Room 101')
            ->assertSee('Room 202')
            ->assertSee("query.set('facility_id', facilityFilter.value)", false)
            ->assertSee("if (info.view.type === 'dayGridMonth')", false)
            ->assertSee('return { domNodes: [weekdayLabel] };', false);
        $this->assertStringContainsString(
            "frame-ancestors 'self'",
            $embeddedCalendar->headers->get('Content-Security-Policy')
        );

        $this->get(route('calendar.manage'))
            ->assertRedirect(route('reserve', ['open_facility_calendar' => 1]));

        $reservePage = $this->get(route('reserve'))->assertOk();
        $this->assertStringContainsString(
            "frame-ancestors 'none'",
            $reservePage->headers->get('Content-Security-Policy')
        );

        $this->getJson('/api/calendar/events?start=2030-05-01&end=2030-05-08&facility_id='.$selectedFacility->id)
            ->assertOk()
            ->assertJsonCount(2)
            ->assertJsonFragment(['id' => $selectedReservation->id])
            ->assertJsonFragment(['id' => 'maintenance-'.$selectedMaintenance->id])
            ->assertJsonMissing(['title' => 'Room 202'])
            ->assertJsonMissing(['title' => 'Unavailable: Room 202']);
    }

    private function createFacility(string $name): Facility
    {
        return Facility::create([
            'name' => $name,
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
    }

    private function createReservation(User $user, Facility $facility): Reservation
    {
        return Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => '2030-05-01',
            'start_time' => '10:00',
            'end_time' => '11:00',
            'purpose' => 'Meeting',
            'status' => 'approved',
        ]);
    }
}
```

tests/Feature/ChatbotPropertyReservationTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
```

```php
use App\Models\FacilityItem;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationPendingApproval;
use App\Notifications\ReservationStatusChanged;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Carbon;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Notification;
use Tests\TestCase;

class ChatbotPropertyReservationTest extends TestCase
{
    use RefreshDatabase;

    protected function setUp(): void
    {
        parent::setUp();
        $this->travelTo(Carbon::parse('2030-04-01'));
    }

    public function test_chatbot_submits_property_reservations_to_the_admin_approval_queue(): void
    {
        Notification::fake();

        $user = User::factory()->create();
        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);
        $item = FacilityItem::create([
            'facility_id' => null,
            'name' => 'Projector',
            'quantity' => 3,
        ]);

        $prompt = $this->actingAs($user)
            ->postJson(route('chatbot.respond'), [
                'message' => 'Reserve 2 projectors on May 1 2030 from 10 AM to 12 PM',
            ]);
        $prompt->assertOk();
        $this->assertStringContainsString('What is the purpose', $prompt->json('reply'));

        $prompt = $this->postJson(route('chatbot.respond'), ['message' => 'Classroom presentation']);
        $prompt->assertOk();
        $this->assertStringContainsString('Should I submit this property reservation request', $prompt->json('reply'));
        $this->assertStringContainsString('use time is 10:00 AM', $prompt->json('reply'));
        $this->assertStringContainsString('return time is 12:00 PM', $prompt->json('reply'));

        $submitted = $this->postJson(route('chatbot.respond'), ['message' => 'yes']);
        $submitted->assertOk();
        $this->assertStringContainsString('submitted as a pending request', $submitted->json('reply'));

        $reservation = Reservation::firstOrFail();
        $this->assertSame('pending', $reservation->status);
        $this->assertNull($reservation->facility_id);
        $this->assertSame('Classroom presentation', $reservation->purpose);
        $this->assertDatabaseHas('reservation_items', [
            'reservation_id' => $reservation->id,
            'facility_item_id' => $item->id,
            'quantity' => 2,
        ]);

        Notification::assertSentTo($admin, ReservationPendingApproval::class, function (ReservationPendingApproval $notification) use ($admin) {
            $message = $notification->toArray($admin)['message'];

            return str_contains($message, 'Use time: 10:00')
                && str_contains($message, 'return time: 12:00')
                && str_contains($message, 'Purpose: Classroom presentation')
                && str_contains($message, 'Projector × 2');
        });
        Notification::assertSentTo($user, ReservationStatusChanged::class, function ($notification, $channels) {
            return $channels === ['database'];
        });
```

```
        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSee('Items:')
            ->assertSee('Projector')
            ->assertSee('× 2')
            ->assertSee('Classroom presentation');

        $this->actingAs($user);
        $status = $this->postJson(route('chatbot.respond'), [
            'message' => 'What is the status of my property reservation?',
        ]);
        $status->assertOk();
        $this->assertStringContainsString('Projector × 2', $status->json('reply'));
        $this->assertStringContainsString('pending', $status->json('reply'));
        $this->assertStringContainsString('use at 10:00 AM, return at 12:00 PM', $status->json('reply'));
        $this->assertStringContainsString('purpose: Classroom presentation', $status->json('reply'));
    }

    public function test_chatbot_rejects_reservation_dates_outside_the_booking_window(): void
    {
        $user = User::factory()->create();

        $this->actingAs($user)
            ->postJson(route('chatbot.respond'), [
                'message' => 'Reserve a projector on May 1 2041 from 10 AM to 12 PM',
            ])
            ->assertOk()
            ->assertJsonPath('reply', 'Reservations can be made up to 2032-04-01. Please choose an earlier date.');

        $this->assertDatabaseCount('reservations', 0);
    }

    public function test_chatbot_asks_for_use_and_return_times_when_property_booking_times_are_missing(): void
    {
        $user = User::factory()->create();
        FacilityItem::create([
            'facility_id' => null,
            'name' => 'Projector',
            'quantity' => 2,
        ]);

        $response = $this->actingAs($user)
            ->postJson(route('chatbot.respond'), [
                'message' => 'I want to reserve a projector tomorrow',
            ]);

        $response->assertOk();
        $this->assertStringContainsString('What is the purpose', $response->json('reply'));

        $response = $this->postJson(route('chatbot.respond'), ['message' => 'Borrowing for a lecture']);
        $response->assertOk();
        $this->assertStringContainsString('What time will you use the items', $response->json('reply'));
        $this->assertStringContainsString('what time will you return them', strtolower($response->json('reply')));
    }

    public function test_chatbot_understands_weekday_dates_and_shared_meridiem_times_with_one_availability_query(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
        Facility::create([
            'name' => 'Study Room 2',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 20,
            'is_active' => true,
        ]);
        Facility::create([
            'name' => 'Conference Room 3',
            'building' => 'North Building',
            'type' => 'Conference',
            'capacity' => 12,
            'is_active' => true,
        ]);
```

```
        $this->travelTo(now()->setDate(2030, 5, 1)->setTime(8, 0));

        $availabilityQueries = 0;
        DB::listen(function ($query) use (&$availabilityQueries): void {
            if (str_contains($query->sql, 'from "reservations"')
                && str_contains($query->sql, '"facility_id" in')) {
                $availabilityQueries++;
            }
        });

        $response = $this->actingAs($user)
            ->postJson(route('chatbot.respond'), [
                'message' => 'Check room availability next Friday from 9 to 11 PM',
            ]);

        $response->assertOk();
        $this->assertStringContainsString('May 3, 2030', $response->json('reply'));
        $this->assertStringContainsString('9:00 PM to 11:00 PM', $response->json('reply'));
        $this->assertStringContainsString($facility->name, $response->json('reply'));
        $this->assertSame(1, $availabilityQueries);
    }

    public function test_chatbot_answers_questions_about_the_next_upcoming_reservation(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $this->travelTo(now()->setDate(2030, 5, 1)->setTime(8, 0));

        Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => '2030-04-30',
            'start_time' => '09:00',
            'end_time' => '10:00',
            'purpose' => 'Past presentation',
            'status' => 'approved',
        ]);
        Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => '2030-05-03',
            'start_time' => '10:00',
            'end_time' => '11:00',
            'purpose' => 'Upcoming presentation',
            'status' => 'approved',
        ]);

        $response = $this->actingAs($user)
            ->postJson(route('chatbot.respond'), ['message' => 'When is my next booking?']);

        $response->assertOk();
        $this->assertStringContainsString('Your next reservation is Lecture Room 1', $response->json('reply'));
        $this->assertStringContainsString('May 3, 2030', $response->json('reply'));
        $this->assertStringContainsString('accepted', $response->json('reply'));
        $this->assertStringNotContainsString('April 30, 2030', $response->json('reply'));
    }

    public function test_chatbot_collects_purpose_before_submitting_a_facility_reservation(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();

        $response = $this->actingAs($user)
            ->postJson(route('chatbot.respond'), [
                'message' => 'Reserve Lecture Room 1 on May 1 2030 from 10 AM to 12 PM',
            ]);

        $response->assertOk();
        $this->assertStringContainsString('What is the purpose', $response->json('reply'));
        $this->assertDatabaseCount('reservations', 0);

        $response = $this->postJson(route('chatbot.respond'), ['message' => 'Student club meeting']);
        $response->assertOk();
        $this->assertStringContainsString('Should I submit this reservation request', $response->json('reply'));
```

```
228
229         $response = $this->postJson(route('chatbot.respond'), ['message' => 'yes']);
230         $response->assertOk();
231         $this->assertDatabaseHas('reservations', [
232             'facility_id' => $facility->id,
233             'purpose' => 'Student club meeting',
234             'status' => 'pending',
235         ]);
236     }
237
238     public function test_chatbot_confirmation_without_a_purpose_collects_it_before_submission(): void
239     {
240         $user = User::factory()->create();
241         $facility = $this->createFacility();
242         $this->actingAs($user);
243
244         $response = $this->withSession([
245             'chatbot_reservation' => [
246                 'facility_id' => $facility->id,
247                 'date' => '2030-05-01',
248                 'start_time' => '10:00',
249                 'end_time' => '12:00',
250                 'reservation_type' => 'facility',
251                 'items' => [],
252                 'reservation_intent' => true,
253                 'awaiting_confirmation' => true,
254             ],
255         ])->postJson(route('chatbot.respond'), ['message' => 'yes']);
256
257         $response->assertOk();
258         $this->assertStringContainsString('What is the purpose', $response->json('reply'));
259         $this->assertDatabaseCount('reservations', 0);
260
261         $response = $this->postJson(route('chatbot.respond'), ['message' => 'Student workshop']);
262         $response->assertOk();
263         $this->assertStringContainsString('submitted as a pending request', $response->json('reply'));
264         $this->assertDatabaseHas('reservations', [
265             'facility_id' => $facility->id,
266             'purpose' => 'Student workshop',
267             'status' => 'pending',
268         ]);
269     }
270
271     public function test_chatbot_reports_when_requested_item_quantity_is_unavailable(): void
272     {
273         $user = User::factory()->create();
274         FacilityItem::create([
275             'facility_id' => null,
276             'name' => 'Projector',
277             'quantity' => 1,
278         ]);
279
280         $response = $this->actingAs($user)
281             ->postJson(route('chatbot.respond'), [
282                 'message' => 'Reserve 2 projectors on May 1 2030 from 10 AM to 12 PM',
283             ]);
284         $response->assertOk();
285         $this->assertStringContainsString('What is the purpose', $response->json('reply'));
286
287         $response = $this->postJson(route('chatbot.respond'), ['message' => 'Class presentation']);
288         $response->assertOk();
289         $this->assertStringContainsString('only 1 available', $response->json('reply'));
290
291         $this->assertDatabaseCount('reservations', 0);
292     }
293
294     private function createFacility(): Facility
295     {
296         return Facility::create([
297             'name' => 'Lecture Room 1',
298             'building' => 'Main Building',
299             'type' => 'Classroom',
300             'capacity' => 30,
301             'is_active' => true,
```

```
        ]);
    }
}
```

tests/Feature/CompleteExpiredReservationsTest.php

```php
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\FacilityItem;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationStatusChanged;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Facades\Artisan;
use Illuminate\Support\Facades\Notification;
use Tests\TestCase;

class CompleteExpiredReservationsTest extends TestCase
{
    use RefreshDatabase;

    public function test_it_completes_all_ended_approved_reservations_and_notifies_the_borrower(): void
    {
        Notification::fake();
        $this->travelTo(now()->setTime(15, 0));

        $user = User::factory()->create();
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
        $pastReservation = $this->createReservation($user, $facility, now()->subDay()->toDateString(), '10:00', '11:00');
        $endedTodayReservation = $this->createReservation($user, $facility, now()->toDateString(), '12:00', '14:00');
        $ongoingReservation = $this->createReservation($user, $facility, now()->toDateString(), '14:00', '16:00');
        $futureReservation = $this->createReservation($user, $facility, now()->addDay()->toDateString(), '10:00', '11:00');
        $propertyReservation = $this->createReservation($user, $facility, now()->subDay()->toDateString(), '12:00', '13:00');
        $item = FacilityItem::create([
            'facility_id' => $facility->id,
            'name' => 'Projector',
            'quantity' => 2,
        ]);
        $propertyReservation->items()->attach($item, ['quantity' => 1]);

        Artisan::call('reservations:complete-expired');

        $this->assertDatabaseHas('reservations', ['id' => $pastReservation->id, 'status' => 'completed']);
        $this->assertDatabaseHas('reservations', ['id' => $endedTodayReservation->id, 'status' => 'completed']);
        $this->assertDatabaseHas('reservations', ['id' => $ongoingReservation->id, 'status' => 'approved']);
        $this->assertDatabaseHas('reservations', ['id' => $futureReservation->id, 'status' => 'approved']);
        $this->assertDatabaseHas('reservations', ['id' => $propertyReservation->id, 'status' => 'completed']);
        Notification::assertSentToTimes($user, ReservationStatusChanged::class, 3);

        Artisan::call('reservations:complete-expired');

        Notification::assertSentToTimes($user, ReservationStatusChanged::class, 3);
    }

    private function createReservation(
        User $user,
        Facility $facility,
        string $date,
        string $startTime,
        string $endTime
    ): Reservation {
        return Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => $date,
```

```
            'start_time' => $startTime,
            'end_time' => $endTime,
            'purpose' => 'Test booking',
            'status' => 'approved',
        ]);
    }
}
```

tests/Feature/DuplicateFacilityTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class DuplicateFacilityTest extends TestCase
{
    use RefreshDatabase;

    public function test_admin_cannot_create_duplicate_facility_name_in_same_building(): void
    {
        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);

        Facility::create([
            'name' => 'Classroom A',
            'building' => 'Academic Building',
            'type' => 'classroom',
            'capacity' => 30,
            'requires_approval' => false,
        ]);

        $this->actingAs($admin)
            ->post(route('admin.facilities.store'), [
                'current_password' => 'password',
                'name' => 'Classroom A',
                'building' => 'Academic Building',
                'type' => 'Classroom',
                'capacity' => 35,
                'amenities' => 'Projector',
            ])
            ->assertSessionHasErrors(['name']);

        $this->assertDatabaseCount('facilities', 1);
    }

    public function test_admin_is_redirected_to_manage_facility_page_with_success_message_after_creation(): void
    {
        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);

        $this->actingAs($admin)
            ->post(route('admin.facilities.store'), [
                'current_password' => 'password',
                'name' => 'Computer Lab',
                'building' => 'Engineering Hall',
                'type' => 'Laboratory',
                'capacity' => 24,
                'amenities' => 'Projector, Whiteboard',
            ])
            ->assertRedirect(route('admin.facilities.create'))
            ->assertSessionHas('status', 'Facility created successfully.');

        $this->assertDatabaseHas('facilities', [
            'name' => 'Computer Lab',
            'building' => 'Engineering Hall',
```

```
        ]);
    }

    public function test_admin_can_delete_a_facility_from_manage_facilities_page(): void
    {
        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);

        $facility = Facility::create([
            'name' => 'Lecture Hall',
            'building' => 'Science Center',
            'type' => 'Auditorium',
            'capacity' => 80,
            'requires_approval' => false,
        ]);

        $this->actingAs($admin)
            ->delete(route('admin.facilities.destroy', $facility), ['current_password' => 'password'])
            ->assertRedirect(route('admin.facilities.create'))
            ->assertSessionHas('status', 'Facility removed successfully.');

        $this->assertDatabaseMissing('facilities', [
            'id' => $facility->id,
        ]);
    }

    public function test_admin_can_update_a_facility_from_manage_facilities_page(): void
    {
        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);

        $facility = Facility::create([
            'name' => 'Old Lab',
            'building' => 'Engineering Hall',
            'type' => 'Laboratory',
            'capacity' => 18,
            'requires_approval' => false,
        ]);

        $this->actingAs($admin)
            ->put(route('admin.facilities.update', $facility), [
                'current_password' => 'password',
                'name' => 'Updated Lab',
                'building' => 'Engineering Hall',
                'type' => 'Laboratory',
                'capacity' => 22,
                'amenities' => 'Projector, Whiteboard',
            ])
            ->assertRedirect(route('admin.facilities.create'))
            ->assertSessionHas('status', 'Facility updated successfully.');

        $this->assertDatabaseHas('facilities', [
            'id' => $facility->id,
            'name' => 'Updated Lab',
            'capacity' => 22,
        ]);
    }
}
```

tests/Feature/ExampleTest.php

```
<?php

namespace Tests\Feature;

// use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class ExampleTest extends TestCase
{
```

```
    /**
     * A basic test example.
     */
    public function test_the_application_returns_a_successful_response(): void
    {
        $response = $this->get('/');

        $response->assertStatus(200)
            ->assertSee('Welcome to')
            ->assertSee('Campus Reserve!')
            ->assertSee('Reserve Now')
            ->assertSee('Explore Calendar')
            ->assertSee('Why Choose Campus Reserve?');
    }
}
```

tests/Feature/FacilityItemManagementTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\FacilityItem;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class FacilityItemManagementTest extends TestCase
{
    use RefreshDatabase;

    public function test_admin_can_view_property_management_and_dashboard_links_to_it(): void
    {
        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
        ]);
        FacilityItem::create([
            'facility_id' => $facility->id,
            'name' => 'Projector',
            'quantity' => 2,
        ]);

        $this->actingAs($admin)
            ->get(route('admin.dashboard'))
            ->assertOk()
            ->assertSeeText('Facilities')
            ->assertSeeText('Property items')
            ->assertSee(route('admin.facilities.create'))
            ->assertSee(route('admin.property-management.index'));

        $this->get(route('admin.property-management.index'))
            ->assertOk()
            ->assertSee('Projector')
            ->assertSee('Property Inventory')
            ->assertDontSee('Room or facility')
            ->assertSee('Admin dashboard navigation')
            ->assertSee('Property items')
            ->assertSee('aria-current="page"', false);

        $this->get(route('admin.facilities.create'))
            ->assertOk()
            ->assertSee('Admin dashboard navigation')
            ->assertSee('Facility Management')
            ->assertSee('aria-current="page"', false);
    }

```

```
    public function test_facility_management_omits_amenities_and_facility_booking_keeps_property_items_separate(): void
    {
        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);
        Facility::create([
            'name' => 'Classroom A',
            'building' => 'Academic Building',
            'type' => 'Classroom',
            'capacity' => 30,
        ]);

        $this->actingAs($admin)
            ->get(route('admin.facilities.create'))
            ->assertOk()
            ->assertDontSee('Amenities');

        $this->get(route('admin.facilities.create'))
            ->assertSee('Classroom A')
            ->assertDontSee('bg-gradient-to-br from-slate-200 to-slate-300');

        $this->get('/calendar/manage')
            ->assertRedirect(route('reserve', ['open_facility_calendar' => 1]));
    }

    public function test_facilities_api_returns_identifiers_for_item_availability_lookup(): void
    {
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);

        $this->getJson('/api/facilities')
            ->assertOk()
            ->assertJsonPath('0.id', $facility->id)
            ->assertJsonPath('0.name', $facility->name);
    }

    public function test_admin_can_add_update_and_remove_facility_items(): void
    {
        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
        ]);

        $this->actingAs($admin)
            ->post(route('admin.property-management.items.store'), [
                'current_password' => 'password',
                'name' => 'Projector',
                'quantity' => 2,
            ])
            ->assertRedirect(route('admin.property-management.index'))
            ->assertSessionHas('status', 'Facility item added successfully.');

        $item = FacilityItem::firstOrFail();
        $this->assertDatabaseHas('facility_items', [
            'id' => $item->id,
            'facility_id' => null,
            'name' => 'Projector',
            'quantity' => 2,
        ]);

        $this->actingAs($admin)
            ->put(route('admin.property-management.items.update', $item), [
```

```
                'current_password' => 'password',
                'name' => 'Interactive Display',
                'quantity' => 1,
            ])
            ->assertRedirect(route('admin.property-management.index'))
            ->assertSessionHas('status', 'Facility item updated successfully.');

        $this->assertDatabaseHas('facility_items', [
            'id' => $item->id,
            'name' => 'Interactive Display',
            'quantity' => 1,
        ]);

        $this->actingAs($admin)
            ->delete(route('admin.property-management.items.destroy', $item), ['current_password' => 'password'])
            ->assertRedirect(route('admin.property-management.index'))
            ->assertSessionHas('status', 'Facility item removed successfully.');

        $this->assertDatabaseMissing('facility_items', ['id' => $item->id]);
    }

    public function test_non_admin_cannot_access_property_management(): void
    {
        $user = User::factory()->create([
            'is_admin' => false,
            'is_super_admin' => false,
        ]);

        $this->actingAs($user)
            ->get(route('admin.property-management.index'))
            ->assertForbidden();
    }
}
```

tests/Feature/FacilityMaintenanceSchedulingTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\FacilityMaintenanceBlock;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationStatusChanged;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Carbon;
use Tests\TestCase;

class FacilityMaintenanceSchedulingTest extends TestCase
{
    use RefreshDatabase;

    protected function setUp(): void
    {
        parent::setUp();
        $this->travelTo(Carbon::parse('2030-04-01'));
    }

    public function test_admin_can_schedule_maintenance_and_overlapping_reservations_are_rejected_and_notified(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $pending = $this->createReservation($user, $facility, 'pending', '09:00', '10:30');
        $approved = $this->createReservation($user, $facility, 'approved', '10:30', '12:00');
        $unaffected = $this->createReservation($user, $facility, 'approved', '13:00', '14:00');

        $this->actingAs($admin)
            ->post(route('admin.reservation-settings.maintenance.store'), [
                'facility_id' => $facility->id,
                'maintenance_date' => '2030-05-01',
                'is_all_day' => '0',
                'start_time' => '10:00',
```

```
                'end_time' => '11:00',
                'reason' => 'Electrical maintenance',
                'current_password' => 'password',
            ])
            ->assertRedirect(route('admin.reservation-settings'))
            ->assertSessionHas('status', 'Maintenance block scheduled. Overlapping reservations were automatically rejected and their owners notified.');

        $this->assertDatabaseHas('facility_maintenance_blocks', [
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-01 00:00:00',
            'start_time' => '10:00',
            'end_time' => '11:00',
            'is_all_day' => 0,
            'reason' => 'Electrical maintenance',
        ]);
        $this->assertSame('rejected', $pending->fresh()->status);
        $this->assertSame('rejected', $approved->fresh()->status);
        $this->assertSame('approved', $unaffected->fresh()->status);
        $this->assertNotNull($approved->fresh()->rejected_at);
        $this->assertDatabaseHas('notifications', [
            'notifiable_id' => $user->id,
            'type' => ReservationStatusChanged::class,
        ]);
        $this->assertDatabaseHas('audit_logs', [
            'reservation_id' => $pending->id,
            'action' => 'rejected',
        ]);
    }

    public function test_all_day_maintenance_rejects_reservations_and_is_visible_on_shared_calendar(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $reservation = $this->createReservation($user, $facility, 'pending', '18:00', '20:00');

        $this->actingAs($admin)
            ->post(route('admin.reservation-settings.maintenance.store'), [
                'facility_id' => $facility->id,
                'maintenance_date' => '2030-05-01',
                'is_all_day' => '1',
                'reason' => 'Room repairs',
                'current_password' => 'password',
            ])
            ->assertRedirect(route('admin.reservation-settings'));

        $this->assertSame('rejected', $reservation->fresh()->status);

        $this->getJson('/api/calendar/events?start=2030-05-01&end=2030-05-02')
            ->assertOk()
            ->assertJsonFragment([
                'id' => 'maintenance-1',
                'title' => 'Unavailable: Lecture Room 1',
                'reason' => 'Room repairs',
                'is_maintenance' => true,
            ]);
    }

    public function test_maintenance_blocks_new_reservations_and_item_availability_but_allows_non_overlapping_time(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $facility = $this->createFacility();

        $this->actingAs($admin)
            ->post(route('admin.reservation-settings.maintenance.store'), [
                'facility_id' => $facility->id,
                'maintenance_date' => '2030-05-01',
                'is_all_day' => '0',
                'start_time' => '10:00',
                'end_time' => '12:00',
                'reason' => 'Equipment inspection',
                'current_password' => 'password',
            ])
```

```
            ->assertRedirect(route('admin.reservation-settings'));

        $this->actingAs($user)
            ->postJson('/api/calendar/reserve', [
                'facility' => (string) $facility->id,
                'purpose' => 'Meeting',
                'date' => '2030-05-01',
                'start_time' => '11:00',
                'end_time' => '13:00',
            ])
            ->assertStatus(409)
            ->assertJsonPath('code', 'facility_maintenance')
            ->assertJsonPath('error', 'This facility is unavailable: Equipment inspection.');

        $this->getJson("/api/facilities/{$facility->id}/item-availability?date=2030-05-01&start_time=11:00&end_time=13:00")
            ->assertStatus(409)
            ->assertJsonPath('message', 'This facility is unavailable: Equipment inspection.');

        $this->postJson('/api/calendar/reserve', [
            'facility' => (string) $facility->id,
            'purpose' => 'Meeting',
            'date' => '2030-05-01',
            'start_time' => '12:00',
            'end_time' => '13:00',
        ])->assertOk()->assertJsonPath('success', true);
    }

    public function test_admin_cannot_approve_reservation_during_maintenance(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $reservation = $this->createReservation($user, $facility, 'pending', '10:00', '11:00');
        FacilityMaintenanceBlock::create([
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-01',
            'start_time' => '10:30',
            'end_time' => '12:00',
            'is_all_day' => false,
            'reason' => 'Emergency repair',
        ]);

        $this->actingAs($admin)
            ->postJson(route('admin.reservations.approve', $reservation), [
                'current_password' => 'password',
            ])
            ->assertStatus(409)
            ->assertJsonPath('message', 'This facility is unavailable: Emergency repair.');

        $this->assertSame('pending', $reservation->fresh()->status);
    }

    public function test_maintenance_requires_a_reason_and_a_valid_time_range_when_not_all_day(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $facility = $this->createFacility();

        $this->actingAs($admin)
            ->from(route('admin.reservation-settings'))
            ->post(route('admin.reservation-settings.maintenance.store'), [
                'facility_id' => $facility->id,
                'maintenance_date' => '2030-05-01',
                'is_all_day' => '0',
                'start_time' => '12:00',
                'end_time' => '11:00',
                'reason' => '',
                'current_password' => 'password',
            ])
            ->assertRedirect(route('admin.reservation-settings'))
            ->assertSessionHasErrors(['end_time', 'reason']);
    }

    public function test_admin_can_remove_a_maintenance_block_with_password_confirmation(): void
    {
```

```
        $admin = User::factory()->create(['is_admin' => true]);
        $facility = $this->createFacility();
        $block = FacilityMaintenanceBlock::create([
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-01',
            'start_time' => null,
            'end_time' => null,
            'is_all_day' => true,
            'reason' => 'Room repairs',
        ]);

        $this->actingAs($admin)
            ->get(route('admin.reservation-settings'))
            ->assertOk()
            ->assertSee('Facility maintenance')
            ->assertSee('Room repairs')
            ->assertSee('password_confirmation_dialog', false)
            ->assertSee('#maintenance-edit-dialog::backdrop', false)
            ->assertSee('backdrop-filter: blur(8px)', false)
            ->assertSee('data-password-confirm', false);

        $this->delete(route('admin.reservation-settings.maintenance.destroy', $block), [
            'current_password' => 'password',
        ])
            ->assertRedirect(route('admin.reservation-settings'))
            ->assertSessionHas('status', 'Maintenance block removed.');

        $this->assertDatabaseMissing('facility_maintenance_blocks', ['id' => $block->id]);
        $this->assertDatabaseHas('audit_logs', ['action' => 'facility_maintenance_removed']);
    }

    public function test_admin_can_edit_maintenance_and_reject_newly_overlapping_reservations(): void
    {
        \Illuminate\Support\Facades\Notification::fake();

        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $reservation = $this->createReservation($user, $facility, 'approved', '10:00', '11:00');
        $block = FacilityMaintenanceBlock::create([
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-03',
            'start_time' => '13:00',
            'end_time' => '14:00',
            'is_all_day' => false,
            'reason' => 'Initial maintenance',
        ]);

        $this->actingAs($admin)
            ->get(route('admin.reservation-settings'))
            ->assertOk()
            ->assertSee('data-edit-maintenance', false)
            ->assertSee('maintenance-edit-dialog', false)
            ->assertSee('Save changes');

        $this->put(route('admin.reservation-settings.maintenance.update', $block), [
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-01',
            'is_all_day' => '0',
            'start_time' => '09:30',
            'end_time' => '10:30',
            'reason' => 'Updated repairs',
            'current_password' => 'password',
        ])
            ->assertRedirect(route('admin.reservation-settings'))
            ->assertSessionHas('status', 'Maintenance block updated. Overlapping reservations were automatically rejected and their owners notified.');

        $this->assertDatabaseHas('facility_maintenance_blocks', [
            'id' => $block->id,
            'maintenance_date' => '2030-05-01 00:00:00',
            'start_time' => '09:30',
            'end_time' => '10:30',
            'reason' => 'Updated repairs',
        ]);
```

```
        $this->assertSame('rejected', $reservation->fresh()->status);
        $this->assertDatabaseHas('audit_logs', [
            'action' => 'facility_maintenance_updated',
            'details' => "Updated maintenance for {$facility->name} on 2030-05-01: Updated repairs.",
        ]);
        \Illuminate\Support\Facades\Notification::assertSentTo($user, ReservationStatusChanged::class);
    }

    public function test_maintenance_edit_requires_password_and_valid_input(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $facility = $this->createFacility();
        $block = FacilityMaintenanceBlock::create([
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-01',
            'start_time' => '10:00',
            'end_time' => '11:00',
            'is_all_day' => false,
            'reason' => 'Room repairs',
        ]);

        $this->actingAs($admin)
            ->putJson(route('admin.reservation-settings.maintenance.update', $block), [
                'facility_id' => $facility->id,
                'maintenance_date' => '2030-05-02',
                'is_all_day' => '0',
                'start_time' => '12:00',
                'end_time' => '11:00',
                'reason' => '',
                'current_password' => 'password',
            ])
            ->assertUnprocessable()
            ->assertJsonValidationErrors(['end_time', 'reason']);

        $this->assertSame('2030-05-01', $block->fresh()->maintenance_date->toDateString());
    }

    public function test_reservation_settings_actions_require_the_correct_password(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $facility = $this->createFacility();
        $block = FacilityMaintenanceBlock::create([
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-01',
            'is_all_day' => true,
            'reason' => 'Room repairs',
        ]);

        $this->actingAs($admin)
            ->put(route('admin.reservation-settings.update'), [
                'start_time' => '08:00',
                'end_time' => '22:00',
                'current_password' => 'incorrect',
            ])
            ->assertSessionHasErrors('current_password');
        $this->assertDatabaseMissing('reservation_settings', ['id' => 1]);

        $this->post(route('admin.reservation-settings.maintenance.store'), [
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-02',
            'is_all_day' => '1',
            'reason' => 'Electrical maintenance',
            'current_password' => 'incorrect',
        ])->assertSessionHasErrors('current_password');
        $this->assertDatabaseMissing('facility_maintenance_blocks', [
            'facility_id' => $facility->id,
            'maintenance_date' => '2030-05-02 00:00:00',
        ]);

        $this->delete(route('admin.reservation-settings.maintenance.destroy', $block), [
            'current_password' => 'incorrect',
        ])->assertSessionHasErrors('current_password');
        $this->assertDatabaseHas('facility_maintenance_blocks', ['id' => $block->id]);
    }
```

```

    private function createFacility(): Facility
    {
        return Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
    }

    private function createReservation(User $user, Facility $facility, string $status, string $start, string $end): Reservation
    {
        return Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => '2030-05-01',
            'start_time' => $start,
            'end_time' => $end,
            'purpose' => 'Meeting',
            'status' => $status,
        ]);
    }
}
```

tests/Feature/GmailApiMailTransportTest.php

```php
<?php

namespace Tests\Feature;

use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Mail;
use Symfony\Component\Mailer\Exception\TransportException;
use Tests\TestCase;

class GmailApiMailTransportTest extends TestCase
{
    public function test_gmail_api_mailer_refreshes_oauth_and_sends_rfc822_message(): void
    {
        config([
            'mail.mailers.gmail_api.client_id' => 'test-client-id',
            'mail.mailers.gmail_api.client_secret' => 'test-client-secret',
            'mail.mailers.gmail_api.refresh_token' => 'test-refresh-token',
            'mail.from.address' => 'sender@gmail.com',
            'mail.from.name' => 'Campus Reserve',
        ]);
        Http::fake([
            'https://oauth2.googleapis.com/token' => Http::response([
                'access_token' => 'test-access-token',
                'expires_in' => 3600,
            ]),
            'https://gmail.googleapis.com/gmail/v1/users/me/messages/send' => Http::response([
                'id' => 'gmail-message-id',
            ]),
        ]);

        Mail::mailer('gmail_api')->raw('Reset your password.', function ($message): void {
            $message->to('user@example.com')->subject('Password reset');
        });

        Http::assertSent(function ($request): bool {
            return $request->url() === 'https://oauth2.googleapis.com/token'
                && $request['client_id'] === 'test-client-id'
                && $request['refresh_token'] === 'test-refresh-token'
                && $request['grant_type'] === 'refresh_token';
        });
        Http::assertSent(function ($request): bool {
            if ($request->url() !== 'https://gmail.googleapis.com/gmail/v1/users/me/messages/send') {
                return false;
            }

            $decodedMessage = base64_decode(strtr($request['raw'], '-_', '+/'));
```

```

            return $request->hasHeader('Authorization', 'Bearer test-access-token')
                && is_string($decodedMessage)
                && str_contains($decodedMessage, 'sender@gmail.com')
                && str_contains($decodedMessage, 'user@example.com')
                && str_contains($decodedMessage, 'Password reset')
                && str_contains($decodedMessage, 'Reset your password.');
        });
    }

    public function test_gmail_api_mailer_reports_oauth_refresh_errors(): void
    {
        config([
            'mail.mailers.gmail_api.client_id' => 'test-client-id',
            'mail.mailers.gmail_api.client_secret' => 'test-client-secret',
            'mail.mailers.gmail_api.refresh_token' => 'invalid-refresh-token',
        ]);
        Http::fake([
            'https://oauth2.googleapis.com/token' => Http::response([
                'error' => 'invalid_grant',
                'error_description' => 'Token has been expired or revoked.',
            ], 400),
        ]);

        $this->expectException(TransportException::class);
        $this->expectExceptionMessage('Token has been expired or revoked.');

        Mail::mailer('gmail_api')->raw('Test email.', function ($message): void {
            $message->to('user@example.com')->subject('Test');
        });
    }

    public function test_gmail_api_mailer_reports_api_rejections(): void
    {
        config([
            'mail.mailers.gmail_api.client_id' => 'test-client-id',
            'mail.mailers.gmail_api.client_secret' => 'test-client-secret',
            'mail.mailers.gmail_api.refresh_token' => 'test-refresh-token',
        ]);
        Http::fake([
            'https://oauth2.googleapis.com/token' => Http::response(['access_token' => 'test-access-token']),
            'https://gmail.googleapis.com/gmail/v1/users/me/messages/send' => Http::response([
                'error' => ['message' => 'Gmail API is disabled'],
            ], 403),
        ]);

        $this->expectException(TransportException::class);
        $this->expectExceptionMessage('HTTP 403: Gmail API is disabled');

        Mail::mailer('gmail_api')->raw('Test email.', function ($message): void {
            $message->to('user@example.com')->subject('Test');
        });
    }

    public function test_gmail_api_mailer_requires_all_oauth_credentials(): void
    {
        config([
            'mail.mailers.gmail_api.client_id' => null,
            'mail.mailers.gmail_api.client_secret' => null,
            'mail.mailers.gmail_api.refresh_token' => null,
        ]);

        $this->expectException(TransportException::class);
        $this->expectExceptionMessage('Gmail API OAuth credentials are not fully configured.');

        Mail::mailer('gmail_api')->raw('Test email.', function ($message): void {
            $message->to('user@example.com')->subject('Test');
        });
    }
}
```

tests/Feature/GoogleSignupTest.php

```
<?php
```

```php

namespace Tests\Feature;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Facades\Auth;
use Tests\TestCase;

class GoogleSignupTest extends TestCase
{
    use RefreshDatabase;

    public function test_google_signup_form_prefills_the_google_name_and_email_over_stale_form_input(): void
    {
        $this->withSession([
            'pending_google_signup' => [
                'name' => 'Google User',
                'email' => 'google.user@example.com',
            ],
            '_old_input' => [
                'name' => 'Previous User',
                'email' => 'previous.user@example.com',
            ],
        ])
            ->get(route('register'))
            ->assertOk()
            ->assertSee('value="Google User"', false)
            ->assertSee('value="google.user@example.com"', false)
            ->assertDontSee('value="previous.user@example.com"', false);
    }

    public function test_new_google_users_can_complete_signup_with_their_own_password(): void
    {
        session()->put('pending_google_signup', [
            'name' => 'Google User',
            'email' => 'google.user@example.com',
        ]);

        $response = $this->post(route('register'), [
            'name' => 'Google User',
            'email' => 'google.user@example.com',
            'password' => 'StrongPass1!',
            'password_confirmation' => 'StrongPass1!',
        ]);

        $response->assertRedirect('/reserve');
        $this->assertTrue(Auth::check());
        $this->assertSame('google.user@example.com', Auth::user()->email);
        $this->assertDatabaseHas('users', [
            'email_hash' => hash('sha256', strtolower('google.user@example.com')),
        ]);
        $this->assertNull(session('pending_google_signup'));
    }

    public function test_user_phone_number_persists_when_updating_profile(): void
    {
        $user = User::factory()->create([
            'name' => 'Profile User',
            'email' => 'profile.user@example.com',
            'phone' => null,
        ]);

        $response = $this->actingAs($user)->post(route('profile.update'), [
            'name' => 'Profile User',
            'phone' => '+639171234567',
            'country_code' => '+63',
        ]);

        $response->assertRedirect();
        $response->assertSessionHas('status', 'Profile updated successfully.');
        $this->assertSame('+639171234567', $user->fresh()->phone);
    }
}
```

tests/Feature/LoginCsrfRecoveryTest.php

```php
<?php

namespace Tests\Feature;

use Tests\TestCase;

class LoginCsrfRecoveryTest extends TestCase
{
    public function test_login_page_explains_expired_csrf_sessions_and_offers_a_refresh(): void
    {
        $this->get(route('login'))
            ->assertOk()
            ->assertSee('Your login session expired or changed. Refresh this page and try again.')
            ->assertSee('Refresh login page')
            ->assertSee('response.status === 419', false);
    }
}
```

tests/Feature/MobileNavigationTest.php

```php
<?php

namespace Tests\Feature;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class MobileNavigationTest extends TestCase
{
    use RefreshDatabase;

    public function test_shared_layout_provides_accessible_mobile_navigation(): void
    {
        $this->get(route('home'))
            ->assertOk()
            ->assertSee('mobile-menu-toggle', false)
            ->assertSee('mobile-navigation', false)
            ->assertSee('mobile-navigation-backdrop', false)
            ->assertSee('fixed bottom-0 left-0', false)
            ->assertSee('backdrop-blur-sm', false)
            ->assertSee('transition-transform duration-300', false)
            ->assertSee('aria-expanded="false"', false)
            ->assertSee('Calendar')
            ->assertSee('Log in');
    }

    public function test_primary_navigation_highlights_the_current_page_on_desktop_and_mobile(): void
    {
        $this->actingAs(User::factory()->create());

        foreach ([
            route('home') => '/',
            route('reserve') => '/reserve',
            route('calendar') => '/calendar',
            route('about') => '/about',
            route('contact') => '/contact',
        ] as $url => $path) {
            $response = $this->get($url)->assertOk();

            $this->assertSame(
                2,
                preg_match_all('/href="' . preg_quote($path, '/') . '"[^>]*aria-current="page"[^>]*>/', $response->getContent()),
                "Expected desktop and mobile navigation to mark {$path} as current."
            );

            $this->assertSame(
                1,
                substr_count($response->getContent(), 'after:bg-green-900'),
                "Expected desktop navigation to underline {$path} as current."
            );

```

```
            $this->assertSame(
                1,
                substr_count($response->getContent(), 'underline decoration-2 underline-offset-4'),
                "Expected desktop and mobile navigation to underline {$path} as current."
            );
        }
    }

    public function test_authenticated_mobile_menu_includes_profile_notifications_and_admin(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);

        $this->actingAs($admin)
            ->get(route('reserve'))
            ->assertOk()
            ->assertSee('Profile settings')
            ->assertSee('Notifications')
            ->assertSee('Staff dashboard')
            ->assertSee('Log out');
    }

    public function test_calendar_pages_use_mobile_friendly_initial_views_and_layouts(): void
    {
        $user = User::factory()->create();

        $this->actingAs($user)
            ->get(route('calendar'))
            ->assertOk()
            ->assertSee("initialView: 'timeGridWeek'", false)
            ->assertDontSee('calendar-mobile-reserve', false)
            ->assertDontSee('open_facility_calendar', false)
            ->assertDontSee('>Today</button>', false)
            ->assertSee('calendar-page-layout', false);

        $this->get(route('calendar.manage', ['embedded' => 1]))
            ->assertOk()
            ->assertSee("initialView: 'timeGridWeek'", false)
        ->assertDontSee('>Today</button>', false)
        ->assertDontSee('Reserve this day')
            ->assertDontSee('Reserve selected day')
            ->assertSee('calendar-manage-layout', false);
    }
}
```

tests/Feature/NotificationUnreadSummaryTest.php

```
<?php

namespace Tests\Feature;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Str;
use Tests\TestCase;

class NotificationUnreadSummaryTest extends TestCase
{
    use RefreshDatabase;

    public function test_unread_summary_returns_count_and_latest_timestamp(): void
    {
        $user = User::factory()->create();
        $earlierAt = now()->subMinutes(2)->toDateTimeString();
        $latestAt = now()->subMinute()->toDateTimeString();
        $readAt = now()->subSeconds(30)->toDateTimeString();

        foreach ([$earlierAt, $latestAt] as $createdAt) {
            $user->notifications()->create([
                'id' => (string) Str::uuid(),
                'type' => 'test',
                'data' => [],
                'created_at' => $createdAt,
                'updated_at' => $createdAt,
            ]);
```

```
        }

        $user->notifications()->create([
            'id' => (string) Str::uuid(),
            'type' => 'test',
            'data' => [],
            'read_at' => $readAt,
            'created_at' => now()->toDateTimeString(),
            'updated_at' => now()->toDateTimeString(),
        ]);

        $this->actingAs($user)
            ->getJson(route('notifications.unreadSummary'))
            ->assertOk()
            ->assertJsonPath('count', 2)
            ->assertJsonPath('latest_at', $latestAt);
    }
}
```

tests/Feature/ProfilePicturePersistenceTest.php

```
<?php

namespace Tests\Feature;

use App\Models\User;
use App\Models\UserProfilePicture;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Http\UploadedFile;
use Tests\TestCase;

class ProfilePicturePersistenceTest extends TestCase
{
    use RefreshDatabase;

    public function test_uploaded_profile_picture_is_stored_in_shared_database_and_available_on_other_devices(): void
    {
        $user = User::factory()->create();
        $anotherDeviceUser = User::factory()->create();
        $image = $this->fakePng('profile.png');
        $expectedContents = file_get_contents($image->getRealPath());

        $upload = $this->actingAs($user)
            ->postJson(route('profile.picture.upload'), [
                'profile_picture' => $image,
            ])
            ->assertOk()
            ->assertJsonPath('status', 'Profile picture uploaded successfully.');

        $picture = UserProfilePicture::where('user_id', $user->id)->firstOrFail();
        $this->assertSame('image/png', $picture->mime_type);
        $this->assertSame(base64_encode($expectedContents), $picture->image_data);
        $this->assertNull($user->fresh()->profile_picture);
        $upload->assertJsonPath('picture_url', $user->fresh()->profilePictureUrl());

        $this->actingAs($anotherDeviceUser)
            ->get(route('profile.picture.show', $user))
            ->assertOk()
            ->assertHeader('Content-Type', 'image/png')
            ->assertHeader('X-Content-Type-Options', 'nosniff')
            ->assertContent(base64_decode($picture->image_data));
    }

    public function test_reupload_replaces_the_database_profile_picture(): void
    {
        $user = User::factory()->create();

        $this->actingAs($user)->postJson(route('profile.picture.upload'), [
            'profile_picture' => $this->fakePng('first.png'),
        ])->assertOk();
        $firstPicture = UserProfilePicture::where('user_id', $user->id)->firstOrFail();

        $this->postJson(route('profile.picture.upload'), [
            'profile_picture' => $this->fakePng('second.png'),
```

```
        ])->assertOk();

        $picture = UserProfilePicture::where('user_id', $user->id)->firstOrFail();
        $this->assertSame($firstPicture->id, $picture->id);
        $this->assertSame('image/png', $picture->mime_type);
        $this->assertSame(1, UserProfilePicture::where('user_id', $user->id)->count());
    }

    private function fakePng(string $name): UploadedFile
    {
        $contents = base64_decode('iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAQAAAC1HAwCAAAAC0lEQVR42mP8/x8AAwMCAO+/lWQAAAAASUVORK5CYII=', true);
        $path = tempnam(sys_get_temp_dir(), 'profile-picture-');
        file_put_contents($path, $contents);

        return new UploadedFile($path, $name, 'image/png', null, true);
    }
}
```

tests/Feature/ProfileSecuritySettingsTest.php

```
<?php

namespace Tests\Feature;

use App\Models\User;
use App\Notifications\PasswordResetLink;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Notifications\SendQueuedNotifications;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Queue;
use OTPHP\TOTP;
use Tests\TestCase;

class ProfileSecuritySettingsTest extends TestCase
{
    use RefreshDatabase;

    public function test_profile_settings_script_initializes_the_password_and_authenticator_controls(): void
    {
        $user = User::factory()->create();

        $this->actingAs($user)
            ->get(route('profile.show'))
            ->assertOk()
            ->assertSee('function syncPhoneFieldsToForm()', false)
            ->assertSee('open-password-modal', false)
            ->assertSee('profile-name-input', false)
            ->assertSee(route('profile.name.update'), false)
            ->assertDontSee('Save settings')
            ->assertDontSee('>Cancel</a>', false)
            ->assertSee('profile-email-edit', false)
            ->assertSee('profile-email-save', false)
            ->assertSee(route('profile.email.update'), false)
            ->assertSee('Forgot password?', false)
            ->assertSee('Resend email', false)
            ->assertSee('resend-password-reset-link', false)
            ->assertSee('generate-authenticator-qr', false)
            ->assertSee('authenticator_otp', false)
            ->assertSee('AbortSignal.timeout(20000)', false)
            ->assertDontSee('name="setup_current_password" required', false)
            ->assertDontSee('name="authenticator_otp" required', false)
            ->assertSee('Finish authenticator setup and verify a code', false)
            ->assertDontSee(route('profile.password.verify'), false)
            ->assertSee("event.preventDefault();\n            const submitButton", false)
            ->assertDontSee('window.location.reload()', false)
            ->assertSee("this.textContent = 'Saving...'", false);
    }

    public function test_profile_settings_can_be_saved_without_a_page_redirect(): void
    {
        $user = User::factory()->create();

        $this->actingAs($user)
```

```
            ->postJson(route('profile.update'), [
                'name' => 'Updated Profile Name',
                'phone' => '+639123456789',
                'country_code' => '+63',
                'email_notifications' => true,
                'two_fa_enabled' => false,
            ])
            ->assertOk()
            ->assertJsonPath('status', 'Profile updated successfully.')
            ->assertJsonPath('user.name', 'Updated Profile Name')
            ->assertJsonPath('user.phone', '+639123456789');

        $this->assertSame('Updated Profile Name', $user->fresh()->name);
        $this->assertSame('+639123456789', $user->fresh()->phone);
    }

    public function test_user_can_change_email_after_confirming_current_password(): void
    {
        $user = User::factory()->create([
            'email' => 'old.address@example.com',
            'password' => 'CurrentPass1!',
        ]);

        $this->actingAs($user)
            ->postJson(route('profile.email.update'), [
                'email' => 'new.address@example.com',
                'current_password' => 'CurrentPass1!',
            ])
            ->assertOk()
            ->assertJsonPath('status', 'Email address updated successfully.')
            ->assertJsonPath('email', 'new.address@example.com');

        $this->assertSame('new.address@example.com', $user->fresh()->email);
        $this->assertSame(
            hash('sha256', strtolower('new.address@example.com')),
            $user->fresh()->email_hash
        );
    }

    public function test_profile_name_can_be_saved_automatically(): void
    {
        $user = User::factory()->create(['name' => 'Original Name']);

        $this->actingAs($user)
            ->postJson(route('profile.name.update'), ['name' => 'Updated Name'])
            ->assertOk()
            ->assertJsonPath('status', 'Name saved automatically.')
            ->assertJsonPath('name', 'Updated Name');

        $this->assertSame('Updated Name', $user->fresh()->name);
    }

    public function test_user_cannot_change_email_with_an_incorrect_password_or_to_an_existing_address(): void
    {
        $user = User::factory()->create([
            'email' => 'owner@example.com',
            'password' => 'CurrentPass1!',
        ]);
        User::factory()->create(['email' => 'taken@example.com']);

        $this->actingAs($user)
            ->postJson(route('profile.email.update'), [
                'email' => 'new.address@example.com',
                'current_password' => 'wrong-password',
            ])
            ->assertUnprocessable()
            ->assertJsonValidationErrors('current_password');

        $this->postJson(route('profile.email.update'), [
            'email' => 'taken@example.com',
            'current_password' => 'CurrentPass1!',
        ])
            ->assertUnprocessable()
            ->assertJsonValidationErrors('email');
```

```
129
130         $this->assertSame('owner@example.com', $user->fresh()->email);
131     }
132
133     public function test_invalid_two_factor_disable_credentials_return_a_json_error(): void
134     {
135         $user = User::factory()->create([
136             'password' => 'CurrentPass1!',
137             'two_fa_enabled' => true,
138             'two_fa_secret' => TOTP::create()->getSecret(),
139         ]);
140
141         $this->actingAs($user)
142             ->postJson(route('profile.update'), [
143                 'name' => $user->name,
144                 'two_fa_enabled' => false,
145                 'disable_current_password' => 'wrong-password',
146                 'disable_2fa_otp' => '123456',
147             ])
148             ->assertUnprocessable()
149             ->assertJsonPath('field', 'disable_current_password')
150             ->assertJsonPath('message', 'The current password is incorrect.');
151
152         $this->assertTrue($user->fresh()->two_fa_enabled);
153     }
154
155     public function test_user_can_change_password_without_an_authenticator_code_when_two_factor_is_off(): void
156     {
157         Queue::fake();
158         $user = User::factory()->create([
159             'password' => 'CurrentPass1!',
160         ]);
161
162         $this->actingAs($user)
163             ->postJson(route('profile.update'), [
164                 'name' => $user->name,
165                 'current_password' => 'CurrentPass1!',
166                 'password' => 'NewPass2!',
167                 'password_confirmation' => 'NewPass2!',
168             ])
169             ->assertOk()
170             ->assertJsonPath('password_changed', true);
171
172         $this->assertTrue(Hash::check('NewPass2!', $user->fresh()->password));
173         Queue::assertPushed(SendQueuedNotifications::class);
174     }
175
176     public function test_user_can_start_authenticator_setup_without_immediately_enabling_two_factor(): void
177     {
178         $user = User::factory()->create([
179             'password' => 'CurrentPass1!',
180         ]);
181
182         $this->actingAs($user)
183             ->postJson(route('profile.2fa.setup'), [
184                 'setup_current_password' => 'CurrentPass1!',
185             ])
186             ->assertOk()
187             ->assertJsonStructure([
188                 'authenticator_setup_uri',
189                 'authenticator_setup_qr',
190                 'authenticator_setup_secret',
191             ]);
192
193         $this->assertFalse($user->fresh()->two_fa_enabled);
194         $this->assertNotEmpty(session('pending_authenticator_secret'));
195     }
196
197     public function test_authenticated_user_can_request_password_reset_link_to_account_email(): void
198     {
199         Queue::fake();
200
201         $user = User::factory()->create([
202             'email' => 'profile.user@example.com',
```

```
        ]);

        $this->actingAs($user)
            ->postJson(route('profile.password.reset-link'))
            ->assertOk()
            ->assertJsonPath('status', 'A password reset email was queued for your account address. Check your inbox and spam folder shortly.');

        $this->assertDatabaseHas('password_reset_tokens', [
            'email' => 'profile.user@example.com',
        ]);

        Queue::assertPushed(SendQueuedNotifications::class, function (SendQueuedNotifications $job): bool {
            return $job->notification instanceof PasswordResetLink
                && str_contains($job->notification->resetUrl, '/reset-password/')
                && str_contains($job->notification->resetUrl, 'email=profile.user%40example.com');
        });
        $notification = Queue::pushed(SendQueuedNotifications::class)[0]->notification;
        parse_str((string) parse_url($notification->resetUrl, PHP_URL_QUERY), $link);
        $link['token'] = basename((string) parse_url($notification->resetUrl, PHP_URL_PATH));
        $tokenHash = DB::table('password_reset_tokens')->where('email', 'profile.user@example.com')->value('token');
        $this->assertNotEmpty($tokenHash);
        $this->assertTrue(Hash::check($link['token'], $tokenHash));

        $this->get($notification->resetUrl)->assertOk();
        $this->post(route('password.update'), [
            'email' => $link['email'],
            'token' => $link['token'],
            'password' => 'ResetPass2!',
            'password_confirmation' => 'ResetPass2!',
        ])->assertRedirect(route('login'));

        $this->assertTrue(Hash::check('ResetPass2!', $user->fresh()->password));
        $this->assertDatabaseMissing('password_reset_tokens', [
            'email' => 'profile.user@example.com',
        ]);
    }

    public function test_authenticator_setup_is_enabled_only_after_valid_code_is_saved(): void
    {
        $user = User::factory()->create([
            'password' => 'CurrentPass1!',
        ]);

        $this->actingAs($user)
            ->postJson(route('profile.2fa.setup'), [
                'setup_current_password' => 'CurrentPass1!',
            ])
            ->assertOk();

        $secret = session('pending_authenticator_secret');
        $this->post(route('profile.2fa.confirm'), [
            'authenticator_otp' => TOTP::createFromSecret($secret)->now(),
        ])
            ->assertRedirect()
            ->assertSessionHas('status', 'Authenticator app 2FA is now enabled.');

        $this->assertTrue($user->fresh()->two_fa_enabled);
        $this->assertSame($secret, $user->fresh()->two_fa_secret);
        $this->assertNull(session('pending_authenticator_secret'));
    }

    public function test_authenticator_confirmation_can_complete_without_a_redirect(): void
    {
        $user = User::factory()->create([
            'password' => 'CurrentPass1!',
        ]);
        $secret = TOTP::create()->getSecret();

        $this->actingAs($user)
            ->withSession(['pending_authenticator_secret' => $secret])
            ->postJson(route('profile.2fa.confirm'), [
                'authenticator_otp' => TOTP::createFromSecret($secret)->now(),
            ])
            ->assertOk()
```

```
            ->assertJsonPath('status', 'Authenticator app 2FA is now enabled.');

        $this->assertTrue($user->fresh()->two_fa_enabled);
    }
}
```

tests/Feature/PropertyItemMaintenanceSchedulingTest.php

```php
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\FacilityItem;
use App\Models\PropertyItemMaintenanceBlock;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationStatusChanged;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Carbon;
use Illuminate\Support\Facades\Notification;
use Tests\TestCase;

class PropertyItemMaintenanceSchedulingTest extends TestCase
{
    use RefreshDatabase;

    protected function setUp(): void
    {
        parent::setUp();
        $this->travelTo(Carbon::parse('2030-04-01 10:30'));
    }

    public function test_scheduling_item_maintenance_rejects_and_notifies_overlapping_reservations(): void
    {
        Notification::fake();

        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $item = $this->createItem();
        $pending = $this->createItemReservation($user, $item, 'pending', '09:00', '10:30');
        $approved = $this->createItemReservation($user, $item, 'approved', '10:30', '12:00');
        $unaffected = $this->createItemReservation($user, $item, 'approved', '12:00', '13:00');

        $this->actingAs($admin)
            ->post(route('admin.reservation-settings.property-maintenance.store'), [
                'facility_item_id' => $item->id,
                'maintenance_date' => '2030-05-01',
                'is_all_day' => '0',
                'start_time' => '10:00',
                'end_time' => '11:00',
                'reason' => 'Repair',
                'current_password' => 'password',
            ])
            ->assertRedirect(route('admin.reservation-settings'))
            ->assertSessionHas('status', 'Property item maintenance scheduled. Overlapping reservations were automatically rejected and their owners notified.');

        $this->assertDatabaseHas('property_item_maintenance_blocks', [
            'facility_item_id' => $item->id,
            'maintenance_date' => '2030-05-01 00:00:00',
            'start_time' => '10:00',
            'end_time' => '11:00',
            'reason' => 'Repair',
        ]);
        $this->assertSame('rejected', $pending->fresh()->status);
        $this->assertSame('rejected', $approved->fresh()->status);
        $this->assertSame('approved', $unaffected->fresh()->status);
        $this->assertNotNull($approved->fresh()->rejected_at);
        $this->assertDatabaseHas('audit_logs', [
            'reservation_id' => $approved->id,
            'action' => 'rejected',
        ]);
        $this->assertDatabaseHas('audit_logs', [
            'action' => 'property_item_maintenance_created',
```

```
        ]);
        Notification::assertSentTo($user, ReservationStatusChanged::class);
    }

    public function test_property_booking_and_availability_respect_maintenance_overlap(): void
    {
        $user = User::factory()->create();
        $item = $this->createItem();
        PropertyItemMaintenanceBlock::create([
            'facility_item_id' => $item->id,
            'maintenance_date' => '2030-05-01',
            'start_time' => '10:00',
            'end_time' => '12:00',
            'is_all_day' => false,
            'reason' => 'Inspection',
        ]);

        $this->actingAs($user)
            ->getJson(route('api.property-items.availability', [
                'date' => '2030-05-01',
                'start_time' => '11:00',
                'end_time' => '13:00',
            ]))
            ->assertOk()
            ->assertJsonPath('items.0.available', 0)
            ->assertJsonPath('items.0.maintenance_reason', 'Inspection');

        $this->postJson('/api/calendar/reserve', [
            'purpose' => 'Presentation',
            'date' => '2030-05-01',
            'start_time' => '11:00',
            'end_time' => '13:00',
            'items' => [[
                'facility_item_id' => $item->id,
                'quantity' => 1,
            ]],
        ])
            ->assertStatus(409)
            ->assertJsonPath('code', 'property_item_maintenance')
            ->assertJsonPath('error', 'This property item is unavailable: Inspection.');

        $this->postJson('/api/calendar/reserve', [
            'purpose' => 'Presentation',
            'date' => '2030-05-01',
            'start_time' => '12:00',
            'end_time' => '13:00',
            'items' => [[
                'facility_item_id' => $item->id,
                'quantity' => 1,
            ]],
        ])->assertOk()->assertJsonPath('success', true);
    }

    public function test_available_property_items_hide_only_during_an_active_maintenance_window(): void
    {
        $user = User::factory()->create();
        $activeItem = $this->createItem('Active item');
        $futureItem = $this->createItem('Future item');

        PropertyItemMaintenanceBlock::create([
            'facility_item_id' => $activeItem->id,
            'maintenance_date' => '2030-04-01',
            'start_time' => '10:00',
            'end_time' => '11:00',
            'is_all_day' => false,
            'reason' => 'Currently being repaired',
        ]);
        PropertyItemMaintenanceBlock::create([
            'facility_item_id' => $futureItem->id,
            'maintenance_date' => '2030-04-02',
            'start_time' => '10:00',
            'end_time' => '11:00',
            'is_all_day' => false,
            'reason' => 'Scheduled repair',
```

```
        ]);

        $this->actingAs($user)
            ->getJson(route('api.facilities.property-items'))
            ->assertOk()
            ->assertJsonMissing(['id' => $activeItem->id])
            ->assertJsonFragment(['id' => $futureItem->id, 'name' => 'Future item', 'quantity' => 2]);

        $this->travelTo(Carbon::parse('2030-04-01 11:00'));
        $this->getJson(route('api.facilities.property-items'))
            ->assertOk()
            ->assertJsonFragment(['id' => $activeItem->id, 'name' => 'Active item', 'quantity' => 2]);
    }

    public function test_admin_cannot_approve_a_property_reservation_during_maintenance(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $item = $this->createItem();
        $reservation = $this->createItemReservation($user, $item, 'pending', '10:00', '11:00');
        PropertyItemMaintenanceBlock::create([
            'facility_item_id' => $item->id,
            'maintenance_date' => '2030-05-01',
            'start_time' => '10:30',
            'end_time' => '12:00',
            'is_all_day' => false,
            'reason' => 'Emergency repair',
        ]);

        $this->actingAs($admin)
            ->postJson(route('admin.reservations.approve', $reservation), [
                'current_password' => 'password',
            ])
            ->assertStatus(409)
            ->assertJsonPath('message', 'Cannot approve this reservation because Projector is under maintenance: Emergency repair.');

        $this->assertSame('pending', $reservation->fresh()->status);
    }

    public function test_chatbot_cannot_submit_a_property_reservation_during_maintenance(): void
    {
        $user = User::factory()->create();
        $item = $this->createItem();
        PropertyItemMaintenanceBlock::create([
            'facility_item_id' => $item->id,
            'maintenance_date' => '2030-05-01',
            'start_time' => '10:00',
            'end_time' => '12:00',
            'is_all_day' => false,
            'reason' => 'Inspection',
        ]);

        $this->actingAs($user)
            ->postJson(route('chatbot.respond'), [
                'message' => 'Reserve 1 projector on May 1 2030 from 10 AM to 12 PM',
            ])
            ->assertOk()
            ->assertJsonPath('reply', 'What is the purpose of your property reservation?');

        $this->postJson(route('chatbot.respond'), ['message' => 'Class presentation'])
            ->assertOk()
            ->assertJsonPath('reply', 'Projector has only 0 available on May 1, 2030 from 10:00 AM to 12:00 PM. Tell me a smaller quantity or another date and time.');

        $this->assertDatabaseCount('reservations', 0);
    }

    public function test_admin_can_edit_and_remove_property_item_maintenance_with_password_confirmation(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $item = $this->createItem();
        $reservation = $this->createItemReservation($user, $item, 'pending', '09:00', '10:00', '2030-05-02');
        $block = PropertyItemMaintenanceBlock::create([
            'facility_item_id' => $item->id,
```

```
            'maintenance_date' => '2030-05-01',
            'start_time' => '10:00',
            'end_time' => '11:00',
            'is_all_day' => false,
            'reason' => 'Old reason',
        ]);

        $this->actingAs($admin)
            ->get(route('admin.reservation-settings'))
            ->assertOk()
            ->assertSee('Property item maintenance')
            ->assertSee('data-edit-property-maintenance', false);

        $this->put(route('admin.reservation-settings.property-maintenance.update', $block), [
            'facility_item_id' => $item->id,
            'maintenance_date' => '2030-05-02',
            'is_all_day' => '1',
            'reason' => 'Updated reason',
            'current_password' => 'password',
        ])
            ->assertRedirect(route('admin.reservation-settings'))
            ->assertSessionHas('status', 'Property item maintenance updated. Overlapping reservations were automatically rejected and their owners notified.');

        $this->assertSame('rejected', $reservation->fresh()->status);
        $this->assertDatabaseHas('property_item_maintenance_blocks', [
            'id' => $block->id,
            'maintenance_date' => '2030-05-02 00:00:00',
            'is_all_day' => 1,
            'reason' => 'Updated reason',
        ]);

        $this->delete(route('admin.reservation-settings.property-maintenance.destroy', $block), [
            'current_password' => 'password',
        ])
            ->assertRedirect(route('admin.reservation-settings'))
            ->assertSessionHas('status', 'Property item maintenance removed.');

        $this->assertDatabaseMissing('property_item_maintenance_blocks', ['id' => $block->id]);
        $this->assertDatabaseHas('audit_logs', ['action' => 'property_item_maintenance_updated']);
        $this->assertDatabaseHas('audit_logs', ['action' => 'property_item_maintenance_removed']);
    }

    private function createItem(string $name = 'Projector'): FacilityItem
    {
        return FacilityItem::create([
            'facility_id' => null,
            'name' => $name,
            'quantity' => 2,
        ]);
    }

    private function createItemReservation(
        User $user,
        FacilityItem $item,
        string $status,
        string $start,
        string $end,
        string $date = '2030-05-01'
    ): Reservation {
        $reservation = Reservation::create([
            'user_id' => $user->id,
            'facility_id' => null,
            'reservation_date' => $date,
            'start_time' => $start,
            'end_time' => $end,
            'purpose' => 'Presentation',
            'status' => $status,
        ]);
        $reservation->items()->attach($item, ['quantity' => 1]);

        return $reservation;
    }
}
```

tests/Feature/PropertyReservationTest.php

```php
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\FacilityItem;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationStatusChanged;
use App\Services\ReservationDateWindow;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Carbon;
use Illuminate\Support\Facades\DB;
use Tests\TestCase;

class PropertyReservationTest extends TestCase
{
    use RefreshDatabase;

    protected function setUp(): void
    {
        parent::setUp();
        $this->travelTo(Carbon::parse('2030-04-01'));
    }

    public function test_reserve_page_shows_both_reservation_sections_without_eager_loading_property_items(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
        FacilityItem::create([
            'facility_id' => $facility->id,
            'name' => 'Projector',
            'quantity' => 3,
        ]);

        $this->actingAs($user)
            ->get(route('reserve'))
            ->assertOk()
            ->assertSee('Welcome,')
            ->assertDontSee('Welcome back,')
            ->assertSee('Available Facilities')
            ->assertSee('Main Building')
            ->assertSee('Available Property Items')
            ->assertSee('Reserve Now')
            ->assertSee('facility-reserve-now', false)
            ->assertSee('facility-calendar-modal', false)
            ->assertSee('role="dialog"', false)
            ->assertSee('property-reserve-now', false)
            ->assertSee('property-booking-item-select', false)
            ->assertSee('lg:grid-cols-2', false)
            ->assertDontSee('data-reservation-tab', false)
            ->assertDontSee('Projector')
            ->assertDontSee('3 in stock')
            ->assertDontSee('data-property-item-id', false)
            ->assertSee('available-facilities-list', false)
            ->assertSee('available-properties-list', false);
    }

    public function test_property_items_are_loaded_from_the_independent_inventory_endpoint(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
        FacilityItem::create([
            'facility_id' => $facility->id,
            'name' => 'Projector',
            'quantity' => 3,
        ]);

        $this->actingAs($user)
            ->get(route('reserve'))
            ->assertOk()
            ->assertDontSee('Projector');
```

```
73
74          $this->getJson(route('api.facilities.property-items'))
75              ->assertOk()
76              ->assertJsonPath('0.id', FacilityItem::first()->id)
77              ->assertJsonPath('0.name', 'Projector')
78              ->assertJsonPath('0.quantity', 3);
79      }
80
81      public function test_property_reservation_dialog_allows_item_selection_from_the_header_action(): void
82      {
83          $user = User::factory()->create();
84
85          $this->actingAs($user)
86              ->get(route('reserve'))
87              ->assertOk()
88              ->assertSee('Selected item and quantity')
89              ->assertSee('Reserve Item')
90              ->assertSee('property-booking-item-select', false)
91              ->assertDontSee('dataset.propertyItemId', false)
92              ->assertDontSee("checkbox.type = 'checkbox';", false);
93      }
94
95      public function test_reserve_page_shows_property_item_quantity_statistics(): void
96      {
97          $user = User::factory()->create();
98          $facility = $this->createFacility();
99          $item = FacilityItem::create([
100             'facility_id' => $facility->id,
101             'name' => 'Projector',
102             'quantity' => 5,
103         ]);
104
105         $borrowed = $this->createReservation($user, $facility, 'approved', '10:00', '12:00');
106         $borrowed->items()->attach($item, ['quantity' => 2]);
107         $returned = $this->createReservation($user, $facility, 'completed', '12:00', '13:00');
108         $returned->items()->attach($item, ['quantity' => 3]);
109         $pending = $this->createReservation($user, $facility, 'pending', '13:00', '14:00');
110         $pending->items()->attach($item, ['quantity' => 1]);
111
112         $this->actingAs($user)
113             ->get(route('reserve'))
114             ->assertOk()
115             ->assertSee('Borrowed Items')
116             ->assertSee('Returned Items')
117             ->assertSeeInOrder(['Borrowed Items', '2</p>', 'Returned Items', '3</p>']);
118     }
119
120     public function test_reserve_page_paginates_reservations_and_keeps_total_statistics(): void
121     {
122         $user = User::factory()->create();
123         $facility = $this->createFacility();
124
125         for ($index = 1; $index <= 21; $index++) {
126             $reservation = $this->createReservation($user, $facility, 'pending', '10:00', '11:00');
127             $reservation->update([
128                 'reservation_date' => now()->addDays($index)->toDateString(),
129                 'purpose' => "Booking {$index}",
130             ]);
131         }
132
133         $this->actingAs($user)
134             ->get(route('reserve'))
135             ->assertOk()
136             ->assertSee('>21</p>', false)
137             ->assertSee('Next')
138             ->assertDontSee('Booking 21');
139     }
140
141     public function test_reservation_search_matches_item_names_without_exposing_other_users_reservations(): void
142     {
143         $user = User::factory()->create();
144         $otherUser = User::factory()->create();
145         $facility = $this->createFacility();
146         $item = FacilityItem::create([
```

```
            'facility_id' => $facility->id,
            'name' => 'Projector',
            'quantity' => 3,
        ]);

        $ownReservation = $this->createReservation($user, $facility, 'pending', '10:00', '11:00');
        $ownReservation->update(['purpose' => 'Student presentation']);
        $ownReservation->items()->attach($item, ['quantity' => 1]);

        $otherReservation = $this->createReservation($otherUser, $facility, 'pending', '11:00', '12:00');
        $otherReservation->update(['purpose' => 'Another user private booking']);
        $otherReservation->items()->attach($item, ['quantity' => 1]);

        $this->actingAs($user)
            ->get(route('reserve', ['reservation_search' => 'projector']))
            ->assertOk()
            ->assertSee('Student presentation')
            ->assertDontSee('Another user private booking');

        $this->get(route('reserve', ['reservation_search' => 'Lecture Room 1']))
            ->assertOk()
            ->assertSee('Student presentation')
            ->assertDontSee('Another user private booking');
    }

    public function test_calendar_shows_all_approved_reservations_but_only_exposes_owner_purpose(): void
    {
        $user = User::factory()->create();
        $otherUser = User::factory()->create();
        $facility = $this->createFacility();
        $ownReservation = $this->createReservation($user, $facility, 'approved', '10:00', '11:00');
        $otherReservation = $this->createReservation($otherUser, $facility, 'approved', '11:00', '12:00');
        $hiddenPendingReservation = $this->createReservation($otherUser, $facility, 'pending', '12:00', '13:00');
        $outsideRangeReservation = $this->createReservation($otherUser, $facility, 'approved', '13:00', '14:00');

        $ownReservation->update(['purpose' => 'My meeting']);
        $otherReservation->update(['purpose' => 'Private meeting']);
        $outsideRangeReservation->update(['reservation_date' => '2030-05-02']);

        $this->actingAs($user)
            ->getJson('/api/calendar/events?start=2030-05-01&end=2030-05-02')
            ->assertOk()
            ->assertJsonCount(2)
            ->assertJsonFragment([
                'id' => $ownReservation->id,
                'purpose' => 'My meeting',
                'is_owner' => true,
            ])
            ->assertJsonFragment([
                'id' => $otherReservation->id,
                'title' => $facility->name,
                'start' => '2030-05-01T11:00:00',
                'end' => '2030-05-01T12:00:00',
                'purpose' => null,
                'is_owner' => false,
            ])
            ->assertJsonMissing(['id' => $hiddenPendingReservation->id])
            ->assertJsonMissing(['id' => $outsideRangeReservation->id]);
    }

    public function test_calendar_events_require_a_date_range(): void
    {
        $user = User::factory()->create();

        $this->actingAs($user)
            ->getJson('/api/calendar/events')
            ->assertUnprocessable()
            ->assertJsonValidationErrors(['start', 'end']);
    }

    public function test_reserving_an_occupied_room_returns_the_friendly_conflict_response(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
```

```
        $this->createReservation($user, $facility, 'approved', '10:00', '12:00');
        $this->assertFalse($facility->fresh()->isAvailable('2030-05-01', '11:00', '13:00'));

        $this->actingAs($user)
            ->postJson('/api/calendar/reserve', [
                'facility' => (string) $facility->id,
                'purpose' => 'Club meeting',
                'date' => '2030-05-01',
                'start_time' => '11:00',
                'end_time' => '13:00',
            ])
            ->assertStatus(409)
            ->assertJsonPath('code', 'schedule_occupied')
            ->assertJsonPath('error', "This schedule and room is already occupied!\nPlease try another time or room!");

        $this->assertDatabaseCount('reservations', 1);
    }

    public function test_same_start_time_database_conflicts_return_the_friendly_response(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $this->createReservation($user, $facility, 'pending', '10:00', '12:00');

        $this->actingAs($user)
            ->postJson('/api/calendar/reserve', [
                'facility' => (string) $facility->id,
                'purpose' => 'Club meeting',
                'date' => '2030-05-01',
                'start_time' => '10:00',
                'end_time' => '11:00',
            ])
            ->assertStatus(409)
            ->assertJsonPath('code', 'schedule_occupied')
            ->assertJsonPath('error', "This schedule and room is already occupied!\nPlease try another time or room!");

        $this->assertDatabaseCount('reservations', 1);
    }

    public function test_user_can_reserve_property_items_without_a_facility(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $item = FacilityItem::create([
            'facility_id' => $facility->id,
            'name' => 'Projector',
            'quantity' => 3,
        ]);

        $this->actingAs($user)
            ->postJson('/api/calendar/reserve', [
                'purpose' => 'Club meeting',
                'date' => '2030-05-01',
                'start_time' => '10:00',
                'end_time' => '12:00',
                'items' => [
                    ['facility_item_id' => $item->id, 'quantity' => 2],
                ],
            ])
            ->assertOk()
            ->assertJsonPath('success', true);

        $reservation = Reservation::firstOrFail();
        $this->assertSame('pending', $reservation->status);
        $this->assertNull($reservation->facility_id);
        $this->assertDatabaseHas('notifications', [
            'notifiable_id' => $user->id,
            'type' => ReservationStatusChanged::class,
        ]);
        $this->assertDatabaseHas('reservation_items', [
            'reservation_id' => $reservation->id,
            'facility_item_id' => $item->id,
            'quantity' => 2,
        ]);
```

```
295     }
296
297     public function test_reservation_date_cannot_exceed_the_two_year_booking_window(): void
298     {
299         $user = User::factory()->create();
300         $facility = $this->createFacility();
301         $maximumDate = app(ReservationDateWindow::class)->maximumDate();
302         $reservationData = [
303             'facility' => (string) $facility->id,
304             'purpose' => 'Club meeting',
305             'start_time' => '10:00',
306             'end_time' => '12:00',
307         ];
308
309         $this->actingAs($user)
310             ->postJson('/api/calendar/reserve', $reservationData + ['date' => $maximumDate])
311             ->assertOk();
312
313         $this->postJson('/api/calendar/reserve', $reservationData + [
314             'date' => Carbon::parse($maximumDate)->addDay()->toDateString(),
315         ])->assertUnprocessable()
316             ->assertJsonValidationErrors('date');
317     }
318
319     public function test_property_reservation_can_use_items_without_facility_assignment(): void
320     {
321         $user = User::factory()->create();
322         $facility = $this->createFacility();
323         $otherFacility = Facility::create([
324             'name' => 'Conference Room',
325             'building' => 'Main Building',
326             'type' => 'Conference Room',
327             'capacity' => 20,
328             'is_active' => true,
329         ]);
330         $item = FacilityItem::create([
331             'facility_id' => $otherFacility->id,
332             'name' => 'Projector',
333             'quantity' => 2,
334         ]);
335
336         $this->actingAs($user)
337             ->postJson('/api/calendar/reserve', [
338                 'purpose' => 'Club meeting',
339                 'date' => '2030-05-01',
340                 'start_time' => '10:00',
341                 'end_time' => '12:00',
342                 'items' => [
343                     ['facility_item_id' => $item->id, 'quantity' => 1],
344                 ],
345             ])
346             ->assertOk()
347             ->assertJsonPath('success', true);
348
349         $this->assertDatabaseCount('reservations', 1);
350         $this->assertDatabaseHas('reservations', [
351             'id' => 1,
352             'facility_id' => null,
353         ]);
354     }
355
356     public function test_property_reservation_can_be_edited_without_a_facility_and_excludes_its_own_stock(): void
357     {
358         $user = User::factory()->create();
359         $item = FacilityItem::create([
360             'facility_id' => null,
361             'name' => 'Projector',
362             'quantity' => 3,
363         ]);
364         $reservation = Reservation::create([
365             'user_id' => $user->id,
366             'facility_id' => null,
367             'reservation_date' => '2030-05-01',
368             'start_time' => '10:00',
```

```
            'end_time' => '12:00',
            'purpose' => 'Presentation',
            'status' => 'approved',
        ]);
        $reservation->items()->attach($item, ['quantity' => 2]);

        $otherReservation = Reservation::create([
            'user_id' => User::factory()->create()->id,
            'facility_id' => null,
            'reservation_date' => '2030-05-01',
            'start_time' => '10:00',
            'end_time' => '12:00',
            'purpose' => 'Another presentation',
            'status' => 'approved',
        ]);
        $otherReservation->items()->attach($item, ['quantity' => 1]);

        $this->actingAs($user)
            ->get(route('reservations.edit', $reservation))
            ->assertOk()
            ->assertSee('Independent property reservation')
            ->assertDontSee('name="facility_id"', false);

        $this->put(route('reservations.update', $reservation), [
            'purpose' => 'Updated presentation',
            'reservation_date' => '2030-05-01',
            'start_time' => '10:00',
            'end_time' => '12:00',
        ])->assertRedirect(route('reserve'));

        $this->assertDatabaseHas('reservations', [
            'id' => $reservation->id,
            'facility_id' => null,
            'purpose' => 'Updated presentation',
            'status' => 'pending',
        ]);
    }

    public function test_user_cannot_edit_a_reservation_beyond_the_booking_window(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $reservation = $this->createReservation($user, $facility, 'approved', '10:00', '12:00');

        $this->actingAs($user)
            ->putJson(route('reservations.update', $reservation), [
                'facility_id' => $facility->id,
                'purpose' => 'Updated presentation',
                'reservation_date' => now()->addYears(2)->addDay()->toDateString(),
                'start_time' => '10:00',
                'end_time' => '12:00',
            ])
            ->assertUnprocessable()
            ->assertJsonValidationErrors('reservation_date');
    }

    public function test_item_availability_counts_only_overlapping_approved_reservations(): void
    {
        $user = User::factory()->create();
        $facility = $this->createFacility();
        $item = FacilityItem::create([
            'facility_id' => $facility->id,
            'name' => 'Projector',
            'quantity' => 4,
        ]);
        $secondItem = FacilityItem::create([
            'facility_id' => $facility->id,
            'name' => 'Whiteboard',
            'quantity' => 5,
        ]);

        $approved = $this->createReservation($user, $facility, 'approved', '10:00', '12:00');
        $approved->items()->attach($item, ['quantity' => 2]);
        $approved->items()->attach($secondItem, ['quantity' => 3]);
```

```
        $pending = $this->createReservation($user, $facility, 'pending', '09:00', '10:45');
        $pending->items()->attach($item, ['quantity' => 1]);
        $nonOverlapping = $this->createReservation($user, $facility, 'approved', '12:00', '13:00');
        $nonOverlapping->items()->attach($item, ['quantity' => 1]);
        $availabilityQueries = 0;
        DB::listen(function ($query) use (&$availabilityQueries): void {
            if (str_contains($query->sql, 'SUM(reservation_items.quantity)')) {
                $availabilityQueries++;
            }
        });

        $this->actingAs($user)
            ->getJson(route('api.property-items.availability', [
                'date' => '2030-05-01',
                'start_time' => '10:30',
                'end_time' => '11:30',
            ]))
            ->assertOk()
            ->assertJsonPath('items.0.id', $item->id)
            ->assertJsonPath('items.0.stock', 4)
            ->assertJsonPath('items.0.available', 2)
            ->assertJsonPath('items.1.id', $secondItem->id)
            ->assertJsonPath('items.1.available', 2);

        $this->assertSame(1, $availabilityQueries);
    }

    private function createFacility(): Facility
    {
        return Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
    }

    private function createReservation(
        User $user,
        Facility $facility,
        string $status,
        string $startTime,
        string $endTime
    ): Reservation {
        return Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => '2030-05-01',
            'start_time' => $startTime,
            'end_time' => $endTime,
            'purpose' => 'Test booking',
            'status' => $status,
        ]);
    }
}
```

tests/Feature/PropertyReturnReminderTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\FacilityItem;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\PropertyReturnDue;
use App\Notifications\PropertyReturnNotConfirmed;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Notifications\DatabaseNotification;
use Tests\TestCase;

class PropertyReturnReminderTest extends TestCase
```

```
{
    use RefreshDatabase;

    public function test_due_approved_property_reservation_is_notified_once(): void
    {
        $this->travelTo(now()->setDate(2030, 5, 1)->setTime(12, 0));
        $user = User::factory()->create();
        [$reservation, $item] = $this->createPropertyReservation($user, 'approved', '11:00');
        $reservation->items()->attach($item, ['quantity' => 2]);

        $this->assertSame('approved', $reservation->fresh()->status);
        $this->assertSame(
            1,
            Reservation::query()
                ->whereDate('reservation_date', now()->toDateString())
                ->whereTime('end_time', '<=', now()->format('H:i:s'))
                ->whereHas('items')
                ->whereNull('return_reminded_at')
                ->count()
        );

        $this->artisan('property-returns:notify-due')
            ->expectsOutput('Sent 1 property return reminder(s).')
            ->assertSuccessful();
        $this->artisan('property-returns:notify-due')->assertSuccessful();

        $this->assertSame(1, $user->notifications()->where('type', PropertyReturnDue::class)->count());
        $this->assertNotNull($reservation->fresh()->return_reminded_at);
    }

    public function test_pending_or_not_yet_due_property_reservations_are_not_notified(): void
    {
        $this->travelTo(now()->setDate(2030, 5, 1)->setTime(12, 0));
        $pendingUser = User::factory()->create();
        [$pendingReservation, $pendingItem] = $this->createPropertyReservation($pendingUser, 'pending', '11:00');
        $pendingReservation->items()->attach($pendingItem, ['quantity' => 1]);

        $futureUser = User::factory()->create();
        [$futureReservation, $futureItem] = $this->createPropertyReservation($futureUser, 'approved', '13:00');
        $futureReservation->items()->attach($futureItem, ['quantity' => 1]);

        $this->artisan('property-returns:notify-due')->assertSuccessful();

        $this->assertSame(0, $pendingUser->notifications()->count());
        $this->assertSame(0, $futureUser->notifications()->count());
    }

    public function test_borrower_can_confirm_return_and_complete_reservation(): void
    {
        $this->travelTo(now()->setDate(2030, 5, 1)->setTime(12, 0));
        $user = User::factory()->create();
        [$reservation, $item] = $this->createPropertyReservation($user, 'approved', '11:00');
        $reservation->items()->attach($item, ['quantity' => 1]);
        $notification = $this->createReturnNotification($user, $reservation);

        $this->actingAs($user)
            ->post(route('notifications.propertyReturn.respond', $reservation), [
                'notification_id' => $notification->id,
                'response' => 'returned',
            ])
            ->assertRedirect();

        $this->assertSame('completed', $reservation->fresh()->status);
        $this->assertSame('returned', $notification->fresh()->data['response']);
        $this->assertNotNull($notification->fresh()->read_at);
    }

    public function test_only_owner_can_respond_and_only_after_return_time(): void
    {
        $this->travelTo(now()->setDate(2030, 5, 1)->setTime(10, 30));
        $user = User::factory()->create();
        $otherUser = User::factory()->create();
        [$reservation, $item] = $this->createPropertyReservation($user, 'approved', '11:00');
        $reservation->items()->attach($item, ['quantity' => 1]);
```

```
        $notification = $this->createReturnNotification($user, $reservation);

        $this->actingAs($otherUser)
            ->post(route('notifications.propertyReturn.respond', $reservation), [
                'notification_id' => $notification->id,
                'response' => 'returned',
            ])
            ->assertNotFound();

        $this->actingAs($user)
            ->post(route('notifications.propertyReturn.respond', $reservation), [
                'notification_id' => $notification->id,
                'response' => 'returned',
            ])
            ->assertUnprocessable();

        $this->assertSame('approved', $reservation->fresh()->status);
        $this->assertArrayNotHasKey('response', $notification->fresh()->data);
    }

    public function test_borrower_reporting_items_not_returned_notifies_admins_without_completing(): void
    {
        $this->travelTo(now()->setDate(2030, 5, 1)->setTime(12, 0));
        $user = User::factory()->create();
        $admin = User::factory()->create(['is_admin' => true]);
        [$reservation, $item] = $this->createPropertyReservation($user, 'approved', '11:00');
        $reservation->items()->attach($item, ['quantity' => 1]);
        $notification = $this->createReturnNotification($user, $reservation);

        $this->actingAs($user)
            ->post(route('notifications.propertyReturn.respond', $reservation), [
                'notification_id' => $notification->id,
                'response' => 'not_returned',
            ])
            ->assertRedirect()
            ->assertSessionHas('success');

        $this->assertSame('approved', $reservation->fresh()->status);
        $this->assertSame('not_returned', $notification->fresh()->data['response']);
        $this->assertSame(1, $admin->notifications()->where('type', PropertyReturnNotConfirmed::class)->count());

        $this->post(route('notifications.propertyReturn.respond', $reservation), [
            'notification_id' => $notification->id,
            'response' => 'not_returned',
        ])->assertRedirect()->assertSessionHas('info');

        $this->assertSame(1, $admin->notifications()->where('type', PropertyReturnNotConfirmed::class)->count());
    }

    public function test_return_reminder_page_offers_yes_and_no_actions(): void
    {
        $this->travelTo(now()->setDate(2030, 5, 1)->setTime(12, 0));
        $user = User::factory()->create();
        [$reservation, $item] = $this->createPropertyReservation($user, 'approved', '11:00');
        $reservation->items()->attach($item, ['quantity' => 1]);
        $this->createReturnNotification($user, $reservation);

        $this->actingAs($user)
            ->get(route('notifications.index'))
            ->assertOk()
            ->assertSee('Yes, returned')
            ->assertSee('No, not returned');
    }

    private function createPropertyReservation(User $user, string $status, string $endTime): array
    {
        $facility = Facility::create([
            'name' => 'Lecture Room '.$user->id,
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
        $item = FacilityItem::create([
```

```
            'facility_id' => $facility->id,
            'name' => 'Projector',
            'quantity' => 3,
        ]);
        $reservation = Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => now()->toDateString(),
            'start_time' => '10:00',
            'end_time' => $endTime,
            'purpose' => 'Class presentation',
            'status' => $status,
        ]);

        return [$reservation, $item];
    }

    private function createReturnNotification(User $user, Reservation $reservation): DatabaseNotification
    {
        $reservation->load(['facility', 'items']);
        $user->notify(new PropertyReturnDue($reservation));

        return $user->notifications()->where('type', PropertyReturnDue::class)->latest()->firstOrFail();
    }
}
```

tests/Feature/RejectExpiredReservationsTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationStatusChanged;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Facades\Artisan;
use Illuminate\Support\Facades\Notification;
use Tests\TestCase;

class RejectExpiredReservationsTest extends TestCase
{
    use RefreshDatabase;

    public function test_it_rejects_past_pending_reservations_and_notifies_the_borrower(): void
    {
        Notification::fake();

        $user = User::factory()->create();
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
        $expiredPending = $this->createReservation($user, $facility, 'pending', now()->subDay()->toDateString(), '10:00');
        $todayPending = $this->createReservation($user, $facility, 'pending', now()->toDateString(), '10:00');
        $upcomingPending = $this->createReservation($user, $facility, 'pending', now()->addDay()->toDateString(), '10:00');
        $expiredApproved = $this->createReservation($user, $facility, 'approved', now()->subDay()->toDateString(), '11:00');

        Artisan::call('reservations:reject-expired');

        $this->assertDatabaseHas('reservations', [
            'id' => $expiredPending->id,
            'status' => 'rejected',
            'admin_remarks' => 'Automatically rejected because the reservation date has passed.',
        ]);
        $this->assertNotNull($expiredPending->fresh()->rejected_at);
        $this->assertDatabaseHas('reservations', ['id' => $todayPending->id, 'status' => 'pending']);
        $this->assertDatabaseHas('reservations', ['id' => $upcomingPending->id, 'status' => 'pending']);
        $this->assertDatabaseHas('reservations', ['id' => $expiredApproved->id, 'status' => 'approved']);
        Notification::assertSentTo($user, ReservationStatusChanged::class, function (ReservationStatusChanged $notification) use ($expiredPending): bool {
```

```
            return $notification->reservation->is($expiredPending)
                && $notification->reservation->status === 'rejected';
        });

        Artisan::call('reservations:reject-expired');

        Notification::assertSentToTimes($user, ReservationStatusChanged::class, 1);
    }

    public function test_it_deletes_rejected_reservations_after_three_full_days(): void
    {
        $user = User::factory()->create();
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);

        $expiredRejected = $this->createReservation($user, $facility, 'rejected', now()->toDateString(), '10:00');
        $expiredRejected->forceFill(['rejected_at' => now()->subDays(3)])->save();
        $recentlyRejected = $this->createReservation($user, $facility, 'rejected', now()->toDateString(), '11:00');
        $recentlyRejected->forceFill(['rejected_at' => now()->subDays(2)])->save();
        $oldPending = $this->createReservation($user, $facility, 'pending', now()->toDateString(), '12:00');
        $oldPending->forceFill(['rejected_at' => now()->subDays(5)])->save();

        Artisan::call('reservations:delete-expired-rejected');

        $this->assertDatabaseMissing('reservations', ['id' => $expiredRejected->id]);
        $this->assertDatabaseHas('reservations', ['id' => $recentlyRejected->id, 'status' => 'rejected']);
        $this->assertDatabaseHas('reservations', ['id' => $oldPending->id, 'status' => 'pending']);
    }

    public function test_it_deletes_completed_reservations_three_days_after_their_scheduled_end_time(): void
    {
        $this->travelTo(now()->setDate(2030, 4, 5)->setTime(11, 0));

        $user = User::factory()->create();
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);

        $completedAtCutoff = $this->createReservation($user, $facility, 'completed', '2030-04-02', '10:00');
        $completedAtCutoffBefore = $this->createReservation($user, $facility, 'completed', '2030-04-02', '09:59');
        $completedNotYetExpired = $this->createReservation($user, $facility, 'completed', '2030-04-02', '10:01');
        $olderCompleted = $this->createReservation($user, $facility, 'completed', '2030-04-01', '15:00');
        $oldRejected = $this->createReservation($user, $facility, 'rejected', '2030-04-01', '10:00');
        $oldApproved = $this->createReservation($user, $facility, 'approved', '2030-04-01', '11:00');
        $completedAtCutoff->update(['end_time' => '11:00']);
        $completedAtCutoffBefore->update(['end_time' => '10:59']);
        $completedNotYetExpired->update(['end_time' => '11:01']);

        Artisan::call('reservations:delete-expired-completed');

        $this->assertDatabaseMissing('reservations', ['id' => $completedAtCutoff->id]);
        $this->assertDatabaseMissing('reservations', ['id' => $completedAtCutoffBefore->id]);
        $this->assertDatabaseMissing('reservations', ['id' => $olderCompleted->id]);
        $this->assertDatabaseHas('reservations', ['id' => $completedNotYetExpired->id, 'status' => 'completed']);
        $this->assertDatabaseHas('reservations', ['id' => $oldRejected->id, 'status' => 'rejected']);
        $this->assertDatabaseHas('reservations', ['id' => $oldApproved->id, 'status' => 'approved']);
    }

    public function test_admin_rejection_records_the_rejection_time(): void
    {
        Notification::fake();

        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
```

```
        ]);
        $user = User::factory()->create();
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
        $reservation = $this->createReservation($user, $facility, 'pending', now()->addDay()->toDateString(), '10:00');

        $this->actingAs($admin)
            ->post(route('admin.reservations.reject', $reservation), ['current_password' => 'password'])
            ->assertRedirect();

        $this->assertDatabaseHas('reservations', [
            'id' => $reservation->id,
            'status' => 'rejected',
        ]);
        $this->assertNotNull($reservation->fresh()->rejected_at);
    }

    public function test_admin_can_reject_a_reservation_without_a_redirect(): void
    {
        Notification::fake();

        $admin = User::factory()->create([
            'is_admin' => true,
            'is_super_admin' => false,
        ]);
        $user = User::factory()->create();
        $facility = Facility::create([
            'name' => 'Lecture Room 2',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);
        $reservation = $this->createReservation($user, $facility, 'pending', now()->addDay()->toDateString(), '12:00');

        $this->actingAs($admin)
            ->postJson(route('admin.reservations.reject', $reservation), ['current_password' => 'password'])
            ->assertOk()
            ->assertJsonPath('status', 'Reservation rejected successfully.')
            ->assertJsonPath('pending_count', 0)
            ->assertJsonPath('audit_log.action', 'rejected');

        $this->assertSame('rejected', $reservation->fresh()->status);
    }

    private function createReservation(
        User $user,
        Facility $facility,
        string $status,
        string $date,
        string $startTime
    ): Reservation {
        return Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => $date,
            'start_time' => $startTime,
            'end_time' => '11:00',
            'purpose' => 'Test booking',
            'status' => $status,
        ]);
    }
}
```

tests/Feature/ResendMailTransportTest.php

```
<?php

namespace Tests\Feature;
```

```php

use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Mail;
use Symfony\Component\Mailer\Exception\TransportException;
use Tests\TestCase;

class ResendMailTransportTest extends TestCase
{
    public function test_resend_mailer_sends_messages_through_the_https_api(): void
    {
        config([
            'mail.mailers.resend.key' => 'test-resend-key',
            'mail.from.address' => 'no-reply@example.com',
            'mail.from.name' => 'Campus Reserve',
        ]);
        Http::fake([
            'https://api.resend.com/emails' => Http::response(['id' => 'resend-message-id'], 200),
        ]);

        Mail::mailer('resend')->raw('Reset your password.', function ($message): void {
            $message->to('user@example.com')->subject('Password reset');
        });

        Http::assertSent(function ($request): bool {
            return $request->url() === 'https://api.resend.com/emails'
                && $request->hasHeader('Authorization')
                && $request['from'] === '"Campus Reserve" <no-reply@example.com>'
                && $request['to'] === ['user@example.com']
                && $request['subject'] === 'Password reset'
                && $request['text'] === 'Reset your password.';
        });
    }

    public function test_resend_mailer_reports_api_rejections(): void
    {
        config(['mail.mailers.resend.key' => 'test-resend-key']);
        Http::fake([
            'https://api.resend.com/emails' => Http::response(['message' => 'Invalid API key'], 401),
        ]);

        $this->expectException(TransportException::class);
        $this->expectExceptionMessage('HTTP 401: Invalid API key');

        Mail::mailer('resend')->raw('Test email.', function ($message): void {
            $message->to('user@example.com')->subject('Test');
        });
    }

    public function test_resend_mailer_requires_an_api_key(): void
    {
        config([
            'mail.mailers.resend.key' => null,
            'services.resend.key' => null,
        ]);

        $this->expectException(TransportException::class);
        $this->expectExceptionMessage('The Resend API key is not configured.');

        Mail::mailer('resend')->raw('Test email.', function ($message): void {
            $message->to('user@example.com')->subject('Test');
        });
    }
}
```

tests/Feature/ReservationEmailNotificationsTest.php

```php
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\Reservation;
use App\Models\User;
use App\Notifications\ReservationStatusChanged;
```

```php
use App\Services\ReservationNotificationDispatcher;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Facades\Notification;
use Tests\TestCase;

class ReservationEmailNotificationsTest extends TestCase
{
    use RefreshDatabase;

    public function test_email_notifications_are_disabled_by_default_and_can_be_enabled_in_profile(): void
    {
        $user = User::factory()->create();

        $this->assertFalse($user->email_notifications);

        $this->actingAs($user)
            ->get(route('profile.show'))
            ->assertOk()
            ->assertSee('Email me about my reservations');

        $this->actingAs($user)
            ->post(route('profile.update'), [
                'name' => $user->name,
                'email_notifications' => '1',
            ])
            ->assertRedirect();

        $this->assertTrue($user->fresh()->email_notifications);

        $this->actingAs($user)
            ->post(route('profile.update'), ['name' => $user->name])
            ->assertRedirect();

        $this->assertFalse($user->fresh()->email_notifications);
    }

    public function test_email_notification_toggle_saves_immediately_without_submitting_the_profile_form(): void
    {
        $user = User::factory()->create(['email_notifications' => false]);

        $this->actingAs($user)
            ->postJson(route('profile.email-notifications.update'), [
                'email_notifications' => true,
            ])
            ->assertOk()
            ->assertJsonPath('status', 'Email notifications enabled.');

        $this->assertTrue($user->fresh()->email_notifications);

        $this->postJson(route('profile.email-notifications.update'), [
            'email_notifications' => false,
        ])
            ->assertOk()
            ->assertJsonPath('status', 'Email notifications disabled.');

        $this->assertFalse($user->fresh()->email_notifications);
    }

    public function test_reservation_update_notifications_use_database_only_when_email_is_disabled(): void
    {
        Notification::fake();
        $user = User::factory()->create(['email_notifications' => false]);
        $reservation = $this->createReservation($user);

        app(ReservationNotificationDispatcher::class)->notifyBorrower($reservation);

        Notification::assertSentTo($user, ReservationStatusChanged::class, function ($notification, $channels) {
            return $channels === ['database']
                && str_contains($notification->toArray(null)['message'], 'is pending and awaiting approval');
        });
        Notification::assertSentToTimes($user, ReservationStatusChanged::class, 1);
    }

    public function test_opted_in_users_receive_reservation_updates_by_database_and_email(): void
```

```
    {
        Notification::fake();
        $user = User::factory()->create(['email_notifications' => true]);
        $reservation = $this->createReservation($user);

        app(ReservationNotificationDispatcher::class)->notifyBorrower($reservation);

        Notification::assertSentTo($user, ReservationStatusChanged::class, function ($notification, $channels) {
            return $channels === ['database'];
        });
        Notification::assertSentTo($user, ReservationStatusChanged::class, function ($notification, $channels) use ($user) {
            return $channels === ['mail']
                && $notification->toMail($user)->subject === 'Reservation Pending - Campus Reserve';
        });
        Notification::assertSentToTimes($user, ReservationStatusChanged::class, 2);
    }

    private function createReservation(User $user): Reservation
    {
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);

        return Reservation::create([
            'user_id' => $user->id,
            'facility_id' => $facility->id,
            'reservation_date' => '2030-05-01',
            'start_time' => '10:00',
            'end_time' => '12:00',
            'purpose' => 'Class',
            'status' => 'pending',
        ]);
    }
}
```

tests/Feature/ReservationScheduleSettingsTest.php

```
<?php

namespace Tests\Feature;

use App\Models\Facility;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Carbon;
use Tests\TestCase;

class ReservationScheduleSettingsTest extends TestCase
{
    use RefreshDatabase;

    protected function setUp(): void
    {
        parent::setUp();
        $this->travelTo(Carbon::parse('2030-04-01'));
    }

    public function test_admin_can_view_and_update_allowed_reservation_hours(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);

        $this->actingAs($admin)
            ->get(route('admin.reservation-settings'))
            ->assertOk()
            ->assertSee('Reservation settings')
            ->assertSee('00:00')
            ->assertSee('23:59')
            ->assertSee('aria-current="page"', false);

        $this->put(route('admin.reservation-settings.update'), [
```

```
            'start_time' => '08:00',
            'end_time' => '22:00',
            'current_password' => 'password',
        ])
            ->assertRedirect(route('admin.reservation-settings'))
            ->assertSessionHas('status', 'Reservation hours updated successfully.');

        $this->assertDatabaseHas('reservation_settings', [
            'id' => 1,
            'start_time' => '08:00',
            'end_time' => '22:00',
        ]);
    }

    public function test_reservations_are_rejected_outside_the_configured_time_window(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();
        $facility = Facility::create([
            'name' => 'Lecture Room 1',
            'building' => 'Main Building',
            'type' => 'Classroom',
            'capacity' => 30,
            'is_active' => true,
        ]);

        $this->actingAs($admin);
        $this->put(route('admin.reservation-settings.update'), [
            'start_time' => '08:00',
            'end_time' => '22:00',
            'current_password' => 'password',
        ])->assertRedirect(route('admin.reservation-settings'));

        $this->actingAs($user);
        $this->postJson('/api/calendar/reserve', [
            'facility' => (string) $facility->id,
            'purpose' => 'Early meeting',
            'date' => '2030-05-01',
            'start_time' => '07:00',
            'end_time' => '09:00',
        ])
            ->assertUnprocessable()
            ->assertJsonPath('error', 'Reservations must be scheduled between 8:00 AM and 10:00 PM.');

        $this->postJson('/api/calendar/reserve', [
            'facility' => (string) $facility->id,
            'purpose' => 'Evening meeting',
            'date' => '2030-05-01',
            'start_time' => '21:00',
            'end_time' => '22:00',
        ])->assertOk()->assertJsonPath('success', true);
    }

    public function test_non_admin_cannot_view_reservation_settings(): void
    {
        $user = User::factory()->create();

        $this->actingAs($user)
            ->get(route('admin.reservation-settings'))
            ->assertForbidden();
    }

    public function test_calendar_page_displays_the_configured_reservation_hours(): void
    {
        $admin = User::factory()->create(['is_admin' => true]);
        $user = User::factory()->create();

        $this->actingAs($admin);
        $this->put(route('admin.reservation-settings.update'), [
            'start_time' => '07:00',
            'end_time' => '19:00',
            'current_password' => 'password',
        ]);

```

```
        $this->actingAs($user);
        $this->get('/calendar')
            ->assertOk()
            ->assertSee('"07:00"', false)
            ->assertSee('"19:00"', false)
            ->assertSee("slotMinTime: reservationOpenTime + ':00'", false)
            ->assertSee("slotMaxTime: reservationCloseTime + ':00'", false);

        $this->get(route('calendar.manage', ['embedded' => 1]))
            ->assertOk()
            ->assertSee('Reservations allowed 7:00 AM–7:00 PM.')
            ->assertSee('min="07:00"', false)
            ->assertSee('max="19:00"', false);
    }
}
```

tests/TestCase.php

```
<?php

namespace Tests;

use Illuminate\Foundation\Testing\TestCase as BaseTestCase;

abstract class TestCase extends BaseTestCase
{
    //
}
```

tests/Unit/ExampleTest.php

```
<?php

namespace Tests\Unit;

use App\Models\User;
use PHPUnit\Framework\TestCase;

class ExampleTest extends TestCase
{
    /**
     * A basic test example.
     */
    public function test_that_true_is_true(): void
    {
        $this->assertTrue(true);
    }

    public function test_invalid_two_fa_secret_does_not_throw(): void
    {
        $user = new User();
        $user->setRawAttributes(['two_fa_secret' => 'not-a-valid-encrypted-value']);

        $this->assertNull($user->two_fa_secret);
    }
}
```